

# Modern Computer Architecture

## Samajh ke Padho

CSA222 · all 15 units · samjho kya ho raha hai, kaise ho raha hai, phir numerical khud solve karo.

**Ye notes alag kyun hain:** Pichle notes mein "one-hot" jaise words bina matlab ke likhe the. Yahan **har** mushkil word ka aasaan matlab, ek chhota example ke saath, wahin pe diya hai jahan woh pehli baar aata hai. Har topic pehle bataata hai **kyun** chahiye, phir **kya** hai, phir **kaise** kaam karta hai (0/1 values ke saath), phir exam jaisa sawal **step by step solve** karke dikhata hai.

Har topic is order mein hai:

**Pehle socho:** ye cheez kis problem ko solve karti hai, ek roz ki zindagi wala example.

**Definition:** exam mein likhne wali English definition.

**Mushkil shabd, aasaan matlab:** har technical word ka matlab, chhote example ke saath.

**Kaise kaam karta hai:** numbered steps, koi step skip nahi.

**Diagram kaise padhein:** diagram mein pehle kya dekhna hai, aur ek value daal ke trace.

**Formula:** har symbol ka matlab ke saath.

**Solved example:** exam jaisa sawal, har line ka calculation, aur last mein **Answer**.

**Galti mat karna:** jo galti students sach mein karte hain.

**Trick:** yaad rakhne ka hook.

**Ek line mein:** pura topic ek sentence mein.

Har unit ke end mein **"Exam mein kya aata hai → kaise solve karein"** table hai: question type aur uska step-by-step method.

### Kaise padhna hai (1 din mein):

1. **Samjho:** "Pehle socho" + "Kaise kaam karta hai" padho. Har step pe khud se poocho "ye kyun?".
2. **Haath se karo:** solved example ka sawal padho, answer dhak do, khud solve karo, phir steps se match karo. Padhne se zyada yaad tab rehta hai jab khud nikaalte ho.
3. **Bol ke dekho:** "Ek line mein" box ko dhak ke topic apne shabdon mein 30 second mein bolo. Atak gaye to wahi part dobara padho.
4. **Diagram ek baar banao:** must-draw diagrams copy pe ek baar haath se banao.
5. **Raat ko:** sirf "Ek line mein" boxes, unit-end tables aur last page ka cheat sheet.

Ye tareeka learning research se liya hai: Feynman technique (bina jargon ke samjhao), worked examples (pehle solved example dekho, phir khud karo), dual coding (words + diagram saath mein), aur retrieval practice (dhak ke khud yaad karo). Content class slides aur labs ke hisaab se hai, Mano (Digital Design), Harris & Harris (Digital Design and Computer Architecture) aur Patterson & Hennessy (Computer Organization and Design) se check kiya hua. Units 14 aur 15 ki class slides nahi thi, woh textbooks se hain.

# Contents

|                                                  |          |
|--------------------------------------------------|----------|
| Unit 1 · Boolean Algebra & Gates                 | page 4   |
| Unit 2 · Logic Minimisation & Universal Gates    | page 22  |
| Unit 3 · Combinational Building Blocks           | page 44  |
| Unit 4 · Number Systems & Two's Complement       | page 66  |
| Unit 5 · Latches, Flip-Flops & Timing            | page 85  |
| Unit 6 · Registers, Counters & Shift Registers   | page 106 |
| Unit 7 · Register Files & Finite State Machines  | page 126 |
| Unit 8 · Memory: SRAM vs DRAM & the Memory Stack | page 149 |
| Unit 9 · Von Neumann vs Harvard Architecture     | page 165 |
| Unit 10 · CISC vs RISC                           | page 180 |
| Unit 11 · MIPS Registers & Instruction Formats   | page 196 |
| Unit 12 · C to MIPS Translation                  | page 217 |
| Unit 13 · The Single-Cycle Datapath              | page 236 |
| Unit 14 · Pipelining                             | page 260 |
| Unit 15 · Pipeline Hazards                       | page 272 |
| Last-hour cheat sheet                            | page 291 |

Ek din mein sab nahi ho paaye to order: 4 (numbers) → 2 (K-map) → 3 → 5 → 6 → 7 → 11 → 12 → 13 → 14 → 15, phir baaki. Har unit ke end ka "Exam mein kya aata hai" table zaroor dekho.

# Unit 1 · Boolean Algebra & Gates

**Is unit ki kahani:** Computer andar se sirf switches ka dher hai: switch ON = 1, OFF = 0. Is unit mein hum seekhte hain ki ye 0/1 kaise "socha" jaata hai: gates (AND, OR, NOT, XOR), unki truth tables, aur Boolean laws jinse ek bada circuit chhota ho jaata hai. Saath mein Verilog, jisme hum circuit ko text ki tarah likhte hain. Unit 2 inhi laws ko K-map se fast banata hai, aur Unit 3 inhi gates se MUX, decoder, adder jaise bade blocks banata hai.

## 1.1 Course Introduction and Roadmap

**Pehle socho:** Ek building ka **naksha** (kitne kamre, kahan darwaza, kahan switch) residents ko dikhta hai. Deewar ke andar ki **wiring aur plumbing** nahi dikhti. Do buildings ka naksha same ho sakta hai par wiring alag. Computer mein bhi yahi do cheezein hain: jo programmer ko dikhta hai (**architecture**) aur jo andar chhupa hai (**organisation**).

**Definition:** **Computer architecture** is the set of attributes of a computer visible to the programmer, mainly the instruction set architecture (ISA): instructions, registers, data types and memory model ("what" the machine does). **Computer organisation** is the hidden implementation of that architecture: pipelines, caches, clock rate, control signals ("how" it does it).

**Mushkil shabd, aasaan matlab:**

### ISA (Instruction Set Architecture)

CPU ki "vocabulary": kaun kaun se instructions hain (ADD, LOAD), kitne registers hain. Jaise "is CPU mein ADD instruction hai" ek ISA statement hai.

### organisation / microarchitecture

Same cheez ke do naam: ADD andar se kaise bana hai. Jaise "ADD ek cycle mein is adder se hota hai" ya "L1 cache 48 KB hai".

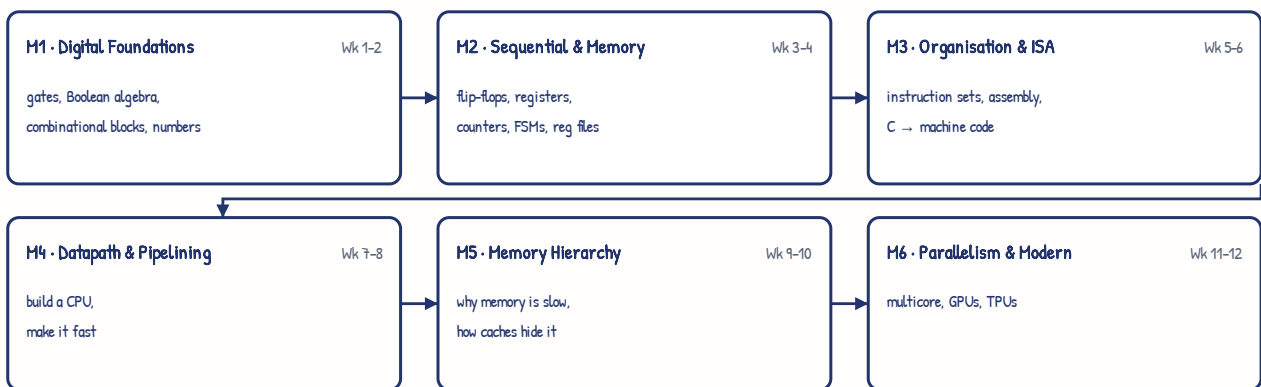
### programmer-visible

Jo cheez program ke result pe asar daale. Register count visible hai (program usse use karta hai). Cache size visible nahi (program same chalega, bas speed badlegi).

**Kaise kaam karta hai (course ka raasta):**

1. Course CSA222 gates se shuru hokar AI accelerator tak jaata hai: 12 weeks, 24 lectures, **6 modules**.
2. M1 Digital Foundations (gates, Boolean algebra, combinational blocks, numbers) → M2 Sequential & Memory (flip-flops, registers, counters, FSM).
3. M3 Organisation & ISA (MIPS, C → machine code) → M4 Datapath & Pipelining (CPU banana, fast karna).
4. M5 Memory Hierarchy (cache) → M6 Parallelism (multicore, GPU, TPU).

5. Grading: end-sem 40%, mid-sem 20%, contests 20%, projects 10%, assignments 10%. Tools: Verilog simulation (M1, M2), MARS MIPS simulator (M3).



### Six-module journey

How to draw: Two rows of three boxes M1-M6 with arrows; write one line of content under each.

**Diagram kaise padhein:** Upar ki row M1 → M2 → M3, neeche ki row M4 → M5 → M6; arrow batata hai ki har module pichle wale pe khada hai. Har box mein top line module ka naam aur weeks hai, neeche ki lines topics.

### Solved example (write the answer like this):

*Differentiate between computer architecture and computer organisation with two examples of each. (5 marks)*

1. Architecture: attributes visible to the programmer, the hardware/software contract (ISA). It answers "what does the machine do?".
2. Organisation: the operational units and their interconnections that implement the architecture. It answers "how does it do it?".
3. Architecture examples: instruction set and formats, number of registers, addressing modes, data types.
4. Organisation examples: pipelining, cache size, clock rate, adder type (ripple vs lookahead), control signals.
5. Impact: changing the architecture breaks old software; changing the organisation keeps programs running, only speed and cost change. Example: Intel and AMD x86-64 chips share the architecture but differ in organisation.

**Answer: Architecture = what (ISA, visible); Organisation = how (implementation, hidden).**

**Galti mat karna:** "L1 cache 48 KB hai" ya "pipeline 5 stage ki hai" ko architecture mat bolna. Program ka result in pe depend nahi karta, sirf speed karti hai, isliye ye **organisation** hai.

**Ek line mein:** Architecture = programmer ko kya dikhta hai (ISA); Organisation = andar se kaise bana hai (pipeline, cache, clock).

## 1.2 Why Architecture Matters: the Abstraction Stack

**Pehle socho:** Car chalte waqt aap steering aur brake use karte ho; engine ke andar petrol kaise jalta hai, ye jaanne ki zaroorat nahi. Har layer neeche wali layer ki details **chhupa** deti hai. Computer bhi aise hi layers mein bana hai, warna ek program likhne ke liye transistors yaad rakhne padte.

**Definition:** The **abstraction stack** is the set of layers from which a computer system is built, each layer using the one below it while hiding its details: AI/ML models → applications & languages → operating system → **ISA** → datapath & control → logic gates → transistors.

**Mushkil shabd, aasaan matlab:**

### **abstraction**

Details chhupa ke sirf kaam ki cheez dikhana. Steering wheel = engine ka abstraction.

### **ISA as contract**

Software (compiler, OS) ISA ke hisaab se likha jaata hai, hardware ISA ko implement karta hai. Dono contract maante rahein to dono alag alag badal sakte hain: purana program naye chip pe chalta hai.

### **datapath & control**

Datapath = registers, ALU, MUX jinse data chalta hai. Control = signals jo batate hain kab kya karna hai.

### **transistor**

Silicon ka switch: ON = 1, OFF = 0. Gates transistors se bante hain.

### **Moore's Law**

Chip pe transistors ki ginti lagbhag har 2 saal mein double hoti hai. Ye clock speed ke baare mein NAHI hai.

### **VLSI flow**

Chip banane ke steps: Specification → RTL (Verilog) → Simulation → Synthesis → Gate netlist → Place & Route → Physical verification (DRC/LVS/STA) → Fabrication → Post-silicon validation.

**Kaise kaam karta hai (ek line code har layer se guzarti hai):**

1. Aap likhte ho  $A[i] = B[i] + C[i]$  (Python / C layer).
2. Compiler isse 4 MIPS instructions banata hai: lw (B[i] lao), lw (C[i] lao), add, sw (A[i] mein rakho). Ye ISA layer hai.
3. Datapath: register file do numbers ALU ko deta hai, ALU jodta hai, result wapas register file mein likha jaata hai.
4. ALU ka adder XOR aur AND gates se bana hai (carry logic).
5. Har gate transistors (switches) se bana hai.
6. Ye course beech ki layers chadhta hai: gates → datapath → ISA.



This course climbs the middle: logic gates → datapath & control → ISA

### The abstraction stack

How to draw: Seven stacked bars, widest at the bottom (transistors); highlight the ISA bar and write "HW/SW contract".

**Diagram kaise padhein:** Neeche sabse chaudi patti transistors, upar sabse abstract (AI/ML). Highlight wali patti ISA hai: uske upar sab software, neeche sab hardware. Left ka arrow upar ki taraf "more abstract" dikhata hai.



### $A[i] = B[i] + C[i]$ through the layers

How to draw: Four boxes down the page: source line → lw/lw/add/sw → regfile→ALU→regfile → XOR/AND carry gates.

**Diagram kaise padhein:** Upar se neeche padho: ek hi line pehle source code, phir lw/lw/add/sw, phir regfile → ALU → regfile, aur aakhir mein XOR/AND gates. Har arrow = ek layer neeche jaana.

**History jo slides mein hai:** 1847 Boole (algebra of logic), 1937 Shannon (switching circuits Boolean algebra follow karte hain), 1947 transistor ne vacuum tube ki jagah li (ENIAC: lagbhag 18,000 tubes, 30 ton), 1958 Kilby ka pehla IC, 1971 Intel 4004: pehla single-chip CPU, lagbhag 2,300 transistors, 740 kHz, 4-bit. Apple M2 (2022): lagbhag 20 billion transistors.

**Solved example (write the answer like this):**

Explain the abstraction stack using  $A[i] = B[i] + C[i]$ . Why is the ISA called the hardware/software contract? (5 marks)

1. Draw the seven layers: AI/ML → applications → OS → ISA → datapath & control → gates → transistors.
2. Compiler maps the statement to `lw, lw, add, sw` (4 MIPS instructions).
3. Control unit sets the datapath: register file sends two operands to the ALU, sum is written back.
4. The ALU adder is built from XOR/AND gates (full adders); gates are CMOS transistors acting as switches.
5. ISA = contract: everything above is written to it, everything below must implement it; both sides can evolve independently.

**Answer: labelled stack + trace through 4 layers + contract line.**

**Galti mat karna:** OS ko ISA ke neeche mat rakhna; OS ISA ke upar chalta hai. Aur Moore's law = transistor count double, clock speed double nahi.

**Ek line mein:** Har layer neeche wali ko chhupati hai; ISA beech mein hardware/software ka contract hai.

## 1.3 Boolean Logic Basics: AND, OR, NOT, XOR Gates

**Pehle socho:** Ek bulb aur do switch. Switch **series** mein lagao: bulb tabhi jalega jab dono ON hon (AND). **Parallel** mein lagao: koi bhi ek ON ho to jalega (OR). Ek ulta switch jo dabane pe band ho jaaye: NOT. Pura computer inhi teen ideas se bana hai.

**Definition:** A **logic gate** is an electronic circuit that performs a Boolean function on one or more binary inputs (0 = LOW/false, 1 = HIGH/true) and produces a single binary output.

**Mushkil shabd, aasaan matlab:**

**binary signal (0/1)**

1 = TRUE = HIGH (lagbhag 3.3 V ya 5 V), 0 = FALSE = LOW (0 V). Sirf "voltage hai / nahi hai" pehchanna hota hai, isliye reliable.

**bubble**

Gate symbol pe chhota gola = NOT (ulta karna). AND + bubble = NAND.

**"+" aur "·"**

Boolean mein + ka matlab OR, · ka matlab AND. Isliye  $1 + 1 = 1$  (2 nahi).

**universal gate**

Aisa gate jisse akele har doosra gate ban jaaye. NAND aur NOR universal hain. Jaise  $\text{NOT } A = A \text{ NAND } A$ .

## parity

1s ki ginti odd hai ya even. XOR = 1 jab 1s ki ginti odd ho.

**Kaise kaam karta hai (har gate ka rule, ek 0/1 example ke saath):**

1. **NOT** ( $Y = A'$ ): input ulta.  $A = 1 \rightarrow Y = 0$ .
2. **AND** ( $Y = A \cdot B$ ): 1 sirf jab **saare** inputs 1.  $A = 1, B = 0 \rightarrow Y = 0$ .
3. **OR** ( $Y = A + B$ ): 1 jab **kam se kam ek** input 1.  $A = 1, B = 0 \rightarrow Y = 1$ .
4. **NAND** = AND phir NOT:  $Y = (AB)'$ . Sirf 1,1 pe 0, baaki sab pe 1.
5. **NOR** = OR phir NOT:  $Y = (A + B)'$ . Sirf 0,0 pe 1, baaki sab pe 0.
6. **XOR** ( $Y = A \oplus B = A'B + AB'$ ): 1 jab inputs **alag** hon.  $1,0 \rightarrow 1; 1,1 \rightarrow 0$ .
7. **XNOR** ( $Y = (A \oplus B)' = A'B' + AB$ ): 1 jab inputs **barabar** hon (equality detector).  $1,1 \rightarrow 1$ .

| A | B | AND | OR | NAND | NOR | XOR | XNOR |
|---|---|-----|----|------|-----|-----|------|
| 0 | 0 | 0   | 0  | 1    | 1   | 0   | 1    |
| 0 | 1 | 0   | 1  | 1    | 0   | 1   | 0    |
| 1 | 0 | 0   | 1  | 1    | 0   | 1   | 0    |
| 1 | 1 | 1   | 1  | 0    | 0   | 0   | 1    |

Table mein dekho: NAND column = AND column ulta; NOR = OR ulta; XNOR = XOR ulta. OR aur XOR sirf last row (1,1) mein alag hain.



**NOT**  
 $Y = A'$



**AND**  
 $Y = A \cdot B$



**OR**  
 $Y = A + B$



**NAND**  
 $Y = (A \cdot B)'$



**NOR**  
 $Y = (A + B)'$



**XOR**  
 $Y = A \oplus B = A'B + AB'$



**XNOR**  
 $Y = (A \oplus B)' = A'B' + AB$

### Symbols of the seven gates with expressions

How to draw: AND = D-shape, OR = shield/curved back, XOR = OR with a second back curve; add a small bubble for NOT/NAND/NOR/XNOR.

**Diagram kaise padhein:** AND = "D" shape (seedhi peeth). OR = curved peeth, nukeeli naak. XOR = OR ke peeche ek extra curve. Output pe bubble ho to ulta: NAND, NOR, XNOR. NOT = triangle + bubble. Har symbol ke neeche uska formula likha hai.

**Formula:** AND  $A \cdot B$  · OR  $A + B$  · NOT  $A'$  · NAND  $(AB)'$  · NOR  $(A + B)'$  · XOR  $A'B + AB'$  · XNOR  $A'B' + AB$

Verilog operators: & AND, | OR, ~ NOT, ^ XOR, ~^ XNOR.

n-input XOR = 1 when the number of 1s is odd.

### Solved example 1 (class quick check):

For  $A = 1, B = 0$ , which of AND, OR, NAND, NOR, XOR, XNOR give 1?

1. AND: dono 1 chahiye,  $B = 0$  hai  $\rightarrow 0$ .
2. OR: ek 1 kaafi hai  $\rightarrow 1$ .
3. NAND = AND ulta =  $0$  ulta  $\rightarrow 1$ .
4. NOR = OR ulta =  $1$  ulta  $\rightarrow 0$ .
5. XOR: inputs alag ( $1$  aur  $0$ )  $\rightarrow 1$ . XNOR = XOR ulta  $\rightarrow 0$ .

**Answer: OR, NAND, XOR give 1.**

### Solved example 2:

(a) A 3-input XOR gets 1, 1, 1. Output? (b) Which gate is assign  $y = \sim(a \wedge b)$ ;

1. (a) 1s ki ginti = 3, jo odd hai. XOR rule: odd  $\rightarrow 1$ . Step by step:  $1 \oplus 1 = 0$ , phir  $0 \oplus 1 = 1$ .
2. (b)  $\wedge$  = XOR, uske bahar  $\sim$  = NOT. NOT of XOR = XNOR.

**Answer: (a) 1; (b) XNOR.**

**Galti mat karna:** "+" ko jod mat samjho:  $A + 1 = 1, 2$  nahi. Aur OR ko XOR mat samjho: 1,1 pe OR = 1 par XOR = 0. NAND  $\neq$  AND (bubble bhoolna common galti hai).

**Trick:** AND = series, OR = parallel. XOR = "alag ho to 1", XNOR = "same ho to 1".

**Ek line mein:** AND sab 1 maange, OR ek 1 maange, XOR alag maange; bubble lagao to ulta.

## 1.4 Truth Tables

**Pehle socho:** Dukaan ki **poori price list**: har item ka daam likha hai, kuch chhoota nahi. Truth table bhi ek circuit ki poori list hai: har possible input ke liye output. Agar do expressions ki truth table same hai, to dono circuits same kaam karte hain, chahe dikhne mein alag hon.

**Definition:** A **truth table** lists the output of a logic function for every possible combination of its inputs. For  $n$  inputs it has  $2^n$  rows, written in binary counting order.

### Mushkil shabd, aasaan matlab:

#### row

Inputs ka ek combination. 3 inputs ( $A, B, C$ )  $\rightarrow 000, 001, 010, 011, 100, 101, 110, 111 = 8$  rows.

#### $2^n$ rows

Har input 0 ya 1 (2 choices).  $n$  inputs  $\rightarrow 2 \times 2 \times 2$  ( $n$  baar)  $= 2^n$ . 4 inputs  $\rightarrow 16$  rows.

### MSB (first variable)

Sabse left wala bit, sabse bada weight.  $ABC = 101$  mein  $A = \text{MSB}$ ,  $\text{value} = 4 + 0 + 1 = 5$ .

### row number / minterm index

Row ka decimal value. Row  $ABC = 011 \rightarrow 0 + 2 + 1 = 3$ , isko  $m_3$  bolte hain (Unit 2 mein detail).

### helper column

Beech ka column jaise  $BC$ , taaki bada expression tukdon mein bane.

### Kaise kaam karta hai (expression se truth table banana):

1. Variables gino:  $n$ . Rows =  $2^n$ .
2. Inputs binary counting order mein likho: 000 se 111 tak ( $A = \text{MSB}$ ).
3. Har chhote tukde (jaise  $BC$ ) ka helper column banao.
4. Helper columns ko jod ke final output column banao.
5. Jin rows mein output 1 hai, unke numbers likho: yahi  $\Sigma m$  list hai.

**Example:**  $Y = A + BC$  (L01 homework).

| row | A | B | C | BC | $Y = A + BC$ |
|-----|---|---|---|----|--------------|
| 0   | 0 | 0 | 0 | 0  | 0            |
| 1   | 0 | 0 | 1 | 0  | 0            |
| 2   | 0 | 1 | 0 | 0  | 0            |
| 3   | 0 | 1 | 1 | 1  | 1            |
| 4   | 1 | 0 | 0 | 0  | 1            |
| 5   | 1 | 0 | 1 | 0  | 1            |
| 6   | 1 | 1 | 0 | 0  | 1            |
| 7   | 1 | 1 | 1 | 1  | 1            |

$A = 1$  wali chaaron rows mein  $Y = 1$  (OR mein ek 1 kaafi).  $A = 0$  pe  $Y = BC$ , jo sirf row 3 mein 1 hai. Isliye  $Y = \Sigma m(3, 4, 5, 6, 7)$ .

### Truth tables of the two-input gates

| A | B | AND | OR | NAND | NOR | XOR | XNOR |
|---|---|-----|----|------|-----|-----|------|
| 0 | 0 | 0   | 0  | 1    | 1   | 0   | 1    |
| 0 | 1 | 0   | 1  | 1    | 0   | 1   | 0    |
| 1 | 0 | 0   | 1  | 1    | 0   | 1   | 0    |
| 1 | 1 | 1   | 1  | 0    | 0   | 0   | 1    |

#### All two-input gates at a glance

How to draw: Write A,B columns 00/01/10/11 once, then one output column per gate.

**Diagram kaise padhein:** Pehle do columns A, B ki chaar rows (00, 01, 10, 11) hain; baaki har column ek gate ka output. Kisi gate ko pehchanna ho to uska column upar se neeche padho: AND = 0001, OR = 0111, XOR = 0110, NAND = 1110, NOR = 1000, XNOR = 1001.

**Formula:** rows =  $2^n$ ; distinct Boolean functions of n variables =  $2^{2^n}$ .

n = 2:  $2^2 = 4$  rows, functions =  $2^4 = 16$ . n = 3:  $2^3 = 8$  rows, functions =  $2^8 = 256$ .

#### Solved example 1 (direct):

(a) Rows in the truth table of a 5-input circuit? (b) How many distinct Boolean functions of 3 variables exist?

- (a)  $2^5 = 2 \times 2 \times 2 \times 2 \times 2 = 32$  rows.
- (b) Rows =  $2^3 = 8$ . Har row ka output 0 ya 1 ho sakta hai, to output column ke  $2 \times 2 \times (8 \text{ baar}) = 2^8 = 256$  alag pattern.

**Answer: (a) 32; (b) 256.**

#### Solved example 2 (exam level):

Construct the truth table of  $F = AB' + A'C + BC'$  and express F in  $\Sigma m$  form. (5 marks)

- 3 variables  $\rightarrow$  8 rows. Helper columns:  $AB'$ ,  $A'C$ ,  $BC'$ .
- $AB' = 1$  sirf jab A = 1, B = 0: rows 100, 101.
- $A'C = 1$  jab A = 0, C = 1: rows 001, 011.
- $BC' = 1$  jab B = 1, C = 0: rows 010, 110.
- F = OR of three: rows 1, 2, 3, 4, 5, 6 mein 1; rows 0 (000) aur 7 (111) mein 0.

| ABC | AB' | A'C | BC' | F |
|-----|-----|-----|-----|---|
| 000 | 0   | 0   | 0   | 0 |
| 001 | 0   | 1   | 0   | 1 |
| 010 | 0   | 0   | 1   | 1 |
| 011 | 0   | 1   | 0   | 1 |
| 100 | 1   | 0   | 0   | 1 |
| 101 | 1   | 0   | 0   | 1 |
| 110 | 0   | 0   | 1   | 1 |
| 111 | 0   | 0   | 0   | 0 |

Answer:  $F = \sum m(1, 2, 3, 4, 5, 6)$  ( $F = 0$  only when  $A = B = C$ ).

**Galti mat karna:** Rows random order mein likhoge to minterm numbers galat ho jaayenge. Hamesha 000, 001, 010 counting order. Aur 4 inputs = 16 rows, 8 nahi.

**Ek line mein:** n inputs  $\rightarrow 2^n$  rows, counting order mein; same truth table = same circuit.

## 1.5 Boolean Laws & Simplification

**Pehle socho:** Normal maths mein " $x \times 1 = x$ ", " $x + 0 = x$ " jaise rules se lamba expression chhota karte ho. Boolean mein bhi aise rules (laws) hain. Chhota expression = kam gates = kam area, kam power, zyada speed. Achhi baat: har variable sirf 0 ya 1 ho sakta hai, isliye har law ko  $A = 0$  aur  $A = 1$  daal ke khud check kar sakte ho.

**Definition: Boolean algebra** is the algebra of variables that take only the values 0 and 1, with the operations AND ( $\cdot$ ), OR ( $+$ ) and NOT ( $'$ ). Its laws allow an expression to be rewritten into a simpler equivalent expression with the same truth table.

**Mushkil shabd, aasaan matlab:**

### literal

Variable ka ek appearance, seedha ya bar ke saath.  $AB' + C$  mein 3 literals ( $A, B', C$ ).

### complement ( $A'$ )

A ka ulta.  $A = 1 \rightarrow A' = 0$ .

### dual

Law ka "mirror":  $+$  ko  $\cdot$  se,  $\cdot$  ko  $+$  se, 0 ko 1 se, 1 ko 0 se badlo (variables same rakho).  $A + 0 = A$  ka dual  $A \cdot 1 = A$ . Ek law sahi hai to uska dual bhi sahi.

### perfect induction

Law ko proof karna har possible 0/1 value daal ke. Sirf 2 ya 4 cases hote hain, isliye easy.

**Kaise kaam karta hai (har law + "kyun sahi hai" 0/1 se):**

| Law               | Formula                            | Kyun sahi hai (0/1 daal ke)                                                                                                         |
|-------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Identity          | $A + 0 = A; A \cdot 1 = A$         | $A = 0: 0 + 0 = 0; A = 1: 1 + 0 = 1$ . Dono baar answer = A.                                                                        |
| Null              | $A + 1 = 1; A \cdot 0 = 0$         | OR mein ek 1 ho to output 1, chahe A kuch bhi ho. AND mein ek 0 ho to output 0.                                                     |
| Idempotent        | $A + A = A; A \cdot A = A$         | $0 + 0 = 0, 1 + 1 = 1$ (OR, jod nahi). Isliye 2A jaisa kuch nahi hota.                                                              |
| Complement        | $A + A' = 1; A \cdot A' = 0$       | A aur A' mein ek hamesha 1 hai (OR $\rightarrow 1$ ) aur ek hamesha 0 hai (AND $\rightarrow 0$ ).                                   |
| Involution        | $(A')' = A$                        | Do baar ulta = seedha: $1 \rightarrow 0 \rightarrow 1$ .                                                                            |
| Commutative       | $A + B = B + A; AB = BA$           | Gate ko farak nahi padta kaunsa wire upar laga hai.                                                                                 |
| Associative       | $(A + B) + C = A + (B + C)$        | OR = "koi ek 1?"; brackets kahin bhi, jawab same. AND ke liye bhi.                                                                  |
| Distributive 1    | $A(B + C) = AB + AC$               | $A = 0$ : dono side 0. $A = 1$ : dono side $B + C$ .                                                                                |
| Distributive 2    | $A + BC = (A + B)(A + C)$          | $A = 1$ : dono side 1. $A = 0$ : left = BC, right = B.C. Same! (Normal maths mein ye nahi hota.)                                    |
| Absorption        | $A + AB = A; A(A + B) = A$         | $A = 1: 1 + B = 1 = A. A = 0: 0 + 0 = 0 = A$ . B ka koi role nahi.                                                                  |
| Redundant Literal | $A + A'B = A + B$                  | $A = 1$ : dono side 1. $A = 0$ : left = $1 \cdot B = B$ , right = B. Same.                                                          |
| De Morgan         | $(AB)' = A' + B'; (A + B)' = A'B'$ | $(AB)'$ sirf $A = B = 1$ pe 0 hai; $A' + B'$ bhi sirf tab 0 jab $A' = B' = 0$ , yaani $A = B = 1$ . Same row.                       |
| Consensus         | $AB + A'C + BC = AB + A'C$         | $BC = 1$ matlab $B = C = 1$ . Tab $A = 1$ ho to $AB = 1$ , $A = 0$ ho to $A'C = 1$ . BC kabhi akela kaam nahi aata, isliye hata do. |

**Formula (sabse zyada use hone wale):**  $A + AB = A \cdot A + A'B = A + B \cdot A + BC = (A + B)(A + C) \cdot (AB)' = A' + B' \cdot (A + B)' = A'B'$

Duality:  $+ \leftrightarrow \cdot, 0 \leftrightarrow 1. A + 0 = A \leftrightarrow A \cdot 1 = A$ .

**Solved example (write the answer like this):**

Prove (i)  $A + AB = A$  and (ii)  $A + A'B = A + B$  using the basic laws, naming each law. (5 marks)

1. (i)  $A + AB = A \cdot 1 + A \cdot B$  [Identity:  $A = A \cdot 1$ ]

2.  $= A(1 + B)$  [Distributive: take A common]

3.  $= A \cdot 1$  [Null:  $1 + B = 1$ ]

4.  $= A$  [Identity]

5. (ii)  $A + A'B = (A + A')(A + B)$  [Distributive 2:  $X + YZ = (X + Y)(X + Z)$ ]

$$6. = 1 \cdot (A + B) \text{ [Complement: } A + A' = 1]$$

$$7. = A + B \text{ [Identity]}$$

8. Check by truth table: both sides of (ii) are 0 only at  $A = B = 0$ , 1 elsewhere.

**Answer:** (i)  $A + AB = A$  (absorption); (ii)  $A + A'B = A + B$ .

**Galti mat karna:**  $(AB)' = A'B'$  likhna GALAT hai. De Morgan mein bar todne ke saath sign bhi badalta hai:  $(AB)' = A' + B'$ . Aur  $A + A = 2A$  nahi,  $A$  hai.

**Trick:** De Morgan = "**bar tod do, sign badlo**". Koi law yaad na aaye to  $A = 0$  aur  $A = 1$  daal ke check kar lo, 30 second lagte hain.

**Ek line mein:** Har Boolean law 0/1 daal ke check ho sakta hai; sabse kaam ke:  $A + AB = A$ ,  $A + A'B = A + B$ , aur De Morgan.

## 1.6 Worked Simplification Examples

**Pehle socho:** L01 ka example:  $Y = AB + AB'$ . Isko seedha banao to 2 AND, 1 OR, 1 NOT = 4 gates. Par  $AB + AB' = A(B + B') = A \cdot 1 = A$ . Matlab poora circuit sirf **ek wire** hai! Simplification ka yahi fayda hai: same kaam, kam hardware.

**Definition: Simplification** is the process of reducing a Boolean expression to fewer terms and literals (hence fewer gates) by applying Boolean laws, while keeping the same truth table.

**Mushkil shabd, aasaan matlab:**

**term**

+ ke beech ka tukda.  $AB' + BC$  mein do terms:  $AB'$  aur  $BC$ .

**common factor**

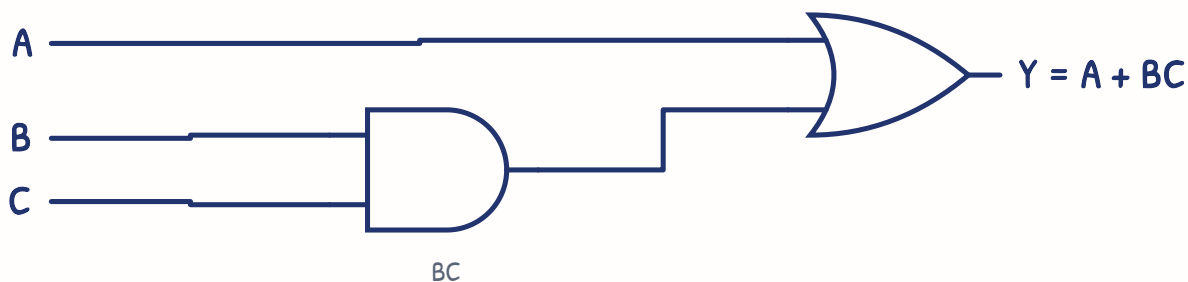
Jo variable do terms mein common hai, use bahar nikalo:  $AB'C + AB'C' = AB'(C + C')$ .

**complement pair**

$X$  aur  $X'$  saath mein:  $X + X' = 1$  (OR mein),  $X \cdot X' = 0$  (AND mein). Yahi variable ko "cancel" karta hai.

**Kaise kaam karta hai (5-step strategy):**

1. Aise do terms dhoondo jo sirf ek variable mein alag hon (ek mein  $X$ , doosre mein  $X'$ ).
2. Common factor bahar nikalo, andar  $X + X' = 1$  bachega, wo variable gayab.
3. Absorption ( $A + AB = A$ ) aur  $A + A'B = A + B$  lagao.
4. Kisi group ke upar bar ho to De Morgan se bar tod do, andar tak.
5. Final answer ko truth table (ya 2 rows) se check karo.



#### Gate circuit for $Y = A + BC$ (L01 homework)

How to draw: AND gate for  $B \cdot C$  first, then an OR gate combining  $A$  with the AND output.

**Diagram kaise padhein:** Left se  $B$  aur  $C$  ek AND gate mein jaate hain (output  $BC$ ). Phir  $A$  aur  $BC$  ek OR gate mein jaate hain, output  $Y = A + BC$ . Trace:  $A = 0, B = 1, C = 1 \rightarrow BC = 1 \rightarrow Y = 0 + 1 = 1$ .

#### Solved example 1 (class quiz):

Simplify  $F = AB'C + A'BC + ABC + AB'C'$ .

1.  $F = AB'C + AB'C' + A'BC + ABC$  [Commutative: rearrange so matching terms are together]
2.  $= AB'(C + C') + BC(A' + A)$  [Distributive: take  $AB'$  and  $BC$  common]
3.  $= AB' \cdot 1 + BC \cdot 1$  [Complement:  $C + C' = 1, A' + A = 1$ ]
4.  $= AB' + BC$  [Identity]
5. Check:  $F$  should be 1 at rows 100, 101 ( $AB'$ ) and 011, 111 ( $BC$ ) =  $m_4, m_5, m_3, m_7$ . Original terms are exactly  $m_5, m_3, m_7, m_4$ . ✓

**Answer:**  $F = AB' + BC$ .

#### Solved example 2 (class quiz, product of sums):

Simplify  $F = (A' + B)(A' + B')(A + B)$ .

1.  $(A' + B)(A' + B') = A' + B \cdot B'$  [Distributive 2:  $(X + Y)(X + Z) = X + YZ$ , with  $X = A'$ ]
2.  $= A' + 0$  [Complement:  $B \cdot B' = 0$ ]
3.  $= A'$  [Identity]
4. Now  $F = A'(A + B) = A'A + A'B$  [Distributive 1]
5.  $= 0 + A'B$  [Complement:  $A'A = 0$ ]
6.  $= A'B$  [Identity]
7. Check  $A = 0, B = 1$ :  $(1 + 1)(1 + 0)(0 + 1) = 1 \cdot 1 \cdot 1 = 1$ , and  $A'B = 1 \cdot 1 = 1$ . ✓

**Answer:**  $F = A'B$ .

### Solved example 3 (exam level, Lab 2 Q27):

Simplify  $F = ((A + B) \cdot C)' + (A \cdot B)'$  and draw the circuit.

1. Inner bar:  $(A + B)' = A'B'$  [De Morgan]
2. So first term =  $(A'B'C)' = A + B + C'$  [De Morgan for 3 variables:  $(XYZ)' = X' + Y' + Z'$ , and  $(A')' = A$ ]
3. Second term:  $(AB)' = A' + B'$  [De Morgan]
4.  $F = A + B + C' + A' + B'$  [just write both together]
5.  $= (A + A') + (B + B') + C'$  [Commutative / associative: regroup]
6.  $= 1 + 1 + C'$  [Complement]
7.  $= 1$  [Null: anything + 1 = 1]

Answer:  $F = 1$  always; the "circuit" is the output tied to logic 1 (VDD), zero gates.

### Solved example 4 (Lab 2 Q12):

Simplify  $F = A'B'C + A'BC + AB'C + ABC + A'B$ .

1.  $A'B'C + A'BC = A'C(B' + B) = A'C$  [Distributive, Complement, Identity]
2.  $AB'C + ABC = AC(B' + B) = AC$  [same three laws]
3.  $A'C + AC = C(A' + A) = C$  [same three laws]
4.  $F = C + A'B$  [nothing more combines:  $A'B$  has no  $C$ ]

Answer:  $F = C + A'B$ .

**Galti mat karna:** Pehla grouping karke ruk mat jao; har step ke baad dobara dekho ki absorption ya complement pair bacha to nahi. Aur factor nikalte waqt koi term gum na ho: wapas multiply karke check karo.

**Trick:** Exam mein har line ke aage bracket mein law ka naam likho; marks usi ke milte hain.

**Ek line mein:** Common factor nikalo  $\rightarrow X + X' = 1$  se variable udao  $\rightarrow$  absorption/De Morgan  $\rightarrow$  truth table se check.

## 1.E1 Extra from slides: Verilog & HDL Fundamentals

**Pehle socho:** Ek billion gates ka chip haath se draw nahi ho sakta. Isliye engineers circuit ko **text** mein likhte hain (jaise code), phir software usse test karta hai (simulation) aur asli gates mein badalta hai (synthesis). Par dhyan: ye "program" nahi hai jo line by line chale; ye ek **wiring diagram** hai jisme sab gates ek saath kaam karte hain.

**Definition:** A **Hardware Description Language (HDL)** such as Verilog describes the structure and behaviour of digital circuits as text, so that they can be **simulated** (behaviour checked over simulated time) and **synthesised** (converted into a gate-level netlist for fabrication).

**Mushkil shabd, aasaan matlab:**

#### **module / endmodule**

Ek hardware block ka dabba, ports (input/output) ke saath. Jaise module and\_gate (input a, input b, output y);

#### **wire**

Asli taar: jo usse chala raha hai uski value hamesha dikhata hai. assign ke saath use hota hai.

#### **reg**

Variable jo always ya initial block ke andar value paata hai aur agle assignment tak rakhta hai. reg ka matlab hamesha flip-flop nahi hai.

#### **assign**

Continuous assignment: assign y = a & b; matlab y hamesha a AND b hai; a ya b badle to y turant badle.

#### **always @(\*)**

Combinational block: koi bhi input badle to block dobara chale. Har case cover karna zaroori (default), warna latch ban jaata hai.

#### **always @(posedge clk)**

Sequential block: sirf clock ke 0→1 edge pe chale. Andar <= (non-blocking) use karo.

#### **testbench**

Bina ports wala module jo design (DUT) ko inputs deta hai aur outputs print karta hai. #10 = 10 time units ruko.

#### **simulation vs synthesis**

Simulation (iverilog/vvp, waveform GTKWave mein) = check karna, kuch banta nahi. Synthesis (Synopsys DC, Cadence Genus) = asli gate netlist banana.

**Kaise kaam karta hai (Lab 1 ka flow):**

1. Design file likho, jaise and\_gate.v.
2. Testbench likho (and\_tb.v) jo saare input combinations deta hai.
3. Compile: iverilog -o and.vvp and\_gate.v and\_tb.v (saari files ek saath).
4. Run: vvp and.vvp (text output aata hai, aur VCD waveform file banti hai).
5. Waveform dekho: gtkwave and.vcd.

```
// 2-input AND gate
module and_gate (
    input wire a,
    input wire b,
    output wire y
);
    assign y = a & b; // y updates whenever a or b changes
endmodule
```

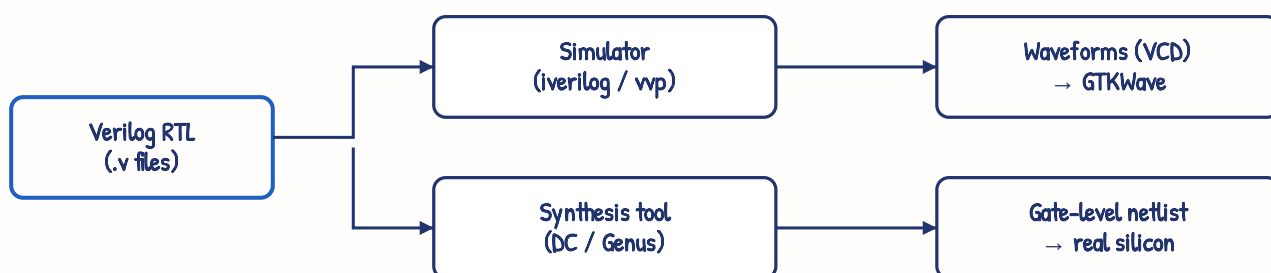
### Line by line:

1. // 2-input AND gate: comment, compiler ignore karta hai.
2. module and\_gate (: naya hardware block shuru, naam and\_gate.
3. input wire a, input wire b: do input taar a aur b.
4. output wire y: ek output taar y.
5. assign y = a & b;; y ko hamesha a AND b se jod do. Ye ek AND gate ban jaata hai.
6. endmodule: block khatam.

```
module y_abc (input a, b, c, output y);
    assign y = a | (b & c); // Y = A + BC, version 1
endmodule

module y_abc_v2 (input a, b, c, output reg y);
    always @(*) // re-run when a, b or c changes
        y = a | (b & c); // same circuit, version 2 (y must be reg)
endmodule
```

Dono modules same circuit (D1.6a) banate hain. Version 2 mein y ko reg declare kiya kyunki usse always block ke andar value mil rahi hai.



same language, two audiences: one pretends to be hardware, one builds it

### Simulation vs synthesis

How to draw: One RTL box splitting into two paths: simulator → waveforms, synthesis → netlist → silicon.

**Diagram kaise padhein:** Left mein ek hi Verilog file. Upar ka raasta: simulator → waveform (sirf check). Neeche ka raasta: synthesis tool → gate netlist → asli silicon. Same language, do kaam.

### Solved example 1 (lab MCQ):

$a = 4'b0101$ ,  $b = 4'b0010$ . Find  $a \& b$  and  $a \&\& b$ .

1. & bitwise hai: bit by bit AND. 0101 AND 0010: bit3 0-0 = 0, bit2 1-0 = 0, bit1 0-1 = 0, bit0 1-0 = 0 → 0000.

2. && logical hai: poor number zero hai ya nahi.  $a = 5$  (non-zero  $\rightarrow$  true),  $b = 2$  (non-zero  $\rightarrow$  true).  
true AND true = 1 (ek bit).

**Answer:**  $a \& b = 4'b0000$ ;  $a \&\& b = 1'b1$ .

### Solved example 2 (write the answer like this):

*Differentiate wire and reg, and simulation and synthesis. (5 marks)*

1. wire: a net driven continuously (by assign or a module output); it has no memory.
2. reg: a variable assigned inside always/initial; it holds its value until the next assignment.
3. Simulation runs the HDL on a computer over simulated time to check behaviour (waveforms); nothing is built.
4. Synthesis converts the same RTL into a gate-level netlist for fabrication or FPGA.
5. Example: assign  $y1 = a \& b$ ; (wire) and always @(\*)  $y2 = a \& b$ ; (reg) both synthesise to one AND gate.

**Answer:** wire  $\leftrightarrow$  assign, reg  $\leftrightarrow$  always/initial; simulation checks, synthesis builds.

**Galti mat karna:** always ke andar wire ko assign karna error hai (reg chahiye). always @(\*) mein case ka default bhoole to latch ban jaata hai. Waveform khaali aaye to check karo \$dumpfile aur \$dumpvars dono likhe hain.

**Trick:** "assign  $\rightarrow$  wire, always  $\rightarrow$  reg". Verilog padhte waqt socho "ye taar kahan jud raha hai", "pehle ye line chalegi" nahi.

**Ek line mein:** Verilog = circuit ka text roop; assign se wire, always se reg; simulation check karta hai, synthesis gates banata hai.

### Exam mein is unit se kya aata hai $\rightarrow$ kaise solve karein

| Question type                                                        | Method (steps)                                                                                                                                                                     |
|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Given inputs, which gate gives 1? / identify gate from output column | 1. Gate table (1.3) yaad karo: AND 0001, OR 0111, NAND 1110, NOR 1000, XOR 0110, XNOR 1001. 2. Given row pe har gate ka rule lagao. 3. Bubble = ulta.                              |
| Count rows / count functions                                         | 1. n gino. 2. Rows = $2^n$ . 3. Functions = $2^{\wedge}(\text{rows})$ . ( $n = 3$ : 8 rows, 256 functions.)                                                                        |
| Truth table from expression, write $\Sigma m$                        | 1. $2^n$ rows counting order. 2. Har term ka helper column. 3. OR karke F. 4. $F = 1$ wali rows ke decimal numbers = $\Sigma m$ .                                                  |
| Simplify an expression (MCQ or 5 marks)                              | 1. $X / X'$ wale term pairs dhoondo. 2. Factor nikalo, $X + X' = 1$ . 3. Absorption, $A + A'B = A + B$ . 4. Bar ho to De Morgan. 5. Har line pe law ka naam. 6. 1-2 rows se check. |

|                                                      |                                                                                                                                                                                              |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prove a law (absorption, $A + A'B$ )                 | 1. Identity se $A = A \cdot 1$ likho. 2. Distributive se factor. 3. Null/complement. 4. Identity. 5. Truth table se verify.                                                                  |
| Dual of an expression                                | $+ \leftrightarrow \cdot$ , $0 \leftrightarrow 1$ ; variables aur bars mat chhuo. $A + 0 = A \rightarrow A \cdot 1 = A$ .                                                                    |
| Architecture vs organisation classify                | Program ke result pe asar? Haan $\rightarrow$ architecture (ISA, registers, formats).<br>Sirf speed pe $\rightarrow$ organisation (cache, pipeline, clock).                                  |
| Abstraction stack / $A[i] = B[i] + C[i]$             | 7 layers draw karo, ISA highlight. Trace: lw, lw, add, sw $\rightarrow$ regfile $\rightarrow$ ALU $\rightarrow$ regfile $\rightarrow$ XOR/AND $\rightarrow$ transistors.                     |
| Verilog: what does this line implement / wire vs reg | 1. Operator padho ( $\&   \sim \wedge \vee$ ). 2. Bahar ka $\sim$ = ulta. 3. always ke andar assign $\rightarrow$ reg; assign $\rightarrow$ wire. 4. $\&\&$ = 1-bit logical, $\&$ = bitwise. |

# Unit 2 · Logic Minimisation & Universal Gates

**Is unit ki kahani:** Unit 1 mein laws se expression chhota kiya, par har baar "kaunsa law lagaun?" sochna padta tha. Is unit mein ek seedha tareeka hai: truth table  $\rightarrow \Sigma m / \Pi M \rightarrow$  **K-map** pe gole banao  $\rightarrow$  minimum expression. Phir De Morgan aur bubble pushing se wahi circuit sirf NAND ya sirf NOR gates se banana seekhte hain, kyunki factory mein NAND sabse sasta hai. Unit 3 mein inhi tricks se MUX, decoder aur adders design honge.

## 2.1 Motivation for Minimisation (Cost/Power)

**Pehle socho:** Ek hi cake ki do recipes: ek mein 16 steps, doosri mein 2. Cake same bana, par doosri mein mehnat, gas, samay sab kam. Circuit mein bhi same truth table kai tarah se ban sakti hai; jisme kam gates hon wo chhota, sasta, thanda aur fast.

**Definition: Logic minimisation** is the process of finding the simplest circuit (fewest gates, gate inputs, literals and logic levels) that implements a given Boolean function, i.e. with the same truth table.

**Mushkil shabd, aasaan matlab:**

### literal

Variable ka ek appearance (seedha ya bar wala).  $AB + AB' + A'B = 6$  literals.

### area

Chip pe gates ki jagah. Chhota die  $\rightarrow$  ek wafer pe zyada chips  $\rightarrow$  sasta.

### dynamic power

Gates ke switch ( $0 \leftrightarrow 1$ ) hone mein lagne wali bijli. Kam gates  $\rightarrow$  kam switching  $\rightarrow$  kam power.

### propagation delay

Input badalne ke baad output badalne mein lagne wala time. Gate ki chain jitni lambi, delay utna zyada.

### logic levels

Input se output tak kitne gates paar karne padte hain. AND-OR = 2 levels.

### logic synthesis

Software (Synopsys Design Compiler, Cadence Genus) jo Verilog ko minimum gates mein badalta hai, automatically.

**Kaise kaam karta hai (cost kaise naapte hain):**

1. Expression likho, terms (products) gino.
2. Literals gino: har variable appearance = 1.
3. Gates gino: har product term ek AND, terms ko jodne ke liye ek OR, har alag bar wala variable ek NOT.

4. Simplify karo (laws ya K-map) aur dobara gino.
5. Chaar fayde likho: **area, power, speed (delay), cost**.

Class slide: ek typical 4-variable function lagbhag 16 gates se 2 gates tak aa sakta hai. Apple M3 mein 98 billion transistors (TSMC N3E) hain, isliye 1% bachat bhi bahut paisa hai. CMOS NAND = 4 transistors, CMOS AND = NAND + inverter = 6 transistors.

### Solved example 1:

$F = AB + AB' + A'B$ . Count literals before and after simplification, and the gates saved.

1. Before:  $AB$  (2) +  $AB'$  (2) +  $A'B$  (2) = 6 literals.
2. Gates before: 3 two-input ANDs + 1 three-input OR + 2 NOTs ( $A'$ ,  $B'$ ) = 6 gates.
3. Simplify:  $AB + AB' = A(B + B') = A \cdot 1 = A$ .
4.  $F = A + A'B = A + B$  (redundant-literal law, 1.5).
5. After:  $A + B$  = 2 literals, 1 OR gate.

**Answer: 6 literals → 2 literals; 6 gates → 1 gate.**

### Solved example 2 (write the answer like this):

*Why is logic minimisation important in VLSI design? (5 marks)*

1. Area: each gate occupies silicon; fewer gates give a smaller die, more dies per wafer and higher yield.
2. Power: dynamic power  $\propto$  switched capacitance  $\times V^2 \times f$ ; fewer gates switch less capacitance.
3. Delay: fewer levels and shorter paths reduce the critical path, allowing a higher clock frequency.
4. Cost: silicon area and yield dominate chip cost; at billions of gates even 1% matters.
5. Example:  $F = AB + AB' = A$  removes four gates; synthesis tools (Design Compiler, Genus) do this automatically.

**Answer: area, power, delay, cost + one example.**

**Galti mat karna:** "Kam terms = hamesha kam literals" nahi hota; dono gino. Aur kabhi kabhi speed (kam levels) ke liye zyada gates lagaye jaate hain.

**Ek line mein:** Same truth table, kam gates → kam area, kam power, zyada speed, kam cost.

## 2.2 Canonical Forms: SOP and POS

**Pehle socho:** Ek alarm ka rule do tarah likh sakte ho. (1) "In situations mein **bajna chahiye**" (list of 1s) = SOP. (2) "In situations mein **chup rehna chahiye**" (list of 0s) = POS. Dono same alarm describe karte

hain. Truth table se seedha expression likhne ka yahi standard tareeka hai.

**Definition:** A **minterm** is a product (AND) term containing every variable exactly once (true or complemented) that equals 1 for exactly one input row. A **maxterm** is a sum (OR) term containing every variable exactly once that equals 0 for exactly one row. **Canonical SOP** (sum of products) is the OR of the minterms of the rows where  $F = 1$ ; **canonical POS** (product of sums) is the AND of the maxterms of the rows where  $F = 0$ .

**Mushkil shabd, aasaan matlab:**

**product term**

Variables ka AND, jaise  $AB'C$ . "Product" = AND.

**sum term**

Variables ka OR, jaise  $(A + B' + C)$ . "Sum" = OR.

**minterm  $m_k$**

Row  $k$  ka AND term, sirf row  $k$  pe 1. Rule: **bit 1  $\rightarrow X$ , bit 0  $\rightarrow X'$** . Row  $101 = 5 \rightarrow m_5 = AB'C$  ( $A = 1, B = 0, C = 1$  daalo:  $1 \cdot 1 \cdot 1 = 1$ ).

**maxterm  $M_k$**

Row  $k$  ka OR term, sirf row  $k$  pe 0. Rule **ulta: bit 0  $\rightarrow X$ , bit 1  $\rightarrow X'$** . Row  $010 = 2 \rightarrow M_2 = A + B' + C$  ( $A = 0, B = 1, C = 0$  daalo:  $0 + 0 + 0 = 0$ ).

**canonical**

"Standard, poora": har term mein har variable hai. Chhota nahi kiya gaya.

**$\Sigma m( )$  aur  $\Pi M( )$**

Shorthand.  $\Sigma$  = OR of minterms,  $\Pi$  = AND of maxterms.  $F = \Sigma m(1, 3)$  matlab  $F = m_1 + m_3$ .

**Kaise kaam karta hai (truth table  $\rightarrow \Sigma m \rightarrow$  expression, aur  $\Pi M$ ):**

1. Truth table mein  $F = 1$  wali rows dhoondo, unke decimal numbers likho  $\rightarrow \Sigma m$  list.
2. Har aisi row ka minterm likho ( $1 \rightarrow X, 0 \rightarrow X'$ ), sabko + se jodo  $\rightarrow$  **canonical SOP**.
3.  $F = 0$  wali rows ke numbers  $\rightarrow \Pi M$  list (ye  $\Sigma m$  list ke bache hue numbers hi hain).
4. Har aisi row ka maxterm likho ( $0 \rightarrow X, 1 \rightarrow X'$ ), brackets ko AND karo  $\rightarrow$  **canonical POS**.
5. Check:  $\Sigma m$  list +  $\Pi M$  list = 0 se  $2^n - 1$  tak har number exactly ek baar.

$$F = \sum m(1, 3, 5, 7) = \prod M(0, 2, 4, 6)$$

| m | A | B | C | F | minterm (if F=1) | maxterm (if F=0) |
|---|---|---|---|---|------------------|------------------|
| 0 | 0 | 0 | 0 | 0 |                  | $M_0 = A+B+C$    |
| 1 | 0 | 0 | 1 | 1 | $m_1 = A'B'C$    |                  |
| 2 | 0 | 1 | 0 | 0 |                  | $M_2 = A+B'+C$   |
| 3 | 0 | 1 | 1 | 1 | $m_3 = A'BC$     |                  |
| 4 | 1 | 0 | 0 | 0 |                  | $M_4 = A'+B+C$   |
| 5 | 1 | 0 | 1 | 1 | $m_5 = AB'C$     |                  |
| 6 | 1 | 1 | 0 | 0 |                  | $M_6 = A'+B'+C$  |
| 7 | 1 | 1 | 1 | 1 | $m_7 = ABC$      |                  |

#### Truth table with minterms and maxterms

How to draw: Add two columns to the truth table: write the minterm on 1-rows and the maxterm on 0-rows.

**Diagram kaise padhein:** Har row mein pehle m (row number), phir A B C, phir F. F = 1 wali rows (highlighted) pe minterm likha hai, F = 0 wali rows pe maxterm. Row 3 dekho: 011, F = 1  $\rightarrow m_3 = A'BC$ . Row 2: 010, F = 0  $\rightarrow M_2 = A + B' + C$ . Isliye  $F = \sum m(1, 3, 5, 7) = \prod M(0, 2, 4, 6)$ .

**Formula:** minterm:  $1 \rightarrow X, 0 \rightarrow X'$  · maxterm:  $0 \rightarrow X, 1 \rightarrow X'$

$$F = \sum m(1\text{-rows}) = \prod M(0\text{-rows}); F' = \sum m(0\text{-rows}); m_k' = M_k.$$

n variables: number of maxterms =  $2^n - (\text{number of minterms})$ . 4 variables with 6 minterms  $\rightarrow 16 - 6 = 10$  maxterms.

#### Solved example 1 (identify terms):

(a) Minterm for  $A = 1, B = 0, C = 1, D = 0$ . (b) Maxterm for  $A = 0, B = 1, C = 1$ .

- (a)  $1010 = 8 + 0 + 2 + 0 = 10$ . Minterm rule ( $1 \rightarrow X, 0 \rightarrow X'$ ):  $A B' C D'$ . So  $m_{10} = AB'CD'$ .
- (b)  $011 = 0 + 2 + 1 = 3$ . Maxterm rule ( $0 \rightarrow X, 1 \rightarrow X'$ ):  $A, B', C'$ . So  $M_3 = A + B' + C'$ .
- Check (b): put 0, 1, 1  $\rightarrow 0 + 0 + 0 = 0$ . Maxterm sirf apni row pe 0 hota hai. ✓

**Answer:**  $m_{10} = AB'CD'$ ;  $M_3 = (A + B' + C')$ .

#### Solved example 2 (full conversion, Lab 2 Q2/Q3):

$F(A, B, C)$  is 1 for rows 001, 010, 100, 111 and 0 elsewhere. Write  $\sum m$ , canonical SOP,  $\prod M$  and canonical POS.

| row | ABC | F | term                |
|-----|-----|---|---------------------|
| 0   | 000 | 0 | $M_0 = A + B + C$   |
| 1   | 001 | 1 | $m_1 = A'B'C$       |
| 2   | 010 | 1 | $m_2 = A'BC'$       |
| 3   | 011 | 0 | $M_3 = A + B' + C'$ |
| 4   | 100 | 1 | $m_4 = AB'C'$       |
| 5   | 101 | 0 | $M_5 = A' + B + C'$ |
| 6   | 110 | 0 | $M_6 = A' + B' + C$ |
| 7   | 111 | 1 | $m_7 = ABC$         |

1. 1-rows: 1, 2, 4, 7  $\rightarrow F = \sum m(1, 2, 4, 7)$ .

2. Minterms ( $1 \rightarrow X, 0 \rightarrow X'$ ):  $m_1 = A'B'C$ ,  $m_2 = A'BC'$ ,  $m_4 = AB'C'$ ,  $m_7 = ABC$ .

3. Canonical SOP:  $F = A'B'C + A'BC' + AB'C' + ABC$ .

4. 0-rows: 0, 3, 5, 6  $\rightarrow F = \prod M(0, 3, 5, 6)$ . (Check: 1, 2, 4, 7 + 0, 3, 5, 6 = all of 0 to 7. ✓)

5. Maxterms ( $0 \rightarrow X, 1 \rightarrow X'$ ):  $M_0 = A + B + C$ ,  $M_3 = A + B' + C'$ ,  $M_5 = A' + B + C'$ ,  $M_6 = A' + B' + C$ .

6. Canonical POS:  $F = (A + B + C)(A + B' + C')(A' + B + C')(A' + B' + C)$ .

**Answer:**  $F = \sum m(1, 2, 4, 7) = \prod M(0, 3, 5, 6)$ . (This F is the 3-input XOR: 1 when the count of 1s is odd.)

### Solved example 3 (expression $\rightarrow \sum m$ , mock NAT):

How many minterms are 1 in  $F(A, B, C) = A + B'C$ ? Write F in  $\sum m$  and  $\prod M$ .

1. Term A:  $A = 1$ , B and C kuch bhi  $\rightarrow$  rows 100, 101, 110, 111 = 4, 5, 6, 7.

2. Term  $B'C$ :  $B = 0$ ,  $C = 1$ , A kuch bhi  $\rightarrow$  rows 001, 101 = 1, 5.

3. Union (5 do baar aaya, ek baar gino): 1, 4, 5, 6, 7  $\rightarrow$  5 minterms.

4. Bache hue: 0, 2, 3  $\rightarrow \prod M(0, 2, 3)$ .

**Answer:** 5 minterms;  $F = \sum m(1, 4, 5, 6, 7) = \prod M(0, 2, 3)$ .

**Galti mat karna:** Maxterm mein minterm wala rule lagana THE trap hai.  $M_2$  (row 010) =  $A + B' + C$  hai,  $A'BC'$  nahi. Yaad rakho: maxterm us row pe 0 dena chahiye, to har literal 0 hona chahiye.

**Trick:** "SOP = 1s = m = seedha 1; POS = 0s = M = seedha 0." Jo bit term ko "seedha" (bina bar) likhwata hai: minterm mein 1, maxterm mein 0.

**Ek line mein:** SOP = 1-rows ke minterms ka OR; POS = 0-rows ke maxterms ka AND; dono lists milke saare rows cover karti hain.

## 2.3 Karnaugh Maps: 2, 3, and 4 Variables

**Pehle socho:** Laws se simplify karne mein asli trick ek hi hai:  $XY + XY' = X$  (jo variable badal raha hai wo cancel). Problem: aise pairs dhoondna mushkil. K-map truth table ko aise ghar mein rakh deta hai ki padosi cells mein sirf ek variable badle. Ab pair dhoondna = padosi 1s pe gola banana. Maths ki jagah pattern dekhna.

**Definition:** A Karnaugh map (K-map) is a grid arrangement of a truth table in which adjacent cells (including wrap-around at the edges) differ in exactly one variable, so that adjacent 1s can be grouped in rectangles of 1, 2, 4, 8 or 16 cells to obtain a minimal SOP expression.

**Mushkil shabd, aasaan matlab:**

**cell**

Map ka ek dabba = truth table ki ek row. Andar F ki value (0, 1 ya X) likhte hain.

**Gray code**

Aisi counting jisme agla number sirf ek bit badalta hai:  $00 \rightarrow 01 \rightarrow 11 \rightarrow 10$ . Binary  $01 \rightarrow 10$  mein do bits badalte hain, isliye binary order K-map mein nahi chalta.

**adjacent (padosi)**

Upar-neeche ya left-right wale cells (diagonal nahi). Map ke edges mude hue maano: left edge right edge ka padosi, top bottom ka, aur 4-variable map ke chaaron corners ek doosre ke padosi.

**group / implicant**

Padosi 1s ka rectangle. Size: pair (2), quad (4), octet (8).

**wrap-around**

Map ko ek "donut" ki tarah socho: right se nikle to left mein aa gaye.

**Kaise kaam karta hai:**

**(A) Map draw karna, cell numbers ke saath** (cell ke andar row number = minterm index likha hai):

| A \ B | 0 | 1 |
|-------|---|---|
| 0     | 0 | 1 |
| 1     | 2 | 3 |

| A \ BC | 00 | 01 | 11 | 10 |
|--------|----|----|----|----|
| 0      | 0  | 1  | 3  | 2  |
| 1      | 4  | 5  | 7  | 6  |

| AB \ CD | 00 | 01 | 11 | 10 |
|---------|----|----|----|----|
| 00      | 0  | 1  | 3  | 2  |
| 01      | 4  | 5  | 7  | 6  |
| 11      | 12 | 13 | 15 | 14 |
| 10      | 8  | 9  | 11 | 10 |

Dhyan do: Gray order ki wajah se columns mein 3 pehle aata hai, 2 baad mein (0 1 3 2), aur rows mein 12 pehle, 8 baad mein. Cell number nikalna: row bits + column bits jodo, jaise row 11, column 01  $\rightarrow 1101 = 8 + 4 + 0 + 1 = 13$ .

**(B) Grouping rules:**

1. Sirf 1s (aur zaroorat ho to X) ko gher lo; 0 kabhi group mein nahi.
2. Group ka size **2 ki power**: 1, 2, 4, 8, 16. 3 ya 6 cells ka group illegal.
3. Group **rectangle** hona chahiye; diagonal nahi.
4. Wrap-around allowed: left-right edge, top-bottom edge, aur chaaron corners (4-variable) ek group.
5. Groups **jitne bade ho sakein** utne bade banao (bada group = kam literals).
6. Groups overlap kar sakte hain; har 1 kam se kam ek group mein ho.
7. Groups **jitne kam ho sakein**; jo group poora doosron se already cover ho, use hata do.

### (C) Group ko term mein padhna:

1. Group ke saare cells ke liye har variable ki value dekho.
2. Jo variable group ke andar **badalta hai** (kahin 0 kahin 1) → wo drop ( $X + X' = 1$ ).
3. Jo **constant** rehta hai: 1 hai to X likho, 0 hai to X'.
4. Bache variables ka AND = us group ka term. Saare terms ka OR = answer.

**(D) Poora procedure:** map bharo → pehle wo 1s dekho jinka group sirf ek tarah ban sakta hai → sabse bade groups gher lo → har 1 cover karo → har group ka term likho → OR karo.

|   |   | BC      |         |         |         |
|---|---|---------|---------|---------|---------|
|   |   | 00      | 01      | 11      | 10      |
| A | 0 | m0<br>0 | m1<br>1 | m3<br>1 | m2<br>0 |
|   | 1 | m4<br>0 | m5<br>1 | m7<br>1 | m6<br>0 |

  $F = C$  (quad)

3-variable map:  $F = \sum m(1,3,5,7) = C$

How to draw: A on rows (0,1), BC on columns in Gray order; the two middle columns form a quad.

**Diagram kaise padhein:** Row = A (0, 1), columns = BC Gray order mein (00, 01, 11, 10). 1s beech ke do columns (01 aur 11) mein hain, dono rows mein = quad. Quad mein A badalta hai (0, 1), B badalta hai (0, 1), C hamesha 1 →  $F = C$ .

|    |    |          |          |          |          |
|----|----|----------|----------|----------|----------|
|    |    | CD       |          |          |          |
|    |    | 00       | 01       | 11       | 10       |
| AB | 00 | m0<br>1  | m1<br>0  | m3<br>0  | m2<br>1  |
|    | 01 | m4<br>1  | m5<br>1  | m7<br>1  | m6<br>1  |
|    | 11 | m12<br>0 | m13<br>1 | m15<br>1 | m14<br>0 |
|    | 10 | m8<br>0  | m9<br>0  | m11<br>0 | m10<br>0 |

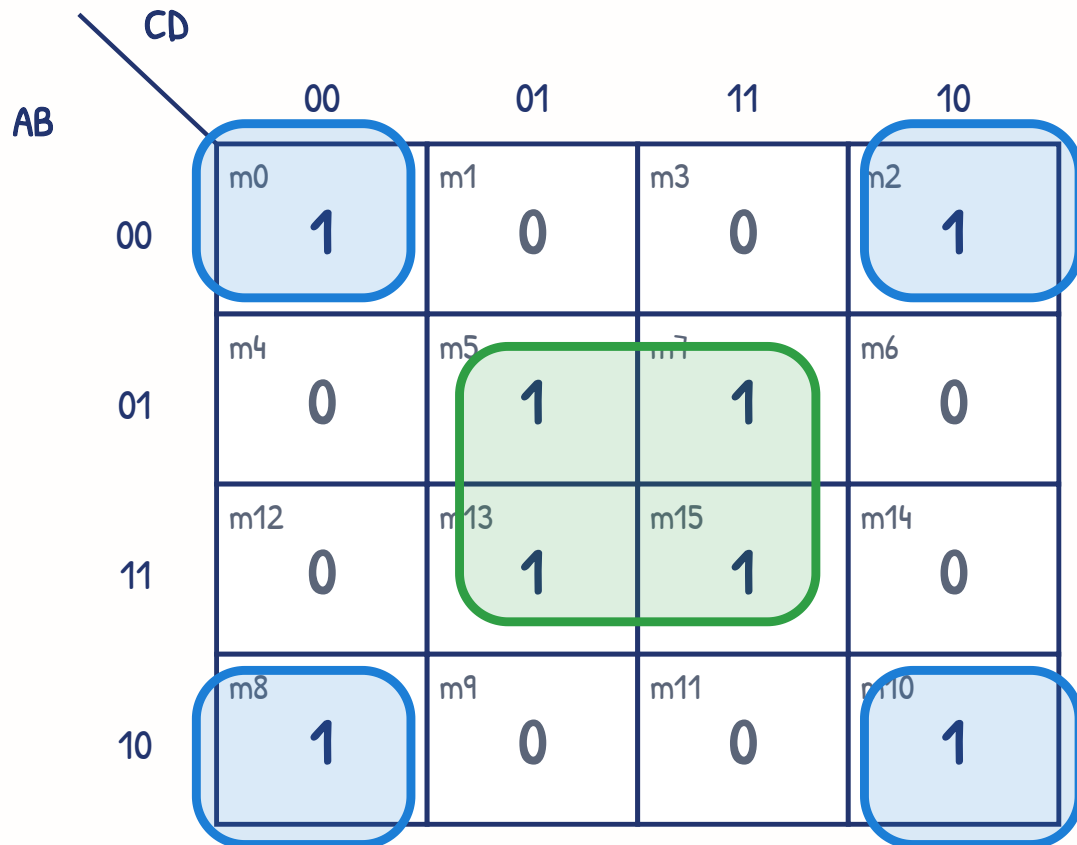
   $A'B$   
   $A'D'$  (wraps)

   $BD$

4-variable map:  $F = A'B + BD + A'D'$  (class example)

How to draw: AB rows, CD columns, both Gray-coded; draw the row quad, the centre quad, then the edge-wrapping quad.

**Diagram kaise padhein:** Rows AB, columns CD, dono Gray order. Row 01 पूरी 1 → quad  $A'B$  ( $A = 0, B = 1$  constant, C, D badalte hain). Beech ka  $2 \times 2$  block (cells 5, 7, 13, 15) →  $B = 1, D = 1$  →  $BD$ . Column 00 aur 10 ke upar ke do rows (cells 0, 2, 4, 6) edge se wrap hote hain →  $A = 0, D = 0$  →  $A'D'$ .  $F = A'B + BD + A'D'$ .



$B'D'$  (four corners)

$BD$

Wrap-around:  $\Sigma m(0, 2, 5, 7, 8, 10, 13, 15) = B'D' + BD$

How to draw: Draw four quarter-loops at the corners and label them as one group.

**Diagram kaise padhein:** Chaaron corners (0, 2, 8, 10) ek hi group hain kyunki map mudta hai. Corners mein  $B = 0$  aur  $D = 0$  hamesha  $\rightarrow B'D'$ . Beech ka quad (5, 7, 13, 15)  $\rightarrow BD$ .  $F = B'D' + BD$ .

**Formula:**  $n$ -variable map mein  $2^k$  cells ka group  $\rightarrow n - k$  literals.

4-variable: pair ( $k = 1$ )  $\rightarrow 3$  literals, quad ( $k = 2$ )  $\rightarrow 2$  literals, octet ( $k = 3$ )  $\rightarrow 1$  literal, all 16  $\rightarrow F = 1$ .

### Solved example 1 (3-variable, Lab 2 Q16):

Minimise  $F(A, B, C) = \Sigma m(0, 2, 3, 5)$  using a K-map.

1. Map bharo (cells 0, 2, 3, 5 mein 1):

| A \ BC | 00     | 01     | 11     | 10     |
|--------|--------|--------|--------|--------|
| 0      | 1 (m0) | 0      | 1 (m3) | 1 (m2) |
| 1      | 0      | 1 (m5) | 0      | 0      |

2. m5 ke padosi: m1 (0), m4 (0), m7 (0). Koi padosi 1 nahi  $\rightarrow$  m5 akela group:  $A = 1, B = 0, C = 1 \rightarrow AB'C$ .

3. m0 ke padosi 1: sirf m2 (wrap-around: column 00 aur column 10 padosi hain). Pair m0, m2:  $A = 0, C = 0$  constant, B badalta  $\rightarrow A'C'$ .
4. m3 ka padosi 1: m2. Pair m2, m3:  $A = 0, B = 1$  constant, C badalta  $\rightarrow A'B$ .
5. Saare 1s cover: m0, m2 ( $A'C'$ ), m3 ( $A'B$ ), m5 ( $AB'C$ ).

**Answer:**  $F = A'C' + A'B + AB'C$ .

### Solved example 2 (4-variable, exam level, mock 6 marks):

Minimise  $F(A, B, C, D) = \sum m(0, 1, 2, 5, 8, 9, 10)$  in SOP and POS.

1. Map bharo:

| AB \ CD | 00    | 01    | 11    | 10    |
|---------|-------|-------|-------|-------|
| 00      | 1(0)  | 1(1)  | 0(3)  | 1(2)  |
| 01      | 0(4)  | 1(5)  | 0(7)  | 0(6)  |
| 11      | 0(12) | 0(13) | 0(15) | 0(14) |
| 10      | 1(8)  | 1(9)  | 0(11) | 1(10) |

2. Four corners 0, 2, 8, 10 sab 1  $\rightarrow$  quad.  $B = 0, D = 0$  constant (A, C badalte)  $\rightarrow B'D'$ .
3. Cells 0, 1, 8, 9 (rows 00 aur 10 wrap, columns 00, 01)  $\rightarrow$  quad.  $B = 0, C = 0$  constant  $\rightarrow B'C'$ .
4. m5 bacha. Padosi: m1 (1), m4 (0), m7 (0), m13 (0). Sirf pair m1, m5  $\rightarrow A = 0, C = 0, D = 1$  constant, B badalta  $\rightarrow A'C'D$ .
5. SOP:  $F = B'D' + B'C' + A'C'D$ .
6. POS ke liye 0s ko group karo ( $F'$  ka SOP): 0s at 3, 4, 6, 7, 11, 12, 13, 14, 15.
7. Zero-group 1: row 11 poori (12, 13, 14, 15)  $\rightarrow A = 1, B = 1 \rightarrow AB$ .
8. Zero-group 2: column 11 poori (3, 7, 15, 11)  $\rightarrow C = 1, D = 1 \rightarrow CD$ .
9. Zero-group 3: rows 01, 11 aur columns 00, 10 (wrap) = cells 4, 6, 12, 14  $\rightarrow B = 1, D = 0 \rightarrow BD'$ .
10. Saare 9 zeros cover:  $F' = AB + CD + BD'$ .
11. De Morgan:  $F = (AB)' \cdot (CD)' \cdot (BD')' = (A' + B')(C' + D')(B' + D)$ .

**Answer:** SOP  $F = B'D' + B'C' + A'C'D$ ; POS  $F = (A' + B')(C' + D')(B' + D)$ .

**Galti mat karna:** Columns ko 00, 01, 10, 11 (binary) likhna sabse common galti hai. 3 cells ka group, diagonal group, aur corners/edges miss karna bhi. Ek group jo poora doosre groups se cover hai, wo extra term hai: hata do.

**Trick:** Gray order: "00, 01, 11, 10": beech mein 11 pehle. Group padhte waqt bolo: "jo badla wo gaya."

**Ek line mein:** Gray order map, 2 ki power ke rectangles (wrap allowed), sabse bade groups, badalne wala variable drop.

## 2.4 DeMorgan's Theorems and Bubble Pushing

**Pehle socho:** "Chai aur coffee dono mil gaye, aisa NAHI hua" = kam se kam ek nahi mila (chai nahi YA coffee nahi). "Na chai mili na coffee" = chai nahi AUR coffee nahi. Ye roz ki baat hi De Morgan hai. Circuit mein isi se NOT ko gates ke andar-bahar khiskaate hain, aur AND-OR circuit ko sirf NAND se bana dete hain.

**Definition: De Morgan's theorems** state that the complement of a product equals the sum of the complements,  $(AB)' = A' + B'$ , and the complement of a sum equals the product of the complements,  $(A + B)' = A'B'$ . **Bubble pushing** is the graphical use of these theorems: moving inversion bubbles from a gate's output to its inputs (or back) while changing AND to OR or OR to AND.

**Mushkil shabd, aasaan matlab:**

**bubble**

Gate ke input ya output pe chhota gola = NOT.

**complement of a function**

$F' = F$  ka ulta: jahan  $F = 1$  tha wahan 0. Nikalne ke liye: har literal ulta karo aur  $+ \leftrightarrow \cdot$  badlo ( $0 \leftrightarrow 1$  bhi).

**double bubble**

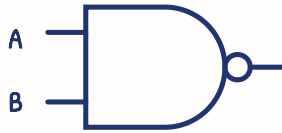
Ek wire pe do NOT = kuch nahi:  $(A')' = A$ . Do bubbles cancel.

**Kaise kaam karta hai:**

| A | B | $(AB)'$ | $A' + B'$ | $(A + B)'$ | $A'B'$ |
|---|---|---------|-----------|------------|--------|
| 0 | 0 | 1       | 1         | 1          | 1      |
| 0 | 1 | 1       | 1         | 0          | 0      |
| 1 | 0 | 1       | 1         | 0          | 0      |
| 1 | 1 | 0       | 0         | 0          | 0      |

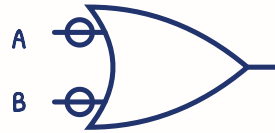
Columns 3 = 4 aur 5 = 6: dono theorems truth table se proved.

- Algebra mein:** sabse bahar wala bar pehle tod do: bar ke neeche ke har tukde pe bar lagao aur sign badlo ( $\cdot \rightarrow +$ ,  $+ \rightarrow \cdot$ ).
- Andar koi bar bacha ho to wahi step dobara. Double bar mile to hata do.
- Picture mein (bubble pushing):** gate ke output ka bubble uthao, gate ke har input pe bubble lagao, aur gate badlo ( $AND \leftrightarrow OR$ ).
- Isliye **NAND = OR with inverted inputs** aur **NOR = AND with inverted inputs**.
- Ek wire pe do bubbles aa jaayein (ek gate ke output pe, agle ke input pe) to dono cancel. Yahi AND-OR ko NAND-NAND banata hai (2.5).



NAND:  $(A \cdot B)'$

$\equiv$

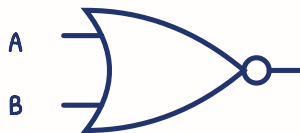


OR of inverted inputs:  $A' + B'$

Bubble pushing: NAND  $\equiv$  OR with inverted inputs

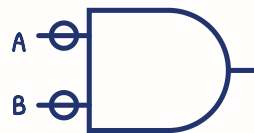
How to draw: Draw an AND with an output bubble, then an OR with bubbles on both inputs, joined by  $\equiv$ .

**Diagram kaise padhein:** Left: NAND (AND + output bubble). Right: OR jiske dono inputs pe bubble. Beech mein  $\equiv$  = dono same. Trace  $A = 1, B = 1$ : NAND  $\rightarrow 0$ ; right side:  $A' = 0, B' = 0$ , OR = 0. ✓



NOR:  $(A + B)'$

$\equiv$



AND of inverted inputs:  $A' \cdot B'$

NOR  $\equiv$  AND with inverted inputs

How to draw: OR with output bubble  $\equiv$  AND with input bubbles.

**Diagram kaise padhein:** Left: NOR (OR + output bubble). Right: AND with bubbled inputs. Trace  $A = 0, B = 0$ : NOR  $\rightarrow 1$ ; right:  $1 \cdot 1 = 1$ . ✓

### Solved example 1 (class slide):

Simplify  $\sim(A \cdot B + C)$ .

1. Bahar ka bar:  $(AB + C)' = (AB)' \cdot C'$  (OR tha  $\rightarrow$  AND, har tukde pe bar).
2. Andar bacha  $(AB)'$ :  $= A' + B'$ .
3. Result:  $(A' + B') \cdot C'$ .

**Answer:  $(\sim A + \sim B) \cdot \sim C$ .**

### Solved example 2 (exam level, Lab 2 Q30):

Simplify  $F = ((A' + B)' + (A \cdot C'))'$ .

1. Andar ke tukde:  $(A' + B)' = A'' \cdot B' = AB'$  (De Morgan, double bar hataya).
2.  $(AC)' = A' + C'$  (De Morgan).
3.  $F = (AB' + A' + C')'$ .
4. Bahar ka bar (teen terms ka OR  $\rightarrow$  teeno ke complement ka AND):  $F = (AB')' \cdot (A')' \cdot (C')'$ .

5.  $= (A' + B) \cdot A \cdot C$  (De Morgan on  $(AB)'$ , involution on the others).

6.  $= A \cdot C \cdot (A' + B) = AA'C + ABC = 0 + ABC$  (distributive, complement).

**Answer:**  $F = ABC$ .

### Solved example 3 (complement of a function, Lab 2 Q13):

Find the complement of  $F = (A + B)(A' + C)$ .

1.  $F' = [(A + B)(A' + C)]' = (A + B)' + (A' + C)'$  (product ka complement = complements ka sum).

2.  $(A + B)' = A'B'$ ;  $(A' + C)' = A \cdot C'$ .

**Answer:**  $F' = A'B' + AC'$ .

**Galti mat karna:**  $(AB)' = A'B'$  GALAT; sign badalna zaroori:  $A' + B'$ . Sirf bahar ka bar tod ke ruk mat jao, andar ke bars bhi tod do. Function ka complement lete waqt constants bhi badlo ( $0 \leftrightarrow 1$ ).

**Trick:** "Bar tod do, sign badlo." Picture mein: "bubble khiskao, gate badlo."

**Ek line mein:**  $(AB)' = A' + B'$ ,  $(A + B)' = A'B'$ ; NAND = bubbled-input OR, NOR = bubbled-input AND.

## 2.5 Universal Gates (NAND/NOR) Demonstration

**Pehle socho:** Ek LEGO set jisme sirf ek hi shape ki brick hai, phir bhi usse kuch bhi ban sakta hai. Factory ke liye ye best hai: ek hi cell design karo, perfect karo, lakhon baar copy karo. CMOS mein NAND sirf 4 transistors ka hai (AND 6 ka), isliye chips zyada NAND se bante hain.

**Definition:** A set of gates is **functionally complete** if every Boolean function can be built from it; {AND, OR, NOT} is such a set. A **universal gate** is a single gate type that is functionally complete by itself; NAND and NOR are universal gates.

**Mushkil shabd, aasaan matlab:**

### functionally complete

Is set se koi bhi truth table ban sakti hai. {AND, OR} complete nahi, kyunki NOT nahi ban sakta.

### NAND-NAND form

Do levels, dono NAND. AND-OR (SOP) circuit ka NAND version.

### NOR-NOR form

OR-AND (POS) circuit ka NOR version.

### inputs tie karna

Gate ke dono inputs ek hi signal se jodna.  $\text{NAND}(A, A) = (A \cdot A)' = A'$ .

## Kaise kaam karta hai (NAND se NOT, AND, OR):

1. **NOT:** NAND ke dono inputs pe A:  $(A \cdot A)' = A'$ . Check  $A = 1$ :  $(1 \cdot 1)' = 0$ . → **1 gate**.
2. **AND:**  $\text{NAND}(A, B) = (AB)'$ , phir ek NAND-inverter:  $((AB)')' = AB$ . → **2 gates**.
3. **OR:** pehle  $A' = \text{NAND}(A, A)$ ,  $B' = \text{NAND}(B, B)$ ; phir  $\text{NAND}(A', B') = (A'B')' = A + B$  (De Morgan). → **3 gates**.
4. **NOR se (mirror):** NOT = NOR(A, A) (1); OR = NOR phir NOR-inverter (2); AND = NOR( $A'$ ,  $B'$ ) =  $(A' + B')' = AB$  (3).
5. **XOR:** 4 NANDs (D2.5c), 5 NORs.

| Build | NAND only | NOR only |
|-------|-----------|----------|
| NOT   | 1         | 1        |
| AND   | 2         | 3        |
| OR    | 3         | 2        |
| XOR   | 4         | 5        |

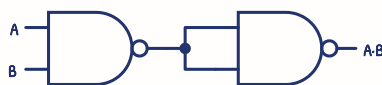
## SOP → NAND-NAND (steps):

1. Minimal SOP likho (K-map se). Circuit = level 1 ANDs, level 2 ek OR.
2. Har AND ke output pe ek bubble aur OR ke har input pe ek bubble lagao (wire pe 2 bubbles = koi farak nahi).
3. Ab AND + output bubble = NAND. OR + input bubbles = NAND (2.4). Sab NAND.
4. Agar koi akela literal (jaise C) seedha OR mein jaa raha tha, to ab use  $C'$  karke daalo (uska bubble cancel karne wala koi nahi).
5. Complemented inputs ( $A'$ ,  $B'$ ) available nahi to har ek ke liye  $\text{NAND}(X, X)$  inverter jodo.

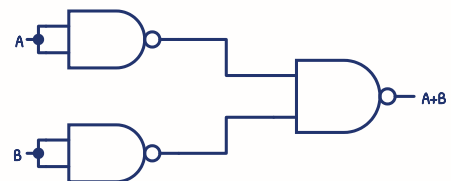
**POS → NOR-NOR:** same steps, OR ki jagah NOR (level 1), AND ki jagah NOR (level 2).



NOT:  $\text{NAND}(A, A)$



AND: 2 gates

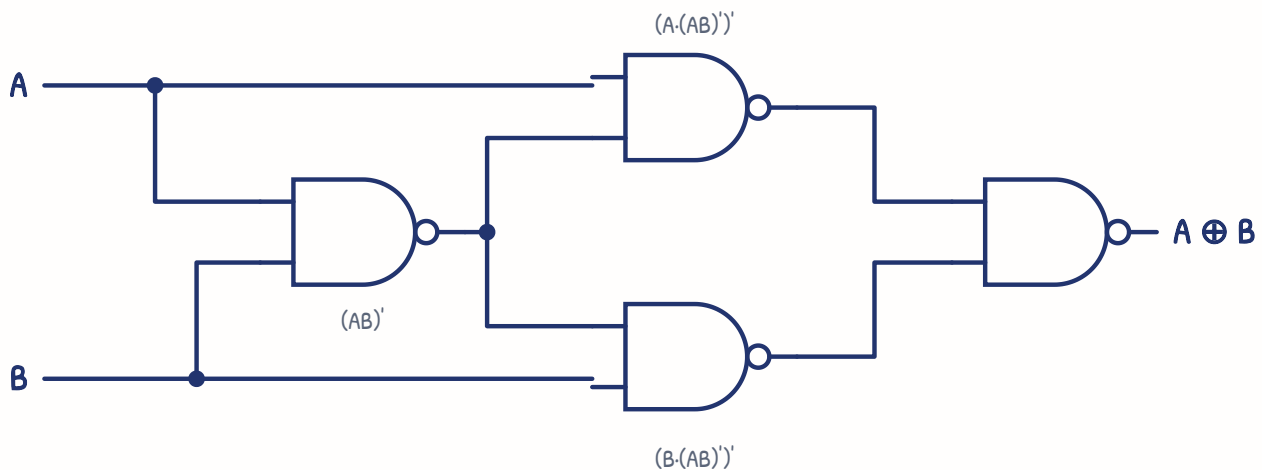


OR: 3 gates

### NOT, AND, OR from NAND gates

How to draw: Tie inputs together for NOT; NAND + NAND-inverter for AND; invert both inputs then NAND for OR.

**Diagram kaise padhein:** Left: ek NAND, dono inputs A se jude →  $A'$ . Beech:  $\text{NAND}(A, B)$  ka output agle NAND ke dono inputs pe →  $AB$  (2 gates). Right: A aur B pehle alag alag NAND-inverters se  $A'$ ,  $B'$  bante hain, phir teesra NAND →  $A + B$  (3 gates). Trace OR with  $A = 0$ ,  $B = 1$ :  $A' = 1$ ,  $B' = 0$ ,  $\text{NAND}(1, 0) = 1$  ✓.



### XOR from four NAND gates

How to draw: First NAND makes  $(AB)'$ ; two NANDs combine it with A and with B; a last NAND joins them.

**Diagram kaise padhein:** Pehla NAND  $G1 = (AB)'$ .  $G1$  do NANDs mein jaata hai:  $G2 = (A \cdot G1)'$ ,  $G3 = (B \cdot G1)'$ . Last NAND  $G4 = (G2 \cdot G3)' = A \oplus B$ . Trace  $A = 1, B = 0$ :  $G1 = 1$ ,  $G2 = (1 \cdot 1)' = 0$ ,  $G3 = (0 \cdot 1)' = 1$ ,  $G4 = (0 \cdot 1)' = 1 \checkmark (1 \oplus 0 = 1)$ .

### Solved example 1 (mock MCQ):

Minimum 2-input NAND gates for a 2-input OR? Verify with a truth table.

- OR =  $(A'B')'$  (De Morgan).  $A' = \text{NAND}(A, A)$ ,  $B' = \text{NAND}(B, B)$ , result =  $\text{NAND}(A', B')$ .
- Check all rows: 00  $\rightarrow A' = 1, B' = 1 \rightarrow \text{NAND} = 0$ . 01  $\rightarrow 1, 0 \rightarrow 1$ . 10  $\rightarrow 0, 1 \rightarrow 1$ . 11  $\rightarrow 0, 0 \rightarrow 1$ .  
Column 0111 = OR  $\checkmark$ .

**Answer: 3 NAND gates.**

### Solved example 2 (exam level, mock 6 marks, second half):

Implement  $F = B'D' + B'C' + A'C'D$  (from 2.3, example 2) using only NAND gates. Only true inputs A, B, C, D are available.

- SOP circuit:  $\text{AND1} = B'D'$ ,  $\text{AND2} = B'C'$ ,  $\text{AND3} = A'C'D$  (3-input), OR of three.
- Bubble push: har AND  $\rightarrow$  NAND, OR (bubbled inputs)  $\rightarrow$  NAND.  $F = ((B'D')' \cdot (B'C')' \cdot (A'C'D)')'$ .
- Complements banao:  $A' = \text{NAND}(A, A)$ ,  $B' = \text{NAND}(B, B)$ ,  $C' = \text{NAND}(C, C)$ ,  $D' = \text{NAND}(D, D) \rightarrow 4$  gates.
- Level 1:  $N1 = \text{NAND}(B', D')$ ,  $N2 = \text{NAND}(B', C')$ ,  $N3 = \text{NAND}(A', C', D)$  (3-input)  $\rightarrow 3$  gates.
- Level 2:  $F = \text{NAND}(N1, N2, N3)$  (3-input)  $\rightarrow 1$  gate.
- Check  $A = 0, B = 1, C = 0, D = 1$  (m5, should be 1):  $N1 = \text{NAND}(0, 0) = 1$ ,  $N2 = \text{NAND}(0, 1) = 1$ ,  $N3 = \text{NAND}(1, 1, 1) = 0$ ,  $F = \text{NAND}(1, 1, 0) = 1 \checkmark$ .

**Answer:**  $F = \text{NAND}(\text{NAND}(B', D'), \text{NAND}(B', C'), \text{NAND}(A', C', D'))$ : 4 inverter NANDs + 3 + 1 = 8 NAND gates.

### Solved example 3 (NOR only):

Implement  $F = (A + B)(C + D)$  using only 2-input NOR gates.

1. POS circuit: OR1 =  $A + B$ , OR2 =  $C + D$ , AND of both.
2. Bubble push: OR + output bubble = NOR; AND with bubbled inputs = NOR.
3.  $F = ((A + B)' + (C + D)')' = \text{NOR}(\text{NOR}(A, B), \text{NOR}(C, D))$ .
4. Check  $A = 1, B = 0, C = 0, D = 0$  ( $F$  should be  $(1)(0) = 0$ ):  $\text{NOR}(1, 0) = 0, \text{NOR}(0, 0) = 1, \text{NOR}(0, 1) = 0 \checkmark$ .

Answer: 3 NOR gates. (Similarly  $AB + CD = \text{NAND}(\text{NAND}(A, B), \text{NAND}(C, D)) = 3 \text{ NANDs.}$ )

**Galti mat karna:** NAND "zyada powerful" nahi, sirf **self-sufficient** hai. NAND-NAND mein akela literal (jaise  $AB + C$  mein  $C$ ) seedha mat daalo:  $C$  daalna padta hai,  $F = ((AB)' \cdot C)'$ . Aur sirf 2-input gates allowed hon to 3-input term ke liye extra gates lagenge.

**Trick:** NAND: **1-2-3** (NOT-AND-OR). NOR: **1-3-2**.  $\text{SOP} \leftrightarrow \text{NAND-NAND}$ ,  $\text{POS} \leftrightarrow \text{NOR-NOR}$  ("S-N-A, P-N-O").

**Ek line mein:** NAND/NOR akele har circuit bana sakte hain; SOP ko NAND-NAND, POS ko NOR-NOR bubble pushing se.

## 2.6 Don't-Cares in K-Maps

**Pehle socho:** Menu mein ek dish hai jo koi kabhi order hi nahi karta. Chef us dish ke liye jo chahe maan le, kisi ko farak nahi padta. Circuit mein bhi kuch input combinations **kabhi aate hi nahi** (jaise BCD mein 1010 se 1111). Unka output hum apni suvidha se 0 ya 1 maan sakte hain, taaki groups bade ho jaayein.

**Definition:** A **don't-care** (written X or d) is an input combination whose output does not matter, because it never occurs or its output is never used. In a K-map it may be treated as 1 or 0, whichever gives a simpler expression.

**Mushkil shabd, aasaan matlab:**

**X / d ( )**

Don't-care ka nishan.  $F = \sum m(5, 6, 7) + d(10, 11)$  matlab 5, 6, 7 pe  $F = 1$  aur 10, 11 pe "kuch bhi".

**BCD**

Binary Coded Decimal: ek decimal digit (0 to 9) ko 4 bits mein likhna. 4 bits 16 values bana sakti hain, par sirf 0000 to 1001 use hote hain; 1010 to 1111 (10 to 15) kabhi nahi aate  $\rightarrow$  don't-cares.

**Kaise kaam karta hai:**

1. Map mein 1s, 0s aur X sab bharo.
2. Har X ke liye socho: kya isse 1 maan ke koi group bada hota hai? Haan  $\rightarrow$  1 maano. Nahi  $\rightarrow$  0 maano.
3. **Sirf X ka group kabhi mat banao** (wo koi 1 cover nahi karta, bekar term).

4. X ko cover karna zaroori nahi; sirf saare 1s cover hone chahiye.
5. Groups ko normal tareeke se padho (badalne wala variable drop).

CD

| AB \ CD | 00       | 01       | 11       | 10       |
|---------|----------|----------|----------|----------|
| 00      | m0<br>1  | m1<br>1  | m3<br>X  | m2<br>0  |
| 01      | m4<br>0  | m5<br>X  | m7<br>X  | m6<br>0  |
| 11      | m12<br>0 | m13<br>0 | m15<br>0 | m14<br>0 |
| 10      | m8<br>0  | m9<br>0  | m11<br>0 | m10<br>0 |

 A'D (X's used as 1)

 A'B'C'

Don't-cares enlarge a group (L02 p16):  $F = A'D + A'B'C'$

How to draw: Fill the 1s and Xs, draw the quad through the Xs, then cover the remaining 1.

**Diagram kaise padhein:** Row 00 mein m0, m1 = 1, m3 = X; row 01 mein m5, m7 = X. Cells 1, 3, 5, 7 ka quad (X ko 1 maana)  $\rightarrow A = 0, D = 1 \rightarrow A'D$ . m0 bacha  $\rightarrow$  pair m0, m1  $\rightarrow A'B'C'$ .  $F = A'D + A'B'C'$ . X ki wajah se ek pair ki jagah quad mila.

### Solved example 1 (3-variable):

Minimise  $F(A, B, C) = \sum m(0, 1, 6) + d(2, 3, 7)$ .

| A \ BC | 00 | 01 | 11 | 10 |
|--------|----|----|----|----|
| 0      | 1  | 1  | X  | X  |
| 1      | 0  | 0  | X  | 1  |

1. Row A = 0: m0, m1 = 1, m3, m2 = X. Chaaron ko 1 maano  $\rightarrow$  quad (0, 1, 3, 2), sirf A = 0 constant  $\rightarrow A'$ .
2. m6 bacha. Cells 2, 3, 6, 7 (columns 11, 10 dono rows) = X, X, 1, X  $\rightarrow$  quad, B = 1 constant  $\rightarrow B$ .

3. Saare 1s (0, 1, 6) cover.

Answer:  $F = A' + B$  (2 literals). Without don't-cares it would be  $A'B' + ABC'$ .

### Solved example 2 (4-variable, exam level):

A circuit gets a BCD digit ABCD and outputs 1 when the digit is 5 or more.  $F = \sum m(5, 6, 7, 8, 9) + d(10, 11, 12, 13, 14, 15)$ . Minimise.

| AB \ CD | 00    | 01    | 11    | 10    |
|---------|-------|-------|-------|-------|
| 00      | 0     | 0     | 0     | 0     |
| 01      | 0     | 1(5)  | 1(7)  | 1(6)  |
| 11      | X(12) | X(13) | X(15) | X(14) |
| 10      | 1(8)  | 1(9)  | X(11) | X(10) |

- m8, m9: rows 11 aur 10 (8 cells: 8, 9, 10, 11, 12, 13, 14, 15) mein baaki sab X  $\rightarrow$  octet,  $A = 1$  constant  $\rightarrow A$ .
- m5, m7: cells 5, 7, 13, 15 (rows 01, 11; columns 01, 11)  $\rightarrow$  quad,  $B = 1, D = 1 \rightarrow BD$ .
- m6: cells 6, 7, 14, 15 (rows 01, 11; columns 11, 10)  $\rightarrow$  quad,  $B = 1, C = 1 \rightarrow BC$ .
- Check: 5 (BD), 6 (BC), 7 (BD), 8 (A), 9 (A) sab cover; 0 to 4 koi group mein nahi  $\checkmark$ .

Answer:  $F = A + BD + BC$  (5 literals). Without don't-cares:  $AB'C' + A'BD + A'BC$  (9 literals).

**Galti mat karna:** X ko cover karne ke liye alag group banana = bekar term. Aur X ko "zaroor 1" mat samjho; jo X kisi group ko bada nahi karta use 0 hi rehne do.

**Ek line mein:** X = "help kare to 1, warna 0"; sirf X ka group kabhi nahi.

## 2.E1 Extra from slides: Prime Implicants, EPIs and the Consensus Theorem

**Pehle socho:** K-map pe gole banate waqt sawaal aata hai: "kaunse group **zaroori** hain aur kaunse optional?" Jaise ek class mein kuch students hi kisi ek topic ko padha sakte hain: jo topic sirf ek student jaanta hai, us student ko team mein lena hi padega. Wahi "essential" hai.

**Definition:** An **implicant** is any product term (group of 1s and don't-cares) for which  $F = 1$ . A **prime implicant (PI)** is an implicant that cannot be enlarged (combined into a bigger group). An **essential prime implicant (EPI)** is a prime implicant that covers at least one 1 that no other prime implicant covers. A minimal SOP contains all EPIs plus the fewest additional PIs needed to cover the remaining 1s.

### Mushkil shabd, aasaan matlab:

#### implicant

Koi bhi valid group (1, 2, 4 cells). Jaise  $m_0, m_2$  ka pair.

#### prime implicant

Group jo aur bada nahi ho sakta. Agar  $m_0, m_2$  ka pair ek quad (0, 2, 8, 10) ka hissa hai to pair prime nahi; quad prime hai.

#### essential PI

Prime group jo kisi ek 1 ko **akela** cover karta hai. Us 1 ke liye koi aur option nahi, to ye answer mein hona hi chahiye.

#### cyclic map

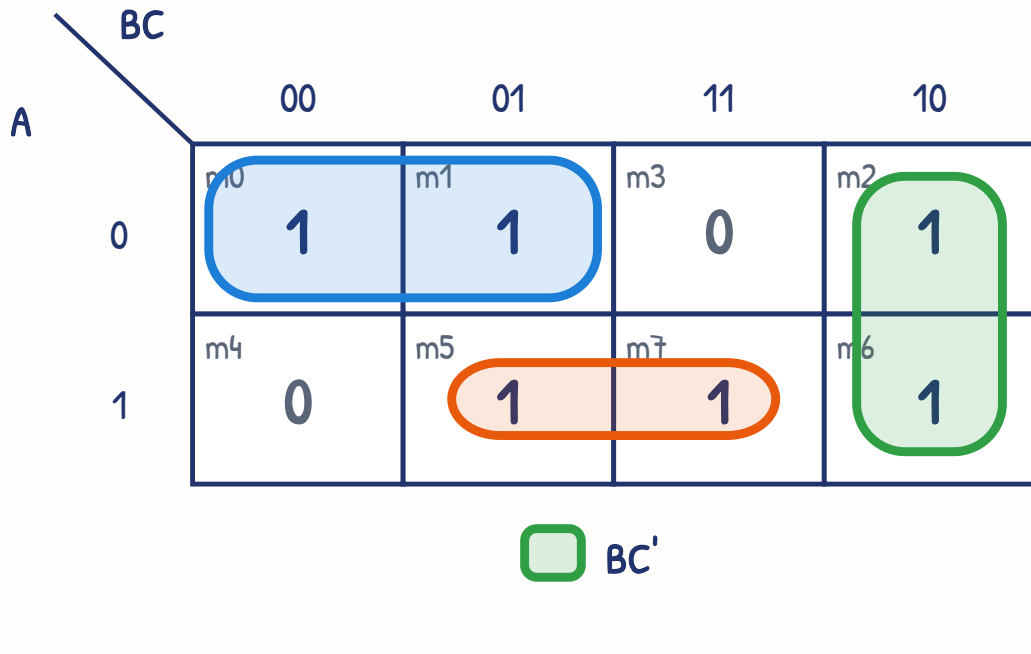
Map jisme har 1 do PIs se cover hota hai, koi EPI nahi. Choice trial se karte hain.

#### consensus term

$XY$  aur  $X'Z$  se bana  $YZ$  ( $X$  hata ke baaki ka AND). Ye redundant hota hai.

### Kaise kaam karta hai (PIs aur EPIs dhoondna):

1. Map bharo. Har 1 ke liye uska **sabse bada possible group** dhoondo. Ye saare alag alag groups = PIs.
2. Har 1 ke liye gino: kitne PIs isko cover karte hain?
3. Jis 1 ko sirf ek PI cover karta hai, wo PI = EPI. Use answer mein daalo.
4. EPIs se jo 1s bach gaye, unhe kam se kam extra PIs se cover karo.
5. Koi EPI na mile (cyclic) → ek PI choose karo, baaki chain ki tarah follow karo.



Cyclic map  $\Sigma m(0,1,2,5,6,7)$ : one minimal cover  $A'B' + BC' + AC$

How to draw: Six 1s in a ring; every 1 has two pair options, so pick three pairs that alternate.

**Diagram kaise padhein:** 6 ones ek ring mein: m0, m1, m5, m7, m6, m2. Har 1 ke do padosi 1 hain, isliye har 1 do pairs mein aa sakta hai. Teen alternate pairs chuno:  $A'B'$  (0, 1),  $AC$  (5, 7),  $BC'$  (2, 6). Saare 6 cover, 3 terms.

### Solved example 1 (EPIs present):

Find all PIs and EPIs of  $F = \Sigma m(0, 1, 2, 5, 8, 9, 10)$  (map of 2.3, example 2).

1. PIs:  $B'D'$  (0, 2, 8, 10),  $B'C'$  (0, 1, 8, 9),  $A'C'D$  (1, 5). Koi aur group inse bada ya alag nahi banta.
2. m5 ko kaun cover karta hai? Sirf  $A'C'D \rightarrow$  EPI.
3. m9: sirf  $B'C'$  (m13, m11 = 0, isliye koi aur group nahi)  $\rightarrow$  EPI.
4. m2 aur m10: sirf  $B'D' \rightarrow$  EPI.
5. Teeno EPIs saare 1s cover karte hain, extra PI nahi chahiye.

**Answer:** 3 PIs, all 3 essential;  $F = B'D' + B'C' + A'C'D$ .

### Solved example 2 (cyclic, Lab 2 Q23):

$F(A, B, C) = \Sigma m(0, 1, 2, 5, 6, 7)$ . How many PIs and EPIs? Give a minimal SOP.

1. Har 1 ke pairs: m0: (0, 1)  $A'B'$ , (0, 2)  $A'C'$ . m1: (1, 0), (1, 5)  $B'C$ . m5: (5, 1), (5, 7)  $AC$ . m7: (7, 5), (7, 6)  $AB$ . m6: (6, 7), (6, 2)  $BC'$ . m2: (2, 0), (2, 6).
2. Koi quad nahi banta (har quad mein koi 0 aata hai). To PIs = 6 pairs:  $A'B'$ ,  $A'C'$ ,  $B'C$ ,  $AC$ ,  $AB$ ,  $BC'$ .
3. Har 1 do PIs se cover hai  $\rightarrow$  koi EPI nahi (0).

4.  $A'B'$  chuno (0, 1). Bache 2, 5, 6, 7:  $AC(5, 7) + BC'(2, 6)$  le lo  $\rightarrow$  sab cover.

**Answer:** 6 PIs, 0 EPIs;  $F = A'B' + AC + BC'$  (other valid cover:  $A'C' + B'C + AB$ ).

**Consensus theorem:**  $XY + X'Z + YZ = XY + X'Z$

Why:  $YZ = 1$  means  $Y = Z = 1$ ; then  $X = 1$  makes  $XY = 1$ ,  $X = 0$  makes  $X'Z = 1$ , so  $YZ$  never adds a new 1.

Example (Lab 2 Q11):  $(A + B)(A' + C) = AA' + AC + A'B + BC = AC + A'B + BC = AC + A'B$  ( $BC$  is the consensus of  $AC$  and  $A'B$ ).

**Galti mat karna:** "PIs ki ginti = EPIs ki ginti" hamesha nahi (cyclic map mein 6 PIs, 0 EPI). EPI kabhi drop nahi hota. Don't-care PI ke andar ho sakta hai, par sirf don't-care cover karne se koi PI essential nahi banta.

**Ek line mein:** PI = sabse bada group; EPI = kisi 1 ka akela sahara, hamesha answer mein; consensus term  $YZ$  redundant.

**Exam mein is unit se kya aata hai  $\rightarrow$  kaise solve karein**

| Question type                                     | Method (steps)                                                                                                                                                                        |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Write minterm / maxterm for a row                 | 1. Bits ka decimal = index. 2. Minterm: $1 \rightarrow X, 0 \rightarrow X'$ . 3. Maxterm: $0 \rightarrow X, 1 \rightarrow X'$ . 4. Row daal ke check (minterm = 1, maxterm = 0).      |
| $\Sigma m \leftrightarrow \Pi M$ , count maxterms | 1. Missing numbers likho (0 to $2^n - 1$ ). 2. Wahi doosri list. 3. Count = $2^n$ - given count.                                                                                      |
| Count minterms of an expression ( $A + B'C$ )     | 1. Har term ki rows likho (missing variable = dono values). 2. Union lo, duplicate ek baar. 3. Gino.                                                                                  |
| K-map minimise (3 or 4 variables) SOP             | 1. Gray order map, cell numbers. 2. 1s bharo. 3. Akele option wale 1s pehle. 4. Sabse bade $2^k$ rectangles, wrap/corners. 5. Badalne wala variable drop. 6. OR karo.                 |
| K-map POS                                         | 1. 0s group karo $\rightarrow F'$ SOP. 2. De Morgan $\rightarrow F =$ product of sums.                                                                                                |
| K-map with don't-cares                            | 1. X bharo. 2. X ko 1 tabhi maano jab group bada ho. 3. Sirf X ka group nahi. 4. Normal padho.                                                                                        |
| De Morgan / complement of a function              | 1. Bahar ka bar tod do, sign badlo. 2. Andar ke bars repeat. 3. Double bar hatao. 4. Laws se simplify.                                                                                |
| NAND/NOR gate count (NOT, AND, OR, XOR)           | NAND 1-2-3, XOR 4; NOR 1-3-2, XOR 5. $(A + B)(C + D)$ with 2-input NANDs = 6.                                                                                                         |
| Implement with NAND only / NOR only               | 1. Minimal SOP (NAND) ya POS (NOR). 2. Bubble push $\rightarrow$ NAND-NAND / NOR-NOR. 3. Akela literal ulta karke. 4. Complements ke liye NAND(X, X). 5. Gates gino, ek row se check. |
| PI / EPI count, cyclic map                        | 1. Har 1 ka sabse bada group = PIs. 2. Jo 1 sirf ek PI mein $\rightarrow$ woh EPI. 3. Koi EPI nahi $\rightarrow$ cyclic, trial se cover.                                              |

Why minimise (theory)

Area, power, delay, cost (1 line each) + example  $AB + AB' = A$ .

## Unit 3 · Combinational Building Blocks

**Is unit ki kahani:** Unit 1–2 mein humne gates aur unhe chhota karna seekha. Par CPU ko "17 gates aise jodo" soch ke koi design nahi karta. Hum gates se kuch **standard blocks** banate hain: MUX (choose karna), DEMUX aur decoder (bhejna / line chunna), adder (jodna). Phir in blocks se ALU aur register file bante hain, jo Unit 13 ke CPU datapath ka dil hain. Unit 4 isi adder se subtraction aur two's complement karega.

### 3.1 Motivation: From Gates to Reusable Blocks

**Pehle socho:** Car factory har screw khud nahi dhaalti; catalogue se standard parts mangwaati hai jo pehle se tested hain. Chip designers bhi ek chhota catalogue use karte hain: **MUX, decoder, adder, register** (lagbhag 5 core types), jo lakhon baar repeat hote hain.

**Definition:** A **combinational building block** is a standard circuit (multiplexer, demultiplexer, decoder, adder) built from logic gates whose outputs depend only on its present inputs, and which is designed and verified once and reused throughout larger designs.

**Mushkil shabd, aasaan matlab:**

#### **combinational**

Output sirf abhi ke inputs pe depend karta hai; koi memory nahi. Input badla, output turant badla. (Ultr: sequential, jo yaad rakhta hai, Unit 5.)

#### **datapath**

CPU ke andar ka raasta jisme data chalta hai: register file → ALU → memory → wapas.

#### **control point**

Datapath ka ek MUX jiska select control unit set karta hai, jaise ALUSrc (ALU ko register de ya immediate).

#### **verification**

Check karna ki block sahi kaam karta hai. Ek baar test kiya, har copy pe bharosa.

**Kaise kaam karta hai (blocks kyun aur kahan):**

1. **Abstraction:** "is input ko select karo" socho, "17 gates" nahi.
2. **Reuse:** ek verified MUX har jagah lagao.
3. **Verification:** block ek baar test, sab copies trusted.
4. **Scale:** 2:1 MUX → 4:1 → 8:1 jodte jaao.
5. CPU mein (Lab 3): ALU operand select (register ya immediate), PC update (PC + 4 ya branch target ya jump target), memory routing (ek bus, kai destinations).

### Solved example (write the answer like this):

Explain, with two datapath examples, why multiplexers are called the fundamental routing block of a CPU. (3 marks)

1. A MUX selects one of several sources and passes it to one output under a control (select) signal.
2. ALUSrc MUX: chooses Read data 2 (register) or the sign-extended immediate as the ALU's second operand.
3. PCSrc MUX: chooses PC + 4 or the branch target as the next PC. (MemtoReg MUX: ALU result or memory data for write-back.)

**Answer: definition + any two datapath MUXes.**

**Galti mat karna:** MUX kuch store nahi karta; wo railway switch hai, godown nahi. Storage register ka kaam hai.

**Ek line mein:** CPU = gates nahi, blocks (MUX, decoder, adder, register) ka jod; har block ek baar bana, baar baar use.

## 3.2 Multiplexers: 2:1, 4:1, and Cascading

**Pehle socho:** Railway junction pe kai patriyaan ek patri mein milti hain. Station master ka **lever** decide karta hai kaunsi train aage jaayegi. Lever badlo, doosri train nikal jaati hai; kuch ruka ya store nahi hota. MUX yahi hai: kai data inputs, ek output, aur **select lines** = lever.

**Definition:** A **multiplexer (MUX)** is a combinational circuit that selects one of  $2^n$  data inputs and passes it to a single output, according to the binary value on  $n$  select lines (a "data selector").

**Mushkil shabd, aasaan matlab:**

**data inputs ( $I_0, I_1, I_2$ )**

Wo signals jinme se ek chuna jaayega (patriyaan).

**select lines ( $S$ )**

Control bits jo batate hain kaunsa input. Select ka binary value = chune gaye input ka number.  $S_1S_0 = 10 = 2 \rightarrow I_2$ .

**why  $2^n$  inputs**

$n$  select bits se  $2^n$  alag numbers ban sakte hain (1 bit: 0, 1  $\rightarrow$  2; 2 bits: 00 to 11  $\rightarrow$  4; 3 bits  $\rightarrow$  8). Har number ek input ko point karta hai. Isliye 8:1 MUX ko 3 select lines.

**cascading / tree**

Chhote MUX jod ke bada MUX banana.

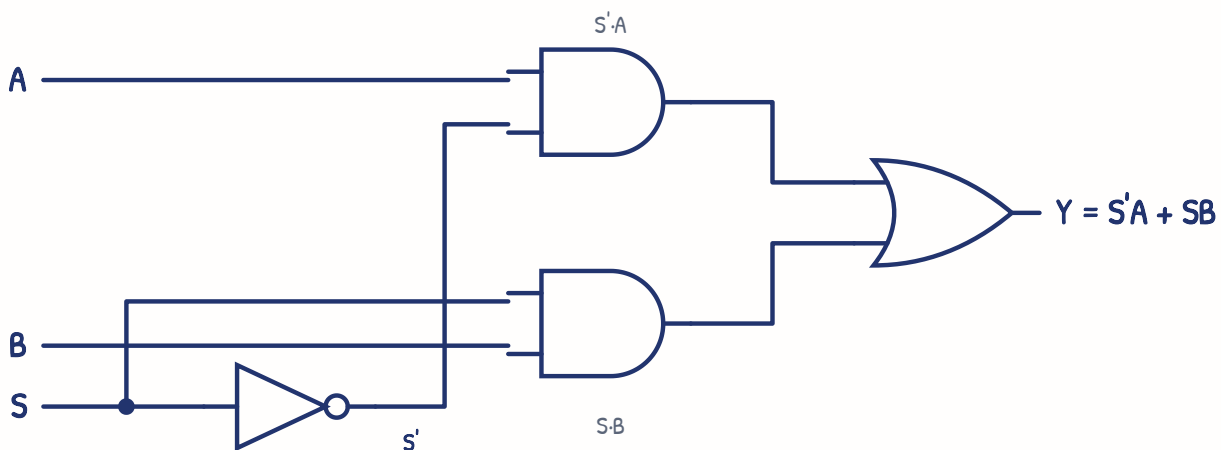
## LUT (look-up table)

MUX jiske data inputs pe truth table ke 0/1 lage hain; select pe variables. FPGA inse bhara hota hai.

### Kaise kaam karta hai:

1. **2:1 MUX** (inputs I0, I1; select S):  $S = 0 \rightarrow Y = I_0$ ;  $S = 1 \rightarrow Y = I_1$ .
2. Equation banao: "S = 0 aur I0" ya "S = 1 aur I1"  $\rightarrow Y = S' \cdot I_0 + S \cdot I_1$ . Gates: 1 NOT, 2 AND, 1 OR.
3. Check:  $I_0 = 1, I_1 = 0, S = 1 \rightarrow Y = 0 \cdot 1 + 1 \cdot 0 = 0$  (S = 1 ne I1 = 0 chuna) ✓.
4. **4:1 MUX** (I0 to I3, selects S1 S0): har input ke saath uske select number ka minterm AND karo:  $Y = S_1' S_0' \cdot I_0 + S_1' S_0 \cdot I_1 + S_1 S_0' \cdot I_2 + S_1 S_0 \cdot I_3$ .
5. Kisi bhi time sirf ek minterm 1 hota hai, isliye sirf ek input output tak pahunchta hai.
6. **Cascading**: 4:1 = teen 2:1 (pehle level pe S0, doosre pe S1). 8:1 = do 4:1 + ek 2:1 (S2 last). General:  $2^n:1$  tree =  $2^n - 1$  two-input MUXes.
7. **MUX se koi bhi function**: variables select pe lagao, aur har data input pe truth table ka us row ka output (0 ya 1) lagao. Ek variable kam select pe ho to data inputs pe 0, 1, C ya C' lagte hain.

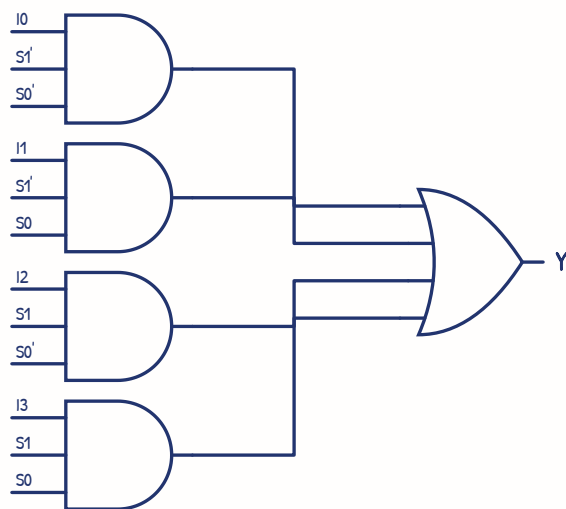
| S1 | S0 | Y  |
|----|----|----|
| 0  | 0  | I0 |
| 0  | 1  | I1 |
| 1  | 0  | I2 |
| 1  | 1  | I3 |



### 2:1 MUX from gates

How to draw: NOT on S, two ANDs (A with S', B with S), one OR.

**Diagram kaise padhein:** S ek NOT se S' banata hai. Upar ka AND: A aur S'; neeche ka AND: B aur S. Dono AND ek OR mein. Trace S = 0: upar ka AND =  $A \cdot 1 = A$ , neeche ka =  $B \cdot 0 = 0 \rightarrow Y = A$ . S = 1 pe ulta  $\rightarrow Y = B$ .



Inverters for  $S1'$ ,  $S0'$  are shared by all four AND gates (not drawn).

$$Y = S1'S0' \cdot I0 + S1'S0 \cdot I1 + S1S0' \cdot I2 + S1S0 \cdot I3$$

$$S1 \ S0 \rightarrow Y$$

$$0 \ 0 \rightarrow I0$$

$$0 \ 1 \rightarrow I1$$

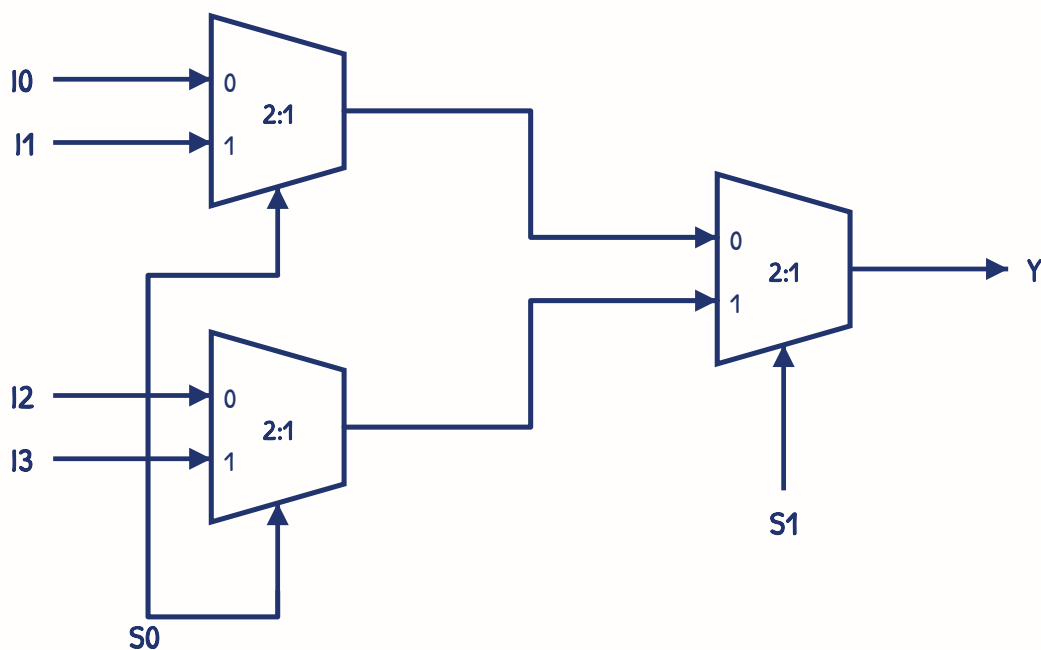
$$1 \ 0 \rightarrow I2$$

$$1 \ 1 \rightarrow I3$$

#### 4:1 MUX gate level with select table

How to draw: Four 3-input ANDs (one per input with its select minterm) into a 4-input OR.

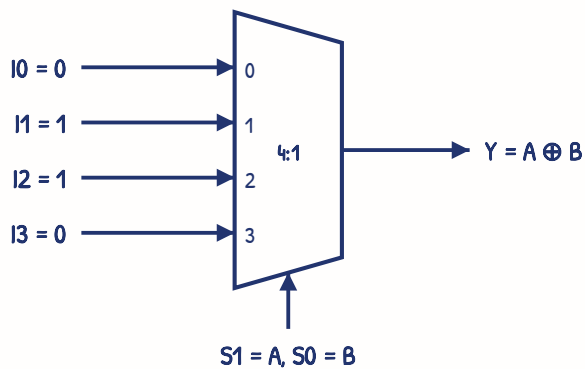
**Diagram kaise padhein:** Chaar 3-input AND gates, har ek pe ek data input aur do select literals. Right mein select table. Trace  $S1S0 = 10$ : sirf teesra AND ( $I2, S1, S0'$ ) ke dono select inputs 1 hain, to  $Y = I2$ .



#### 4:1 MUX from three 2:1 MUXes

How to draw: Two MUXes on  $S0$  feeding a third on  $S1$ .

**Diagram kaise padhein:** Left mein do 2:1 MUX, dono ka select  $S0$ : upar wala  $I0/I1$  mein se, neeche wala  $I2/I3$  mein se chunta hai. Right ka MUX  $S1$  se un dono mein se ek chunta hai. Trace  $S1S0 = 11$ : upar  $\rightarrow I1$ , neeche  $\rightarrow I3$ , right ( $S1 = 1$ )  $\rightarrow I3$  ✓.



Rule: feed the variables to the select lines, wire each data input to the truth-table output (FPGA LUT).

### XOR implemented with a 4:1 MUX (LUT idea)

How to draw: Selects = A, B; data inputs = the XOR output column 0, 1, 1, 0.

**Diagram kaise padhein:** Select pe A ( $S_1$ ) aur B ( $S_0$ ). Data inputs = XOR ka output column 0, 1, 1, 0.  $AB = 10 \rightarrow I_2 = 1 \rightarrow Y = 1 = 1 \oplus 0 \checkmark$ . Ye LUT idea hai.

**Formula:**  $2^n:1$  MUX  $\rightarrow n$  select lines;  $2:1$ :  $Y = S'_1 I_0 + S_1 I_1$ ;  $4:1$ :  $Y = S'_1 S'_0 I_0 + S'_1 S_0 I_1 + S_1 S'_0 I_2 + S_1 S_0 I_3$

$2^n:1$  from  $2:1$  MUXes:  $2^n - 1$ .

Verilog: assign  $y = sel ? b : a$ ; ( $2:1$  in one line).

### Solved example 1 (class quiz):

(a)  $2:1$  MUX with  $A (I_0) = 1$ ,  $B (I_1) = 0$ ,  $S = 1$ :  $Y$ ? (b)  $4:1$  MUX with  $S_1 = 1$ ,  $S_0 = 0$ : which input? (c) Select lines of a  $32:1$  MUX? (d)  $2:1$  MUXes for a  $16:1$  MUX?

- (a)  $S = 1 \rightarrow B$  chuna  $\rightarrow Y = 0$ . (Equation:  $0 \cdot 1 + 1 \cdot 0 = 0$ .)
- (b)  $S_1 S_0 = 10 = 1 \times 2 + 0 = 2 \rightarrow I_2$ .
- (c)  $2^n = 32 \rightarrow 2^5 = 32 \rightarrow n = 5$ .
- (d) Tree: 16 inputs  $\rightarrow 8$  MUX (16 se 8),  $\rightarrow 4$  MUX (8 se 4),  $\rightarrow 2$  MUX (4 se 2),  $\rightarrow 1$  MUX (2 se 1).  $8 + 4 + 2 + 1 = 15 = 16 - 1$ .

**Answer:** (a) 0; (b)  $I_2$ ; (c) 5; (d) 15.

### Solved example 2 (exam level, mock diagram question):

Implement  $F(A, B, C) = \sum m(0, 3, 5, 6)$  using a  $4:1$  MUX with A, B on the select lines ( $S_1 = A$ ,  $S_0 = B$ ).

1. Truth table likho, rows ko AB ke hisaab se jodiyon mein baanto (har jodi mein  $C = 0$  aur  $C = 1$ ):

| AB | C = 0 row           | C = 1 row           | F (C = 0, C = 1) | Data input |
|----|---------------------|---------------------|------------------|------------|
| 00 | $m_0 \rightarrow 1$ | $m_1 \rightarrow 0$ | 1, 0             | $I_0 = C'$ |
| 01 | $m_2 \rightarrow 0$ | $m_3 \rightarrow 1$ | 0, 1             | $I_1 = C$  |
| 10 | $m_4 \rightarrow 0$ | $m_5 \rightarrow 1$ | 0, 1             | $I_2 = C$  |
| 11 | $m_6 \rightarrow 1$ | $m_7 \rightarrow 0$ | 1, 0             | $I_3 = C'$ |

2. Har jodi ke liye F ko C ke roop mein likho:  $(0, 0) \rightarrow 0$ ;  $(1, 1) \rightarrow 1$ ;  $(0, 1) \rightarrow C$ ;  $(1, 0) \rightarrow C'$ .
3.  $AB = 00$ :  $F = 1$ ,  $0 \rightarrow C'$ .  $AB = 01$ :  $0, 1 \rightarrow C$ .  $AB = 10$ :  $0, 1 \rightarrow C$ .  $AB = 11$ :  $1, 0 \rightarrow C'$ .
4. Circuit: 4:1 MUX (gate level D3.2c),  $S1 = A$ ,  $S0 = B$ ,  $I0 = C'$ ,  $I1 = C$ ,  $I2 = C$ ,  $I3 = C'$  ( $C'$  ke liye ek NOT).
5. Check  $A = 1$ ,  $B = 1$ ,  $C = 0$  (m6, F should be 1): select 11  $\rightarrow I3 = C' = 1 \checkmark$ .

**Answer:**  $I0 = C'$ ,  $I1 = C$ ,  $I2 = C$ ,  $I3 = C'$ . (With an 8:1 MUX and A, B, C on selects:  $I0$  to  $I7 = 1, 0, 0, 1, 0, 1, 1, 0$ .)

**Galti mat karna:** 8:1 MUX ko 8 select lines nahi, 3 chahiye. MUX (many  $\rightarrow$  one) ko DEMUX (one  $\rightarrow$  many) se mix mat karo. Verilog mein {s1, s0} ka order ulta mat karo, aur case mein default bhoole to latch.

**Trick:** "Select ka number = input ka number." Function ke liye: "variables select pe, truth table data pe."

**Ek line mein:** n select lines  $\rightarrow 2^n$  inputs mein se ek output pe; 4:1 = har input AND uska select-minterm, sab OR.

### 3.3a Demultiplexer (DEMUX)

**Pehle socho:** Post office ka sorting counter: ek parcel aata hai, aur pin code (select) dekh ke wo **kai dabbon mein se ek** mein daal diya जाता है. Baaki dabbe khaali rehte hain. MUX ka bilkul ulta kaam: ek input, kai outputs.

**Definition:** A demultiplexer (DEMUX) is a combinational circuit that routes a single data input to one of  $2^n$  outputs selected by n select lines; the selected output equals the data input and all other outputs are 0.

**Mushkil shabd, aasaan matlab:**

**data input (D)**

Jo signal bhejna hai (parcel).  $D = 0$  ya  $1$  kuch bhi ho sakta hai.

**select lines**

Address: batate hain kaunsa output.  $S1S0 = 11 \rightarrow Y3$ .

**route**

Signal ko ek raaste pe bhejna. Chuna hua output = D, baaki = 0.

**1:2, 1:4 naming**

Course mein "1 input : outputs" likhte hain. 1:4 DEMUX = 1 data in, 4 out, 2 selects. "2:1 DEMUX" ulta padhta hai, mat likho.

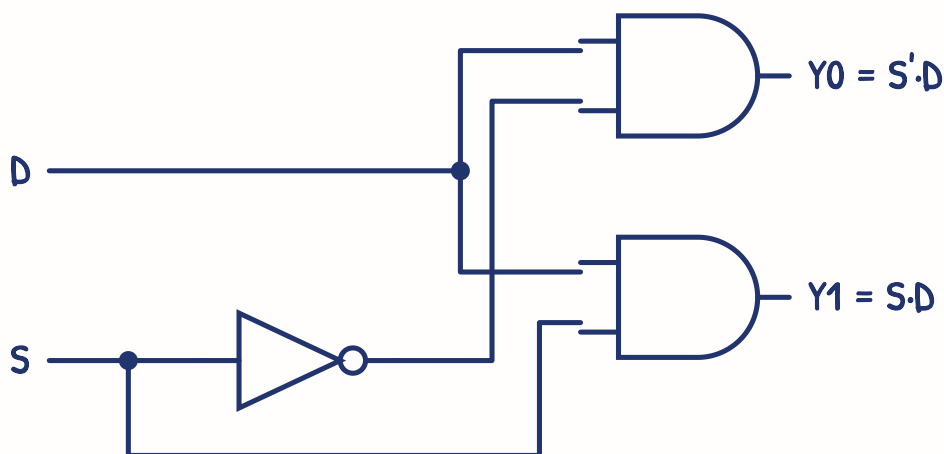
**Kaise kaam karta hai:**

1. **1:2 DEMUX** (input D, select S):  $S = 0 \rightarrow Y0 = D, Y1 = 0$ .  $S = 1 \rightarrow Y0 = 0, Y1 = D$ .
2. Equations:  $Y0 = S' \cdot D, Y1 = S \cdot D$  (1 NOT + 2 AND).
3. **1:4 DEMUX** (selects S1 S0): har output = D AND us output ke number ka select-minterm.
4.  $Y0 = S1'S0'D, Y1 = S1'S0D, Y2 = S1S0'D, Y3 = S1S0D$  (4 three-input AND + 2 NOT).
5. Agar D = 1 permanently laga do to chuna hua output 1 aur baaki 0: ye **decoder** ban jaata hai (3.3b).

| S1 | S0 | Y0 | Y1 | Y2 | Y3 |
|----|----|----|----|----|----|
| 0  | 0  | D  | 0  | 0  | 0  |
| 0  | 1  | 0  | D  | 0  | 0  |
| 1  | 0  | 0  | 0  | D  | 0  |
| 1  | 1  | 0  | 0  | 0  | D  |

**Teeno blocks ek saath** (decoder agle topic mein detail se):

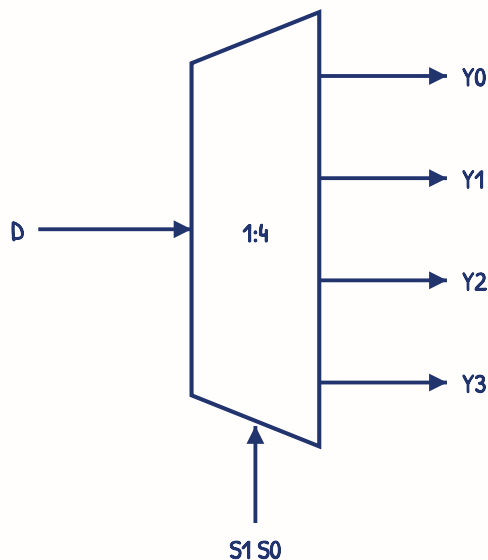
|         | MUX                   | DEMUX                             | Decoder                          |
|---------|-----------------------|-----------------------------------|----------------------------------|
| Inputs  | $2^n$ data + n select | 1 data + n select                 | n code bits (+ enable)           |
| Outputs | 1                     | $2^n$                             | $2^n$ (one-hot)                  |
| Kaam    | kai mein se ek chunna | ek ko kai mein se ek jagah bhejna | code ke hisaab se ek line jalana |



#### 1:2 DEMUX from gates

How to draw: NOT on S;  $Y0 = \text{AND}(D, S')$ ,  $Y1 = \text{AND}(D, S)$ .

**Diagram kaise padhein:** D dono AND gates mein jaata hai. S seedha neeche wale AND mein ( $Y1$ ), aur NOT ke baad  $S'$  upar wale AND mein ( $Y0$ ). Trace  $D = 1, S = 1$ : upar =  $1 \cdot 0 = 0$ , neeche =  $1 \cdot 1 = 1 \rightarrow Y1 = 1$ .



$S1\ S0 \mid Y0\ Y1\ Y2\ Y3$

0 0 | D 0 0 0

0 1 | 0 D 0 0

1 0 | 0 0 D 0

1 1 | 0 0 0 D

$Y0 = S1'S0'D$   $Y1 = S1'S0D$

$Y2 = S1S0'D$   $Y3 = S1S0D$

### 1:4 DEMUX symbol and routing table

How to draw: Reverse trapezoid with one input on the left, four outputs on the right, 2-bit select below.

**Diagram kaise padhein:** Ulta trapezoid: left se ek D, right mein Y0 to Y3, neeche S1 S0. Right ki table: jis row mein select hai, wahan D likha hai, baaki 0.

### Solved example 1:

A 1:4 DEMUX has  $D = 1$ ,  $S1S0 = 10$ . Find  $Y0$  to  $Y3$ . What if  $D = 0$ ?

1.  $S1S0 = 10 = 2 \rightarrow$  chuna hua output  $Y2$ .
2.  $Y2 = D = 1$ ;  $Y0 = Y1 = Y3 = 0$ .
3.  $D = 0$ :  $Y2 = D = 0$ , baaki bhi 0  $\rightarrow$  sab 0.

**Answer:**  $Y3\ Y2\ Y1\ Y0 = 0100$ ; with  $D = 0$ , 0000.

### Solved example 2 (exam level):

Design a 1:4 DEMUX: equations, gate count, and explain how it becomes a 2-to-4 decoder.

1. Har output =  $D \cdot$  (select minterm):  $Y0 = S1'S0'D$ ,  $Y1 = S1'S0D$ ,  $Y2 = S1S0'D$ ,  $Y3 = S1S0D$ .
2. Gates:  $S1'$  aur  $S0'$  ke liye 2 NOT; har output ke liye ek 3-input AND  $\rightarrow$  4 AND.
3. Check  $S1S0 = 01$ ,  $D = 1$ :  $Y1 = 1 \cdot 1 \cdot 1 = 1$ ;  $Y0 = 1 \cdot 0 \cdot 1 = 0$ ;  $Y2 = 0$ ;  $Y3 = 0$  ✓.
4. D ko "enable" naam do aur 1 pe rakho:  $Y_k = m_k$  (select ka minterm). Yahi 2-to-4 decoder hai;  $D = 0 \rightarrow$  sab outputs 0 = decoder disabled.

**Answer:** 2 NOT + 4 three-input AND; decoder = DEMUX with data input tied to 1 (enable).

**Galti mat karna:** Na chune gaye outputs ko "purani value" ya "X" mat likho: wo 0 hote hain. Verilog mein isliye pehle sab outputs 0 kar dete hain ( $y = 4'b0000$ ;) phir case.

**Trick:** MUX = "Many  $\rightarrow$  1" (funnel), DEMUX = "1  $\rightarrow$  Many" (sorter).

**Ek line mein:** DEMUX ek data input ko select ke number wale output pe bhejta hai; baaki outputs 0.

### 3.3b Decoder ( $n$ -to- $2^n$ )

**Pehle socho:** Hotel mein 4 rooms hain: 0, 1, 2, 3. Reception pe ek board hai jisme har room ki ek bulb hai. Receptionist room number *binary* mein daalta hai (do switch: 00, 01, 10, 11) aur sirf usi room ki bulb jalti hai, baaki teen band rehti hain. Ye kaam karne wala circuit hi **decoder** hai: ek "code" (binary number) leke batata hai "kaunsi line?".

**Definition:** A **decoder** is a combinational circuit that converts an  $n$ -bit binary input into  $2^n$  output lines, such that for each input combination exactly one output line is active (1) and all others are 0.

**Mushkil shabd, aasaan matlab:**

**$n$ -to- $2^n$**

$n$  input bits,  $2^n$  output lines. 2 input bits  $\rightarrow 2^2 = 4$  outputs (2-to-4). 3 input bits  $\rightarrow 2^3 = 8$  outputs (3-to-8).

**one-hot**

"Ek hi garam". Outputs ke group mein **exactly ek** bit 1 hai, baaki sab 0. Jaise 4 outputs D3 D2 D1 D0 = **0100**: sirf D2 "hot" hai. 0110 one-hot NAHI hai (do 1 hain), 0000 bhi nahi (koi 1 nahi). Decoder ka output hamesha one-hot hota hai (jab enable = 1).

**active / asserted**

Output "ON" hai, yaani 1 (active-high decoder mein).

**enable (E)**

Master switch.  $E = 0 \rightarrow$  saare outputs 0 (decoder band).  $E = 1 \rightarrow$  normal kaam.

**minterm**

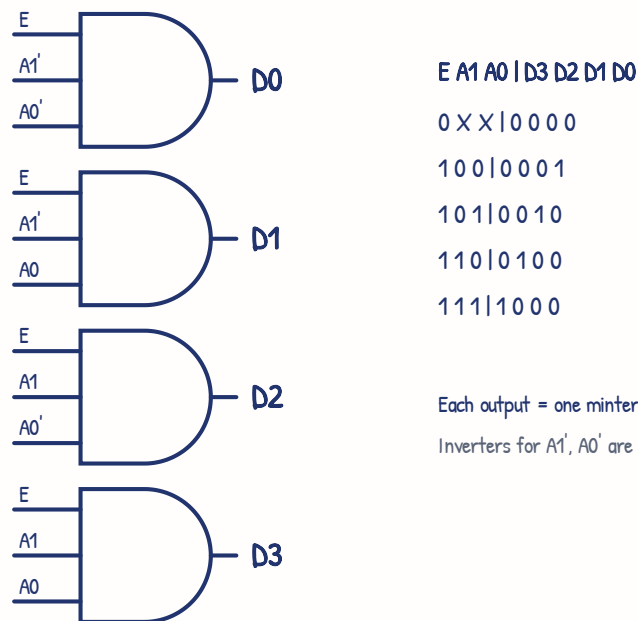
Ek AND term jisme har input exactly ek baar aata hai (seedha ya bar ke saath) aur jo sirf **ek** row mein 1 hota hai.  $A_1'A_0'$  sirf row 00 mein 1 hai, isliye ye minterm  $m_0$  hai.

**Kaise kaam karta hai (2-to-4 decoder, inputs  $A_1 A_0$ ):**

1. Input ek 2-bit number hai:  $A_1 A_0 = 00, 01, 10$  ya  $11$ , yaani decimal 0, 1, 2 ya 3.
2. Har number ke liye ek output line hai: D0, D1, D2, D3.
3. Rule: input ka decimal value  $k$  hai to sirf  $D_k = 1$ .
4. Har output ek AND gate hai jo sirf apne number pe 1 deta hai. D2 ko "10" chahiye, yaani  $A_1 = 1$  aur  $A_0 = 0$ . Isliye  $D_2 = A_1 \cdot A_0'$  ( $A_0$  pe NOT lagao taaki  $A_0 = 0$  hone pe AND ko 1 mile).
5. Isi tarah:  $D_0 = A_1'A_0'$ ,  $D_1 = A_1'A_0$ ,  $D_2 = A_1A_0'$ ,  $D_3 = A_1A_0$ . Har output = ek minterm.
6. Enable ho to har AND mein E bhi jod do:  $D_2 = E \cdot A_1A_0'$ .

| A1 | A0 | D3 | D2 | D1 | D0 |
|----|----|----|----|----|----|
| 0  | 0  | 0  | 0  | 0  | 1  |
| 0  | 1  | 0  | 0  | 1  | 0  |
| 1  | 0  | 0  | 1  | 0  | 0  |
| 1  | 1  | 1  | 0  | 0  | 0  |

Table mein dekho: har row mein sirf ek 1 hai, aur woh diagonal pe chalta hai. Yahi "one-hot" hai.



Each output = one minterm (one-hot).  
Inverters for A1', A0' are shared.

### 2-to-4 decoder with enable

How to draw: Four 3-input ANDs: E with each combination of A1/A1' and A0/A0'.

**Diagram kaise padhein:** Left se A1, A0 aate hain; har ek ka NOT bhi banta hai (A1', A0'). Right mein 4 AND gates hain, har AND ko teen wire milte hain: E, A1 ya A1', A0 ya A0'. Trace karo A1A0 = 10, E = 1: sirf D2 wale AND ko (1, 1, A0' = 1) teeno 1 milte hain, to sirf D2 = 1.

**Formula:** n inputs  $\rightarrow$  2<sup>n</sup> outputs, D<sub>k</sub> = m<sub>k</sub> (output k = minterm k).

**Gates:** 2<sup>n</sup> AND gates (each with n inputs, n + 1 with enable) + n NOT gates.

3-to-8: 8 AND gates of 3 inputs + 3 NOT gates.

### Solved example 1 (direct):

A 3-to-8 decoder has inputs A2A1A0 = 101 and E = 1. Which output is 1? Write the output word D7 to D0.

- 101 ko decimal mein badlo:  $1 \times 4 + 0 \times 2 + 1 \times 1 = 4 + 0 + 1 = 5$ .
- Rule: sirf D5 = 1.
- D7 D6 D5 D4 D3 D2 D1 D0 likho, D5 ki jagah 1: 0 0 1 0 0 0 0 0.

**Answer:** D5 = 1; output = 0010 0000 (one-hot).

### Solved example 2 (exam favourite):

*Implement a full adder using a 3-to-8 decoder and OR gates.*

1. Full adder ke inputs A, B, Cin ko decoder ke A2, A1, A0 pe lagao. Ab decoder har row ka minterm khud bana deta hai.
2. Sum = 1 jab 1s ki ginti odd ho: rows 001, 010, 100, 111 → decimal 1, 2, 4, 7. Isliye Sum =  $\sum m(1, 2, 4, 7)$ .
3. Cout = 1 jab kam se kam do 1 hon: rows 011, 101, 110, 111 → 3, 5, 6, 7. Isliye Cout =  $\sum m(3, 5, 6, 7)$ .
4. Ek 4-input OR gate lo: inputs D1, D2, D4, D7 → output Sum.
5. Doosra 4-input OR: D3, D5, D6, D7 → output Cout.

**Answer:** Sum = D1 + D2 + D4 + D7; Cout = D3 + D5 + D6 + D7. (Any function = OR of the decoder outputs of its minterms.)

**Galti mat karna:** Decoder aur MUX ko mix mat karo. **MUX data pass karta hai** (many in → one out). **Decoder data pass nahi karta**, sirf batata hai "kaunsi line" (n in → 2<sup>n</sup> out, one-hot). DEMUX = decoder jiska enable hi data input hai.

**Trick:** "De-code = number padho, bulb jalao." Input ka decimal value = jalne wali bulb ka number.

**Ek line mein:** n-bit number in → 2<sup>n</sup> lines out → sirf us number wali line 1 (one-hot); har output ek minterm hai.

## 3.4 Half Adder Design

**Pehle socho:** Do single digits jodo: 7 + 5 = 12 → likho 2, carry 1. Binary mein bhi same: 1 + 1 = 10 (do) → likho 0, carry 1. Ek bit ke do numbers jodne wala sabse chhota circuit **half adder** hai. CPU ka har addition (PC + 4, address calculation, subtraction) adders se hota hai.

**Definition:** A **half adder** is a combinational circuit that adds two single bits A and B and produces a Sum bit and a Carry bit; Sum =  $A \oplus B$  and Carry =  $A \cdot B$ . It has no carry input.

**Mushkil shabd, aasaan matlab:**

**Sum (S)**

Jod ka right wala bit (jo isi column mein likha jaata hai).  $1 + 1 = 10 \rightarrow S = 0$ .

**Carry (C)**

Agle column mein jaane wala bit.  $1 + 1 = 10 \rightarrow C = 1$ .

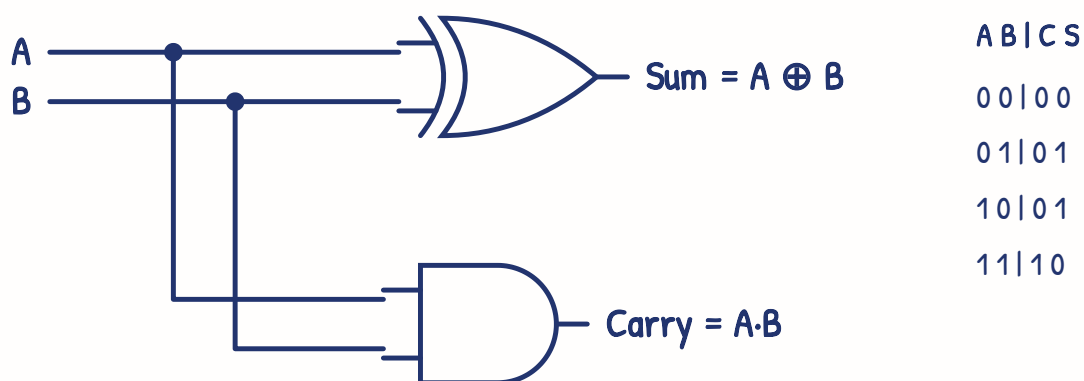
## carry-in (Cin)

Pichle column se aaya carry. Half adder mein ye input hai hi nahi, isliye "half".

**Kaise kaam karta hai (truth table se derive karo):**

1. Chaaron cases jodo:  $0 + 0 = 00$ ,  $0 + 1 = 01$ ,  $1 + 0 = 01$ ,  $1 + 1 = 10$  (Carry Sum).
2. Table banao (neeche).
3. Sum column = 0, 1, 1, 0 → 1-rows 01 aur 10 → SOP:  $\text{Sum} = A'B + AB' = A \oplus B$  (XOR ka pattern).
4. Carry column = 0, 0, 0, 1 → sirf row 11 →  $\text{Carry} = A \cdot B$  (AND ka pattern).
5. Circuit: ek XOR + ek AND, dono ko A aur B milte hain.

| A | B | Carry | Sum | A + B (decimal) |
|---|---|-------|-----|-----------------|
| 0 | 0 | 0     | 0   | 0               |
| 0 | 1 | 0     | 1   | 1               |
| 1 | 0 | 0     | 1   | 1               |
| 1 | 1 | 1     | 0   | $2 = 10_2$      |



**Half adder: circuit and truth table**

How to draw: XOR and AND side by side, both fed by A and B.

**Diagram kaise padhein:** A aur B dono gates ko jaate hain (black dots = wire split). Upar XOR → Sum, neeche AND → Carry. Right mein truth table. Trace  $A = 1, B = 1$ : XOR = 0, AND = 1 → Carry Sum = 10.

**Formula:**  $\text{Sum} = A \oplus B = A'B + AB'$  ·  $\text{Carry} = A \cdot B$

Gates: 1 XOR + 1 AND. With 2-input NANDs only: 5 (4 for XOR, the first NAND  $(AB)'$  reused, 1 more NAND inverts it to  $AB$ ).

### Solved example 1:

Half adder inputs  $A = 1, B = 1$ . Write "Carry Sum".

1.  $\text{Sum} = 1 \oplus 1 = 0$  (inputs same → 0).
2.  $\text{Carry} = 1 \cdot 1 = 1$ .
3. Check:  $1 + 1 = 2 = 10_2$  ✓.

Answer: 10.

### Solved example 2 (exam level):

Minimum number of 2-input NAND gates to build a half adder (Sum and Carry)?

1. XOR from NANDs (2.5):  $G1 = (AB)'$ ,  $G2 = (A \cdot G1)'$ ,  $G3 = (B \cdot G1)'$ , Sum =  $G4 = (G2 \cdot G3)'$  → 4 gates.
2. Carry =  $AB = (G1)'$  → G1 already hai, bas ek NAND-inverter:  $G5 = \text{NAND}(G1, G1) = AB$ .
3. Total =  $4 + 1 = 5$ .
4. Check  $A = 1, B = 1$ :  $G1 = 0, G2 = 1, G3 = 1, G4 = 0$  (Sum),  $G5 = 1$  (Carry) ✓.

Answer: 5 NAND gates.

**Galti mat karna:** Sum =  $A + B$  (OR) GALAT: 1 + 1 pe OR 1 deta hai par Sum 0 chahiye. Aur multi-bit addition ke bit 1 pe half adder nahi chalega: wahan teen bits ( $A1, B1$ , pichla carry) jodne hain. Example 11 + 01: bit 0 pe carry banta hai jo bit 1 mein jaana chahiye.

**Ek line mein:** Half adder: Sum =  $A \oplus B$ , Carry =  $AB$ ; carry-in nahi hai, isliye chain nahi ho sakta.

## 3.5 Full Adder Design (and Ripple-Carry Adder)

**Pehle socho:** Column addition mein har column mein **teen** cheezein judti hain: upar ka digit, neeche ka digit, aur pichle column ka carry. Binary mein bhi multi-bit addition ke liye har bit pe  $A, B$  aur  $Cin$  jodne padte hain. Ye **full adder** karta hai.  $n$  full adders ko line mein jodo to  $n$ -bit adder ban jaata hai.

**Definition:** A **full adder** adds three bits,  $A, B$  and a carry-in  $Cin$ , and produces Sum =  $A \oplus B \oplus Cin$  and carry-out  $Cout = AB + Cin(A \oplus B)$ . An  **$n$ -bit ripple-carry adder (RCA)** chains  $n$  full adders, the carry-out of each stage feeding the carry-in of the next.

**Mushkil shabd, aasaan matlab:**

**parity (Sum)**

Teen bits mein 1s ki ginti odd ho to Sum = 1.  $1 + 1 + 1 = 3 = 11_2 \rightarrow$  Sum 1.

**majority (Cout)**

Teen mein se kam se kam do 1 hon to Cout = 1 (voting).

**ripple**

Carry ek stage se agle mein lehar ki tarah chalta hai: bit 0 ka carry bit 1 mein, phir bit 2 mein.

**propagation delay**

Input aane ke baad output sahi hone mein laga time. Ek gate delay = ek gate paar karne ka time ( $\Delta$ ).

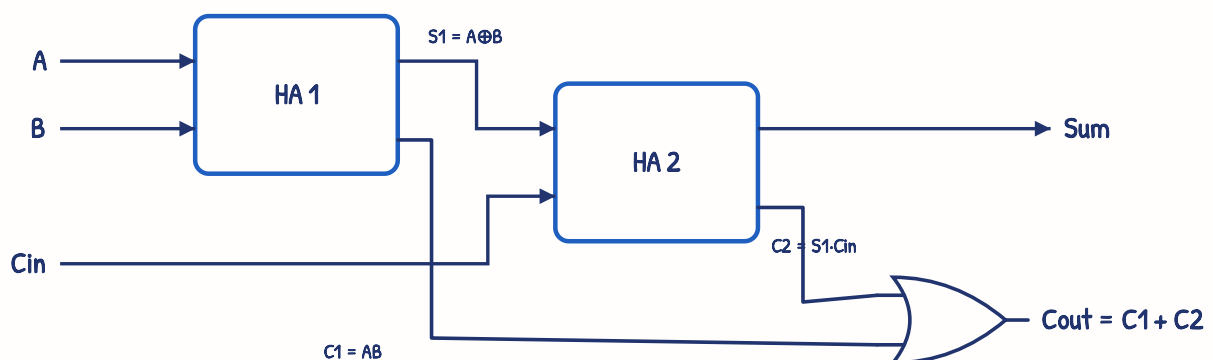
### Kaise kaam karta hai (truth table se derive):

- 8 rows (3 inputs). Har row mein  $A + B + \text{Cin}$  jodo aur 2-bit answer Cout Sum likho.
- Sum = 1 rows: 001, 010, 100, 111  $\rightarrow \Sigma m(1, 2, 4, 7)$ . Cout = 1 rows: 011, 101, 110, 111  $\rightarrow \Sigma m(3, 5, 6, 7)$ .
- Sum ka K-map checkerboard (koi do 1 padosi nahi)  $\rightarrow$  group nahi banta  $\rightarrow$  **Sum =  $A \oplus B \oplus \text{Cin}$** .
- Cout ka K-map: teen pairs, sab m7 ke saath: (3, 7)  $\rightarrow B \cdot \text{Cin}$ , (5, 7)  $\rightarrow A \cdot \text{Cin}$ , (6, 7)  $\rightarrow A \cdot B \rightarrow$  **Cout =  $AB + BCin + ACin$** .
- Do half adders wala form:  $\text{HA1}(A, B) \rightarrow S1 = A \oplus B, C1 = AB$ .  $\text{HA2}(S1, \text{Cin}) \rightarrow \text{Sum} = S1 \oplus \text{Cin}, C2 = S1 \cdot \text{Cin}$ . **Cout =  $C1 + C2 = AB + \text{Cin}(A \oplus B)$** .
- C1 aur C2 kabhi saath mein 1 nahi ( $A = B = 1$  ho to  $S1 = 0$ , to  $C2 = 0$ ), isliye OR theek hai.

| row | A | B | Cin | Cout | Sum |
|-----|---|---|-----|------|-----|
| 0   | 0 | 0 | 0   | 0    | 0   |
| 1   | 0 | 0 | 1   | 0    | 1   |
| 2   | 0 | 1 | 0   | 0    | 1   |
| 3   | 0 | 1 | 1   | 1    | 0   |
| 4   | 1 | 0 | 0   | 0    | 1   |
| 5   | 1 | 0 | 1   | 1    | 0   |
| 6   | 1 | 1 | 0   | 1    | 0   |
| 7   | 1 | 1 | 1   | 1    | 1   |

| Sum: A \ B Cin | 00 | 01 | 11 | 10 |
|----------------|----|----|----|----|
| 0              | 0  | 1  | 0  | 1  |
| 1              | 1  | 0  | 1  | 0  |

| Cout: A \ B Cin | 00 | 01 | 11 | 10 |
|-----------------|----|----|----|----|
| 0               | 0  | 0  | 1  | 0  |
| 1               | 0  | 1  | 1  | 1  |

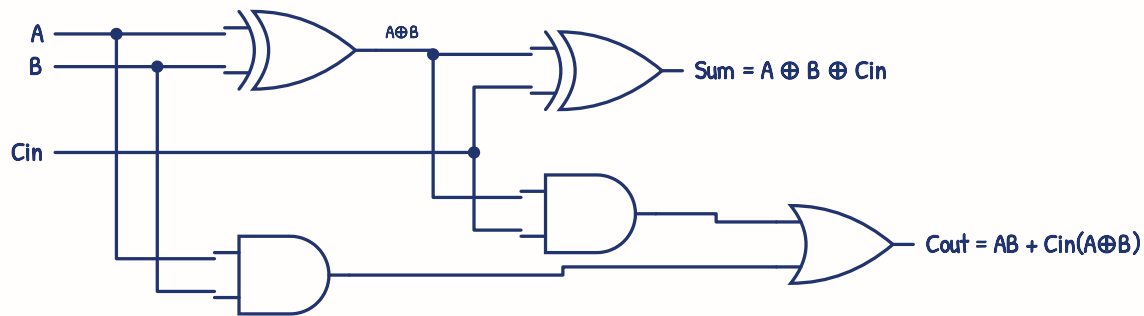


C1 and C2 are never 1 together (if  $A = B = 1$  then  $S1 = 0$ ), so an OR (or XOR) merges them.

Full adder from two half adders + OR

How to draw: Two HA boxes in series, OR their carries.

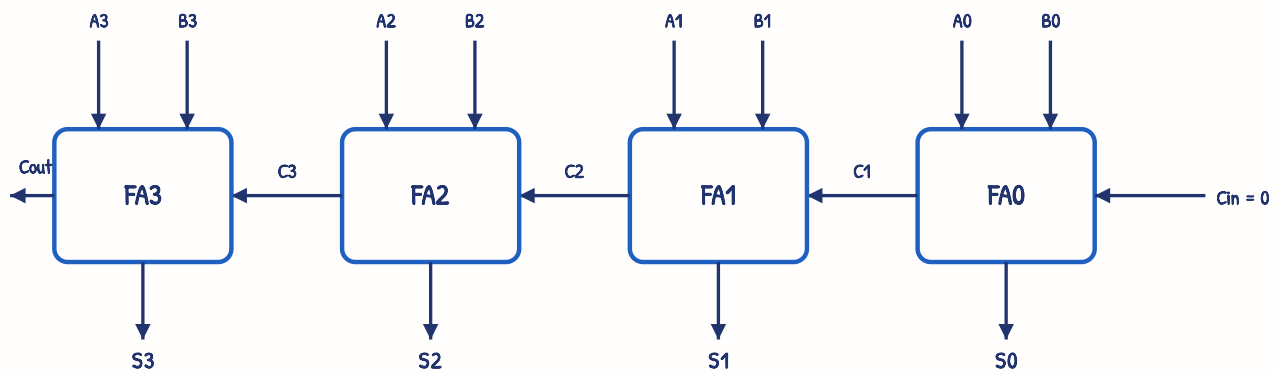
**Diagram kaise padhein:** HA 1 ko A, B milte hain; uska  $S1 = A \oplus B$ , HA 2 mein  $Cin$  ke saath jaata hai  $\rightarrow$  Sum. Dono half adders ke carries ( $C1 = AB$ ,  $C2 = S1 \cdot Cin$ ) ek OR mein  $\rightarrow$  Cout. Trace  $A = 1$ ,  $B = 0$ ,  $Cin = 1$ :  $S1 = 1$ ,  $Sum = 1 \oplus 1 = 0$ ,  $C1 = 0$ ,  $C2 = 1 \cdot 1 = 1 \rightarrow Cout = 1$  ( $1 + 0 + 1 = 10_2$  ✓).



#### Full adder gate level

How to draw: Two XORs in series for Sum; two ANDs ( $AB$  and  $Cin \cdot (A \oplus B)$ ) into an OR for Cout.

**Diagram kaise padhein:** Same circuit gate level: do XOR series mein (Sum), do AND ( $AB$  aur  $Cin \cdot (A \oplus B)$ ) ek OR mein (Cout).



Carry ripples right  $\rightarrow$  left; worst-case delay  $\approx n \times t(\text{carry})$ . 32-bit  $\rightarrow$  32 stages.

#### 4-bit ripple-carry adder

How to draw: Four FA boxes right to left (FA0 on the right), carry arrows leftward,  $Cin = 0$  at the right.

**Diagram kaise padhein:** FA0 right mein (LSB), FA3 left mein (MSB). Har FA ko upar se  $A_i$ ,  $B_i$  milte hain, neeche  $S_i$  nikalta hai. Carry right se left chalta hai:  $Cin = 0 \rightarrow FA0 \rightarrow C1 \rightarrow FA1 \rightarrow C2 \rightarrow FA2 \rightarrow C3 \rightarrow FA3 \rightarrow Cout$ . FA3 ka Sum tab tak sahi nahi jab tak  $C3$  nahi aata.

**Formula:**  $Sum = A \oplus B \oplus Cin$ ,  $Cout = AB + Cin(A \oplus B) = AB + BCin + ACin$

RCA worst-case delay =  $n \times t(\text{carry per stage})$ ; class model: 2 gate delays per stage  $\rightarrow 2n$  gate delays (4-bit 8, 16-bit 32, 64-bit 128).

FA = 2 HA + 1 OR; n-bit RCA = n FAs = 2n HAs.

#### Solved example 1 (class quiz, trace):

A 4-bit RCA ( $Cin = 0$ ) adds 0111 + 0001. Find the sum and Cout, showing every carry.

1. Bit 0:  $A0 = 1$ ,  $B0 = 1$ ,  $Cin = 0 \rightarrow 1 + 1 + 0 = 2 = 10_2 \rightarrow S0 = 0$ ,  $C1 = 1$ .

2. Bit 1:  $A_1 = 1, B_1 = 0, C_1 = 1 \rightarrow 1 + 0 + 1 = 2 \rightarrow S_1 = 0, C_2 = 1$ .
3. Bit 2:  $A_2 = 1, B_2 = 0, C_2 = 1 \rightarrow 2 \rightarrow S_2 = 0, C_3 = 1$ .
4. Bit 3:  $A_3 = 0, B_3 = 0, C_3 = 1 \rightarrow 1 \rightarrow S_3 = 1, C_{out} = 0$ .
5. Sum =  $S_3 S_2 S_1 S_0 = 1000$ . Check:  $7 + 1 = 8 = 1000_2 \checkmark$ . Carry 3 stages tak ripple hua.

**Answer: 1000, Cout = 0.**

### Solved example 2 (exam level, delay):

A 32-bit ripple-carry adder uses full adders whose carry path is 2 gate delays; one gate delay = 25 ps.

(a) Worst-case carry delay? (b) Maximum clock frequency if this adder is the only logic between registers? (c) What limits its speed?

1. (a) Gate delays =  $2 \times 32 = 64$ . Time =  $64 \times 25 \text{ ps} = 1600 \text{ ps} = 1.6 \text{ ns}$ .
2. (b)  $f = 1 / T = 1 / 1.6 \text{ ns} = 0.625 \text{ GHz} = 625 \text{ MHz}$ .
3. (c) Worst case (jaise saare 1s + 1, 4 bits mein  $1111 + 0001$ ) mein carry ko bit 0 se bit 31 tak har stage se guzarana padta hai; har stage pichle carry ka wait karta hai. Delay n ke saath linearly badhta hai.
4. Fix: carry-lookahead adder (3.E1) jo carries parallel mein nikalta hai.

**Answer: (a) 1.6 ns; (b) 625 MHz; (c) serial carry propagation, delay  $\propto n$ .**

**Galti mat karna:** Cout =  $C_1 \text{ AND } C_2$  likha to kabhi 1 nahi hoga (dono saath 1 nahi hote): OR chahiye. Cout =  $AB + C_{in}$  bhi galat ( $A = B = 0, C_{in} = 1$  pe 1 de dega). Ripple-carry ko "fast" mat likhna: wo simple hai, fast nahi.

**Trick:** Sum = "odd hai to 1" (parity), Cout = "do ya zyada to 1" (majority). FA = "2 HA + 1 OR".

**Ek line mein:** Full adder teen bits jodta hai (Sum =  $A \oplus B \oplus C_{in}$ , Cout = majority); n FAs ki chain = RCA, delay  $2n$  gate delays.

## 3.6 CPU Preview: ALU and Register File

**Pehle socho:** CPU ki table pe ek chhoti **daraaz** (register file: kuch fast slots jinme numbers rakhe hain) aur ek **calculator** (ALU) hai. Har instruction jaise add: daraaz se do numbers nikalo, calculator pe jodo, result wapas daraaz mein. Ye dono isi unit ke blocks (MUX, decoder, adder) se bante hain.

**Definition:** The **ALU (Arithmetic Logic Unit)** is the combinational unit of the CPU that performs arithmetic (add, subtract) and logic (AND, OR, compare) operations on two operands, selected by ALU control bits. The **register file** is a small, fast array of registers ( $32 \times 32$  bits in MIPS) with two read ports and one write port that supplies the ALU's operands and stores its results.

## Mushkil shabd, aasaan matlab:

### 1-bit ALU slice

Ek bit ka ALU: AND gate, OR gate, full adder, aur ek MUX jo inme se ek result chunta hai. 32 slices = 32-bit ALU.

### Operation (select)

MUX ke select bits: 00 = AND, 01 = OR, 10 = add.

### Binvert

Ek 2:1 MUX jo B ya B' adder ko deta hai.  $B' + \text{CarryIn} = 1$  se  $A - B$  banta hai (two's complement, Unit 4).

### Zero flag

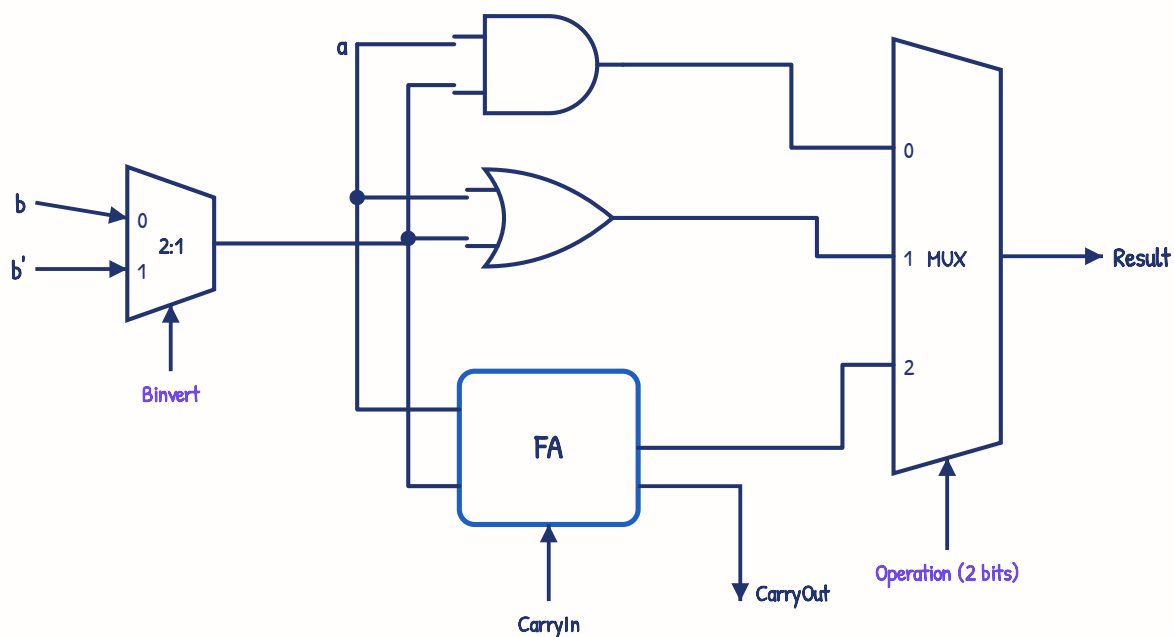
Output jo 1 hota hai jab result ke saare bits 0 hon. NOR of all result bits. beq isi se compare karta hai.

### read port / write port

Read port = register ki value bahar dena (MUX se). Write port = ek register mein likhna (decoder + write enable, clock edge pe).

## Kaise kaam karta hai:

- 1-bit slice ke andar a aur b (ya b') teen jagah jaate hain: AND, OR, full adder. Teeno parallel mein result banate hain.
- Operation MUX inme se ek ko Result pe bhejta hai.
- Subtract: Binvert = 1 (b' liya) aur pehle slice ka CarryIn = 1  $\rightarrow A + B' + 1 = A - B$ .
- 32 slices jodo: slice i ka CarryOut  $\rightarrow$  slice i+1 ka CarryIn (ripple, 3.5).
- Zero = NOR(R31, R30, down to R0): sab 0 hon tabhi 1.
- 4-bit ALU control (Patterson & Hennessy): bits = Ainvert, Bnegate, Operation(2). **0000 AND, 0001 OR, 0010 add, 0110 sub** (Bnegate = 1 + add), 0111 slt, 1100 NOR ( $A' \text{ AND } B' = (A + B)'$ ).
- Register file  $\rightarrow$  ALU  $\rightarrow$  register file: har R-type instruction ka loop (Unit 7 aur 13 mein detail).

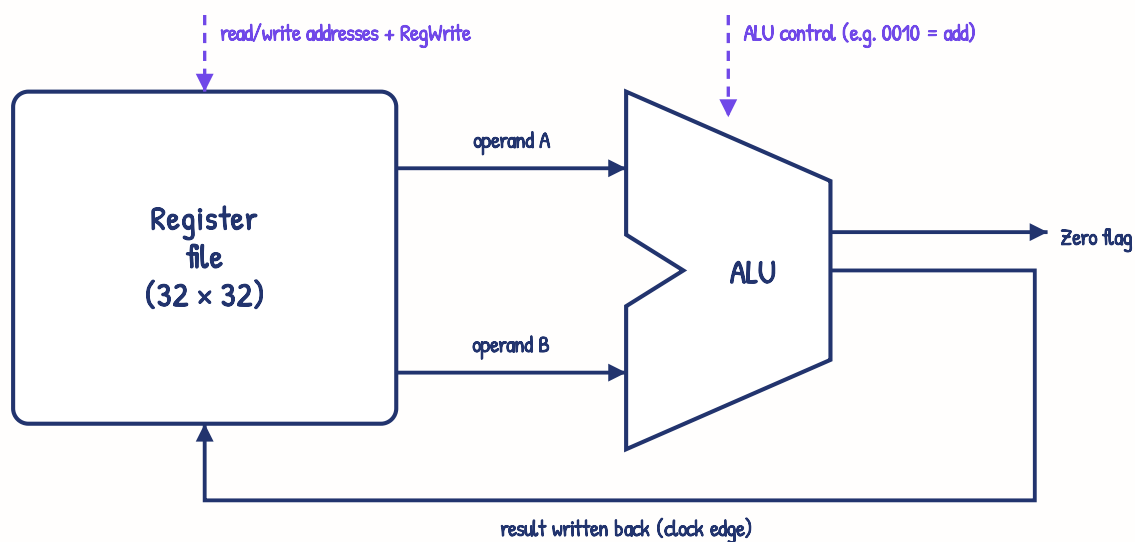


Op 00 = AND, 01 = OR, 10 = add (subtract with Binvert = 1 and CarryIn = 1). 32 slices chained = 32-bit ALU.

#### 1-bit ALU slice (AND, OR, add/subtract)

How to draw: Binvert MUX on b, then AND, OR and FA in parallel, a 3:1 MUX on Operation picks the result.

**Diagram kaise padhein:** Left mein Binvert MUX b ya b' chunta hai. Beech mein AND, OR aur FA (FA ko neeche se CarryIn, bahar CarryOut). Right ka MUX (Operation, 2 bits) teeno mein se Result chunta hai. Trace a = 1, b = 1, Operation = 00: AND = 1 → Result = 1.



#### Preview: register file feeding the ALU

How to draw: Register-file box with two outputs into an ALU; the result loops back to the register file.

**Diagram kaise padhein:** Register file ke do outputs (operand A, operand B) ALU mein. ALU control upar se aata hai (0010 = add). Result neeche se ghoom ke wapas register file mein likha jaata hai (clock edge pe). Zero flag right mein nikalta hai.

#### Solved example 1 (exam level):

A 4-bit ALU gets  $a = 0101$ ,  $b = 0011$  and ALU control  $0110$ . Show how it computes the result and the Zero flag.

1.  $0110$  = subtract: Binvert (Bnegate) = 1, Operation = add, CarryIn = 1.
2.  $b' = 0011 \text{ ulta} = 1100$ .
3.  $a + b' + 1 = 0101 + 1100 + 1$ :  $0101 + 1100 = 10001$ , 4 bits rakho  $\rightarrow 0001$ ; phir  $+ 1 \rightarrow 0010$ .
4. Check:  $5 - 3 = 2 = 0010 \checkmark$ .
5. Result ke bits 0, 0, 1, 0: sab zero nahi  $\rightarrow$  Zero = 0. (Agar  $a = b = 0011$  hota to result  $0000$ , Zero = 1.)

**Answer: Result = 0010, Zero = 0.**

### Solved example 2 (write the answer like this):

Draw a 1-bit ALU for AND, OR, add and subtract. How do 32 slices form a 32-bit ALU and how is Zero produced? (5 marks)

1. Draw D3.6a: Binvert MUX on b; AND, OR and full adder in parallel; 3:1 Operation MUX gives Result.
2. Operation 00 = AND, 01 = OR, 10 = add; subtraction = add with Binvert = 1 and CarryIn = 1.
3. 32 slices: CarryOut of slice  $i \rightarrow$  CarryIn of slice  $i + 1$ ; slice 0's CarryIn = 1 for subtract.
4. Zero = NOR of all 32 result bits (tree of ORs + one inverter); used by beq.

**Answer: diagram + subtract rule + chaining + Zero = NOR(result bits).**

**Galti mat karna:** ALU kuch store nahi karta (combinational); storage register file hai. Subtraction mein sirf  $B'$  kaafi nahi, CarryIn = 1 bhi chahiye (warna one's complement, answer 1 kam).

**Trick:** "add 0010, sub 0110": sub = add + Bnegate bit (bit 2) on.

**Ek line mein:** ALU = AND + OR + adder + MUX per bit (sub =  $B'$  + carry 1, Zero = NOR); register file = 2 read ports (MUX) + 1 write port (decoder).

## 3.E1 Extra from slides: Carry-Lookahead Adder

**Pehle socho:** Ripple adder mein bit 31 ko apne carry ke liye bit 0 se 30 tak sabka intezaar karna padta hai, jaise line mein khade log ek ek karke message aage bolte hain. Kya har aadmi seedha shuru wale se message nahi le sakta? CLA yahi karta hai: **har carry ko seedha inputs se nikal leta hai**, bina wait ke.

**Definition:** A carry-lookahead adder (CLA) computes all carries in parallel directly from the inputs, using a generate signal  $G_i = A_i \cdot B_i$  and a propagate signal  $P_i = A_i \oplus B_i$  for each bit, with  $C_{(i+1)} = G_i + P_i \cdot C_i$  expanded into two-level logic, instead of waiting for the carry to ripple.

**Mushkil shabd, aasaan matlab:**

**Generate  $G_i = A_i \cdot B_i$**

"Ye bit khud carry banata hai", chahe neeche se carry aaye ya na aaye.  $A_i = B_i = 1 \rightarrow 1 + 1 = 10 \rightarrow$  carry pakka.

**Propagate  $P_i = A_i \oplus B_i$**

"Ye bit neeche se aaya carry aage pass kar deta hai".  $A_i, B_i$  mein ek 1:  $1 + 0 + C_i \rightarrow C_i = 1$  ho to carry aage jaata hai.

**$C_0$**

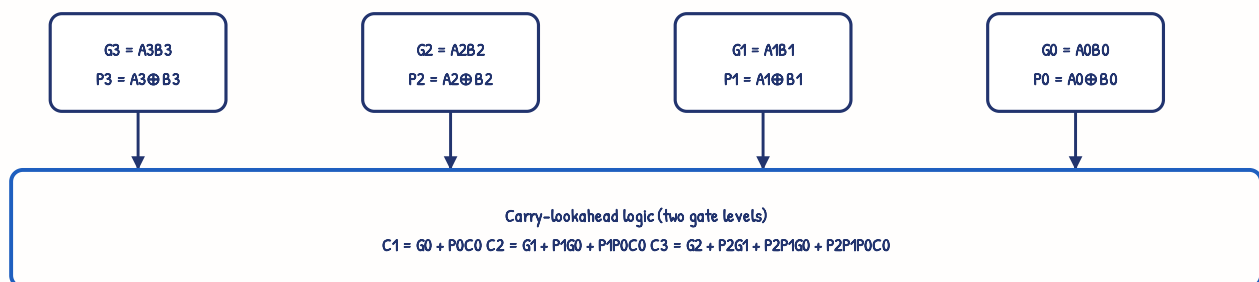
Sabse pehla carry-in (addition ke liye 0, subtraction ke liye 1).

**two-level logic**

AND phir OR (SOP): sirf 2 gate delays, chahe term kitne bhi hon.

**Kaise kaam karta hai ( $C_1$  se  $C_4$  nikalna):**

1. Basic rule: bit  $i$  se carry tab nikalega jab bit  $i$  khud generate kare, YA propagate kare aur neeche se carry aaye:  $C_{(i+1)} = G_i + P_i \cdot C_i$ .
2.  $i = 0$ :  $C_1 = G_0 + P_0 C_0$ .
3.  $i = 1$ :  $C_2 = G_1 + P_1 C_1$ .  $C_1$  ki jagah upar wala daalo:  $C_2 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$ .
4.  $i = 2$ :  $C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$ .
5.  $i = 3$ :  $C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$ .
6. Har carry ab sirf G, P aur  $C_0$  pe depend karta hai, kisi doosre carry pe nahi. Isliye sab carries **ek saath** bante hain.
7. Sum:  $S_i = P_i \oplus C_i$ .
8. Delay: G, P (1 gate)  $\rightarrow$  saare carries (AND-OR = 2 gates)  $\rightarrow$  Sum (1 XOR)  $\approx$  4 gate delays, bit position se independent. 4-bit RCA mein carry ke liye 8.
9. Keemat: zyada gates ( $C_4$  mein 5-input AND tak), isliye area badhta hai; real CPUs 4-bit CLA blocks jodte hain ya hybrid adders use karte hain.



$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0; S_i = P_i \oplus C_i$$

All carries are computed in parallel from the inputs: delay no longer grows linearly with  $n$  (more gates = more area).

**Carry-lookahead adder: G/P generation and parallel carry logic**

How to draw: Row of G/P boxes on top, one wide "lookahead" box below producing  $C_1$ – $C_4$ .

**Diagram kaise padhein:** Upar chaar boxes, har bit ke liye  $G_i = A_i B_i$  aur  $P_i = A_i \oplus B_i$ . Sab neeche ek bade "lookahead" box mein jaate hain jo  $C_1, C_2, C_3$  ek saath nikalta hai ( $C_4$  neeche likha hai). Koi carry arrow ek box se doosre mein nahi jaata: yahi ripple se farak hai.

**Formula:**  $G_i = A_i B_i$ ,  $P_i = A_i \oplus B_i$ ,  $C_{(i+1)} = G_i + P_i C_i$ ,  $S_i = P_i \oplus C_i$

$$C_1 = G_0 + P_0 C_0 \quad C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 \quad C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

### Solved example 1 (exam level):

$A = 1011, B = 0110, C_0 = 0$ . Find all  $G_i, P_i$ , the carries  $C_1$  to  $C_4$  with the lookahead equations, and the sum.

1. Bit by bit (bit 0 = right):  $A_0 = 1, B_0 = 0 \rightarrow G_0 = 0, P_0 = 1$ .  $A_1 = 1, B_1 = 1 \rightarrow G_1 = 1, P_1 = 0$ .  $A_2 = 0, B_2 = 1 \rightarrow G_2 = 0, P_2 = 1$ .  $A_3 = 1, B_3 = 0 \rightarrow G_3 = 0, P_3 = 1$ .
2.  $C_1 = G_0 + P_0 C_0 = 0 + 1 \cdot 0 = 0$ .
3.  $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 = 1 + 0 + 0 = 1$ .
4.  $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 = 0 + 1 \cdot 1 + 0 + 0 = 1$ .
5.  $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 = 0 + 0 + 1 \cdot 1 \cdot 1 + 0 + 0 = 1$ .
6. Sums:  $S_0 = P_0 \oplus C_0 = 1 \oplus 0 = 1$ ;  $S_1 = 0 \oplus 0 = 0$ ;  $S_2 = 1 \oplus 1 = 0$ ;  $S_3 = 1 \oplus 1 = 0$ .
7. Check:  $1011 = 11, 0110 = 6, 11 + 6 = 17 = 10001_2 \rightarrow C_4 = 1, S = 0001 \checkmark$ .

**Answer:**  $C_1 C_2 C_3 C_4 = 0 1 1 1$ ; Sum = 0001 with carry-out 1.

### Solved example 2 (write the answer like this):

Explain generate and propagate and why a CLA is faster than a ripple-carry adder. (5 marks)

1.  $G_i = A_i B_i$ : bit  $i$  produces a carry by itself.  $P_i = A_i \oplus B_i$ : bit  $i$  passes an incoming carry to the next bit.
2.  $C_{(i+1)} = G_i + P_i C_i$ ; substituting repeatedly gives each carry in terms of  $G, P$  and  $C_0$  only (write  $C_1$  to  $C_4$ ).
3. All carries are therefore computed in parallel by two-level AND-OR logic, after one level for  $G$  and  $P$ .
4. Delay  $\approx$  constant (about 4 gate delays for a 4-bit block) instead of  $2n$  for ripple; cost is more gates (area, fan-in).

**Answer:** G/P meaning + equations + parallel carries  $\rightarrow$  speed, at the cost of area.

**Galti mat karna:**  $G$  aur  $P$  ulta mat karo:  $G = \text{AND (generate)}$ ,  $P = \text{XOR (propagate)}$ . Kuch books  $P = A + B$  likhti hain, wo carry ke liye chalta hai par Sum =  $P \oplus C$  ke liye nahi; course XOR use karta hai. Aur CLA "kam gates" wala nahi hai: zyada gates, kam delay.

**Trick:** "**G = Ghar ka carry** (khud banaya), **P = Pass karo.**" Ck ka pattern: " $G + P \cdot (\text{pichla carry})$ " ko khol do.

**Ek line mein:** CLA:  $G = AB$ ,  $P = A \oplus B$ ,  $C(i+1) = G + PC$  ko kholke saare carries ek saath; area zyada, delay kam.

**Exam mein is unit se kya aata hai → kaise solve karein**

| Question type                                  | Method (steps)                                                                                                                                                                                       |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MUX: select lines / which input / output value | 1. $2^n = \text{inputs} \rightarrow n = \text{selects}$ (8:1 → 3). 2. Select bits ka decimal = input number. 3. $Y =$ us input ki value.                                                             |
| 2:1 MUXes needed for a big MUX                 | $2^n:1 \rightarrow 2^n - 1$ (16:1 → 15). Or count levels: $8 + 4 + 2 + 1$ .                                                                                                                          |
| Implement F with a 4:1 MUX (A, B on selects)   | 1. Truth table. 2. $AB = 00, 01, 10, 11$ ki jodiyen. 3. Har jodi ( $C = 0, C = 1$ ): $00 \rightarrow 0, 11 \rightarrow 1, 01 \rightarrow C, 10 \rightarrow C'$ . 4. I0 to I3 likho. 5. Ek row check. |
| Draw 4:1 MUX with gates                        | 4 three-input ANDs (1k + select minterm) → 4-input OR; 2 NOTs for $S_1', S_0'$ ; select table.                                                                                                       |
| DEMUX outputs / equations                      | 1. Select ka decimal = output number. 2. Woh output = D, baaki 0. 3. $Y_k = D \cdot (\text{select minterm } k)$ .                                                                                    |
| Decoder: outputs, AND count, which line        | 1. $n$ -to- $2^n$ : outputs $2^n$ , ANDs $2^n$ , NOTs $n$ . 2. Input ka decimal $k \rightarrow$ sirf $D_k = 1$ (enable = 1). 3. One-hot word likho.                                                  |
| Implement F (or full adder) with decoder + OR  | 1. Inputs decoder pe. 2. F ki $\Sigma m$ list. 3. OR gate pe un $D_k$ ko jodo. FA: Sum = $D_1 + D_2 + D_4 + D_7$ , Cout = $D_3 + D_5 + D_6 + D_7$ .                                                  |
| Half / full adder design (5 to 6 marks)        | 1. Truth table. 2. Sum K-map (checkerboard → XOR), Cout K-map (3 pairs → majority). 3. Equations. 4. Gate circuit. 5. FA = 2 HA + OR diagram, why OR works.                                          |
| RCA result trace                               | Bit 0 se shuru: $A + B + C \rightarrow$ Sum bit, carry agle bit mein. Last carry = Cout. Decimal se check.                                                                                           |
| RCA delay / max frequency                      | 1. Delay = $n \times (\text{gate delays per stage, usually } 2) \times t_{\text{gate}}$ . 2. $f = 1 / \text{delay}$ . 3. Reason: serial carry, $\propto n$ .                                         |
| CLA equations / carries for given A, B         | 1. $G_i = A_i B_i$ , $P_i = A_i \oplus B_i$ per bit. 2. $C(i+1) = G_i + P_i C_i$ , expand $C_1$ to $C_4$ . 3. Values daalo. 4. $S_i = P_i \oplus C_i$ . 5. Decimal check.                            |
| ALU: subtract, control codes, Zero             | 1. Sub = Binvert 1 + CarryIn 1 ( $A + B' + 1$ ). 2. Codes: 0000 AND, 0001 OR, 0010 add, 0110 sub, 0111 slt. 3. Zero = NOR of result bits.                                                            |
| Which block? (MUX vs DEMUX vs decoder)         | Many → one = MUX; one data → one of many = DEMUX; code → one line high (no data) = decoder.                                                                                                          |

# Unit 4 · Number Systems & Two's Complement

**Is unit ki kahani:** Unit 3 mein humne adder banaya jo bits jodta hai. Par bits ka *matlab* kya hai? 1011 ek number hai, par kaunsa: 11 ya -5? Ye unit sikhati hai ki numbers binary aur hex mein kaise likhte hain, negative numbers kaise likhte hain (sign-magnitude, 1's complement, 2's complement), chhote number ko bade register mein kaise daalte hain (sign extension), add/subtract kaise hota hai, aur kab answer box mein fit nahi hota (overflow). Agla unit (5) batayega ki ye bits *yaad* kaise rakhe jaate hain (latches, flip-flops).

## 4.1 Number systems: binary, decimal, hex conversions

**Pehle socho:** Tum roz decimal use karte ho kyunki insaan ke 10 ungliyan hain: digits 0 se 9. Computer ke andar har wire pe sirf do haalat hain: current hai (1) ya nahi hai (0), jaise switch ON/OFF. Isliye computer **base 2** (binary) mein ginta hai. Lambi binary strings padhna mushkil hai (1011 0101 1100 0011), isliye insaan unhe chhota karke **hex** (base 16) mein likhte hain. Teen systems ek hi idea pe chalte hain: **har position ki ek "keemat" (weight) hoti hai.**

**Definition:** A **positional number system** with base (radix)  $r$  represents a value as the sum of each digit multiplied by  $r$  raised to the power of its position, positions being numbered 0, 1, 2 from the right. Binary uses base 2 (digits 0, 1), decimal base 10 (0 to 9) and hexadecimal base 16 (0 to 9, A to F).

**Mushkil shabd, aasaan matlab:**

### base / radix

Kitne alag digits hain, aur har position pichhli se kitni guna keemti hai. Decimal: base 10. Binary: base 2. Hex: base 16.

### place value / weight

Kisi position ki keemat. Decimal mein right se: 1, 10, 100, 1000. Binary mein: 1, 2, 4, 8, 16, 32 (har baar  $\times 2$ ).

### bit

Ek binary digit, 0 ya 1. "101" mein 3 bits hain.

### nibble / byte

4 bits = nibble (ek hex digit). 8 bits = byte (do hex digits). 1011 0101 ek byte hai.

### MSB / LSB

Most Significant Bit = sabse left wala bit (sabse bada weight). Least Significant Bit = sabse right wala (weight 1). 101101 mein MSB ka weight 32, LSB ka 1.

### subscript 2 10 16, prefix 0x

Number kis base mein hai ye batata hai.  $101_2 = \text{five}$ ,  $101_{10} = \text{one hundred one}$ .  $0x2A$  ka matlab hex 2A.

### remainder

Divide karne ke baad jo bachta hai.  $45 \div 2 = 22$ , remainder 1 (kyunki  $22 \times 2 = 44$ ,  $45 - 44 = 1$ ).

### Kaise kaam karta hai:

1. **Decimal se shuru (place value):**  $237$  ka matlab hai 2 sau + 3 das + 7 ek =  $2 \times 100 + 3 \times 10 + 7 \times 1 = 200 + 30 + 7 = 237$ . Har position pichhli se 10 guna keemti hai.
2. **Binary mein bilkul wahi, bas  $\times 2$ :** weights right se 1, 2, 4, 8, 16, 32, 64, 128.  $101101_2 = 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 32 + 8 + 4 + 1 = 45$ . Rule: jahan bit 1 hai, us position ka weight jodo; 0 wali positions chhod do.
3. **Decimal  $\rightarrow$  binary, method 1 (baar baar 2 se divide):** number ko 2 se divide karo, remainder (0 ya 1) likho, quotient ko phir 2 se divide karo, jab tak quotient 0 na ho jaaye. Phir remainders ko neeche se upar padho. 45 ke liye:

| Division    | Quotient | Remainder |
|-------------|----------|-----------|
| $45 \div 2$ | 22       | 1 (LSB)   |
| $22 \div 2$ | 11       | 0         |
| $11 \div 2$ | 5        | 1         |
| $5 \div 2$  | 2        | 1         |
| $2 \div 2$  | 1        | 0         |
| $1 \div 2$  | 0        | 1 (MSB)   |

Neeche se upar: 1 0 1 1 0 1  $\rightarrow$  **101101**. Check:  $32 + 8 + 4 + 1 = 45$ . Ye kyun kaam karta hai:  $237 \div 10$  karo to remainder 7 aata hai, jo 237 ka last digit hai. Waise hi  $\div 2$  ka remainder binary ka last bit (LSB) hai. Isliye pehla remainder sabse right mein jaata hai.

4. **Decimal  $\rightarrow$  binary, method 2 (powers of 2 ghatao):** sabse bada power of 2 dhoondo jo number se chhota ya barabar ho, use ghatao, us position pe 1 likho; baaki se phir yahi karo. 45:  $32 \leq 45 \rightarrow 45 - 32 = 13$ ;  $16 > 13 \rightarrow 0$ ;  $8 \leq 13 \rightarrow 13 - 8 = 5$ ;  $4 \leq 5 \rightarrow 5 - 4 = 1$ ;  $2 > 1 \rightarrow 0$ ;  $1 \leq 1 \rightarrow 1 - 1 = 0$ . Positions 32, 16, 8, 4, 2, 1  $\rightarrow$  1 0 1 1 0 1 = **101101**. Same answer.
5. **Binary  $\rightarrow$  hex (4 ke group):**  $16 = 2^4$ , isliye ek hex digit = exactly 4 bits. Bits ko right se 4-4 ke group mein todo, har group ko hex digit mein badlo.  $10110101 \rightarrow 1011 \mid 0101 \rightarrow B \mid 5 \rightarrow$  **0xB5**. Agar bits 4 ke multiple nahi hain to left mein 0 jodo:  $1101101$  (7 bits)  $\rightarrow 0110 \mid 1101 \rightarrow$  **0x6D**.
6. **Hex  $\rightarrow$  binary:** ulta karo, har hex digit ko 4 bits mein likho.  $0x2A \rightarrow 2 = 0010$ ,  $A = 1010 \rightarrow 0010 \ 1010$ .
7. **Hex  $\rightarrow$  decimal:** hex weights right se 1, 16, 256, 4096.  $0xB5 = 11 \times 16 + 5 \times 1 = 176 + 5 = 181$ .  $0x3E8 = 3 \times 256 + 14 \times 16 + 8 \times 1 = 768 + 224 + 8 = 1000$ .  $0x3C = 3 \times 16 + 12 = 48 + 12 = 60$ .
8. **Decimal  $\rightarrow$  hex:** 16 se divide karo, remainders neeche se upar.  $181 \div 16 = 11$  remainder 5 ( $11 \times 16 = 176$ );  $11 \div 16 = 0$  remainder 11 = B  $\rightarrow$  **B5**.

9. **Octal (base 8):**  $8 = 2^3$ , isliye ek octal digit = 3 bits.  $(1217)_8 \rightarrow 001\ 010\ 001\ 111 \rightarrow$  ab 4 ke group banao right se:  $0010\ 1000\ 1111 \rightarrow 0x28F = 2 \times 256 + 8 \times 16 + 15 = 512 + 128 + 15 = 655$ .

10. **Fraction (binary point ke baad):** weights  $1/2, 1/4, 1/8$  (0.5, 0.25, 0.125).  $0.101_2 = 0.5 + 0 + 0.125 = 0.625$ . Uلتا (0.625  $\rightarrow$  binary):  $\times 2$  karte jao, integer part likho:  $0.625 \times 2 = 1.25 \rightarrow 1$ ;  $0.25 \times 2 = 0.5 \rightarrow 0$ ;  $0.5 \times 2 = 1.0 \rightarrow 1 \rightarrow 0.101$  (yahan upar se neeche padho).

11. **n bits mein kitna aata hai (unsigned):** 0 se  $2^n - 1$ . 4 bits: 0 se 15. 8 bits: 0 se 255. 1000 tak ke liye kitne bits?  $2^9 = 512$  (kam),  $2^{10} = 1024$  (kaafi)  $\rightarrow 10$  bits.

| Hex    | 0    | 1    | 2      | 3      | 4      | 5      | 6      | 7      |
|--------|------|------|--------|--------|--------|--------|--------|--------|
| Binary | 0000 | 0001 | 0010   | 0011   | 0100   | 0101   | 0110   | 0111   |
| Hex    | 8    | 9    | A (10) | B (11) | C (12) | D (13) | E (14) | F (15) |
| Binary | 1000 | 1001 | 1010   | 1011   | 1100   | 1101   | 1110   | 1111   |

Powers of 2 ratti lo: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024.  $2^{10} = 1024 \approx 1\text{ K}$ ,  $2^{20} \approx 1\text{ M}$ ,  $2^{30} \approx 1\text{ G}$ .

Positional notation:  $101101_2 = 45_{10}$

| bit      | 1  | 0  | 1 | 1 | 0 | 1 | value |
|----------|----|----|---|---|---|---|-------|
| weight   | 32 | 16 | 8 | 4 | 2 | 1 |       |
| contrib. | 32 | 0  | 8 | 4 | 0 | 1 | = 45  |

#### Positional weights

How to draw: Write the bits, the powers of 2 underneath, and add the weights where the bit is 1.

**Diagram kaise padhein:** Upar wali row mein bits 1 0 1 1 0 1 hain. Beech wali row mein har bit ka weight (32, 16, 8, 4, 2, 1). Neeche wali row "contribution" hai: bit 1 hai to weight aata hai, bit 0 hai to 0. Highlighted columns (32, 8, 4, 1) jodo:  $32 + 8 + 4 + 1 = 45$ .

**Formula:** Value =  $\sum (\text{digit} \times \text{base}^{\text{position}})$ , position 0 = rightmost.

Unsigned n bits: 0 to  $2^n - 1$ . Bits needed for value N: smallest n with  $2^n - 1 \geq N$ .

1 hex digit = 4 bits, 1 octal digit = 3 bits.

#### Solved example 1 (exam favourite, 5 marks):

Convert (a)  $157_{10}$  to binary, octal and hex, (b)  $0x3E8$  to decimal, (c)  $10110.011_2$  to decimal. Show working.

1. (a) Repeated  $\div 2$ :  $157 \div 2 = 78\text{ r }1$ ;  $78 \div 2 = 39\text{ r }0$ ;  $39 \div 2 = 19\text{ r }1$ ;  $19 \div 2 = 9\text{ r }1$ ;  $9 \div 2 = 4\text{ r }1$ ;  $4 \div 2 = 2\text{ r }0$ ;  $2 \div 2 = 1\text{ r }0$ ;  $1 \div 2 = 0\text{ r }1$ .

2. Remainders neeche se upar:  $1\ 0\ 0\ 1\ 1\ 1\ 0\ 1 \rightarrow 157_{10} = 1001\ 1101_2$ . Check:  $128 + 16 + 8 + 4 + 1 = 157$ .

3. Octal: right se 3 ke group:  $10\ | \ 011\ | \ 101 \rightarrow 010\ | \ 011\ | \ 101 \rightarrow 2\ | \ 3\ | \ 5 \rightarrow 235_8$ . Check:  $2 \times 64 + 3 \times 8 + 5 = 128 + 24 + 5 = 157$ .

4. Hex: right se 4 ke group:  $1001 | 1101 \rightarrow 9 | D \rightarrow 0x9D$ . Check:  $9 \times 16 + 13 = 144 + 13 = 157$ .
5. (b)  $0x3E8 = 3 \times 256 + 14 \times 16 + 8 \times 1 = 768 + 224 + 8 = 1000$ .
6. (c)  $10110.011$ : left part  $16 + 0 + 4 + 2 + 0 = 22$ ; right part  $0 \times 0.5 + 1 \times 0.25 + 1 \times 0.125 = 0.375 \rightarrow 22.375$ .

Answer:  $157 = 10011101_2 = 235_8 = 0x9D$ ;  $0x3E8 = 1000$ ;  $10110.011_2 = 22.375$ .

### Solved example 2 (MCQ / NAT type):

(i) Convert  $181_{10}$  to hex. (ii) How many bits are needed to store unsigned numbers up to 1000?

- (i)  $181 \div 16 = 11$  remainder 5 ( $16 \times 11 = 176$ ,  $181 - 176 = 5$ ).  $11 \div 16 = 0$  remainder 11 = B.
- Neeche se upar: B, 5  $\rightarrow 0xB5$ . Check via binary:  $10110101 = 128 + 32 + 16 + 4 + 1 = 181$ .
- (ii) 9 bits max =  $2^9 - 1 = 511 < 1000$  (kam). 10 bits max =  $2^{10} - 1 = 1023 \geq 1000$  (kaafi).

Answer: (i)  $0xB5$  (ii) 10 bits.

**Galti mat karna:** (1) Remainders upar se neeche padhna: 42 ke remainders upar se neeche 0 1 0 1 0 1 hain; aise padhoge to  $010101 = 21$  aayega (galat). Sahi: neeche se upar =  $101010 = 42$ . (2) Hex group left se banana:  $1101101$  ko  $1101 | 101$  karna galat hai, group hamesha **right se**. (3)  $0x10$  ko "das" samajhna:  $0x10 = 1 \times 16 + 0 = 16$ .

**Trick:** 4 bits ke weights "**8 4 2 1**" ratt lo. Koi bhi nibble 2 second mein:  $1011 = 8 + 2 + 1 = 11 = B$ .

**Ek line mein:** Har position ka weight base ki power hai; binary  $\rightarrow$  decimal = 1 wale weights jodo, decimal  $\rightarrow$  binary =  $\div 2$  aur remainders neeche se upar, binary  $\leftrightarrow$  hex = right se 4-4 bits ka group.

## 4.2 Signed representations overview

**Pehle socho:** Kaagaz pe  $-5$  likhne ke liye hum "-" sign lagate hain. Par computer ke paas sirf 0 aur 1 hain, koi "-" nahi. To negative number kaise likhein? Teen ideas try kiye gaye: (1) ek bit ko sign bana do (**sign-magnitude**), (2) saare bits ulat do (**1's complement**), (3) ulat ke 1 jodo (**2's complement**). Pehle do mein problems hain, teesra jeet gaya. Teenon ko ek hi number pe dekhenge.

**Definition:** A **signed number representation** encodes both positive and negative integers using bits. In **sign-magnitude** the MSB is the sign ( $0 = +$ ,  $1 = -$ ) and the remaining bits give the magnitude; in **one's complement** a negative number is formed by inverting every bit of the positive number; in **two's complement** it is formed by inverting every bit and adding 1.

**Mushkil shabd, aasaan matlab:**

**sign bit**

Sabse left wala bit (MSB). 0 = positive, 1 = negative. Teeno systems mein yahi rule hai.

### magnitude

Sign ke bina size. -5 ka magnitude 5.

### invert / complement

Har bit ulta karo:  $0 \rightarrow 1, 1 \rightarrow 0$ . 0101 ka invert = 1010.

### two zeros

Zero ke do alag patterns hona (+0 aur -0). Bekaar hai: ek pattern waste, aur "kya ye zero hai?" check karne ke liye do comparisons.

### end-around carry

1's complement addition mein jo carry left se bahar nikle, use wapas right wale bit mein jodna padta hai. Extra kaam.

### range

Sabse chhota se sabse bada number jo n bits mein likh sakte ho.

### Kaise kaam karta hai (same number, teeno tarike, 4 bits):

1. Pehle +5 likho: 0101 (teeno systems mein positive number same hota hai).
2. **Sign-magnitude -5**: sirf sign bit 1 kar do, magnitude 101 waisa hi  $\rightarrow$  **1101**.
3. **1's complement -5**: 0101 ke saare bits ulto  $\rightarrow$  **1010**.
4. **2's complement -5**: 1010 mein 1 jodo  $\rightarrow$  **1011**.
5. 8-bit mein  $\pm 18$  (class slide): +18 = 0001 0010. Sign-magnitude -18 = **1001 0010**. 1's complement -18 = **1110 1101**. 2's complement -18 = **1110 1110**.
6. **Two zeros problem**: sign-magnitude mein 0000 (+0) aur 1000 (-0). 1's complement mein 0000 (+0) aur 1111 (-0). 2's complement mein sirf 0000 (1111 ka matlab -1 hai).
7. **Addition problem (sign-magnitude)**:  $5 + (-3)$  seedha jodo:  $0101 + 1011 = 1\ 0000 \rightarrow 0000 = 0$ . Galat (answer 2 hona chahiye). Isliye sign-magnitude ko jodne se pehle signs compare karne wala alag circuit chahiye.
8. **Addition (1's complement)**:  $5 + (-3) = 0101 + 1100 = 1\ 0001$ . Bahar nikla carry wapas jodo:  $0001 + 1 = 0010 = 2$ . Sahi, par extra "end-around" step lagta hai.
9. **Addition (2's complement)**:  $5 + (-3) = 0101 + 1101 = 1\ 0010 \rightarrow$  carry chhod do  $\rightarrow 0010 = 2$ . Seedha sahi.  
Isliye har CPU 2's complement use karta hai.

| Bits | Unsigned | Sign-mag | 1's comp | 2's comp | Bits | Unsigned | Sign-mag | 1's comp | 2's comp |
|------|----------|----------|----------|----------|------|----------|----------|----------|----------|
| 0000 | 0        | +0       | +0       | 0        | 1000 | 8        | -0       | -7       | -8       |
| 0001 | 1        | 1        | 1        | 1        | 1001 | 9        | -1       | -6       | -7       |
| 0010 | 2        | 2        | 2        | 2        | 1010 | 10       | -2       | -5       | -6       |
| 0011 | 3        | 3        | 3        | 3        | 1011 | 11       | -3       | -4       | -5       |
| 0100 | 4        | 4        | 4        | 4        | 1100 | 12       | -4       | -3       | -4       |
| 0101 | 5        | 5        | 5        | 5        | 1101 | 13       | -5       | -2       | -3       |
| 0110 | 6        | 6        | 6        | 6        | 1110 | 14       | -6       | -1       | -2       |
| 0111 | 7        | 7        | 7        | 7        | 1111 | 15       | -7       | -0       | -1       |

Table mein dekho: left half (MSB = 0) mein sab systems same hain. Right half (MSB = 1) mein har system alag padhta hai. Bits ka khud koi matlab nahi; instruction decide karti hai ki unsigned padhna hai ya signed (jaise MIPS add vs addu, lb vs lbu).

**Formula:** Sign-magnitude and 1's complement range:  $-(2^{n-1} - 1)$  to  $+(2^{n-1} - 1)$  (two zeros). 2's complement range:  $-2^{n-1}$  to  $+(2^{n-1} - 1)$  (one zero).

1's complement of  $x = (2^n - 1) - x$ ; 2's complement of  $x = 2^n - x$ .

4 bits: S-M and 1's: -7 to +7; 2's: -8 to +7. 8 bits: S-M and 1's: -127 to +127; 2's: -128 to +127.

### Solved example (exam 5 marks):

Represent -23 in 8-bit sign-magnitude, one's complement and two's complement. Compare the three representations.

1. +23 binary:  $23 = 16 + 4 + 2 + 1 \rightarrow 0001\ 0111$ .
2. Sign-magnitude: MSB 1, baaki magnitude 001 0111  $\rightarrow 1001\ 0111$ .
3. One's complement: 0001 0111 ka har bit ulto  $\rightarrow 1110\ 1000$ .
4. Two's complement:  $1110\ 1000 + 1 = 1110\ 1001$ . Check:  $-128 + 64 + 32 + 8 + 1 = -23$ .
5. Comparison (likhne ka tareeka): "Sign-magnitude and one's complement both have two zeros (+0 and -0) and a symmetric range -127 to +127 in 8 bits. Sign-magnitude addition needs sign-comparison logic; one's complement needs an end-around carry. Two's complement has a single zero, range -128 to +127, and the same adder performs both addition and subtraction, so it is used in all modern CPUs. Sign-magnitude survives only as the sign bit of floating-point numbers."

**Answer:** S-M 1001 0111; 1's 1110 1000; 2's 1110 1001.

**Galti mat karna:** 1000 0000 ko 2's complement mein "-0" mat bolo. 2's complement mein ye -128 hai. "-0" sirf sign-magnitude (1000 0000) aur 1's complement (1111 1111) mein hota hai.

**Trick:** "S-M: sign flip. 1's: sab flip. 2's: sab flip + 1."

**Ek line mein:** Negative likhne ke teen tarike hain; sign-magnitude aur 1's complement mein do zero aur addition ki pareshani hai, 2's complement mein ek zero aur seedha addition, isliye wahi use hota hai.

### 4.3 Two's complement deep dive: negation and range

**Pehle socho:** Gaadi ka purana 3-digit odometer (km meter) socho, jo 000 pe hai. Use ek km *peeche* chalao to wo **999** dikhayega. To 3 digit ki duniya mein "999" ka matlab hi "-1" ho gaya: kyunki  $999 + 1 = 1000$ , aur 3 digit mein 1000 ka "1" gir jaata hai, bachta hai 000. 2's complement bilkul yahi hai, binary odometer pe: 4-bit 0000 se ek peeche = **1111** = -1. Isliye addition apne aap sahi aata hai.

**Definition:** In **n-bit two's complement**, the MSB has weight  $-2^{n-1}$  and every other bit has its normal positive weight. The negative of  $x$  is  $2^n - x$ , which is obtained by inverting all bits of  $x$  and adding 1. The range is  $-2^{n-1}$  to  $2^{n-1} - 1$ .

**Mushkil shabd, aasaan matlab:**

#### wrap-around

Sabse bade pattern ke baad wapas sabse chhote pe aa jaana. 4-bit mein  $1111 + 1 = 0000$  (carry bahar gir jaata hai).

#### $2^n$ (modulus)

$n$  bits mein total patterns. 4 bits:  $2^4 = 16$  patterns (0000 se 1111). Wheel pe 16 jagah hain; 16 kadam chalo to wapas wahiin.

#### negate

Number ka ulta sign wala version banana:  $+18 \rightarrow -18$ , ya  $-18 \rightarrow +18$ .

#### MSB ka negative weight

8-bit mein MSB ka weight +128 nahi, **-128** hai.  $1000\ 0001 = -128 + 1 = -127$ .

#### asymmetric range

Negative side ek number zyada: 4-bit mein -8 hai par +8 nahi.

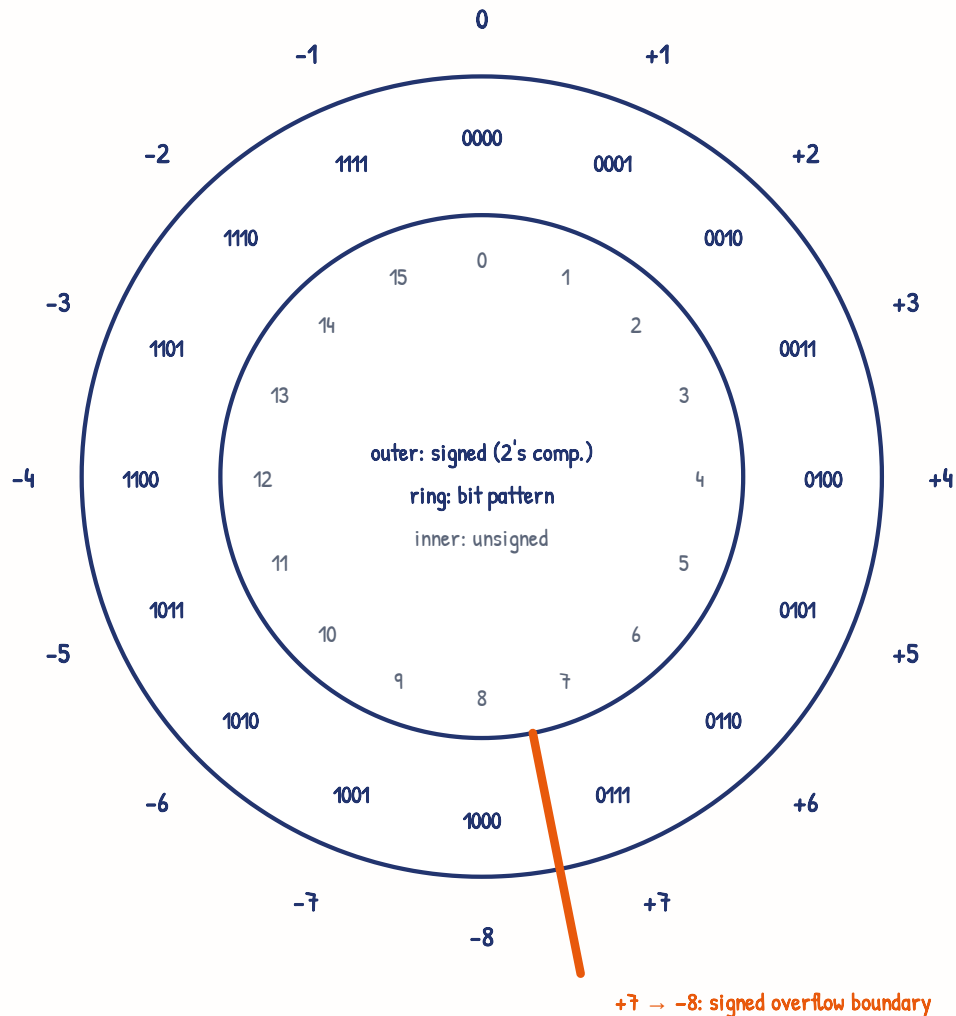
**Kaise kaam karta hai:**

- Wheel idea:** 4 bits ke 16 patterns ek ghadi ki tarah gol ghare mein lagao: 0000 upar, clockwise 0001, 0010 se 1111 tak. 1111 ke baad phir 0000. Har kadam = +1.
- 0000 se ek kadam ulta (anticlockwise) = 1111. To 1111 = -1. Do kadam ulta = 1110 = -2. Teen ulta = 1101 = -3. **Negative numbers bas "zero se peeche" wale positions hain.**
- Formula kyun  $-x = 2^n - x$ :** 16 kadam chalo to wapas wahiin aate ho, isliye "3 kadam peeche" = "16 - 3 = 13 kadam aage". 13 = 1101. Dekho, upar humne -3 = 1101 hi paaya. To  **$-x$  ka pattern =  $(2^n - x)$  ka binary.**
- Invert + 1 kyun kaam karta hai:**  $x$  aur uska invert  $x'$  jodo to har column mein  $1 + 0 = 1$  aata hai:  $0101 + 1010 = 1111 = 2^n - 1$  (koi carry nahi). Matlab  $x' = (2^n - 1) - x$ . Ab 1 jodo:  $x' + 1 = 2^n - x = -x$ . Bas, isliye

**Invert karo, 1 jodo.**

5. **Negation steps (8-bit, +18 → -18):** (a) +18 = 0001 0010. (b) Invert: 1110 1101. (c) 1 jodo: 1110 1101 + 1 → right bit 1 + 1 = 10, likho 0 carry 1; agla bit 0 + 1 = 1, carry khatam → **1110 1110**.
6. **Shortcut (copy till first 1):** right se chalo, bits **copy** karte jao jab tak pehla 1 na mile (us 1 ko bhi copy karo), uske baad baaki saare bits **ulto**. +18 = 0001 0010: right ka 0 copy → 0; agla 1 (pehla 1) copy → 1; baaki 0001 00 ulto → 1110 11. Joda: **1110 1110**. Same answer, addition ki zarurat nahi. Ek aur: +40 = 0010 1000 → right ke 000 copy, phir pehla 1 copy → "1000"; baaki 0010 ulto → 1101 → **1101 1000** = -40.
7. **Negative pattern ko padhna (method 1, weights):** 1110 1110 = -128 + 64 + 32 + 0 + 8 + 4 + 2 + 0 = -128 + 110 = **-18**.
8. **Method 2 (wapas negate):** MSB 1 hai to negative hai. Invert + 1 karo magnitude milega: 1110 1110 → 0001 0001 → +1 → 0001 0010 = 18. To value -18. (Negate dono taraf kaam karta hai.)
9. **Range nikalna, n = 4:** 16 patterns. MSB = 0 wale 8 patterns: 0000 se 0111 = 0 se +7 (zero ne ek jagah le li, isliye positive sirf 7 tak). MSB = 1 wale 8 patterns: 1000 se 1111 = -8 se -1. Range **-8 to +7**. Formula:  $-2^n = -8$  to  $2^n - 1 = 7$ .
10. **Range, n = 8:**  $-2^8 = -128$  to  $2^8 - 1 = +127$ . Sabse negative 1000 0000 = -128, sabse positive 0111 1111 = +127, -1 = 1111 1111. (16-bit: -32768 to +32767.)
11. **Sabse negative ka koi jodidaar nahi:** -8 = 1000 ko negate karo: invert 0111, +1 → 1000. Wapas -8! Kyunki +8 4 bits mein fit hi nahi hota.

1111 → 0000 (-1 → 0): unsigned carry-out



#### 4-bit number wheel (unsigned, pattern, signed)

How to draw: Circle with 16 positions; 0000 at the top, increase clockwise; label signed values outside and mark the +7/-8 boundary.

**Diagram kaise padhein:** Teen ghare hain. Beech wali ring mein bit pattern, andar ki taraf (halka) unsigned value, bahar signed (2's complement) value. 0000 sabse upar hai, clockwise chalo to +1 hota hai. 0111 (+7) ke baad 1000 (-8) aata hai; ye highlighted line **signed overflow ki deewar** hai. 1111 (-1) se ek aage 0000 (0): ye unsigned ka carry-out point hai (15 → 0). Trace: 1101 dhoondo, bahar likha hai -3, andar 13;  $13 + 3 = 16 = 2^4$ .

**Formula:**  $-x = x' + 1 = 2^n - x$  · Range =  $-2^{n-1}$  to  $2^{n-1} - 1$  · MSB weight =  $-2^{n-1}$

$n = 4$ : -8 to 7 ·  $n = 6$ : -32 to 31 ·  $n = 8$ : -128 to 127 ·  $n = 16$ : -32768 to 32767. All ones = -1.

#### Solved example 1 (class exit ticket):

Write -45 in 8-bit two's complement and verify.

1. +45:  $45 = 32 + 8 + 4 + 1 \rightarrow 0010\ 1101$ .
2. Invert: 1101 0010.
3. +1: right bit  $0 + 1 = 1$ , koi carry nahi  $\rightarrow 1101\ 0011$ .

4. Shortcut se check: 0010 1101 ka right bit hi pehla 1 hai, use copy karo; baaki 0010 110 ulto  $\rightarrow$  1101 001  $\rightarrow$  1101 0011. Same.
5. Verify weights se:  $-128 + 64 + 16 + 2 + 1 = -128 + 83 = -45$ .

**Answer:  $-45 = 1101\ 0011$ .**

### Solved example 2 (MCQ/NAT type):

(i)  $0xB7$  as an 8-bit signed integer? (ii) Decimal value of 1011 0100 (8-bit two's complement)? (iii) Range of a 6-bit two's complement number? (iv) Show  $+18 + (-18) = 0$ .

- (i)  $0xB7 = 1011\ 0111$ . MSB 1  $\rightarrow$  negative. Weights:  $-128 + 32 + 16 + 4 + 2 + 1 = -128 + 55 = -73$ . (Unsigned padho to 183.)
- (ii)  $1011\ 0100 = -128 + 32 + 16 + 4 = -128 + 52 = -76$ .
- (iii)  $n = 6$ :  $-2^5 = -32$  to  $2^5 - 1 = 31 \rightarrow -32$  to  $+31$ .
- (iv)  $0001\ 0010 + 1110\ 1110$ : bit 0:  $0 + 0 = 0$ . Bit 1:  $1 + 1 = 10 \rightarrow 0$ , carry 1. Bits 2 se 7: har column mein ek 1 hai, plus aaya hua carry 1 =  $10 \rightarrow 0$ , carry 1. Result 1 0000 0000. 9th bit (carry) 8-bit box mein fit nahi, gir jaata hai  $\rightarrow 0000\ 0000 = 0$ .

**Answer:  $-73$ ;  $-76$ ;  $-32$  to  $+31$ ; sum = 0 (carry discarded).**

**Galti mat karna:** (1) "+1" bhoool jaana: sirf invert karoge to 1's complement banega, 2's nahi ( $-45 = 1101\ 0010$  galat). (2) Range ko symmetric likhna ( $-127$  to  $+127$ ): ye sign-magnitude ka hai; 2's complement mein  $-128$  to  $+127$ . (3) Shortcut mein left se copy shuru karna: copy hamesha **right** se.

**Trick: "Odometer peeche chalo."**  $-1$  hamesha all ones (1111 1111). Range: "negative side pe ek extra, kyunki zero positive side pe baitha hai."

**Ek line mein:** 2's complement mein  $-x = \text{invert} + 1 = 2^n - x$ , MSB ka weight  $-2^{n-1}$  hai, aur range  $-2^{n-1}$  se  $2^{n-1} - 1$ .

## 4.4 Sign extension

**Pehle socho:** Odometer wapas lo. 3-digit meter pe  $-1 = 999$ . Ab gaadi mein 6-digit meter lag gaya.  $-1$  ab kya dikhega? **999999**, na ki 000999 (wo to 999 km hai, positive!). Matlab chhote number ko bade box mein daalte waqt naye khaali digits ko "sign wale digit" se bharna padta hai. Binary mein bhi: 4-bit ya 16-bit number ko 8-bit ya 32-bit register mein daalna ho (MIPS mein 16-bit immediate ko 32-bit ALU tak), to naye upar wale bits mein MSB copy karo.

**Definition: Sign extension** increases the number of bits of a two's complement number without changing its value, by copying the sign bit (MSB) into every new higher-order bit. **Zero extension** fills the new bits with 0 and is correct only for unsigned numbers.

**Mushkil shabd, aasaan matlab:**

**extend (widen)**

Bits badhana: 4-bit → 8-bit, 16-bit → 32-bit.

**sign bit**

MSB. 1 = negative, 0 = positive (2's complement mein).

**immediate**

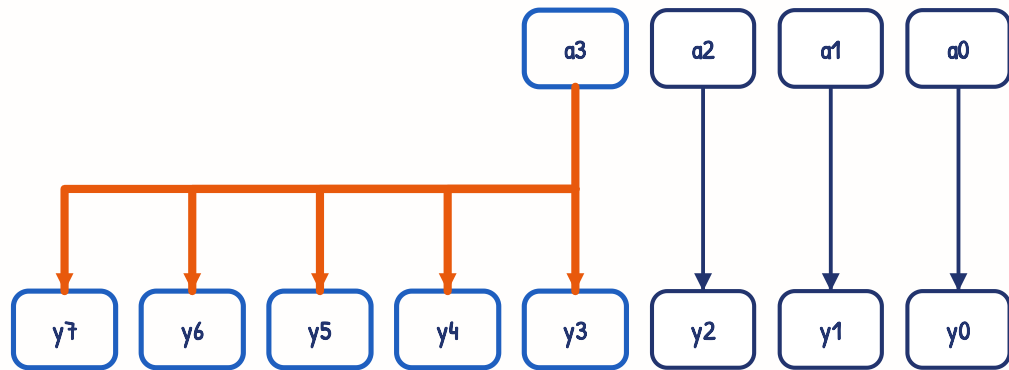
Instruction ke andar likha constant number. MIPS mein 16 bits ka hota hai, jaise addi \$t0, \$t0, -1 mein -1.

**lb / lbu**

MIPS "load byte" (signed, sign-extend karta hai) aur "load byte unsigned" (zero-extend karta hai).

**Kaise kaam karta hai:**

1. Purane number ka MSB dekho.
2. Naye jitne bits chahiye, sab mein wahi MSB likho, purane bits ko right mein waise hi rakho.
3. +5 (4-bit) = 0101, MSB 0 → 0000 0101 (8-bit). Value 5 hi.
4. -5 (4-bit) = 1011, MSB 1 → **1111 1011**. Check:  $-128 + 64 + 32 + 16 + 8 + 2 + 1 = -128 + 123 = -5$ . Value same.
5. Agar galti se 0 bhar diye: 0000 1011 =  $8 + 2 + 1 = +11$ . Bilkul galat.
6. **Kyun copy karne se value nahi badalti:** 4-bit mein MSB ka weight -8 tha. 8-bit mein bits 7 se 3 ab 1 1 1 1 hain:  $-128 + 64 + 32 + 16 + 8 = -8$ . Yaani naye saare 1s milke wahi -8 dete hain jo pehle akela MSB deta tha. Baaki bits (2 + 1 = 3) same → total  $-8 + 3 = -5$ .
7. **MIPS mein:** addi, lw, sw, slti aur branch offsets ka 16-bit immediate sign-extend hota hai. 0xFFFFB (-5) → 0xFFFFFFF B. 0x0004 → 0x00000004. Logical ones (andi, ori, xori) zero-extend karte hain. lb byte 0xF0 → 0xFFFFFFF0; lbu 0xF0 → 0x000000F0.



$y = \{4\{a[3]\}, a\}$ : the sign bit  $a_3$  is copied into every new upper bit.  $-5 = 1011 \rightarrow 1111\ 1011$ ;  $+5 = 0101 \rightarrow 0000\ 0101$ .

#### Sign extension wiring (4 $\rightarrow$ 8 bits)

How to draw: Draw the 4 input bits above the 8 output bits; straight wires for the low 4, fan the MSB to the top 5 output bits.

**Diagram kaise padhein:** Upar 4 input bits  $a_3\ a_2\ a_1\ a_0$ , neeche 8 output bits  $y_7$  se  $y_0$ . Neeche ke right 4 bits seedhe wire se upar wale se jude hain ( $y_3 = a_3$ ,  $y_2 = a_2$ ,  $y_1 = a_1$ ,  $y_0 = a_0$ ). Highlighted wire  $a_3$  (sign bit) se nikal ke  $y_7$ ,  $y_6$ ,  $y_5$ ,  $y_4$  sab mein jaata hai. Trace:  $a = 1011 \rightarrow a_3 = 1 \rightarrow y_7$  se  $y_4 = 1111 \rightarrow y = 1111\ 1011$ .

**Formula:** sign-extend  $n \rightarrow m$  bits: new bits = copies of the MSB. Verilog:  $y = \{4\{a[3]\}, a\}$ ;  
( $a[3]$  ko 4 baar repeat karo, phir  $a$ )

#### Solved example 1:

Sign-extend the 4-bit value 1010 to 8 bits and give its decimal value.

1. MSB = 1.
2. Naye 4 bits mein 1 bharo: 1111 1010.
3. Value: 4-bit 1010 =  $-8 + 2 = -6$ . 8-bit 1111 1010 =  $-128 + 64 + 32 + 16 + 8 + 2 = -128 + 122 = -6$ . Same.

**Answer:** 1111 1010 = -6.

#### Solved example 2 (MCQ + 4 marks):

(i) Sign-extend the 8-bit value 0xF4 to 16 bits. (ii) What is sign extension and why does the MIPS datapath need it? Give a positive and a negative 16-bit example.

1. (i)  $0xF4 = 1111\ 0100$ , MSB 1  $\rightarrow$  upper 8 bits all 1 =  $0xFF \rightarrow 0xFFFF4 (= -12)$ .
2. (ii) Model answer: "Sign extension widens a two's complement number by copying its sign bit into the new upper bits, so the value is unchanged. The MIPS ALU and registers are 32 bits wide but the immediate field of I-format instructions is only 16 bits, so offsets of lw/sw, constants of addi and branch offsets are sign-extended before use. Example:  $0x0004 (+4) \rightarrow 0x00000004$ ;  $0xFFFFC (-4) \rightarrow 0xFFFFFFF4$ ."

Answer:  $0xFF4$ ; definition + need + two examples as above.

**Galti mat karna:** Negative number mein 0 bharna (zero-padding) value badal deta hai. Aur ulta: unsigned data (lbu, andi, ori) ko sign-extend nahi karte, wahan 0 bharte hain.

**Trick:** "Signed  $\rightarrow$  MSB copy. Unsigned  $\rightarrow$  zero copy."

**Ek line mein:** 2's complement number ko chauda karte waqt naye bits mein MSB copy karo, value same rehti hai ( $-5: 1011 \rightarrow 1111\ 1011$ ).

## 4.5 Binary addition and subtraction

**Pehle socho:** School mein  $47 + 38$  aise karte the:  $7 + 8 = 15$ , likho 5, 1 carry upar. Binary mein bhi bilkul wahi, bas **2 hote hi carry** lagta hai (10 pe nahi). Aur subtraction ke liye alag circuit nahi chahiye:  $A - B = A + (-B)$ , aur  $-B$  to invert + 1 se ban jaata hai. Ek hi adder dono kaam karta hai.

**Definition:** Binary addition adds column by column from the LSB, producing a sum bit and a carry into the next column. In two's complement, subtraction  $A - B$  is performed as  $A + B' + 1$ . An **adder-subtractor** uses one XOR gate per bit of B and a control input K (or M) that also feeds the carry-in, so that  $K = 0$  gives  $A + B$  and  $K = 1$  gives  $A - B$ .

**Mushkil shabd, aasaan matlab:**

**carry**

Column ka sum 2 ya 3 ho to agle column mein jaane wala 1.  $1 + 1 = 10_2$ : likho 0, carry 1.

**carry-in ( $C_{in}$ ) / carry-out ( $C_{out}$ )**

Kisi column mein aane wala carry / usse nikalne wala carry. 4-bit adder ka final carry-out  $C_4$  kehlata hai.

**discard the carry**

Signed answer mein 4 bits se bahar nikla 5th bit ( $C_4$ ) phenk do. Wo answer ka hissa nahi.

**borrow**

Unsigned subtraction mein "udhaar".  $A + B' + 1$  method mein  $C_4 = 0$  ka matlab borrow hua ( $A < B$ ),  $C_4 = 1$  ka matlab borrow nahi ( $A \geq B$ ).

**XOR as controlled inverter**

$B \oplus 0 = B$  (waisa hi),  $B \oplus 1 = B'$  (ulta). Isliye ek control K se B ko ulta ya seedha kar sakte hain.

**Kaise kaam karta hai:**

1. **Chaar rules:**  $0 + 0 = 0$  (carry 0);  $0 + 1 = 1$  (carry 0);  $1 + 1 = 10$  (likho 0, carry 1);  $1 + 1 + 1 = 11$  (likho 1, carry 1).

2. Hamesha right column (bit 0) se shuru karo, har column mein A bit + B bit + aaya hua carry.

3. **Full example A (unsigned, 13 + 11 = 24):** 1101 + 1011.

|          | C4 (bahar) | bit 3 | bit 2 | bit 1 | bit 0 |
|----------|------------|-------|-------|-------|-------|
| carry in |            | 1     | 1     | 1     | 0     |
| A = 13   |            | 1     | 1     | 0     | 1     |
| B = 11   |            | 1     | 0     | 1     | 1     |
| Sum      | <b>1</b>   | 1     | 0     | 0     | 0     |

Bit 0:  $1 + 1 + 0 = 2 = 10 \rightarrow$  likho 0, carry 1. Bit 1:  $0 + 1 + 1 = 2 \rightarrow$  likho 0, carry 1. Bit 2:  $1 + 0 + 1 = 2 \rightarrow$  likho 0, carry 1. Bit 3:  $1 + 1 + 1 = 3 = 11 \rightarrow$  likho 1, carry 1 (C4). Result **1 1000** = 16 + 8 = 24. 4-bit box mein sirf 1000 = 8 bachta hai; C4 = 1 batata hai ki unsigned answer 15 se bada ho gaya.

4. **Subtraction rule:**  $A - B = A + (B \text{ ka invert}) + 1$ . Steps: B ko invert karo, A mein jodo, carry-in 1 rakho (ya pehle B ka 2's complement bana lo).

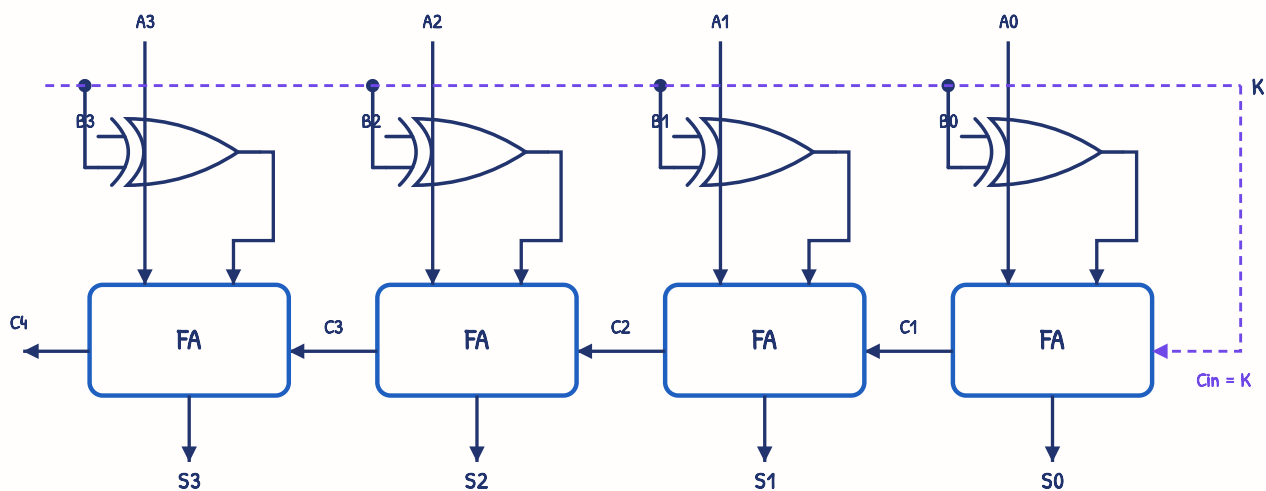
5. **Full example B (signed, 5 - 3, final carry discarded):**  $-3 = \text{invert}(0011) + 1 = 1100 + 1 = 1101$ . Ab 0101 + 1101.

|          | C4 (bahar)        | bit 3 | bit 2 | bit 1 | bit 0 |
|----------|-------------------|-------|-------|-------|-------|
| carry in |                   | 1     | 0     | 1     | 0     |
| A = +5   |                   | 0     | 1     | 0     | 1     |
| B = -3   |                   | 1     | 1     | 0     | 1     |
| Sum      | <b>1 (phenko)</b> | 0     | 0     | 1     | 0     |

Bit 0:  $1 + 1 = 10 \rightarrow$  0, carry 1. Bit 1:  $0 + 0 + 1 = 1 \rightarrow$  1, carry 0. Bit 2:  $1 + 1 + 0 = 10 \rightarrow$  0, carry 1. Bit 3:  $0 + 1 + 1 = 10 \rightarrow$  0, carry 1 (C4). Result 1 0010  $\rightarrow$  C4 discard  $\rightarrow$  **0010** = +2. Sahi ( $5 - 3 = 2$ ).

6. **Adder-subtractor circuit:** har B bit ek XOR se guzarta hai jiska doosra input K hai, aur K hi Cin bhi hai.  $K = 0$ : B seedha,  $Cin = 0 \rightarrow A + B$ .  $K = 1$ : B ulta,  $Cin = 1 \rightarrow A + B' + 1 = A - B$ .

7. **Unsigned borrow:**  $3 - 5$ :  $0011 + 1010 + 1 = 1110$ ,  $C4 = 0 \rightarrow$  borrow hua ( $3 < 5$ ). Signed padho to  $1110 = -2$ , sahi.



$K = 0$ : B passes,  $Cin = 0 \rightarrow A + B$ .  $K = 1$ : B inverted,  $Cin = 1 \rightarrow A + B' + 1 = A - B$ .

#### 4-bit adder-subtractor (Lab 5)

How to draw: Four FA boxes; an XOR in front of each B input with the shared control K; K also feeds Cin of FA0.

**Diagram kaise padhein:** Chaar FA (full adder) boxes right se left: FA0 (bit 0) right mein. Har FA ke upar A seedha aata hai aur B ek XOR se hokar. Upar ki K line har XOR mein jaati hai aur right side se neeche aakar FA0 ka Cin banti hai. Carries C1, C2, C3 right se left jaate hain, C4 left se bahar. Trace  $K = 1$ ,  $A = 0101$ ,  $B = 0011$ : XOR outputs 1100,  $Cin = 1$ , sum  $0101 + 1100 + 1 = 10010 \rightarrow S = 0010$ ,  $C4 = 1$ .

**Formula:**  $A - B = A + B' + 1$  · adder-subtractor: input to FA =  $B_i \oplus K$ ,  $Cin = K$ .

Signed result: C4 phenko. Unsigned subtraction:  $C4 = 0 \rightarrow$  borrow ( $A < B$ ).

#### Solved example (5 marks):

Draw a 4-bit adder-subtractor and explain how one control input selects addition or subtraction. Trace  $0101 - 0011$ .

1. Circuit: draw D4.5a (4 FAs in a chain, XOR on every B input, K to all XORs and to Cin of FA0).
2.  $K = 0$ :  $B_i \oplus 0 = B_i$  and  $Cin = 0 \rightarrow S = A + B$ .
3.  $K = 1$ :  $B_i \oplus 1 = B_i'$  and  $Cin = 1 \rightarrow S = A + B' + 1 = A + (-B) = A - B$ .
4. Trace:  $K = 1$ .  $B = 0011 \rightarrow$  XOR outputs 1100.  $0101 + 1100 + 1$ : bit 0:  $1 + 0 + 1 = 10 \rightarrow 0$  c1; bit 1:  $0 + 0 + 1 = 1 \rightarrow 1$  c0; bit 2:  $1 + 1 + 0 = 10 \rightarrow 0$  c1; bit 3:  $0 + 1 + 1 = 10 \rightarrow 0$  c1.
5.  $S = 0010$  (+2),  $C4 = 1$  (no borrow, kyunki  $5 \geq 3$ ).

**Answer:**  $0101 - 0011 = 0010$  (+2),  $C4 = 1$ .

**Galti mat karna:** (1) Subtract karte waqt sirf invert karna aur  $Cin = 1$  bhool jaana ( $5 - 3$  ka answer 0001 aayega, galat). (2) Signed answer mein C4 ko 5th bit bana ke likhna ( $10010$  ko 18 padhna). (3)  $C4 = 1$  ko "overflow" samajhna: signed overflow alag cheez hai (next topic).

**Trick:** "Minus = ulta + ek." Ek K wire do kaam karti hai: B ko ulta karti hai aur wahi +1 deti hai.

**Ek line mein:** Binary addition school wala column addition hai (2 pe carry), aur  $A - B = A + B' + 1$ , jo ek XOR-wale adder ( $K = 1$ ) se ho jaata hai.

## 4.6 Overflow detection rules

**Pehle socho:** 2-litre ki bottle mein 3 litre paani daalo, 1 litre bahar gir jaata hai aur bottle mein galat maatra dikhti hai. 4-bit signed box sirf  $-8$  se  $+7$  tak hold kar sakta hai.  $5 + 4 = 9$  is box mein fit nahi hota, to adder ek galat pattern deta hai ( $1001 = -7$ ). Ise **overflow** kehte hain. Aur ye chupchaap hota hai, koi alarm nahi bajta (odometer 999 se 000 pe chala jaata hai bina beep ke). Isliye CPU ek **flag** bit set karta hai taaki software check kar sake.

**Definition: Overflow** occurs when the true result of an arithmetic operation lies outside the range representable with the available bits. For two's complement (signed) addition it is indicated by the overflow flag  $V (OF) = C_{in,MSB} \oplus C_{out,MSB}$ ; for unsigned addition by the carry flag  $CF = C_{out,MSB}$ .

**Mushkil shabd, aasaan matlab:**

### overflow

Sahi answer box ki range se bahar. 4-bit signed: answer  $+9$  ya  $-9$  aaye to overflow.

### C3 / C4 (4-bit)

$C3$  = MSB column (bit 3) mein **aane** wala carry.  $C4$  = MSB column se **bahar jaane** wala carry.

### V / OF (overflow flag)

Signed overflow ka flag.  $V = C3 \oplus C4$  (dono alag hon to 1).

### CF (carry flag)

Unsigned overflow ka flag =  $C4$ .

### flag

Ek bit jo CPU result ke baare mein set karta hai, jaise "overflow hua" ya "result zero hai".

### trap

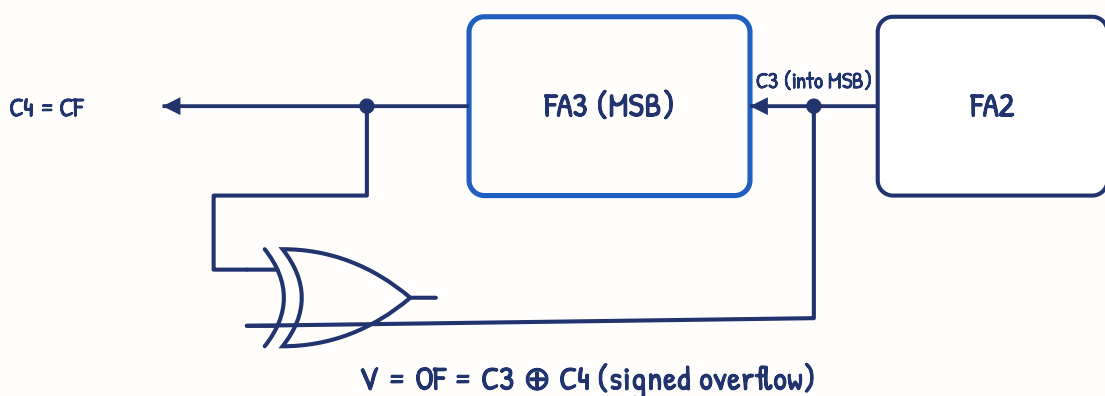
Error pe program ka ruk jaana. MIPS add/addi signed overflow pe trap karte hain; addu/addiu nahi.

**Kaise kaam karta hai:**

1. **Seedha matlab:** answer ko decimal mein socho. 4-bit signed range  $-8$  to  $+7$ . Agar sahi answer is range ke bahar hai to overflow.
2. **Rule 1: positive + positive = negative** aaya  $\rightarrow$  overflow. (Do positive ka sum kabhi negative nahi ho sakta; MSB 1 aaya matlab number bahut bada ho gaya.)
3. **Rule 2: negative + negative = positive** aaya  $\rightarrow$  overflow.
4. **Opposite signs kabhi overflow nahi karte:**  $+5 + (-3)$  ka answer hamesha dono ke beech mein aata hai ( $-3$  aur  $+5$  ke beech), aur dono already range mein hain, to answer bhi range mein.

5. **Rule 3 (hardware):  $C3 \neq C4 \rightarrow \text{overflow}$ .** MSB column mein aane wala carry aur jaane wala carry alag hon to  $V = 1$ . Circuit mein bas ek XOR:  $V = C3 \oplus C4$ .
6. **Rule 3 kyun sahi hai (MSB column dekho):** (a) Dono MSB 0 (dono positive):  $C4 = 0$  hi hoga; sum bit =  $C3$ .  $C3 = 1$  aaya to answer ka MSB 1 (negative)  $\rightarrow$  overflow, aur tab  $C3 = 1 \neq C4 = 0$ . (b) Dono MSB 1 (dono negative):  $C4 = 1$  hi hoga; sum bit =  $C3$ .  $C3 = 0$  aaya to answer positive  $\rightarrow$  overflow, aur tab  $C3 = 0 \neq C4 = 1$ . (c) MSB alag: column mein  $0 + 1 + C3$ , isliye  $C4 = C3$  hamesha  $\rightarrow V = 0$ . Teeno cases rule se match.
7. **Carry-out  $\neq$  signed overflow (bahut important):**  $-1 + 1 = 1111 + 0001 = 1\ 0000$ .  $C4 = 1$  aaya, par signed answer  $0000 = 0$  bilkul sahi hai!  $C3 = 1, C4 = 1 \rightarrow V = 0$ . Uta,  $7 + 1 = 0111 + 0001 = 1000$ :  $C4 = 0$  par signed answer  $-8$  galat ( $V = 1$ ). Matlab  **$C4$  unsigned ka flag hai, signed ke liye  $C3 \oplus C4$  dekho.**
8. ALU dono flags hamesha banata hai; instruction decide karti hai kaunsa matter karta hai.

| A + B (4-bit) | bits        | result    | C3 | C4 = CF | V | verdict                |
|---------------|-------------|-----------|----|---------|---|------------------------|
| 5 + 4         | 0101 + 0100 | 1001 (-7) | 1  | 0       | 1 | signed overflow        |
| -6 + -5       | 1010 + 1011 | 0101 (+5) | 0  | 1       | 1 | signed overflow        |
| 5 + -3        | 0101 + 1101 | 0010 (+2) | 1  | 1       | 0 | OK                     |
| 3 + 2         | 0011 + 0010 | 0101 (+5) | 0  | 0       | 0 | OK                     |
| -1 + 1        | 1111 + 0001 | 0000      | 1  | 1       | 0 | unsigned overflow only |
| 7 + 1         | 0111 + 0001 | 1000 (-8) | 1  | 0       | 1 | signed overflow only   |
| -8 + -8       | 1000 + 1000 | 0000      | 0  | 1       | 1 | both                   |



CF (unsigned overflow) =  $C4$  alone. Same adder, two flags — the instruction decides which one matters.

#### Overflow detection with one XOR at the MSB

How to draw: Draw FA3 with  $C3$  entering and  $C4$  leaving; tap both into an XOR labelled V.

**Diagram kaise padhein:** Right mein FA2, left mein FA3 (MSB). FA2 se FA3 mein jaane wala carry  $C3$  hai, FA3 se bahar  $C4 (= CF)$ . Dono wires tap karke ek XOR mein jaate hain; XOR ka output V hai. Trace  $5 + 4$ :  $C3 = 1, C4 = 0 \rightarrow \text{XOR}(1, 0) = 1 \rightarrow V = 1$  (overflow).

**Formula:**  $V = C_3 \oplus C_4$  (n-bit:  $C_{n-1} \oplus C_n$ ) · **CF** =  $C_4$

Sign form:  $V = A_3'B_3'S_3 + A_3B_3S_3'$  (dono 0 aur sum 1, ya dono 1 aur sum 0).

### Solved example 1 (overflow hota hai):

In 4-bit two's complement compute  $0101 + 0100$ . Is there overflow?

1. Bit 0:  $1 + 0 + 0 = 1 \rightarrow$  likho 1, carry 0.
2. Bit 1:  $0 + 0 + 0 = 0 \rightarrow$  likho 0, carry 0.
3. Bit 2:  $1 + 1 + 0 = 10 \rightarrow$  likho 0, carry 1 (ye  $C_3$  hai, MSB mein jaayega).
4. Bit 3 (MSB):  $0 + 0 + 1 = 1 \rightarrow$  likho 1, carry 0 ( $C_4 = 0$ ).
5. Result 1001. Rule 1:  $+5 + +4 =$  negative (MSB 1)  $\rightarrow$  overflow. Rule 3:  $C_3 = 1, C_4 = 0 \rightarrow V = 1$ .  
Decimal:  $9 > 7$ , fit nahi.

**Answer: 1001 (reads as  $-7$ ),  $V = 1 \rightarrow$  signed overflow;  $CF = 0$ .**

### Solved example 2 (overflow nahi hota):

Compute  $0101 + 1101 (+5 + -3)$  and state  $V$  and  $CF$ .

1. Bit 0:  $1 + 1 = 10 \rightarrow 0$ , carry 1.
2. Bit 1:  $0 + 0 + 1 = 1 \rightarrow 1$ , carry 0.
3. Bit 2:  $1 + 1 + 0 = 10 \rightarrow 0$ , carry 1 ( $C_3 = 1$ ).
4. Bit 3:  $0 + 1 + 1 = 10 \rightarrow 0$ , carry 1 ( $C_4 = 1$ ).
5. Result 0010 = +2 (sahi). Signs opposite the  $\rightarrow$  overflow possible hi nahi.  $C_3 = 1 = C_4 \rightarrow V = 0$ .  $CF = 1$  (par signed mein ignore).

**Answer: 0010 = +2,  $V = 0$ ,  $CF = 1$  (no signed overflow).**

### Solved example 3 (mock 6 marks):

Compute  $0110 - 1101$  (two's complement) on the adder-subtractor and state the overflow flag.

1.  $K = 1 \rightarrow B' = 0010, C_{in} = 1$ .
2.  $0110 + 0010 + 1$ : bit 0:  $0 + 0 + 1 = 1 \rightarrow 1, c_0$ ; bit 1:  $1 + 1 + 0 = 10 \rightarrow 0, c_1$ ; bit 2:  $1 + 0 + 1 = 10 \rightarrow 0, c_1$  ( $C_3 = 1$ ); bit 3:  $0 + 0 + 1 = 1 \rightarrow 1, c_0$  ( $C_4 = 0$ ).
3. Result 1001. Decimal check:  $6 - (-3) = +9$ , jo 4-bit range (max  $+7$ ) se bahar hai; 1001 ko  $-7$  padha jaayega.
4.  $C_3 = 1, C_4 = 0 \rightarrow V = 1$ .

**Answer: 1001,  $V = 1$  (overflow).**

**Galti mat karna:** (1) Carry-out  $C_4$  ko signed overflow flag bolna ( $1111 + 0001$  mein  $C_4 = 1$  par  $V = 0$ ).  
 (2) "Positive + negative bhi overflow kar sakta hai" likhna: kabhi nahi. (3) Ye maanna ki overflow pe CPU khud error dega: by default chupchaap galat answer aata hai, flag check karna padta hai.

**Trick:** "Same sign andar, ulta sign bahar = overflow." Hardware ke liye: "MSB mein carry aaya aur gaya alag = overflow."

**Ek line mein:** Signed overflow tab hai jab do same-sign numbers ka sum ulta sign deta hai, hardware mein  $V = C_3 \oplus C_4$ ; unsigned overflow sirf  $CF = C_4$ .

**Exam mein is unit se kya aata hai → kaise solve karein**

| Question type                                                                   | Method (steps)                                                                                                                                                                        |
|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Binary → decimal ( $101101_2 = ?$ )                                             | 1. Bits ke neeche weights likho (right se 1, 2, 4, 8). 2. Jahan bit 1 hai wo weights jodo. 3. Fraction ho to 0.5, 0.25, 0.125 weights.                                                |
| Decimal → binary / hex / octal                                                  | 1. $\div 2$ (ya $\div 16$ ) baar baar, har remainder likho. 2. Neeche se upar padho. 3. Binary se octal: right se 3 ke group; hex: right se 4 ke group. 4. Weights jod ke check karo. |
| Hex → decimal ( $0x3E8$ , $0x3C$ )                                              | 1. Har digit $\times 16$ ki power (1, 16, 256). 2. $A = 10$ se $F = 15$ . 3. Jodo.                                                                                                    |
| Bits needed for N (unsigned)                                                    | 1. Smallest $n$ jisme $2^n - 1 \geq N$ . 2. $1000 \rightarrow 10$ bits, $20 \rightarrow 5$ bits.                                                                                      |
| $-x$ in S-M / 1's / 2's (8-bit)                                                 | 1. $+x$ binary (8 bits). 2. S-M: MSB 1. 3. 1's: sab ulto. 4. 2's: 1's + 1 (ya copy-till-first-1 shortcut). 5. Weights se verify.                                                      |
| Signed value of a pattern ( $0xB7$ , $10110100$ )                               | 1. Binary banao. 2. MSB 0 → normal. 3. MSB 1 → $-2^{n-1}$ + baaki weights (ya invert + 1 karke minus lagao).                                                                          |
| Range of n-bit 2's complement                                                   | $-2^{n-1}$ to $2^{n-1} - 1$ (6-bit: -32 to 31; 8-bit: -128 to 127). S-M / 1's: $\pm(2^{n-1} - 1)$ .                                                                                   |
| Sign extension ( $0xF4 \rightarrow 16$ bits, $1010 \rightarrow 8$ bits)         | 1. MSB dekho. 2. Naye bits mein MSB copy. 3. Hex mein: MSB 1 → upar FF, MSB 0 → upar 00.                                                                                              |
| Add / subtract with carries                                                     | 1. Subtract ho to B invert + $C_{in}$ 1. 2. Right se column by column, carry row likho. 3. Signed: $C_4$ phenko. 4. Decimal mein check.                                               |
| Overflow? (V and CF)                                                            | 1. Add karo, $C_3$ aur $C_4$ note karo. 2. $V = C_3 \oplus C_4$ , $CF = C_4$ . 3. Sign rule se cross-check (same sign in, ulta out). 4. Decimal answer range mein hai ya nahi.        |
| Theory: why 2's complement / adder-subtractor / $V = C_3 \oplus C_4$ derivation | 1. One zero, one adder for add and subtract, MSB = sign, no end-around carry. 2. Circuit D4.5a with K. 3. MSB column ke 3 cases (dono 0, dono 1, alag).                               |

## Unit 5 · Latches, Flip-Flops & Timing

**Is unit ki kahani:** Unit 3 aur 4 ke circuits (adder, MUX, decoder) kuch yaad nahi rakhte: input hata do, output gaya. Par computer ko yaad rakhna padta hai: agla instruction kahan hai (PC), register mein kya hai. Ye unit dikhata hai ki **feedback** (output ko wapas input mein dena) se memory kaise banti hai: SR latch → D latch → D flip-flop. Phir **clock** aata hai jo sabko ek saath, sahi time pe update karata hai, aur timing rules (setup, hold) jo batate hain clock kitna fast chal sakta hai. Unit 6 in flip-flops ko jodkar registers aur counters banayega.

### 5.1 Combinational vs sequential circuits

**Pehle socho:** Calculator mein  $2 + 3$  dabao, hamesha 5 aata hai: answer sirf *abhi* ke input pe depend karta hai. Ab TV remote ka "channel +" button socho: dabane pe channel 5 aayega ya 8, ye isse depend karta hai ki *pehle* kaunsa channel tha. TV ko pichhla channel **yaad** hai. Pehla type **combinational** hai (koi yaaddasht nahi), doosra **sequential** (yaaddasht hai). Yaaddasht aati hai **feedback loop** se: output ko ghuma ke wapas input mein do, to circuit apni value khud ko baar baar "bolta" rehta hai.

**Definition:** A **combinational circuit** is one whose outputs depend only on its present inputs. A **sequential circuit** is one whose outputs depend on the present inputs and on the past history of inputs (its stored **state**); it contains memory elements formed by feedback.

**Mushkil shabd, aasaan matlab:**

**state**

Circuit ne jo yaad rakha hai, wo value. Counter ka state = abhi ka count (jaise 5).

**feedback loop**

Output wire ko ghuma ke apne hi input mein jodna. Isse circuit apni value hold kar sakta hai.

**bistable**

"Do stable haalat". Do inverters ka loop: ya to  $Q = 1$  ya  $Q = 0$ , aur jo bhi ho usi mein tika rehta hai. 1 bit ki sabse simple memory.

**clock**

Ek signal jo baar baar 0-1-0-1 hota rehta hai (square wave). Sequential circuit ko batata hai "ab update karo".

**Kaise kaam karta hai:**

1. Combinational: inputs → gates → output. Koi wire wapas nahi jaata. Example: adder, MUX, decoder, encoder.

2. Sequential: combinational logic + memory. Memory ka output wapas logic ke input mein aata hai (feedback). Example: latch, flip-flop, register, counter, FSM.
3. **Bistable trace (do inverters, ek doosre mein):** maan lo  $Q = 1$ . Neeche wala inverter  $Q$  ko ulta karke  $Q' = 0$  deta hai. Upar wala inverter  $Q' = 0$  ko ulta karke 1 deta hai, jo  $Q$  hi hai. Loop poora,  $Q = 1$  tika rehta hai.
4. Ab maan lo  $Q = 0$ :  $Q' = 1$ , upar wala inverter 1 ko ulta karke  $0 \rightarrow Q = 0$  phir tika. To **do stable states hain:  $(Q, Q') = (1, 0)$  ya  $(0, 1)$ .**
5. Problem: is loop mein koi input hi nahi, to value badal nahi sakte. Agle topic mein inverters ki jagah NOR gates lagake S aur R inputs denge (SR latch).

| Combinational                                | Sequential                                     |
|----------------------------------------------|------------------------------------------------|
| Output = $f(\text{inputs now})$              | Output = $f(\text{inputs, past state})$        |
| No memory, no feedback                       | Memory through a feedback loop                 |
| Same inputs $\rightarrow$ always same output | Same inputs can give different outputs         |
| Adder, MUX, decoder                          | Latch, flip-flop, register, counter, FSM       |
| No clock needed                              | Clock decides when state updates (synchronous) |



combinational: signal flows once, no memory

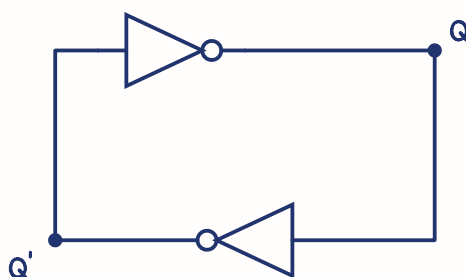


sequential: output fed back  $\rightarrow$  it can hold a value (state)

**Feedback turns logic into memory**

How to draw: Same logic box twice; on the right draw the output looping back into an input.

**Diagram kaise padhein:** Left mein logic box: inputs andar, output bahar, bas. Right mein wahi box, par output ka ek wire ghoom ke wapas input mein jaata hai. Wahi ghoomta wire memory hai.



**Two stable states:**

$Q = 1, Q' = 0$  or  $Q = 0, Q' = 1$

Holds forever (while powered) —  
but there is no input to change it.

**Bistable element (two cross-coupled inverters)**

How to draw: Two inverters pointing opposite ways, each output wired to the other's input; mark  $Q$  and  $Q'$ .

**Diagram kaise padhein:** Do inverters ulta dishaon mein. Upar wale ka output ( $Q$ ) neeche wale ka input hai; neeche wale ka output ( $Q'$ ) upar wale ka input. Trace  $Q = 1 \rightarrow Q' = 0 \rightarrow Q = 1$ , tika hua.

### Solved example (4 marks, "write the answer like this"):

*Distinguish between combinational and sequential circuits with block diagrams and two examples of each.*

1. Draw D5.1a: left block "combinational logic" (inputs → outputs); right block "combinational logic + memory" with the feedback wire and a clock into the memory.
2. Write: "A combinational circuit's output depends only on its present inputs; it has no memory and no feedback. Examples: full adder, 4:1 multiplexer."
3. Write: "A sequential circuit's output depends on present inputs and on its stored state; memory is created by feedback and is usually updated by a clock edge. Examples: binary counter, register."
4. Add the comparison table above (3 to 4 rows).

**Answer: definitions + D5.1a + table + examples (adder, MUX / counter, register).**

**Galti mat karna:** "Bahut saare gates hain to sequential" galat hai. 64-bit adder mein hazaron gates hain par feedback nahi, isliye combinational. Counter ko gates se akele nahi bana sakte; use state chahiye. Aur combinational ka delay zero nahi hota, bas memory nahi hoti.

**Trick:** "Calculator = combinational, TV remote = sequential." Sawaal pucho: "kya isko pichhla value pata hona chahiye?" Haan → sequential.

**Ek line mein:** Combinational = sirf abhi ke inputs; sequential = inputs + yaad kiya hua state, aur yaad feedback loop se aati hai.

## 5.2 SR latch: structure and forbidden state

**Pehle socho:** Ek light ke do button: **S (Set)** dabao → light ON, **R (Reset)** dabao → light OFF, kuch mat dabao → jaisi thi waisi rehti hai (yaad hai). Dono ek saath dabana bekaar sawaal hai ("ON bhi karo aur OFF bhi?"). Ye circuit do NOR gates ko cross mein jodkar banta hai, jisse feedback milta hai.

**Definition:** An **SR latch** is a basic memory element made of two cross-coupled NOR gates (or NAND gates) with inputs Set and Reset:  $S = 1$  makes  $Q = 1$ ,  $R = 1$  makes  $Q = 0$ , and  $S = R = 0$  holds the previous value. For the NOR latch,  $S = R = 1$  is **forbidden** because it forces  $Q = Q' = 0$  and releasing both inputs together gives an unpredictable result.

### Mushkil shabd, aasaan matlab:

#### NOR gate

OR ka ulta. Output 1 sirf tab jab dono inputs 0 hon. **Koi bhi input 1 → output 0.**  $NOR(0, 0) = 1$ ,  $NOR(1, 0) = 0$ ,  $NOR(0, 1) = 0$ .

### cross-coupled

Do gates jinme har ek ka output doosre ka input hai (X ki shape mein wires).

### set / reset / hold

Set = Q ko 1 karo. Reset = Q ko 0 karo. Hold = jo hai wahi rakho.

### Q and Q'

Do outputs. Normally Q' hamesha Q ka ulta hota hai ( $Q = 1 \rightarrow Q' = 0$ ).

### forbidden (invalid) state

Input combination jo use nahi karna chahiye, kyunki output galat ya unpredictable hota hai.

### race

Do gates ek saath badalne ki koshish karte hain; jo thoda tez hai wo jeet jaata hai. Result pehle se pata nahi.

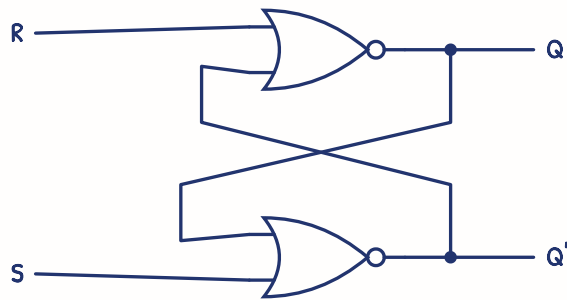
### active-low

Input 0 hone pe "dabaya hua" maana jaata hai (NAND latch mein). Naam pe bar lagta hai: S', R'.

**Kaise kaam karta hai (NOR latch: gate N1 ke inputs R aur Q', output Q; gate N2 ke inputs S aur Q, output Q'):**

1. **Set (S = 1, R = 0), shuru mein Q = 0:** N2 ko S = 1 mila  $\rightarrow$  "koi 1  $\rightarrow$  output 0"  $\rightarrow$  Q' = 0. Ab N1 ko R = 0 aur Q' = 0 mile  $\rightarrow$  dono 0  $\rightarrow$  Q = 1. Ab N2 ko S = 1, Q = 1  $\rightarrow$  Q' = 0 hi rehta hai. Stable: **Q = 1, Q' = 0**.
2. **Reset (S = 0, R = 1), shuru mein Q = 1:** N1 ko R = 1  $\rightarrow$  Q = 0. N2 ko S = 0, Q = 0  $\rightarrow$  Q' = 1. N1 ko R = 1 (ab bhi)  $\rightarrow$  Q = 0 tika. Stable: **Q = 0, Q' = 1**.
3. **Hold (S = 0, R = 0), pehle Q = 1, Q' = 0:** N1 ko R = 0, Q' = 0  $\rightarrow$  Q = 1. N2 ko S = 0, Q = 1  $\rightarrow$  Q' = 0. Kuch nahi badla. Agar pehle Q = 0, Q' = 1 tha: N1 ko 0, 1  $\rightarrow$  Q = 0; N2 ko 0, 0  $\rightarrow$  Q' = 1. Phir bhi kuch nahi badla. **S = R = 0 pe latch pichhli value yaad rakhta hai**; yahi memory hai.
4. **Forbidden (S = 1, R = 1):** N1 ko R = 1  $\rightarrow$  Q = 0. N2 ko S = 1  $\rightarrow$  Q' = 0. Ab **Q = Q' = 0**. Pehli problem: Q' ko Q ka ulta hona chahiye tha, rule toot gaya; jo circuit Q' use karta hai use galat info milti hai.
5. **Asli khatra, dono ko ek saath 0 karna:** abhi Q = Q' = 0 hai. S, R dono 0 ho gaye. N1 dekhta hai (0, 0)  $\rightarrow$  Q = 1 banane chalta hai. N2 bhi (0, 0) dekhta hai  $\rightarrow$  Q' = 1 banane chalta hai. Agar dono same speed ke hain to dono 1 ho jaate hain, phir dono ko ek 1 dikhta hai to dono 0 ho jaate hain: 00  $\rightarrow$  11  $\rightarrow$  00  $\rightarrow$  11, **oscillation**. Real chip mein ek gate thoda tez hota hai aur wo jeet jaata hai, par kaun jeetega pata nahi. Memory jo random value store kare wo bekaar hai. Isliye 11 **forbidden**.
6. **NAND SR latch:** NAND ka rule ulta: "koi bhi input 0  $\rightarrow$  output 1". Isliye inputs active-low hain (S', R'): S' = 0  $\rightarrow$  set (Q = 1); R' = 0  $\rightarrow$  reset (Q = 0); S' = R' = 1  $\rightarrow$  hold; S' = R' = 0  $\rightarrow$  forbidden (Q = Q' = 1).
7. **Gated SR latch:** S aur R ke aage do AND gates lagao jinka doosra input E (enable) hai. E = 0  $\rightarrow$  andar S = R = 0  $\rightarrow$  hold, chahe bahar kuch bhi ho. E = 1  $\rightarrow$  normal SR latch.
8. **Characteristic equation (derivation):** Q<sup>+</sup> (next Q) kab 1 hai? (a) S = 1 hai (set), ya (b) R = 0 hai aur Q pehle se 1 tha (hold kiya 1). To **Q<sup>+</sup> = S + R'Q**, condition S·R = 0.

| S | R | Q (old) | Q <sup>+</sup> | Action    |
|---|---|---------|----------------|-----------|
| 0 | 0 | 0       | 0              | hold      |
| 0 | 0 | 1       | 1              |           |
| 0 | 1 | x       | 0              | reset     |
| 1 | 0 | x       | 1              | set       |
| 1 | 1 | x       | Q = Q' = 0     | forbidden |



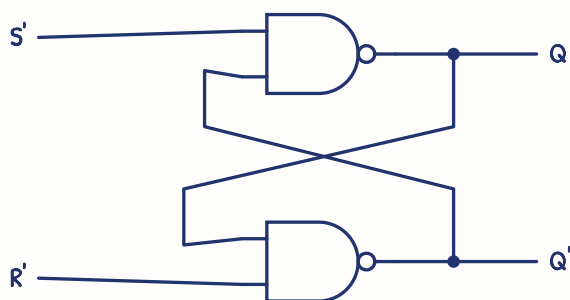
| S | R | Q <sup>+</sup> | action             |
|---|---|----------------|--------------------|
| 0 | 0 | Q              | hold               |
| 1 | 0 | 1              | set                |
| 0 | 1 | 0              | reset              |
| 1 | 1 | -              | forbidden (Q=Q'=0) |

NOR latch: inputs are active-HIGH. Each output feeds the other gate.

#### NOR SR latch + characteristic table

How to draw: Two NOR gates stacked; draw the output of each crossing to the second input of the other; R on the gate that makes Q.

**Diagram kaise padhein:** Do NOR gates upar neeche. Upar wale gate mein R aata hai aur uska output Q hai; neeche wale mein S aata hai aur output Q'. Beech mein do tirchhe wires (X shape): Q neeche wale gate mein jaata hai, Q' upar wale mein. Trace S = 1: neeche wala gate turant 0 deta hai (Q' = 0), wo upar jaake R = 0 ke saath mil ke Q = 1 banata hai.



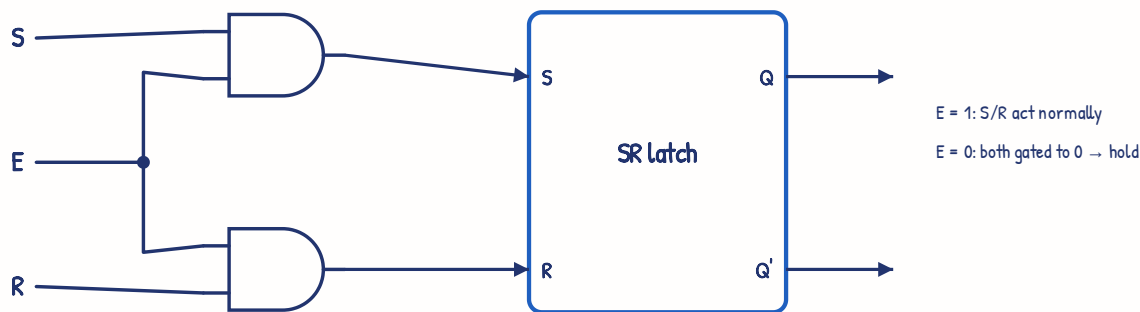
| S' | R' | Q <sup>+</sup> | action             |
|----|----|----------------|--------------------|
| 1  | 1  | Q              | hold               |
| 0  | 1  | 1              | set                |
| 1  | 0  | 0              | reset              |
| 0  | 0  | -              | forbidden (Q=Q'=1) |

NAND latch: inputs are active-LOW (a 0 sets/resets).

#### NAND SR latch (active-low) + table

How to draw: Same shape with NAND gates; S' enters the gate that produces Q.

**Diagram kaise padhein:** Same shape, NAND gates ke saath, inputs pe bar (S', R'). S' wala gate Q banata hai. Trace S' = 0: NAND ko ek 0 mila → Q = 1.



### Gated SR latch (enable)

How to draw: Two AND gates in front of an SR latch box; E feeds both.

**Diagram kaise padhein:** S aur R pehle AND gates mein jaate hain, dono AND ko E bhi milta hai. E = 0 → dono AND ka output 0 → latch hold. E = 1 → S, R seedhe latch tak.

### Solved example 1 (sequence trace):

A NOR SR latch starts with Q unknown. Inputs (S, R) are applied in order: (1, 0), (0, 0), (0, 1), (0, 0), (1, 1), (0, 0). Give Q and Q' after each step.

1. (1, 0): set → Q = 1, Q' = 0.
2. (0, 0): hold → Q = 1, Q' = 0.
3. (0, 1): reset → Q = 0, Q' = 1.
4. (0, 0): hold → Q = 0, Q' = 1.
5. (1, 1): forbidden → Q = 0, Q' = 0 (dono NOR ko ek 1 mila).
6. (0, 0) dono ek saath: race → unpredictable (simulation mein oscillate ya X).

**Answer:** Q = 1, 1, 0, 0, 0, unpredictable (Q' = 0, 0, 1, 1, 0, unpredictable).

### Solved example 2 (5 marks):

Draw an SR latch using NOR gates and explain its operation with the characteristic table. Why is S = R = 1 forbidden? How does the NAND version differ?

1. Draw D5.2a and the table above.
2. Operation: "S = 1 forces Q' = 0, so Q = (R + Q')' = (0 + 0)' = 1 (set). R = 1 forces Q = 0 (reset). With S = R = 0 each gate keeps the other in its present state through feedback, so the latch holds."
3. Forbidden: "S = R = 1 makes both outputs 0, so Q' is no longer the complement of Q. If both inputs then return to 0 together, the two gates race and the final state depends on tiny delay differences; it is unpredictable."
4. NAND: "Inputs are active-low: S' = 0 sets, R' = 0 resets, S' = R' = 1 holds, and S' = R' = 0 is forbidden (Q = Q' = 1)."
5. Equation:  $Q^+ = S + R'Q$  with  $SR = 0$ .

Answer: diagram + table + set/reset/hold explanation + race explanation + NAND difference.

**Galti mat karna:** (1) NOR latch mein R ko galat gate pe lagana: **R us gate mein jaata hai jo Q banata hai** ( $R = 1 \rightarrow Q = 0$ ). (2) NAND latch ke inputs ko active-high samajhna. (3) Sirf "11 pe output galat hai" likhna; asli problem release pe race hai, wo zaroor likho.

**Trick:** "NOR: 11 mana. NAND: 00 mana." NOR ka forbidden output 00, NAND ka 11 (ulta).

**Ek line mein:** Do cross-coupled NOR gates: S = 1 set, R = 1 reset, 00 hold (memory), 11 forbidden kyunki  $Q = Q' = 0$  aur release pe race;  $Q^+ = S + R'Q$ .

### 5.3 D latch (level-triggered)

**Pehle socho:** SR latch ki do pareshaniyan: do alag control wires, aur 11 ka khatra. Hal: sirf **ek data wire D** rakho, aur  $S = D$ ,  $R = D'$  bana do. Ab S aur R kabhi ek saath 1 ho hi nahi sakte. Upar se ek **EN (enable)** lagao. Socho ek khidki: jab khuli hai ( $EN = 1$ ) to andar wala bahar jo bhi ho raha hai sab dekhta hai; jab band hai ( $EN = 0$ ) to aakhri dekhi hui tasveer yaad rehti hai.

**Definition:** A **D latch** is a gated latch with a data input D and an enable EN, built from an SR latch with  $S = D \cdot EN$  and  $R = D' \cdot EN$ . While  $EN = 1$  it is **transparent** (Q follows D); while  $EN = 0$  it holds the last value. It is **level-triggered**.

**Mushkil shabd, aasaan matlab:**

**D (data)**

Wo bit jo store karna hai.

**EN (enable)**

Khidki ka switch. 1 = khuli, 0 = band.

**transparent**

"Aar-paar dikhna".  $EN = 1$  pe  $Q = D$ , D badle to Q bhi turant badle.

**opaque / hold**

$EN = 0$  pe Q freeze, D kuch bhi kare.

**level-triggered**

EN ke *level* (1 rehne ke poore time) pe kaam karta hai, kisi ek instant pe nahi.

**glitch**

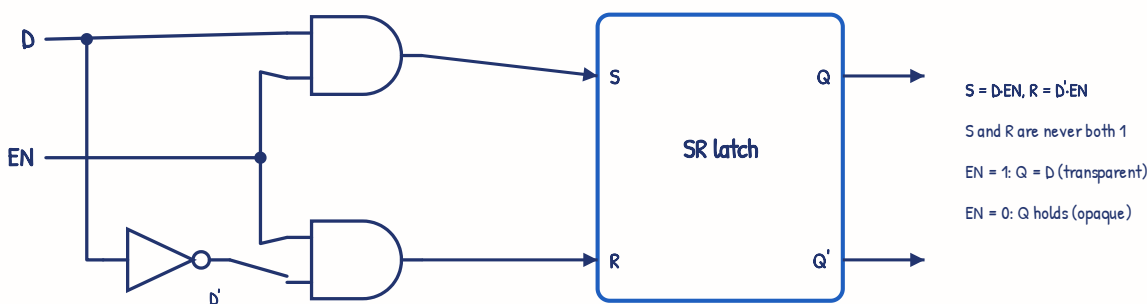
Signal mein chhoti si galat jhatka ( $0 \rightarrow 1 \rightarrow 0$  bahut thode time ke liye).

**Kaise kaam karta hai:**

1. Andar:  $S = D \cdot EN$ ,  $R = D' \cdot EN$  (do AND gates + ek inverter), phir SR latch.

2. **EN = 0:**  $S = D \cdot 0 = 0, R = D' \cdot 0 = 0 \rightarrow$  SR latch hold. Q purana hi.
3. **EN = 1, D = 1:**  $S = 1, R = 0 \rightarrow$  set  $\rightarrow Q = 1$ .
4. **EN = 1, D = 0:**  $S = 0, R = 1 \rightarrow$  reset  $\rightarrow Q = 0$ .
5. Kya  $S = R = 1$  ho sakta hai? Uske liye  $D = 1$  aur  $D' = 1$  dono chahiye, jo impossible hai. **Forbidden state construction se hi khatam.**
6. Equation:  $Q^+ = EN \cdot D + EN' \cdot Q$  ( $EN = 1 \rightarrow D, EN = 0 \rightarrow Q$ ).
7. Kami: jab tak  $EN = 1$  hai, D ka har badlav (glitch bhi) Q tak pahunch jaata hai. Agar do latches line mein hon aur dono ki khidki ek saath khuli ho, data ek hi baar mein dono paar kar jaata hai. Isliye CPU pipelines mein edge-triggered flip-flop use hote hain.

| EN | D | S | R | Q <sup>+</sup> |
|----|---|---|---|----------------|
| 0  | x | 0 | 0 | Q (hold)       |
| 1  | 0 | 0 | 1 | 0              |
| 1  | 1 | 1 | 0 | 1              |

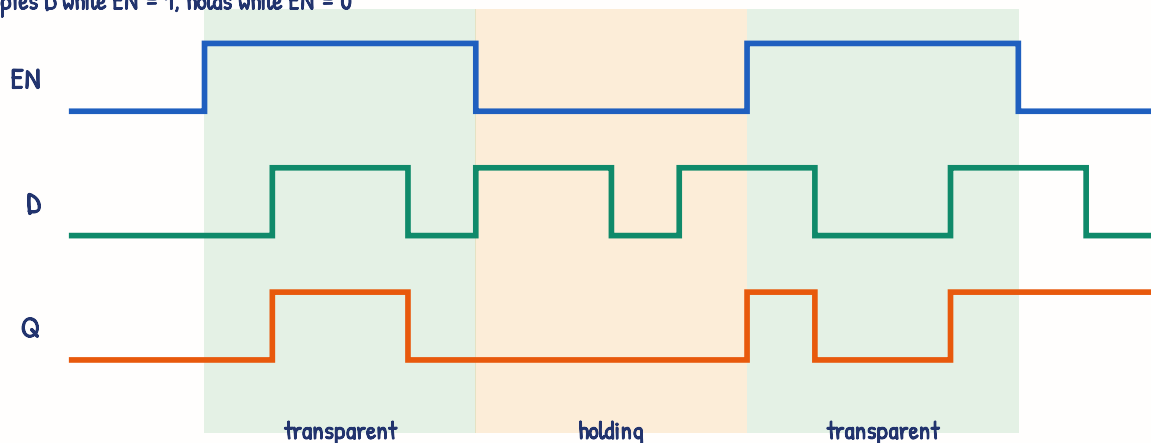


#### D latch from an SR latch

How to draw: Inverter on D, two AND gates with EN, feeding S and R of an SR latch box.

**Diagram kaise padhein:** D upar wale AND mein seedha jaata hai (S banta hai) aur inverter se hokar neeche wale AND mein (R banta hai). Dono AND ko EN milta hai. Trace  $EN = 1, D = 1: S = 1, R = 0 \rightarrow Q = 1$ .  $EN = 0$ : dono AND 0  $\rightarrow$  hold.

D latch: Q copies D while  $EN = 1$ , holds while  $EN = 0$



#### D latch transparency waveform

How to draw: Draw EN, D, Q rows; shade the  $EN = 1$  windows; inside them  $Q = D$ , outside Q is flat.

**Diagram kaise padhein:** Teen rows: EN, D, Q. Hare shaded hisse wo time hain jab EN = 1 (transparent): wahan Q bilkul D ki copy hai, D ke saath upar neeche. EN = 0 wale hisse ("holding") mein Q seedhi line hai, us value pe jo D ki thi jab EN gira. Pehle window ke end mein D = 0 tha, isliye beech mein Q = 0 rehta hai, chahe D upar neeche hota rahe.

### Solved example (waveform question):

A D latch has  $Q = 0$  initially.  $EN = 1$  from  $t = 10\text{ ns}$  to  $20\text{ ns}$  and 0 otherwise. D is 0 until  $12\text{ ns}$ , 1 from  $12\text{ ns}$  to  $18\text{ ns}$ , and 0 after  $18\text{ ns}$ . Sketch Q. What is Q after  $20\text{ ns}$ ?

1.  $t < 10\text{ ns}$ :  $EN = 0 \rightarrow \text{hold} \rightarrow Q = 0$ .
2.  $10\text{ to }12\text{ ns}$ :  $EN = 1, D = 0 \rightarrow Q = 0$ .
3.  $12\text{ ns}$ :  $D = 0 \rightarrow 1$ , khidki khuli  $\rightarrow Q$  turant 1.
4.  $18\text{ ns}$ :  $D = 1 \rightarrow 0 \rightarrow Q$  turant 0.
5.  $20\text{ ns}$ : EN gira. Us waqt  $D = 0 \rightarrow Q = 0$  freeze.
6. Uske baad D kuch bhi kare,  $Q = 0$ .

**Answer:**  $Q = 0$  till  $12\text{ ns}$ , 1 from  $12$  to  $18\text{ ns}$ , 0 after; Q after  $20\text{ ns} = 0$ .

**Galti mat karna:** D latch ko "edge-triggered" mat bolo. Ye  $EN = 1$  ke **poore time** D copy karta hai, ek instant pe sample nahi karta. Q tab ki value hold karta hai jab EN **gira**, na ki jab utha.

**Trick:** "Latch = khidki (level), flip-flop = camera click (edge)."

**Ek line mein:** D latch = SR latch with  $S = D \cdot EN$ ,  $R = D' \cdot EN$ ;  $EN = 1$  pe transparent ( $Q = D$ ),  $EN = 0$  pe hold, aur forbidden state possible hi nahi.

## 5.4 D flip-flop (edge-triggered)

**Pehle socho:** Latch ki khidki poore  $EN = 1$  time khuli rehti hai, isliye galat jhatke bhi andar aa jaate hain. Hum chahte hain ki data sirf **ek instant** pe liya jaaye, jaise camera flash: click ke pal ki photo, baaki time kuch record nahi. Wo instant hai **clock edge**. Poore chip ke saare flip-flops ek hi clock edge pe update hote hain, to sab kuch ek taal mein chalta hai, jaise drum beat pe march karte soldiers.

**Definition:** A **D flip-flop** is an edge-triggered memory element that copies its D input to Q only at the active edge of the clock (usually the rising edge) and keeps Q constant at all other times. Its characteristic equation is  $Q(n+1) = D$ . It is commonly built as a **master-slave** pair of D latches enabled on opposite clock phases.

**Mushkil shabd, aasaan matlab:**

**clock (CLK)**

Square wave: 0, 1, 0, 1 barabar time ke liye. Har cycle = ek "beat".

### period T, frequency f

Ek cycle ka time = T. Ek second mein kitne cycles =  $f = 1/T$ .  $T = 1 \text{ ns} \rightarrow f = 1 \text{ GHz}$ .  $3 \text{ GHz} \rightarrow T \approx 0.33 \text{ ns}$ .

### rising edge / falling edge

Clock ka  $0 \rightarrow 1$  hona = rising (positive) edge.  $1 \rightarrow 0$  = falling (negative) edge.

### edge-triggered

Sirf edge ke instant pe kaam karta hai. Symbol mein clock pin pe chhota triangle ( $\triangleright$ ) = edge-triggered.

### master / slave

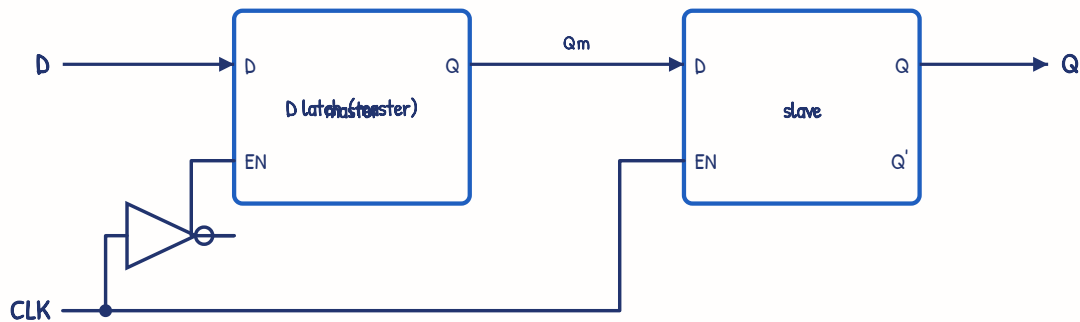
Flip-flop ke andar ke do latches. Master pehle data pakadta hai, slave use output pe dikhata hai.

### ring oscillator

Odd number (3, 5, 7) inverters ka loop. Kabhi stable nahi hota, apne aap  $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$  karta rehta hai: clock banane ka simple tareeka.

### Kaise kaam karta hai:

1. **Behaviour:** rising edge pe D ki jo value hai wo Q mein copy. Do edges ke beech D chahe 10 baar badle, Q nahi badalta.
2. **Master-slave, saral shabdon mein:** do darwaze wala airlock socho (ya canal lock): dono darwaze kabhi ek saath nahi khulte. Pehla darwaza (master latch) aur doosra (slave latch) ulte clock phases pe khulte hain (ek ko CLK, doosre ko CLK' milta hai).
3. **CLK = 0 (textbook / Harris version):** master khula (transparent), uska output Qm D ke peeche peeche chalta hai. Slave band, Q purana hold.
4. **CLK  $0 \rightarrow 1$  (rising edge):** master band ho jaata hai, aur us instant D ki jo value thi wo Qm mein phas jaati hai. Usi samay slave khulta hai aur Qm ko Q pe dikha deta hai. Q badla, sirf edge pe.
5. **CLK = 1:** master band hai, to D ke naye badlav andar nahi aate. Slave khula hai par Qm fixed hai, isliye Q fixed.
6. **CLK  $1 \rightarrow 0$ :** slave band (Q hold), master phir khul jaata hai. Cycle repeat. Result: **Q sirf rising edge pe badalta hai.**
7. **Class slide version:** master CLK = 1 pe khula, slave CLK = 0 pe. Tab Q **falling edge** pe badalta hai (negative-edge FF). Class MCQ ka answer: "falling edge". Exam mein apna convention likh do.
8. **Cost:** flip-flop = 2 latches  $\approx 2 \times$  gates, par timing predictable. Har register, PC, pipeline register D flip-flops ki row hai.
9. **Ring oscillator se clock:** N inverters (odd) ka loop. Signal ko poora period banane ke liye loop do baar ghoomna padta hai (ek baar  $1 \rightarrow 0$ , ek baar  $0 \rightarrow 1$ ), isliye  $T = 2 \cdot N \cdot t_{pd}$ . Even N hota to wo bistable ban jaata (5.1), oscillate nahi karta.
10. **Verilog:** always @(posedge clk) q <= d; (clocked block mein non-blocking <=).



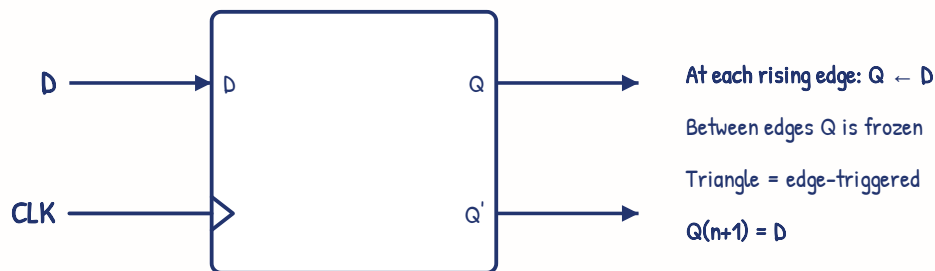
CLK = 0: master transparent (Qm follows D), slave holds. CLK 0 → 1: master closes, slave opens → Q takes D at the RISING edge.

Class slide variant: master enabled on CLK = 1 → the output changes at the FALLING edge (negative-edge FF).

### Master-slave D flip-flop

How to draw: Two D-latch boxes in series; CLK to one enable directly and to the other through an inverter.

**Diagram kaise padhein:** Left box master latch, right box slave latch. D master mein jaata hai, master ka Q slave ka D hai, slave ka Q final output. CLK neeche se aata hai: slave ke EN mein seedha, aur master ke EN mein inverter se hokar (master ko CLK' milta hai), isliye ek khula to doosra band. Neeche likha hai: CLK = 0 master transparent; rising edge pe slave Q update karta hai. Grey line class version (falling edge) batati hai.

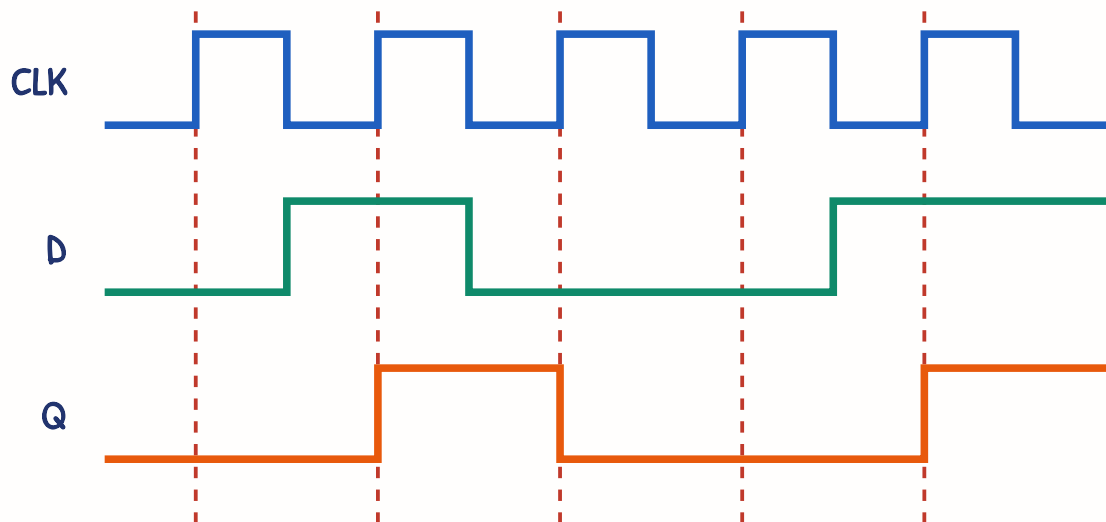


### D flip-flop symbol

How to draw: Box with D, CLK (triangle), Q and Q'.

**Diagram kaise padhein:** Box mein D (input), Q aur Q' (outputs), aur clock pin pe triangle. Triangle dekho to samjho "edge-triggered flip-flop", bina triangle "latch".

Trace Q: sample D only at rising edges (class slide p12)



D flip-flop waveform trace (class slide)

How to draw: Draw dashed lines at every rising edge; read D there and hold that value until the next edge.

**Diagram kaise padhein:** Teen rows: CLK, D, Q. Dashed lines rising edges pe hain (time 1, 3, 5, 7, 9). Har dashed line pe D ki value padho; Q us value pe chala jaata hai aur agli dashed line tak wahi rehta hai.

### Solved example 1 (timing trace, class slide):

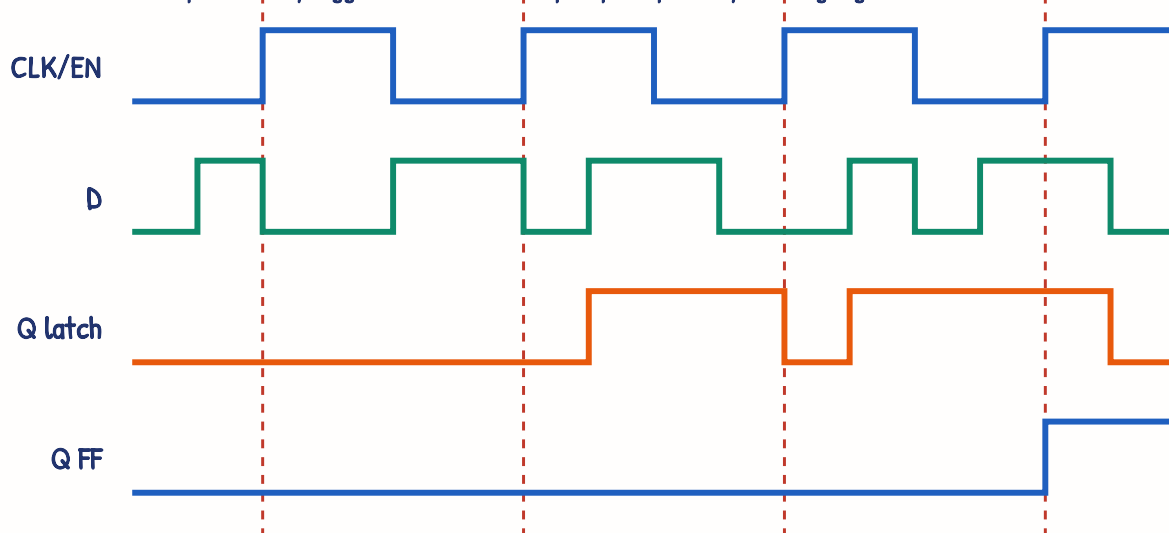
Rising clock edges at  $t = 1, 3, 5, 7, 9$ .  $D = 0$  before  $t = 2$ , 1 from  $t = 2$  to 4, 0 from  $t = 4$  to 8, and 1 after  $t = 8$ . Q starts at 0. Draw Q.

| Edge at $t$ | D at that instant | Q after edge | Kya hua                                                 |
|-------------|-------------------|--------------|---------------------------------------------------------|
| 1           | 0                 | 0            | $D = 0$ copy                                            |
| 3           | 1                 | 1            | $D$ $t = 2$ pe 1 hua tha, par Q ne edge 3 tak wait kiya |
| 5           | 0                 | 0            | $D$ $t = 4$ pe 0 hua, Q edge 5 pe gira                  |
| 7           | 0                 | 0            | D abhi bhi 0, koi badlav nahi                           |
| 9           | 1                 | 1            | $D$ $t = 8$ pe 1 hua, Q edge 9 pe utha                  |

1. Har rising edge pe vertical line kheencho.
2. Har line pe D padho (upar table ka column 2).
3. Q ko us value pe le jao aur agli line tak flat rakho.
4. Dhyani do: D ke badlav ( $t = 2, 4, 8$ ) Q ko turant nahi badalte; Q hamesha agle edge pe badalta hai.

**Answer:** Q = 0, 1, 0, 0, 1 after the five edges (string 01001).

Same clock and data: the latch passes every wiggle while EN = 1, the flip-flop samples only at rising edges



#### Latch vs flip-flop on the same inputs

How to draw: Four rows: CLK, D, Q(latch) follows D while CLK high, Q(FF) changes only at edges.

**Diagram kaise padhein:** Same CLK/EN aur same D, do outputs. "Q latch" har us time D ke saath hilta hai jab CLK = 1 hai (beech mein 0 → 1 → 0 sab dikhta hai). "Q FF" sirf rising edges (dashed) pe D padhta hai; pehle teen edges pe D = 0 tha isliye 0, aakhri edge pe D = 1 to Q = 1. Same input, alag output: yahi latch vs flip-flop ka farq hai.

#### Solved example 2 (NAT):

A ring oscillator has 5 inverters, each with 20 ps delay. Find its frequency. (Mock variant: 7 inverters of 10 ps.)

1.  $T = 2 \times N \times t_{pd} = 2 \times 5 \times 20 \text{ ps} = 200 \text{ ps}$ .
2.  $f = 1 / 200 \text{ ps}$ .  $1 / 1 \text{ ps} = 1000 \text{ GHz}$ , isliye  $f = 1000 / 200 = 5 \text{ GHz}$ .
3. Mock:  $T = 2 \times 7 \times 10 = 140 \text{ ps} \rightarrow f = 1000 / 140 = 7.14 \text{ GHz}$ .

**Answer: 5 GHz (mock: 7.14 GHz).**

#### Solved example 3 (5 marks, model answer):

Explain the master-slave D flip-flop with a neat diagram, sketch Q for given CLK and D, and compare latch and flip-flop.

1. Draw D5.4c (two D latches in series, CLK to one, CLK' to the other).
2. "The master is transparent while CLK = 0 and the slave holds. At the rising edge the master closes, capturing D, and the slave opens and passes this value to Q. While CLK = 1 the master is closed, so changes on D cannot reach Q. Hence Q changes only at the clock edge. (If the master is enabled on CLK = 1, as in the class slide, Q changes at the falling edge.)"
3. Waveform: mark every active edge, read D there, hold that value until the next edge (D5.4e).
4. Comparison: latch is level-triggered, transparent while enabled, passes glitches, fewer gates; flip-flop is edge-triggered, never transparent, ignores glitches between edges, about twice the gates,

used for registers and pipelines.

**Answer: diagram + operation + waveform + 4-point comparison.**

**Galti mat karna:** (1) "Master-slave mein dono latches ek saath khulte hain" galat: kabhi nahi. (2) Edge convention mix karna: class MCQ = falling edge, textbook diagram = rising edge; dono yaad rakho, answer mein convention likho. (3) Waveform mein D ke badalte hi Q badal dena: wo latch hai, flip-flop nahi.

**Trick: "Edge pe dekho, agle edge tak rakho."** Waveform ke liye bas itna.

**Ek line mein:** D flip-flop sirf clock edge pe  $Q = D$  karta hai ( $Q(n+1) = D$ ), andar do latches ulte phases pe hain jo kabhi ek saath nahi khulte.

## 5.5 Setup time, hold time, metastability

**Pehle socho:** Camera se chalti cheez ki photo lo. Shutter click hone se **thoda pehle** se aur **thoda baad** tak cheez ko hilna nahi chahiye, warna photo blur. Aur photo screen pe click ke **thodi der baad** aati hai. Flip-flop bhi yahi hai: clock edge = shutter. D ko edge se pehle kuch time (setup) aur baad kuch time (hold) stable rehna chahiye, aur Q edge ke kuch time baad (clock-to-Q) badalta hai. Agar D "hil gaya" to output blur jaisa: na 0 na 1, ise **metastability** kehte hain.

**Definition:** Setup time ( $t_{\text{setup}}$ ) is the minimum time the D input must be stable before the active clock edge; hold time ( $t_{\text{hold}}$ ) is the minimum time it must remain stable after the edge; clock-to-Q delay ( $t_{\text{pcq}}$ ) is the time from the edge until Q shows the new value. **Metastability** is a condition, caused by a setup or hold violation, in which the flip-flop output stays between 0 and 1 for an unpredictable time before settling to either value.

**Mushkil shabd, aasaan matlab:**

### setup violation

D edge ke bahut paas (late) badla: data der se pahuncha. Kaaran: logic path bahut slow / clock bahut fast.

### hold violation

D edge ke turant baad badal gaya: naya data bahut jaldi aa gaya. Kaaran: path bahut fast (short path).

### $t_{\text{pd}}$ (logic delay)

Combinational logic ka sabse lamba (worst) delay.  $t_{\text{cd}}$  = sabse chhota delay.

### critical path

Do flip-flops ke beech sabse lamba (slowest) path. Yahi max clock speed decide karta hai.

### asynchronous input

Aisa signal jo clock se tune nahi hai (button press, doosre clock wala chip). Ye kabhi na kabhi setup/hold window mein badlega hi.

### synchronizer

2 ya 3 flip-flops ki line jo async signal ko apne clock ke saath "safe" banati hai.

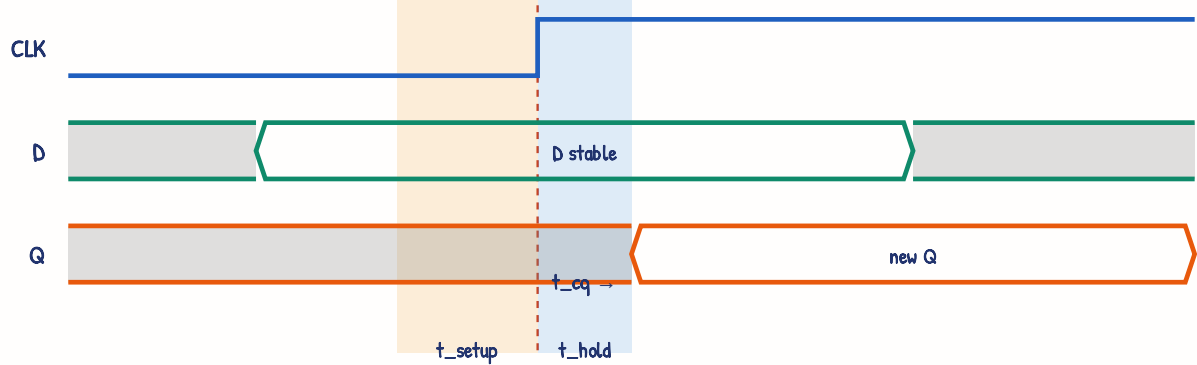
### MTBF

Mean Time Between Failures: average kitne time mein ek baar failure. Synchronizer se ise hazaron saal kar dete hain.

### Kaise kaam karta hai:

1. Edge se  $t_{\text{setup}}$  pehle se  $t_{\text{hold}}$  baad tak ek "window" hai; is window mein D flat (stable) hona chahiye.
2. Edge ke  $t_{\text{pcq}}$  baad Q naya value dikhata hai.
3. **Max clock frequency (derivation):** path: FF1  $\rightarrow$  Logic  $\rightarrow$  FF2, dono same clock. Edge 1 pe FF1 data launch karta hai. (a) FF1 ka Q  $t_{\text{pcq}}$  baad aata hai. (b) Logic se guzarne mein  $t_{\text{pd}}$ . (c) FF2 pe agle edge se  $t_{\text{setup}}$  pehle pahunchna zaroori. To ek clock period mein ye teeno fit hone chahiye:  $T_c \geq t_{\text{pcq}} + t_{\text{pd}} + t_{\text{setup}}$ , aur  $f_{\text{max}} = 1 / T_c$ .
4. **Hold check:** edge pe FF1 ka naya data sabse fast path se FF2 tak pahunchne, wo FF2 ke hold time ke baad hi pahunchna chahiye:  $t_{\text{acq}} + t_{\text{cd}} \geq t_{\text{hold}}$ . Isme  $T_c$  hai hi nahi, isliye clock slow karne se hold violation theek nahi hota; short path mein buffers (delay) daalo.
5. **Metastability (ball on hill):** do ghaatiyan (0 aur 1) aur beech mein pahaadi. Normal mein gend kisi ghaati mein. Violation pe gend pahaadi ki choti pe ruk jaati hai: kab aur kis taraf giregi, pata nahi.
6. **Synchronizer:** async signal pehle FF1 mein, FF1 ka output FF2 mein, dono receiving clock pe. FF1 metastable ho bhi jaaye, to uske paas settle hone ke liye ek poora clock period hai, uske baad hi FF2 sample karta hai. Ek period ke baad bhi metastable rehne ka chance bahut hi kam hai. Metastability **khatam nahi hoti**, sirf bahut rare ho jaati hai.
7. Overclocking  $T_c$  ko chhota karta hai  $\rightarrow$  setup margin khatam  $\rightarrow$  setup failures.

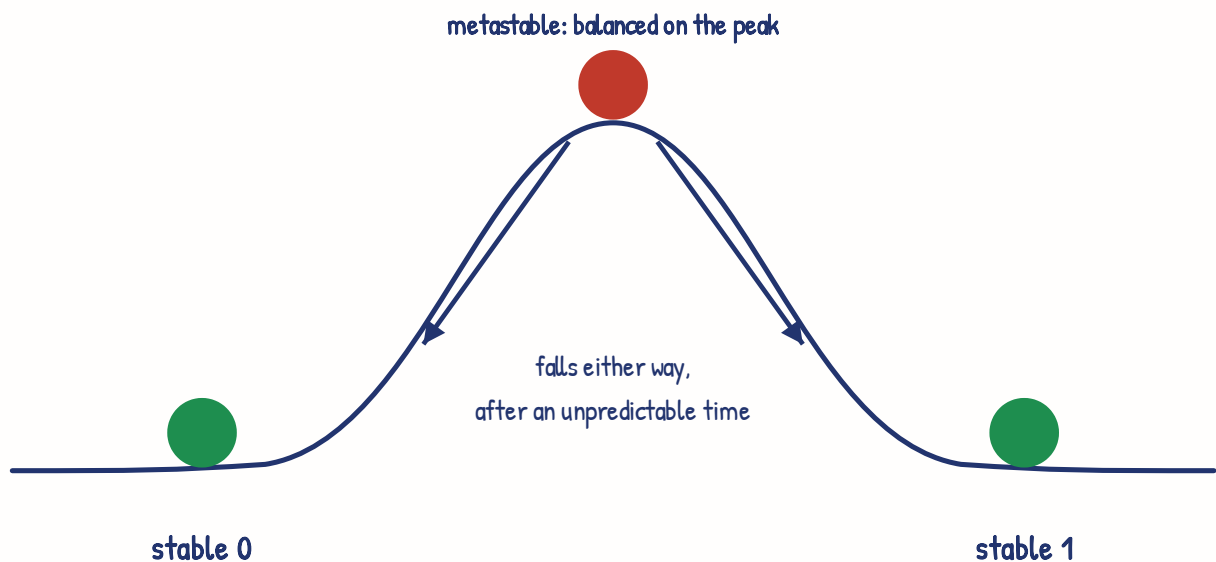
D must be stable from  $t_{\text{setup}}$  before to  $t_{\text{hold}}$  after the edge; Q valid  $t_{\text{clk-to-q}}$  later



### Setup, hold and clock-to-Q on a timing diagram

How to draw: One rising edge; shade the window before (setup) and after (hold) where D must be flat; Q changes  $t_{\text{cq}}$  after the edge.

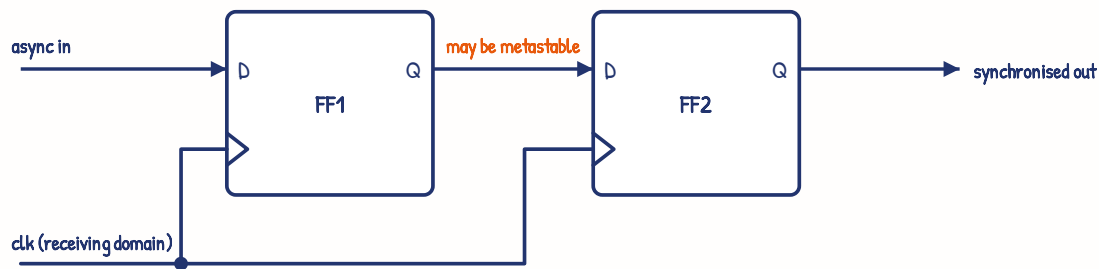
**Diagram kaise padhein:** Ek rising edge (beech mein). Edge se pehle ka shaded hissa  $t_{\text{setup}}$ , baad ka  $t_{\text{hold}}$ ; is poore hisse mein D row flat ("D stable") hai. Q row edge ke  $t_{\text{cq}}$  baad naya value ("new Q") dikhata hai.



### Metastability: the ball on the hill

How to draw: Two valleys (0 and 1) and a hill; ball on the peak = metastable.

**Diagram kaise padhein:** Do hare gend ghaatiyon mein = stable 0 aur stable 1. Laal gend choti pe = metastable. Teer dikhate hain ki wo kisi bhi taraf gir sakti hai, unpredictable time ke baad.



FF1 gets a whole clock period to settle before FF2 samples it → failure probability falls exponentially (MTBF of years).

### Two flip-flop synchronizer

How to draw: Two D FFs in series clocked by the receiving clock; the async signal enters the first.

**Diagram kaise padhein:** Async input left se FF1 ke D mein. FF1 ka Q FF2 ke D mein. Dono ko same (receiving) clock. Safe output FF2 ke Q se lo, FF1 se kabhi nahi.

**Formula:**  $T_c \geq t_{pcq} + t_{pd} + t_{setup}$   $f_{max} = 1 / T_c$  · Hold:  $t_{ccq} + t_{cd} \geq t_{hold}$

Unit trick:  $f \text{ (GHz)} = 1 / T \text{ (ns)} = 1000 / T \text{ (ps)}$ .

### Solved example 1 (class homework):

A path has 0.28 ns delay (including clock-to-Q) and setup time is 0.05 ns. Fastest safe clock?

1.  $T_{c,min} = 0.28 + 0.05 = 0.33 \text{ ns}$ .
2.  $f = 1 / 0.33 \text{ ns} = 3.03 \text{ GHz}$  ( $1 / \text{ns} = \text{GHz}$ ).

**Answer:  $\approx 3.03 \text{ GHz}$ .**

### Solved example 2 (mock NAT + hold check):

(i)  $t_{pcq} = 50 \text{ ps}$ , worst logic delay 300 ps,  $t_{setup} = 50 \text{ ps}$ . Max frequency? (ii) Another path:  $t_{ccq} = 30 \text{ ps}$ ,  $t_{cd} = 10 \text{ ps}$ ,  $t_{hold} = 50 \text{ ps}$ . Is hold met?

1. (i)  $T = 50 + 300 + 50 = 400 \text{ ps}$ .
2.  $f = 1000 / 400 = 2.5 \text{ GHz}$ .
3. (ii) Data jaldi se jaldi pahunchta hai  $30 + 10 = 40 \text{ ps}$  mein.
4. Hold chahiye 50 ps.  $40 < 50 \rightarrow$  **hold violation**.
5. Fix: short path mein kam se kam 10 ps ka buffer delay daalo. Clock slow karna kaam nahi karega.

**Answer: (i) 2.5 GHz (ii) violated; add delay to the short path.**

**Galti mat karna:** (1) "Clock slow karo, hold violation theek" galat: hold mein clock period aata hi nahi. (2)  $f_{max}$  mein  $t_{pcq}$  bhoool jaana. (3) "Synchronizer metastability eliminate kar deta hai" galat: sirf probability ghatata hai.

**Trick:** "Setup = pehle, hold = baad. Slow clock setup bachata hai, hold nahi."

**Ek line mein:** D ko edge se  $t_{\text{setup}}$  pehle se  $t_{\text{hold}}$  baad tak stable rehna chahiye;  $T_c \geq t_{\text{pcq}} + t_{\text{pd}} + t_{\text{setup}}$ ; violation se metastability, jise 2-FF synchronizer rare banata hai.

## 5.E1 JK and T flip-flops; characteristic and excitation tables

**Pehle socho:** SR flip-flop ka 11 input bekaar (forbidden) tha. Kyun na us input ko koi kaam de diya jaaye? JK flip-flop mein 11 ka matlab "ulta kar do" (toggle). Aur agar J aur K ko jod do to ek hi input T bachta hai:  $T = 1 \rightarrow \text{toggle}$ ,  $T = 0 \rightarrow \text{hold}$ . Toggle karne wala flip-flop har clock pe ulta hota hai, matlab clock ki aadhi speed: counters isi se bante hain (Unit 6).

**Definition:** A JK flip-flop behaves like an SR flip-flop ( $J = \text{set}$ ,  $K = \text{reset}$ ) except that  $J = K = 1$  toggles the output; its characteristic equation is  $Q^+ = JQ' + K'Q$ . A T (toggle) flip-flop toggles when  $T = 1$  and holds when  $T = 0$ ;  $Q^+ = T \oplus Q$ . An **excitation table** lists the inputs required to produce a given transition  $Q \rightarrow Q^+$ .

**Mushkil shabd, aasaan matlab:**

**toggle**

Ualta karna:  $Q = 0 \rightarrow 1$ ,  $Q = 1 \rightarrow 0$ .

**characteristic table / equation**

"Inputs diye, next Q kya hoga?"  $Q^+$  ka formula.

**excitation table**

Ualta sawaal: "Q ko 0 se 1 le jaana hai, input kya do?" Counter design mein use hota hai.

**X (don't care)**

0 ho ya 1, farq nahi padta.

**$Q^+$**

Clock edge ke baad ka Q (next state).  $Q(n+1)$  bhi likhte hain.

**Kaise kaam karta hai:**

- JK table:**  $JK = 00 \rightarrow \text{hold } (Q^+ = Q)$ ;  $01 \rightarrow \text{reset } (0)$ ;  $10 \rightarrow \text{set } (1)$ ;  $11 \rightarrow \text{toggle } (Q^+ = Q')$ .
- JK equation derivation:** 8 rows (J, K, Q) likho aur  $Q^+ = 1$  wali rows chuno: (J K Q) = 001, 100, 101, 110.  
Group 1:  $J = 1$  aur  $Q = 0$  (rows 100, 110; K kuch bhi)  $\rightarrow JQ'$ . Group 2:  $K = 0$  aur  $Q = 1$  (rows 001, 101; J kuch bhi)  $\rightarrow K'Q$ . Chaaron rows cover ho gayi (100 aur 110 group 1 mein; 001 aur 101 group 2 mein).  $Q^+ = JQ' + K'Q$ . Check  $J = K = 1, Q = 1$ :  $1 \cdot 0 + 0 \cdot 1 = 0$  (toggle) ✓.
- Andar se JK:** SR core ke aage AND gates:  $S = J \cdot Q'$ ,  $R = K \cdot Q$ .  $Q = 0$  ho to sirf J kaam karta hai (set ho sakta hai),  $Q = 1$  ho to sirf K (reset). Isliye  $J = K = 1$  pe jo bhi Q hai uska ulta hota hai.

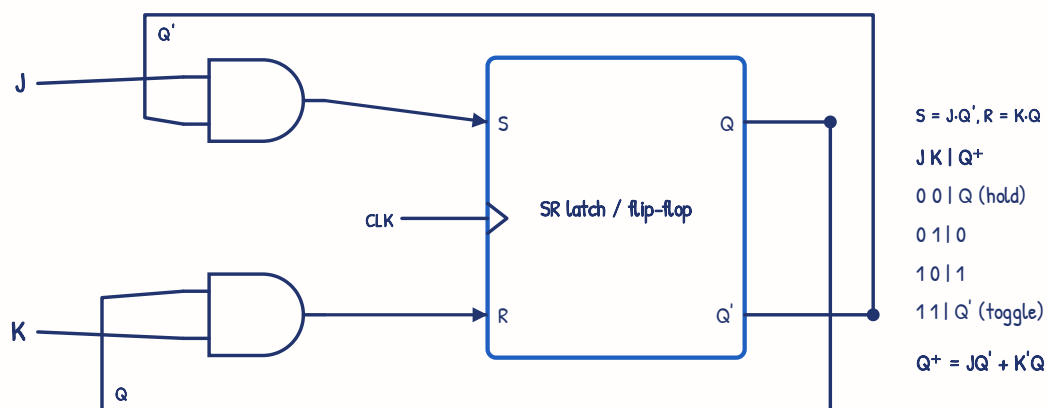
4. **T table aur equation:**  $T = 0 \rightarrow Q^+ = Q$ ;  $T = 1 \rightarrow Q^+ = Q'$ . Rows:  $(T, Q) = (0, 0) \rightarrow 0$ ;  $(0, 1) \rightarrow 1$ ;  $(1, 0) \rightarrow 1$ ;  $(1, 1) \rightarrow 0$ .  $Q^+ = 1$  jab T aur Q alag hon  $\rightarrow Q^+ = T \oplus Q (= T'Q + TQ')$ .

5. **T banana:** JK se:  $J = K = T$  (00 hold, 11 toggle). D se:  $D = T \oplus Q$  (XOR gate D ke aage).

6. **Excitation (ulta padhna):** D ke liye  $D = Q^+$  (jo chahiye wahi do). T ke liye  $T = Q \oplus Q^+$  (badalna hai to 1). JK ke liye:  $0 \rightarrow 1$  chahiye to  $J = 1$  ( $K = X$ , kyunki set ya toggle dono 1 denge);  $1 \rightarrow 0$  chahiye to  $K = 1$  ( $J = X$ ).

| J | K | $Q^+$ | Action | T | $Q^+$ | Action |
|---|---|-------|--------|---|-------|--------|
| 0 | 0 | Q     | hold   | 0 | Q     | hold   |
| 0 | 1 | 0     | reset  | 1 | $Q'$  | toggle |
| 1 | 0 | 1     | set    |   |       |        |
| 1 | 1 | $Q'$  | toggle |   |       |        |

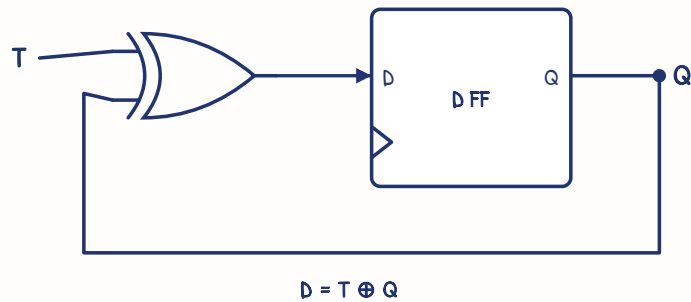
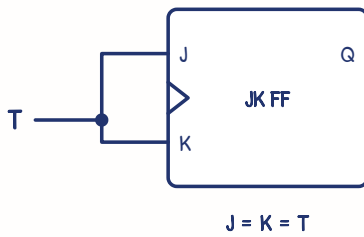
| $Q \rightarrow Q^+$ | D | T | JK | SR |
|---------------------|---|---|----|----|
| $0 \rightarrow 0$   | 0 | 0 | 0X | 0X |
| $0 \rightarrow 1$   | 1 | 1 | 1X | 10 |
| $1 \rightarrow 0$   | 0 | 1 | X1 | 01 |
| $1 \rightarrow 1$   | 1 | 0 | X0 | X0 |



#### JK flip-flop (SR core with feedback gating) + table

How to draw: Two AND gates: J with  $Q'$ , K with Q, feeding S and R of a clocked SR core; draw the feedback wires from the outputs.

**Diagram kaise padhein:** J ek AND mein  $Q'$  ke saath jaata hai (S banta hai), K doosre AND mein Q ke saath (R banta hai). Outputs se feedback wires wapas AND gates tak aate hain. Trace  $Q = 1$ ,  $J = K = 1$ :  $S = 1 \cdot 0 = 0$ ,  $R = 1 \cdot 1 = 1 \rightarrow \text{reset} \rightarrow Q = 0$  (toggle hua).



$T = 1$  toggles,  $T = 0$  holds:  $Q^+ = TQ' + T'Q = T \oplus Q$

**T flip-flop from a JK ( $J = K = T$ ) and from a D ( $D = T \oplus Q$ )**

How to draw: JK box with J and K joined; D box with an XOR of T and Q in front.

**Diagram kaise padhein:** Left: JK box jiske J aur K ek hi wire T se jude hain. Right: D box ke aage XOR gate jiske inputs T aur Q hain. Dono mein  $T = 1 \rightarrow$  toggle,  $T = 0 \rightarrow$  hold.

**Formula:** SR:  $Q^+ = S + R'Q$  ( $SR = 0$ ) · D:  $Q^+ = D$  · JK:  $Q^+ = JQ' + K'Q$  · T:  $Q^+ = T \oplus Q$

Excitation:  $D = Q^+$ ,  $T = Q \oplus Q^+$ . Uses: D for registers and pipelines, T for counters and dividers, JK general purpose; real chips and FPGAs use D flip-flops almost always.

### Solved example (5 marks):

Write the characteristic equations of SR, D, JK and T flip-flops and their excitation tables. Show how to convert a JK flip-flop into a T flip-flop.

1. List the four equations in the formula box above.
2. Draw the excitation table ( $Q \rightarrow Q^+$  vs D, T, JK, SR) above.
3. JK  $\rightarrow$  T: required behaviour  $T = 0$  hold,  $T = 1$  toggle. JK hold = 00, JK toggle = 11. So connect  $J = K = T$ .
4. Check with equation:  $Q^+ = TQ' + T'Q = T \oplus Q$  ✓.

**Answer:** as above; JK  $\rightarrow$  T by tying J and K together (D  $\rightarrow$  T:  $D = T \oplus Q$ ).

**Galti mat karna:** JK 11 ko forbidden mat likho (wo SR ka hai); JK mein 11 = toggle. Aur T excitation mein  $1 \rightarrow 1$  ke liye  $T = 0$  (koi badlav nahi), 1 nahi.

**Trick:** JK: "00 so jao, 01 zero, 10 one, 11 ulat do." T: " $T = 1 =$  toggle."

**Ek line mein:** JK = SR jisme 11 toggle hai ( $Q^+ = JQ' + K'Q$ );  $T = 1$  pe toggle ( $Q^+ = T \oplus Q$ ); excitation table batati hai transition ke liye kaunsa input chahiye.

**Exam mein is unit se kya aata hai  $\rightarrow$  kaise solve karein**

Question type

Method (steps)

|                                                      |                                                                                                                                                                                          |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Combinational vs sequential (identify / distinguish) | 1. Pucho: pichhla value chahiye? Haan → sequential. 2. Definitions + D5.1a + table + 2 examples each.                                                                                    |
| SR latch output for given S, R (NOR / NAND)          | 1. NOR: 10 set, 01 reset, 00 hold, 11 forbidden ( $Q = Q' = 0$ ). 2. NAND: active-low, 11 hold, 00 forbidden ( $Q = Q' = 1$ ). 3. Sequence: har step pe pichhla Q yaad rakho.            |
| Why is $S = R = 1$ forbidden?                        | 1. $Q = Q' = 0$ , complement rule tootna hai. 2. Dono ek saath 0 → race → unpredictable / oscillation.                                                                                   |
| D latch waveform                                     | 1. EN = 1 ke hisse shade karo. 2. Andar $Q = D$ (copy). 3. EN girne pe jo D tha wahi Q hold.                                                                                             |
| D flip-flop waveform                                 | 1. Har active edge pe line. 2. Wahan D padho. 3. Agli line tak wahi Q. 4. Beech ke D badlav ignore.                                                                                      |
| Master-slave explanation (which edge?)               | 1. Do latches, ulte clock phases, kabhi ek saath nahi khulte. 2. Master CLK = 0 pe khula → rising edge FF; class version master CLK = 1 → falling edge. 3. Diagram D5.4c.                |
| Latch vs flip-flop comparison                        | Level vs edge; transparent vs never; glitches pass vs ignored; $\sim 1\times$ vs $\sim 2\times$ gates; latch for SRAM / time borrowing, FF for registers and pipelines.                  |
| $f_{\max}$ numerical                                 | 1. $T = t_{pcq} + t_{pd} + t_{setup}$ . 2. $f = 1000 / T(\text{ps})$ GHz ya $1 / T(\text{ns})$ GHz.                                                                                      |
| Hold violation check / fix                           | 1. $t_{ccq} + t_{cd} \geq t_{hold}$ ? 2. Nahi → short path mein buffer delay; clock slow karna kaam nahi karta.                                                                          |
| Ring oscillator frequency                            | 1. $T = 2 \times N \times t_{pd}$ . 2. $f = 1 / T$ . 3. N odd hona chahiye.                                                                                                              |
| Metastability / synchronizer theory                  | 1. Definition (violation → beech mein atka output, unpredictable time). 2. Ball on hill. 3. 2-FF synchronizer: FF1 ko ek period settle karne ko milta hai. 4. Eliminate nahi, sirf rare. |
| JK / T tables, equations, conversion                 | 1. Tables ratt lo. 2. $Q^+ = 1$ wali rows se group karke equation. 3. JK → T: $J = K = T$ ; D → T: $D = T \oplus Q$ . 4. Excitation: $D = Q^+$ , $T = Q \oplus Q^+$ .                    |

# Unit 6 · Registers, Counters & Shift Registers

**Is unit ki kahani:** Unit 5 mein ek D flip-flop = 1 bit ki memory bani. Ab inhe jodte hain. **Ek line mein rakho, ek clock do** → register (poora word, jaise PC). **Thodi logic jodo jo next value banaye** → counter (0, 1, 2, 3 ginta hai; timers, PC). **Ek ka Q agle ke D mein do** → shift register (bits ek ek karke khisakte hain; serial ↔ parallel, UART). Unit 7 in registers se register file aur FSM banayega, aur Unit 13 ka datapath inhi pe chalta hai.

## 6.1 Scaling from flip-flops to registers

**Pehle socho:** Ek flip-flop sirf 1 bit yaad rakhta hai. Par CPU ko poora number yaad rakhna hai, jaise 32-bit address. To 32 flip-flops ek line mein lagao aur **sabko ek hi clock do**, taaki saare bits ek hi instant pe ek saath save hon. Jaise ek board pe 8 switches jinhe ek hi haath ek saath dabata hai.

**Definition:** A **register** is a group of  $n$  flip-flops sharing a common clock that stores an  $n$ -bit word; all  $n$  bits are captured simultaneously at the active clock edge.

**Mushkil shabd, aasaan matlab:**

**word**

Bits ka group jo ek unit ki tarah use hota hai: 8, 16, 32 ya 64 bits.

**common clock**

Ek hi CLK wire jo har flip-flop ke clock pin tak jaati hai.

**PC / IR**

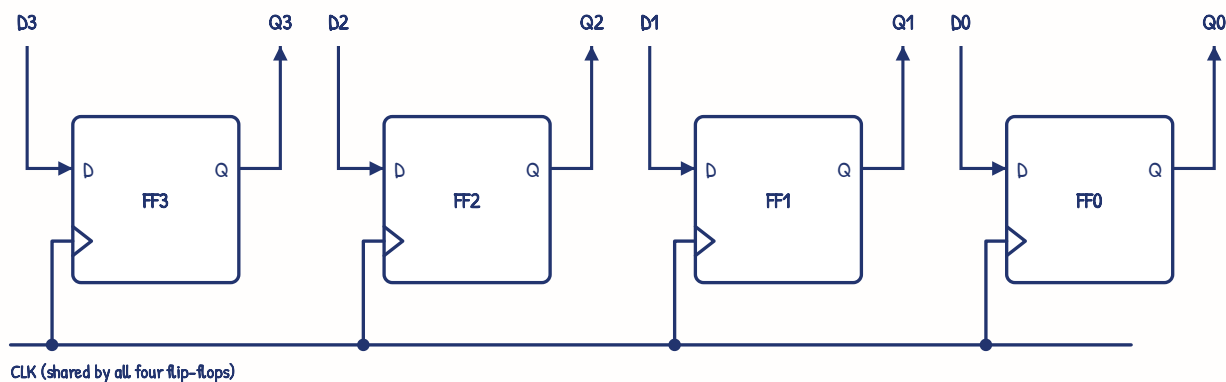
Program Counter (agle instruction ka address), Instruction Register (abhi fetch hua instruction).  
Dono registers hain.

**pipeline register**

Pipeline ke do stages ke beech ka register (Unit 14), jaise IF/ID.

**Kaise kaam karta hai:**

1.  $n$ -bit register =  $n$  D flip-flops, ek bit ke liye ek.
2. Har flip-flop ka apna D input (D0, D1, D2, D3) aur apna Q output (Q0 se Q3).
3. Clock sab ka common. Rising edge pe:  $Q_3Q_2Q_1Q_0 \leftarrow D_3D_2D_1D_0$ , ek saath.
4. Example:  $D = 1010$  aur edge aaya →  $Q = 1010$ . Agle edge pe  $D = 0110$  →  $Q = 0110$ .
5. Problem: ye plain register **har** edge pe naya D le leta hai, value hold nahi kar sakta. Isliye real registers mein **load enable** aur **reset** chahiye (6.2).
6. Cost linear hai: 32-bit register = 32 flip-flops. Isliye registers kam hote hain aur badi memory SRAM/DRAM se banti hai (Unit 8).



A register =  $n$  D flip-flops sharing one clock; all  $n$  bits are captured at the same rising edge.

#### 4-bit register (4 D flip-flops, one clock)

How to draw: Four D-FF boxes in a row, D inputs above/left, Q outputs right, one horizontal clock line underneath tapping each clock pin.

**Diagram kaise padhein:** Chahr D-FF boxes ek line mein. Har box ka D upar/left se, Q right se bahar. Neeche ek horizontal CLK line jo har box ke clock pin ko chhooti hai. Ek hi CLK line = ek hi instant pe sab load.

#### Solved example (4 marks, model answer):

*What is a register? Draw a 4-bit register using D flip-flops and name four registers found in a CPU.*

*How many flip-flops does a 32-bit register need?*

1. Definition: "A register is a group of  $n$  D flip-flops sharing one clock that stores an  $n$ -bit word, all bits being loaded at the same clock edge."
2. Draw D6.1a.
3. CPU registers: PC, IR, general-purpose registers (\$t0, \$s0), pipeline registers (IF/ID).
4. 32-bit  $\rightarrow$  ek bit ek FF  $\rightarrow$  32 flip-flops.

**Answer: definition + diagram + PC, IR, GPRs, pipeline registers; 32 flip-flops.**

**Galti mat karna:** Har bit ke liye alag clock mat banao; common clock hi register ka matlab hai. Aur Q  $\rightarrow$  D chain mat banao (wo shift register hai, 6.5). ALU register nahi hai (wo combinational hai).

**Ek line mein:** Register =  $n$  D flip-flops + ek common clock, jo  $n$ -bit word ek saath store karta hai.

## 6.2 Registers: load enable, reset, CPU preview

**Pehle socho:** Locker socho. Ghanti (clock) har second bajti hai, par locker ka saamaan tabhi badalta hai jab tum darwaza kholte ho (**LOAD = 1**). Darwaza band ho (**LOAD = 0**) to ghanti chahe kitni baar baje, andar wahi rehta hai. **RESET** = locker khaali karna (sab 0). Plain register har edge pe overwrite karta tha; load enable usse hold karna sikhata hai.

**Definition: Load enable** lets a register either capture new data or keep its present value at each clock edge; it is built with a 2:1 multiplexer in front of each flip-flop that selects D when LOAD = 1 and the flip-flop's own Q when LOAD = 0. **Reset** forces the register to a known value (all 0s): a **synchronous reset** acts at the next active clock edge, an **asynchronous reset** acts immediately.

**Mushkil shabd, aasaan matlab:**

### 2:1 MUX

Do inputs mein se ek chunne wala switch. Select = 1 → input 1 bahar, select = 0 → input 0 bahar.

### feedback (Q → MUX)

Flip-flop ka apna output MUX ke input 0 pe wapas. LOAD = 0 pe flip-flop apni hi value dobara load karta hai, yaani hold.

### LOAD (ld)

Control bit: 1 = naya data lo, 0 = purana rakho.

### synchronous / asynchronous reset

Sync: rst = 1 karke *agle edge* ka wait. Async: rst = 1 hote hi turant 0, clock ka wait nahi.

### clock gating

Clock ko AND gate se rokna. Yahan galat tareeka: delay (skew) aur glitch laata hai.

### priority

Dono ek saath aayein to kaun jeetega. Labs mein: reset > load > hold.

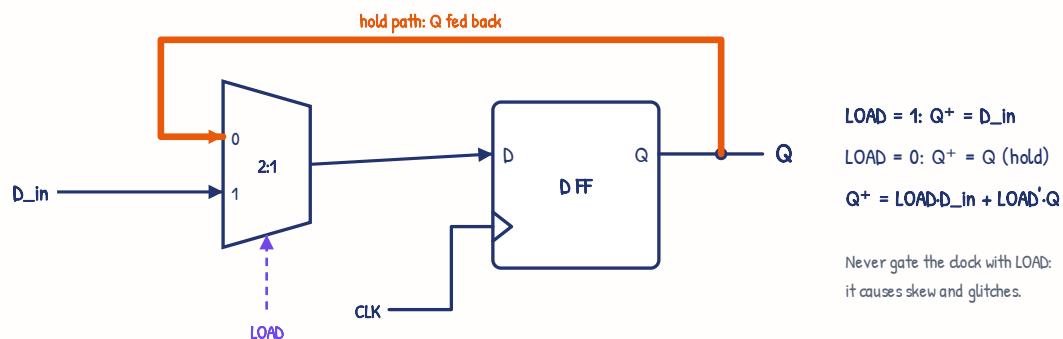
**Kaise kaam karta hai:**

1. Har bit ke D pin ke aage ek 2:1 MUX. MUX input 1 = naya data, input 0 = us flip-flop ka apna Q. Select = LOAD.
2. LOAD = 1 → MUX naya data deta hai → edge pe Q = data.
3. LOAD = 0 → MUX Q hi wapas deta hai → edge pe Q = Q (wahi value dobara) → **hold**.
4. Equation:  $Q^+ = \text{LOAD} \cdot D + \text{LOAD}' \cdot Q$  (yahi 2:1 MUX ka formula hai).
5. Clock ko kabhi mat roko (AND(CLK, LOAD) mat banao); clock saaf rakho aur MUX se decide karo.
6. Reset upar se: rst = 1 →  $Q^+ = 0$ , chahe LOAD kuch bhi ho.
7. Verilog (sync reset): `always @(posedge clk) if (rst) q <= 0; else if (ld) q <= d;` (else nahi likha → hold). Async: `always @(posedge clk or posedge rst).`

**Clock-by-clock trace (8-bit register, lab Q1 and Q2):**

| Edge / time     | rst | ld | d         | MUX chunta hai | q after                          |
|-----------------|-----|----|-----------|----------------|----------------------------------|
| edge (t = 5)    | 0   | 1  | 1010 1010 | d              | 1010 1010                        |
| edge (t = 15)   | 0   | 1  | 1100 1100 | d              | 1100 1100                        |
| edge (t = 25)   | 0   | 0  | 1111 1111 | Q (feedback)   | 1100 1100 (hold)                 |
| t = 30, no edge | 1   | 1  | 1111 1111 | (edge ka wait) | 1100 1100 (sync reset abhi nahi) |
| edge (t = 35)   | 1   | 1  | 1111 1111 | reset jeeta    | 0000 0000                        |

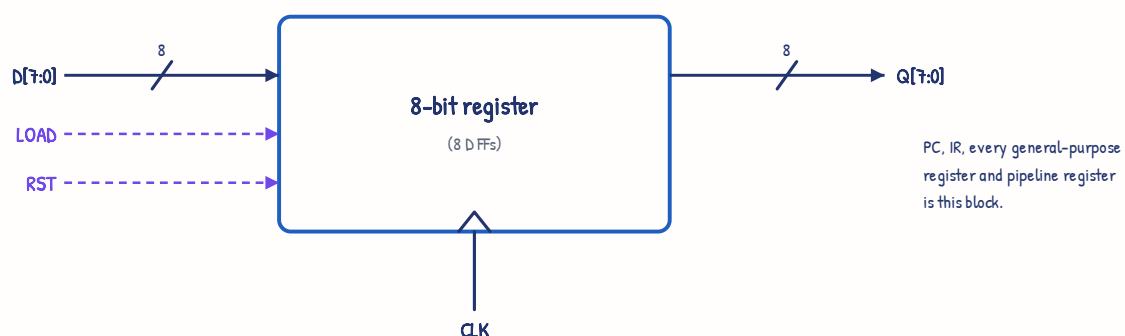
|              | Synchronous reset           | Asynchronous reset |
|--------------|-----------------------------|--------------------|
| Acts         | at the next clock edge      | immediately        |
| Needs clock? | yes                         | no                 |
| Typical use  | mod-N counters, FSM restart | power-on reset     |



**Load enable: 2:1 MUX in front of each D**

How to draw: Trapezoid MUX feeding a D FF; input 1 = new data, input 0 = wire looped back from Q; LOAD on the MUX select.

**Diagram kaise padhein:** Trapezoid = MUX, uska output D-FF ke D mein. MUX ka input "1" upar se naya data, input "0" ek wire jo FF ke Q se ghoom ke wapas aata hai. LOAD MUX ke select pe. Trace LOAD = 0, Q = 1: MUX Q = 1 bhejta hai → edge pe Q = 1 (hold).



Priority on each rising edge: RST (clear to 0) > LOAD (capture D) > hold.

**8-bit register block with LOAD and RST**

How to draw: One box labelled 8-bit register; 8-bit buses (slash 8) in and out; LOAD and RST control lines; clock triangle at the bottom.

**Diagram kaise padhein:** Ek box "8-bit register".  $D[7:0]$  bus andar,  $Q[7:0]$  bahar; bus pe tirschhi line aur "8" = 8 wires. LOAD aur RST control lines, neeche triangle = clock. PC, IR, saare registers isi block jaise dikhte hain.

### Solved example (5 marks):

Explain load enable and reset in a register. Draw one bit with a 2:1 MUX and write Verilog for an 8-bit register with synchronous reset and load.

1. Draw D6.2a; write  $Q^+ = \text{LOAD} \cdot D + \text{LOAD}' \cdot Q$ .
2. "LOAD = 1 selects the new data; LOAD = 0 feeds Q back so the register reloads its own value and holds. The clock itself is never gated."
3. Reset: "Synchronous reset clears the register at the next active edge; asynchronous reset clears it immediately without the clock (used for power-on)."
4. Verilog:

```
module register_8bit_rst (input clk, rst, ld, input [7:0] d, output reg [7:0] q);
    always @(posedge clk)
        if (rst)      q <= 8'b0;          // highest priority
        else if (ld)  q <= d;             // load
                                // else: hold
endmodule
```

**Answer:** MUX per bit with Q feedback; priority  $\text{rst} > \text{ld} > \text{hold}$ ; sync reset waits for the edge.

**Galti mat karna:** (1) LOAD = 0 pe MUX "0" bhejta hai samajhna: nahi, wo Q bhejta hai (0 bhejega to wo reset ban jaayega). (2) Sync reset ko turant maan lena:  $t = 30$  pe  $\text{rst} = 1$  hua par  $q$   $t = 35$  (edge) pe hi 0 hua. (3) if-else mein load ko reset se pehle likhna.

**Trick:** "Load band = apna hi Q wapas."

**Ek line mein:** Har bit ke aage 2:1 MUX: LOAD = 1 naya D, LOAD = 0 apna Q (hold); reset sabse upar, sync reset edge ka wait karta hai, async turant.

## 6.3 Asynchronous (ripple) vs synchronous counters

**Pehle socho:** Binary mein ginti dekho: 0111  $\rightarrow$  1000. Koi bit kab badalta hai? Jab uske neeche (right) ke saare bits 1 hon. Do tarike se ye kara sakte hain. **Ripple:** dominoes ki line, pehla gira tab doosra, phir teesra: sasta par slow, aur girte waqt beech mein galat ginti dikhti hai. **Synchronous:** soldiers jo drum beat pe sab ek saath kadam rakhte hain: thoda extra logic, par fast aur saaf.

**Definition:** A **counter** is a sequential circuit that steps through a fixed sequence of states (usually binary counting) on successive clock pulses. In an **asynchronous (ripple) counter** only the first flip-flop receives the clock and each later flip-flop is clocked by the output of the previous one; in a **synchronous counter** all flip-flops share the same clock and logic decides which bits toggle.

**Mushkil shabd, aasaan matlab:**

### T flip-flop (reminder)

$T = 1 \rightarrow$  har active edge pe ulta (toggle);  $T = 0 \rightarrow$  hold.

### propagation delay $t_{pd}$

Clock aane se Q badalne tak ka time (ek flip-flop ka). Jaise 10 ns.

### ripple

Lahar: badlav ek flip-flop se agle, phir agle tak, ek ke baad ek pahunchta hai.

### settle time

Clock ke baad sahi final count dikhne mein lagne wala time. Ripple:  $n \times t_{pd}$ .

### glitch / false state

Thodi der ke liye dikhne wala galat count (jaise 7  $\rightarrow$  8 ke beech 6, 4, 0).

### divide-by-2

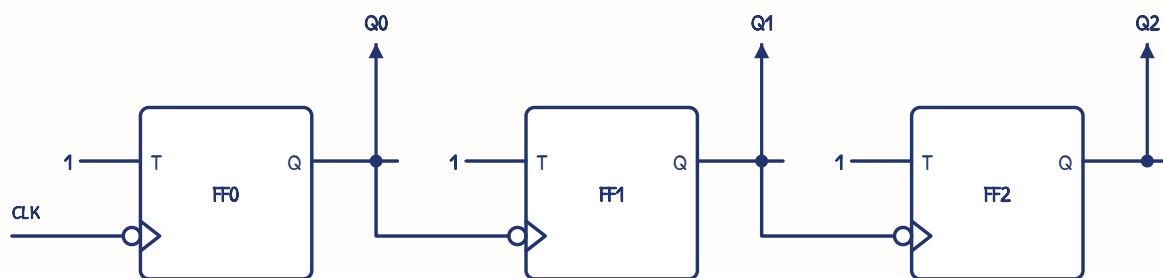
Toggle FF ka output clock ki aadhi frequency pe chalta hai: 2 clock pulses = ek poora output cycle.

**Kaise kaam karta hai:**

1. **Counting rule:** Q0 har clock pe badalta hai; Q1 tab jab Q0 = 1; Q2 tab jab Q1 = Q0 = 1. Isliye Q0 = clock  $\div 2$ , Q1 =  $\div 4$ , Q2 =  $\div 8$ ; bit k ki frequency =  $f_{clk} / 2^{k+1}$ .
2. **Ripple build:** saare T = 1 (sab toggle karne ko ready). Sirf FF0 ko clock. FF1 ka clock = Q0, FF2 ka clock = Q1. Flip-flops falling-edge (bubble) wale hain, to FF1 tab toggle karta hai jab Q0 1  $\rightarrow$  0 girta hai. Ye "neeche ke bits 1 se 0 hue = carry" hi hai.
3. **Ripple trace (3-bit, 0  $\rightarrow$  7  $\rightarrow$  0):** clock 1: Q0 0  $\rightarrow$  1  $\rightarrow$  001. Clock 2: Q0 1  $\rightarrow$  0 (gira)  $\rightarrow$  Q1 toggle 0  $\rightarrow$  1  $\rightarrow$  010. Clock 3: Q0  $\rightarrow$  1  $\rightarrow$  011. Clock 4: Q0 gira  $\rightarrow$  Q1 1  $\rightarrow$  0 gira  $\rightarrow$  Q2 0  $\rightarrow$  1  $\rightarrow$  100. Isi rule se 101, 110, 111, aur clock 8 pe teeno girte hain  $\rightarrow$  000.
4. **Ripple delay:** clock 4 jaise case mein Q2 tab badalta hai jab Q1 badal chuka, jo Q0 ke baad badla: 3 delays. n-bit mein worst case  $n \times t_{pd}$ . Clock period isse lamba hona chahiye:  $f_{max} = 1 / (n \times t_{pd})$ .
5. **False states (4-bit, 7  $\rightarrow$  8):** neeche table. Har step ek  $t_{pd}$  baad.
6. **Synchronous:** sab FFs ko same clock. T inputs AND gates se: T0 = 1, T1 = Q0, T2 = Q0·Q1, T3 = Q0·Q1·Q2 (6.4). Edge pe jo bits badalne hain sab ek saath badalte hain  $\rightarrow$  koi false state nahi; delay = ek FF delay + AND chain, n ke saath nahi badhta.

| Time after the clock edge | Kya hua              | Q3 Q2 Q1 Q0 | Count dikhta hai |
|---------------------------|----------------------|-------------|------------------|
| 0                         | purani value         | 0111        | 7                |
| $1 \times t_{pd}$         | Q0 1 $\rightarrow$ 0 | 0110        | 6 (galat)        |
| $2 \times t_{pd}$         | Q1 1 $\rightarrow$ 0 | 0100        | 4 (galat)        |
| $3 \times t_{pd}$         | Q2 1 $\rightarrow$ 0 | 0000        | 0 (galat)        |
| $4 \times t_{pd}$         | Q3 0 $\rightarrow$ 1 | 1000        | 8 (sahi)         |

|                      | Ripple (async)                                                          | Synchronous                         |
|----------------------|-------------------------------------------------------------------------|-------------------------------------|
| Clock                | FF0 only; others clocked by previous Q                                  | all FFs share one clock             |
| Delay to valid count | $n \times t_{pd}$ (grows with n)                                        | one FF delay + AND logic (constant) |
| False states         | yes (7 $\rightarrow$ 6 $\rightarrow$ 4 $\rightarrow$ 0 $\rightarrow$ 8) | no                                  |
| Hardware             | minimum (no gates)                                                      | extra AND gates                     |
| Use                  | slow dividers (watch: 32,768 Hz $\rightarrow$ 1 Hz)                     | CPUs, PC, timers, anything fast     |



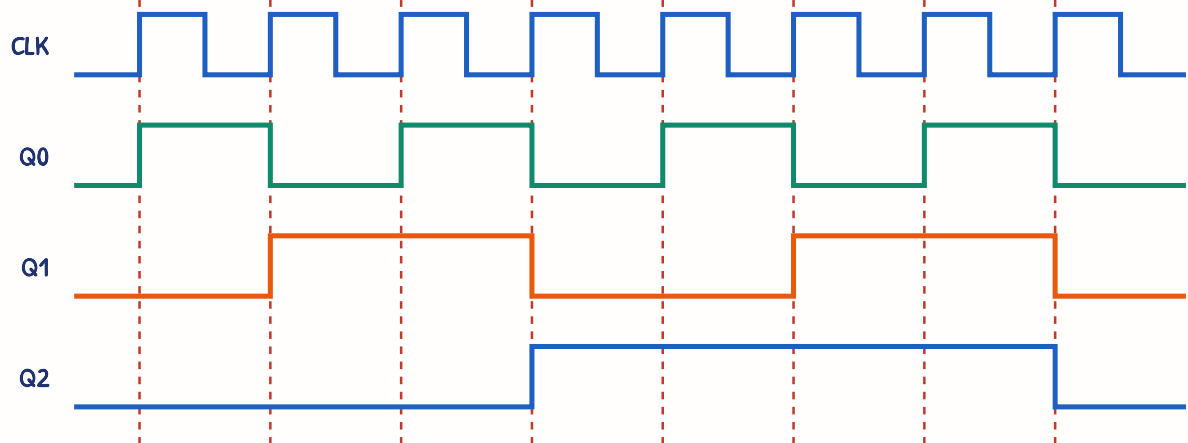
Only FF0 sees the clock. Each next FF is clocked by the previous Q (bubble = falling-edge trigger), so FF1 toggles when Q0 falls 1  $\rightarrow$  0. Bits change one after another: settle time =  $n \times t_{pd}$ .

### 3-bit ripple counter (T flip-flops, T = 1)

How to draw: Three T-FF boxes, T tied to 1; clock only into FF0; draw each Q both upward (output) and into the next FF's clock with a bubble.

**Diagram kaise padhein:** Teen T-FF boxes, sab ka T = 1. CLK sirf FF0 mein (left). Har Q do jagah jaata hai: upar output (Q0, Q1, Q2) aur agle FF ke clock pin pe (bubble = falling edge). Trace: Q0 1  $\rightarrow$  0 gira  $\rightarrow$  FF1 ko falling edge mila  $\rightarrow$  Q1 toggle.

3-bit up count 0 → 7 → 0:  $Q0 = clk \div 2$ ,  $Q1 = clk \div 4$ ,  $Q2 = clk \div 8$  (synchronous view)

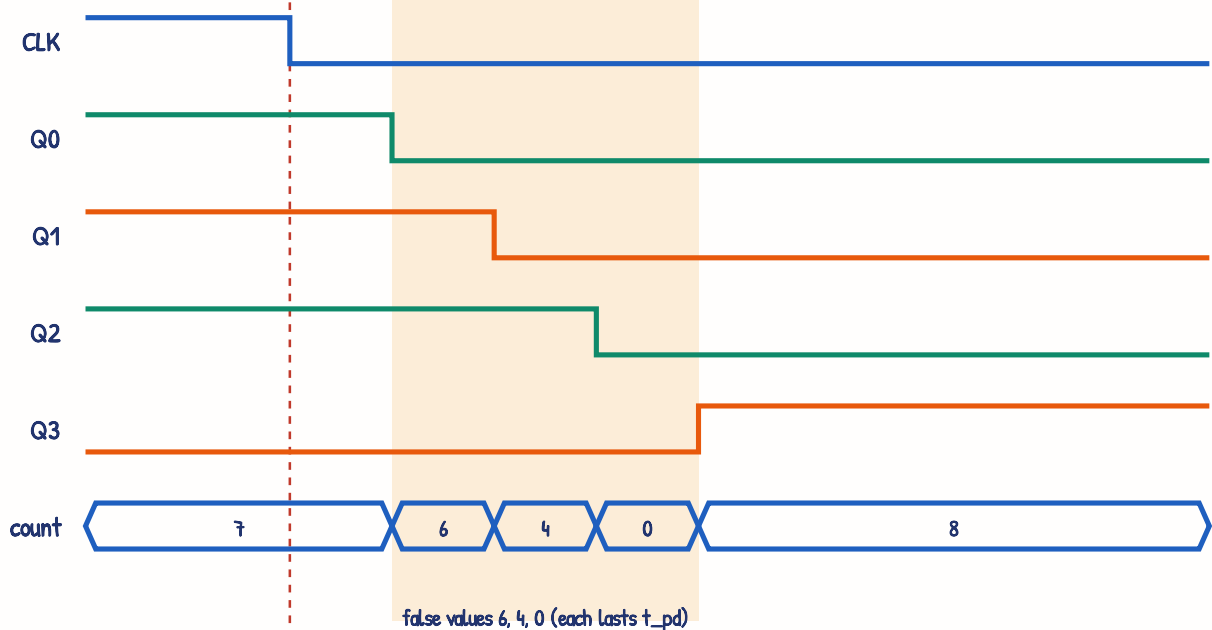


#### Counter waveforms Q0, Q1, Q2

How to draw: Clock on top; Q0 toggles at every active edge, Q1 at every second, Q2 at every fourth; read the column values to check 0..7.

**Diagram kaise padhein:** Upar clock, phir Q0, Q1, Q2. Q0 har edge pe badalta hai, Q1 har doosre pe, Q2 har chauthhe pe. Kisi bhi column mein Q2 Q1 Q0 padho: 000, 001, 010 se 111 tak, phir 000. Q0 ka ek poora cycle = 2 clock cycles ( $\div 2$ ), Q2 ka = 8 ( $\div 8$ ).

Ripple counter 0111 → 1000: each bit waits for the one before it



#### Ripple counter 7 → 8 transition timeline

How to draw: Stagger each falling Q by one  $t_{pd}$  to the right; write the momentary counts 7, 6, 4, 0, 8 underneath.

**Diagram kaise padhein:** Ek clock edge ke baad Q0, Q1, Q2 ek ke baad ek (har ek  $t_{pd}$  right shift hokar) girte hain, aur aakhir mein Q3 uthta hai. Neeche "count" row: 7, 6, 4, 0, 8. Shaded hissa = false values.

**Formula:** Ripple settle time =  $n \times t_{pd}$ ;  $f_{max} = 1 / (n \times t_{pd})$ ; bit k frequency =  $f_{clk} / 2^{k+1}$ .

1 / ns = GHz = 1000 MHz. So  $f$  (MHz) = 1000 / T (ns).

#### Solved example 1 (class quiz type):

(i) A 4-bit ripple counter has  $t_{pd} = 10$  ns. Max frequency? (ii) A 12-bit ripple counter with 0.6 ns flip-flops? (iii) Mock: 12-bit with 2 ns? (iv) 3-bit with 5 ns: worst settling time?

1. (i)  $4 \times 10 = 40$  ns  $\rightarrow f = 1000 / 40 = 25$  MHz.
2. (ii)  $12 \times 0.6 = 7.2$  ns  $\rightarrow f = 1000 / 7.2 = 138.9$  MHz  $\approx 139$  MHz.
3. (iii)  $12 \times 2 = 24$  ns  $\rightarrow f = 1000 / 24 = 41.67$  MHz.
4. (iv)  $3 \times 5 = 15$  ns.

**Answer: 25 MHz;  $\approx 139$  MHz; 41.67 MHz; 15 ns.**

### Solved example 2 (divide-by):

(i) A binary counter is driven by a 16 MHz clock. Frequency of Q2? (ii) How many T flip-flops divide a 32,768 Hz watch crystal down to 1 Hz?

1. (i)  $Q2 = f / 2^{2+1} = 16 / 8 = 2$  MHz.
2. (ii) Har T flip-flop  $\div 2$  karta hai.  $32,768 = 2^{15}$  ( $2^{10} = 1024$ ,  $\times 32 = 32,768$ ). To 15 baar  $\div 2 \rightarrow 1$  Hz  $\rightarrow 15$  flip-flops.

**Answer: 2 MHz; 15 flip-flops.**

**Galti mat karna:** (1) "Ripple counter mein clock hai, to synchronous hai" galat: clock sirf FF0 ko milta hai. (2) Ripple delay ko sirf  $t_{pd}$  likhna: ye  $n \times t_{pd}$  hai. (3) Rising-edge FF ka clock seedha Q se dena: wo down counter ban jaata hai; up ke liye falling-edge FF (ya Q') chahiye.

**Trick: "Ripple = dominoes, sync = drum pe march."** 7  $\rightarrow$  8 ripple: "7, 6, 4, 0, 8" (har baar ek aur bit gira).

**Ek line mein:** Ripple counter mein har FF pichhle ke Q se clock hota hai (sasta,  $n \times t_{pd}$  delay, false states); synchronous mein sab ek clock pe, AND logic se toggle (fast, glitch-free).

## 6.4 4-bit synchronous up-counter design

**Pehle socho:** Synchronous counter mein sab flip-flops ek saath clock hote hain, to har flip-flop ko pehle se batana padega: "agle edge pe tum badloge ya nahi?" T flip-flop ke liye ye bas T input hai:  $T = 1 \rightarrow$  badlo. Kaam: har bit ke liye T ka formula nikalna. Rule wahi: bit tab badalta hai jab neeche ke saare bits 1 hon, isliye  $T =$  neeche ke saare Q ka AND.

**Definition:** A synchronous up-counter is designed by writing the count sequence as a present-state/next-state table, finding each flip-flop input from the flip-flop's excitation table, minimising each input with a K-map, and wiring all flip-flops to a common clock. For T flip-flops this gives  $T0 = 1$ ,  $T1 = Q0$ ,  $T2 = Q0 \cdot Q1$ ,  $T3 = Q0 \cdot Q1 \cdot Q2$ .

**Mushkil shabd, aasaan matlab:**

**present state / next state**

Abhi ka count (Q) aur agle edge ke baad ka count (Q<sup>+</sup>). 0101 ka next state 0110.

**excitation table (reminder)**

Q → Q<sup>+</sup> ke liye input kya do. T flip-flop: T = Q ⊕ Q<sup>+</sup> (bit badalna hai to 1). D flip-flop: D = Q<sup>+</sup>.

**count enable (EN)**

EN = 0 → counter ruka; EN = 1 → ginta hai. Har T mein EN AND kar do.

**terminal count (tc)**

Output jo sirf aakhri count (1111) pe 1 hota hai. Agle counter ko enable karne ke kaam aata hai (cascading).

**$\lceil \log_2 N \rceil$**

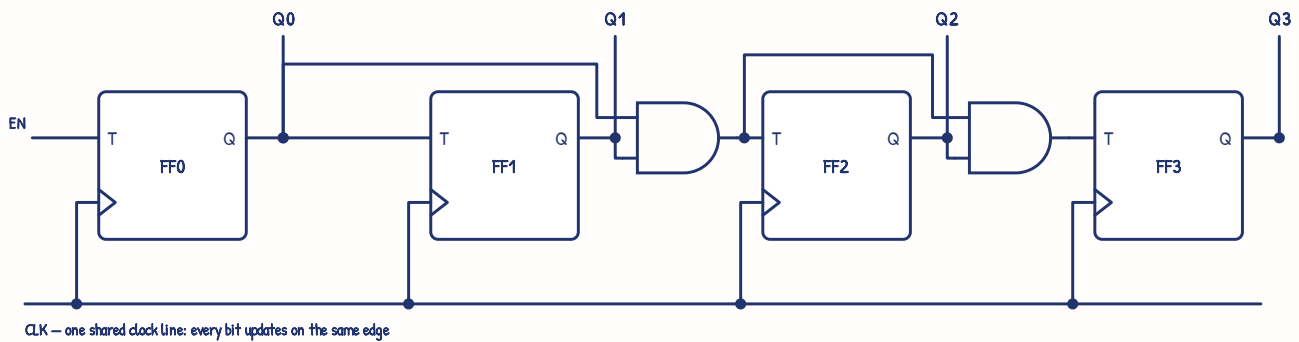
Wo sabse chhota k jisme  $2^k \geq N$ .  $N = 12 \rightarrow 2^4 = 16 \geq 12 \rightarrow 4$ .

**Kaise kaam karta hai (design steps, class flow):**

- Step 1, sequence:** 0000 → 0001 → 0010, har edge pe +1 karte hue 1111 tak, phir wapas 0000 (16 states, 4 FFs).
- Step 2, state table:** har present state ke saamne next state (present + 1) likho.
- Step 3, excitation:** har bit ke liye T = Q ⊕ Q<sup>+</sup>: jo bit badla wahan 1, jo nahi badla wahan 0.
- Step 4, minimise:** T columns dekho (neeche table). T0 column mein sab 1 → **T0 = 1**. T1 sirf un rows mein 1 jahan Q0 = 1 → **T1 = Q0**. T2 sirf jahan Q1 = Q0 = 1 (0011, 0111, 1011, 1111) → **T2 = Q1·Q0**. T3 sirf 0111 aur 1111 mein, yaani Q2 = Q1 = Q0 = 1 → **T3 = Q2·Q1·Q0**. (K-map mein bhi: T2 ke 1s 4 cells ka group banate hain jahan Q1 = Q0 = 1, Q3 aur Q2 cancel.)
- Step 5, wire:** 4 T-FFs, common clock; T0 = 1 (ya EN), T1 se Q0, ek AND (Q0, Q1) se T2, doosra AND (pichhla AND, Q2) se T3. Ye "AND chain" hai.

| Q3Q2Q1Q0 | Q <sup>+</sup> | T3 | T2 | T1 | T0 | Q3Q2Q1Q0 | Q <sup>+</sup> | T3 | T2 | T1 | T0 |
|----------|----------------|----|----|----|----|----------|----------------|----|----|----|----|
| 0000     | 0001           | 0  | 0  | 0  | 1  | 1000     | 1001           | 0  | 0  | 0  | 1  |
| 0001     | 0010           | 0  | 0  | 1  | 1  | 1001     | 1010           | 0  | 0  | 1  | 1  |
| 0010     | 0011           | 0  | 0  | 0  | 1  | 1010     | 1011           | 0  | 0  | 0  | 1  |
| 0011     | 0100           | 0  | 1  | 1  | 1  | 1011     | 1100           | 0  | 1  | 1  | 1  |
| 0100     | 0101           | 0  | 0  | 0  | 1  | 1100     | 1101           | 0  | 0  | 0  | 1  |
| 0101     | 0110           | 0  | 0  | 1  | 1  | 1101     | 1110           | 0  | 0  | 1  | 1  |
| 0110     | 0111           | 0  | 0  | 0  | 1  | 1110     | 1111           | 0  | 0  | 0  | 1  |
| 0111     | 1000           | 1  | 1  | 1  | 1  | 1111     | 0000           | 1  | 1  | 1  | 1  |

Check ek row: 0111 → 1000. Chaaron bits badle → T3T2T1T0 = 1111. Formula: T1 = Q0 = 1, T2 = Q1·Q0 = 1, T3 = Q2·Q1·Q0 = 1 ✓.



$T_0 = EN(1)$ ,  $T_1 = Q_0$ ,  $T_2 = Q_0 \cdot Q_1$ ,  $T_3 = Q_0 \cdot Q_1 \cdot Q_2$  — a bit toggles when all lower bits are 1.

#### 4-bit synchronous up-counter (T flip-flops + AND chain)

How to draw: Four T FFs in a row on one clock line;  $T_0 = EN$ ;  $T_1$  from  $Q_0$ ; an AND gate ( $Q_0, Q_1$ ) feeds  $T_2$ ; a second AND (previous AND,  $Q_2$ ) feeds  $T_3$ .

**Diagram kaise padhein:** Chaar T-FFs ek line mein, neeche ek shared CLK line. Left mein  $EN \rightarrow T_0$ .  $Q_0$  seedha  $T_1$  mein. Pehla AND gate  $Q_0$  aur  $Q_1$  leta hai  $\rightarrow T_2$ . Doosra AND pehle AND ka output aur  $Q_2$  leta hai  $\rightarrow T_3$ . Trace count = 0011:  $T_1 = 1$ ,  $AND_1 = 1 \cdot 1 = 1 \rightarrow T_2 = 1$ ,  $AND_2 = 1 \cdot 0 = 0 \rightarrow T_3 = 0$ . Edge pe  $Q_0, Q_1, Q_2$  toggle  $\rightarrow 0100 \checkmark$ .

**Formula:** Up:  $T_0 = 1$ ,  $T_1 = Q_0$ ,  $T_2 = Q_0 \cdot Q_1$ ,  $T_3 = Q_0 \cdot Q_1 \cdot Q_2$  (with enable: AND every T with EN).

D flip-flops:  $D = Q \oplus T \rightarrow D_0 = Q_0'$ ,  $D_1 = Q_1 \oplus Q_0$ ,  $D_2 = Q_2 \oplus Q_1 \cdot Q_0$ ,  $D_3 = Q_3 \oplus Q_2 \cdot Q_1 \cdot Q_0$ .

Down:  $T_0 = 1$ ,  $T_1 = Q_0'$ ,  $T_2 = Q_0' \cdot Q_1'$ . Up/down:  $T_1 = UP \cdot Q_0 + UP' \cdot Q_0'$ .  $t_c = Q_3 Q_2 Q_1 Q_0$ . FFs for mod-N =  $\lceil \log_2 N \rceil$ .

#### Solved example 1 (mock 6 marks, D flip-flops):

Design a 3-bit synchronous up-counter using D flip-flops: state table, next-state equations and circuit.

- State table:  $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 110 \rightarrow 111 \rightarrow 000$ .
- D flip-flop excitation:  $D = Q^+$ , isliye  $D_2 D_1 D_0 =$  next state column.
- Bit 0 har baar ulta:  $D_0 = Q_0'$ .
- Bit 1 tab ulta jab  $Q_0 = 1$ : "ulta jab  $X = 1$ " =  $Q \oplus X \rightarrow D_1 = Q_1 \oplus Q_0$ .
- Bit 2 tab ulta jab  $Q_1 = Q_0 = 1 \rightarrow D_2 = Q_2 \oplus (Q_1 \cdot Q_0)$ .
- Check 011:  $D_0 = 0$ ,  $D_1 = 1 \oplus 1 = 0$ ,  $D_2 = 0 \oplus 1 = 1 \rightarrow 100 \checkmark$ . Check 111:  $D_0 = 0$ ,  $D_1 = 0$ ,  $D_2 = 1 \oplus 1 = 0 \rightarrow 000 \checkmark$ .
- Circuit: 3 D-FFs on one clock; inverter  $Q_0 \rightarrow D_0$ ;  $XOR(Q_1, Q_0) \rightarrow D_1$ ;  $AND(Q_1, Q_0) \rightarrow XOR$  with  $Q_2 \rightarrow D_2$ .

**Answer:**  $D_0 = Q_0'$ ,  $D_1 = Q_1 \oplus Q_0$ ,  $D_2 = Q_2 \oplus Q_1 \cdot Q_0$ .

#### Solved example 2 (down counter + FF count):

(i) Find the T equations of a 3-bit synchronous DOWN counter. (ii) Minimum flip-flops for mod-12, mod-20 and for dividing 1 MHz to 1 kHz?

- (i) Down count mein bit tab badalta hai jab neeche ke saare bits 0 hon ( $100 \rightarrow 011$ : teeno badle).  $T_0 = 1, T_1 = Q_0', T_2 = Q_0'Q_1'$ .
- Check  $100 \rightarrow 011$ :  $T_0 = 1, T_1 = 1 (Q_0 = 0), T_2 = 1 \cdot 1 = 1 \rightarrow$  teeno toggle  $\rightarrow 011 \checkmark$ . Check  $010 \rightarrow 001$ :  $T_1 = 1, T_2 = Q_0'Q_1' = 1 \cdot 0 = 0 \rightarrow Q_1, Q_0$  toggle  $\rightarrow 001 \checkmark$ .
- (ii) mod-12:  $2^3 = 8 < 12 \leq 16 = 2^4 \rightarrow$  4. mod-20:  $16 < 20 \leq 32 \rightarrow$  5.  $1 \text{ MHz} \rightarrow 1 \text{ kHz} = \div 1000 \rightarrow$  mod-1000:  $512 < 1000 \leq 1024 \rightarrow$  10.

**Answer:**  $T_0 = 1, T_1 = Q_0', T_2 = Q_0'Q_1'$ ; 4, 5, 10 flip-flops.

**Galti mat karna:**  $T_2 = Q_1$  likhna (galat;  $T_2 = Q_0Q_1$ , saare neeche ke bits). Down counter mein Q ki jagah  $Q'$  use karna bhoool jaana. Mod-N ke FFs  $N/2$  ya  $\log_2 N$  ko round-down karna: hamesha round up.

**Trick:** "Neeche sab 1  $\rightarrow$  main palat jaata hoon" (up). Down mein "neeche sab 0".

**Ek line mein:** Synchronous up-counter: common clock,  $T_0 = 1$  aur har agle  $T_k =$  neeche ke saare Q ka AND ( $T_1 = Q_0, T_2 = Q_0Q_1, T_3 = Q_0Q_1Q_2$ ).

## 6.5 Shift registers: SISO, SIPO, PISO, PIPO

**Pehle socho:** Aag bujhane wali bucket line socho: har whistle (clock) pe har aadmi apni bucket agle ko de deta hai. Bits bhi aise hi ek flip-flop se agle mein khisakte hain. Kyun chahiye? Bahut se cables sirf **ek wire** pe data bhejte hain (USB, UART): CPU ke paas 8-bit byte hai (parallel) aur wire pe ek ek bit bhejna hai (serial). Shift register yahi badalta hai: parallel  $\leftrightarrow$  serial.

**Definition:** A **shift register** is a chain of flip-flops on a common clock in which the output of each flip-flop drives the input of the next, so the stored bits move one position per clock pulse. By how data enters and leaves it is classified as SISO, SIPO, PISO or PIPO.

**Mushkil shabd, aasaan matlab:**

**serial**

Ek wire, ek time pe ek bit. 4 bits bhejne mein 4 clocks.

**parallel**

n wires, saare bits ek saath, 1 clock.

**SISO / SIPO / PISO / PIPO**

Serial-In Serial-Out, Serial-In Parallel-Out, Parallel-In Serial-Out, Parallel-In Parallel-Out. Pehla shabd = data kaise andar, doosra = kaise bahar.

**UART**

Serial communication port. Transmitter (TX) = PISO, Receiver (RX) = SIPO.

**Shift/Load' (sh\_ld)**

PISO ka control: ek value pe parallel load, doosri pe shift. Portal code: sh\_ld = 0 load, 1 shift.

### delay line

Signal ko kuch clocks late karna. 4-bit SISO = 4 clock delay.

### Kaise kaam karta hai:

1. **SISO:** 4 D-FFs chain: serial in  $\rightarrow$  FF0  $\rightarrow$  FF1  $\rightarrow$  FF2  $\rightarrow$  FF3  $\rightarrow$  serial out (Q3). Har edge pe har bit ek aage. Verilog: `q <= {q[2:0], serial_in}; serial_out = q[3];`
2. **SIPO:** wahi chain, par har Q bahar nikala (taps). 4 clocks baad poora word Q3Q2Q1Q0 pe ek saath padh lo.
3. **PISO:** har D ke aage 2:1 MUX. Load mode: MUX parallel bit chunta hai, ek edge pe poora word andar. Shift mode: MUX pichhle FF ka Q chunta hai, har edge pe ek bit bahar. Plain chain parallel data le hi nahi sakti, isliye MUX zaroori.
4. **PIPO:** koi chain nahi; ye bas load wala register hai (buffer).

**SISO / SIPO clock-by-clock trace** (4-bit, new bit enters q[0], shifts toward q[3], start 0000, input 1, 0, 1, 1, 0, 0, 0, 0):

| Edge | serial in | q3 q2 q1 q0 | serial out (q3)                |
|------|-----------|-------------|--------------------------------|
| 1    | 1         | 0001        | 0                              |
| 2    | 0         | 0010        | 0                              |
| 3    | 1         | 0101        | 0                              |
| 4    | 1         | 1011        | 1 (pehla 1 bahar, 4th edge pe) |
| 5    | 0         | 0110        | 0                              |
| 6    | 0         | 1100        | 1                              |
| 7    | 0         | 1000        | 1                              |
| 8    | 0         | 0000        | 0                              |

Edges 1 to 4 SIPO view hain: stream 1, 0, 1, 1  $\rightarrow$  q = 0001, 0010, 0101, **1011** (poora word ek saath). SISO view: serial out pe input wahi pattern 4 clocks late aata hai (1, 0, 1, 1 edges 4 se 7 pe).

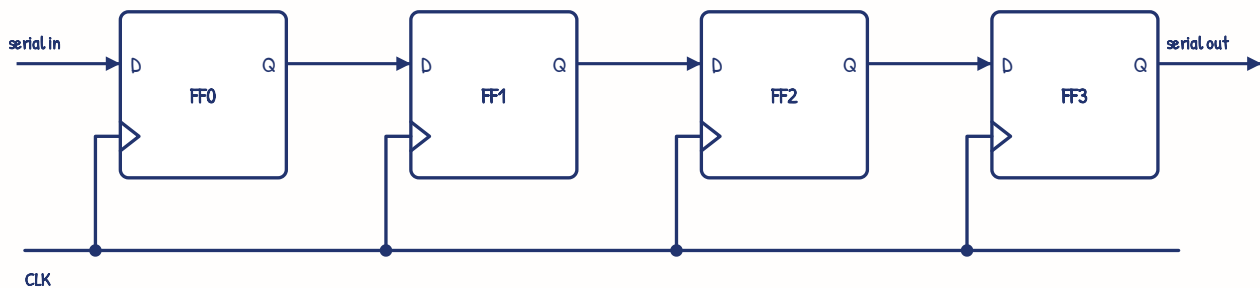
**PISO trace** (portal: sh\_ld = 0 load, 1 shift right, d\_out = q[0], 0 enters from the left):

| Edge | sh_ld         | q3 q2 q1 q0 | d_out = q0 |
|------|---------------|-------------|------------|
| 1    | 0 (load 1011) | 1011        | 1          |
| 2    | 1 (shift)     | 0101        | 1          |
| 3    | 1             | 0010        | 0          |
| 4    | 1             | 0001        | 1          |
| 5    | 1             | 0000        | 0          |

d\_out = 1, 1, 0, 1: ye 1011 ke bits **right se** hain (q0, q1, q2, q3), yaani **LSB pehle bahar**. Class study-pack ke problem title mein "MSB first" likha hai, par d\_out = q[0] aur right shift ke saath LSB hi pehle nikalta hai (slide error, LSB first sahi).

| Type | Input  | Output | Clocks for n bits | Use                                  |
|------|--------|--------|-------------------|--------------------------------------|
| SISO | 1bit   | 1bit   | n                 | delay line                           |
| SIPO | 1bit   | n bits | n                 | UART receiver (serial → parallel)    |
| PISO | n bits | 1bit   | 1 load + n shifts | UART transmitter (parallel → serial) |
| PIPO | n bits | n bits | 1                 | buffer / register                    |

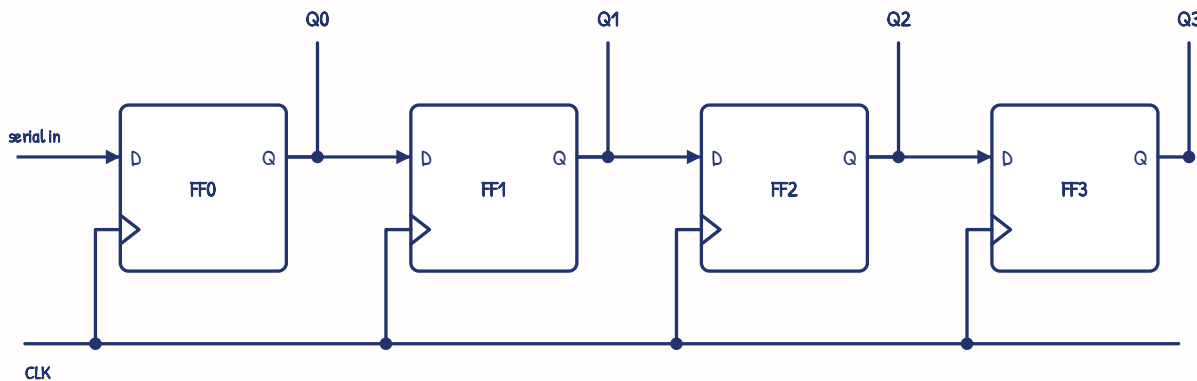
Extra uses: shift left =  $\times 2$ , shift right =  $\div 2$  (logical), LFSR random numbers (6.E1).



SISO: Q of each FF drives D of the next. A bit entering now appears at serial out 4 clocks later (delay line).

**SISO (4-bit)**  
 How to draw: Four D FFs in a row, Q → D arrows between them, serial in on the left, serial out on the right, shared clock below.

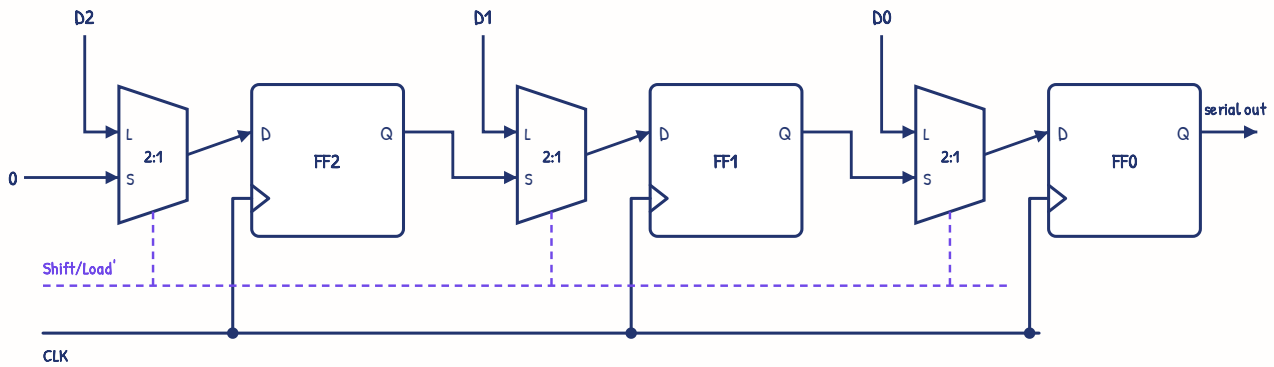
**Diagram kaise padhein:** Chaar D-FFs left se right (FF0 se FF3). Har FF ka Q teer se agle ke D mein. Left mein serial in, right mein serial out. Neeche common CLK. Ek bit ko serial out tak pahunchne mein 4 edges.



SIPO: same chain, every Q tapped. After 4 clocks the whole word is on Q0-Q3 at once (UART receiver).

**SIPO (4-bit)**  
 How to draw: Same as SISO, then draw a vertical tap from every Q up to labelled outputs Q0-Q3.

**Diagram kaise padhein:** SISO jaisa hi, bas har Q se ek wire upar (Q0, Q1, Q2, Q3). Ye taps hain jahan se poora word ek saath padhte ho.

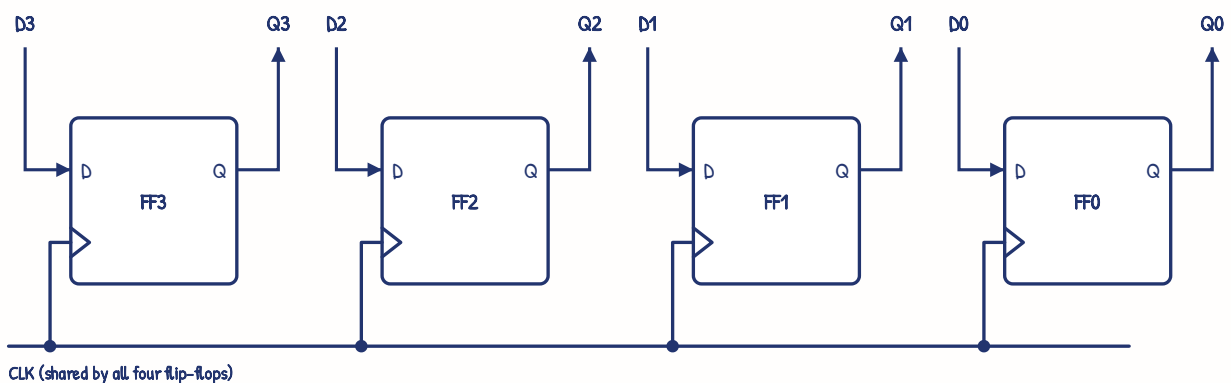


PISO (3 of 4 bits shown): a 2:1 MUX before each D picks the parallel bit (L = load) or the neighbour's Q (S = shift).

#### PISO with a load/shift MUX before each D

How to draw: Put a 2:1 MUX in front of each D: input L from the parallel bit above, input S from the previous FF's Q; one Shift/Load' line to every select.

**Diagram kaise padhein:** Har D-FF se pehle ek MUX. MUX ka input L upar se parallel bit (D2, D1, D0), input S left wale FF ka Q (sabse left MUX ke S pe 0). Neeche Shift/Load' line har MUX ke select pe. Right wale FF (FF0) ka Q serial out hai. Load pe saare L ek saath andar; shift pe har bit ek jagah right, aur FF0 ka bit bahar.



PIPO: no chain at all — D3..D0 loaded in parallel on one edge, Q3..Q0 read in parallel (buffer / register).

#### PIPO (4-bit)

How to draw: Four independent D FFs: parallel inputs from the top, parallel outputs on the right, one clock.

**Diagram kaise padhein:** Chaar alag D-FFs, koi Q → D connection nahi. D3 se D0 upar se andar, Q3 se Q0 right se bahar, ek clock. Ek edge pe poora word load.

#### Solved example 1 (5 marks):

Draw SISO, SIPO, PISO and PIPO shift registers (4-bit) and state one application of each. Show the SIPO contents for the input stream 1011.

1. Draw D6.5a, D6.5b, D6.5c, D6.5d.
2. Uses: SISO delay line; SIPO UART receiver; PISO UART transmitter; PIPO buffer register.
3. SIPO trace (start 0000, new bit into q0): after 1 → 0001; after 0 → 0010; after 1 → 0101; after 1 → 1011.

**Answer: four diagrams + uses; SIPO = 0001, 0010, 0101, 1011.**

### Solved example 2 (lab MCQ type):

(i) A 4-bit SIPO starts at 0000; bits 1, 1, 0, 1 arrive. Final  $q[3:0]$ ? (ii) A PISO loads 1100 and shifts right with  $d\_out = q[0]$ . First four  $d\_out$  values (including the load edge)?

1. (i) Har edge pe  $q = \{q[2:0], \text{new bit}\}$ : 0001  $\rightarrow$  0011  $\rightarrow$  0110  $\rightarrow$  1101.

2. (ii) Load:  $q = 1100$ ,  $d\_out = q[0] = 0$ . Shift: 0110  $\rightarrow$   $d\_out$  0. Shift: 0011  $\rightarrow$  1. Shift: 0001  $\rightarrow$  1.

**Answer:** (i) 1101 (ii) 0, 0, 1, 1.

**Galti mat karna:** (1) PISO output ko "MSB first" likhna jab  $d\_out = q[0]$  hai: LSB first. (2) PISO bina MUX ke draw karna. (3) Verilog mein blocking = se shift likhna:  $q[1] = q[0]$  naya  $q[0]$  copy kar lega aur saare bits  $d\_in$  ban jaayenge;  $\leq$  use karo.

**Trick:** "Receive = SIPO, Transmit = PISO." Naam ka pehla hissa = andar kaise, doosra = bahar kaise.

**Ek line mein:** Shift register = D-FFs ki chain ( $Q \rightarrow D$ , common clock); SISO delay, SIPO serial  $\rightarrow$  parallel (RX), PISO parallel  $\rightarrow$  serial with load MUX (TX), PIPO buffer.

## 6.E1 Ring, Johnson, mod-N, divide-by-N, LFSR

**Pehle socho:** Shift register ke last output ko wapas pehle input mein jod do to bits gol gol ghoomte rehte hain, jaise musical chairs. Isse aise counters bante hain jinhe decode karna aasaan hai (ring, Johnson). Kabhi 16 states nahi, sirf 6 ya 10 chahiye (ghadi ke seconds, decimal digit): mod-N counter. Aur feedback mein XOR daalo to "random jaisa" sequence milta hai (LFSR), jo testing aur CRC mein kaam aata hai.

**Definition:** A **ring counter** is a shift register whose last output is fed back to its first input, circulating a single 1 through  $n$  states. A **Johnson (twisted-ring) counter** feeds back the complement of the last output and has  $2n$  states. A **mod-N counter** counts 0 to  $N - 1$  and then returns to 0. An **LFSR** (linear feedback shift register) feeds back the XOR of selected bits; a maximal-length  $n$ -bit LFSR cycles through  $2^n - 1$  non-zero states.

### Mushkil shabd, aasaan matlab:

#### preset

Shuru mein ek fixed value daal dena (ring counter mein 0001). Bina preset ke 0000  $\rightarrow$  0000 hi ghoomta rahega.

#### one-hot

Exactly ek bit 1 (0100). Ring counter ka har state one-hot hai, isliye decoder ki zarurat nahi.

#### mod-N

$N$  states wala counter. Mod-6: 0, 1, 2, 3, 4, 5, 0.

### decode & reset

Jab counter N pe pahunche, AND gate se pehchaano aur clear kar do.

### unused / invalid states

Patterns jo sequence mein kabhi nahi aate. 4-bit Johnson:  $16 - 8 = 8$ .

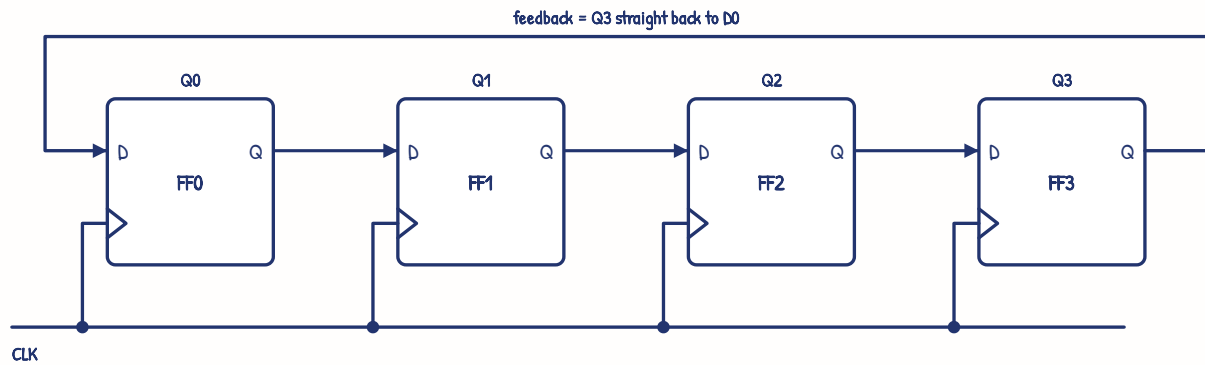
### tap

LFSR ke wo bits jinka XOR feedback banta hai.

### Kaise kaam karta hai:

1. **Ring counter (4-bit):** SISO chain +  $Q_3 \rightarrow D_0$ . Preset 0001. Har edge pe 1 ek jagah left khisakta hai,  $Q_3$  ka bit wapas  $Q_0$  mein:  $0001 \rightarrow 0010 \rightarrow 0100 \rightarrow 1000 \rightarrow 0001$ . **n FFs = n states (4)**.
2. **Johnson counter (4-bit):** feedback  $Q_3'$  (ulta)  $\rightarrow D_0$ . Start 0000. Trace: 0000 ( $Q_3 = 0$ ,  $Q_3' = 1$  andar)  $\rightarrow 0001 \rightarrow 0011 \rightarrow 0111 \rightarrow 1111$  (ab  $Q_3 = 1$ , andar 0 jaata hai)  $\rightarrow 1110 \rightarrow 1100 \rightarrow 1000 \rightarrow 0000$ . **8 states =  $2n$** . Har step mein sirf ek bit badalta hai, isliye 2-input gate se glitch-free decode.
3. **3-bit Johnson (mock):**  $000 \rightarrow 001 \rightarrow 011 \rightarrow 111 \rightarrow 110 \rightarrow 100 \rightarrow 000$ : 6 states =  $2 \times 3$ .
4. **Mod-N, method A (detect N, async reset):** normal binary counter, aur jaise hi count N banta hai, AND gate use pehchaan ke saare FFs clear kar deta hai. N thodi der (few ns) ke liye dikhta hai: glitch state.
5. **Mod-N, method B (detect N - 1, synchronous reset / parallel load):** jab count N - 1 ho, agle edge pe 0 load karo. Koi glitch state nahi, isliye production designs mein preferred. Verilog: `if (q == N-1) q <= 0; else q <= q + 1;`
6. **Divide-by-N:** mod-N counter ka terminal count (ya MSB) har N clocks mein ek baar  $\rightarrow f / N$ . Binary counter ka bit  $k = f / 2^{k+1}$ .
7. **LFSR (4-bit,  $x' + x^3 + 1$ ):** shift left, naya  $Q_0 = Q_3 \oplus Q_2$ . Start 0001:  $0001 \rightarrow 0010 \rightarrow 0100 \rightarrow 1001 \rightarrow 0011 \rightarrow 0110 \rightarrow 1101 \rightarrow 1010 \rightarrow 0101 \rightarrow 1011 \rightarrow 0111 \rightarrow 1111 \rightarrow 1110 \rightarrow 1100 \rightarrow 1000 \rightarrow 0001$ . 15 states =  $2^4 - 1$ . 0000 kabhi nahi aata: 0000 se  $0 \oplus 0 = 0$ , hamesha 0000 (lock-up), isliye wo exclude.

| Counter (n FFs)        | States    | n = 4 | Feedback                     |
|------------------------|-----------|-------|------------------------------|
| Binary (ripple / sync) | $2^n$     | 16    | AND / adder logic            |
| Ring                   | n         | 4     | $Q_{(n-1)} \rightarrow D_0$  |
| Johnson                | $2n$      | 8     | $Q_{(n-1)}' \rightarrow D_0$ |
| LFSR (maximal)         | $2^n - 1$ | 15    | XOR of taps                  |

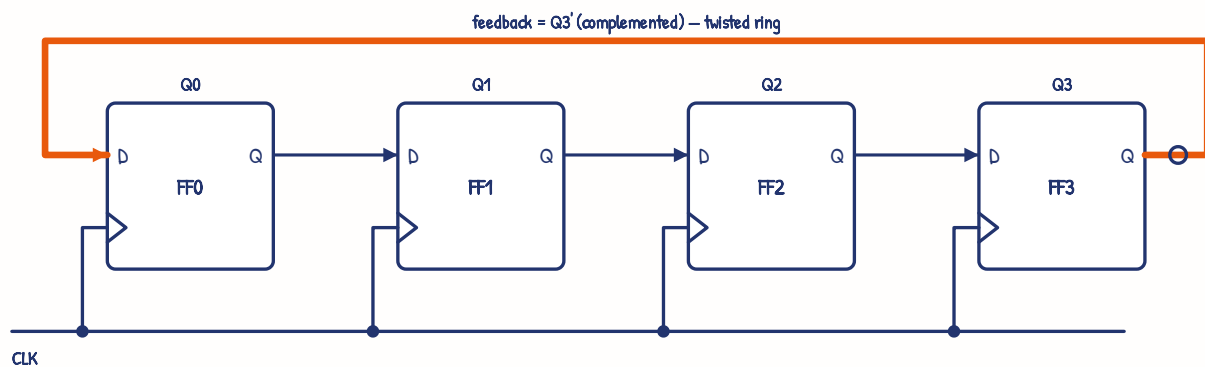


Ring (preset 0001): 0001 → 0010 → 0100 → 1000 → 0001 ( $n = 4$  states, one-hot).

#### 4-bit ring counter

How to draw: Four D FFs in a SISO chain; wire Q3 back to D0 over the top; write the preset 0001 and the 4-state sequence.

**Diagram kaise padhein:** SISO jaisi chain FF0 se FF3, aur upar se ek lambi wire Q3 se wapas D0 tak. Preset 0001 likha hai. Trace: 1000 pe Q3 = 1 wapas D0 mein → agla 0001.



Johnson: 0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → 1000 → 0000 ( $2n = 8$  states).

#### 4-bit Johnson (twisted-ring) counter

How to draw: Same chain, but the feedback comes from Q3' (draw a bubble); list the 8 states.

**Diagram kaise padhein:** Same chain, par feedback Q3' (bubble) se. 8 states ki list dekho: pehle left se 1 bharte hain (0001, 0011, 0111, 1111), phir 0 bharte hain (1110, 1100, 1000, 0000).

#### Solved example 1 (mod-6 counter):

*Design a mod-6 counter (0 to 5) from a binary up-counter. Give the reset logic for both methods and the counting sequence.*

- FFs:  $\lceil \log_2 6 \rceil = 3$  ( $2^3 = 8 \geq 6$ ).
- Sequence: 000 → 001 → 010 → 011 → 100 → 101 → 000 (6 states).
- Method A (class: reset at 6):**  $6 = 110$ . 0 se 6 tak sirf 6 mein Q2 = Q1 = 1 (7 = 111 kabhi aata hi nahi). To CLR =  $Q_2 \cdot Q_1 \rightarrow$  async clear. 110 bas kuch ns dikhta hai, phir 000.
- Method B (sync, no glitch):**  $5 = 101$ . 0 se 5 mein sirf 5 mein Q2 = Q0 = 1. Jab  $Q_2 \cdot Q_0 = 1 \rightarrow$  agle edge pe 000 load.
- Check method B: 100 pe Q0 = 0 → normal count → 101. 101 pe  $Q_2 \cdot Q_0 = 1 \rightarrow$  edge pe 000 ✓.

**Answer:** 3 FFs; async clear =  $Q_2 \cdot Q_1$  (detect 6) or sync reset when  $Q_2 \cdot Q_0 = 1$  (detect 5); sequence 0 to 5.

### Solved example 2 (mod-10 / decade counter):

*Design a mod-10 counter. How many flip-flops, and what reset logic?*

1. FFs:  $2^3 = 8 < 10 \leq 16 = 2^4 \rightarrow 4$  FFs.
2. Sequence 0000 to 1001 (0 to 9), phir 0000.
3. Method A: 10 = 1010. Counting upar jaate hue  $Q_3 = Q_1 = 1$  pehli baar 1010 pe hota hai (8 = 1000, 9 = 1001 mein  $Q_1 = 0$ ). CLR =  $Q_3 \cdot Q_1$ .
4. Method B: 9 = 1001. 0 se 9 mein sirf 9 mein  $Q_3 = Q_0 = 1$  (8 = 1000 mein  $Q_0 = 0$ ). Sync reset jab  $Q_3 \cdot Q_0 = 1$ .
5. Unused states:  $16 - 10 = 6$  (1010 se 1111).

**Answer:** 4 FFs; CLR =  $Q_3 \cdot Q_1$  (async, detect 10) or sync reset on  $Q_3 \cdot Q_0$  (detect 9).

### Solved example 3 (quiz NAT/MCQ):

*(i) States of a 5-bit Johnson counter? (ii) Unused states of a 4-bit Johnson counter? (iii) States of an 8-bit maximal LFSR? (iv) A 4-bit ring counter preset to 1000 rotating left: next state?*

1. (i)  $2n = 2 \times 5 = 10$ .
2. (ii)  $2^n - 2 \times 4 = 16 - 8 = 8$ .
3. (iii)  $2^8 - 1 = 255$ .
4. (iv) Rotate left:  $\{q[2:0], q[3]\} = 000$  followed by 1  $\rightarrow 0001$ .

**Answer:** 10; 8; 255; 0001.

**Galti mat karna:** (1) Ring counter ko preset na karna (0000 hamesha 0000). (2) Johnson ke states  $2^n$  likhna:  $2n$  hai. (3) LFSR ke  $2^n$  states likhna: all-zero exclude  $\rightarrow 2^n - 1$ . (4) "Decode & reset" ko glitch-free bolna: async clear mein N thodi der dikhta hai; parallel-load / sync method glitch-free hai.

**Trick:** 4 FFs ke liye "4, 8, 15, 16" = ring, Johnson, LFSR, binary.

**Ek line mein:** Ring = n states ( $Q_3 \rightarrow D_0$ ), Johnson =  $2n$  ( $Q_3' \rightarrow D_0$ ), LFSR =  $2^n - 1$  (XOR feedback); mod-N = count N (async clear) ya  $N - 1$  (sync reset, no glitch) detect karke 0 pe wapas.

**Exam mein is unit se kya aata hai  $\rightarrow$  kaise solve karein**

| Question type | Method (steps) |
|---------------|----------------|
|---------------|----------------|

|                                                    |                                                                                                                                                                                                                                     |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Register definition / FF count                     | 1. $n$ D-FFs + common clock. 2. $n$ -bit $\rightarrow n$ FFs. 3. D6.1a + PC, IR, GPRs, pipeline registers.                                                                                                                          |
| Load enable / reset (MUX, priority, sync vs async) | 1. 2:1 MUX per bit, $\text{LOAD} = 0 \rightarrow Q$ feedback. 2. $Q^+ = \text{LOAD} \cdot D + \text{LOAD}' \cdot Q$ . 3. $\text{rst} > \text{ld} > \text{hold}$ . 4. Sync waits for edge, async immediate. 5. Never gate the clock. |
| Ripple counter $f_{\max}$ / settle time            | 1. $T = n \times t_{pd}$ . 2. $f = 1000 / T(\text{ns})$ MHz. 3. 3-bit, 5 ns $\rightarrow$ 15 ns.                                                                                                                                    |
| Ripple 7 $\rightarrow$ 8 transient values          | Bits right se ek ek girte hain: $0111 \rightarrow 0110 \rightarrow 0100 \rightarrow 0000 \rightarrow 1000 = 7, 6, 4, 0, 8$ .                                                                                                        |
| Ripple vs synchronous comparison                   | Clock, delay, glitches, hardware, use (table in 6.3) + D6.3a and D6.4a.                                                                                                                                                             |
| Divide-by / frequency of bit $k$                   | 1. $f_k = f / 2^{k+1}$ . 2. $\div 2^m \rightarrow m$ FFs (32, 768 $\rightarrow$ 15).                                                                                                                                                |
| Synchronous counter design (T or D FFs)            | 1. Sequence. 2. PS/NS table. 3. $T = Q \oplus Q^+$ (ya $D = Q^+$ ). 4. Columns dekh ke / K-map: $T_0 = 1, T_1 = Q_0, T_2 = Q_0Q_1, T_3 = Q_0Q_1Q_2$ . 5. D version: $D_k = Q_k \oplus T_k$ . 6. Circuit + waveform.                 |
| Down / up-down counter equations                   | Down: bit badalta jab neeche sab 0: $T_1 = Q_0', T_2 = Q_0'Q_1'$ . Up/down: $T_1 = \text{UP} \cdot Q_0 + \text{UP}' \cdot Q_0'$ .                                                                                                   |
| Min FFs for mod- $N$                               | Smallest $k$ with $2^k \geq N$ : mod-12 $\rightarrow 4$ , mod-20 $\rightarrow 5$ , mod-1000 $\rightarrow 10$ .                                                                                                                      |
| Shift register trace (SISO / SIPO / PISO)          | 1. Direction note karo (naya bit kahan aata hai). 2. Clock-by-clock table banao. 3. PISO with $d_{\text{out}} = q[0]$ : LSB first.                                                                                                  |
| Which shift register for UART TX / RX              | TX = PISO, RX = SIPO, delay line = SISO, buffer = PIPO.                                                                                                                                                                             |
| Ring / Johnson / LFSR states                       | Ring $n$ , Johnson $2n$ (unused $2^n - 2n$ ), LFSR $2^n - 1$ , binary $2^n$ . Sequence trace karke likho.                                                                                                                           |
| Mod- $N$ counter design                            | 1. FFs = $\lceil \log_2 N \rceil$ . 2. Async: detect $N$ (AND of its 1 bits) $\rightarrow$ clear (glitch). 3. Sync: detect $N - 1 \rightarrow$ load 0 next edge (no glitch). 4. Sequence table.                                     |

# Unit 7 · Register Files & Finite State Machines

**Is unit ki kahani:** Unit 5–6 mein humne flip-flop aur register (flip-flops ki row) banaye. Ab do bade sawaal: (1) CPU ke andar 32 registers hain, instruction unhe *number* se bulata hai, to inhe ek saath kaise organise karein? Jawab: **register file** (decoder se likho, MUX se padho, Unit 3 ke blocks). (2) CPU har cycle kya kare, ye kaun decide karta hai? Jawab: ek **Finite State Machine (FSM)**, jo yaad rakhti hai "abhi main kahan hoon" aur input dekh ke aage badhti hai. Unit 8 mein register file se badi memory (SRAM, DRAM) aayegi, aur Unit 13 mein register file + FSM (control) milke poora CPU banayenge.

## 7.1 From Registers to Register Files

**Pehle socho:** Instruction `add R1, R2, R3` ka matlab hai: R2 aur R3 padho, jodo, jawab R1 mein likho. Isme **2 registers padhne hain aur 1 likhna hai, aur sab ek hi clock cycle mein**. Agar har register alag-alag wire se juda ho to 32 registers ke liye wiring ka jungle ban jayega. Socho railway station ke **numbered lockers ki deewar**: tum locker ka number bolte ho aur woh khul jaata hai. Is deewar mein do "dekhne ki khidkiyan" hain (do lockers ek saath dekh sakte ho) aur ek "daalne ka slot" hai (ek locker mein saamaan rakh sakte ho). Yahi register file hai.

**Definition:** A **register file** is an array of registers inside the CPU with address-based access, which allows several registers to be read and one register to be written in the same clock cycle. The MIPS / RISC-V register file has 32 registers of 32 bits each, with 2 read ports and 1 write port.

**Mushkil shabd, aasaan matlab:**

### register

Flip-flops ki ek row jo ek saath ek word (jaise 32 bits) store karti hai. 32-bit register = 32 D flip-flops, sabka clock same.

### address (register number)

Register ka "locker number" binary mein. 32 registers hain to number 0 se 31 tak. R5 ka address = 00101.

### port

Register file ka ek "darwaza" jisse ek register padha ya likha jaata hai. 2 read ports = ek saath do alag registers padh sakte ho. 1 write port = ek cycle mein ek register likh sakte ho.

### width (word size)

Ek register mein kitne bits hain. MIPS mein 32. Width aur number of registers alag cheezein hain: "32 × 32" ka pehla 32 = kitne registers, doosra 32 = har ek ki width.

### log<sub>2</sub>

"2 ko kitni baar multiply karein ki ye number mile".  $\log_2 32 = 5$  kyunki  $2 \times 2 \times 2 \times 2 \times 2 = 32$ . Isi se pata chalta hai address ke kitne bits chahiye.

#### Kaise kaam karta hai:

1. CPU ke paas 32 registers hain: R0 se R31.
2. Har register ko number (address) se bulaya jaata hai. 32 alag numbers banane ke liye kitne bits?  $2^5 = 32$ , isliye **5 bits** (00000 se 11111).
3. add R1, R2, R3 aaya: Read address 1 = 00010 (R2), Read address 2 = 00011 (R3), Write address = 00001 (R1).
4. Dono read ports R2 aur R3 ka data turant ALU ko de dete hain.
5. ALU jodta hai, aur clock edge pe jawab write port se R1 mein chala jaata hai.
6. Isliye ports ki ginti instruction se match karti hai: **2 source padhne = 2 read ports, 1 destination likhna = 1 write port.**
7. Speed: register file sabse tez storage hai (ns ka ek hissa), phir L1 cache, L2, RAM (Unit 8).

**Formula:** Address bits per port =  $\log_2(\text{number of registers})$  · Total storage bits (= flip-flops) = **registers × width**

32 registers →  $\log_2 32 = 5$  bits.  $32 \times 32 = 1024$  bits.

#### Solved example 1 (direct):

*A register file is designed with exactly 64 registers. What is the minimum number of bits required for the write address?*

1. Sawaal: 64 alag numbers banane ke liye kitne bits?
2. 2 ki powers likho:  $2^1 = 2$ ,  $2^2 = 4$ ,  $2^3 = 8$ ,  $2^4 = 16$ ,  $2^5 = 32$ ,  $2^6 = 64$ .
3.  $2^6 = 64$ , isliye  $\log_2 64 = 6$ .

**Answer: 6 bits.**

#### Solved example 2 (exam-level, class quiz):

*A register file has 16 registers, each 8 bits wide. How many flip-flops does it contain, and how wide must each read address be? Also find the total storage of the MIPS file ( $32 \times 32$ ) and of a  $64 \times 32$  file.*

1. Har bit = ek flip-flop. Flip-flops = registers × width =  $16 \times 8 = 128$ .
2. Read address:  $\log_2 16$ .  $2^4 = 16$ , isliye 4 bits.
3. MIPS:  $32 \times 32 = 1024$  bits.
4.  $64 \times 32$ :  $64 \times 32 = 2048$  bits (address 6 bits, kyunki  $2^6 = 64$ ).

**Answer: 128 flip-flops, 4-bit address; MIPS = 1024 bits;  $64 \times 32 = 2048$  bits.**

**Galti mat karna:** "Number of registers" aur "width" ko mix mat karo. Address bits sirf **number of registers** se aate hain (16 registers  $\rightarrow$  4 bits), width se nahi.  $16 \times 8$  file mein address 8 bits NAHI hai, 4 bits hai. Width (8) sirf data wires aur total bits mein aati hai.

**Trick:** "Ginti  $\rightarrow$  address, chaudai  $\rightarrow$  data." Kitne registers (ginti) se address bits; kitne bits per register (chaudai) se data bus.

**Ek line mein:** Register file = number se chunne wale registers ka bank; MIPS:  $32 \times 32$ -bit, 5-bit address, 2 read + 1 write port, kyunki add rd, rs, rt do padhta hai aur ek likhta hai.

## 7.2 Register File Architecture (1 Write, 2 Read Ports)

**Pehle socho:** Locker wali deewar mein do kaam hain. **Saamaan daalna (write):** pehle decide karo *kaunse* locker mein (number), phir decide karo *daalna hai ya nahi* (permission). **Dekhna (read):** number bolo, aur us locker ka saamaan khidki mein dikh jaaye. Unit 3 ke do blocks bilkul yahi karte hain: **decoder** number leke ek hi line ON karta hai (kaunsa locker), aur **MUX** number leke us ek input ko output pe bhej deta hai (khidki).

**Definition:** The **write port** of a register file uses an  $n$ -to- $2^n$  decoder whose one-hot outputs are ANDed with the write-enable signal (RegWrite) to load exactly one register at the clock edge (synchronous write). Each **read port** is a  $2^n$ :1 multiplexer whose select lines are the read address, so read data appears after the gate delay without any clock (combinational read).

**Mushkil shabd, aasaan matlab:**

**decoder (5-to-32)**

5-bit number andar, 32 lines bahar, sirf us number wali line 1. Input 00011  $\rightarrow$  sirf line 3 = 1.

**one-hot**

Lines ke group mein exactly ek line 1, baaki sab 0. Jaise 0000 1000: sirf line 3 "garam" hai.

**RegWrite / WE (write enable)**

Ek control bit: 1 = "is cycle mein likhna hai", 0 = "kuch mat likho". sw ya beq jaise instructions register nahi likhte, unke liye RegWrite = 0.

**load enable**

Register ka apna "haan, nayi value lo" signal. Enable = 1 to clock edge pe naya data andar, enable = 0 to purani value rakhe (Unit 6).

**MUX (32:1)**

32 inputs mein se ek ko output pe bhejne wala switch. Kaunsa? Ye 5 select bits batate hain ( $2^5 = 32$ ). Select = 00010  $\rightarrow$  input 2 bahar.

**combinational**

Output sirf abhi ke inputs pe depend karta hai, clock ka intezaar nahi. Input badla to thodi der (gate delay) baad output badal gaya.

### synchronous

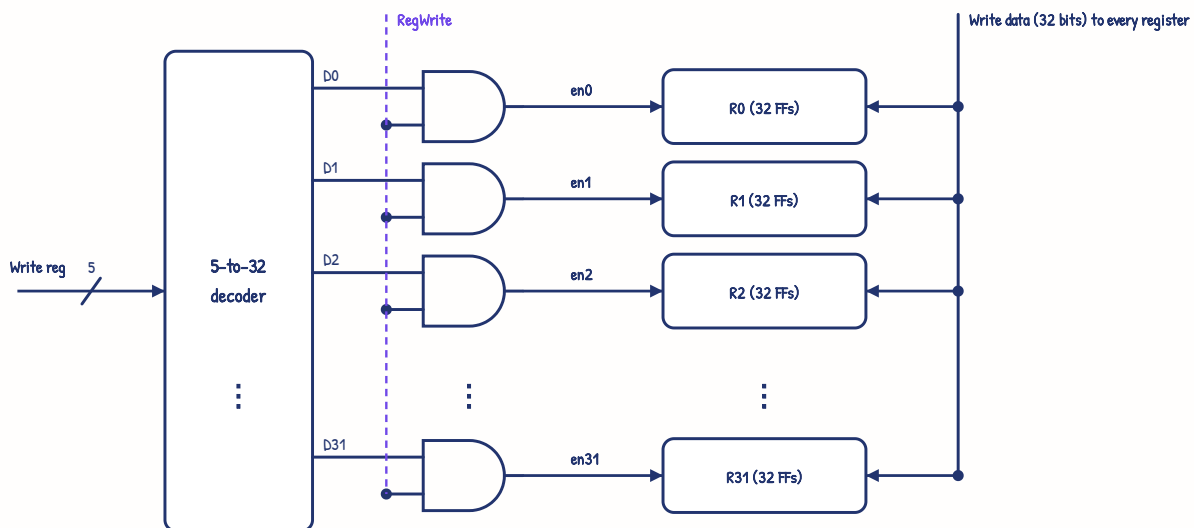
Kaam sirf clock edge ( $0 \rightarrow 1$  hone ke pal) pe hota hai. Beech mein inputs badalte rahein, register nahi badalta.

### hardwired zero

R0 (\$zero, RISC-V mein x0) hamesha 0 padhta hai, usme likhoge to ignore ho jaata hai.

### Kaise kaam karta hai (WRITE port, synchronous):

1. Write register address (5 bits, jaise 00001 = R1) 5-to-32 decoder mein jaata hai.
2. Decoder ki 32 output lines mein se sirf line 1 = 1 (one-hot). Ye batata hai **KAHAN** likhna hai.
3. Har decoder line ek 2-input AND gate mein jaati hai; AND ka doosra input **RegWrite** hai. Isliye 32 AND gates.
4. AND ka output = us register ka load enable.  $en1 = D1 \cdot \text{RegWrite} = 1 \cdot 1 = 1$ . Baaki sab  $en = 0$  (kyunki unki decoder line 0 hai).
5. Write data (32 bits) **saare 32 registers** tak wire se pahunchta hai, lekin sirf jiska enable 1 hai woh clock edge pe use pakadta hai.
6. Agar  $\text{RegWrite} = 0$  to saare AND outputs 0: koi register nahi badlega. **Decoder batata hai KAHAN, RegWrite batata hai KYA LIKHNA HAI YA NAHI.**



The decoder picks WHERE (one-hot), RegWrite picks WHETHER. Only the enabled register captures the data at the edge.

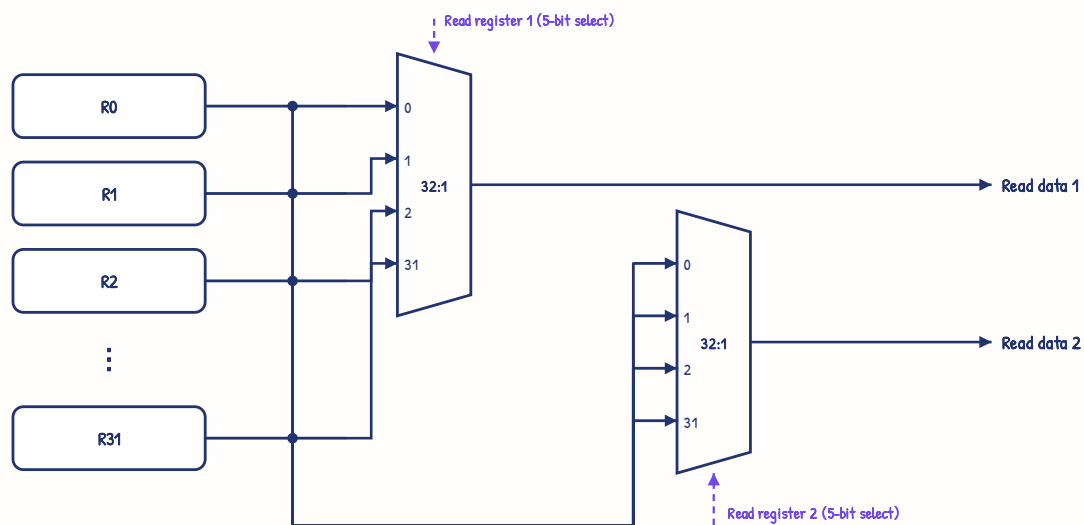
### Write port: 5-to-32 decoder + RegWrite → register enables

How to draw: Tall decoder box; each output into an AND gate with RegWrite; AND output = load enable of R0..R31; one Write data bus to all registers.

**Diagram kaise padhein:** Left mein lamba box = 5-to-32 decoder, andar 5-bit "Write reg" aata hai. Har decoder output (D0, D1, D2, D31) ek AND gate mein jaata hai; upar se aane wali RegWrite line har AND ka doosra input hai. AND ka output (en0, en1) us register box (32 FFs) ka enable hai. Right mein Write data ki ek lambi line saare registers ko chhooti hai. Trace: Write reg = 00010, RegWrite = 1 → D2 = 1 → en2 = 1 → sirf R2 clock edge pe naya data lega.

## Kaise kaam karta hai (READ ports, combinational):

1. Har register ka 32-bit output do jagah jaata hai: MUX 1 aur MUX 2 (dono 32:1).
2. Read register 1 (5 bits) MUX 1 ka select hai; Read register 2 (5 bits) MUX 2 ka select hai.
3. Read register 1 = 00010 → MUX 1 R2 ko "Read data 1" pe bhejta hai. Read register 2 = 00011 → MUX 2 R3 ko "Read data 2" pe bhejta hai.
4. Koi clock nahi: address diya, gate delay ke baad data aa gaya. Isliye ALU ko operands usi cycle mein mil jaate hain.
5. "32 bits wide" MUX ka matlab: asal mein 32 chhote 1-bit 32:1 MUX side-by-side, sabka select same 5 bits.

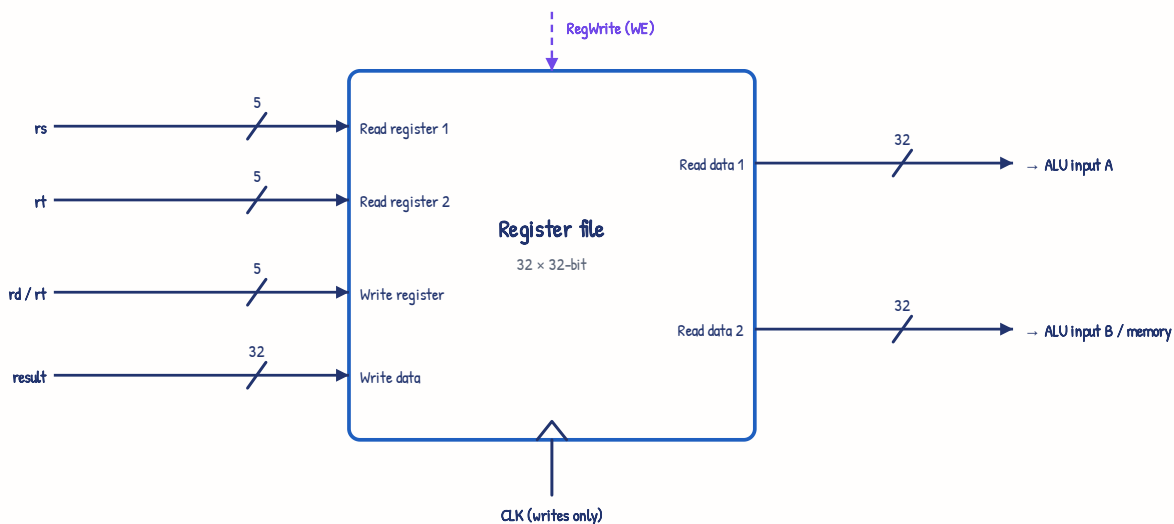


Every register output fans out to BOTH 32:1 MUXes (wide 32-bit buses, drawn as single lines). No clock: data appears after gate delay.

### Read ports: two 32:1 MUXes

How to draw: Stack of register boxes on the left; every output goes to both MUXes; Read reg 1 / 2 are the MUX selects; outputs Read data 1 / 2.

**Diagram kaise padhein:** Left mein register boxes R0, R1, R2, R31. Har box se line nikal ke DONO trapezium (32:1 MUX) mein jaati hai, isliye har register ka dot (fan-out) dikhta hai. Upar wala MUX ka select = Read register 1, neeche wale ka = Read register 2. Trace: Read register 1 = 00001 → upar wala MUX R1 ko "Read data 1" pe nikalta hai, koi clock arrow nahi hai.



Blue side = 2 read ports (combinational). Write port = address + data + WE, updates one register at the clock edge.

### Register file block (MIPS $32 \times 32$ , $2R + 1W$ )

How to draw: One big box; Left side: Read reg 1, Read reg 2, Write reg (5 bits each), Write data (32); right side: Read data 1 and 2 (32); RegWrite on top; clock at the bottom.

**Diagram kaise padhein:** Ye poore register file ka bahar se dikhne wala box hai (exam mein yahi sabse pehle banao). Left: Read register 1 (rs, 5 bits), Read register 2 (rt, 5), Write register (rd ya rt, 5), Write data (32). Right: Read data 1 (ALU ka input A), Read data 2 (ALU input B ya memory). Upar RegWrite, neeche CLK jo sirf write ke liye hai. Wire pe tircha slash aur number = us wire mein kitne bits (5 ya 32).

### Read kyun combinational aur write kyun clock edge pe?

| READ = combinational                                               | WRITE = synchronous (clock edge)                                                   |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Clock ka intezaar nahi, sirf gate delay                            | Sirf clock edge pe register badalta hai                                            |
| ALU ko operands usi cycle mein chahiye, warna ek extra cycle waste | Address, data, RegWrite pehle set ho jaate hain, edge pe saaf-saaf likha jaata hai |
| Do ports ek saath kaam karte hain                                  | Beech mein badalte (adhure) value likhe nahi jaate, race nahi hota                 |
| Cycle ki shuruaat mein purani value padhi                          | Nayi value agle cycle se padhne ko milti hai                                       |

Ek cycle ki kahani (add R1, R1, R2): cycle shuru → purana R1 aur R2 padha (combinational) → ALU ne joda → clock edge pe jawab R1 mein likha. Agar write bhi bina clock ke hota, to naya R1 turant read port pe aata, ALU phir se jodta, phir likhta: ek hi cycle mein value baar-baar badalti (race). Clock edge isko rokta hai. Yahi **read-before-write** hai.

**Formula (n registers, w bits each):** address bits  $k = \log_2 n$  · write decoder = **k-to-n** · AND gates =  $n$  · read MUX = **n:1**, w bits wide, k select lines, **one per read port** · storage =  $n \times w$  bits.

MIPS: 5-to-32 decoder, 32 AND gates, two 32:1 MUXes (32-bit wide, 5 select bits each), 1024 bits.

### Solved example 1 (class homework):

An  $8 \times 16$  register file (8 registers, 16 bits each) has 1 write and 2 read ports. Find the address width, decoder size and number of decoder output lines, MUX size, and total bits.

1. Registers = 8  $\rightarrow$  address bits =  $\log_2 8 = 3$  ( $2^3 = 8$ ).
2. Write decoder: 3 inputs,  $2^3 = 8$  outputs  $\rightarrow$  **3-to-8 decoder, 8 output lines**.
3. AND gates with RegWrite: ek per register = 8.
4. Read ports: 2 MUXes, har ek **8:1**, 16 bits wide, 3 select lines.
5. Total bits =  $8 \times 16 = 128$ .

**Answer: 3-bit address, 3-to-8 decoder (8 outputs), two 16-bit 8:1 MUXes, 128 bits.**

### Solved example 2 (mock: $4 \times 32$ file drawing):

Draw / describe a register file with 4 registers of 32 bits, one write port and two read ports. Label the decoder, write enable and MUX select widths. What does it output if the hardwired-zero version is written with FACEFACE at address 00 and then address 00 is read?

1. Address bits =  $\log_2 4 = 2$  ( $2^2 = 4$ ).
2. Write side: **2-to-4 decoder**; 4 AND gates (decoder line  $\cdot$  WE)  $\rightarrow$  load enables of R0 to R3; 32-bit write data bus to all 4.
3. Read side: two **4:1 MUXes**, 32 bits wide, each with a **2-bit select** (rd\_addr1, rd\_addr2).
4. Write on posedge clk only if WE = 1; reads are plain assign statements (combinational).
5. Hardwired zero: write to address 00 is dropped, and reading 00 always gives 0.

**Answer: 2-to-4 decoder + WE ANDs, two 32-bit 4:1 MUXes with 2-bit selects; reading R0 gives 00000000.**

```
always @(posedge clk)                // WRITE: only at the clock edge
    if (we) reg_file[wr_addr] <= wr_data;
assign rd_data1 = reg_file[rd_addr1]; // READ: combinational (no clock)
assign rd_data2 = reg_file[rd_addr2];
```

**Galti mat karna:** (1) Read port ko clock mat karo: operand ek cycle late milega. (2) RegWrite bhoolna: phir har cycle har register overwrite ho jayega. (3) 32:1 MUX ka select 32 bits NAHI, **5 bits** hai. (4) Write side pe MUX aur read side pe decoder likh dena ulta hai: **write = decoder, read = MUX**.

**Trick: "Likhne ke Liye Decoder, Dekhne ke Liye MUX."** Decoder ek line jalata hai (kahan rakhun), MUX ek cheez dikhata hai (kya hai).

**Ek line mein:** Write = k-to-n decoder (kahan) AND RegWrite (likhun ya nahi), clock edge pe; Read = har port ka ek n:1 MUX, address = select, bina clock.

## 7.3 Introduction to Finite State Machines

**Pehle socho:** Vending machine socho: ek drink ₹10 ki, tum ₹5 ke coin daalte ho. Pehla coin daala, kuch nahi nikla, par machine ko *yaad* hai "₹5 aa chuke". Doosra coin daala, ab drink nikli. Matlab machine ka jawab sirf abhi ke coin pe nahi, **pichli history (jo yaad hai) pe bhi depend karta hai**. Ye "yaad" hi **state** hai. **Traffic light** bhi aisi hi hai: GREEN hai, timer khatam hua to YELLOW, phir RED, phir GREEN. Har light ek state hai. CPU ka control unit bhi yahi karta hai: fetch → decode → execute → write-back.

**Definition:** A **finite state machine (FSM)** is a sequential circuit that has a finite number of states, moves from one state to another (a transition) according to its present state and inputs, and produces outputs. Formally it is the 5-tuple **(S, I, O,  $\delta$ ,  $\lambda$ )**: S = set of states, I = inputs, O = outputs,  $\delta$  = next-state (transition) function,  $\lambda$  = output function.

**Mushkil shabd, aasaan matlab:**

### state

Machine ki "yaad", yaani history ka summary. Vending machine: "₹0 aaye", "₹5 aaye". Hardware mein state flip-flops mein binary number ke roop mein rehti hai.

### transition

Ek state se doosri state pe jaana, clock edge pe, input ke hisaab se. Diagram mein arrow.

### present state / next state

Present = abhi flip-flops mein kya hai. Next = agli clock edge pe kya hoga.

### next-state logic ( $\delta$ )

Combinational circuit: (present state, input) leke next state nikalta hai. Traffic: (GREEN, T = 1) → YELLOW.

### state register

Flip-flops jo present state store karte hain; clock edge pe next state andar le lete hain. FSM ka **sirf yahi hissa** clocked hai.

### output logic ( $\lambda$ )

Combinational circuit jo state (aur Mealy mein input bhi) dekh ke output banata hai. Traffic: state RED → output "Stop".

### state diagram

Circle = state, arrow = transition, arrow pe input likha hota hai.

### state table

Wahi diagram table mein: har row = ek present state, har column = ek input value, cell mein next state.

### encoding

Har state ko binary code dena. Binary: G = 00, Y = 01, R = 10. One-hot: G = 001, Y = 010, R = 100 (har state ka apna flip-flop, ek hi 1).

## ⌈ ⌋ (ceiling)

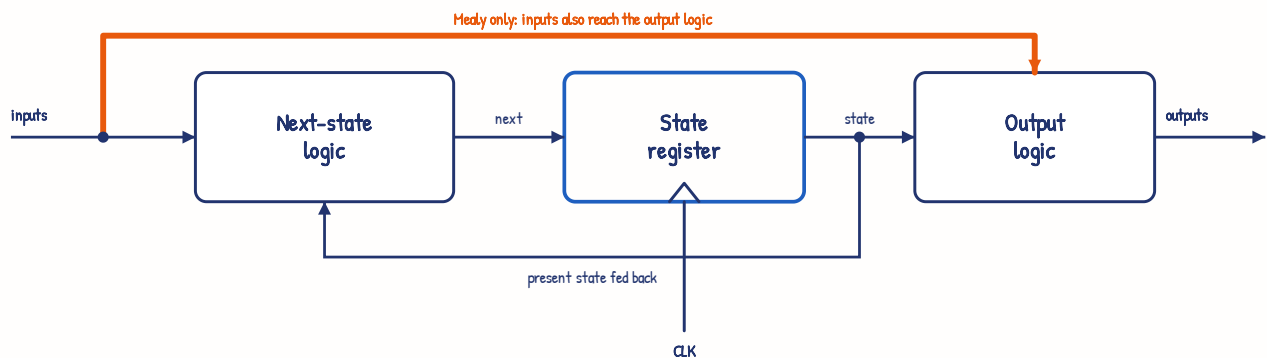
"Upar ki taraf round karo".  $\lceil 2.32 \rceil = 3$ ,  $\lceil 3 \rceil = 3$ .

## reset state

Power on hone pe machine kahan se shuru kare (S0). Diagram mein bahar se aata arrow.

## Kaise kaam karta hai (hardware ke 3 blocks):

1. **State register** (flip-flops) mein present state rakhi hai, jaise 00 = GREEN.
2. **Next-state logic** present state + input dekhta hai: GREEN aur T = 1  $\rightarrow$  next = 01 (YELLOW).
3. Clock edge aata hai: state register 01 le leta hai. Ab present state = YELLOW.
4. **Output logic** state dekh ke output deta hai: YELLOW  $\rightarrow$  "Slow".
5. Present state wapas next-state logic mein jaati hai (feedback), aur cycle chalta rehta hai.



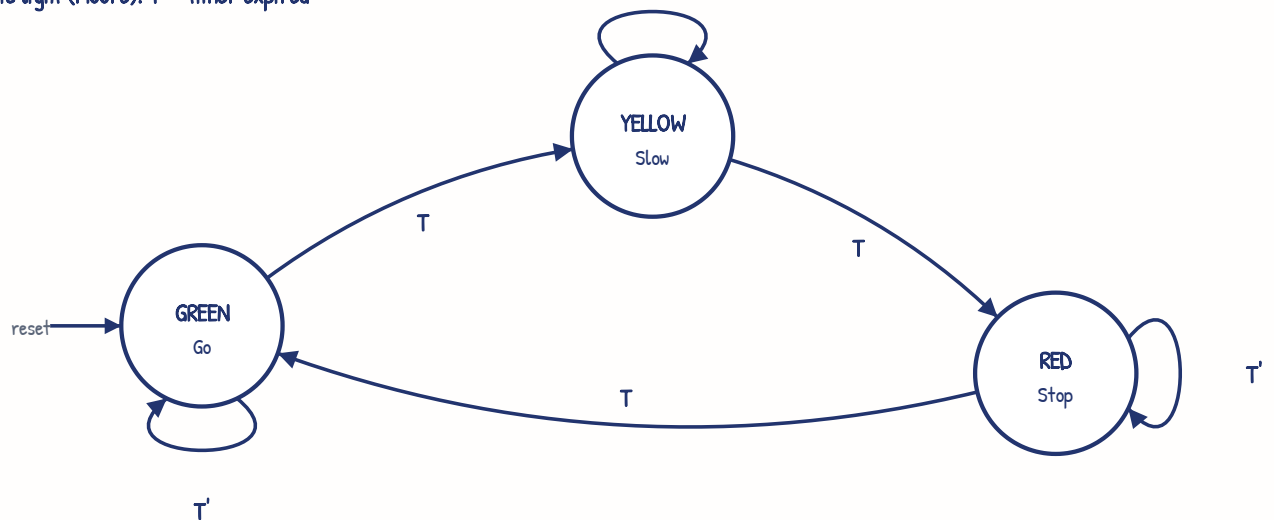
Moore: output =  $\lambda(\text{state})$ . Mealy: output =  $\lambda(\text{state}, \text{input})$ . Next state =  $\delta(\text{state}, \text{input})$  for both.

## FSM anatomy (Moore and Mealy)

How to draw: Three boxes in a row (next-state logic, state register with clock, output logic), feedback arrow from the register back to next-state logic; for Mealy add an arrow from the inputs to the output logic.

**Diagram kaise padhein:** Teen box left se right: next-state logic  $\rightarrow$  state register (neeche CLK, sirf isme flip-flops)  $\rightarrow$  output logic. State register ke output se ek arrow neeche ghoom ke wapas next-state logic mein jaata hai: yahi "yaad" ka feedback hai. Upar ka highlighted arrow (inputs se seedha output logic tak) sirf **Mealy** mein hota hai; Moore mein output logic sirf state dekhta hai. Neeche likha hai: Moore output =  $\lambda(\text{state})$ , Mealy output =  $\lambda(\text{state}, \text{input})$ .

Traffic light (Moore): T = timer expired



### Traffic-light FSM (Moore)

How to draw: Three circles GREEN/YELLOW/RED with outputs Go/Slow/Stop inside; arrows labelled T around the cycle; self-loops labelled T'.

**Diagram kaise padhein:** Teen circle = teen states, har circle mein state ka naam aur output (GREEN/Go, YELLOW/Slow, RED/Stop). Bahar se aata arrow GREEN pe = reset state. Arrow pe "T" = timer khatam hua to aage jao; har circle pe chhota loop "T'" = timer abhi chal raha hai to wahin raho. Trace: GREEN, T = 0 → GREEN; T = 1 → YELLOW; T = 1 → RED; T = 1 → GREEN. Output circle ke andar hai, isliye ye Moore machine hai.

### FSM design karne ka step-by-step tarika (traffic light pe poora karke):

- Spec samjho:** Input T (timer khatam = 1). Output: Go, Slow ya Stop. Order GREEN → YELLOW → RED → GREEN, T = 0 pe wahin ruko.
- States list karo (meaning ke saath):** G = "green chal raha hai", Y = "yellow", R = "red". Reset state = G.
- State diagram banao (D7.3b):** har state se **har input value** ka arrow (T = 0 aur T = 1), dono.
- State table banao:**

| Present | T = 0 | T = 1 | Output |
|---------|-------|-------|--------|
| G       | G     | Y     | Go     |
| Y       | Y     | R     | Slow   |
| R       | R     | G     | Stop   |

- Encoding karo:** 3 states →  $\lceil \log_2 3 \rceil = 2$  flip-flops (Q1 Q0). G = 00, Y = 01, R = 10. Code 11 kabhi use nahi hota (don't care).
- Encoded table + equations:**

| Q1Q0   | T | Q1 <sup>+</sup> Q0 <sup>+</sup> |
|--------|---|---------------------------------|
| 00 (G) | 0 | 00                              |
| 00 (G) | 1 | 01                              |
| 01 (Y) | 0 | 01                              |
| 01 (Y) | 1 | 10                              |
| 10 (R) | 0 | 10                              |
| 10 (R) | 1 | 00                              |

$Q1^+ = 1$  wali rows: (Y, T = 1) aur (R, T = 0)  $\rightarrow Q1^+ = Q0 \cdot T + Q1 \cdot T'$ .

$Q0^+ = 1$  wali rows: (G, T = 1) aur (Y, T = 0)  $\rightarrow Q0^+ = Q1' \cdot Q0' \cdot T + Q0 \cdot T'$ .

Output (11 ko don't care maan ke): **Go** =  $Q1' \cdot Q0'$ , **Slow** =  $Q0$ , **Stop** =  $Q1$ .

7. **Circuit banao / simulate karo:** 2 D flip-flops ( $D1 = Q1^+$ ,  $D0 = Q0^+$ ), upar ke gates, aur reset jo 00 pe le aaye. Har row check karo.

**Formula:** Flip-flops (binary encoding) =  $\lceil \log_2 N \rceil$  · Flip-flops (one-hot) =  $N$

$N$  = number of states. Shortcut: sabse chhoti power of 2 dhoondo jo  $N$  ke barabar ya badi ho; uska exponent = flip-flops.

### Solved example 1 (flip-flop count):

Find the minimum number of flip-flops (binary encoding) and the number for one-hot encoding for FSMs with 3, 5, 6 and 9 states.

1. Powers of 2:  $2^1 = 2$ ,  $2^2 = 4$ ,  $2^3 = 8$ ,  $2^4 = 16$ .
2.  $N = 3$ : 2 se zyada, 4 tak fit  $\rightarrow 2^2 \rightarrow$  **2 FFs**. One-hot: 3.
3.  $N = 5$ : 4 se zyada, 8 tak fit  $\rightarrow 2^3 \rightarrow$  **3 FFs**. One-hot: 5.
4.  $N = 6$ : 8 tak fit  $\rightarrow$  **3 FFs**. One-hot: 6.
5.  $N = 9$ : 8 se zyada ( $2^3 = 8$  sirf 8 codes deta hai), 16 tak fit  $\rightarrow 2^4 \rightarrow$  **4 FFs**. One-hot: 9.

**Answer:** 3  $\rightarrow$  2 / 3; 5  $\rightarrow$  3 / 5; 6  $\rightarrow$  3 / 6; 9  $\rightarrow$  4 / 9 (binary / one-hot).

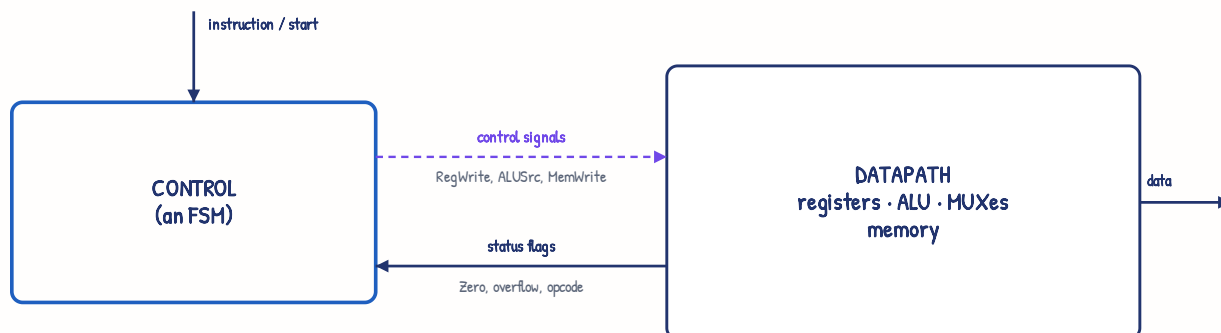
### Solved example 2 (write the answer like this):

What is a finite state machine? Draw its general hardware block diagram. (3 marks)

1. A finite state machine is a sequential circuit with a finite set of states  $S$ , inputs  $I$  and outputs  $O$ ;  $\delta$  gives the next state and  $\lambda$  gives the output.
2. It has three blocks: next-state logic (combinational), the state register (flip-flops, clocked) and output logic (combinational).
3. The present state is fed back to the next-state logic, so the next state depends on the present state and the input.
4. In a Moore machine the output depends on the state only; in a Mealy machine also on the input.

5. Draw D7.3a. Example: traffic light (GREEN → YELLOW → RED on timer T), or the CPU control unit.

Answer: definition + (S, I, O,  $\delta$ ,  $\lambda$ ) + 3-block diagram with feedback + one example.



Control decides WHAT happens each cycle; the datapath does the work and reports back. (Unit 13 builds both.)

#### Datapath + control

How to draw: Control box (FSM) on the left, datapath box on the right; dashed control-signal arrow right, status-flag arrow back left.

**Diagram kaise padhein:** Left box CONTROL ek FSM hai; right box DATAPATH (registers, ALU, MUX, memory) asli kaam karta hai. Upar ka dashed arrow = control signals (RegWrite, ALUSrc, MemWrite) control se datapath ko. Neeche ka arrow = status flags (Zero, overflow, opcode) datapath se control ko wapas. Isliye ALU ka Zero flag ek **status** signal hai, control signal nahi.

**Galti mat karna:** (1) Reset state bhoolna. (2) Kisi state se kisi input ka arrow chhod dena: har state se har input value ka arrow chahiye (traffic light mein T' self-loops bhi). (3) One-hot ke liye  $\lceil \log_2 N \rceil$  likhna: one-hot mein N flip-flops lagte hain. (4) 9 states ke liye 3 FFs bolna: 3 FFs sirf 8 codes dete hain.

**Trick:** FSM = "Socho (next-state logic) → Yaad rakho (state register) → Bolo (output logic)". Sirf "yaad rakho" wale dabbe mein clock hai.

**Ek line mein:** FSM = states + inputs + transitions + outputs; hardware = next-state logic → state register (clock) → output logic, state ka feedback; FFs =  $\lceil \log_2 N \rceil$  (binary) ya N (one-hot).

## 7.4 Moore vs Mealy Machines

**Pehle socho:** Do tarah ke chowkidar. **Moore chowkidar** sirf apni "position" dekh ke bolta hai: woh gate pe khada hai to "andar aao" bolega, bas. Uska jawab tab badalta hai jab woh position badalta hai (agla clock edge). **Mealy chowkidar** position + tumhara chehra (abhi ka input) dono dekhta hai, aur *turant* bol deta hai, position badalne ka intezaar nahi karta. Isliye **Mealy ek cycle pehle jawab deta hai**, par agar input hil-dul raha ho (glitch) to uska jawab bhi hilita hai.

**Definition:** In a **Moore machine** the outputs depend only on the present state,  $\lambda = f(\text{state})$ . In a **Mealy machine** the outputs depend on the present state and the present inputs,  $\lambda = f(\text{state}, \text{input})$ .

### Mushkil shabd, aasaan matlab:

#### output in the circle (Moore)

State diagram mein output state ke circle ke andar likha jaata hai: "S4 / 1". Us state mein ho to output 1, chahe input kuch bhi ho.

#### output on the arrow (Mealy)

Arrow pe "input / output" likhte hain: "1/1" = input 1 aaya to output 1 aur is arrow pe jao.

#### glitch

Signal mein galti se aaya chhota sa spike, jaise input ek pal ke liye  $0 \rightarrow 1 \rightarrow 0$  ho gaya. Mealy output input se seedha juda hai, isliye ye spike output mein bhi aa sakta hai.

#### registered output

Output ke aage ek flip-flop laga dena; glitch hat jaata hai par output ek cycle late ho jaata hai (Moore jaisa).

### Kaise kaam karta hai (timing ka farak):

1. Maan lo detector ko "1011" dhoondhna hai aur abhi "101" dekh chuka hai (state S3).
2. Ab bit 4 = 1 aata hai, clock edge abhi nahi aaya.
3. **Mealy:** output logic ko state S3 aur input 1 dono dikh rahe hain, to **usi cycle mein** output = 1 (arrow S3 — 1/1→).
4. **Moore:** output sirf state se aata hai. Abhi state S3 hai jiska output 0. Clock edge pe state S4 banti hai, tab output = 1, yaani **agle cycle mein**.
5. Isliye Moore ko ek extra state (S4, "match ho gaya") chahiye jisme output 1 likha ho. Mealy mein ye kaam arrow kar deta hai.
6. Dono same pattern pakadte hain; farak sirf **kab** output aata hai aur **kitni states** lagti hain.

| Point               | Moore                                | Mealy                                       |
|---------------------|--------------------------------------|---------------------------------------------|
| Output depends on   | state only                           | state + input                               |
| Diagram notation    | output inside circle (S1/1)          | output on arrow (in/out)                    |
| When output changes | only when state changes (clock edge) | as soon as input changes, same cycle        |
| Timing              | one cycle after the input            | same cycle as the input (one cycle earlier) |
| Glitches            | glitch-free (stable for whole cycle) | can glitch if input glitches                |
| Number of states    | usually more (1011: 5)               | usually fewer (1011: 4)                     |
| Block diagram       | output logic fed by state only       | output logic fed by state and inputs        |

### Solved example (write the answer like this):

*Compare Moore and Mealy machines (any five points). (5 marks)*

1. Moore: output is a function of the present state only; Mealy: output is a function of the present state and the present input.

2. Moore writes the output inside the state circle; Mealy writes input/output on each transition arrow.
3. Mealy output responds in the same cycle as the input; Moore output appears one clock cycle later.
4. Moore output is glitch-free; Mealy output can glitch when the input glitches.
5. Mealy usually needs fewer states (1011 detector: Mealy 4, Moore 5).
6. Draw D7.3a twice: Moore without, Mealy with the arrow from inputs to output logic.

**Answer:** five rows of the table above + the two block diagrams.

**Mealy → Moore conversion (class homework):** jis Mealy state mein alag-alag outputs wale arrows *aate* hain, use har output ke liye do copies mein tod do (ek copy "out 0", ek "out 1"). 1011 Mealy mein S1 do tarah se aata hai: S0 —1/0→ S1 aur S3 —1/1→ S1. To S1 ki ek copy "out 1" wali banti hai: wahi Moore ki S4 hai. Isliye Moore mein ek state zyada.

**Galti mat karna:** (1) Ek hi diagram mein output circle mein BHI aur arrow pe BHI likh dena: ek notation choose karo. (2) Ye kehna ki Moore aur Mealy alag patterns detect karte hain: pattern same, sirf timing alag. (3) "Mealy late hai" bolna: ulta, Mealy ek cycle **pehle** output deta hai.

**Trick:** Moore = "O" = gol circle ke andar output. Mealy = arrow pe output. Mealy "jaldi bolta hai" (same cycle), Moore "soch ke bolta hai" (agle cycle).

**Ek line mein:** Moore = output sirf state se (circle mein, ek cycle baad, glitch-free, zyada states); Mealy = output state + input se (arrow pe, same cycle, glitch ho sakta hai, kam states).

## 7.5 Worked Example: Sequence Detector 1011

**Pehle socho:** Tum ek lambi line mein aane wale bits dekh rahe ho: 1, 0, 1, 1, 0, 1, 1, 0. Kaam: jab bhi aakhri 4 bits "1011" hon, ghanti bajao. Tumhe saare purane bits yaad rakhne ki zarurat nahi, sirf itna ki "abhi tak pattern ka kitna hissa match ho chuka hai". Jaise password type karte waqt: "maine pehle 3 letters sahi daale hain". Galat letter aaya to zero se shuru karna zaruri nahi: ho sakta hai galat letter ke saath ka aakhri hissa naye password ki shuruaat ho. Telecom mein frame ki shuruaat dhoondhne ke liye yahi detector use hota hai (DS1 pattern 001011, SONET 11110110).

**Definition:** A **sequence detector** is an FSM that receives one bit per clock cycle on a serial input and outputs 1 when a specified bit pattern has just appeared. With **overlapping** detection, the end of one match may be the beginning of the next match; with **non-overlapping** detection, the machine starts afresh after every match.

**Mushkil shabd, aasaan matlab:**

**serial input**

Ek wire, har clock pe ek bit. Stream 10110110 ka matlab: pehle 1, phir 0, phir 1 (bit 1 sabse pehle).

## prefix

Pattern ki shuruaat ke tukde. 1011 ke prefixes: "1", "10", "101", "1011".

## suffix

Ab tak aaye input ke aakhri ke tukde. "1010" ke suffixes: "0", "10", "010", "1010".

## longest suffix that is a prefix

Aakhri wale tukdon mein se sabse lamba jo pattern ki shuruaat bhi ho. "1010" mein: "10" prefix hai (1011 bhi "10" se shuru hota hai), "010" nahi. To jawab "10". Yahi batata hai galti ke baad kahan jaana hai.

## overlap

Match ke aakhri bits agle match mein reuse ho sakte hain. Stream 1011011: bits 1-4 = 1011 aur bits 4-7 = 1011, bit 4 dono mein. Overlap: 2 matches. Non-overlap: 1.

## fall-back edge (failure arrow)

Galat bit aane pe peeche ki state pe jaane wala arrow.

## States ka matlab (dono machines mein same):

| State           | Matlab (Hinglish)               | Pattern ka kitna match |
|-----------------|---------------------------------|------------------------|
| S0              | kuch kaam ka nahi dekha (start) | ""                     |
| S1              | abhi tak "1" dekha              | 1                      |
| S2              | abhi tak "10" dekha             | 10                     |
| S3              | abhi tak "101" dekha            | 101                    |
| S4 (sirf Moore) | "1011" poora mil gaya, output 1 | 1011                   |

## Har arrow kahan jaata hai aur KYUN (rule: naya string banao, uska longest suffix jo 1011 ka prefix ho):

1. **S0, input 0:** string "0". 1011 "1" se shuru hota hai, "0" kisi kaam ka nahi → **S0**.
2. **S0, input 1:** "1" = pehla prefix → **S1**.
3. **S1, input 0:** "10" = prefix → **S2** (aage badho).
4. **S1, input 1:** "11". "11" prefix nahi, par uska aakhri "1" prefix hai → **S1 pe hi raho** (naya 1 nayi shuruaat ho sakta hai).
5. **S2, input 1:** "101" = prefix → **S3**.
6. **S2, input 0:** "100". Suffixes "00", "0": koi bhi "1" se shuru nahi → **S0**.
7. **S3, input 0:** "1010". Suffix "010" prefix nahi, par **"10" prefix hai** → **S2** (S0 nahi! "10" bacha hai, use mat phenko).
8. **S3, input 1:** "1011" = MATCH, output 1. Ab kahan jaayein?  
Overlap: aakhri "1" agle 1011 ki shuruaat ho sakta hai → **S1**.  
Non-overlap: match ke bits phenk do → **S0**.
9. **Moore S4** ("1011" dekha, output 1) ke arrows: S4 ki "yaad" bhi aakhir mein "1" hai, isliye overlap mein S4 bilkul S1 jaisa behave karta hai: input 0 → "10" → **S2**; input 1 → "11" → aakhri "1" → **S1**. Non-overlap

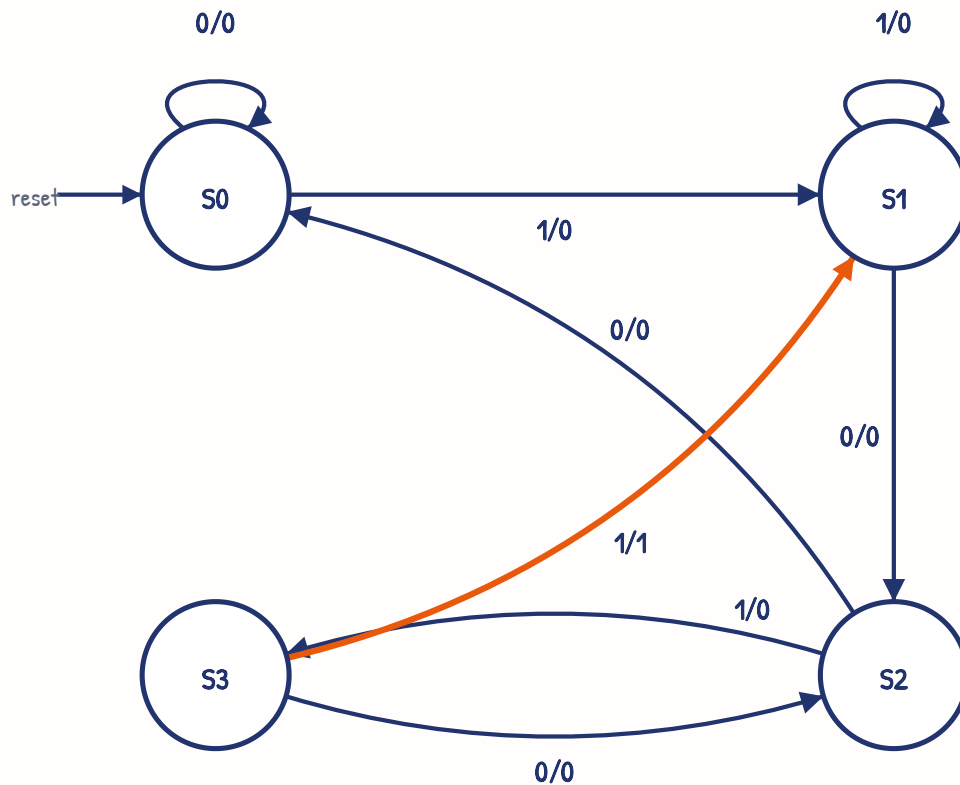
mein S4 ko S0 jaisa maano: input 0 → **S0**, input 1 → **S1**.

**State tables (Lab 8, Q1 to Q4):**

| Present | Mealy overlap (next / out) |               | Mealy non-overlap |               |
|---------|----------------------------|---------------|-------------------|---------------|
|         | in = 0                     | in = 1        | in = 0            | in = 1        |
| S0      | S0 / 0                     | S1 / 0        | S0 / 0            | S1 / 0        |
| S1      | S2 / 0                     | S1 / 0        | S2 / 0            | S1 / 0        |
| S2      | S0 / 0                     | S3 / 0        | S0 / 0            | S3 / 0        |
| S3      | S2 / 0                     | <b>S1 / 1</b> | S2 / 0            | <b>S0 / 1</b> |

| Present | Moore overlap |        |          | Moore non-overlap |        |          |
|---------|---------------|--------|----------|-------------------|--------|----------|
|         | in = 0        | in = 1 | out      | in = 0            | in = 1 | out      |
| S0      | S0            | S1     | 0        | S0                | S1     | 0        |
| S1      | S2            | S1     | 0        | S2                | S1     | 0        |
| S2      | S0            | S3     | 0        | S0                | S3     | 0        |
| S3      | S2            | S4     | 0        | S2                | S4     | 0        |
| S4      | <b>S2</b>     | S1     | <b>1</b> | <b>S0</b>         | S1     | <b>1</b> |

Mealy 1011, overlap: S3  $\xrightarrow{1/1}$  S1 (input/output on arrows)

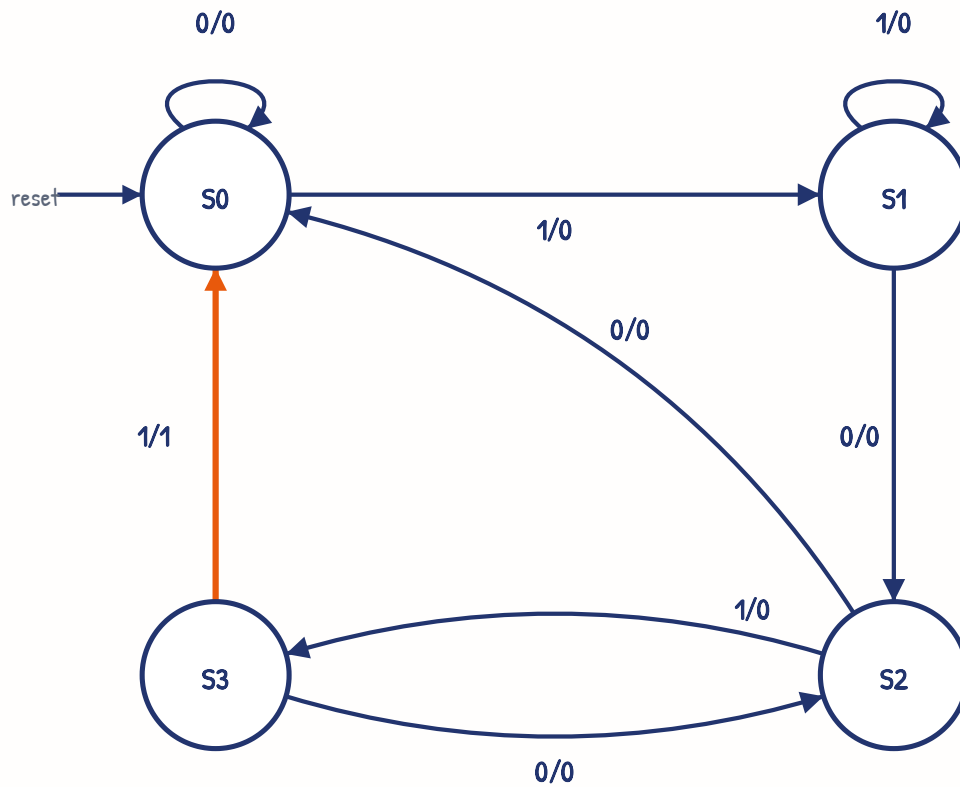


#### Mealy 1011 detector, overlapping

How to draw: Four circles S0–S3 in a square; forward edges 1/0, 0/0, 1/0; fall-back edges; the amber S3  $\rightarrow$  S1 edge labelled 1/1.

**Diagram kaise padhein:** Chaar circle S0 (upar-left, reset arrow), S1 (upar-right), S2 (neeche-right), S3 (neeche-left). Arrow pe "input/output". Aage badhne wale arrows: S0  $\xrightarrow{1/0}$  S1  $\xrightarrow{0/0}$  S2  $\xrightarrow{1/0}$  S3. Self-loops: S0 pe 0/0, S1 pe 1/0. Fall-back: S2  $\xrightarrow{0/0}$  S0, S3  $\xrightarrow{0/0}$  S2. Amber arrow S3  $\xrightarrow{1/1}$  S1 = match + overlap. Trace "1011": S0  $\rightarrow$  (1) S1  $\rightarrow$  (0) S2  $\rightarrow$  (1) S3  $\rightarrow$  (1, output 1) S1.

Mealy 1011, no overlap:  $S3 \xrightarrow{1/1} S0$  (input/output on arrows)

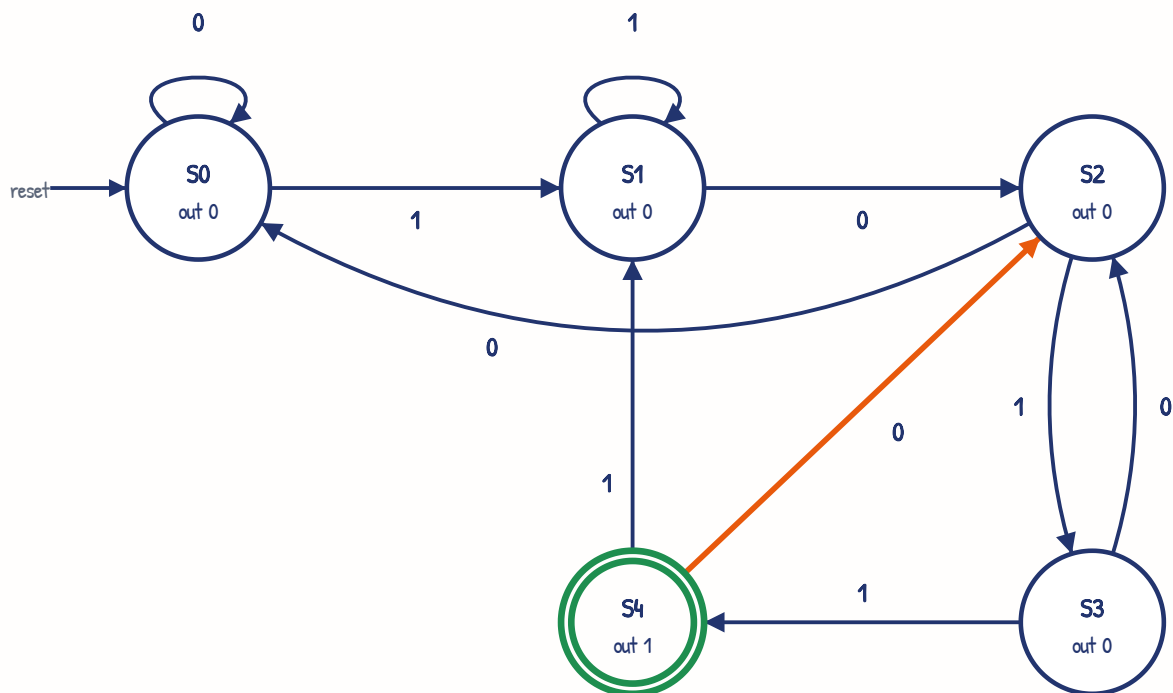


Mealy 1011 detector, non-overlapping

How to draw: Same as overlap, only the matching edge changes to  $S3 \rightarrow S0$  (1/1).

**Diagram kaise padhein:** Bilkul D7.5a jaisa, sirf amber matching arrow badla:  $S3 \xrightarrow{1/1} S0$ . Matlab match ke baad sab bhool jao.

Moore 1011, overlap:  $S_4 \xrightarrow{0} S_2, \xrightarrow{1} S_1$  (output in the circle)

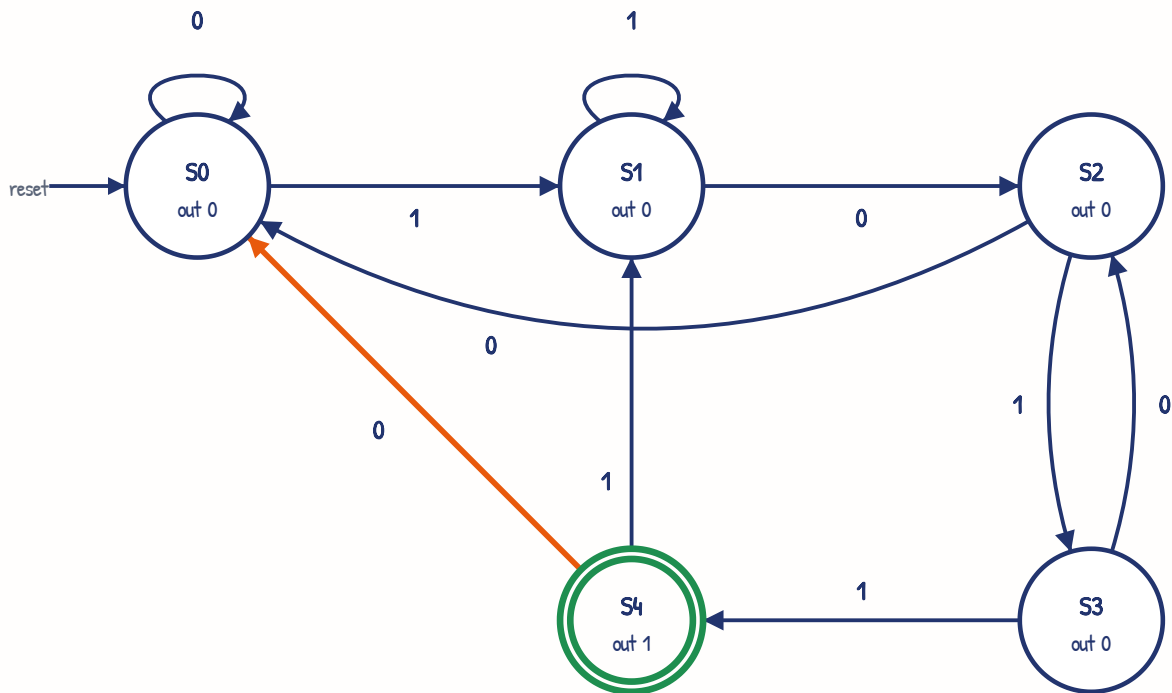


Moore 1011 detector, overlapping (5 states)

How to draw: Five circles; output written inside ( $S_4 = 1$ );  $S_4$  on 1  $\rightarrow$   $S_1$ , on 0  $\rightarrow$   $S_2$ .

**Diagram kaise padhein:** Paanch circle, har circle mein "out 0" ya "out 1" likha hai (Moore). Sirf  $S_4$  (double circle) mein out 1. Arrows pe sirf input. Forward:  $S_0 \xrightarrow{1} S_1 \xrightarrow{0} S_2 \xrightarrow{1} S_3 \xrightarrow{1} S_4$ .  $S_4$  se: 1  $\rightarrow$   $S_1$ , 0  $\rightarrow$   $S_2$  (amber, overlap). Trace "1011":  $S_0, S_1, S_2, S_3, S_4$ ; output 1 tab dikhta hai jab machine  $S_4$  mein pahunch jaati hai (agle cycle mein).

Moore 1011, no overlap:  $S_4 \xrightarrow{0} S_0, \xrightarrow{1} S_1$  (output in the circle)



Moore 1011 detector, non-overlapping

How to draw: Same as Moore overlap but  $S_4$  on  $0 \rightarrow S_0$ .

**Diagram kaise padhein:** D7.5c jaisa, farak sirf amber arrow:  $S_4 \xrightarrow{0} S_0$ .  $S_4 \xrightarrow{1} S_1$  waisa hi rehta hai (naya "1" to naya prefix hai hi).

**Input stream trace: 10110110 (Lab 8 ka shared stream), OVERLAP**

Mealy column: present state  $\rightarrow$  next state, aur output *isi cycle* mein. Moore column: present state, aur output = 1 sirf jab present state  $S_4$  ho.

| Bit no. | Input | Mealy state $\rightarrow$ next | Mealy out | Moore state $\rightarrow$ next | Moore out |
|---------|-------|--------------------------------|-----------|--------------------------------|-----------|
| 1       | 1     | $S_0 \rightarrow S_1$          | 0         | $S_0 \rightarrow S_1$          | 0         |
| 2       | 0     | $S_1 \rightarrow S_2$          | 0         | $S_1 \rightarrow S_2$          | 0         |
| 3       | 1     | $S_2 \rightarrow S_3$          | 0         | $S_2 \rightarrow S_3$          | 0         |
| 4       | 1     | $S_3 \rightarrow S_1$          | 1         | $S_3 \rightarrow S_4$          | 0         |
| 5       | 0     | $S_1 \rightarrow S_2$          | 0         | $S_4 \rightarrow S_2$          | 1         |
| 6       | 1     | $S_2 \rightarrow S_3$          | 0         | $S_2 \rightarrow S_3$          | 0         |
| 7       | 1     | $S_3 \rightarrow S_1$          | 1         | $S_3 \rightarrow S_4$          | 0         |
| 8       | 0     | $S_1 \rightarrow S_2$          | 0         | $S_4 \rightarrow S_2$          | 1         |

Dekho: Mealy bit 4 aur bit 7 pe 1 deta hai; Moore wahi do matches **ek cycle baad** (bit 5 aur bit 8 wale cycle mein) dikhata hai. Total **2 matches** (bits 1-4 aur 4-7, bit 4 share hua).

**Same stream, NON-overlap:**

| Bit no. | Input | Mealy state → next | Mealy out | Moore state → next | Moore out |
|---------|-------|--------------------|-----------|--------------------|-----------|
| 1       | 1     | S0 → S1            | 0         | S0 → S1            | 0         |
| 2       | 0     | S1 → S2            | 0         | S1 → S2            | 0         |
| 3       | 1     | S2 → S3            | 0         | S2 → S3            | 0         |
| 4       | 1     | S3 → S0            | 1         | S3 → S4            | 0         |
| 5       | 0     | S0 → S0            | 0         | S4 → S0            | 1         |
| 6       | 1     | S0 → S1            | 0         | S0 → S1            | 0         |
| 7       | 1     | S1 → S1            | 0         | S1 → S1            | 0         |
| 8       | 0     | S1 → S2            | 0         | S1 → S2            | 0         |

Bit 4 ke baad sab bhool gaye, to bits 5-7 sirf "011" bane: **1 match**.

**Flip-flops:** Mealy 4 states →  $\lceil \log_2 4 \rceil = 2$  FFs (one-hot 4). Moore 5 states →  $\lceil \log_2 5 \rceil = 3$  FFs (one-hot 5).

**Mealy overlap equations** ( $S0 = 00, S1 = 01, S2 = 10, S3 = 11$ , input  $x$ ):  $Q1^+ = Q0 \cdot x' + Q1 \cdot Q0' \cdot x$ ,  $Q0^+ = x$ , out  $z = Q1 \cdot Q0 \cdot x$ .

$Q0^+ = x$  kyun? Table dekho: input 1 pe next state hamesha S1 ya S3 ( $Q0 = 1$ ), input 0 pe hamesha S0 ya S2 ( $Q0 = 0$ ).

### Solved example 1 (class exit ticket):

Trace the Mealy 1011 (overlapping) detector on the input 101011. At which input bit (1 = first) does the output pulse?

- Bit 1 = 1:  $S0 \rightarrow S1$  (out 0). Ab "1" dekha.
- Bit 2 = 0:  $S1 \rightarrow S2$  (out 0). "10".
- Bit 3 = 1:  $S2 \rightarrow S3$  (out 0). "101".
- Bit 4 = 0:  $S3 \rightarrow S2$  (out 0). "1010" ka suffix "10" bacha.
- Bit 5 = 1:  $S2 \rightarrow S3$  (out 0). "101" phir se.
- Bit 6 = 1:  $S3 \rightarrow S1$ , **out 1**. Bits 3-6 = 1011.

**Answer: output pulses only at bit 6.**

### Solved example 2 (count matches):

How many times does a 1011 detector fire on 1011011 (a) with overlap, (b) without overlap?

- Positions likho: bit 1 = 1, 2 = 0, 3 = 1, 4 = 1, 5 = 0, 6 = 1, 7 = 1.
- Bits 1-4 = 1 0 1 1 = match.
- (a) Overlap: bit 4 reuse ho sakta hai. Bits 4-7 = 1 0 1 1 = match. Total 2.
- (b) Non-overlap: bit 4 ke baad S0 se shuru. Bits 5-7 = 0 1 1, sirf 3 bits, match nahi. Total 1.

Answer: (a) 2, (b) 1.

### Solved example 3 (mock variant: same method, different pattern):

Draw the state table of a Mealy machine that detects 110 (overlapping), and of a Moore machine that detects 101 (overlapping).

1. **Mealy 110.** States:  $S_0 = ""$ ,  $S_1 = "1"$ ,  $S_2 = "11"$ .
2.  $S_0$ :  $0 \rightarrow "0" \rightarrow S_0/0$ ;  $1 \rightarrow "1" \rightarrow S_1/0$ .
3.  $S_1$ :  $0 \rightarrow "10"$ : suffixes "10", "0", koi bhi "1" ya "11" nahi  $\rightarrow S_0/0$ ;  $1 \rightarrow "11" \rightarrow S_2/0$ .
4.  $S_2$ :  $1 \rightarrow "111"$ , suffix "11"  $\rightarrow S_2/0$  (self-loop);  $0 \rightarrow "110" = \text{match}$ , output 1; suffixes "10", "0" prefix nahi  $\rightarrow S_0/1$ .
5. **Moore 101.** States:  $S_0 = ""$  (0),  $S_1 = "1"$  (0),  $S_2 = "10"$  (0),  $S_3 = "101"$  (out 1).
6.  $S_0$ :  $0 \rightarrow S_0$ ,  $1 \rightarrow S_1$ .  $S_1$ :  $0 \rightarrow S_2$ ,  $1 \rightarrow S_1$  ("11"  $\rightarrow$  "1").  $S_2$ :  $0 \rightarrow S_0$  ("100"),  $1 \rightarrow S_3$ .
7.  $S_3$  ("101"):  $0 \rightarrow "1010" \rightarrow \text{suffix "10"} \rightarrow S_2$ ;  $1 \rightarrow "1011" \rightarrow \text{suffix "1"} \rightarrow S_1$ .

Answer: Mealy 110:  $S_0 \{0: S_0/0, 1: S_1/0\}$ ,  $S_1 \{0: S_0/0, 1: S_2/0\}$ ,  $S_2 \{0: S_0/1, 1: S_2/0\}$  (2 FFs). Moore 101:  $S_0 \{S_0, S_1, 0\}$ ,  $S_1 \{S_2, S_1, 0\}$ ,  $S_2 \{S_0, S_3, 0\}$ ,  $S_3 \{S_2, S_1, 1\}$  (2 FFs).

**Galti mat karna:** (1) Overlap allowed ho aur match ke baad  $S_0$  pe chale jaana: trailing "1" ek nayi shuruaat hai,  $S_1$  pe jao (Mealy) /  $S_4$  ko  $S_1$  jaisa banao (Moore). (2)  $S_3$  pe 0 aaya to  $S_0$  likhna: "1010" ke aakhir mein "10" hai, isliye  **$S_2$** . (3)  $S_1$  pe 1 aaya to  $S_0$  likhna: "11" ka aakhri "1" bacha hai,  **$S_1$  pe raho**. (4) Moore ko sirf 4 states dena: output 1 ke liye apni state chahiye, isliye 5.

**Trick:** Har galat bit pe khud se poocho: "Abhi tak ke string ka aakhri hissa, pattern ki shuruaat se kitna milta hai?" Utne letters wali state pe jao. Moore  $S_4 = \text{"overlap mein } S_1 \text{ ka judwa, non-overlap mein } S_0 \text{ ka judwa"}$ .

**Ek line mein:** State = "pattern ka kitna hissa mil chuka"; galti pe longest suffix-jo-prefix-hai wali state pe jao; Mealy 1011 = 4 states ( $S_3 \xrightarrow{1/1} S_1$  overlap /  $S_0$  non-overlap), Moore = 5 states ( $S_4$  out 1, ek cycle late); 10110110 pe overlap 2, non-overlap 1.

### Exam mein is unit se kya aata hai $\rightarrow$ kaise solve karein

| Question type                            | Method (steps)                                                                                                                                          |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Address bits for n registers             | 1. 2 ki powers likho. 2. Jo power n ke barabar (ya just badi) ho, uska exponent = bits. $32 \rightarrow 5$ , $64 \rightarrow 6$ , $16 \rightarrow 4$ .  |
| Total bits / flip-flops in register file | registers $\times$ width. $16 \times 8 = 128$ , $32 \times 32 = 1024$ , $64 \times 32 = 2048$ .                                                         |
| Decoder / MUX size for a register file   | 1. $k = \log_2 n$ . 2. Write: k-to-n decoder (n output lines) + n AND gates with RegWrite. 3. Read: n:1 MUX per read port, w bits wide, k select lines. |

|                                                        |                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Draw register file ( $4 \times 32$ or $32 \times 32$ ) | 1. Outer block (D7.2a) with widths. 2. Write port: decoder $\rightarrow$ AND with WE $\rightarrow$ load enables, data bus to all (D7.2b). 3. Read ports: two MUXes (D7.2c). 4. Write "read combinational, write on clock edge" with one reason each.                                                                                                                                                          |
| Why read combinational, write synchronous              | Read: operands needed in the same cycle, MUX needs no clock. Write: at the edge only, so no race / half-written values; read-before-write in one cycle.                                                                                                                                                                                                                                                       |
| Hardwired zero output (lab MCQ)                        | Write to R0 ignored; read of R0 always 00000000.                                                                                                                                                                                                                                                                                                                                                              |
| Flip-flops for an N-state FSM                          | Binary: smallest $k$ with $2^k \geq N$ ( $3 \rightarrow 2$ , $5 \rightarrow 3$ , $6 \rightarrow 3$ , $9 \rightarrow 4$ ). One-hot: $N$ .                                                                                                                                                                                                                                                                      |
| Define FSM / draw block diagram                        | ( $S, I, O, \delta, \lambda$ ); next-state logic $\rightarrow$ state register (clock) $\rightarrow$ output logic, feedback arrow; Mealy adds input $\rightarrow$ output logic arrow.                                                                                                                                                                                                                          |
| Design an FSM (traffic light)                          | 1. Spec. 2. States with meaning + reset. 3. Diagram with every input arrow. 4. State table. 5. Encode ( $\lceil \log_2 N \rceil$ FFs). 6. Next-state + output equations. 7. Circuit.                                                                                                                                                                                                                          |
| Moore vs Mealy comparison                              | Five rows: depends on, notation, timing (Mealy one cycle earlier), glitches, number of states; plus two block diagrams.                                                                                                                                                                                                                                                                                       |
| Design a sequence detector (1011, 110, 101)            | 1. States = prefixes of the pattern ("", "1", "10", "101" for 1011). 2. Forward arrows. 3. For each wrong bit: new string $\rightarrow$ longest suffix that is a prefix $\rightarrow$ that state. 4. Match edge: overlap $\rightarrow$ state of the reusable suffix, non-overlap $\rightarrow$ S0. 5. Moore: add a match state with out 1 that copies the outgoing arrows of that state. 6. Table + FF count. |
| Trace a detector on a stream / count matches           | Make a table: bit no., input, present $\rightarrow$ next, output. Mealy out on the matching arrow (same bit); Moore out when present state = match state (next bit). Count the 1s.                                                                                                                                                                                                                            |

## Unit 8 · Memory: SRAM vs DRAM & the Memory Stack

**Is unit ki kahani:** Unit 7 ka register file flip-flops se bana hai: bahut tez, par sirf 32 words. Ek photo hi lakhon bytes ki hoti hai, OS billions bytes ka. Itni badi memory flip-flops se banana bahut mehenga hai. Is unit mein do sasti memory cells seekhenge: **SRAM** (6 transistor, tez, cache ke liye) aur **DRAM** (1 transistor + 1 capacitor, sasti aur dense, main memory ke liye). Phir dekhenge ki inhe **memory hierarchy** mein kaise layer karte hain taaki computer "upar jitna tez, neeche jitna bada" lage. Unit 9 mein yahi memory CPU se bus ke through judegi (Von Neumann / Harvard).

### 8.1 Register Files; the Need for Bigger Memory

**Pehle socho:** Padhai karte waqt teen jagah cheezein hoti hain: **desk pe kaagaz** (turant haath mein, par 2-3 hi aate hain), **bookshelf** (thoda uthna padta hai, 50 kitaabein), **library** (jaana padta hai, lakhon kitaabein). Sab kuch desk pe rakhna possible nahi: desk itni badi banegi hi nahi, aur bahut mehengi hogi. Computer mein bhi yahi hai: register file = desk, cache (SRAM) = bookshelf, main memory (DRAM) = library. Koi bhi ek memory tez + badi + sasti teeno ek saath nahi ho sakti.

**Definition:** Register files are fast but tiny because each bit is stored in a flip-flop (about 20 or more transistors per bit). Large memories therefore use denser storage cells, **SRAM** (6 transistors per bit) and **DRAM** (1 transistor + 1 capacitor per bit), trading speed for capacity. Since no single technology is fast, large and cheap at the same time, computers combine several of them in a memory hierarchy.

**Mushkil shabd, aasaan matlab:**

#### transistor

Chip ka sabse chhota electronic switch (ON/OFF). Gate, latch, memory cell sab inhi se bante hain. Ek bit mein jitne kam transistors, utni zyada bits ek chip mein aayengi.

#### transistors per bit

Ek bit store karne mein kitne transistors lage. Flip-flop: 20+. SRAM: 6. DRAM: 1 (plus ek capacitor).

#### density

Ek mm<sup>2</sup> chip area mein kitni bits. Kam transistors per bit = zyada density = sasti memory.

#### cost per bit

Ek bit store karne ka kharcha. Area zyada = kharcha zyada.

#### volatile

Power gayi to data gaya. Flip-flop, SRAM, DRAM teeno volatile hain.

#### word / address line

Memory ek lambi list hai; har khaane (word) ka ek number (address). Address bhejne wale wires = address lines.

### Kaise kaam karta hai (problem step by step):

1. Register file:  $32 \text{ registers} \times 32 \text{ bits} = 1024 \text{ bits} = \text{sirf } 128 \text{ bytes}$ . ALU ke bilkul paas, ek cycle se kam mein access.
2. Har bit ek flip-flop hai, aur ek flip-flop  $\approx 20+$  transistors.
3. Agar 1 GB ( $\approx 8 \times 10^9$  bits) flip-flops se banayein:  $8 \times 10^9 \times 20 = 1.6 \times 10^{11}$  transistors sirf memory mein. Chip itni badi aur mehengi ho jayegi ki banana practical nahi.
4. Isliye chhoti cells chahiye: SRAM (6T) caches ke liye, DRAM (1T1C) main memory ke liye.
5. **Core tension:** fast memory = chhoti aur mehengi; badi memory = slow aur sasti. Solution: sab ko layer karo (memory hierarchy, 8.5).

**Formula:** Address lines =  $\log_2(\text{number of words})$  · Capacity =  $2^{\text{address lines}} \times \text{word width}$  ·

Transistors = bits  $\times$  transistors per bit

Same rule as the register file (32 registers  $\rightarrow$  5 bits). Memory with  $2^n$  words needs  $n$  address lines.

### Solved example 1 (transistors per bit):

*How many transistors are needed to store 1 KB (8192 bits) using flip-flops (20 T/bit), SRAM and DRAM?*

1. 1 KB =  $1024 \text{ bytes} \times 8 = 8192 \text{ bits}$ .
2. Flip-flops:  $8192 \times 20 = 163,840$  transistors.
3. SRAM:  $8192 \times 6 = 49,152$  transistors.
4. DRAM:  $8192 \times 1 = 8,192$  transistors (+ 8,192 capacitors).

**Answer:** FF  $\approx 163,840$ ; SRAM 49,152; DRAM 8,192 (+ 8,192 capacitors). Isliye badi memory DRAM se banti hai.

### Solved example 2 (address lines):

*A memory has 64 K words of 8 bits each (64 K = 65,536). How many address lines does it need, and what is its capacity? How many address lines does a 1 M  $\times$  8 memory need?*

1.  $65,536 = 2^{16}$  ( $2^{10} = 1024$ ,  $2^6 = 64$ ,  $1024 \times 64 = 65,536$ ). Isliye address lines = 16.
2. Capacity =  $65,536 \text{ words} \times 8 \text{ bits} = 65,536 \text{ bytes} = 64 \text{ KB}$ .
3.  $1 \text{ M} = 2^{20} = 1,048,576 \text{ words} \rightarrow 20 \text{ address lines, } 8 \text{ data lines, capacity } 1 \text{ MB}$ .

**Answer:** 16 address lines, 64 KB; 1 M  $\times$  8 needs 20 address lines.

**Galti mat karna:** (1) "Registers DRAM se bane hain" galat hai: registers flip-flops hain, caches SRAM, main memory DRAM. (2) "Register file aur bada kar do" free nahi hai: area, power aur access time teeno badhte hain. (3) Address lines width se nahi, **words ki ginti** se aate hain.

**Trick:** Transistor count yaad rakho: **20 → 6 → 1** (flip-flop → SRAM → DRAM). Number jitna chhota, memory utni badi aur slow.

**Ek line mein:** Flip-flop (20+ T/bit) bahut mehenga hai, isliye badi memory SRAM (6T) aur DRAM (1T1C) se banti hai; koi memory tez + badi + sasti ek saath nahi, isliye hierarchy.

## 8.2 SRAM: 6T Cell Structure and Behaviour

**Pehle socho:** Ek **see-saw** socho jisko do log dono taraf se pakad ke rakhte hain: ek side neeche hai to woh neeche hi rahegi, kyunki dono log usse lagatar dhakel rahe hain. Thoda hila bhi to turant wapas apni jagah. SRAM ka bit aise hi "zinda" tarike se pakda rehta hai (**active storage**): do inverters ek doosre ko lagatar sahi value dete rehte hain. Isliye jab tak bijli hai, data rehta hai, refresh ki zarurat nahi. Isiliye naam **Static** RAM.

**Definition: SRAM (Static RAM)** stores each bit in a bistable latch made of two cross-coupled inverters (4 transistors) plus two access transistors that connect the latch to the bit lines when the word line is high, giving a 6-transistor (6T) cell. It keeps its data without refresh as long as power is applied, and its read is non-destructive.

**Mushkil shabd, aasaan matlab:**

**inverter**

NOT gate: 1 andar → 0 bahar. CMOS mein 2 transistors (1 PMOS + 1 NMOS).

**cross-coupled inverters**

Do inverters ek gol ghare mein: pehle ka output doosre ka input, doosre ka output pehle ka input.  $Q = 1 \rightarrow$  doosra inverter  $Q' = 0$  deta hai → pehla inverter  $Q = 1$  hi rakhta hai. Ye loop khud ko sahi rakhta hai.

**bistable latch**

Circuit jiski do stable halat hain ( $Q = 1$  ya  $Q = 0$ ) aur woh kisi ek mein tiki rehti hai. Unit 5 ka basic bistable yahi hai.

**access transistor (M5, M6)**

Darwaze: ON hon to latch bit lines se jud jaata hai, OFF hon to latch akela band rehta hai.

**word line (WL)**

Row select wire.  $WL = 1 \rightarrow$  us row ke saare cells ke darwaze khul jaate hain. Matlab "ye cell chuno".

**bit lines (BL, BL')**

Data wires: BL pe bit, BL' pe uska ulta (complement). Inhi se data andar-bahar jaata hai.

### pre-charge

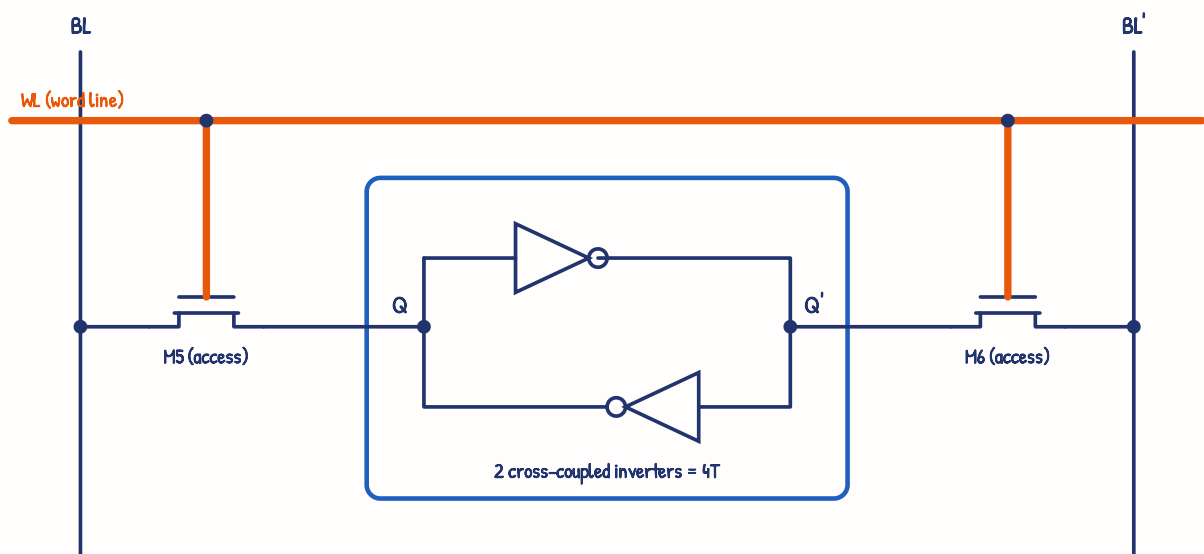
Read se pehle dono bit lines ko 1 (high voltage) pe charge karke tayaar rakhna.

### sense amplifier

Ek sensitive circuit jo dono bit lines ka bahut chhota voltage farak pakad ke use poora 0 ya 1 bana deta hai.

### non-destructive read

Padhne se stored value nahi mit-ti.



4 transistors (2 inverters, each 1 PMOS + 1 NMOS) store the bit actively; WL = 1 connects Q to BL and Q' to BL'.

Total 6T per bit. Reads are non-destructive; the latch holds the value as long as power is on (static).

### 6T SRAM cell

How to draw: Two vertical bit lines (BL, BL'), a word line across the top; in the middle two inverters drawn back-to-back in a loop; an access transistor on each side between the loop node (Q / Q') and its bit line, gates tied to WL.

**Diagram kaise padhein:** Left aur right mein do khadi lines BL aur BL' (data wires). Upar aadi line WL (highlighted). Beech ke box mein do inverters ek loop mein: left node Q, right node Q'. Q se BL tak M5, Q' se BL' tak M6, dono ke gate WL se jude. Count: inverters  $2 \times 2 = 4$  transistors + M5 + M6 = 6. Trace: WL = 0  $\rightarrow$  M5, M6 OFF  $\rightarrow$  latch akela, value hold. WL = 1  $\rightarrow$  Q BL se aur Q' BL' se jud jaate hain.

### Kaise kaam karta hai:

1. **Hold (kuch nahi ho raha):** WL = 0, darwaze band. Maan lo Q = 1, to doosra inverter Q' = 0 banata hai, jo pehle inverter ko Q = 1 rakhne pe majboor karta hai. Thoda leakage hua to power supply turant theek kar deti hai.
2. **Read step 1:** dono bit lines BL aur BL' ko high pe pre-charge karo.
3. **Read step 2:** WL = 1 karo, M5 aur M6 ON.
4. **Read step 3:** jis side 0 stored hai (yahan Q' = 0), woh apni bit line (BL') ko thoda neeche kheenchni hai. BL high hi rehti hai.

5. **Read step 4:** sense amplifier dekhta hai "BL' gira, BL nahi" → output 1. Latch apni value khud pakde hue hai, isliye read **non-destructive**.
6. **Write step 1:** nayi value bit lines pe zor se daalo: 0 likhna hai to  $BL = 0$ ,  $BL' = 1$ .
7. **Write step 2:**  $WL = 1$  karo.
8. **Write step 3:** bit line drivers latch se zyada strong hain, to Q ko 0 aur Q' ko 1 kar dete hain (overpower).
9. **Write step 4:**  $WL = 0$ . Latch palat chuka hai aur ab nayi value hold karta hai.

#### Solved example (write the answer like this):

*Draw the 6T SRAM cell and explain how a stored bit is read. Why is the read non-destructive? (5 marks)*

1. Draw D8.2a: two cross-coupled inverters (4 transistors) holding Q and Q'; access transistors M5, M6 connect Q to BL and Q' to BL'; their gates are driven by the word line WL.
2. Read: both bit lines are pre-charged high; WL is raised; the side storing 0 pulls its bit line slightly low; a sense amplifier detects which bit line dropped and outputs the bit.
3. Write: the bit lines are driven to the new value and its complement; WL is raised; the strong drivers overpower the latch, which flips and holds the new value.
4. The read is non-destructive because the latch actively drives its nodes from the power supply, so it keeps its value while being read.
5. SRAM is static (no refresh), fast (a few ns) but needs 6 transistors per bit, so it is used for caches and register files.

**Answer: diagram + 4 read steps + 4 write steps + "active latch, so non-destructive".**

**Scorecard:** very fast (a few ns) · no refresh · non-destructive read · low density (6T/bit) · expensive · constant leakage (static) power → **caches L1/L2/L3, register files, buffers.**

**Galti mat karna:** (1) "SRAM ko refresh chahiye" galat: refresh DRAM ko chahiye. (2) Latch mein 6 transistors bolna: latch = 4, baaki 2 access transistors hain (M5, M6). (3) "SRAM read data mita deta hai" galat: woh DRAM hai. (4) Word line pe data nahi chalta, data bit lines pe chalta hai; WL sirf select karti hai.

**Trick: 6T = 4 + 2:** "4 pakadte hain (latch), 2 darwaza kholte hain (access)". Word line = Which cell; Bit line = Bit ka data.

**Ek line mein:** SRAM 6T = 2 cross-coupled inverters (4T, bit ko zinda pakadte hain) + 2 access transistors (WL se khulte hain); read = pre-charge, WL, sense amp, non-destructive; refresh nahi, tez, mehenga → cache.

## 8.3 DRAM: 1T1C Cell, Refresh, Destructive Read

**Pehle socho:** Ek **chhed wali baalti** socho. Baalti mein paani = 1, khaali = 0. Problem: paani dheere-dheere tapakta rehta hai, to thodi der mein "1" bhi "0" jaisa dikhne lagega. Isliye har thodi der mein koi aake baalti check karta hai aur bhar deta hai: ye **refresh** hai. Aur baalti itni chhoti hai ki paani dekhne ke liye use bade bartan (bit line) mein undelna padta hai, to baalti khaali ho jaati hai: ye **destructive read** hai, isliye padhne ke baad wapas bharna padta hai. Fayda: baalti bahut chhoti aur sasti hai, to lakhon-karodon rakh sakte ho. Yahi **DRAM** hai: **passive storage**, bahut dense, par refresh aur write-back chahiye.

**Definition: DRAM (Dynamic RAM)** stores each bit as charge on a tiny capacitor that is connected to the bit line through one access transistor (a 1T1C cell). Because the charge leaks away, every row must be refreshed periodically (about every 64 ms), and because reading shares the capacitor's charge with the bit line, the read is destructive and the value must be written back.

**Mushkil shabd, aasaan matlab:**

### capacitor

Charge (bijli) store karne wala chhota "bucket". Charged = 1, empty = 0.

### 1T1C

1 transistor (darwaza) + 1 capacitor (bucket) per bit. Sabse chhota memory cell.

### leakage

Capacitor ka charge dheere-dheere apne aap nikal jaana, jaise baalti se paani tapakna.

### retention time

Kitni der tak charge itna rehta hai ki 1 pehchana ja sake. Class mein: 64 ms ke andar refresh karna hai.

### refresh

Row ko padh ke turant wapas poori value likhna, taaki charge phir se full ho jaaye. Har row lagbhag har 64 ms mein.

### row

DRAM cells ek grid mein hote hain; ek word line ek poori row ko ek saath kholti hai. Refresh row-by-row hota hai.

### charge sharing

Capacitor ko bit line se jodne pe uska charge bit line mein bant jaata hai. Bit line badi hai, to voltage bas thoda sa badalta hai, aur capacitor khaali ho jaata hai.

### sense amplifier

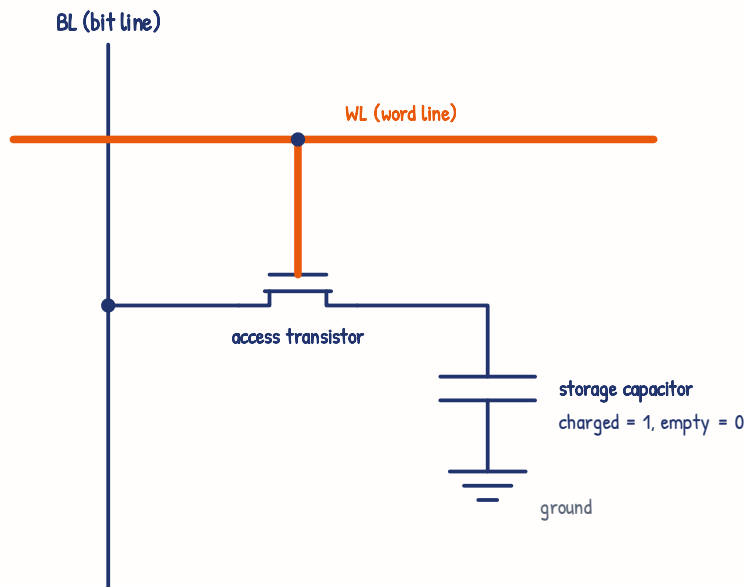
Bit line ke us chhote se badlaav ko pakad ke poora 0 ya 1 bana deta hai, aur wahi value wapas cell mein likh deta hai.

### write-back

Read ke baad value cell mein wapas likhna (kyunki read ne use mita diya).

### **μs, ms, ns**

1 ms = 1000 μs; 1 μs = 1000 ns. 64 ms = 64,000 μs.



WL = 1 opens the transistor and connects the capacitor to the bit line (write: charge/drain it; read: share its charge).

1 transistor + 1 capacitor per bit → about 6× denser than SRAM, but the charge leaks (needs refresh).

### **1T1C DRAM cell**

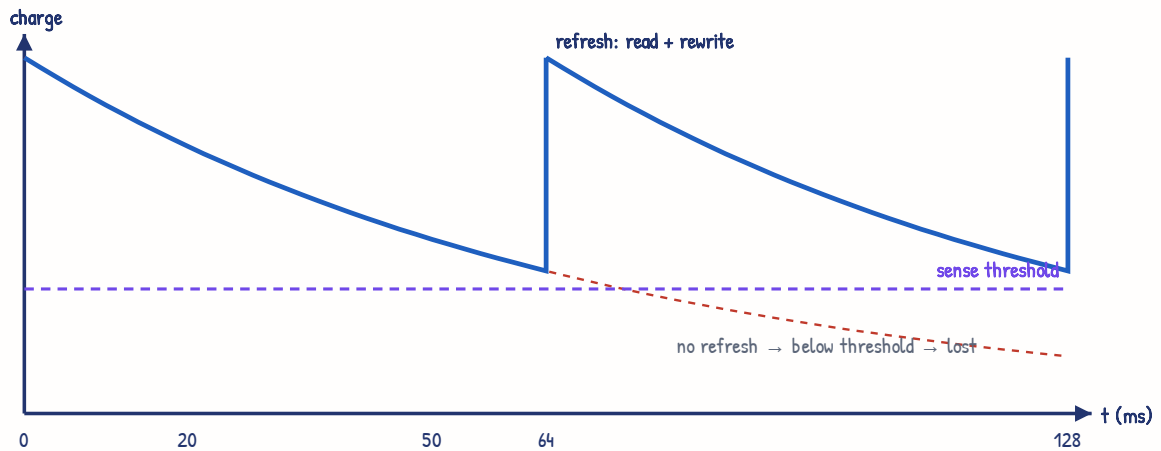
How to draw: Vertical bit line, horizontal word line; one transistor from the bit line to a capacitor whose other plate goes to ground; gate tied to the word line.

**Diagram kaise padhein:** Left mein khadi line = bit line (BL). Upar aadi line = word line (WL). Beech mein ek transistor: ek taraf BL, doosri taraf capacitor (do parallel plates), capacitor ka doosra sira ground. Transistor ka gate WL se juda. Trace: WL = 1 → transistor ON → capacitor BL se juda. Write 1: BL ko high karo → capacitor charge. Write 0: BL low → capacitor khaali.

### **Kaise kaam karta hai:**

1. **Store:** WL = 0, transistor OFF, capacitor akela. Charge hai to 1, nahi to 0.
2. **Write:** BL ko desired value pe rakho, WL = 1. Capacitor charge (1) ya discharge (0) ho jaata hai. WL = 0.
3. **Leakage:** capacitor ka charge dheere-dheere nikalta hai. Class curve: 0 ms pe full, 20 ms pe kam hota hua, 50 ms pe weak, 64 ms tak sense threshold ke neeche (lost).
4. **Refresh:** DRAM controller har row ko ~64 ms ke andar padh ke wapas likhta hai. Us waqt woh hissa memory busy rehta hai (performance + power cost). Isiliye naam **Dynamic**.
5. **Read step 1 (access):** WL = 1. Capacitor ka charge bit line ke saath share hota hai.
6. **Read step 2 (tiny signal):** bit line ka voltage bahut thoda sa upar ya neeche hota hai. Capacitor ab apni value kho chuka hai.
7. **Read step 3 (sense amp):** sense amplifier us chhote farak ko poori 0 ya 1 bana deta hai.
8. **Read step 4 (write-back):** wahi poori value turant capacitor mein wapas likh di jaati hai. Har DRAM read asal mein **read + rewrite** hai.

9. **Speed:** 50–100 ns (sense time + row open karna + pre-charge), SRAM se 10 guna se zyada slow. Use: **main memory** (8/16/32 GB RAM stick), GPU memory (GDDR).



Charge fades within milliseconds: every row is read and rewritten about every 64 ms.

#### Charge leak and refresh (sawtooth)

How to draw: Axes charge vs time; an exponential decay that would drop below the sense threshold by 64 ms; a sawtooth showing refresh restoring full charge every 64 ms.

**Diagram kaise padhein:** X-axis = time (ms), Y-axis = capacitor charge. Dashed curve = bina refresh ke charge girta jaata hai aur "sense threshold" line ke neeche chala jaata hai (data lost). Solid sawtooth = har 64 ms pe refresh charge ko wapas full kar deta hai, isliye charge kabhi threshold ke neeche nahi jaata.

**Formula:** Per-row refresh interval =  $\text{retention time} \div \text{number of rows}$  · Refresh busy fraction =  $\frac{\text{time per row refresh}}{\text{per-row interval}}$  (= rows × time per row ÷ retention time)

64 ms = 64,000  $\mu\text{s}$ . 8192 rows  $\rightarrow 64,000 \div 8192 = 7.8125 \mu\text{s}$ .

#### Solved example 1 (class homework):

A DRAM with 8192 rows must refresh every row within 64 ms. What is the interval between row refreshes? Repeat for 4096 and 16384 rows.

1. ms ko  $\mu\text{s}$  mein badlo:  $64 \text{ ms} \times 1000 = 64,000 \mu\text{s}$ .
2. 8192 rows:  $64,000 \div 8192$ .  $8192 \times 8 = 65,536$  (thoda zyada),  $8192 \times 7.8 = 63,897.6$ ,  $8192 \times 7.8125 = 64,000$ . To  $7.8125 \mu\text{s} \approx 7.8 \mu\text{s}$ .
3. 4096 rows (aadhi rows  $\rightarrow$  double time):  $64,000 \div 4096 = 15.625 \mu\text{s} \approx 15.6 \mu\text{s}$ .
4. 16384 rows (double rows  $\rightarrow$  aadha time):  $64,000 \div 16384 = 3.90625 \mu\text{s} \approx 3.91 \mu\text{s}$ .

**Answer:** 7.8  $\mu\text{s}$  (8192), 15.6  $\mu\text{s}$  (4096), 3.91  $\mu\text{s}$  (16384).

#### Solved example 2 (exam-level, refresh overhead):

A DRAM bank has 8192 rows, a 64 ms retention window, and each row refresh blocks the bank for 50 ns. What fraction of time is spent refreshing? Repeat for 60 ns per row (mock 2).

1. Per-row interval =  $7.8125 \mu s = 7812.5 \text{ ns} (\times 1000)$ .
2. Har  $7812.5 \text{ ns}$  mein  $50 \text{ ns}$  busy. Fraction =  $50 \div 7812.5 = 0.0064$ .
3. Percent:  $0.0064 \times 100 = 0.64 \%$ .
4.  $60 \text{ ns}$ :  $60 \div 7812.5 = 0.00768 \rightarrow 0.77 \%$ .
5. Check doosre tarike se:  $8192 \times 60 \text{ ns} = 491,520 \text{ ns} = 0.49 \text{ ms}$  busy har  $64 \text{ ms}$  mein;  $0.49 \div 64 = 0.0077 \rightarrow 0.77 \%$ .

**Answer: 0.64 % (50 ns), 0.77 % (60 ns).**

**Galti mat karna:** (1) "Transistor bit store karta hai" galat: **capacitor** karta hai, transistor sirf darwaza hai. (2) DRAM read ke baad write-back bhool jaana. (3) Poori chip ka refresh period ( $64 \text{ ms}$ ) aur per-row interval ( $7.8 \mu s$ ) mix karna. (4)  $\text{ms}$  ko  $\mu s$  mein badalna bhoolna ( $\times 1000$ ).

**Trick:** DRAM = Dynamic = Drains (charge tapakta hai)  $\rightarrow$  refresh. " $64 \div 8192 = 7.8$ " seedha yaad kar lo.

**Ek line mein:** DRAM = 1T1C (capacitor mein charge = bit); charge leak hota hai  $\rightarrow$  har row  $\sim 64 \text{ ms}$  mein refresh ( $8192 \text{ rows} \rightarrow 7.8 \mu s$  per row); read charge baant deta hai  $\rightarrow$  sense amp + write-back; dense, sasta,  $50\text{--}100 \text{ ns} \rightarrow$  main memory.

## 8.4 SRAM vs DRAM Comparison

**Pehle socho:** Dono mein "better" koi nahi. SRAM ek **bodyguard** jaisa hai jo bit ko lagatar pakad ke rakhta hai: tez aur bharosemand, par mehenga aur jagah leta hai. DRAM ek **baalti** hai: sasti, chhoti, karodon rakh lo, par bharte rehna padta hai. Har farak ek hi baat se aata hai: SRAM active storage (latch), DRAM passive storage (charge).

**Definition:** SRAM stores a bit actively in a 6-transistor latch, which makes it fast, refresh-free and non-destructive to read but low in density and expensive; DRAM stores a bit passively as charge in a 1T1C cell, which makes it dense and cheap but slower, and it needs refresh and a write-back after every read. SRAM trades density for speed; DRAM trades speed for density.

**Mushkil shabd, aasaan matlab:**

**active storage**

Circuit khud power supply se value ko lagatar sahi rakhta hai (SRAM latch).

**passive storage**

Value bas charge ke roop mein padi hai, koi use pakad ke nahi rakhta (DRAM capacitor).

**GDDR**

Graphics card ki DRAM (GPU memory).

### Kaise yaad karein (table + har row ka KYUN):

| Point        | SRAM                                    | DRAM                           | Kyun?                                                                                                                                       |
|--------------|-----------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Cell         | 6 transistors                           | 1 transistor + 1 capacitor     | Latch ko 2 inverters (4T) + 2 darwaze chahiye; DRAM ko sirf bucket + darwaza.                                                               |
| Storage      | active (latch)                          | passive (charge)               | Inverters lagatar drive karte hain; capacitor bas charge rakhta hai.                                                                        |
| Density      | low                                     | high ( $\approx 6\times$ SRAM) | 6 transistors vs 1: DRAM cell bahut chhota.                                                                                                 |
| Speed        | fast (a few ns)                         | slower (50–100 ns)             | Latch bit line ko zor se chalata hai; DRAM ka chhota charge sense karna, row kholna, pre-charge karna padta hai, aur chip CPU se bahar hai. |
| Refresh      | not needed                              | every row $\approx 64$ ms      | Latch khud ko theek rakhta hai; capacitor leak hota hai.                                                                                    |
| Read         | non-destructive                         | destructive (write-back)       | Latch value pakde rehta hai; capacitor ka charge bit line mein bant jaata hai.                                                              |
| Cost per bit | high                                    | low                            | Zyada area per bit = zyada kharcha.                                                                                                         |
| Power        | constant leakage (static) power         | refresh power                  | SRAM ke inverters hamesha ON; DRAM ko baar-baar refresh karna padta hai.                                                                    |
| Volatile?    | yes                                     | yes                            | Dono ko power chahiye; power gayi to data gaya.                                                                                             |
| Used for     | L1/L2/L3 cache, register files, buffers | main memory, GPU memory (GDDR) | Chhota + tez kaam SRAM; bada + sasta kaam DRAM.                                                                                             |

#### Solved example 1 (class exit ticket):

Give three reasons why SRAM is faster than DRAM.

1. SRAM ka latch bit lines ko actively drive karta hai: strong signal, jaldi sense hota hai. DRAM ka chhota capacitor sirf thoda sa signal deta hai.
2. SRAM mein destructive read / write-back nahi, aur refresh ki rukawat nahi.
3. SRAM chhote arrays mein CPU chip pe hi (same logic process) banti hai; DRAM chip ke bahar hai aur har access mein row activation + pre-charge chahiye.

**Answer: active drive, no write-back/refresh, on-chip small arrays (DRAM off-chip with row activation and pre-charge).**

#### Solved example 2 (match type + density):

Match with SRAM or DRAM: needs refresh; six transistors per bit; used for L1 cache; destructive read; higher density. If DRAM is about  $6\times$  denser, how many DRAM bits fit in the area of 100 SRAM bits?

1. Needs refresh  $\rightarrow$  DRAM (capacitor leaks).
2. Six transistors  $\rightarrow$  SRAM.

3. L1 cache → SRAM (fast).
4. Destructive read → DRAM.
5. Higher density → DRAM.
6. Density:  $100 \times 6 = 600$ .

**Answer:** DRAM, SRAM, SRAM, DRAM, DRAM; 600 bits.

**Galti mat karna:** (1) "DRAM non-volatile hai" galat: dono power ke bina data kho dete hain (non-volatile = flash/SSD). (2) Uses ulta likhna: **cache = SRAM, main memory / RAM stick = DRAM**. (3) "SRAM better hai" likhna: dono alag kaam ke liye hain.

**Trick:** SRAM = Speed, Stable, Small (cache). DRAM = Dense, Dynamic (refresh), Destructive read.

**Ek line mein:** SRAM = 6T active latch (tez, refresh nahi, mehenga, cache); DRAM = 1T1C passive charge (dense  $\approx 6\times$ , sasta, refresh + write-back, main memory); har farak active vs passive se aata hai.

## 8.5 Memory Hierarchy and the Memory Wall

**Pehle socho:** Tum desk pe padh rahe ho. Jo abhi chahiye woh desk pe (registers), thoda pehle wali kitaabein drawer mein (L1), shelf pe (L2/L3), library mein (DRAM), aur purana saamaan godown mein (disk). Tum 90 % baar desk/drawer se hi kaam chala lete ho kyunki **jo abhi use kiya woh phir use karoge**, aur **jo kitaab nikali uske paas wale page bhi padhoge**. Isse poor system **upar jitna tez aur neeche jitna bada** lagta hai. Problem (memory wall): CPU har saal bahut tez hota gaya, par library (DRAM) tak jaane ka time lagbhag waisa hi raha.

**Definition:** The **memory hierarchy** arranges storage in levels (registers, L1, L2, L3 caches, main memory, disk) from small, fast and expensive per bit to large, slow and cheap per bit; it works because programs show **locality**, so most accesses are served by the fast upper levels. The **memory wall** is the growing gap between processor speed and DRAM latency, which makes memory access, not computation, the main limit on performance.

**Mushkil shabd, aasaan matlab:**

**cache**

CPU ke paas chhoti SRAM memory jo haal hi mein use hua data rakhti hai. L1 sabse chhota/tez, phir L2, L3.

**latency**

Maangne se milne tak ka time.  $L1 \approx 1 \text{ ns}$ ,  $DRAM \approx 50\text{--}100 \text{ ns}$ .

**temporal locality**

"Time" wali: jo data abhi use hua, woh jaldi phir use hoga. Loop counter i har iteration mein.

## spatial locality

"Space/jagah" wali: jo address use hua, uske paas wale address jaldi use honge. Array ke elements  $a[0]$ ,  $a[1]$ ,  $a[2]$  ek ke baad ek.

## block (cache line)

Cache ek byte nahi, paas-paas ke bytes ka poora tukda (jaise 64 bytes) ek saath laata hai. Isse spatial locality ka fayda.

## hit / miss

Hit = data cache mein mil gaya (tez). Miss = nahi mila, neeche wale level se laana pada (slow).

## miss rate / miss penalty

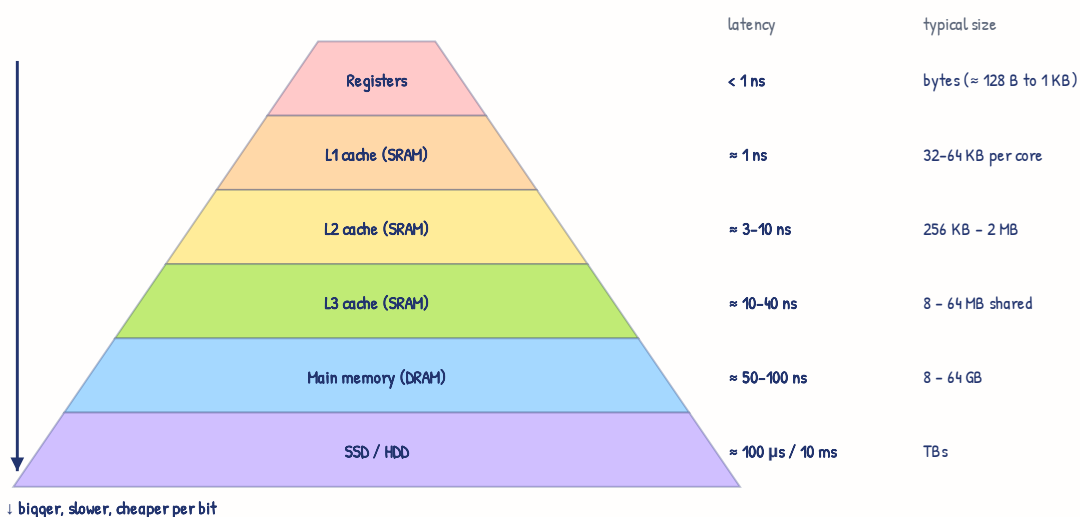
Miss rate = kitne % access miss hue. Miss penalty = miss pe neeche se laane mein extra time.

## AMAT

Average Memory Access Time: average mein ek access kitna time leta hai.

## memory wall

CPU aur DRAM ki speed ka badhta farak, latency ka farak (capacity ka nahi).



↓ bigger, slower, cheaper per bit

Locality (temporal + spatial) lets the small fast levels serve most accesses — the system looks as fast as the top and as big as the bottom.

## Memory hierarchy pyramid

How to draw: Triangle split into six bands (registers at the narrow top, disk at the wide base); write latency and size beside each band; arrow downward "bigger, slower, cheaper".

**Diagram kaise padhein:** Pyramid ka sabse patla upar wala hissa = registers (sabse chhote, sabse tez). Neeche jaate hue har band chhota: L1, L2, L3 (SRAM), main memory (DRAM), sabse neeche SSD/HDD (TBs). Right mein har level ka latency aur size likha hai. Left ka arrow: neeche jao to bada, slow, sasta per bit. Upar = tez + chhota + mehenga.

| Level     | Technology | Latency           | Size              |
|-----------|------------|-------------------|-------------------|
| Registers | flip-flops | < 1 ns            | ≈ 1 KB            |
| L1        | SRAM       | ≈ 1 ns (4 cycles) | 32–64 KB per core |
| L2        | SRAM       | ≈ 3–10 ns         | 256 KB – 2 MB     |
| L3        | SRAM       | ≈ 10–40 ns        | 8 – 64 MB shared  |

|             |                  |                                                 |           |
|-------------|------------------|-------------------------------------------------|-----------|
| Main memory | DRAM             | $\approx 50\text{--}100\text{ ns}$              | 8 – 64 GB |
| SSD / HDD   | flash / magnetic | $\approx 100\text{ }\mu\text{s} / 10\text{ ms}$ | TBs       |

### Kaise kaam karta hai:

1. CPU ko ek data chahiye. Pehle L1 cache dekhta hai.
2. Mil gaya (**hit**)  $\rightarrow \approx 1\text{ ns}$  mein kaam ho gaya.
3. Nahi mila (**miss**)  $\rightarrow$  L2 dekho, phir L3, phir DRAM. Jahan mila, wahan se data (poora block) upar wale levels mein copy ho jaata hai.
4. Har level apne neeche wale level ka ek chhota hissa (subset) rakhta hai.
5. **Temporal locality** ka fayda: abhi laya data cache mein rehta hai, agli baar hit. Example:

```
sum = 0;
for (i = 0; i < 1000; i++)
    sum = sum + a[i];
```

sum aur i har iteration mein phir use hote hain  $\rightarrow$  temporal. Loop ke instructions bhi baar-baar chalte hain  $\rightarrow$  temporal.

6. **Spatial locality** ka fayda: a[0] laane pe poora block (a[0] se a[15], agar 64-byte block aur 4-byte int) aa jaata hai, to a[1] se a[15] sab hit. Instructions bhi ek ke baad ek address pe hain  $\rightarrow$  spatial.
7. Result: zyadatar accesses upar ke tez levels se, isliye average time chhota.

### Formula: $\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}$

hit time = fast level ka time; miss rate = fraction (5 % = 0.05); miss penalty = neeche se laane ka extra time.

### Solved example 1 (AMAT):

L1 hit time 1 ns, miss rate 5 %, miss penalty 80 ns. Find AMAT. What if the miss rate is halved?

1. 5 % = 0.05.
2.  $0.05 \times 80 = 4\text{ ns}$ .
3.  $\text{AMAT} = 1 + 4 = 5\text{ ns}$ .
4. Miss rate 2.5 % = 0.025:  $0.025 \times 80 = 2$ ;  $\text{AMAT} = 1 + 2 = 3\text{ ns}$ .

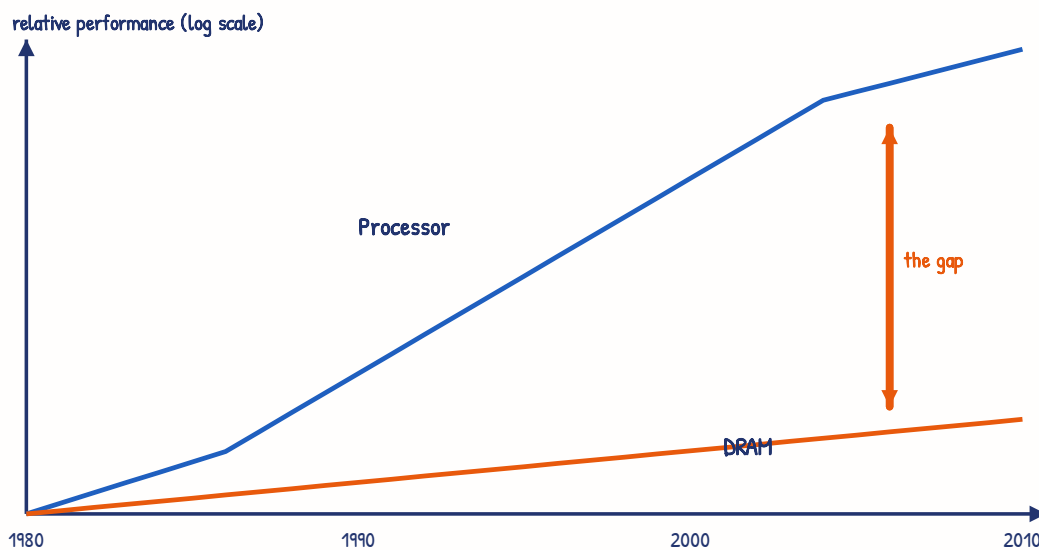
**Answer: 5 ns; halving the miss rate gives 3 ns (isliye cache itna important hai).**

### Solved example 2 (exam-level):

Hit time 2 ns, miss rate 10 %, miss penalty 100 ns. AMAT? Also order L2, DRAM, registers, L1, SSD from fastest to slowest.

1. 10 % = 0.1.  $0.1 \times 100 = 10\text{ ns}$ .
2.  $\text{AMAT} = 2 + 10 = 12\text{ ns}$ .
3. Order: registers (flip-flops, ALU ke paas)  $\rightarrow$  L1 (chhota number = CPU ke paas)  $\rightarrow$  L2  $\rightarrow$  DRAM  $\rightarrow$  SSD.

Answer: 12 ns; registers, L1, L2, DRAM, SSD.



CPU speed grew  $\approx 50\%$  per year for two decades; DRAM latency improved only  $\approx 7\%$  per year (Hennessy & Patterson).

#### Processor-memory performance gap

How to draw: Log-scale axes vs year; a steep processor curve and a nearly flat DRAM curve; double-headed arrow labelled "the gap".

**Diagram kaise padhein:** X-axis = year (1980 se 2010), Y-axis = relative performance (log scale: har step 10 guna). Upar ki steep line = Processor ( $\approx 50\%$  per year, 1986–2004). Neeche ki lagbhag flat line = DRAM latency ( $\approx 7\%$  per year). Dono ke beech ka double arrow = "the gap" = memory wall. Gap har saal badhta gaya.

#### Memory wall step by step:

1. CPU performance  $\approx 50\%$  per year badha (1986–2004); DRAM latency sirf  $\approx 7\%$  per year sudhri.
2. Rough idea: 10 saal mein CPU  $1.5^{10} \approx 58$  guna, DRAM  $1.07^{10} \approx 2$  guna. Farak  $\approx 29$  guna badh gaya.
3. Aaj: 3 GHz CPU ka ek cycle = 0.33 ns. DRAM access 100 ns =  $100 \times 3 = 300$  cycles ka intezaar.
4. Matlab ALU khaali baitha rehta hai, memory hi limit hai (Wulf & McKee, 1995).
5. Responses: deep multi-level caches, prefetching (data pehle se le aana), out-of-order execution (wait karte hue doosre instructions chalana), wider/faster DRAM interfaces, 3-D stacked memory (HBM).

#### Solved example 3 (write the answer like this):

Draw the memory hierarchy and explain locality and the memory wall. (5 marks)

1. Draw D8.5a: registers, L1, L2, L3 (SRAM), main memory (DRAM), SSD/HDD, with latency and size; going down, levels are larger, slower and cheaper per bit.
2. The hierarchy works because of locality. Temporal locality: recently used data is reused soon (loop variables). Spatial locality: nearby addresses are used soon (arrays, sequential code); caches fetch whole blocks.
3. So most accesses hit the small fast levels; AMAT = hit time + miss rate  $\times$  miss penalty.
4. Memory wall: processor performance grew about  $50\%$  per year while DRAM latency improved about  $7\%$  per year, so a DRAM access now costs hundreds of CPU cycles.

5. Remedies: multi-level caches, prefetching, out-of-order execution, wider/faster DRAM interfaces, HBM.

**Answer:** pyramid + two types of locality with examples + 50 % vs 7 % gap + remedies.

**Galti mat karna:** (1) L2 ko L1 ke upar rakhna: chhota number = CPU ke zyada paas = zyada tez. (2) "Memory wall = memory capacity ruk gayi" galat: ye **latency (speed) ka gap** hai. (3) Temporal aur spatial ulta karna: **temporal = same data phir se, spatial = paas wala data**. (4) Registers hata dena memory wall ka solution nahi; registers sabse tez level hain.

**Trick:** Temporal = Time = "phir se wahi". Spatial = Space = "padosi". Hierarchy = desk → drawer → shelf → library → godown (registers → L1 → L2/L3 → DRAM → disk).

**Ek line mein:** Registers → L1 → L2 → L3 → DRAM → SSD/HDD (neeche bada, slow, sasta); locality (temporal + spatial) se zyadatar access upar se serve hote hain; AMAT = hit + miss rate × penalty; memory wall = CPU 50 %/yr vs DRAM 7 %/yr ka latency gap.

**Exam mein is unit se kya aata hai → kaise solve karein**

| Question type                                         | Method (steps)                                                                                                                                                                                         |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why several memory types / why not flip-flops for RAM | Flip-flop 20+ T/bit, SRAM 6T, DRAM 1T1C; fast = small + expensive, big = slow + cheap; no single memory is all three → hierarchy.                                                                      |
| Transistors / address lines numericals                | Transistors = bits × T per bit (20, 6, 1). Address lines = $\log_2(\text{words})$ ; capacity = $2^{\text{lines}} \times \text{width}$ .                                                                |
| Draw + explain 6T SRAM, read/write                    | 1. D8.2a with WL, BL, BL', Q, Q', M5, M6. 2. Read: pre-charge, WL high, 0-side bit line dips, sense amp, non-destructive. 3. Write: drive bit lines, WL high, overpower latch. 4. Static = no refresh. |
| MCQ: transistors in SRAM latch / cell                 | Latch 4, access 2, cell 6. DRAM 1T1C.                                                                                                                                                                  |
| Draw 1T1C DRAM, why refresh, why destructive          | 1. D8.3a. 2. Capacitor leaks → refresh every row ~64 ms (D8.3b). 3. Read = charge sharing → tiny signal → sense amp → write-back.                                                                      |
| Refresh interval per row                              | 1. ms → $\mu\text{s}$ (× 1000). 2. Divide by rows. 64 ms / 8192 = 7.8 $\mu\text{s}$ ; / 4096 = 15.6 $\mu\text{s}$ ; / 16384 = 3.91 $\mu\text{s}$ .                                                     |
| Refresh overhead fraction                             | 1. Per-row interval in ns. 2. Time per row refresh ÷ interval × 100. 50 ns → 0.64 %; 60 ns → 0.77 %.                                                                                                   |
| SRAM vs DRAM table (six points) + uses                | Cell, storage, density, speed, refresh, read, cost, use; add one "why" from active vs passive. SRAM: caches/RF; DRAM: main memory/GDDR.                                                                |
| Three reasons SRAM is faster                          | Active drive of bit lines; no write-back / refresh; small on-chip arrays (DRAM off-chip, row activation + pre-charge).                                                                                 |

|                      |                                                                                                                                                            |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Order the levels     | registers < L1 < L2 < L3 < DRAM < SSD < HDD (fastest to slowest).                                                                                          |
| AMAT                 | Convert % to decimal; AMAT = hit time + miss rate $\times$ miss penalty ( $1 + 0.05 \times 80 = 5$ ns; $2 + 0.1 \times 100 = 12$ ns).                      |
| Locality example MCQ | Same variable reused (i, sum, loop code) = temporal; neighbouring addresses (array elements, sequential instructions, same block) = spatial.               |
| Memory wall          | Latency gap: CPU $\sim 50$ %/yr vs DRAM $\sim 7$ %/yr (D8.5b); hundreds of cycles per DRAM access; fixes: caches, prefetch, out-of-order, wider DRAM, HBM. |

## Unit 9 · Von Neumann vs Harvard Architecture

**Is unit ki kahani:** Units 1–8 mein humne saare parts bana liye: gates, ALU, flip-flops, register file, SRAM, DRAM. Ab sawaal hai ki **inhe jodein kaise?** CPU aur memory ke beech kitne raaste (buses) hon, aur program kahan rakha jaaye? Do classic blueprints hain: **Von Neumann** (ek memory, ek bus: simple, par bus pe traffic jam = bottleneck) aur **Harvard** (instructions aur data ke liye alag memory, alag bus). Aaj ke CPUs dono ka mix hain: **modified Harvard** (andar split L1 cache, bahar ek DRAM). Unit 10 mein dekhenge ki CPU ko kaunsi instructions di jaati hain (CISC vs RISC).

### 9.1 From Gates to a Complete Machine

**Pehle socho:** Ek **sheher** socho. Factories (CPU) saamaan banati hain, godown (memory) saamaan rakhte hain, bandargah / port (I/O) bahar ki duniya se len-den karta hai, aur **sadkein (bus)** sabko jodti hain. Factory kitni bhi tez ho, agar sadak ek hi hai aur patli hai to sheher dheema chalega. Isliye **parts ke saath-saath unka jodne ka tarika (organisation) bhi speed decide karta hai.**

**Definition:** A complete computer consists of a **CPU** (compute), **memory** (store programs and data), **I/O** (communicate with the outside world) and a **bus** (a set of address, data and control lines that connects them). **Computer organisation** is the study of how exactly these parts are connected, for example how many buses join the CPU and memory.

**Mushkil shabd, aasaan matlab:**

#### CPU

Central Processing Unit: control unit + ALU + registers. Hisaab karta hai aur sabko chalata hai.

#### I/O

Input/Output: keyboard, screen, disk, network. Computer ka bahar se baat karne ka tarika.

#### bus

Wires ka group jo parts ko jodta hai. Teen hisse: **address lines** (kahan: "address 0x40"), **data lines** (kya: "value 42"), **control lines** (kya karna hai: "read" ya "write").

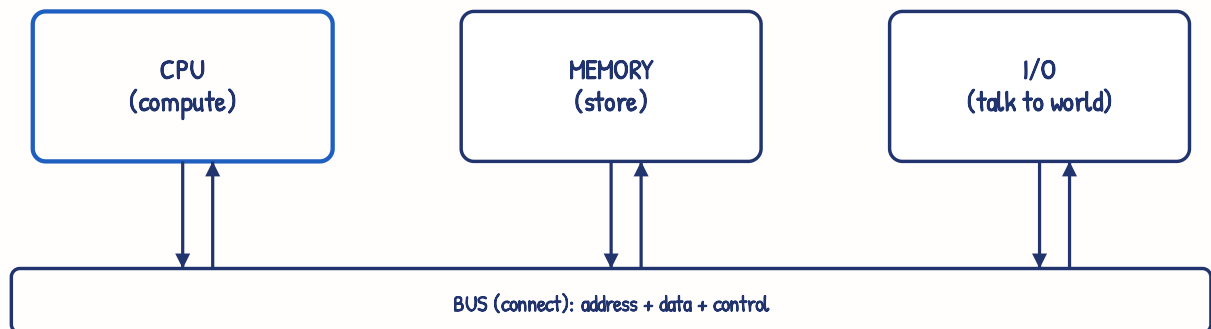
#### computer organisation

Hardware ko kaise joda gaya hai (kitni buses, kitni memories). ISA (instruction set) isse alag hai: woh programmer ko dikhne wala "contract" hai.

**Kaise kaam karta hai:**

- Units 1–8: gates → adder/ALU → flip-flops → registers/register file → SRAM/DRAM. Saare parts tayaar.
- CPU ko memory se ek word padhna hai: address lines pe address daalta hai (0x40).

- Control line pe "read" signal deta hai.
- Memory data lines pe value bhejti hai (0x2A), CPU le leta hai.
- I/O devices bhi isi tarah bus se jude hain.
- Design ka sawaal: CPU aur memory ke beech **ek** bus ho ya **do**? Jawab se do blueprints bante hain: Von Neumann (ek) aur Harvard (do).



Computer organisation asks: how exactly do these connect – and how many buses?

#### CPU – Memory – I/O on a shared bus

How to draw: Three boxes in a row above one long horizontal bus; two-way arrows from each box to the bus.

**Diagram kaise padhein:** Upar teen box: CPU (compute), MEMORY (store), I/O (talk to world). Neeche ek lambi patti = BUS (address + data + control). Har box se bus tak do arrows (upar-neeche) = do-taraf ka len-den. Exam mein yahi 4 blocks aur unka kaam likhna hai.

#### Solved example (write the answer like this):

*Draw the block diagram of a basic computer system and explain the role of each component. (3 marks)*

1. Draw D9.1a: CPU, memory and I/O connected to one system bus.
2. CPU: executes instructions; it contains the control unit, ALU and registers.
3. Memory: stores the program and the data.
4. I/O: connects the computer to keyboards, screens, disks and networks.
5. Bus: carries address, data and control signals between these units.

**Answer: diagram + one line for each of CPU, memory, I/O, bus.**

**Galti mat karna:** "Bus sirf data le jaati hai" galat hai: bus mein **address + data + control** teeno lines hoti hain. Aur "kitni buses" wala sawaal **computer organisation** ka hai, ISA ka nahi.

**Trick: C-M-I-B = "Compute, Memorise, Interact, Bridge".** Bus ki lines: "**A-D-C**": Address (kahan), Data (kya), Control (kaise).

**Ek line mein:** Computer = CPU + memory + I/O + bus (address, data, control); organisation = in parts ko kaise jodte hain, khaaskar kitni buses.

## 9.2 Von Neumann Architecture: Structure and History

**Pehle socho:** 1945 se pehle computer ek hi kaam ke liye **wire kiye jaate the**. Naya program chalana hai? Plugs aur wires hath se badlo, din lag jaate the. Jaise ek **washing machine** jo sirf ek tarah ka wash kar sake, doosra wash chahiye to machine kholo. Von Neumann ka idea: **program ko bhi numbers bana ke usi memory mein rakh do jahan data hai**. Naya program = memory mein naye numbers load karo. Machine wahi, kaam naya. Yahin se "software" paida hua.

**Definition:** The **Von Neumann (stored-program) architecture** keeps program instructions and data, both encoded as numbers, in the same memory, and moves both over a single shared bus between the memory and the CPU. Its four classic components are the control unit, the ALU, the memory and I/O. It was described by John von Neumann in the "First Draft of a Report on the EDVAC" (1945).

**Mushkil shabd, aasaan matlab:**

### **stored-program concept**

Program instructions ko numbers bana ke memory mein data ke saath rakhna. `add r3, r1, r2` bhi memory mein bas ek 32-bit number hai.

### **control unit**

CPU ka "manager": memory se instruction laata (fetch) hai, samajhta (decode) hai, aur baaki parts ko signals deta hai. Ye ek FSM hai (Unit 7).

### **ALU**

Arithmetic Logic Unit: add, subtract, compare, AND/OR karta hai.

### **unified memory**

Ek hi memory, ek hi address space: address `0x00` pe instruction ho sakti hai, `0x40` pe data.

### **PC (program counter)**

Register jo batata hai agli instruction kis address pe hai.

### **IR (instruction register)**

Register jisme abhi chal rahi instruction rakhi hai.

### **EDVAC**

Ek early computer jiske liye von Neumann ne 1945 mein ye design likha.

### **JIT compiler**

Just-In-Time: program chalte-chalte naya machine code banata hai (Java, JavaScript). Ye tabhi possible hai jab code = data.

### Stored program: code and data side by side in ONE memory (class slide)

| Address | Contents       | Kind        |
|---------|----------------|-------------|
| 0x00    | lw r1, 0x40    | instruction |
| 0x04    | add r3, r1, r2 | instruction |
| 0x08    | sw r3, 0x44    | instruction |
| 0x40    | 0x0000002A     | data        |
| 0x44    | 0x00000000     | data        |

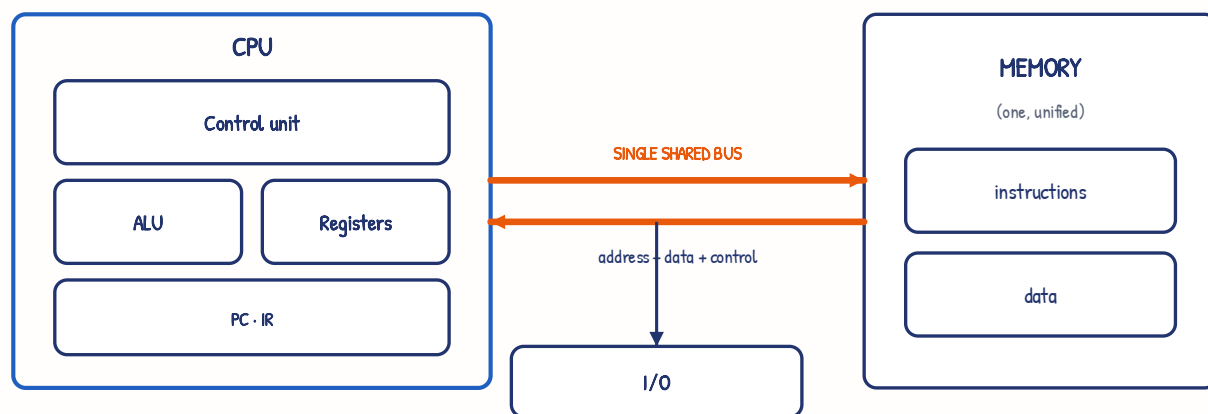
#### Stored-program memory map

How to draw: A two-column table of addresses and contents: three instructions at 0x00–0x08, data words at 0x40 and 0x44.

**Diagram kaise padhein:** Ek hi memory ki table: Address, Contents, Kind. 0x00, 0x04, 0x08 pe teen instructions (lw, add, sw), highlighted. 0x40 aur 0x44 pe data words (0x0000002A = 42 decimal, aur 0). Dhyani do: instructions aur data **same memory** mein side-by-side hain. Program ka matlab: 0x40 se 42 load karo (r1), r1 + r2 karo (r3), r3 ko 0x44 pe store karo.

#### Kaise kaam karta hai (ek instruction ka safar):

1. PC = 0x00. Control unit bus pe address 0x00 bhejta hai: **instruction fetch**. Memory "lw r1, 0x40" bhejti hai, woh IR mein jaati hai.
2. Control unit decode karta hai: "memory se load karna hai".
3. Ab **wahi bus** pe address 0x40 jaata hai: **data access**. Memory 0x2A bhejti hai, woh r1 mein.
4. PC = 0x04. Next instruction "add r3, r1, r2" fetch hoti hai (bus phir use hui). ALU jodta hai.
5. PC = 0x08. "sw r3, 0x44" fetch, phir wahi bus se r3 ki value 0x44 pe likhi jaati hai.
6. Har kadam pe instructions aur data **ek hi bus** pe baari-baari chalte hain. Yahi design ki pehchaan hai (aur limit bhi, 9.3).



Instructions and data take turns on the one bus — the defining feature and the built-in limit.

### Von Neumann block diagram

How to draw: CPU box (control unit, ALU, registers) on the left, one memory box holding both instructions and data on the right, a single two-way bus between them; I/O on the bus.

**Diagram kaise padhein:** Left mein CPU box: andar Control unit, ALU, Registers, PC · IR. Right mein ek MEMORY box " (one, unified)" jisme instructions aur data dono hain. Beech mein highlighted do arrows = **SINGLE SHARED BUS** (address + data + control). Neeche I/O bhi isi bus se juda. Ek hi raasta, isliye ek waqt pe ek hi transfer.

### "Programs are data" ka matlab aur asar:

1. Compiler aur assembler program ko data ki tarah *likhte* hain (output file = numbers).
2. OS program ko chalane ke liye bas uske bytes memory mein copy karta hai (loader).
3. JIT compiler chalte waqt naya code banata hai.
4. Self-modifying code possible hai, aur yahi kai security attacks (code injection) ki jad hai: attacker data ki jagah code ghusa deta hai.

**History aur kyun jeeta:** John von Neumann ne **1945** mein "First Draft of a Report on the EDVAC" mein likha (Eckert, Mauchly aur Turing ke kaam pe based). Jeeta kyunki (1) **simplicity**: ek memory, ek bus, sasta; (2) **generality**: wahi hardware koi bhi program chala sakta hai. Lagbhag har general-purpose computer isi model pe chalta hai.

### Solved example (write the answer like this):

*Explain the stored-program concept and draw the Von Neumann architecture, naming its four classic components. (5 marks)*

1. Stored-program concept: instructions are encoded as numbers and stored in the same memory as data; to run a different program, different numbers are loaded, so no rewiring is needed (memory map D9.2b).
2. Draw D9.2a: CPU (control unit, ALU, registers, PC, IR), one unified memory, one shared bus, I/O on the bus.
3. Control unit fetches and decodes instructions and directs all other units; ALU performs arithmetic and logic; memory holds program and data with uniform addressing; I/O communicates with the

outside world.

4. Described by John von Neumann in the First Draft of a Report on the EDVAC (1945).
5. Advantages: simplicity (one memory, one bus) and generality (any program on the same hardware).

**Answer:** stored-program idea + diagram + four components with roles + 1945 EDVAC + simplicity/generality.

**Galti mat karna:** (1) "Von Neumann mein instruction aur data ki alag memory hai" galat: woh **Harvard** hai. Von Neumann = **ek** memory, **ek** bus. (2) Stored-program idea ko Harvard Mark I ka bolna galat: Mark I mein program paper tape pe tha, memory mein nahi. (3) Four components mein "bus" ya "registers" likhna: classic chaar = **Control unit, ALU, Memory, I/O**.

**Trick:** Von Neumann = "**ek ghar, ek darwaza**": code aur data ek hi ghar (memory) mein, ek hi darwaze (bus) se aate-jaate hain. Chaar parts: "**CAMI**" = Control, ALU, Memory, I/O.

**Ek line mein:** Von Neumann = stored program (code bhi numbers, data ke saath ek memory mein) + ek shared bus; parts: control unit, ALU, memory, I/O; EDVAC report 1945; simple aur general.

### 9.3 The Von Neumann Bottleneck

**Pehle socho:** Ek dukaan socho jiska **sirf ek darwaza** hai, aur woh itna patla ki ek waqt mein ek hi aadmi nikal sake. Andar ka kaarigar (CPU) bahut tez hai. Usko do cheezein chahiye: **order slip** (instruction: kya banana hai) aur **kachcha maal** (data). Dono usi ek darwaze se aate hain, baari-baari. Kaarigar kitna bhi tez ho, **darwaza hi speed tay karta hai**; aadha time woh khaali baitha intezaar karta hai. Yahi **Von Neumann bottleneck** hai (bottleneck = botal ki patli gardan, jahan se sab dheere nikalta hai).

**Definition:** The **Von Neumann bottleneck** (term coined by John Backus in 1977) is the performance limit caused by instructions and data sharing a single bus between CPU and memory: an instruction fetch and a data access cannot happen in the same cycle, and the CPU can compute faster than the bus can supply instructions and data.

**Mushkil shabd, aasaan matlab:**

**bottleneck**

Kisi system ka sabse dheema hissa jo poore system ki speed rok deta hai. Botal ki gardan.

**bus contention**

Do kaam ek hi bus maang rahe hain (fetch aur data); ek ko rukna padega.

**IPC**

Instructions Per Cycle: ek clock cycle mein kitni instructions पूरी होती हैं। Bus एक ही transfer दे सकती है तो IPC पे cap लग जाता है।

### bandwidth

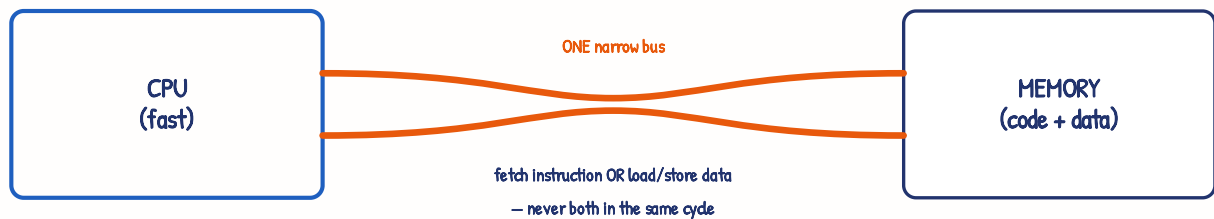
Bus एक second में किता data ले जा सकती है। Unit: bits/s या bytes/s (GB/s).

### GHz

$10^9$  cycles per second. 1 GHz = हर second 1,000,000,000 cycles.

### Gbit/s vs GB/s

1 byte = 8 bits. Gbit/s को 8 से divide करो तो GB/s. 64 Gbit/s = 8 GB/s.



Backus (1977) called this traffic jam the "von Neumann bottleneck": the CPU computes faster than one bus can feed it.

### The single-bus bottleneck

How to draw: CPU and memory boxes joined by a bus drawn pinched in the middle; label "fetch OR data, never both".

**Diagram kaise padhein:** Left में CPU (fast), right में MEMORY (code + data). बीच की दो lines बीच में patli हो जाती हैं = "ONE narrow bus". नीचे लिखा: "fetch instruction OR load/store data, never both in the same cycle". Wahi patla hissa bottleneck है।

### Kaise kaam karta hai (kyun performance rukti hai):

1. **Bus contention:** 1 instruction को पहले fetch (bus use), फिर data load (bus फिर use). Dono एक साथ नहीं।
2. **IPC cap:** instructions किती तेजी से आती हैं, वो bus decide करती है, ALU नहीं।
3. **Idle compute units:** data का इंतजार करते हुए ALU खाली बैठा रहता है।
4. **Worsens with CPU speed:** CPU तेज होता गया, bus उतनी तेज नहीं बढ़ी, तो gap बढ़ता गया (Unit 8 का memory wall भी इसी से जुड़ा है)।
5. **Responses:** caches (गर्म code और data chip पे ही rakho), wider/faster buses, Harvard-style अलग रास्ते।

### Solved example 1 (cycle count, single door):

A program has 10 instructions, of which 4 are loads/stores (each needs one data access). Each memory transfer takes 1 bus cycle and there is no cache. How many bus cycles does memory traffic take on a Von Neumann machine and on a Harvard machine?

1. हर instruction को fetch करना है: 10 fetches.

2. Loads/stores ka data access: 4.

3. Von Neumann: ek hi bus, sab baari-baari:  $10 + 4 = 14$  bus cycles.

4. Harvard: fetch I-bus pe, data D-bus pe, ek saath. 4 data accesses apni instruction ke saath overlap ho jaate hain, to sirf 10 fetch cycles: **10 cycles**.

5. Farak:  $14 \div 10 = 1.4$  guna. Agar har instruction load/store hoti ( $10 + 10 = 20$  vs  $10$ ), to 2 guna. Isliye "up to  $2\times$ ".

**Answer: Von Neumann 14 cycles, Harvard 10 cycles ( $1.4\times$ ; best case  $2\times$ ).**

**Formula:** Bus bandwidth (bytes/s) = bits moved per cycle  $\times$  clock rate  $\div 8$

Von Neumann: one bus carries instruction bits + data bits. Harvard: each bus carries only its own share.

**Solved example 2 (class worked, p28):**

*A 1 GHz CPU must fetch a 32-bit instruction and access a 32-bit data word every cycle. Find the required bandwidth of the single Von Neumann bus, and of each Harvard bus.*

1. Bits per cycle (VN) =  $32 + 32 = 64$  bits.

2. Per second:  $64 \times 10^9 = 64$  Gbit/s.

3. Bytes:  $64 \div 8 = 8$  GB/s.

4. Harvard: I-bus  $32 \times 10^9 \div 8 = 4$  GB/s; D-bus bhi 4 GB/s.

**Answer: 8 GB/s on the single VN bus; 4 GB/s + 4 GB/s on the Harvard buses.**

**Solved example 3 (class homework + mock):**

*(a) 2 GHz, 64-bit instruction + 64-bit data per cycle. (b) 2 GHz, 32-bit instruction + 32-bit data per cycle. Required Von Neumann bus bandwidth?*

1. (a)  $64 + 64 = 128$  bits per cycle.  $128 \times 2 \times 10^9 = 256$  Gbit/s.  $256 \div 8 = 32$  GB/s. Harvard: 16 GB/s per bus.

2. (b)  $32 + 32 = 64$  bits.  $64 \times 2 \times 10^9 = 128$  Gbit/s.  $128 \div 8 = 16$  GB/s.

**Answer: (a) 32 GB/s (Harvard 16 + 16); (b) 16 GB/s.**

**Galti mat karna:** (1) ALU ko dosh dena: bottleneck **shared raasta (bus)** hai, ALU nahi. (2) Bytes/s maanga ho to 8 se divide karna bhoolna. (3) Term ka credit von Neumann ko dena: **John Backus, 1977** ne naam diya.

**Trick: "Ek darwaza, do mehmaan:** instruction aur data ek saath andar nahi aa sakte." Bandwidth: "**bits jodo, GHz se guna, 8 se bhaag**".

**Ek line mein:** Bottleneck = ek bus pe instruction fetch aur data access baari-baari (Backus 1977);  
effects: contention, IPC cap, idle ALU, CPU ke saath badhta; fixes: cache, wider bus, Harvard split;  
bandwidth = bits × GHz ÷ 8.

## 9.4 Harvard Architecture: Separate I and D Memories

**Pehle socho:** Dukaan ka ek darwaza problem tha. Solution: **do darwaze** bana do: ek sirf order slips (instructions) ke liye, ek sirf kachche maal (data) ke liye. Ab dono ek saath andar aa sakte hain, kaarigar ko intezaar nahi. Kharcha: do darwaze, do raaste, zyada deewar todni padi. Aur ab order slip wale darwaze se maal nahi aa sakta (code aur data alag duniya). Yahi **Harvard** hai: **alag instruction memory + alag data memory + do alag buses**.

**Definition:** The **Harvard architecture** uses physically separate memories and separate buses for instructions (I-memory, I-bus) and data (D-memory, D-bus), so an instruction fetch and a data access can take place in the same clock cycle. It is named after the Harvard Mark I (1944), which held instructions on punched paper tape and data in relay counters.

**Mushkil shabd, aasaan matlab:**

### I-memory / I-bus

Sirf instructions ki memory aur uska raasta. CPU yahan se sirf fetch karta hai.

### D-memory / D-bus

Sirf data ki memory aur uska raasta. Load/store yahan hote hain (do-taraf).

### throughput

Ek second mein kitna kaam (yahan: kitne memory transfers) ho sakta hai.

### address space

Addresses ki "duniya". Harvard mein do: I-address 0x10 aur D-address 0x10 alag jagah hain.

### DSP

Digital Signal Processor: audio/video filter, FFT jaisa lagatar data stream ka kaam karne wala chip.

### microcontroller

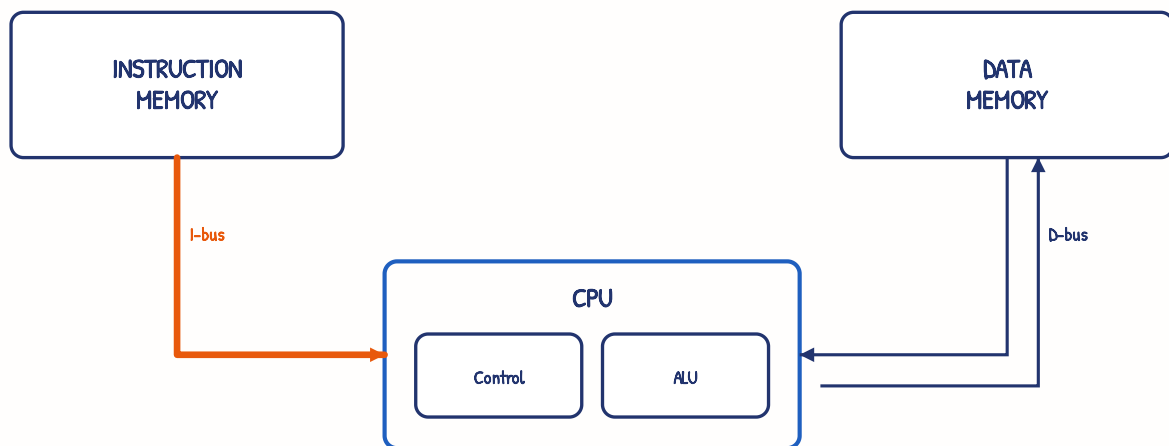
Chhota computer-on-a-chip (CPU + memory + I/O), jaise Arduino ka ATmega328.

### flash

Non-volatile memory: power jaane pe bhi data rehta hai. ATmega328 mein program flash mein, data SRAM mein.

### deterministic timing

Har baar kaam bilkul utne hi time mein: real-time systems (airbag, motor control) ke liye zaruri.



Instruction fetch (I-bus) and data access (D-bus) happen in the **SAME** cycle — no contention.

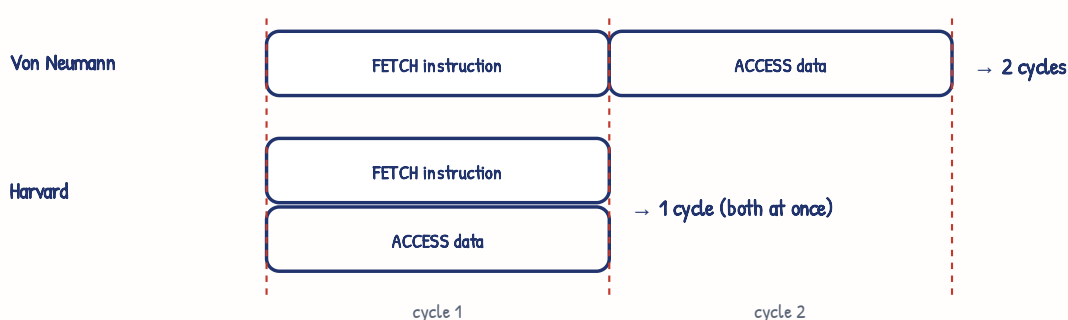
### Harvard block diagram

How to draw: Instruction memory top-left, data memory top-right, CPU in the middle below; a one-way I-bus from instruction memory to CPU and a two-way D-bus between CPU and data memory.

**Diagram kaise padhein:** Upar-left INSTRUCTION MEMORY, upar-right DATA MEMORY, neeche beech mein CPU (Control, ALU). Left ka highlighted arrow = I-bus, sirf instruction memory se CPU ki taraf (ek-taraf, kyunki instructions sirf padhi jaati hain). Right mein do arrows = D-bus, dono taraf (load aur store). Do alag raaste, isliye ek hi cycle mein dono transfer.

### Kaise kaam karta hai:

1. Cycle shuru: CPU I-bus pe PC ka address bhejta hai, instruction memory se instruction aati hai.
2. **Usi cycle mein** pichli 1w instruction D-bus pe data memory se apna data le aati hai.
3. Do alag buses, koi takraav (contention) nahi.
4. Isliye memory throughput Von Neumann se **2 guna tak** (jab har cycle mein fetch aur data dono hon).
5. Kharcha: do memories, do bus systems: zyada wires, pins, silicon. Code aur data alag address spaces mein, to program code ko data ki tarah aasani se nahi padh/likh sakta (kam flexibility).



Same work, half the time: the two transfers ride separate buses simultaneously.

### Timeline: Von Neumann 2 cycles vs Harvard 1 cycle

How to draw: Two rows: VN shows FETCH then ACCESS in consecutive cycles; Harvard shows FETCH and ACCESS stacked in the same cycle.

**Diagram kaise padhein:** Do rows. Upar "Von Neumann": FETCH instruction (cycle 1), phir ACCESS data (cycle 2), total 2 cycles. Neeche "Harvard": FETCH aur ACCESS ek ke upar ek, dono cycle 1 mein, total 1 cycle. Khadi dashed lines cycle

ki seemayein hain. Same kaam, aadha time.

### History aur kahan use hota hai:

1. **Harvard Mark I (1944):** instructions punched paper tape pe, data electro-mechanical relay counters mein. Alag rakhna majboori thi (alag technology).
2. Aaj alag rakhna ek **design choice** hai (on-chip alag memories / caches).
3. **DSPs:** filters/FFT mein har cycle instruction + data dono chahiye.
4. **Real-time systems:** bus takraav nahi, to timing pakki (deterministic).
5. **Microcontrollers: ATmega328 (Arduino Uno) = true Harvard:** code flash mein, data SRAM mein. Embedded / IoT.

| Point               | Von Neumann                              | Harvard                                       |
|---------------------|------------------------------------------|-----------------------------------------------|
| Memory              | one (unified code + data)                | two (separate I and D)                        |
| Buses               | one shared                               | two independent (I-bus, D-bus)                |
| Fetch + data access | sequential (2 cycles)                    | simultaneous (1 cycle)                        |
| Throughput          | limited by the bottleneck                | up to 2× higher                               |
| Cost / complexity   | lower                                    | higher (more wires, pins, silicon)            |
| Flexibility         | high (code = data, one address space)    | lower (separate address spaces)               |
| Origin              | EDVAC report, 1945                       | Harvard Mark I, 1944                          |
| Typical use         | general-purpose computers (memory model) | DSPs, microcontrollers (ATmega328), real-time |

#### Solved example 1 (Harvard bandwidth):

*A 1 GHz Harvard CPU fetches a 32-bit instruction and a 32-bit data word per cycle. Bandwidth needed on EACH bus?*

1. Har bus pe sirf apna hissa: 32 bits per cycle.
2.  $32 \times 10^9 = 32 \text{ Gbit/s}$ .
3.  $32 \div 8 = 4 \text{ GB/s}$ .

**Answer: 4 GB/s per bus (VN would need 8 GB/s on one bus).**

#### Solved example 2 (write the answer like this):

*Draw block diagrams of Von Neumann and Harvard architectures and compare them (six points). Where is each used? (5 marks)*

1. Draw D9.2a (one memory, one shared bus) and D9.4a (instruction memory on the I-bus, data memory on the D-bus).
2. Write six rows of the table: memory, buses, fetch + data timing, throughput, cost, flexibility.
3. Add the timeline D9.4b: VN needs 2 cycles, Harvard 1 cycle for a fetch plus a data access.

4. Uses: Von Neumann for general-purpose computers; Harvard for DSPs and microcontrollers such as the ATmega328; modern CPUs use modified Harvard.

**Answer: two diagrams + six-point table + uses.**

**Galti mat karna:** (1) "Harvard = do CPU" galat: CPU ek, **memories aur buses do**. (2) "Harvard program ko 2 guna tez chalata hai" galat: sirf memory transfer throughput **up to 2×**. (3) "Harvard ka throughput kam hai" galat: zyada hai; uske nuksaan hain zyada pins/wiring, do memory systems, aur code ko data ki tarah treat na kar paana.

**Trick:** Harvard = "H" ke do khambe = **do memories, do buses**. Mark I = paper tape (instructions) + relays (data).

**Ek line mein:** Harvard = alag instruction memory + alag data memory + do buses → fetch aur data ek cycle mein (up to 2× throughput); kharcha: zyada pins, kam flexibility; Mark I 1944; DSP, ATmega328.

## 9.5 Modified Harvard in Modern CPUs

**Pehle socho:** Ek bade restaurant ki rasoi socho. Chef ke **bilkul paas do chhoti trolleys** hain: ek mein recipe cards (instructions), ek mein masale-sabzi (data), taaki chef dono ek saath utha sake (Harvard speed). Par peeche **ek hi bada store room** hai jisme sab kuch rakha hai (ek memory, aasaan management). Trolleys khaali hon to store room se bhar lo. Yahi **modified Harvard** hai: **andar se Harvard (split L1), bahar se Von Neumann (ek DRAM)**.

**Definition:** A **modified Harvard architecture** splits only the level-1 cache into a separate instruction cache (L1-I) and data cache (L1-D), so the core can fetch an instruction and access data in the same cycle, while the L2/L3 caches and the main memory (DRAM) are unified and hold both code and data in one address space. Almost all modern CPUs, such as x86-64 desktops and the ARM Cortex-M4, use it.

**Mushkil shabd, aasaan matlab:**

### L1-I / L1-D

Level-1 Instruction cache aur Level-1 Data cache: core ke sabse paas ki do alag chhoti SRAM memories.

### unified cache / memory

Ek hi memory jisme code aur data dono (L2, L3, DRAM).

### cache coherence

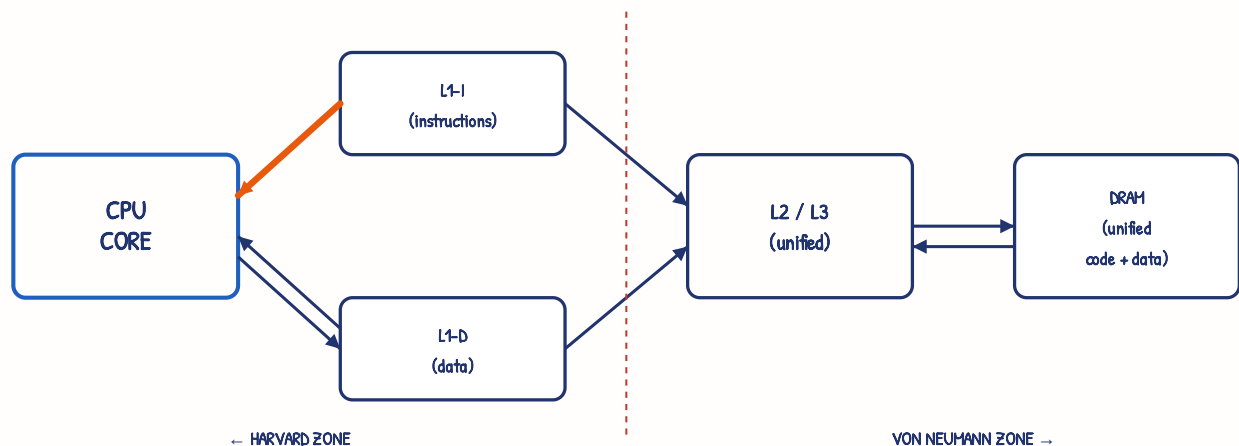
Hardware jo dhyan rakhta hai ki saari copies (L1-I, L1-D, L2, DRAM) ek doosre se match karein, taaki software ko ek hi memory dikhe.

### pipeline (preview)

CPU ek saath kai instructions ke alag-alag stage chalata hai (Unit 14). IF stage = instruction fetch, MEM stage = data access.

### structural hazard (preview)

Do stages ko ek hi hardware chahiye ek hi cycle mein. Ek hi memory ho to IF aur MEM takrate hain; split L1 ye hata deta hai (Unit 15).



Split at L1 for parallel fetch; unified from L2 down for simplicity. Modified Harvard in one picture.

### Split L1-I / L1-D over unified L2/L3 and DRAM

How to draw: CPU core box with two small caches L1-I and L1-D beside it; both feed one unified L2/L3 box, then DRAM; mark the Harvard zone (L1) and the Von Neumann zone (L2 down).

**Diagram kaise padhein:** Left mein CPU CORE. Uske right mein do chhote box: upar L1-I (instructions, highlighted arrow core ki taraf), neeche L1-D (data, do-taraf arrows). Dono ka raasta ek box L2 / L3 (unified) mein milta hai, phir DRAM (unified code + data). Beech ki khadi dashed line: left = "HARVARD ZONE" (split L1), right = "VON NEUMANN ZONE" (L2 se neeche sab ek).

### Kaise kaam karta hai:

1. Program aur data dono **ek hi DRAM** mein (ek address space): OS load karta hai, linker jodta hai, sab aasaan (Von Neumann fayda).
2. Core ke paas L1 do hisson mein: L1-I aur L1-D.
3. Har cycle pipeline L1-I se instruction **aur** L1-D se data ek saath le leta hai: koi contention nahi (Harvard fayda).
4. L1 mein miss hua to unified L2/L3, phir DRAM se block aata hai.
5. Cache coherence hardware ensure karta hai ki copies match karein, to programmer ko ek hi memory dikhti hai.
6. Yahi split L1 5-stage pipeline mein IF aur MEM ka structural hazard hatata hai (Unit 15).

| Point        | Von Neumann | Harvard             | Modified Harvard     |
|--------------|-------------|---------------------|----------------------|
| Where split? | nowhere     | whole memory system | only L1 (L1-I, L1-D) |
| Main memory  | one         | two                 | one (unified DRAM)   |

|                                 |                                               |                                   |                                                 |
|---------------------------------|-----------------------------------------------|-----------------------------------|-------------------------------------------------|
| Fetch + data same cycle?        | no                                            | yes                               | yes (on L1 hits)                                |
| Address spaces seen by software | one                                           | two                               | one                                             |
| Example (class activity)        | EDVAC-style designs                           | ATmega328 (Arduino Uno)           | ARM Cortex-M4, x86-64 desktop                   |
| Used today in                   | the memory model of general-purpose computers | DSPs, microcontrollers, real-time | almost all modern CPUs (desktop, laptop, phone) |

### Solved example 1 (class activity):

*Classify: ATmega328 (Arduino Uno), ARM Cortex-M4, x86-64 desktop.*

1. ATmega328: program flash mein, data SRAM mein, alag buses → **true Harvard**.
2. ARM Cortex-M4: alag instruction aur data paths, par ek unified address space → **modified Harvard**.
3. x86-64 desktop: split L1-I/L1-D, unified L2/L3 aur DRAM → **modified Harvard** (pure Von Neumann NAHI).

**Answer: true Harvard; modified Harvard; modified Harvard.**

### Solved example 2 (write the answer like this):

*Explain modified Harvard architecture with a diagram. Why do almost all modern CPUs use it? (4 marks)*

1. Draw D9.5a: core → split L1-I and L1-D → unified L2/L3 → unified DRAM; mark the Harvard zone and the Von Neumann zone.
2. The split L1 lets the pipeline fetch an instruction and access data in the same cycle, avoiding the Von Neumann bottleneck where it matters most.
3. The unified main memory gives one address space, so loading, linking and programming stay simple and flexible.
4. Hardware cache coherence keeps the split caches consistent, so software sees a single memory; the design scales to real workloads.

**Answer: diagram + "Harvard speed inside, Von Neumann flexibility outside" + coherence.**

**Galti mat karna:** (1) Modern x86 ko "pure Von Neumann" kehna: uska L1 split hai, to modified Harvard. (2) "Modern CPU ke programs ko do address spaces dikhte hain" galat: software ko **ek** memory dikhti hai. (3) Split ko DRAM ya register file level pe batana: split sirf **L1** pe hai.

**Trick: "Andar Harvard, bahar Von Neumann" ya "L1 split, baaki sab ek".**

**Ek line mein:** Modified Harvard = split L1-I/L1-D (fetch + data ek cycle mein) + unified L2/L3/DRAM (ek address space), coherence hardware se consistent; x86-64 aur Cortex-M4 isi pe, ATmega328 true Harvard.

### Exam mein is unit se kya aata hai → kaise solve karen

| Question type                                | Method (steps)                                                                                                                                                                                                                                 |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Basic computer block diagram                 | CPU, memory, I/O on one bus (D9.1a); one line per role; bus = address + data + control.                                                                                                                                                        |
| Stored-program concept / Von Neumann diagram | 1. Code stored as numbers with data in one memory (D9.2b). 2. D9.2a: CPU (CU, ALU, registers), one memory, one bus, I/O. 3. Four components: control unit, ALU, memory, I/O. 4. EDVAC 1945. 5. Simplicity + generality.                        |
| MCQ: defining feature of VN / Harvard        | VN: one memory + one shared bus. Harvard: separate I and D memories + two buses, fetch and data in the same cycle.                                                                                                                             |
| Who / when (one-word)                        | Bottleneck term: John Backus, 1977. Stored program: von Neumann, EDVAC report 1945. Harvard Mark I: 1944, punched paper tape + relays.                                                                                                         |
| Explain the bottleneck                       | One bus shared → fetch OR data per cycle (D9.3a); four effects: contention, IPC cap, idle ALU, worsens with CPU speed; fixes: caches, wider buses, Harvard split.                                                                              |
| Bandwidth numerical                          | 1. Add bits per cycle (VN: instr + data; Harvard: each bus separately). 2. × clock (GHz = $10^9$ ). 3. ÷ 8 for bytes. 1 GHz $32+32 \rightarrow 8$ GB/s (Harvard $4+4$ ); 2 GHz $64+64 \rightarrow 32$ GB/s; 2 GHz $32+32 \rightarrow 16$ GB/s. |
| Cycle count VN vs Harvard                    | VN cycles = fetches + data accesses; Harvard cycles = fetches (data overlaps). Ratio is at most 2.                                                                                                                                             |
| VN vs Harvard comparison (six points)        | Memory, buses, fetch + data timing (2 vs 1 cycle, D9.4b), throughput, cost, flexibility, use. Draw both diagrams.                                                                                                                              |
| Harvard disadvantages (MSQ)                  | More pins/wiring, two memory systems, code cannot easily be treated as data. NOT lower throughput.                                                                                                                                             |
| Modified Harvard + why modern CPUs use it    | D9.5a; split L1 for parallel fetch + data; unified L2/L3 + DRAM for one address space; hardware cache coherence; split is at L1 only.                                                                                                          |
| Classify a processor                         | ATmega328 → true Harvard; ARM Cortex-M4 → modified Harvard; x86-64 desktop → modified Harvard.                                                                                                                                                 |

## Unit 10 · CISC vs RISC

**Is unit ki kahani:** Unit 9 mein humne dekha CPU memory se *kaise* juda hai (Von Neumann, Harvard). Ab sawaal hai: CPU *kaunsi* bhasha samajhta hai? Us bhasha ko **ISA** kehte hain. ISA design karne ke do tareeke hain: har instruction mein bahut kaam bhar do (**CISC**, jaise x86) ya har instruction ko chhota aur simple rakho (**RISC**, jaise MIPS, ARM). Is unit mein dono ka matlab, fayda, nuksaan aur history samjhenge. Unit 11 mein ek RISC (MIPS) ki bhasha bit-by-bit seekhenge, aur Unit 14 mein dekhenge ki RISC pipelining ke liye kyun perfect hai.

### 10.1 Recap (Von Neumann / Harvard) aur ISA ka matlab

**Pehle socho:** Tumhare paas ek Android app hai. Wahi app Samsung ke phone pe bhi chalti hai aur Xiaomi pe bhi, jabki dono ke andar chip alag company ne banayi hai. Kaise? Kyunki dono chips **ek hi "bhasha" (ARM ISA) samajhti hain**. Jaise do alag restaurants ek hi menu card follow karein: customer (software) menu se order deta hai, kitchen (hardware) andar kaise banata hai, customer ko farak nahi padta. Ye "menu card" hi **ISA** hai.

**Definition:** The **Instruction Set Architecture (ISA)** is the part of a processor visible to the programmer: its instructions, registers, data types, addressing modes and memory model. It is the **contract between software and hardware**: compilers translate programs *to* the ISA, and the hardware (microarchitecture) *executes* the ISA.

**Mushkil shabd, aasaan matlab:**

#### Von Neumann (recap)

Ek hi memory aur ek hi bus, jisme instructions aur data dono rehte hain. Fetch aur data access baari-baari hote hain (bottleneck).

#### Harvard (recap)

Instruction memory alag, data memory alag, bus bhi alag. Ek hi cycle mein instruction fetch aur data read dono ho sakte hain.

#### Modified Harvard (recap)

Aaj ke CPU: L1 cache do hisson mein (L1-I instructions ke liye, L1-D data ke liye), neeche L2/L3 aur RAM ek hi (unified).

#### ISA

CPU ki "bhasha": kaunse instructions hain (add, lw, beq), kitne registers hain (\$0 se \$31), memory kaise address hoti hai. Example: x86, ARM, MIPS, RISC-V.

#### Microarchitecture (organisation)

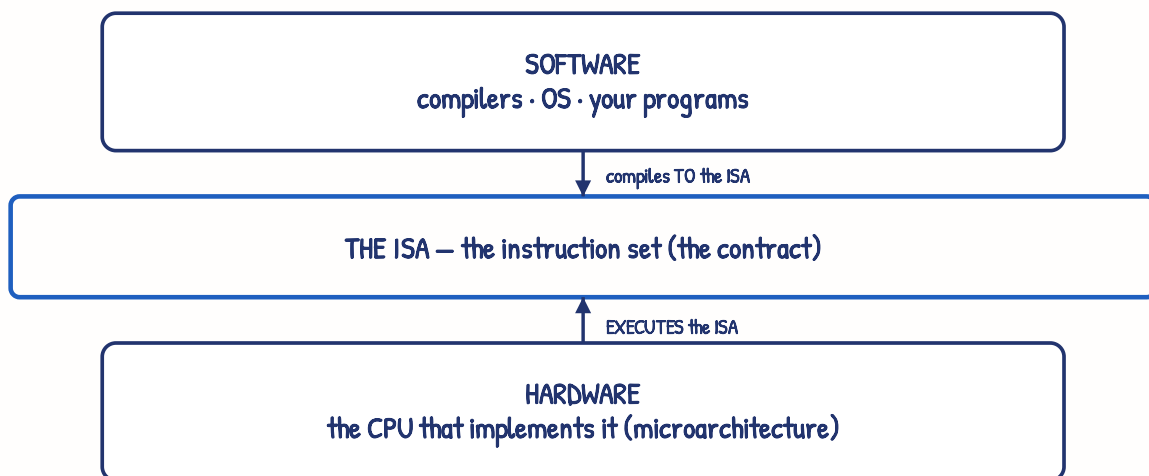
ISA ko andar se *kaise* banaya: kitne pipeline stages, cache kitna bada. Intel i3 aur i9 ka ISA same (x86-64), microarchitecture alag.

### Memory operand

Instruction ka input/output seedha memory address ho, register nahi. Jaise `add [x], eax, x` memory mein hai.

### Kaise kaam karta hai (recap se ISA tak ka pul):

1. Unit 9 ne bataya: CPU aur memory ke beech bus hai, aur har memory access mein time lagta hai.
2. Ab har instruction kitni baar memory chhoota hai, ye ISA decide karta hai.
3. CISC instruction `add [mem1], [mem2]`: 1 baar instruction fetch + 2 baar data read + 1 baar result write = ek hi instruction mein bus pe 4 trips.
4. RISC ka `add $t0, $t1, $t2`: sirf instruction fetch, data registers mein hai, bus pe koi data trip nahi. Memory sirf `lw/sw` chhoota hain, aur woh bhi ek baar.
5. Isliye recap pehle aata hai: ISA ka design seedha memory traffic decide karta hai.
6. ISA ek contract hai: upar software (compiler, OS, apps) ISA mein compile hota hai, neeche hardware ISA ko execute karta hai. Dono contract maante hain to Intel naya chip banaye, purana program phir bhi chalega.



Both sides honour the ISA → hardware and software evolve independently (old programs run on new chips).

### The ISA as the hardware/software contract

How to draw: Three stacked bars: software on top, the ISA in the middle (highlighted), hardware at the bottom; arrows "compiles to" and "executes".

**Diagram kaise padhein:** Teen patti upar se neeche: Software, beech mein highlighted ISA, neeche Hardware. Upar wala arrow "compiles TO the ISA" (tumhara C code ISA ke instructions ban jata hai), neeche wala "EXECUTES the ISA" (chip un instructions ko chalata hai). Trace:  $c = a + b$  → compiler add instruction banata hai (ISA level) → chip ka ALU usse jodta hai (hardware level).

### Solved example (write the answer like this):

*Differentiate between architecture (ISA) and organisation. Why is the ISA called a contract? (3 marks)*

1. Architecture (ISA) = the "what": instructions, registers, addressing modes visible to the programmer.
2. Organisation (microarchitecture) = the "how": pipelines, caches, buses that implement the ISA.
3. Example: Intel Core i3 and i9 share the x86-64 ISA but have different organisation.
4. Contract: software is compiled only against the ISA; any hardware that honours the ISA runs it. So hardware and software can be improved independently (old programs run on new chips).

**Answer:** ISA = what the CPU understands; organisation = how it is built. Same ISA, different chips, same software.

**Galti mat karna:** "Architecture" aur "organisation" ko same mat likhna. Cache size, pipeline depth = organisation. Instruction list, register count = architecture (ISA).

**Ek line mein:** ISA = CPU ki bhasha aur software-hardware ka contract; Unit 9 batata hai CPU memory tak kaise pahunchta hai, Unit 10 batata hai CPU kya samajhta hai.

## 10.2 RISC vs CISC: definitions aur design goals

**Pehle socho:** Do tarah ke cook socho. **Cook A (CISC)** ko tum ek hi bada order dete ho: "Paneer butter masala bana do". Ek command, par cook ko andar se 20 kaam khud karne padte hain, aur aisi saikdon recipes yaad rakhni padti hain. **Cook B (RISC)** sirf chhote simple steps samajhta hai: "fridge se paneer nikalo", "kaato", "kadhai mein daalo", "milao". Tumhe (compiler ko) zyada lines likhni padti hain, par har step itna simple hai ki cook B bahut tez aur bina confusion ke kaam karta hai, aur ek step khatam hone se pehle agla shuru kar sakta hai (pipelining). **CISC = ek complex command; RISC = bahut saare simple steps.**

**Definition:** A CISC (Complex Instruction Set Computer) provides a large number of powerful, **variable-length** instructions, many of which can operate **directly on memory**, often implemented with microcode (e.g. x86, VAX, 68000). A RISC (Reduced Instruction Set Computer) provides a small set of simple, **fixed-length (32-bit)** instructions, each doing one simple operation, in a **load/store architecture** where only load and store instructions access memory (e.g. MIPS, ARM, RISC-V, SPARC).

**Mushkil shabd, aasaan matlab:**

### Variable length

Instructions alag-alag size ke. x86 mein ek instruction 1 byte ka bhi ho sakta hai aur 15 bytes ka bhi.

### Fixed length

Har instruction same size. MIPS mein har instruction 32 bits = 4 bytes. Isliye agla instruction hamesha PC + 4 pe.

### Load/store architecture

Sirf do kaam memory chhoo sakte hain: **load** (memory → register, lw) aur **store** (register → memory, sw). add, sub sirf registers pe kaam karte hain.

### Decoder

CPU ka woh hissa jo instruction ke bits padh ke samajhta hai "ye kaunsa instruction hai, kaunse registers". (Unit 3 wala decoder ka bada roop.)

### Microcode

CPU ke andar chhupa ek mini-program. Ek complex CISC instruction aane pe CPU ROM se uska chhota program chalata hai jo usse kai chhote steps mein karta hai.

### Pipelining

Assembly line: ek instruction decode ho raha hai tab tak agla fetch ho raha hai. (Unit 14.)

### Code density

Kitne bytes mein kitna kaam. Kam bytes mein zyada kaam = high density (CISC ka fayda).

**Kaise kaam karta hai (same task, dono styles):** Task:  $C = A + B$ , teeno variables memory mein hain.

1. **CISC soch:** ek instruction memory se operand utha sakta hai. VAX mein ek hi instruction ADDL3 A, B, C (memory A + memory B → memory C) poora kaam kar deta tha. x86 thoda kam flexible hai (ek instruction mein max ek memory operand), to wahan `mov eax, [A]; add eax, [B]; mov [C], eax`.
2. **RISC soch:** pehle data ko register mein lao, register pe kaam karo, phir wapas memory mein rakho.
3. `lw $t1, A`: A ki value memory se \$t1 mein.
4. `lw $t2, B`: B ki value \$t2 mein.
5. `add $t3, $t1, $t2`: registers pe jod (memory ko chhua bhi nahi).
6. `sw $t3, C`: result memory ke C mein rakho.
7. **Result:** CISC ne kam instructions likhe, RISC ne 4 likhe, par RISC ke chaaron instructions simple, fixed 32-bit, aur ek-ek cycle wale hain.

; Class activity:  $C = A + B$  ;  $D = C * 2$

; CISC style (x86-like): 5 instructions, memory operands allowed

```
mov ax, [A]      ; ax = A (memory se seedha)
add ax, [B]      ; ax = A + B (B memory operand hai)
mov [C], ax      ; C = A + B, memory mein likh diya
shl ax, 1        ; ax = C * 2 (left shift 1 = 2 se guna)
mov [D], ax      ; D = C * 2
```

# RISC style (MIPS-like): 6 instructions, sirf lw/sw memory chhoote hain

```
lw r1, A        # r1 = A (load: memory se register)
lw r2, B        # r2 = B
add r3, r1, r2   # r3 = A + B (sirf registers)
sw r3, C        # C = r3 (store: register se memory)
add r4, r3, r3   # r4 = C + C = C * 2
sw r4, D        # D = r4
```

RISC – FIXED 32-bit: every instruction the same width



next instruction always starts 4 bytes later → trivial to fetch and decode in parallel

CISC (x86) – VARIABLE length: 1 to 15 bytes



the CPU must decode one instruction before it knows where the next begins → harder to pipeline

### Fixed (RISC) vs variable (CISC) instruction encoding

How to draw: Top row: three equal-width 32-bit boxes with fields; bottom row: boxes of different lengths (prefix, op, modRM, disp, imm) with boundaries that must be found by decoding.

**Diagram kaise padhein:** Upar wali line RISC: teen barabar chaudi ke dabbe, har ek 32 bits, fields (opcode, rs, rt, rd, shamt, funct) hamesha same jagah. CPU ko pata hai agle instruction 4 bytes baad shuru hoga, to woh bina padhe agle fetch kar sakta hai. Neeche wali line CISC: dabbe alag lambai ke (prefix, op, modRM, disp, imm), aur dabbe ki boundary tab tak pata nahi jab tak pichla instruction decode na ho jaye. Isliye CISC ko pipeline karna mushkil hai.

| Point                  | CISC                               | RISC              | WHY (kyun)                                                                                                               |
|------------------------|------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------|
| Number of instructions | hundreds, many complex             | few, simple       | CISC: har common kaam ke liye hardware mein ready-made instruction. RISC: sirf woh jo compiler sach mein use karta hai.  |
| Instruction length     | variable (x86: 1 to 15 bytes)      | fixed 32 bits     | CISC: chhota instruction kam byte le, memory bache. RISC: fixed size → agle instruction hamesha PC + 4, fetch aasaan.    |
| Memory access          | arithmetic can use memory operands | load/store only   | CISC: programmer ka kaam kam. RISC: memory sirf 2 instructions mein, baaki sab registers pe → predictable timing.        |
| Decoder / control      | complex, often microcoded          | simple, hardwired | Variable format padhne ke liye bada decoder chahiye. Fixed format ke fields hamesha same bit pe, to decode lagbhag free. |

|                          |                                           |                          |                                                                                                       |
|--------------------------|-------------------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------|
| Cycles per instruction   | many, varies per instruction              | about 1 (ideally)        | Ek CISC instruction andar kai kaam karta hai; RISC instruction ek hi simple kaam.                     |
| Instructions per program | fewer                                     | more                     | Ek CISC instruction = kai RISC instructions ka kaam.                                                  |
| Code density             | high (small programs)                     | lower                    | Kam aur chhote instructions → kam bytes.                                                              |
| Pipelining               | hard                                      | easy                     | Fixed length, same format, same time per instruction → assembly line smooth.                          |
| Registers                | fewer general-purpose (x86: 8 originally) | many (32 in MIPS)        | Load/store mein data registers mein rakhna padta hai, isliye RISC ko zyada registers chahiye.         |
| Complexity lives in      | hardware                                  | compiler                 | Kaam kahin to hoga: CISC mein chip karti hai, RISC mein compiler instructions ki sequence banata hai. |
| Examples                 | x86, VAX, Motorola 68000                  | MIPS, ARM, RISC-V, SPARC | Class slide ki list.                                                                                  |

### Solved example 1 (direct):

*Name the two hallmarks of a RISC ISA. (class exit ticket)*

1. RISC ki pehchaan wo cheez hai jo CISC mein nahi hoti.
2. Pehli: har instruction same size (fixed 32-bit).
3. Doosri: memory sirf load/store se (load/store architecture).

**Answer: (1) fixed-length instructions, (2) load/store-only memory access.**

### Solved example 2 (exam favourite, 5 marks):

*Compare CISC and RISC (six points) with examples, and write  $C = A + B$ ;  $D = 2C$  in both styles.*

1. Definition dono ki ek-ek line mein (upar wali definition).
2. Upar wali table ki pehli 6 rows likho: number of instructions, length, memory access, decoder, CPI, pipelining (har row ke saath "why" ek chhoti line).
3. Examples: CISC = x86, VAX, 68000; RISC = MIPS, ARM, RISC-V, SPARC.
4. Upar wala code likho: CISC 5 instructions (mov, add, mov, shl, mov), RISC 6 (lw, lw, add, sw, add, sw).
5. Conclusion line: RISC zyada instructions chalata hai, par har ek simple aur pipeline-friendly hai; complexity hardware se compiler mein shift hoti hai.

**Answer: 6-point table + code (CISC 5 instructions, RISC 6) + "complexity moves from hardware to compiler".**

**Galti mat karna:** "RISC = kam instructions chalte hain" GALAT hai. RISC mein instruction **types** kam hain, par ek program chalane mein instructions **zyada** lagte hain (upar: 6 vs 5). "Reduced" ka matlab har instruction ka kaam reduced (simple) hai.

**Trick:** RISC ke do hallmarks = "**Fixed + Load/Store**", jaise railway: fixed time-table (har instruction 4 bytes) aur chadhna-utarna sirf platform pe (memory sirf lw/sw se).

**Ek line mein:** CISC = kam lekin bhaari, variable-length, memory-operand instructions (complexity hardware mein); RISC = zyada lekin halke, fixed 32-bit, load/store instructions (complexity compiler mein).

### 10.3 Historical context: VAX, x86, MIPS, ARM, RISC-V

**Pehle socho:** 1970s mein memory bahut mehengi thi (kuch KB ke liye lakhon rupaye) aur programmer zyada tar assembly *haath se* likhte the. Aise mein sabse samajhdari kya thi? Program ko chhota rakho aur programmer ka kaam aasaan karo: "ek instruction mein zyada kaam". Isliye pehle **CISC** aaya. 1980s tak memory sasti ho gayi, compilers smart ho gaye, aur researchers ne dekha ki compiler complex instructions use hi nahi karta. Tab **RISC** aaya: simple instructions, tez chip, pipelining.

**Definition:** The **ISA family tree** shows two lineages: the **CISC** line (VAX 1977, Motorola 68000 1979, Intel x86 1978 → x86-64 2003) and the **RISC** line (IBM 801 → MIPS (Stanford, 1984), SPARC, ARM (1985) → RISC-V (Berkeley, 2010)). CISC came first because memory was scarce and expensive; RISC followed when better compilers and the need for pipelining made simple instructions more attractive.

**Mushkil shabd, aasaan matlab:**

#### Lineage

Vansh / khaandan. Kaunsa ISA kis se prerit hua.

#### Backward compatibility

Naya chip purane programs bina badle chala sake. x86 ka sabse bada hathiyar: 1978 ka code aaj bhi chal sakta hai.

#### Licensing (ARM model)

ARM khud chip nahi bechta, design ka licence bechta hai. Qualcomm, Apple, Samsung licence leke apni chip banate hain.

#### Royalty-free

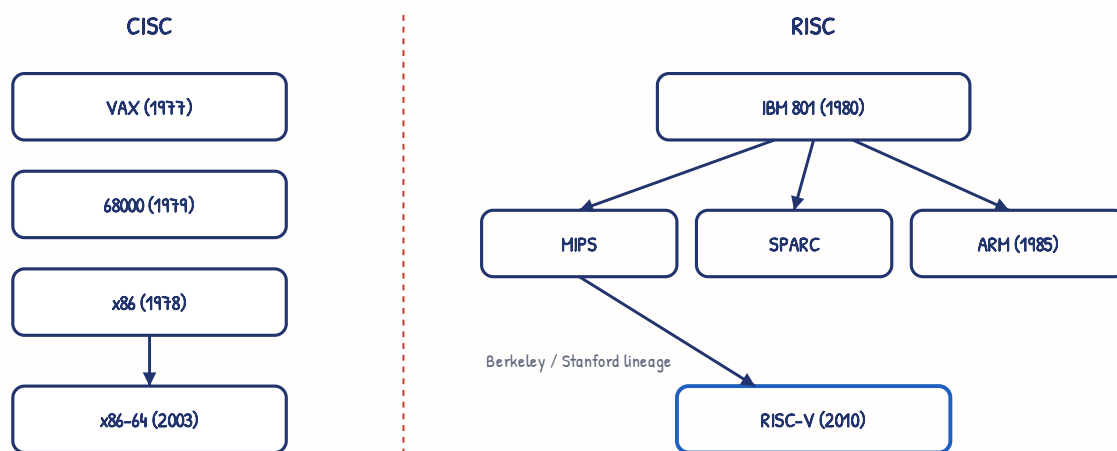
Use karne ke liye koi fees nahi. RISC-V aisa hai.

#### Compiler

Program jo C jaisi bhasha ko ISA instructions mein badalta hai.

**Kaise hua (time-line, step by step):**

1. **1970s, CISC kyun pehle:** memory mehengi aur chhoti → program kam bytes mein hona chahiye → ek instruction mein zyada kaam (high code density). Programmer assembly haath se likhte the → powerful instructions unka kaam aasaan karte the. Compilers kamzor the.
2. **CISC family:** VAX (DEC, 1977), x86 (Intel 8086, 1978), Motorola 68000 (1979). Baad mein AMD ne x86 ko 64-bit kiya: x86-64 (2003).
3. **1980, RISC kyun aaya:** IBM 801 project ne dekha ki compiler zyada tar simple instructions hi use karta hai. Memory sasti ho rahi thi, compilers achhe ho rahe the. Simple instructions → chhota decoder → tez clock aur **pipelining** aasaan.
4. **RISC family:** IBM 801 → MIPS (Stanford, John Hennessy, 1984), SPARC (Sun, Berkeley RISC se), ARM (1985), aur RISC-V (Berkeley, 2010).
5. **MIPS kahan chala (Lab 11):** SGI workstations, original PlayStation (R3000A, 33.8688 MHz), Nintendo 64 (VR4300, 93.75 MHz), PlayStation 2 (R5900). Hennessy aur Patterson ko RISC ke liye 2017 Turing Award mila. 2021 mein MIPS company RISC-V pe shift ho gayi.
6. **1990s se, do samrajya:** ARM ne mobile jeeta (simple design = kam power = lambi battery, aur licence model se sab ARM chip bana sakte hain; ab Apple laptops bhi). x86 ne desktop/server rakha (backward compatibility, dashakon ka software, Intel/AMD ka R&D).
7. **Lesson:** "best ISA" jeetta nahi; ecosystem, power budget aur compatibility jeetate hain.



Two lineages defined 40 years of chips — and have since converged (x86 runs RISC-like  $\mu$ ops inside).

### ISA family tree

How to draw: Two columns: CISC (VAX, 68000, x86 → x86-64) and RISC (IBM 801 → MIPS, SPARC, ARM, then RISC-V); a dashed line between the camps.

**Diagram kaise padhein:** Beech mein dashed line dono camps ko alag karti hai. Left (CISC): VAX, 68000, x86, aur x86 se arrow neeche x86-64 (2003). Right (RISC): upar IBM 801, usse teen arrows MIPS, SPARC, ARM ki taraf; MIPS (Berkeley/Stanford lineage) se arrow RISC-V (2010) tak. Neeche ki line: dono family ab mil gayi hain (x86 andar RISC jaise  $\mu$ ops chalata hai, 10.5 dekho).

**Solved example (write the answer like this, 4 marks):**

Trace the history of CISC and RISC. Why did CISC come first, and why did ARM win mobile while x86 kept the desktop?

1. CISC first (1970s): memory was expensive and small, programs were hand-written in assembly and compilers were weak, so dense, powerful instructions (VAX 1977, x86 1978, 68000 1979) saved memory and programmer effort.
2. RISC (1980s): IBM 801 showed compilers mainly use simple instructions; cheaper memory and better compilers allowed simple fixed-length instructions that pipeline well (MIPS 1984, SPARC, ARM 1985, RISC-V 2010).
3. ARM won mobile: simple core → low power → long battery life; licensable design.
4. x86 kept desktop: backward compatibility with decades of software, huge ecosystem and R&D.
5. Draw the family tree (D10.3a).

**Answer:** CISC first because memory was costly; RISC when compilers + pipelining mattered; ARM = power + licensing, x86 = compatibility.

**Galti mat karna:** RISC-V 1980s ka nahi, **2010** ka hai. ARM RISC hai, CISC nahi. x86 1978 (8086), x86-64 2003 (AMD).

**Trick:** Saal yaad rakho: "77 VAX, 78 x86, 84 MIPS, 85 ARM, 2003 x86-64, 2010 RISC-V". CISC pehle kyun? "Memory mehengi thi" (bas yahi ek reason exam mein likhna kaafi hai, saath mein compilers kamzor).

**Ek line mein:** Mehengi memory ke zamane mein CISC aaya (dense code), sasti memory + smart compilers + pipelining ke zamane mein RISC aaya; ARM ne power se mobile jeeta, x86 ne compatibility se desktop rakha.

## 10.4 Compiler implications aur code density

**Pehle socho:** Ek SMS mein 160 characters ki limit hai. Agar tum "tomorrow" ki jagah "tmrw" likho, to kam jagah mein zyada baat aa jaati hai, par padhne wale ko thoda dimaag lagana padta hai. CISC "tmrw" jaisa hai: kam bytes mein zyada kaam (dense code), par padhna (decode) mushkil. RISC poore spelling jaisa: zyada jagah, par padhna turant. Ye trade-off kabhi khatam nahi hota, sirf jagah badalta hai.

**Definition:** Code density is the amount of work a program does per byte of instruction memory. CISC has higher code density (smaller programs, less instruction-cache pressure), while RISC's fixed-length, uniform instructions are easier to decode and pipeline. Performance is given by the **iron law**: CPU time = Instruction count × CPI × Clock period.

**Mushkil shabd, aasaan matlab:**

**I-cache (instruction cache)**

CPU ke paas chhoti fast memory jisme instructions rakhe jaate hain. Program chhota ho to zyada instructions isme fit hote hain.

### Instruction count (IC)

Program chalne mein kul kitne instructions execute hue. Jaise 2 million.

### CPI (cycles per instruction)

Ek instruction ko average kitne clock cycles lage.  $CPI = 1.5$  matlab average 1.5 cycle.

### Clock period (T)

Ek cycle ka time =  $1 / \text{frequency}$ .  $1 \text{ GHz} \rightarrow 1 \text{ ns}$ ;  $2 \text{ GHz} \rightarrow 0.5 \text{ ns}$ .

### ARM Thumb / RISC-V "C"

RISC ke 16-bit chhote instructions (compressed). Inse RISC ki code density CISC ke paas aa jaati hai.

### Register allocation

Compiler decide karta hai kaunsa variable kaunse register mein rahe. RISC mein bahut zaroori kyunki maths sirf registers pe hota hai.

### Kaise kaam karta hai:

1. **CISC ka fayda (density):** kam aur chhote instructions  $\rightarrow$  program kam bytes  $\rightarrow$  fetch kam bytes  $\rightarrow$  I-cache mein zyada fit. Memory kam ho (embedded) to bada fayda.
2. **RISC ka nuksaan:** har instruction 4 bytes, chahe kaam chhota ho. Fix: ARM Thumb aur RISC-V C (16-bit instructions).
3. **RISC ka fayda (pipelining), 4 reasons:** (a) fixed length = fetch aasaan (agla hamesha PC + 4); (b) uniform format = decode ek stage mein; (c) har instruction ek simple step = flow smooth; (d) load/store = sirf 2 instructions memory chhoote, hazards pakadna aasaan.
4. **Compiler pe asar:** RISC compiler ko registers achhe se baantne padte hain aur zyada simple instructions ko sahi order mein lagana padta hai (load ka wait chhupane ke liye reorder bhi kar sakta hai). CISC compiler rich instructions use kar sakta hai, par asal mein instruction set ka bada hissa use hi nahi hota.
5. **Iron law se faisla:** CISC IC (instruction count) kam karta hai; RISC CPI aur clock period kam karta hai. Teeno ka product hi asli time hai.

**Formula: CPU time =  $IC \times CPI \times T = IC \times CPI / f$**

IC = instruction count, CPI = average cycles per instruction, T = clock period =  $1/f$  (f = clock frequency).  $1 \text{ GHz} \rightarrow T = 1 \text{ ns}$ ;  $2 \text{ GHz} \rightarrow T = 0.5 \text{ ns}$ .

| CISC strengths                   | RISC strengths               |
|----------------------------------|------------------------------|
| denser code                      | simple, fast, cheap decoder  |
| fewer instructions per task      | easy, efficient pipelining   |
| less instruction-fetch bandwidth | higher clock speeds possible |
| rich, expressive operations      | simpler to design and verify |

### Solved example 1 (easy):

A program runs 2 million instructions with CPI 1.5 on a 1 GHz clock. Find the CPU time in ms.

1. IC = 2 million =  $2 \times 10^6$ .
2.  $T = 1 / 1 \text{ GHz} = 1 \text{ ns} = 1 \times 10^{-9} \text{ s}$ .
3. CPU time =  $2 \times 10^6 \times 1.5 \times 1 \text{ ns} = 3 \times 10^6 \text{ ns}$ .
4.  $3 \times 10^6 \text{ ns} = 3 \times 10^6 \times 10^{-9} \text{ s} = 3 \times 10^{-3} \text{ s} = 3 \text{ ms}$ .

**Answer: 3 ms.**

### Solved example 2 (exam-level comparison):

A program on a CISC machine executes 1.0 million instructions at CPI 2.0. Compiled for a RISC machine it executes 1.4 million instructions at CPI 1.1. Both run at 2 GHz. Which is faster?

1.  $T = 1 / 2 \text{ GHz} = 0.5 \text{ ns}$ .
2. CISC:  $1.0 \times 10^6 \times 2.0 = 2.0 \times 10^6 \text{ cycles}$ .  $\times 0.5 \text{ ns} = 1.0 \times 10^6 \text{ ns} = 1.0 \text{ ms}$ .
3. RISC:  $1.4 \times 10^6 \times 1.1 = 1.54 \times 10^6 \text{ cycles}$ .  $\times 0.5 \text{ ns} = 0.77 \times 10^6 \text{ ns} = 0.77 \text{ ms}$ .
4. Compare:  $0.77 \text{ ms} < 1.0 \text{ ms}$ , to RISC tez hai.
5. Speed-up =  $1.0 / 0.77 \approx 1.30$ .

**Answer: RISC is faster (0.77 ms vs 1.0 ms, about 1.3×) even though it runs 40% more instructions.**

### Solved example 3 (class homework):

Count the instructions for a straight-line 3-element vector add  $C[i] = A[i] + B[i]$  in CISC and RISC style.

1. CISC (x86) har element: `mov eax, [A+4i]; add eax, [B+4i]; mov [C+4i], eax` = 3 instructions.
2. 3 elements  $\times 3 = 9$  instructions.
3. RISC har element: `lw; lw; add; sw` = 4 instructions.
4. 3 elements  $\times 4 = 12$  instructions.

**Answer: CISC 9, RISC 12 (RISC has more instructions, but each is simple and pipelines at about one per cycle).**

**Galti mat karna:** (1) "RISC hamesha tez hai" galat; time  $IC \times CPI \times T$  pe depend karta hai. (2) GHz ko ns mein badalte waqt:  $2 \text{ GHz} \rightarrow 0.5 \text{ ns}$ ,  $2 \text{ ns}$  nahi. (3) Answer ms mein maanga ho to ns ko  $10^6$  se divide karo.

**Trick: "CISC count ghatata hai, RISC cycle ghatata hai."** Pipelining ke 4 reasons: "**F-U-O-L**" = Fixed length, Uniform decode, One cycle, Load/store.

**Ek line mein:** CISC ka hathihaar code density, RISC ka hathihaar pipelining; asli speed =  $IC \times CPI \times T$ .

## 10.5 Modern reality: x86 ke andar micro-ops

**Pehle socho:** Ek restaurant ka waiter (front) customer se bada order leta hai: "ek thali". Kitchen (back) mein woh order chhote kaamon mein toota jaata hai: "roti banao", "dal garam karo", "plate lagao", aur kai cooks ek saath assembly line pe kaam karte hain. Customer ko lagta hai ek order diya, par andar simple kaam chal rahe hain. x86 bahar se CISC hai (purane software ke liye), andar se RISC jaisa.

**Definition: Micro-operations ( $\mu$ ops)** are simple, RISC-like internal operations into which the front-end decoder of a modern x86 processor splits each complex CISC instruction. The back end then pipelines, renames and reorders these  $\mu$ ops like a RISC machine.  $\mu$ ops are internal and not visible to the programmer.

**Mushkil shabd, aasaan matlab:**

### $\mu$ op (micro-op)

" $\mu$ " = micro = chhota. Ek chhota simple kaam: sirf load, ya sirf add, ya sirf store.

### Front end

CPU ka woh hissa jo instructions fetch aur decode karta hai (yahan CISC  $\rightarrow$   $\mu$ ops todna hota hai).

### Back end (core)

Woh hissa jo asli kaam (ALU, memory) karta hai; ye  $\mu$ ops ko RISC ki tarah chalata hai.

### Out-of-order execution

Jo  $\mu$ op ready hai woh pehle chal jaye, chahe program mein baad mein likha ho (result phir bhi sahi order mein).

### Register renaming

Andar chhupe extra registers use karke false clashes hatana. (Exam mein sirf naam kaafi.)

### tmp

CPU ka andar ka temporary register jo programmer ko nahi dikhta.

**Kaise kaam karta hai (worked example: add [mem], eax):**

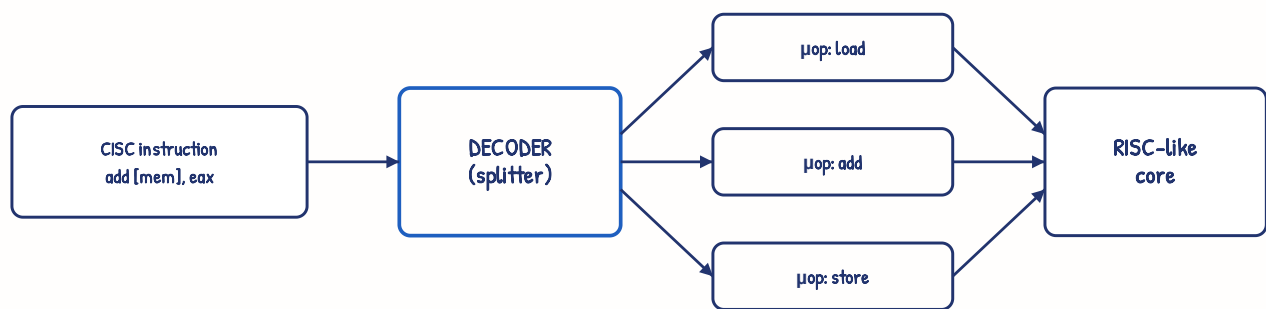
1. Instruction ka matlab: "memory address mem pe jo number hai, usme eax jodo, aur result wapas usi memory address pe likho".
2. Isme teen alag kaam chhupe hain: memory se padhna, jodna, memory mein likhna.
3.  **$\mu$ op 1 (load):**  $tmp \leftarrow Mem[mem]$ . Maan lo  $Mem[mem] = 10$ , to  $tmp = 10$ .
4.  **$\mu$ op 2 (add):**  $tmp \leftarrow tmp + eax$ . Maan lo  $eax = 5$ , to  $tmp = 15$ . (Sirf registers, RISC jaisa.)
5.  **$\mu$ op 3 (store):**  $Mem[mem] \leftarrow tmp$ . Ab memory mein 15.

6. Dekho: ye bilkul RISC ka lw, add, sw hai. Decoder ne CISC instruction ko RISC jaise 3  $\mu$ ops mein tod diya.
7. Back end in  $\mu$ ops ko pipeline mein, out-of-order, register renaming ke saath chalata hai, jaise ek RISC core.
8. **Kyun?** Binary compatibility bani rahe (purane x86 programs chalein) aur saath mein RISC wali pipelining, out-of-order execution aur high clock bhi mile.

x86 instruction (CISC, programmer ko dikhta hai):  
`add [mem], eax ; Mem[mem] = Mem[mem] + eax`

Decoder ke baad 3 micro-ops (andar, chhupe hue):  
`uop1: load tmp <- Mem[mem] ; jaise MIPS lw`  
`uop2: add tmp <- tmp + eax ; jaise MIPS add`  
`uop3: store Mem[mem] <- tmp ; jaise MIPS sw`

Values ke saath: Mem[mem] = 10, eax = 5  
`uop1: tmp = 10`  
`uop2: tmp = 10 + 5 = 15`  
`uop3: Mem[mem] = 15`



Front end: complex x86 instruction → several simple  $\mu$ ops. Back end: pipelines, renames and reorders  $\mu$ ops like a RISC machine.

**x86 micro-op decode: CISC skin, RISC core**

How to draw: One x86 instruction box → a decoder box → three small  $\mu$ op boxes (load, add, store) → a RISC-like execution core box.

**Diagram kaise padhein:** Left se right chalo. Pehla dabba: CISC instruction `add [mem], eax`. Woh DECODER (splitter) mein jaata hai. Decoder se teen arrows teen chhote dabbon ( $\mu$ op load,  $\mu$ op add,  $\mu$ op store) mein. Teeno  $\mu$ ops ek "RISC-like core" mein jaate hain. Neeche ki line: front end tod-ta hai, back end RISC ki tarah chalata hai.

### Solved example (write the answer like this, 5 marks):

*Explain with a diagram how a modern x86 processor executes CISC instructions using micro-operations. Why is it said that CISC and RISC have merged?*

1. Draw D10.5a: x86 instruction → decoder →  $\mu$ ops (load, add, store) → RISC-like core.
2. The front-end decoder splits each complex x86 instruction into simple RISC-like  $\mu$ ops, e.g. `add [mem], eax` → load, add, store.
3. The back end pipelines, renames and reorders  $\mu$ ops like a RISC core.
4. Reason: keeps binary compatibility with decades of x86 software while gaining RISC-style pipelining and high clock rates.

5. Merged: CISC became RISC inside; RISC (ARM, RISC-V) added hundreds of instructions, SIMD/vector and 16-bit compressed instructions (a CISC-style density idea). Microarchitecture now matters more than the label.

**Answer:** "CISC outside, RISC inside"; both camps borrowed from each other, so the boundary has blurred.

**Galti mat karna:** (1)  $\mu$ ops programmer ko nahi dikhte; tum assembly mein  $\mu$ op nahi likh sakte. (2) "CISC obsolete hai" galat: x86 aaj bhi desktop/server pe raaj karta hai. (3) Class example mein `add [mem], eax = 3`  $\mu$ ops (load, add, store).

**Trick:** "Bahar CISC ka kurta, andar RISC ka body." Teen  $\mu$ ops = "Utha, Jod, Rakh" (load, add, store).

**Ek line mein:** Modern x86 ka decoder har CISC instruction ko RISC jaise  $\mu$ ops mein todta hai (add [mem], eax  $\rightarrow$  load, add, store), isliye aaj CISC aur RISC ki deewar dhundhli ho gayi hai.

## 10.6 Trends: RISC-V open ISA aur AI accelerators

**Pehle socho:** ARM ka design use karne ke liye paise (licence fee) dene padte hain. Socho ek "open recipe" ho jise koi bhi free mein use kare aur apni zaroorat ke hisaab se masale (extensions) jode: ye **RISC-V** hai. Doosra trend: agar tumhe sirf ek hi kaam baar-baar karna hai (jaise AI mein matrix multiply), to general-purpose CPU ki jagah usi kaam ke liye special chip banao. Jaise roti ke liye alag roti-maker, sab kuch karne wale mixer ki jagah.

**Definition:** **RISC-V** is an open, royalty-free, modular RISC ISA created at UC Berkeley in 2010: a small base integer ISA plus optional standard extensions (M, A, F, D, C, V). A **domain-specific ISA** is an instruction set designed for one class of workload (e.g. graphics or neural networks), trading generality for much higher efficiency, e.g. NVIDIA PTX for GPUs and Google TPU (matrix multiply).

**Mushkil shabd, aasaan matlab:**

**Open / royalty-free**

Specification sabke liye free; RISC-V chip banane ke liye kisi ko fees nahi deni.

**Modular**

Chhota base (RV32I) + jo chahiye woh extension jodo. M = multiply/divide, A = atomic, F = float, D = double, C = compressed 16-bit, V = vector.

**Accelerator**

Ek kaam ke liye bani special chip jo us kaam mein CPU se kai guna tez aur kam power wali hoti hai.

**GPU / PTX**

GPU mein hazaron simple cores jo ek saath (parallel) kaam karte hain. PTX NVIDIA ka GPU ISA hai.

## TPU

Google ki Tensor Processing Unit; iska ISA **matrix multiplication** ke around bana hai, jo neural networks ka main kaam hai.

### Kaise kaam karta hai:

1. **RISC-V open hai:** koi bhi company ya college bina fees RISC-V chip bana sakta hai.
2. **RISC-V modular hai:** microcontroller ko sirf base chahiye (RV32I); laptop chip base + M + A + F + D + C le sakta hai (jaise RV32IMAFDC).
3. **Adoption:** teaching se shuru, ab microcontrollers, data-centre aur AI chips tak. (2021 mein MIPS company bhi RISC-V pe aa gayi.)
4. **Domain-specific ISA:** workload fixed hai (jaise sirf matrix multiply) → generality hata do → wahi transistors us ek kaam pe lagao → bahut zyada efficiency.
5. **Examples:** GPU (NVIDIA PTX, hazaron simple cores, massive parallelism); Google TPU (matrix multiply ISA).
6. **Direction:** multicore + GPU + AI accelerators: architecture ka future specialisation hai (Module 6).

### Solved example (write the answer like this, 4 marks):

Write short notes on (a) RISC-V and (b) domain-specific ISAs for AI.

1. (a) RISC-V: open and royalty-free ISA from UC Berkeley (2010); modular: base RV32I (about 40 instructions) plus extensions M, A, F, D, C, V; clean RISC (fixed 32-bit, load/store, 32 registers); used from teaching to microcontrollers to data-centre/AI chips.
2. (b) Domain-specific ISAs: designed for one workload; NVIDIA PTX for GPUs (thousands of simple parallel cores); Google TPU centred on matrix multiply (core of neural networks).
3. Reason for (b): a fixed workload lets designers remove general-purpose features and gain large efficiency.

**Answer:** RISC-V = open + modular + Berkeley 2010; DSAs = specialise for efficiency (PTX, TPU).

**Galti mat karna:** RISC-V kisi ek company ki property nahi hai (open hai). Har RISC-V chip mein float nahi hota; float sirf F/D extension ke saath. TPU general-purpose CPU nahi hai.

**Trick:** RISC-V extension ka letter = uske English naam ka pehla akshar: **M**ultiply, **A**tomic, **F**loat, **D**ouble, **C**ompressed, **V**ector. Naam padho, ISA samjho: RV32IMAFD = base + M + A + F + D.

**Ek line mein:** RISC-V = free, open, modular RISC (Berkeley 2010); AI ke liye special ISAs (PTX, TPU) generality chhod ke efficiency jeet-te hain.

**Exam mein is unit se kya aata hai → kaise solve karein**

| Question type                                       | Method (steps)                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MCQ: "defining property of RISC" / "hallmarks"      | 1. Fixed length + load/store = RISC. 2. Variable length, memory operands, microcode = CISC. 3. Option mein jo RISC hallmark ho woh chuno.                                                                                                                                      |
| MCQ: x86 instruction length                         | Variable, 1 to 15 bytes. (RISC: fixed 32 bits = 4 bytes.)                                                                                                                                                                                                                      |
| Compare CISC vs RISC (5 marks)                      | 1. Dono ki definition. 2. Table: instructions count, length, memory access, decoder, CPI, pipelining, complexity location (har row pe ek "why"). 3. Examples. 4. $C = A + B$ ; $D = 2C$ code dono styles (5 vs 6 instructions). 5. Conclusion: complexity hardware ↔ compiler. |
| Same task in CISC vs RISC code / count instructions | 1. CISC: memory operand allowed (add eax, [B]). 2. RISC: har memory value ke liye lw, kaam registers pe, result ke liye sw. 3. Count karo (vector add 3 elements: 9 vs 12).                                                                                                    |
| Iron law numerical (CPU time)                       | 1. $T = 1/f$ (2 GHz → 0.5 ns). 2. Time = IC × CPI × T. 3. Units: ns → ms ( $\div 10^6$ ). 4. Comparison mein dono ka time nikaal ke chhota wala tez.                                                                                                                           |
| History / why CISC first / ARM vs x86               | 1. CISC first: memory mehengi, assembly haath se, compilers kamzor. 2. RISC: IBM 801, compilers + pipelining. 3. Dates: VAX 77, x86 78, MIPS 84, ARM 85, x86-64 2003, RISC-V 2010. 4. ARM = low power + licensing; x86 = compatibility. 5. Family tree draw karo.              |
| Code density vs pipelining (5 marks)                | 1. Density = CISC ka fayda (chhote programs, l-cache). 2. Thumb / RISC-V C se RISC gap bharta hai. 3. Pipelining ke 4 reasons (Fixed, Uniform, One cycle, Load/store). 4. Strength table. 5. Iron law line.                                                                    |
| Micro-ops (x86)                                     | 1. Diagram D10.5a. 2. add [mem], eax → load, add, store (3 $\mu$ ops). 3. Why: compatibility + RISC-style pipelining/out-of-order. 4. "Merged" points.                                                                                                                         |
| RISC-V / domain-specific ISA short notes            | RISC-V: open, royalty-free, modular (M A F D C V), Berkeley 2010. DSA: PTX (GPU), TPU (matrix multiply), specialisation = efficiency.                                                                                                                                          |

# Unit 11 · MIPS Registers & Instruction Formats

**Is unit ki kahani:** Unit 10 mein humne kaha "RISC = fixed 32-bit, load/store". Ab ek asli RISC, **MIPS**, ko andar se dekhte hain: CPU ke paas 32 registers hain (har ek ka kaam software ne tay kiya hai), aur har instruction 32 bits ka hai jo teen "formats" (R, I, J) mein se ek mein bata hota hai. Is unit ke baad tum koi bhi MIPS instruction ko binary/hex mein badal paoge aur hex ko wapas assembly mein. Unit 12 mein inhi instructions se C program likhenge, aur Unit 13 mein dekhenge ki hardware in fields ko kaise use karta hai.

## 11.1 Why MIPS for teaching

**Pehle socho:** Gaadi chalana seekhna ho to log pehle simple manual car pe seekhte hain, Formula-1 pe nahi. Ek baar clutch-gear samajh gaye to koi bhi gaadi chala loge. MIPS computer architecture ki "simple manual car" hai: **sirf 3 formats, har instruction 32 bits**, poora ek hafte mein seekh sakte ho, aur yahi soch RISC-V aur ARM mein seedhi kaam aati hai.

**Definition:** MIPS (Microprocessor without Interlocked Pipeline Stages) is a classic RISC ISA from Stanford (John Hennessy, 1984) with 32 general-purpose 32-bit registers, fixed 32-bit instructions in three formats (R, I, J) and a load/store memory model. It is used for teaching because it is clean, small and well supported by simulators and textbooks.

**Mushkil shabd, aasaan matlab:**

### Format

32 bits ko alag-alag hisson (fields) mein baantne ka tareeka. MIPS mein sirf 3: R, I, J.

### MARS / SPIM

Computer pe chalne wale MIPS simulators. Code likho, ek-ek line chalao (single-step), har register ki value dekho. Lab mein MARS 4.5.

### Patterson & Hennessy (P&H)

Course ki textbook "Computer Organization and Design"; iska datapath aur pipeline MIPS pe bana hai (Units 13 to 15).

**Kyun MIPS (4 reasons, exam mein yahi likho):**

1. **Clean, fixed-width design:** har instruction 32 bits, sirf 3 formats → poora ISA chhota, seekhna aasaan.
2. **Tooling:** MARS simulator mein step-by-step chala ke har register dekh sakte ho.
3. **Textbook:** P&H ka datapath aur pipeline MIPS pe bana hai.
4. **Transferable:** registers + formats wali soch RISC-V aur ARM mein waise hi chalti hai; sirf syntax badalta hai.

5. **Real history:** Stanford 1984; SGI workstations, PlayStation 1 (R3000A), Nintendo 64 (VR4300), PS2 (R5900) mein chala.

#### Solved example (direct):

*How many bytes long is every MIPS instruction, and how many instruction formats does MIPS have?*

1. Har instruction 32 bits.
2. 1 byte = 8 bits, to  $32 / 8 = 4$  bytes. Isliye agla instruction PC + 4 pe.
3. Formats: R, I, J = 3.

**Answer: 4 bytes; 3 formats (R, I, J).**

**Galti mat karna:** MIPS ke 3 formats hain, RISC-V ke 6 (R, I, S, B, U, J). Dono ko mix mat karna.

**Ek line mein:** MIPS = simple, fixed 32-bit, 3-format RISC jo seekhne mein aasaan aur RISC-V/ARM tak transfer hota hai.

## 11.2 MIPS register set aur conventions

**Pehle socho:** Hostel ke ek room mein 32 lockers hain. Hardware ke liye sab lockers ek jaise hain, par hostel walon ne rule bana diya: locker 4 se 7 mein "doston ke liye saamaan" (arguments), locker 2-3 mein "wapas milne wala jawab" (return value), locker 29 mein "stack ka pata", locker 31 mein "wapas kahan aana hai" ka parchi. Agar sab ye rule maanein to kisi ka bhi likha function kisi ke bhi function ko call kar sakta hai. Hardware ko farak nahi padta, ye rules software ka agreement (convention) hain.

**Definition:** MIPS has **32 general-purpose registers** of 32 bits each (\$0 to \$31). The hardware treats them alike (except \$zero, which is always 0), but the **register convention** gives each a software role. **Caller-saved** registers (\$t, \$a, \$v) may be changed by a called function, so the caller must save them if it needs their values. **Callee-saved** registers (\$s, \$sp, \$fp, \$ra, \$gp) must be restored by the called function before it returns. PC, HI and LO are separate special registers.

**Mushkil shabd, aasaan matlab:**

#### Register

CPU ke andar ek chhoti, bahut tez 32-bit memory. Sab maths registers pe hota hai.

#### Caller

Jo function doosre ko call karta hai (jaise main).

#### Callee

Jo function call hota hai (jaise sum).

#### Caller-saved

"Bulaane wala khud bachaye". Callee isko bina bataye badal sakta hai. Caller ko value chahiye to call se pehle stack pe rakh le.

### Callee-saved

"Bulaya gaya bachaye". Callee use kare to pehle purani value save kare aur return se pehle wapas daal de. Caller ke liye value "safe" hai.

### Stack

Memory ka ek hissa jahan functions temporary cheezein rakhte hain (Unit 12). \$sp uska top batata hai.

### PC (program counter)

Agle instruction ka address. Ye 32 registers mein se nahi hai; khud PC + 4 hota hai ya branch/jump se badalta hai.

### HI, LO

Do special registers jahan mult aur div apna result rakhte hain. mfhi/mflo se padhte hain.

## Har register ka naam, number, kaam aur KYUN:

1. **\$zero (\$0): hamesha 0.** Isme likho to bhi 0 hi rahega (write ignore). Kyun: 0 itna common hai (copy karna, compare karna, clear karna) ki ek register ko permanently 0 bana diya. Jaise `move $t0, $t1` asal mein `addu $t0, $zero, $t1` hai ( $0 + \$t1$ ).
2. **\$at (\$1): assembler temporary.** Kyun: pseudo-instructions (jaise `blt`, `la`, `bada li`) ko assembler 2 asli instructions mein todta hai aur beech ka result \$at mein rakhta hai. Tum ise mat chhuo.
3. **\$v0, \$v1 (\$2, \$3): "values" = return values.** Function apna jawab \$v0 mein deta hai (64-bit jawab ho to \$v1 bhi). Syscall ka code bhi \$v0 mein daalte hain.
4. **\$a0 to \$a3 (\$4 to \$7): "arguments".** Function ko pehle 4 inputs inme dete hain; zyada hon to stack pe. Kyun registers mein: memory se tez.
5. **\$t0 to \$t7 (\$8 to \$15), \$t8, \$t9 (\$24, \$25): "temporaries", caller-saved.** Chhote-mote hisaab ke liye. Callee inhe bina puchhe badal sakta hai, isliye inko save karne ki zimmedari caller ki. Kyun aisa rule: chhota function bina stack ke jaldi kaam kar le.
6. **\$s0 to \$s7 (\$16 to \$23): "saved", callee-saved.** Lambi zindagi wale variables (jaise loop ka sum jo function call ke baad bhi chahiye). Callee inhe use kare to pehle stack pe save, return se pehle restore.
7. **\$k0, \$k1 (\$26, \$27): kernel.** OS ke interrupt handler ke liye reserved; user program inhe use na kare.
8. **\$gp (\$28): global pointer.** Global/static data area ke beech mein point karta hai, taaki global variable ek hi lw (offset ke saath) se mil jaaye.
9. **\$sp (\$29): stack pointer.** Stack ke top (aakhri pushed word) ka address. Stack neeche ki taraf badhta hai.
10. **\$fp (\$30): frame pointer.** Current function ke stack frame ki shuruaat pe fixed rehta hai, jabki \$sp function ke beech badal sakta hai. Locals ko fixed offset se dhoondhne mein madad.
11. **\$ra (\$31): return address.** jal isme PC + 4 likhta hai; jr \$ra se wapas aate hain.

### MIPS register conventions (PC, HI and LO are separate special registers)

| Name      | Number | Use                          | Saved by |
|-----------|--------|------------------------------|----------|
| \$zero    | 0      | constant 0 (writes ignored)  | —        |
| \$at      | 1      | assembler temporary          | —        |
| \$v0-\$v1 | 2-3    | return values / syscall code | caller   |
| \$a0-\$a3 | 4-7    | function arguments           | caller   |
| \$t0-\$t7 | 8-15   | temporaries                  | caller   |
| \$s0-\$s7 | 16-23  | saved values                 | CALLEE   |
| \$t8-\$t9 | 24-25  | more temporaries             | caller   |
| \$k0-\$k1 | 26-27  | reserved for OS kernel       | —        |
| \$gp      | 28     | global pointer               | callee   |
| \$sp      | 29     | stack pointer                | callee   |
| \$fp      | 30     | frame pointer                | callee   |
| \$ra      | 31     | return address (set by jal)  | callee   |

#### MIPS register table

How to draw: Four columns (name, number, use, saved by); memorise the number ranges: 0, 1, 2-3, 4-7, 8-15, 16-23, 24-25, 26-27, 28, 29, 30, 31.

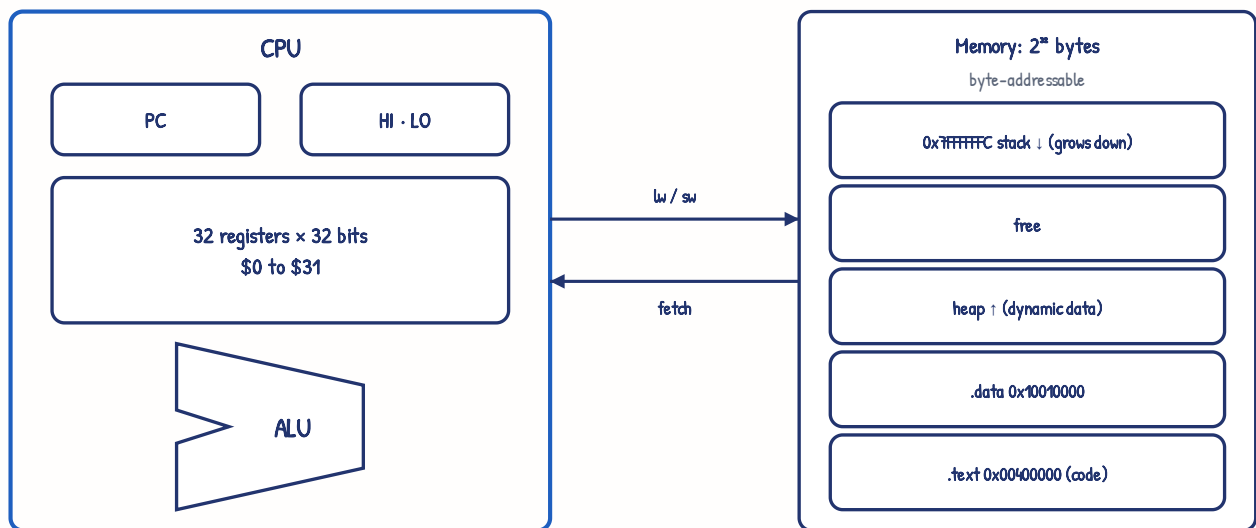
**Diagram kaise padhein:** Chaar columns: naam, number, use, saved by. Pehle number column dekho aur ranges yaad karo: 0, 1, 2-3, 4-7, 8-15, 16-23, 24-25, 26-27, 28, 29, 30, 31. Phir "saved by" column: \$t aur \$a/\$v ke saamne "caller", \$s ke saamne "CALLEE" (highlighted). Example: \$sp ki row = 29, stack pointer, callee.

#### Caller vs callee, values ke saath:

*main has a running total 50 in \$s0 and a temporary 7 in \$t0. It calls f. Inside, f uses both \$s0 and \$t0. What does main see after f returns?*

1. \$t0 caller-saved hai → f use badal sakta hai (maan lo 99 kar diya) bina wapas kiye.
2. Agar main ko 7 baad mein chahiye tha, to main ko call se pehle 7 stack pe save karna chahiye tha.
3. \$s0 callee-saved hai → f ne shuru mein 50 stack pe rakha (sw \$s0, 0(\$sp)), apna kaam kiya, aur jir \$ra se pehle 50 wapas load kiya (lw \$s0, 0(\$sp)).
4. Isliye main ko \$s0 = 50 milta hai (guaranteed), \$t0 kuch bhi ho sakta hai.

**Answer: \$s0 = 50 (preserved by f); \$t0 is not guaranteed (main must save it itself if needed).**



Only lw/sw (and lb/sb, lh/sh) move data between registers and memory; everything else works on registers (load/store).

### MIPS machine model: CPU + byte-addressable memory

How to draw: CPU box with PC, HI/LO, register file and ALU; memory box divided top-to-bottom into stack (down), free, heap (up), .data, .text; arrows lw/sw and fetch.

**Diagram kaise padhein:** Left box CPU: PC, HI-LO, 32 registers aur ALU. Right box memory (2<sup>32</sup> bytes, har byte ka apna address). Memory upar se neeche: stack (0x7FFFFFFC se neeche badhta hai), free, heap (upar badhta hai), .data (0x10010000), .text (0x00400000, code). Beech ke arrows: data ke liye sirf lw/sw, aur instruction ke liye fetch. Yahi load/store model hai.

### Solved example (exam MCQ / match):

Match: \$zero, \$a0, \$t0, \$sp, \$ra with numbers. After `addi $zero, $zero, 5`, what is in \$zero?

- \$zero = 0, \$a0 = 4 (\$a0-\$a3 = 4-7 ka pehla), \$t0 = 8 (\$t0-\$t7 = 8-15 ka pehla), \$sp = 29, \$ra = 31.
- \$zero mein likhna ignore hota hai → value 0 hi rahegi.

**Answer: 0, 4, 8, 29, 31; \$zero still contains 0.**

**Galti mat karna:** (1) \$t aur \$s ke roles ulte mat karna: **\$s callee-saved**. (2) PC ko 32 registers mein mat gin-na. (3) \$t8, \$t9 = 24, 25 (16-23 nahi, wahan \$s hain). (4) Return value \$v0 mein, \$a0 mein nahi.

**Trick:** "t = temporary, don't trust it; s = saved, it survives." Numbers ke liye ginti: **0 zero, 1 at, 2 v, 4 a, 8 t, 16 s, 24 t, 26 k, 28 gp, 29 sp, 30 fp, 31 ra** (2, 4, 8, 16 = doubling!).

**Ek line mein:** 32 registers, roles convention se: \$zero = 0, \$v = jawab, \$a = arguments, \$t = kachcha kaam (caller bachaye), \$s = pakka kaam (callee bachaye), \$sp stack, \$ra wapsi ka pata.

## 11.3 R-format (register to register)

**Pehle socho:** Ek form socho jisme fixed boxes hain: "kaam ka type", "pehla number kahan se", "doosra kahan se", "jawab kahan rakhna". Har form same size ka hai, boxes hamesha same jagah. Clerk (CPU) bina soche har box seedha padh leta hai. add \$t0, \$s1, \$s2 jaise instruction mein teeno cheezein registers hain, isliye iska form "Register format" hai.

**Definition:** The R-format is used for instructions whose operands are all registers (add, sub, and, or, slt, shifts, jr). Its 32 bits are divided as **opcode (6) | rs (5) | rt (5) | rd (5) | shamt (5) | funct (6)**. The opcode is 0 for all R-type instructions; the funct field selects the operation; rs and rt are the source registers and rd is the destination.

**Mushkil shabd, aasaan matlab:**

### Field

32 bits ka ek hissa jiska ek kaam hai.

### opcode (6 bits, bits 31-26)

"Operation code": instruction ka type. 6 bits  $\rightarrow 2^6 = 64$  possible. R-type ke liye hamesha 000000.

### rs (5 bits, 25-21)

"Register source": pehla input register ka *number*. 5 bits kyun? 32 registers =  $2^5$ , to 0 se 31 tak 5 bits mein.

### rt (5 bits, 20-16)

"Register target": doosra input register (R-type mein).

### rd (5 bits, 15-11)

"Register destination": jahan result likhna hai.

### shamt (5 bits, 10-6)

"Shift amount": sirf sll/srl/sra mein; kitni position shift. Baaki sab mein 00000. 5 bits kyun: 32-bit number ko 0 se 31 tak shift karna kaafi.

### funct (6 bits, 5-0)

"Function": opcode 0 hai, to asli operation yahi batata hai. add = 100000 (0x20), sub = 100010 (0x22).

### Hex

Base 16. 4 bits = 1 hex digit (0000 = 0, 1010 = A, 1111 = F). 32 bits = 8 hex digits.

**Kaise kaam karta hai (32 bits kyun aur kaise bante hain):**

- 3 registers  $\times$  5 bits = 15 bits. opcode 6 + funct 6 = 12 bits. shamt 5 bits. Total 15 + 12 + 5 = 32. Poora word bhar gaya.
- Kyun fixed 32: har instruction 4 bytes  $\rightarrow$  PC + 4 se agla instruction, aur fields hamesha same jagah  $\rightarrow$  decoder simple (Unit 10).

3. **Assembly order ≠ binary order:** assembly mein likhte hain add rd, rs, rt (destination pehle), par binary mein order rs, rt, rd.
4. **Encoding recipe:** (a) register naam → number; (b) rs, rt, rd mein sahi jagah rakho; (c) har field binary mein, sahi width (5 bits ke liye aage 0 lagao); (d) 32 bits ko 4-4 ke group mein todo; (e) har group → hex digit.
5. **Shifts:** sll rd, rt, shamt: source **rt** mein jata hai, rs = 00000, shamt = shift amount.

| Instr | funct (bin) | hex  | Instr       | funct (bin)     | hex         |
|-------|-------------|------|-------------|-----------------|-------------|
| add   | 100000      | 0x20 | sll         | 000000          | 0x00        |
| addu  | 100001      | 0x21 | srl         | 000010          | 0x02        |
| sub   | 100010      | 0x22 | sra         | 000011          | 0x03        |
| and   | 100100      | 0x24 | jr          | 001000          | 0x08        |
| or    | 100101      | 0x25 |             |                 |             |
| xor   | 100110      | 0x26 | mult / div  | 011000 / 011010 | 0x18 / 0x1A |
| nor   | 100111      | 0x27 | mfhi / mflo | 010000 / 010010 | 0x10 / 0x12 |

R-format: add \$t1, \$t2, \$t3 = 0x014B4820



### R-format fields (add \$t1, \$t2, \$t3)

How to draw: A 32-bit bar split 6|5|5|5|5|6 with bit numbers 31-26, 25-21, 20-16, 15-11, 10-6, 5-0; write the field names and the binary values inside.

**Diagram kaise padhein:** Ek 32-bit patti, chhe dabbe: 6 | 5 | 5 | 5 | 5 | 6. Har dabbe mein naam, binary value aur neeche note. Class example add \$t1, \$t2, \$t3: opcode 000000, rs 01010 (\$t2 = 10), rt 01011 (\$t3 = 11), rd 01001 (\$t1 = 9, DEST), shamt 00000, funct 100000 (add). Total = 0x014B4820.

### Solved example 1 (full encoding walkthrough):

Encode add \$t0, \$s1, \$s2 in binary and hex.

- Format: add = R-type → opcode = 000000, funct = 100000 (0x20).
- Assembly add rd, rs, rt se: rd = \$t0, rs = \$s1, rt = \$s2.
- Numbers: \$t0 = 8 (\$t0-\$t7 = 8-15 ka pehla). \$s1 = 17 (\$s0 = 16, to \$s1 = 17). \$s2 = 18.
- 5-bit binary: 17 = 16 + 1 → **10001**. 18 = 16 + 2 → **10010**. 8 → **01000**.
- shamt = 0 → 00000 (add shift nahi karta).
- Binary order mein rakho (rs, rt, rd):

| opcode | rs (\$s1) | rt (\$s2) | rd (\$t0) | shamt | funct  |
|--------|-----------|-----------|-----------|-------|--------|
| 000000 | 10001     | 10010     | 01000     | 00000 | 100000 |
| 6 bits | 5         | 5         | 5         | 5     | 6      |

7. Jodo: 00000010001100100100000000100000 (count: 6 + 5 + 5 + 5 + 5 + 6 = 32 bits ✓).

8. 4-4 ke group: 0000 | 0010 | 0011 | 0010 | 0100 | 0000 | 0010 | 0000.

9. Hex digits: 0000 = 0, 0010 = 2, 0011 = 3, 0010 = 2, 0100 = 4, 0000 = 0, 0010 = 2, 0000 = 0.

10. Check (ulta): 0x02324020 ko wapas 6-5-5-5-5-6 mein todo → 0, 17, 18, 8, 0, 0x20 → add \$t0, \$s1, \$s2 ✓.

**Answer: 000000 10001 10010 01000 00000 100000 = 0x02324020.**

### Solved example 2 (decode hex → assembly, class homework):

Decode 0x012A4020.

1. Har hex digit ko 4 bits mein: 0 = 0000, 1 = 0001, 2 = 0010, A = 1010, 4 = 0100, 0 = 0000, 2 = 0010, 0 = 0000.
2. Poora: 0000 0001 0010 1010 0100 0000 0010 0000.
3. Pehle 6 bits (opcode) = 000000 → R-type. To ab 6-5-5-5-5-6 mein todo: 000000 | 01001 | 01010 | 01000 | 00000 | 100000.
4. funct = 100000 = 0x20 → add.
5. rs = 01001 = 9 = \$t1; rt = 01010 = 10 = \$t2; rd = 01000 = 8 = \$t0; shamt = 0.
6. Assembly mein rd pehle: add rd, rs, rt.

**Answer: add \$t0, \$t1, \$t2.**

### Solved example 3 (shift):

Encode sll \$t0, \$t1, 2.

1. R-type, funct sll = 000000, opcode 000000.
2. Shift mein source **rt** = \$t1 = 9 = 01001; rs = 00000 (use nahi hota); rd = \$t0 = 8 = 01000; shamt = 2 = 00010.
3. 000000 00000 01001 01000 00010 000000 → 0000 0000 0000 1001 0100 0000 1000 0000.

**Answer: 0x00094080.**

**Galti mat karna:** (1) Binary mein rd ko pehle mat likhna; order **rs, rt, rd**. (2) R-type ka opcode hamesha 0; operation **funct** se. (3) 5-bit field mein aage ke 0 mat bhoolna (8 = 01000, sirf 1000 nahi). (4) Shift mein source rs mein nahi, rt mein.

**Trick:** R-format widths: "6-5-5-5-6" (dono kinaare 6, beech mein chaar 5). Field order: "Old Rats Take Dinner Some Food" = opcode, rs, rt, rd, shamt, funct.

**Ek line mein:** R = op 6 | rs 5 | rt 5 | rd 5 | shamt 5 | funct 6; opcode 0, funct operation batata hai, rd destination hai par binary mein teesre number pe aata hai.

## 11.4 I-format (immediate, loads, stores, branches)

**Pehle socho:** Kabhi-kabhi instruction ko register nahi, ek **seedha number** chahiye: "\$sp mein se 8 ghatao", "address \$sp + 16 se load karo", "agar barabar hain to 5 instructions aage jao". Ye number instruction ke andar hi likha hota hai (immediate = "turant, instruction ke saath hi"). Teen registers ki jagah do registers + ek 16-bit number: yahi Immediate format hai.

**Definition:** The **I-format** is used for instructions with a 16-bit constant: arithmetic/logic immediates (addi, andi, ori, slti, lui), loads and stores (lw, sw) and conditional branches (beq, bne). Its fields are **opcode (6) | rs (5) | rt (5) | immediate (16)**. For addi and lw, rt is the **destination**; for sw, rt is the data to store; for beq/bne, rs and rt are compared and the immediate is a word offset from PC + 4.

**Mushkil shabd, aasaan matlab:**

### Immediate

Instruction ke andar likha number (constant). addi \$t0, \$t0, 5 mein 5.

### Two's complement (recap Unit 4)

Negative number likhne ka tareeka: positive ke bits ulto (invert) aur 1 jodo. 16 bits mein range -32768 se +32767.

### Sign extension

16-bit number ko 32-bit banana uska sign bit (bit 15) upar copy karke. 0xFFFF (-5) → 0xFFFFFFFF (-5) (phir bhi -5). 0x0005 → 0x00000005.

### Zero extension

Upar ke 16 bits mein hamesha 0 bharna. Sirf andi, ori, xori karte hain.

### Base + offset

lw \$t1, 16(\$sp): address = \$sp (base register) + 16 (offset). Jaise "gali ka number + ghar number".

### Branch offset

beq/bne mein immediate = kitne *instructions* (words) aage/peeche jaana hai, **PC + 4** se gin ke.

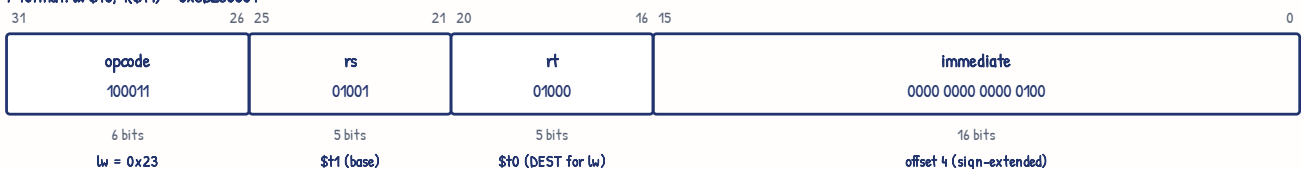
**Kaise kaam karta hai:**

1. Fields: opcode 6 + rs 5 + rt 5 + immediate 16 = 32 bits.

- addi rt, rs, imm:**  $rt \leftarrow rs + \text{signext}(\text{imm})$ . Yahan **rt destination** hai (R-type ki tarah rd nahi, kyunki rd ki jagah immediate ne le li).
- lw rt, offset(rs):**  $rt \leftarrow \text{Memory}[rs + \text{signext}(\text{offset})]$ . rs = base, rt = destination.
- sw rt, offset(rs):**  $\text{Memory}[rs + \text{signext}(\text{offset})] \leftarrow rt$ . Yahan rt **source** (data) hai.
- beq rs, rt, label:** agar  $rs == rt$  to  $PC \leftarrow PC + 4 + \text{signext}(\text{imm}) \times 4$ , warna  $PC \leftarrow PC + 4$ .
- Sign vs zero extend:** addi, lw, sw, beq, bne, slti sign-extend; andi, ori, xori zero-extend; lui rt, imm imm ko upar ke 16 bits mein daalta hai.

| Instr      | opcode (bin)    | hex         | Instr   | opcode (bin)    | hex         |
|------------|-----------------|-------------|---------|-----------------|-------------|
| addi       | 001000          | 0x08        | beq     | 000100          | 0x04        |
| addiu      | 001001          | 0x09        | bne     | 000101          | 0x05        |
| slti       | 001010          | 0x0A        | lw      | 100011          | 0x23        |
| andi / ori | 001100 / 001101 | 0x0C / 0x0D | sw      | 101011          | 0x2B        |
| lui        | 001111          | 0x0F        | lb / sb | 100000 / 101000 | 0x20 / 0x28 |

I-format: lw \$t0, 4(\$t1) = 0x8D280004



**I-format fields (lw \$t0, 4(\$t1))**

How to draw: A 32-bit bar split 6|5|5|16; label opcode, rs (base), rt (destination for loads), immediate (offset); write the binary.

**Diagram kaise padhein:** 32-bit patti chaar dabbon mein: 6 | 5 | 5 | 16. lw \$t0, 4(\$t1): opcode 100011 (lw = 0x23), rs 01001 (\$t1, base), rt 01000 (\$t0, DEST), immediate 0000 0000 0000 0100 (4). Address = \$t1 + 4; wahan ka word \$t0 mein. Total 0x8D280004.

**Formula:** Effective address =  $rs + \text{signext}(\text{imm})$

Branch target =  $(PC + 4) + \text{signext}(\text{imm}) \times 4 \Leftrightarrow \text{imm} = (\text{target} - (PC + 4)) / 4$

PC = address of the branch itself; PC + 4 = next instruction;  $\times 4$  because the offset counts words (4 bytes each).

**Solved example 1 (load, class homework):**

Encode `lw $t1, 16($sp)`.

- lw = I-type, opcode 0x23 = 100011.
- lw rt, offset(rs):  $rt = \$t1 = 9 = 01001$ ;  $rs = \$sp = 29 = 11101$  ( $16 + 8 + 4 + 1 = 29$ ); imm = 16.
- 16 in 16 bits = 0000 0000 0001 0000.
- Rakho: 100011 | 11101 | 01001 | 0000000000010000.
- 4-4 group: 1000 1111 1010 1001 0000 0000 0001 0000  $\rightarrow$  8 F A 9 0 0 1 0.

**Answer: 0x8FA90010.**

### Solved example 2 (negative immediate, two's complement):

Encode `addi $t0, $s1, -5`.

1. `addi` opcode =  $0x08 = 001000$ .
2.  $rs = \$s1 = 17 = 10001$  (source);  $rt = \$t0 = 8 = 01000$  (destination).
3.  $-5$  ko 16-bit two's complement mein:  $+5 = 0000\ 0000\ 0000\ 0101$ .
4. Invert ( $0 \leftrightarrow 1$ ):  $1111\ 1111\ 1111\ 1010$ .
5.  $+1$ :  $1111\ 1111\ 1111\ 1010 + 1 = 1111\ 1111\ 1111\ 1011 = 0xFFFF$ .
6. Check:  $0xFFFF = 65531$ ;  $65531 - 65536 = -5 \checkmark$ .
7. Rakho:  $001000 \mid 10001 \mid 01000 \mid 111111111111011$ .
8. 4-4 group:  $0010\ 0010\ 0010\ 1000\ 1111\ 1111\ 1111\ 1011 \rightarrow 2\ 2\ 2\ 8\ F\ F\ F\ B$ .

**Answer:  $0x2228FFFF$ . (Isi tarah `addi $sp, $sp, -8 = 0x23BDDFFF`, kyunki  $-8 = 0xFFFF$ .)**

### Solved example 3 (branch, easy, forward):

`beq $t0, $t1, Done` is at address  $0x00400008$  and `Done` is at  $0x00400020$ . Find the immediate and encode it.

1.  $PC + 4 = 0x00400008 + 4 = 0x0040000C$ .
2. Door (bytes) = target - ( $PC + 4$ ) =  $0x00400020 - 0x0040000C = 0x14 = 20$  bytes ( $0x20 = 32$ ,  $0x0C = 12$ ,  $32 - 12 = 20$ ).
3. Words =  $20 / 4 = 5$ . Isliye  $imm = 5 = 0000\ 0000\ 0000\ 0101$ .
4. `beq` opcode  $000100$ ;  $rs = \$t0 = 01000$ ;  $rt = \$t1 = 01001$ .
5.  $000100\ 01000\ 01001\ 0000000000000101 \rightarrow 0001\ 0001\ 0000\ 1001\ 0000\ 0000\ 0000\ 0101$ .
6. Check:  $0x0040000C + 5 \times 4 = 0x0040000C + 0x14 = 0x00400020 \checkmark$ .

**Answer:  $imm = 5$ ; word =  $0x11090005$ .**

### Solved example 4 (branch, exam-level, backward loop):

A loop ends with `bne $t0, $zero, Loop` at address  $0x00400100$ ; Loop is at  $0x004000F0$ . Find the 16-bit immediate (decimal and hex) and the machine word.

1.  $PC + 4 = 0x00400100 + 4 = 0x00400104$ .
2. target - ( $PC + 4$ ) =  $0x004000F0 - 0x00400104$ . Sirf last digits:  $0xF0 = 240$ ,  $0x104 = 260$ .  $240 - 260 = -20$  bytes.
3. Words =  $-20 / 4 = -5$  (negative = peeche jaana).
4.  $-5$  in 16 bits =  $0xFFFF$  (example 2 jaisa:  $0000\ 0000\ 0000\ 0101 \rightarrow$  invert  $1111\ 1111\ 1111\ 1010 \rightarrow +1 \rightarrow 1111\ 1111\ 1111\ 1011$ ).
5. `bne` opcode =  $000101$ ;  $rs = \$t0 = 01000$ ;  $rt = \$zero = 00000$ .

6. 000101 01000 00000 111111111111011 → 0001 0101 0000 0000 1111 1111 1111 1011.

7. Check:  $0x00400104 + (-5 \times 4) = 0x00400104 - 0x14 = 0x004000F0$  ✓.

Answer: imm = -5 = 0xFFFFB; word = 0x1500FFFB.

### Solved example 5 (decode, hex → assembly):

Decode (a) 0x8D280004 (Lab 11) and (b) 0x2108FFFC (addi).

1. (a) 8 D 2 8 0 0 0 4 → 1000 1101 0010 1000 0000 0000 0000 0100.

2. Pehle 6 bits = 100011 = 0x23 = lw → I-type, to 6-5-5-16 mein todo: 100011 | 01001 | 01000 | 00000000000000100.

3. rs = 01001 = 9 = \$t1 (base); rt = 01000 = 8 = \$t0 (dest); imm = 4. Likho lw rt, imm(rs) → **lw \$t0, 4(\$t1)**.

4. (b) 2 1 0 8 F F F C → 0010 0001 0000 1000 1111 1111 1111 1100 → 001000 | 01000 | 01000 | 1111111111111100.

5. Opcode 001000 = addi; rs = \$t0; rt = \$t0; imm = 0xFFFFC.

6. 0xFFFFC ka bit 15 = 1 → negative. Invert: 0000 0000 0000 0011, +1 = 0000 0000 0000 0100 = 4 → imm = -4.

Answer: (a) lw \$t0, 4(\$t1); (b) addi \$t0, \$t0, -4.

**Galti mat karna:** (1) lw mein rt **destination** hai, source nahi. (2) 0xFFFFB ko 65531 mat padhna; sign-extend karke -5. (3) Branch offset PC se nahi, **PC + 4** se; aur bytes nahi, **words** (÷ 4). (4) andi/ori/xori zero-extend karte hain, sign-extend nahi.

**Trick:** I-format = "6-5-5-16". Branch: "agle se gino, chaar se bhaago" =  $(\text{target} - (\text{PC} + 4)) \div 4$ .

**Ek line mein:** I = op 6 | rs 5 | rt 5 | imm 16; addi/lw mein rt destination, address = rs + signext(imm), branch imm =  $(\text{target} - (\text{PC} + 4)) / 4$ .

## 11.5 J-format (jumps)

**Pehle socho:** Branch sirf  $\pm 32768$  instructions tak ja sakta hai (16-bit offset). Door kisi function pe jaana ho to? Jump instruction ko register ki zaroorat nahi, bas ek bada address chahiye. To opcode ke baad bache **saare 26 bits** address ko de do. Aur kyunki har instruction ka address 4 ka multiple hai (last 2 bits hamesha 00), un 2 bits ko likhne ki zaroorat hi nahi: 26 bits asal mein 28 bits ka kaam karte hain.

**Definition:** The **J-format** is used for the unconditional jumps **j** (opcode 2) and **jal** (opcode 3). Its fields are **opcode (6) | address (26)**. The 32-bit target is formed pseudo-directly: **target = PC+4[31:28] || address || 00**, so a jump can reach any instruction in the current 256 MB region. **jal** also stores PC + 4 in \$ra.

**Mushkil shabd, aasaan matlab:**

### address field (26 bits)

Target ka **word number** (byte address  $\div 4$ ), bina upar ke 4 bits ke.

### || (concatenate)

Bits ko saath mein chipkana.  $0000 || 11 = 000011$ .

### Pseudo-direct

"Lagbhag seedha" address: 28 bits instruction se, upar ke 4 bits current PC + 4 se.

### 256 MB region

28 bits =  $2^{28}$  bytes = 256 MB. Jump sirf usi 256 MB block ke andar ja sakta hai jisme PC + 4 hai.

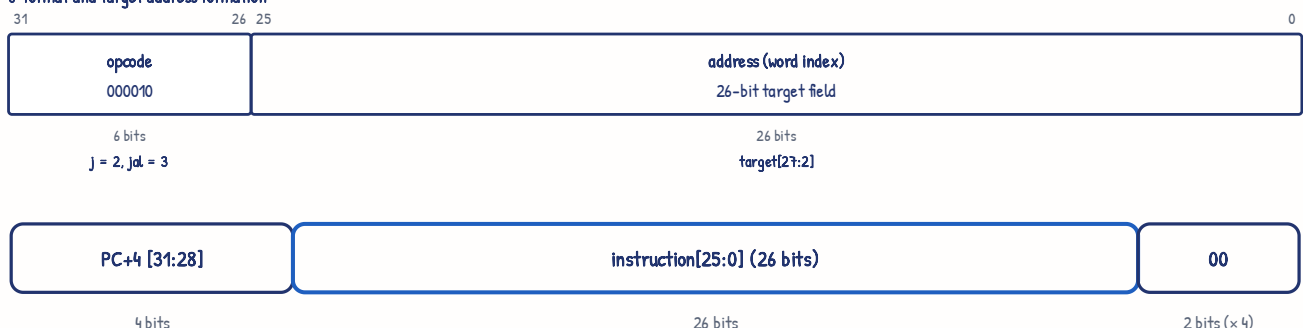
### j<sub>r</sub>

Jump register: register mein rakhe address pe jao. Ye R-type hai (funct 0x08), J-type nahi.

### Kaise kaam karta hai (encode aur target banana):

1. Opcode: **j** = 000010, **jal** = 000011.
2. Encode: address field = target  $\div 4$  (= target  $\gg 2$ ), sirf neeche ke 26 bits rakho.
3. Decode (CPU kya karta hai): 26-bit field ko 2 baar left shift ( $\times 4$ )  $\rightarrow$  28 bits.
4. Upar ke 4 bits PC + 4 se lo (bits 31-28).
5. Target = PC+4[31:28] || address || 00 (4 + 26 + 2 = 32 bits).
6. **jal** pehle \$ra  $\leftarrow$  PC + 4 likhta hai (wapas aane ka pata), phir jump.
7. 256 MB se door jaana ho to address register mein load karke **j<sub>r</sub>** use karo.

J-format and target address formation



Jump address (32 bits) = PC+4[31:28] || address || 00  $\rightarrow$  reaches any word in the current 256 MB region.

Example: **j** 0x00400018  $\rightarrow$  field = 0x00400018  $\gg 2$  = 0x100006  $\rightarrow$  word 0x08100006.

### J-format and jump target formation

How to draw: A 6|26 bar; below it, the 32-bit target built from 4 bits of PC+4, the 26-bit field and two zero bits.

**Diagram kaise padhein:** Upar 6 | 26 patti: opcode ( $j = 2$ ,  $jal = 3$ ) aur 26-bit address. Neeche target banane ki patti: left mein  $PC+4$  ke upar ke 4 bits, beech mein instruction ke 26 bits, right mein "00" ( $\times 4$ ). Example line:  $j\ 0x00400018 \rightarrow$  field =  $0x100006 \rightarrow$  word  $0x08100006$ .

### Solved example 1 (encode j):

Encode  $j\ 0x00400018$  and verify the target.

1. Target binary:  $0x00400018 = 0000\ 0000\ 0100\ 0000\ 0000\ 0000\ 0001\ 1000$ .
2.  $\div 4$  (do bits right shift):  $0x00400018 \gg 2 = 0x00100006$ . (Decimal:  $4194328 / 4 = 1048582 = 0x100006$ .)
3. 26 bits mein:  $00\ 0001\ 0000\ 0000\ 0000\ 0000\ 0110$ .
4. Opcode  $j = 000010$ . Jodo:  $000010\ |\ 0000010000000000000000000000110$ .
5. 4-4 group:  $0000\ 1000\ 0001\ 0000\ 0000\ 0000\ 0000\ 0110 \rightarrow 0\ 8\ 1\ 0\ 0\ 0\ 0\ 6$ .
6. Verify:  $PC+4$  upar ke 4 bits =  $0000$  (code  $0x004xxxxx$  mein hai).  $0x100006 \times 4 = 0x0400018$  (28 bits). Aage  $0000$  chipkao  $\rightarrow 0x00400018 \checkmark$ .

**Answer:  $0x08100006$ .**

### Solved example 2 (jal):

Encode  $jal\ 0x00400030$ . If this  $jal$  is at address  $0x00400020$ , what goes into  $\$ra$ ?

1.  $0x00400030 \gg 2 = 0x0010000C$  ( $0x30 / 4 = 0x0C$ ,  $0x400000 / 4 = 0x100000$ ).
2. Opcode  $jal = 000011$ .  $000011\ |\ 00\ 0001\ 0000\ 0000\ 0000\ 0000\ 1100$ .
3. 4-4 group:  $0000\ 1100\ 0001\ 0000\ 0000\ 0000\ 0000\ 1100 = 0x0C10000C$ .
4.  $\$ra = PC + 4 = 0x00400020 + 4 = 0x00400024$ .

**Answer:  $0x0C10000C$ ;  $\$ra = 0x00400024$  (= 4194340 decimal).**

**Galti mat karna:** (1) Field mein poora address mat likhna; pehle  $\div 4$ . (2)  $j$  kisi bhi 32-bit address pe nahi ja sakta, sirf current 256 MB block mein. (3)  $jr$  J-format nahi, R-format hai. (4) Class ki "MIPS Encoding Reference" sheet  $j$  ko PC-relative dikhati hai (teaching variant); exam mein lecture wala rule ( $PC+4[31:28] \parallel \text{addr} \parallel 00$ ) likho.

**Trick:**  $J = "6 + 26"$ . Encode: " $\div 4$ , upar ke 4 phenko". Decode: " $\times 4$ , upar PC ke 4 chipkao".

**Ek line mein:**  $J = \text{op } 6\ |\ \text{address } 26$ ; target =  $PC+4[31:28] \parallel \text{address} \parallel 00$  (256 MB region);  $j = 2$ ,  $jal = 3$  ( $jal\ \$ra$  mein  $PC + 4$  bhi rakhta hai).

## 11.6 RISC-V ka modular ISA

**Pehle socho:** Pizza order karte waqt base (dough + sauce) fixed hai, toppings tum chunte ho: cheese, olives, paneer. Chhote bachche ke liye sirf base, party ke liye sab toppings. RISC-V aisa hi hai: **chhota base ISA (RV32I) + jo extension chahiye woh jodo**. Aur MIPS jaisi hi RISC DNA: 32 registers, zero register, fixed 32-bit, load/store.

**Definition:** RISC-V is an open, royalty-free RISC ISA organised as a small mandatory **base integer ISA** (RV32I: about 40 instructions, registers x0 to x31 with x0 = 0) plus optional standard **extensions**: M (multiply/divide), A (atomic), F (single float), D (double float), C (compressed 16-bit), V (vector). It uses six basic formats: R, I, S, B, U, J.

**Mushkil shabd, aasaan matlab:**

### Base ISA (RV32I)

Har RISC-V chip mein zaroori minimum: integer add/sub, logic, load/store, branch, jump. "32" = 32-bit registers, "I" = integer.

### Extension

Optional instruction groups. Naam mein letter jodte hain: RV32IMAFD = base + M + A + F + D.

### Atomic

Ek operation jo beech mein toot nahi sakta; multicore mein do cores ek hi variable ko ek saath na bigadein.

### S, B, U formats

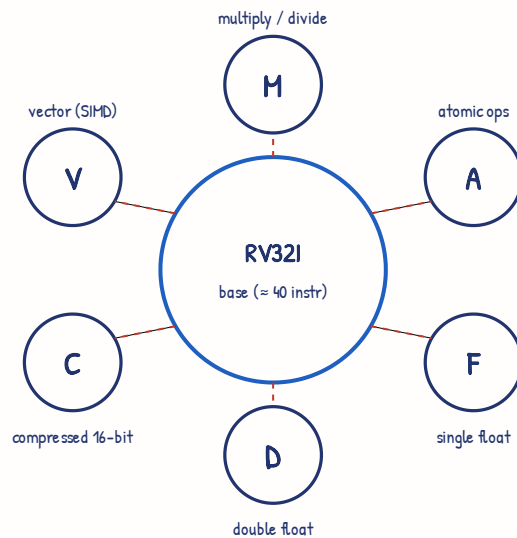
S = store, B = branch, U = upper immediate (lui jaisa). MIPS mein ye sab I-format mein the.

### x0

RISC-V ka \$zero: hamesha 0.

**Kaise kaam karta hai:**

1. Base RV32I chuno (about 40 instructions): har chip mein.
2. Zaroorat ke hisaab se letters jodo: microcontroller = RV32IMC; laptop-class = RV64IMAFDC + V.
3. Formats 6 hain (MIPS ke 3 ki jagah) kyunki store, branch aur upper-immediate ko alag format diye gaye taaki **register fields hamesha same bit positions** pe rahein (rd hamesha bits 11-7, rs1 bits 19-15, rs2 bits 24-20). Isse hardware register file ko instruction poora decode hone se pehle hi padh sakta hai.
4. Field order bhi alag: RISC-V R-type = funct7 (7) | rs2 (5) | rs1 (5) | funct3 (3) | rd (5) | opcode (7). Opcode neeche (bits 6-0) aur 7 bits ka; MIPS mein opcode upar (31-26) aur 6 bits.
5. Immediate: RISC-V I-type mein 12 bits; MIPS I-type mein 16 bits.



Pick your letters, get your ISA: RV32IMAFD = base + multiply + atomic + float + double.

### RV32I base plus standard extensions

How to draw: A central circle RV32I with six satellite circles M, A, F, D, C, V, each labelled with its purpose.

**Diagram kaise padhein:** Beech mein bada gola RV32I (base, about 40 instructions). Chaaron taraf chhe chhote gole: M (multiply/divide), A (atomic), F (single float), D (double float), C (compressed 16-bit), V (vector). Har gola base se line se juda hai: "pick your letters, get your ISA". Neeche: RV32IMAFD = base + M + A + F + D.

| Point               | MIPS32                     | RISC-V (RV32)             |
|---------------------|----------------------------|---------------------------|
| Registers           | 32 (\$0 to \$31)           | 32 (x0 to x31)            |
| Zero register       | \$zero (\$0)               | x0                        |
| Instruction width   | fixed 32-bit               | 32-bit (+16-bit with C)   |
| Formats             | 3: R, I, J                 | 6: R, I, S, B, U, J       |
| Opcode field        | 6 bits, bits 31-26         | 7 bits, bits 6-0          |
| Immediate in I-type | 16 bits                    | 12 bits                   |
| ISA model           | one fixed set              | modular base + extensions |
| Licensing           | proprietary (MIPS company) | open, royalty-free        |

### Solved example (write the answer like this, 4 marks):

*Explain RISC-V's modular ISA and compare it with MIPS32 (five points).*

1. Modular: mandatory base RV32I (about 40 integer instructions, x0 to x31, x0 = 0) plus optional extensions M, A, F, D, C, V; a chip implements only what it needs, e.g. RV32IMAFD.
2. Draw D11.6a.
3. Comparison: both 32 registers with a hardwired zero; both fixed 32-bit load/store RISC; MIPS has 3 formats, RISC-V 6 (R, I, S, B, U, J) with register fields in fixed positions; RISC-V adds 16-bit compressed instructions (C); MIPS proprietary, RISC-V open and royalty-free.

**Answer:** base + extensions model, and the five-row comparison table.

**Galti mat karna:** RISC-V ke 3 formats nahi, 6. Har RISC-V chip floating point nahi karti (sirf F/D ke saath). RISC-V proprietary nahi hai.

**Ek line mein:** RISC-V = MIPS jaisa RISC, par open aur modular: base RV32I + M A F D C V, aur 6 formats jisme register fields hamesha same jagah.

## 11.E1 Extra (Lab 11): MIPS assembly toolkit

**Pehle socho:** Instructions to aa gaye, par asli program chalane ke liye aur cheezein chahiye: data kahan rakhein (directives), screen pe kaise print karein (syscall), aur kuch shortcuts (pseudo-instructions) jo assembler khud asli instructions mein badal de. Ye lab ka "toolkit" hai. **syscall** hi bahar ki duniya (keyboard, screen) ki khidki hai.

**Definition:** An **assembler directive** (.data, .text, .word, .ascii, .space, .globl) tells the assembler how to lay out the program and data; it is not executed. A **pseudo-instruction** (li, la, move, blt, bge, nop) is a convenience instruction that the assembler expands into one or more real instructions, using \$at if needed. A **syscall** requests an operating-system service selected by the code in \$v0, with arguments in \$a0 and \$a1.

**Mushkil shabd, aasaan matlab:**

**.data / .text**

.data ke neeche variables (0x10010000 se), .text ke neeche code (0x00400000 se).

**.word / .ascii / .space n**

.word 10 = ek 4-byte number. .ascii "Hi" = string + end mein 0 byte. .space 400 = 400 bytes khaali jagah (100 words).

**.globl main**

main label ko bahar (simulator) ko dikhana, taaki program wahan se shuru ho.

**la vs lw**

la \$t0, x = x ka **address** lao. lw \$t1, x = x ki **value** lao.

**HI / LO**

mult ka 64-bit result: upar ke 32 bits HI mein, neeche ke LO mein. div: **LO = quotient (bhaagfal)**, **HI = remainder (shesh)**.

**sl / srl / sra**

Shift left logical ( $\times 2^n$ ), shift right logical (left se 0 bhara), shift right arithmetic (left se sign bit copy karo, negative number negative hi rahe).

## Alignment

lw/sw ka address 4 ka multiple hona chahiye (0, 4, 8). 2 pe lw = error.

## Endianness

4-byte word ke bytes memory mein kis order mein. Big-endian: bada (most significant) byte pehle.

Little-endian: chhota byte pehle.

### Kaise kaam karta hai (syscall ka 3-step pattern):

1. `li $v0, code`: kaunsi service chahiye.
2. Argument set karo: `$a0` mein number (print int) ya string ka address (print string).
3. `syscall` likho. Read int ke baad answer `$v0` mein aata hai, turant move karke kahin aur rakh lo (agla `syscall` `$v0` badal dega).

| \$v0 | Service                | Argument                                  | Result                         |
|------|------------------------|-------------------------------------------|--------------------------------|
| 1    | print integer          | <code>\$a0 = value</code>                 | none                           |
| 4    | print string           | <code>\$a0 = address of string</code>     | none                           |
| 5    | read integer           | none                                      | <code>\$v0 = value read</code> |
| 8    | read string            | <code>\$a0 = buffer, \$a1 = length</code> | none                           |
| 9    | allocate memory (sbrk) | <code>\$a0 = bytes</code>                 | <code>\$v0 = address</code>    |
| 10   | exit                   | none                                      | none                           |
| 11   | print character        | <code>\$a0 = ASCII code</code>            | none                           |

| Pseudo-instruction               | Real instruction(s) it becomes                                 |
|----------------------------------|----------------------------------------------------------------|
| <code>li \$t0, 7</code>          | <code>addiu \$t0, \$zero, 7 (= 0x24080007)</code>              |
| <code>li \$t0, 0x12345678</code> | <code>lui \$t0, 0x1234 ; ori \$t0, \$t0, 0x5678</code>         |
| <code>la \$t0, label</code>      | <code>lui \$at, upper half ; ori \$t0, \$at, lower half</code> |
| <code>move \$t0, \$t1</code>     | <code>addu \$t0, \$zero, \$t1</code>                           |
| <code>blt \$a, \$b, L</code>     | <code>slt \$at, \$a, \$b ; bne \$at, \$zero, L</code>          |
| <code>bge \$a, \$b, L</code>     | <code>slt \$at, \$a, \$b ; beq \$at, \$zero, L</code>          |
| <code>bgt \$a, \$b, L</code>     | <code>slt \$at, \$b, \$a ; bne \$at, \$zero, L</code>          |
| <code>ble \$a, \$b, L</code>     | <code>slt \$at, \$b, \$a ; beq \$at, \$zero, L</code>          |
| <code>b L</code>                 | <code>beq \$zero, \$zero, L</code>                             |
| <code>nop</code>                 | <code>sll \$zero, \$zero, 0 (word 0x00000000)</code>           |

```

# Lab 11 Program A: do number jodo aur print karo
.data
msg:      .asciiz "Sum = "      # string, end mein 0 byte
.text
.globl main
main:
    li    $t0, 7                # a = 7 (pseudo: addiu $t0,$zero,7)
    li    $t1, 5                # b = 5
    add   $t2, $t0, $t1         # sum = a + b = 12
    li    $v0, 4                # service 4 = print string
    la    $a0, msg              # $a0 = msg ka ADDRESS
    syscall                      # screen: "Sum = "
    li    $v0, 1                # service 1 = print integer
    move  $a0, $t2              # $a0 = 12 (VALUE)
    syscall                      # screen: "12"
    li    $v0, 10               # service 10 = exit
    syscall                     # program khatam

```

Output: Sum = 12. (add ki jagah sub → 2; mult + mflo → 35.)

### Solved example 1 (HI/LO, Lab question):

After `div $t0, $t1` with  $\$t0 = 17$  and  $\$t1 = 5$ , which instruction gets the remainder, and what are HI and LO? What about  $-17 \div 5$ ?

1.  $17 \div 5$ :  $5 \times 3 = 15$ ,  $17 - 15 = 2$ . Quotient 3, remainder 2.
2. Rule: LO = quotient = 3, HI = remainder = 2.
3. Remainder chahiye → mfhi (HI se nikaalo).
4.  $-17 \div 5$ : MIPS zero ki taraf kaatta hai → quotient = -3; remainder =  $-17 - (-3 \times 5) = -17 + 15 = -2$ .

**Answer: mfhi; LO = 3, HI = 2. For  $-17 \div 5$ : LO = -3, HI = -2.**

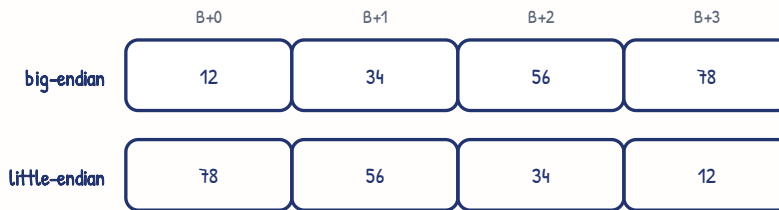
### Solved example 2 (shifts, logic, mult):

(a)  $\$t0 = -16$ . Find `sll`, `srl`, `sra` by 2. (b) `0xCA AND/OR/XOR 0x0F`. (c)  $100000 \times 100000$ : HI and LO?

1. (a)  $-16 = 0xFFFFFFF0$ . `sll 2` →  $\times 4 = -64$ . `srl 2` → left se 0 aate hain →  $0x3FFFFFFC = 1073741820$  (bada positive). `sra 2` → sign bit (1) copy →  $-16 / 4 = -4$ .
2. (b) `0xCA = 1100 1010`, `0x0F = 0000 1111`. `AND` = `0000 1010` = `0x0A`. `OR` = `1100 1111` = `0xCF`. `XOR` = `1100 0101` = `0xC5`. `NOR` = `NOT(OR)` → low byte `0011 0000` = `0x30` (32 bits mein `0xFFFFF30`).
3. (c)  $100000 \times 100000 = 10^{10} = 0x2540BE400$  (34 bits, 32 mein fit nahi). Upar wala hissa HI = `0x2` = 2; neeche ke 32 bits LO = `0x540BE400` = `1410065408`.

**Answer: (a) -64, 1073741820, -4; (b) 0x0A, 0xCF, 0xC5; (c) HI = 2, LO = 1410065408.**

Word 0x12345678 stored at address B



lb \$t1, 0(\$t0) gives 0x12 on big-endian, 0x78 on little-endian. Network order = big; x86 = little; MIPS is bi-endian.

Alignment: lw/sw need an address that is a multiple of 4 (lw \$t1, 2(\$t0) → address error); lb/sb have no restriction.

### Endianness and alignment

How to draw: Two rows of four byte boxes B+0..B+3: big-endian 12 34 56 78, little-endian 78 56 34 12.

**Diagram kaise padhein:** Word 0x12345678 address B pe. Upar ki row big-endian: B+0 = 12, B+1 = 34, B+2 = 56, B+3 = 78 (bada byte pehle). Neeche little-endian: B+0 = 78, B+1 = 56, B+2 = 34, B+3 = 12 (chhota byte pehle). lb \$t1, 0(\$t0) big pe 0x12, little pe 0x78 deta hai. Neeche alignment rule: lw/sw ko 4 ka multiple address chahiye.

### Solved example 3 (bug hunt, Lab 11):

*x is .word 42. A student writes la \$a0, x then print-integer and gets a large number like 268500992. Why? Fix it.*

1. la x ka **address** deta hai (0x10010000 = 268500992), value nahi.
2. Value 42 chahiye → lw \$a0, x.
3. Saath mein li \$v0, 1 zaroor ho, warna syscall ko pata nahi kaunsi service.

**Answer:** replace la with lw \$a0, x (la = address, lw = value).

**Galti mat karna:** (1) syscall se pehle li \$v0, n bhoolna. (2) Doosra integer padhne se pehle pehla \$v0 se copy na karna. (3) div ke baad mflo/mfhi ulta: **mflo = quotient, mfhi = remainder**. (4) lw \$t1, 2(\$t0) = unaligned (2 multiple of 4 nahi). (5) Encoding question mein blt jaise pseudo ko asli instruction maan lena.

**Trick:** syscall codes: "1 print int, 4 print string, 5 read int, 10 exit" = "1-4 likho, 5 padho, 10 pe bye". div: "LO = Lena (quotient), HI = Hisaab baaki (remainder)".

**Ek line mein:** Directives layout batate hain, pseudo-instructions assembler ke shortcuts hain, syscall = li \$v0 → \$a0 → syscall; div mein LO quotient, HI remainder; la address, lw value.

### Exam mein is unit se kya aata hai → kaise solve karein

| Question type                              | Method (steps)                                                                                                                                                            |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Register number / match register to number | Ranges: 0 zero, 1 at, 2-3 v, 4-7 a, 8-15 t0-t7, 16-23 s, 24-25 t8-t9, 26-27 k, 28 gp, 29 sp, 30 fp, 31 ra. Naam ke number ko range ke start se jodo (\$s3 = 16 + 3 = 19). |

|                                                       |                                                                                                                                                                                                                                                     |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Caller-saved vs callee-saved (MCQ / 5 marks)          | 1. \$t (and \$a, \$v) caller-saved: callee badal sakta hai. 2. \$s (and \$sp, \$ra) callee-saved: callee restore kare. 3. Example: main ka sum \$s0 mein safe; \$t0 main khud save kare. 4. Register table draw karo.                               |
| Encode R-type to hex                                  | 1. Register numbers. 2. Order rs, rt, rd (assembly ka pehla = rd). 3. opcode 000000, shamt 0 (shift ke alawa), funct table se. 4. 6-5-5-5-6 binary. 5. 4-4 group → hex. 6. Ulta decode karke check.                                                 |
| Encode I-type (addi / lw / sw)                        | 1. opcode table se. 2. rs = source/base, rt = destination (addi, lw) ya data (sw). 3. imm 16-bit; negative ho to invert + 1. 4. 6-5-5-16 → hex.                                                                                                     |
| Branch offset (beq/bne immediate)                     | 1. PC + 4 nikaalo. 2. target - (PC + 4) bytes mein. 3. ÷ 4 = words. 4. Negative ho to 16-bit two's complement. 5. Check: PC + 4 + imm × 4 = target.                                                                                                 |
| J-type encode / jump target                           | 1. target ÷ 4 (>> 2). 2. Neeche ke 26 bits. 3. opcode j = 000010, jal = 000011. 4. Hex. Target banana: PC+4[31:28]    addr    00; reach = 256 MB.                                                                                                   |
| Decode hex → assembly                                 | 1. Hex → 32 bits. 2. Pehle 6 bits = opcode: 0 → R (funct dekho), 2/3 → J, baaki I. 3. Format ke hisaab se fields todo. 4. Numbers → register naam. 5. Assembly order mein likho (R: rd, rs, rt; lw: rt, imm(rs)). 6. imm ka bit 15 = 1 to negative. |
| Sign-extended immediate value (e.g. 0xFFFFC)          | 1. Bit 15 = 1 → negative. 2. Invert + 1 → magnitude. 3. Minus lagao (0xFFFFC → -4, 0xFFFFB → -5).                                                                                                                                                   |
| Match instruction to format                           | R: add, sub, and, or, slt, sll, jr. I: addi, lw, sw, beq, bne, andi, lui. J: j, jal.                                                                                                                                                                |
| Field widths / why 32 bits                            | R 6-5-5-5-5-6, I 6-5-5-16, J 6-26. 5-bit register fields kyunki 32 = 2 <sup>5</sup> registers. Fixed 32 → PC + 4, simple decode.                                                                                                                    |
| RISC-V vs MIPS                                        | Base RV32I + M A F D C V; 6 formats; open/royalty-free; dono 32 registers + zero register; comparison table.                                                                                                                                        |
| Lab: syscall / pseudo / HI-LO / la vs lw / endianness | Syscall table (1, 4, 5, 10, 11); pseudo expansion table; div: LO quotient, HI remainder; la address vs lw value; 0x12345678 little-endian B+0 = 78.                                                                                                 |

## Unit 12 · C to MIPS Translation

**Is unit ki kahani:** Unit 11 mein humne MIPS ke "shabd" seekhe: registers aur instructions (add, lw, beq, j, jal). Ab un shabdon se "vaakya" banayenge: C ka if, while, for, arrays aur functions ko haath se MIPS mein badalna, bilkul waise jaise compiler karta hai. Sabse mushkil hissa functions hain: arguments kaise bhejin, jawab kaise lein, aur recursive function mein **stack** kaise use karein. Unit 13 mein dekhenge ki ye instructions hardware (datapath) mein kaise chalte hain.

### 12.1 Recap of formats aur translation pattern

**Pehle socho:** C code ek building ka naksha hai jisme kamre (if-block, loop-block) saaf dikhte hain. Assembly ek seedhi lambi line hai, jisme kamron ki jagah **labels** (naam ke board) hain aur darwaazon ki jagah **branches/jumps**. Translate karna matlab: **variables ko registers mein bithao, conditions ko compare + branch banao, aur blocks ko labels se jodo.**

**Definition: Translating C to MIPS** means mapping each variable to a register, each expression to arithmetic instructions, each condition to a compare-and-branch (slt, beq, bne), and each control structure (if, loop, function) to labelled blocks connected by branches and jumps. Array elements are reached by  $\text{address} = \text{base} + \text{index} \times 4$  for word arrays.

**Mushkil shabd, aasaan matlab:**

**Recap: R / I / J**

R = sab registers (add, sub, slt). I = 16-bit number wale (addi, lw, sw, beq, bne). J = jumps (j, jal).  
Har instruction 32 bits.

**Label**

Code mein ek naam + colon, jaise Loop:. Ye us instruction ka address hai; branch/jump yahan aa sakte hain.

**Base address**

Array ka pehla element A[0] jis memory address pe hai. Usually ek \$s register mein.

**Offset**

Base se kitne bytes aage. A[i] ka offset =  $4 \times i$  (har int 4 bytes).

**Byte addressing**

Memory mein har byte ka alag address. Isliye 4-byte word ke liye agla word 4 address aage.

**Translation ke 5 rules:**

1. **Variables → registers:** har C variable ko ek \$s ya \$t register do (question mein diya hota hai, jaise "f in \$s0").

- Expressions → arithmetic:** ek baar mein ek operation; beech ke results \$t mein.
- Conditions → compare + branch:** == / != ke liye beq/bne; < > ke liye pehle slt, phir beq/bne \$zero ke saath.
- Reverse the condition:** "agar cond, to body" ko likho "agar NOT cond, to body skip karo".
- Arrays:** A[k] (k constant) → lw \$t0, 4k(\$base). A[i] (i variable) → pehle 4i banao (sll by 2), base mein jodo, phir lw \$t0, 0(\$t1) (\$t1 = base + 4i).

#### Solved example 1 (expression):

$f = (g + h) - (i + j)$ ; with  $f, g, h, i, j$  in \$s0 to \$s4.

- Pehle  $g + h$ : add \$t0, \$s1, \$s2.
- Phir  $i + j$ : add \$t1, \$s3, \$s4.
- Dono ka difference  $f$  mein: sub \$s0, \$t0, \$t1.

**Answer:** add \$t0, \$s1, \$s2 ; add \$t1, \$s3, \$s4 ; sub \$s0, \$t0, \$t1.

#### Solved example 2 (array, constant index):

$A[12] = h + A[8]$ ; with  $h$  in \$s2 and the base of int array  $A$  in \$s3.

- $A[8]$  ka offset =  $8 \times 4 = 32$  bytes → lw \$t0, 32(\$s3).
- Jodo: add \$t0, \$s2, \$t0 ( $h + A[8]$ ).
- $A[12]$  ka offset =  $12 \times 4 = 48$  → sw \$t0, 48(\$s3).

**Answer:** lw \$t0, 32(\$s3) ; add \$t0, \$s2, \$t0 ; sw \$t0, 48(\$s3).

#### Solved example 3 (array, variable index):

$g = A[i]$ ; with  $g$  in \$s1,  $i$  in \$s4, base of  $A$  in \$s3. If  $A$  starts at  $0x10010000$  and  $i = 5$ , which address is read?

```
sll $t1, $s4, 2    # t1 = i * 4 (2 baar left shift = 4 se guna)
add $t1, $t1, $s3  # t1 = base + 4i = &A[i] (A[i] ka address)
lw $s1, 0($t1)     # g = A[i]
```

- sll by 2 kyun: 1 baar left shift =  $\times 2$ , 2 baar =  $\times 2 \times 2 = \times 4$ .  $i = 5 = 101 \rightarrow 10100 = 20$ .
- Address =  $0x10010000 + 20 = 0x10010000 + 0x14 = 0x10010014$ .

**Answer:** offset 20 bytes, address  $0x10010014$ .

**Galti mat karna:** Index ko 4 se guna karna mat bhoolna. lw \$t0, 8(\$s3)  $A[8]$  nahi,  $A[2]$  hai ( $8 / 4 = 2$ ). Aur lw \$t0, \$s4(\$s3) jaisa kuch MIPS mein hota hi nahi; offset hamesha constant, isliye variable index ke liye address khud banao.

**Trick:** "Index  $\times 4 =$  offset; sll 2 =  $\times 4$ ". Variable index ka 3-line mantra: sll, add, lw.

**Ek line mein:** Variables → registers, conditions → slt + beq/bne (ulti condition pe skip), blocks → labels, A[i] ka address = base + 4i.

## 12.2 If/else aur conditional branching

**Pehle socho:** Railway track pe points (switch) hote hain: train ya to seedhi jaati hai ya doosri line pe mud jaati hai. MIPS mein bhi CPU seedha agla instruction chalata hai, jab tak koi **branch** use dusri line (label) pe na mode. Twist ye hai ki MIPS ke paas sirf do asli conditional points hain: beq (barabar ho to modo) aur bne (barabar na ho to modo). Isliye if (a == b) ke liye bne use hota hai: "barabar NAHI hai to body ke upar se kood jao".

**Definition:** A **conditional branch** (beq, bne) changes the PC to a label only if its condition is true; otherwise execution falls through to the next instruction. An **if** statement is translated by branching past the body on the **opposite (reversed) condition**; an **if/else** uses one conditional branch to the else-block and an unconditional j at the end of the if-block to skip the else-block. Ordering tests (<, >) use **slt** to set a flag that is then tested against \$zero.

**Mushkil shabd, aasaan matlab:**

**beq a, b, L**

Branch if equal: a == b ho to L pe jao, warna agli line.

**bne a, b, L**

Branch if not equal: a != b ho to L pe jao.

**slt d, a, b**

Set on less than: a < b ho to d = 1, warna d = 0. Jaise a = 2, b = 7 → d = 1; a = 7, b = 2 → d = 0.

**Flag**

Ek register jisme sirf 0 ya 1 hai (haan/na ka jawab).

**Fall through**

Branch nahi liya to CPU apne aap agli line pe chala jaata hai.

**Reversed condition**

Ulti condition. == ka ulta !=, < ka ulta >=.

**blt / bge / bgt / ble**

Pseudo-instructions (asli nahi): assembler inhe slt + beq/bne mein badal deta hai (\$at use karke).

**Kaise kaam karta hai (if/else ko translate karne ke steps):**

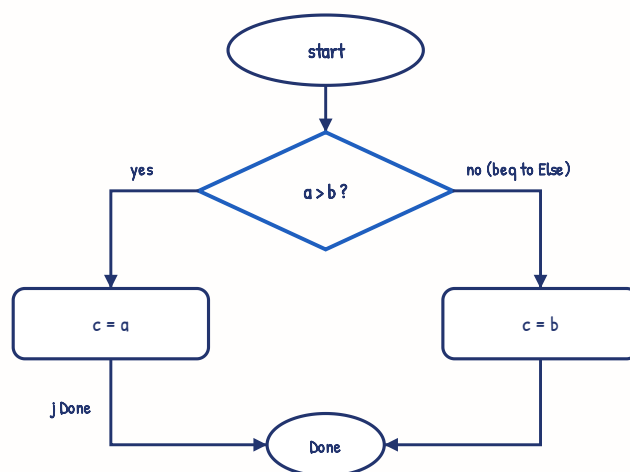
1. C condition likho: a == b.
2. Uta karo: a != b. Kyun? Hum chahte hain condition sahi ho to CPU *seedha* neeche body mein chale (fall through), aur galat ho to body ke upar se kood jaaye.

3. Ulti condition ka branch Else label pe: `bne $t0, $t1, Else`.
4. Uske neeche if-body likho.
5. If-body ke end mein `j Done`, warna CPU girte-girte else-body bhi chala dega.
6. Else: label ke neeche else-body.
7. Done: label: dono raste yahan milte hain.
8. `< / >` ke liye: pehle `slt` se flag, phir flag ko `$zero` se compare.  $a > b = b < a$ , to `slt` ke operands ulte: `slt $t3, $t1, $t0`.

```
# if (a == b) c = 1; else c = 0;      a = $t0, b = $t1, c = $t2
    bne $t0, $t1, Else             # ULTI condition: a != b ho to Else pe kood
    li  $t2, 1                     # if-body: c = 1 (yahan sirf a == b pe aate hain)
    j   Done                       # ZAROORI: else-body ko skip karo
Else: li  $t2, 0                   # else-body: c = 0
Done:                               # dono raste yahan milte hain
```

**Trace:**  $a = 4, b = 4 \rightarrow$  bne nahi liya (barabar hain)  $\rightarrow c = 1 \rightarrow j$  Done.  $a = 4, b = 5 \rightarrow$  bne liya  $\rightarrow$  Else  $\rightarrow c = 0$ .

| C condition (body kab chale) | MIPS: body SKIP karne ke liye                 | Kyun                                              |
|------------------------------|-----------------------------------------------|---------------------------------------------------|
| $a == b$                     | <code>bne a, b, Skip</code>                   | barabar nahi $\rightarrow$ skip                   |
| $a != b$                     | <code>beq a, b, Skip</code>                   | barabar $\rightarrow$ skip                        |
| $a < b$                      | <code>slt t, a, b; beq t, \$zero, Skip</code> | $t = 0$ matlab $a < b$ jhooth $\rightarrow$ skip  |
| $a >= b$                     | <code>slt t, a, b; bne t, \$zero, Skip</code> | $t = 1$ matlab $a < b$ sach $\rightarrow$ skip    |
| $a > b$                      | <code>slt t, b, a; beq t, \$zero, Skip</code> | $a > b = b < a$ (operands ulte)                   |
| $a <= b$                     | <code>slt t, b, a; bne t, \$zero, Skip</code> | $b < a$ sach $\rightarrow a > b \rightarrow$ skip |



```
slt $t3, $t1, $t0 # t3 = (b < a)
beq $t3, $zero, Else
move $t2, $t0 # c = a
j Done
Else: move $t2, $t1 # c = b
Done:
```

Every if/else has this shape: a test that splits into two blocks that merge again. Branches are the arrows.

### Max of two: flowchart and MIPS

How to draw: Start oval  $\rightarrow$  diamond " $a > b$ ?"  $\rightarrow$  yes box  $c = a$ , no box  $c = b \rightarrow$  both arrows merge at Done; write the matching MIPS beside it.

**Diagram kaise padhein:** Upar "start" oval, neeche diamond " $a > b$ ?". Yes wala arrow left box  $c = a$  mein, no wala (beq to Else) right box  $c = b$  mein. Dono boxes ke neeche arrows "Done" pe milte hain; left wale arrow pe "j Done" likha

hai, yahi woh jump hai jo else ko skip karata hai. Right mein matching MIPS code. Trace  $a = 9, b = 4$ : `slt $t3, $t1, $t0`  
 $= (4 < 9) = 1 \rightarrow$  beq nahi liya  $\rightarrow c = 9 \rightarrow j$  Done.

### Solved example 1 (slt idiom, easy):

Translate *if* ( $a < b$ )  $c = a + b$ ; ( $a = \$t0, b = \$t1, c = \$t2$ ). Trace for  $a = 2, b = 7$  and  $a = 7, b = 2$ .

```
slt  $t3, $t0, $t1    # t3 = 1 agar a < b, warna 0
beq  $t3, $zero, Skip # flag 0 (a >= b) → body skip
add  $t2, $t0, $t1    # body: c = a + b
Skip:                               # aage ka code
```

1.  $a = 2, b = 7$ :  $2 < 7 \rightarrow \$t3 = 1 \rightarrow$  beq nahi liya  $\rightarrow c = 9$ .
2.  $a = 7, b = 2$ :  $7 < 2$  jhooth  $\rightarrow \$t3 = 0 \rightarrow$  beq liya  $\rightarrow$  Skip,  $c$  nahi badla.

**Answer: `slt + beq` on `$zero`; body runs only when  $a < b$ .**

### Solved example 2 (exam favourite, 5 marks):

Translate ( $a$  in `$t0`,  $b$  in `$t1`,  $c$  in `$t2`): *if* ( $a > b$ )  $c = a$ ; *else*  $c = b$ ; Explain why `j Done` is needed.

```
slt  $t3, $t1, $t0    # t3 = (b < a) = (a > b) ; operands ulte!
beq  $t3, $zero, Else # a > b jhooth (t3 = 0) → Else pe jao
move $t2, $t0         # if-body: c = a
j    Done             # else-body skip karo
Else: move $t2, $t1    # else-body: c = b
Done:
```

1.  $a > b$  ke liye seedha `slt` nahi hai, to  $b < a$  likho: `slt $t3, $t1, $t0`.
2. Ulti condition (flag = 0) pe Else pe branch.
3. Trace  $a = 9, b = 4$ :  $t3 = 1 \rightarrow c = 9 \rightarrow j$  Done.  $a = 3, b = 8$ :  $t3 = 0 \rightarrow$  Else  $\rightarrow c = 8$ .  $a = b = 5$ :  $t3 = 0 \rightarrow c = 5$ .
4. `j Done` ke bina:  $a = 9, b = 4$  pe  $c = 9$  hota, phir girte hue Else wali line bhi chalti aur  $c = 4$  ho jaata (galat).

**Answer: code above; `j Done` stops the if-block from falling through into the else-block.**

### Solved example 3 (portal: even or odd):

Read an integer and print Even or Odd.

```

.data
even: .asciiz "Even"
odd: .asciiz "Odd"
.text
.globl main
main:
    li    $v0, 5          # service 5: integer padho → $v0
    syscall
    andi  $t0, $v0, 1      # sirf last bit rakho: 0 = even, 1 = odd
    bne   $t0, $zero, print_odd # last bit 1 → odd wale block pe
print_even:
    la    $a0, even        # "Even" string ka address
    li    $v0, 4           # service 4: print string
    syscall
    j     end              # odd wala block skip
print_odd:
    la    $a0, odd         # "Odd" string ka address
    li    $v0, 4
    syscall
end:
    li    $v0, 10          # exit
    syscall

```

1. Kyun andi 1: binary ka last bit 0 ho to number even (6 = 110), 1 ho to odd (7 = 111).
2. Input 7 → 7 AND 1 = 1 → bne liya → "Odd". Input 2 → 0 → fall through → "Even".

**Answer:** andi \$t0, \$v0, 1 then bne on \$zero. 2 → Even, 7 → Odd, -3 → Odd.

**Galti mat karna:** (1) j Done bhoolna → if-block else-block mein gir jaata hai. (2)  $a > b$  ke liye `slt $t3, $t0, $t1` likhna (ye  $a < b$  hai; operands ulto). (3) `==` ke liye `beq` likh ke body neeche rakhna: tab body ulti condition pe chalegi. (4) Encoding question mein `blt` ko asli instruction maanna.

**Trick: "Uلتa socho, kood jao":** C mein jo condition hai, uska ulta branch karo. **"Bada chahiye? slt ulta likho":**  $a > b = \text{slt } t, b, a$ .

**Ek line mein:** MIPS mein sirf `beq/bne` (+ `slt`) hain; `if` = ulti condition pe body skip, `if/else` = branch to Else + `if-body` ke end mein `j Done`.

## 12.3 Loops: while aur for

**Pehle socho:** Ek ghoomne wala darwaza socho. Andar aate hi guard puchhta hai "abhi aur chakkar lagana hai?" Haan → ek chakkar (body) lagao, counter badhao, wapas guard ke paas. Na → bahar nikal jao. Assembly loop bilkul aisa hai: **top pe test** (ulti condition pe bahar), **body**, **increment**, aur **j Loop se wapas top pe**.

**Definition:** A **loop** in assembly is a labelled block that tests its condition at the top (branching out to an End label on the reversed condition), executes the body, updates the loop variable and jumps back to the label with j. A **for** loop is a while loop with the initialisation placed before the label and the increment placed just before the jump back.

**Mushkil shabd, aasaan matlab:**

### Iteration

Loop ka ek chakkar.

### Init

Loop shuru hone se pehle variable set karna ( $i = 0$ ). Label se **pehle** likhte hain, taaki har chakkar mein dobara na ho.

### Loop variable / counter

Jo har chakkar mein badalta hai ( $i++$ ).

### Off-by-one

Loop ek baar zyada ya ek baar kam chalna. Sabse common bug.

### slt t, a, k

slt ka immediate version:  $a < k$  (constant) ho to  $t = 1$ .

### Pointer

Register jisme kisi element ka address hai; agle word ke liye +4.

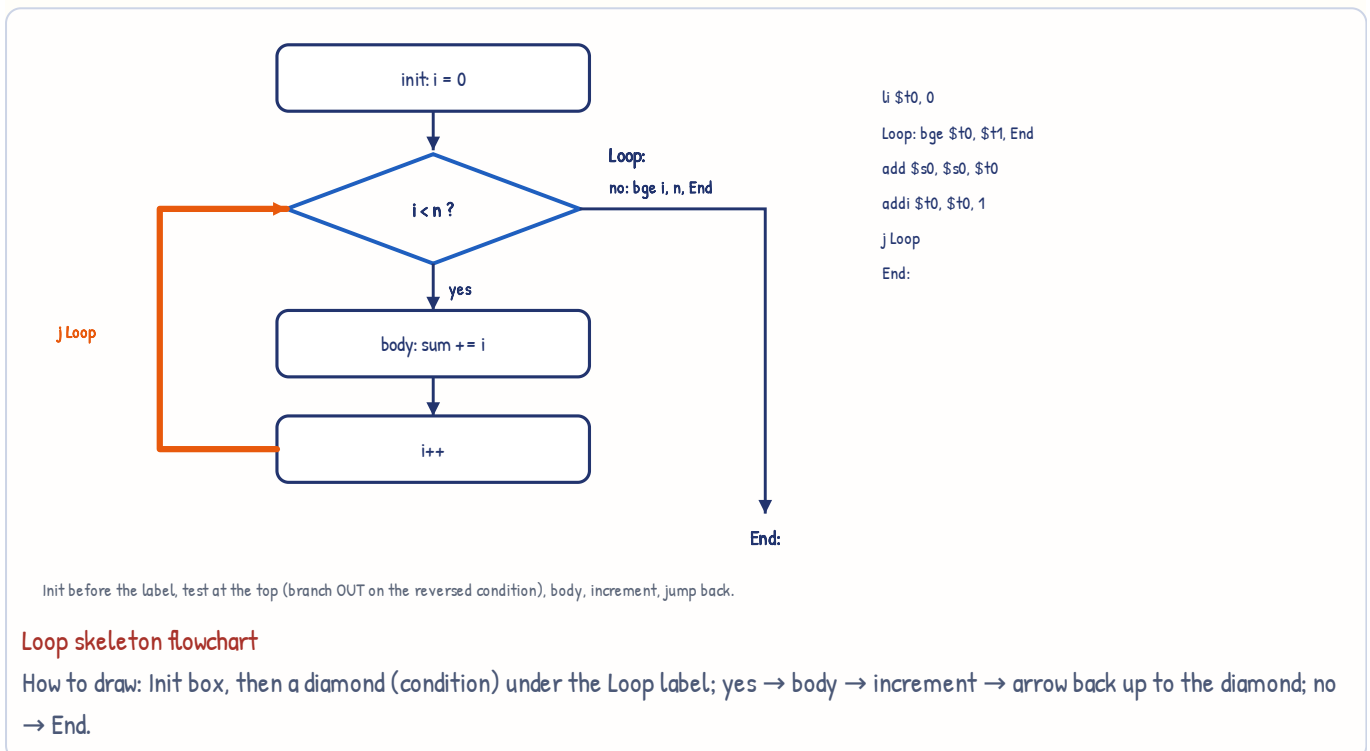
**Kaise kaam karta hai (loop skeleton, 5 hisse):**

1. **Init** (label se pehle): `li $t0, 0 ( $i = 0$ ).`
2. **Label + test**: Loop: phir ulta condition pe bahar: while ( $i < n$ ) ka ulta  $i \geq n \rightarrow$  `bge $t0, $t1, End.`
3. **Body**: asli kaam (`sum += i`).
4. **Increment**: `addi $t0, $t0, 1 ( $i++$ ).`
5. **Jump back**: `j Loop`; aur loop ke baad `End: label.`

```
# while (i < n) { sum += i; i++; }      i = $t0, n = $t1, sum = $s0
Loop: bge  $t0, $t1, End                # TEST: i >= n ho gaya to bahar (ulti condition)
      add  $s0, $s0, $t0                # BODY: sum = sum + i
      addi $t0, $t0, 1                  # INCREMENT: i = i + 1
      j    Loop                        # wapas test pe
End:                                     # loop ke baad yahan
```

| Chakkar | i (\$t0) | i >= n (n = 4)? | sum (\$s0) baad mein |
|---------|----------|-----------------|----------------------|
| 1       | 0        | no              | 0                    |
| 2       | 1        | no              | 1                    |
| 3       | 2        | no              | 3                    |
| 4       | 3        | no              | 6                    |
| test    | 4        | yes → End       | 6                    |

**Slide correction:** class slide ka test blt \$t1, \$t0, End (n < i pe bahar) hai; ye i == n pe bhi ek baar body chala deta hai (n = 4 pe sum = 10 aata, 6 nahi). Sahi: bge \$t0, \$t1, End ya slt \$t2, \$t0, \$t1; beq \$t2, \$zero, End.



**Diagram kaise padhein:** Upar "init: i = 0" box (label ke bahar). Neeche "Loop:" label pe diamond "i < n?". Yes → body box (sum += i) → i++ box → highlighted "j Loop" arrow wapas diamond tak. No → right side ka arrow "bge i, n, End" se End: tak. Right mein matching code. Trace n = 2: i = 0 yes, i = 1 yes, i = 2 no → End.

### Solved example 1 (for loop, class):

Translate for (i = 0; i < 10; i++) sum += i; (i = \$t0, sum = \$s0 = 0). How many times does the body run, and what is sum?

```

li    $t0, 0           # INIT: i = 0 (label se pehle, sirf ek baar)
Loop: slti $t2, $t0, 10 # t2 = 1 agar i < 10
      beq  $t2, $zero, End # i < 10 jhooth → bahar
      add  $s0, $s0, $t0   # BODY: sum += i
      addi $t0, $t0, 1     # INCREMENT: i++
      j    Loop           # wapas test pe
End:
  
```

1. Body i = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ke liye chalti hai = 10 baar.

2.  $\text{sum} = 0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$  (formula:  $9 \times 10 / 2 = 45$ ).

**Answer: body runs 10 times; sum = 45.**

### Solved example 2 (exam 5 marks: array sum):

Translate  $\text{sum} = 0$ ; for ( $i = 0$ ;  $i < n$ ;  $i++$ )  $\text{sum} = \text{sum} + A[i]$ ; (A base in  $\$s1$ ,  $n$  in  $\$s0$ ,  $\text{sum}$  in  $\$s2$ ). Explain each part.

```
li    $s2, 0          # sum = 0
li    $t0, 0          # INIT: i = 0
Loop: slt    $t1, $t0, $s0  # t1 = 1 agar i < n
      beq    $t1, $zero, End # TEST: i >= n → bahar
      sll    $t2, $t0, 2    # t2 = 4i (byte offset)
      add    $t2, $t2, $s1  # t2 = base + 4i = &A[i]
      lw     $t3, 0($t2)    # t3 = A[i]
      add    $s2, $s2, $t3  # BODY: sum += A[i]
      addi   $t0, $t0, 1    # INCREMENT: i++
      j      Loop          # wapas test pe
End:
```

1. Init: sum aur i dono 0, label se pehle.

2. Test: slt se flag ( $i < n$ ), flag 0 → End (ulti condition).

3. Address: sll 2 ( $\times 4$ ) → base mein jodo → lw. Ye har chakkar mein  $A[i]$  ka naya address banata hai.

4. Increment + j Loop.

5. Trace  $A = \{10, 20, 30, 40, 50\}$ ,  $n = 5$ :  $\text{sum} = 10 \rightarrow 30 \rightarrow 60 \rightarrow 100 \rightarrow 150$ .

**Answer: code above; for  $A = \{10, 20, 30, 40, 50\}$ , sum = 150.**

### Solved example 3 (mock paper, 6 marks: loop + if inside):

A base in  $\$s0$ ,  $n$  in  $\$s1$ ,  $\text{max}$  in  $\$s2$ ,  $n \geq 1$ :  $\text{max} = A[0]$ ; for ( $i = 1$ ;  $i < n$ ;  $i++$ ) if ( $A[i] > \text{max}$ )  $\text{max} = A[i]$ ;

```
lw     $s2, 0($s0)    # max = A[0] (offset 0)
li     $t0, 1          # i = 1
Loop:  slt    $t1, $t0, $s1  # i < n ?
      beq    $t1, $zero, End # nahi → loop khatam
      sll    $t2, $t0, 2    # 4i
      add    $t2, $t2, $s0  # &A[i]
      lw     $t3, 0($t2)    # t3 = A[i]
      slt    $t4, $s2, $t3  # t4 = (max < A[i]) = (A[i] > max)
      beq    $t4, $zero, Skip # ULTA: A[i] > max nahi → update skip
      move   $s2, $t3       # max = A[i]
Skip:  addi   $t0, $t0, 1    # i++
      j      Loop
End:
```

1. " $A[i] > \text{max}$ " = " $\text{max} < A[i]$ " → slt  $\$t4, \$s2, \$t3$  (operands ulte).

2. Reversed: flag 0 hone pe Skip pe kood (update mat karo).

3. Trace  $A = \{-7, 3, -1, 2\}$ :  $\text{max} = -7 \rightarrow 3$  (update) →  $-1$  nahi →  $2$  nahi →  $3$ .

Answer: code above; for  $\{-7, 3, -1, 2\}$   $\max = 3$ .

#### Solved example 4 (portal: iterative factorial, counting down):

Compute  $5!$  with a loop that counts down.

```
li    $t0, 1          # result = 1
li    $t1, 5          # counter = 5
loop: beq $t1, $zero, done # counter 0 ho gaya → bahar
mul   $t0, $t0, $t1    # result = result * counter
addi  $t1, $t1, -1     # counter-- (MARS mein subi bhi chalta hai)
j     loop
done: move $a0, $t0     # print ke liye $a0 = result
```

1. result:  $1 \rightarrow 5 \rightarrow 20 \rightarrow 60 \rightarrow 120 \rightarrow 120$  (counter 5, 4, 3, 2, 1 ke baad 0).

Answer: 120.

**Galti mat karna:** (1) Init ko label ke *neeche* likhna → har chakkar mein  $i = 0$ , infinite loop. (2) Off-by-one exit (slide ka `blt $t1, $t0`). (3) Array pointer ko +1 karna; word ke liye **+4**. (4) Test sirf neeche rakhna (do-while) →  $n = 0$  pe bhi body ek baar chal jaati hai.

**Trick:** Loop ka mantra: "**Init, Label, Test, Body, Badhao, Wapas**" (I-L-T-B-B-W). Test hamesha ulti condition pe bahar.

**Ek line mein:** Loop = init (label se pehle) → Loop: ulti condition pe End → body → increment → j Loop;  
while ( $i < n$ ) ka exit = `bge $t0, $t1, End`.

## 12.4 Calling convention: $\$a$ , $\$v$ , $\$ra$

**Pehle socho:** Tum dost ko phone karke kehte ho "ye 2 numbers jod do" (arguments). Dost jodta hai aur jawab batata hai (return value), phir call khatam karke tum wahin se kaam continue karte ho jahan ruke the (return address). Agar har koi alag tareeke se numbers bheje to gadbad. Isliye **sab ek rule maante hain:** arguments  $\$a0$ – $\$a3$  mein, jawab  $\$v0$  mein, wapsi ka pata  $\$ra$  mein. Yahi calling convention hai.

**Definition:** The MIPS calling convention is the agreed set of rules for function calls: the caller places up to four arguments in  $\$a0$  to  $\$a3$  and calls with `jal`, which stores the return address ( $PC + 4$ ) in  $\$ra$  and jumps; the callee returns results in  $\$v0$  (and  $\$v1$ ) and returns with `jr $ra`.  $\$t$  registers may be overwritten by the callee (caller-saved);  $\$s$  registers must be preserved by the callee (callee-saved).

**Mushkil shabd, aasaan matlab:**

**jal Label (jump and link)**

Do kaam: (1)  $\$ra \leftarrow PC + 4$  (agle instruction ka address "link"), (2) Label pe jump.

## Jr \$ra (jump register)

\$ra mein jo address hai wahan jao. Function se wapas aane ka tareeka.

## Return address

Caller mein jal ke baad wala instruction. Wahin se caller continue karta hai.

## Caller / callee

Caller = bulaane wala (main). Callee = bulaya gaya (sum).

## Leaf function

Jo kisi aur function ko call nahi karta (12.5).

## Kaise kaam karta hai (6 steps):

1. Caller arguments rakhta hai: `li $a0, 10 (n = 10)`.
2. Caller jal `sum: $ra = jal ke baad wale instruction ka address, PC = sum`.
3. Callee \$a0 padhta hai aur kaam karta hai (zaroorat ho to \$s/\$ra stack pe save).
4. Callee jawab \$v0 mein rakhta hai.
5. Callee `jr $ra`: wapas caller mein, jal ke theek baad.
6. Caller \$v0 se result uthata hai: `move $a0, $v0`.



More than four arguments → extra ones go on the stack. \$t registers may be clobbered by the callee; \$s must be preserved.

## Caller / callee register flow

How to draw: Two boxes (caller, callee); arrows right labelled \$a0-\$a3 and jal; arrows left labelled \$v0 and jr \$ra; number the six steps.

**Diagram kaise padhein:** Left box CALLER, right box CALLEE, steps 1 se 6 numbered. Upar ke do arrows right ki taraf: "\$a0-\$a3 (arguments)" aur highlighted "jal: jump + link". Neeche ke do arrows left ki taraf: "\$v0-\$v1 (return values)" aur highlighted "jr \$ra: back to the instruction after jal". Neeche note: 4 se zyada arguments stack pe; \$t callee badal sakta hai, \$s nahi.

## Solved example 1 (class homework: function sum(n)):

Write `int sum(int n) { s = 0; for (i = 1; i <= n; i++) s += i; return s; }` and call it with `n = 10` from main.

```

.text
.globl main
main:
    li    $a0, 10           # argument n = 10, $a0 mein
    jal   sum               # $ra = agli line ka address, sum pe jao
    move  $a0, $v0          # wapas aaye: result $v0 mein hai
    li    $v0, 1            # print integer
    syscall                 # prints 55
    li    $v0, 10           # exit
    syscall

sum:
    li    $v0, 0            # s = 0 (seedha $v0 mein, yahi return hoga)
    li    $t0, 1            # i = 1
sum_loop:
    bgt   $t0, $a0, sum_done # i > n → loop khatam
    add   $v0, $v0, $t0      # s += i
    addi  $t0, $t0, 1        # i++
    j     sum_loop
sum_done:
    jr    $ra               # main mein jal ke baad wali line pe wapas

```

1.  $n = 10 \rightarrow 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10$ ; formula  $n(n+1)/2 = 10 \times 11 / 2 = 55$ .
2. sum leaf hai aur sirf \$t0, \$v0 use karta hai → koi save nahi.

**Answer: prints 55.**

### Solved example 2 (numerical):

*jal sum is at address 0x00400020. What is written into \$ra, and where does jr \$ra return?*

1. jal:  $\$ra \leftarrow PC + 4$ .
2.  $0x00400020 + 4 = 0x00400024$ .
3. Decimal:  $0x00400024 = 4 \times 16^5 + 2 \times 16 + 4 = 4194304 + 32 + 4 = 4194340$ .
4. jr \$ra wapas 0x00400024 pe, yaani jal ke baad wala instruction (move \$a0, \$v0).

**Answer: \$ra = 0x00400024 (4194340).**

**Galti mat karna:** (1) Result \$a0 mein return karna; return hamesha **\$v0**. (2) jal ko "\$ra stack pe push karta hai" samajhna; jal sirf register \$ra mein likhta hai, stack software ka kaam. (3) \$t registers ko call ke baad same samajhna. (4) jr \$ra ki jagah j sum\_done jaisa kuch likh dena.

**Trick:** "a = andar (arguments), v = vapas (values), ra = return address". jal = "jump + yaad rakho", jr \$ra = "yaad wali jagah lauto".

**Ek line mein:** Arguments \$a0-\$a3 mein, jal ( $\$ra \leftarrow PC + 4$ , jump), jawab \$v0 mein, jr \$ra se wapas; \$t callee bigaad sakta hai, \$s nahi.

## 12.5 Stack frames: leaf vs non-leaf

**Pehle socho:** Canteen mein thaaliyon ka dher (stack) socho: nayi thaali hamesha upar rakhte ho, aur uthate bhi upar se ho. Function calls bhi aise hi: main ne A ko bulaya, A ne B ko; B pehle khatam hoga, phir A, phir main. Problem: **\$ra sirf ek hai**. Agar A khud jaal B kare to \$ra mein B ka return address aa jaayega aur A ka apna wapsi ka pata (main mein) mit jaayega. Isliye A ko **apna \$ra pehle stack pe rakhna padta hai**. Har function ka stack pe ye apna hissa = **stack frame**.

**Definition:** A **stack frame** is the block of stack memory a function allocates on entry to save registers (\$ra, \$s registers, arguments it still needs) and local data, and frees before returning. The MIPS stack **grows downward**: \$sp is decreased to push and increased to pop. A **leaf function** calls no other function; a **non-leaf function** calls others (or itself) and must save \$ra (and any \$a/\$s values needed after the call) on the stack.

**Mushkil shabd, aasaan matlab:**

### Stack

Memory ka hissa (upar ke addresses,  $0x7FFFFFFC$  ke paas se) jo LIFO (last in, first out) chalta hai: aakhri rakha, pehle nikla.

### \$sp (stack pointer)

Stack ke top (aakhri pushed word) ka address.

### Grows down

Naya data **chhote** address pe. Push = `addi $sp, $sp, -8` (8 bytes = 2 words ki jagah), pop = `addi $sp, $sp, 8`.

### Push / pop

Push = \$sp ghatao + sw. Pop = lw + \$sp badhao.

### Prologue / epilogue

Function ki shuru ki lines (frame banao, save karo) aur aakhri lines (restore, frame hatao, jr \$ra).

### Leaf / non-leaf

Leaf = ped ki patti, aage koi shaakha nahi (koi call nahi). Non-leaf = shaakha, aage aur calls.

### Balanced stack

Jitna push kiya utna hi pop; jr \$ra ke time \$sp wahi jo function shuru hone pe tha.

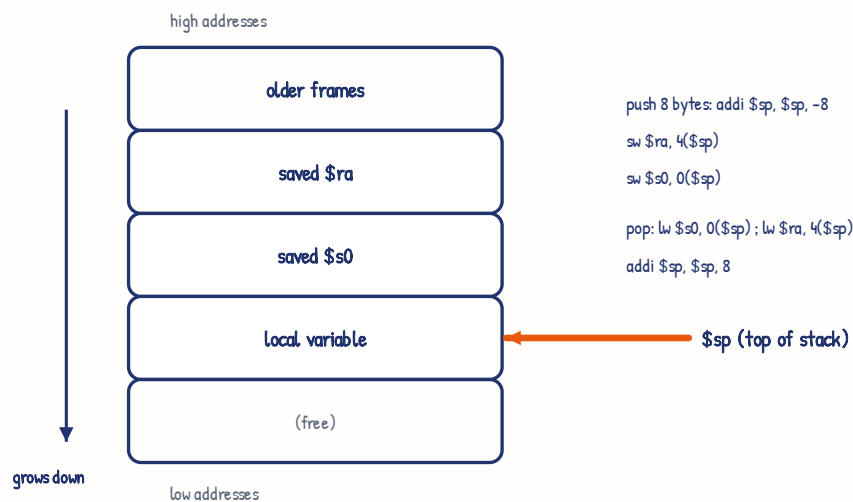
**Kaise kaam karta hai (5-part function template):**

1. **Prologue:** `addi $sp, $sp, -8` (jagah banao), `sw $ra, 4($sp)`, `sw $a0, 0($sp)` (jo baad mein chahiye woh save).
2. **Base case / kaam:** simple case ho to seedha jawab \$v0 mein.
3. **Call:** argument badlo aur jaal (ab \$ra overwrite hoga, par humne save kar liya hai).

4. **Combine:** stack se apna saved value wapas lo (`lw $a0, 0($sp)`) aur jawab banao.

5. **Epilogue:** `lw $ra, 4($sp), addi $sp, $sp, 8` (frame hatao), `jr $ra`.

6. **Leaf function** ko `$ra` save nahi karna (koi jal nahi). Agar sirf `$t/$a/$v` use kare to stack bilkul nahi; agar `$s` use kare to sirf `$s` save/restore.



Down to push, up to pop: `$sp` always points at the last word pushed; the stack must be balanced before `jr $ra`.

### The stack grows downward

How to draw: A tall rectangle: high addresses at the top (older frames), then saved `$ra`, saved `$s0`, local variable; arrow for `$sp` pointing at the lowest used word; write the push/pop code.

**Diagram kaise padhein:** Lamba dabba: upar high addresses (older frames), neeche low addresses. Upar se: older frames, saved `$ra`, saved `$s0`, local variable, phir (free). Left mein arrow "grows down". Right mein highlighted arrow "`$sp` (top of stack)" sabse neeche wale used word pe. Saath mein push code (`addi -8, sw $ra 4, sw $s0 0`) aur pop code (`lw, lw, addi +8`).

### Solved example 1 (leaf function that uses `$s0`, P&H):

```
int leaf(int g, int h, int i, int j) { int f = (g + h) - (i + j); return f; }
with f in $s0.
```

```
leaf:
    addi $sp, $sp, -4      # 1 word (4 bytes) ki jagah banao
    sw    $s0, 0($sp)      # $s0 callee-saved hai: purani value bachao
    add   $t0, $a0, $a1     # t0 = g + h
    add   $t1, $a2, $a3     # t1 = i + j
    sub   $s0, $t0, $t1     # f = (g + h) - (i + j)
    move  $v0, $s0         # return value $v0 mein
    lw    $s0, 0($sp)      # caller ka $s0 wapas daalo
    addi  $sp, $sp, 4       # jagah wapas (stack balanced)
    jr    $ra              # jal nahi kiya tha, to $ra abhi bhi sahi hai
```

1. `leaf(5, 6, 1, 2)`:  $(5 + 6) - (1 + 2) = 11 - 3 = 8$ .

2. `$ra` save nahi kiya kyunki andar koi jal nahi; `$s0` save kiya kyunki callee-saved hai.

**Answer: returns 8; only `$s0` is saved, `$ra` is not needed.**

### Solved example 2 (class: recursive factorial, non-leaf, full):

Write `int fact(int n) { if (n <= 1) return 1; return n * fact(n - 1); }` with main, and draw the stack during `fact(3)`.

```
.text
.globl main
main:
    li    $v0, 5          # n padho
    syscall
    move  $a0, $v0        # argument n → $a0
    jal   fact            # $ra = ret_main (agli line), fact pe jao
    move  $s0, $v0        # ret_main: result yahan aata hai
    move  $a0, $s0        # print ke liye
    li    $v0, 1          # print integer
    syscall
    li    $v0, 10         # exit
    syscall

fact:
    # 1. PROLOGUE: 8 bytes ka frame, $ra aur n save
    addi  $sp, $sp, -8    # stack neeche badhao (2 words)
    sw    $ra, 4($sp)     # apna wapsi ka pata bachao
    sw    $a0, 0($sp)     # n bachao (call ke baad chahiye)
    # 2. BASE CASE: n <= 1 → return 1
    li    $t0, 1
    ble   $a0, $t0, base_case
    # 3. RECURSIVE STEP: fact(n - 1)
    addi  $a0, $a0, -1    # argument = n - 1
    jal   fact            # $ra = ret_fact (agli line)
    # 4. COMBINE (ret_fact)
    lw    $a0, 0($sp)     # apna n stack se wapas ($a0 badal gaya tha)
    mul   $v0, $a0, $v0    # v0 = n * fact(n - 1)
    j     fact_exit
base_case:
    li    $v0, 1          # fact(1) = 1
fact_exit:
    # 5. EPILOGUE: restore, frame hatao, wapas
    lw    $ra, 4($sp)     # apna $ra wapas
    addi  $sp, $sp, 8     # frame hatao (stack balanced)
    jr    $ra             # caller ke paas lauto
```

**Stack har step pe** (shuru mein `$sp = 0x7FFF000`; `ret_main` = main mein jal ke baad wali line, `ret_fact` = fact mein jal ke baad wali line):

Step 1: main calls fact(3)  
 addr        value  
 0x7FFFEFFC \$ra = ret\_main  
 0x7FFFEFF8 n = 3    <- \$sp

Step 2: fact(3) calls fact(2)  
 addr        value  
 0x7FFFEFFC \$ra = ret\_main  
 0x7FFFEFF8 n = 3  
 0x7FFFEFF4 \$ra = ret\_fact  
 0x7FFFEFF0 n = 2    <- \$sp

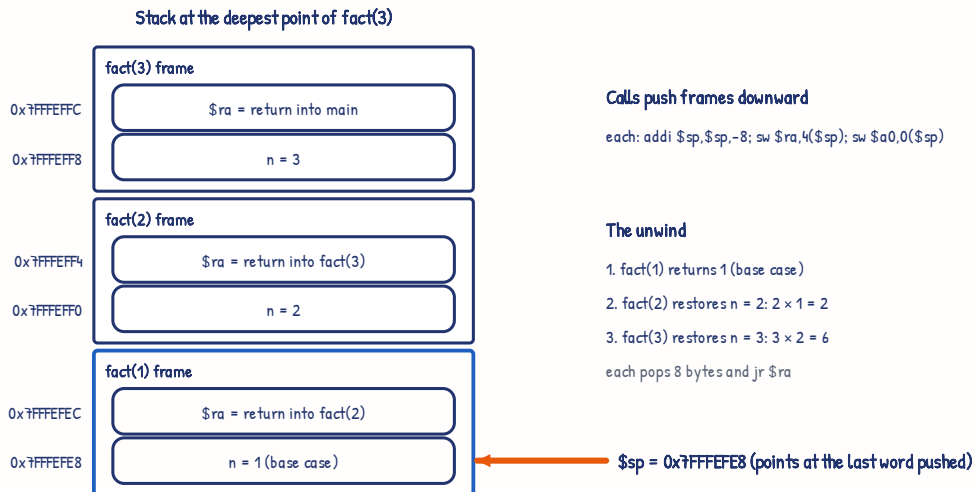
Step 3: fact(2) calls fact(1) (DEEPEST: 3 frames = 24 bytes)  
 0x7FFFEFFC \$ra = ret\_main | fact(3) ka frame  
 0x7FFFEFF8 n = 3 |  
 0x7FFFEFF4 \$ra = ret\_fact | fact(2) ka frame  
 0x7FFFEFF0 n = 2 |  
 0x7FFFEFEC \$ra = ret\_fact | fact(1) ka frame  
 0x7FFFEFE8 n = 1    <- \$sp |  
 fact(1): n <= 1 → \$v0 = 1, pop → \$sp = 0x7FFFEFF0, jr → fact(2)

Step 4: fact(2) combine  
 lw n = 2 (from 0x7FFFEFF0)  
 \$v0 = 2 \* 1 = 2  
 pop → \$sp = 0x7FFFEFF8  
 jr \$ra (ret\_fact) → fact(3)

Step 5: fact(3) combine  
 lw n = 3 (from 0x7FFFEFF8)  
 \$v0 = 3 \* 2 = 6  
 pop → \$sp = 0x7FFFF000  
 jr \$ra (ret\_main) → main, prints 6

1. Har call apna 8-byte frame banata hai: upar (offset 4) \$ra, neeche (offset 0) n.
2. fact(3) → fact(2) → fact(1): 3 frames, peak =  $3 \times 8 = 24$  bytes; \$sp sabse neeche 0x7FFFEFE8.
3. Unwind ulte order mein:  $1 \rightarrow 2 \times 1 = 2 \rightarrow 3 \times 2 = 6$ .
4. **\$ra kyun save:** andar wala jal fact \$ra ko ret\_fact se overwrite karta hai; save na karo to fact(3) kabhi main mein wapas nahi pahunchega.
5. **\$a0 (n) kyun save:** recursive call se pehle humne \$a0 = n - 1 kar diya; multiply ke liye asli n chahiye, jo sirf stack pe bacha hai.

**Answer: fact(3) = 6; peak 3 frames (24 bytes); \$ra and n are saved because jal and the argument change overwrite them.**



3 calls  $\rightarrow$  3 frames (24 bytes) at peak, then unwound one by one. Addresses assume  $\$sp = 0x7FFFF000$  before the first call.

### fact(3): frames at the deepest point and the unwind

How to draw: Three stacked frames (fact(3), fact(2), fact(1)), each holding saved \$ra and n; \$sp arrow at the bottom frame; list the three unwind steps.

**Diagram kaise padhein:** Left mein teen frames upar se neeche: fact(3), fact(2), fact(1) (sabse neeche highlighted, base case). Har frame mein do dabbe: saved \$ra (kahan lautna hai) aur n. Bagal mein addresses (0x7FFFEFFC se 0x7FFFEFE8). Neeche arrow: \$sp = 0x7FFFEFE8. Right mein "The unwind": fact(1) returns 1, fact(2) =  $2 \times 1 = 2$ , fact(3) =  $3 \times 2 = 6$ ; har ek 8 bytes pop karke jr \$ra.

### Solved example 3 (mock paper, 6 marks):

Write a recursive MIPS function for `int sum(int n) { if (n == 0) return 0; return n + sum(n - 1); }`. Show what is saved on the stack and why.

```
sum:  addi $sp, $sp, -8      # 8-byte frame
      sw  $ra, 4($sp)      # non-leaf: jal $ra ko overwrite karega
      sw  $a0, 0($sp)      # n call ke baad chahiye
      bne $a0, $zero, rec  # n != 0  $\rightarrow$  recursive case (n == 0 ka ulta)
      li  $v0, 0           # base case: sum(0) = 0
      j   done
rec:  addi $a0, $a0, -1     # argument = n - 1
      jal sum              # $v0 = sum(n - 1)
      lw  $a0, 0($sp)      # apna n wapas
      add $v0, $a0, $v0     # n + sum(n - 1)
done: lw  $ra, 4($sp)      # $ra wapas
      addi $sp, $sp, 8     # frame hatao
      jr  $ra
```

1. Saved: \$ra (kyunki andar jal hai) aur \$a0 = n (kyunki \$a0 ko  $n - 1$  bana diya, par add ke liye n chahiye).
2. Yahan base case ka bhi frame banta hai:  $\text{sum}(3) \rightarrow \text{sum}(2) \rightarrow \text{sum}(1) \rightarrow \text{sum}(0) = 4 \text{ frames} = 32 \text{ bytes peak}$ .
3. Unwind:  $0 \rightarrow 1 + 0 = 1 \rightarrow 2 + 1 = 3 \rightarrow 3 + 3 = 6$ .

**Answer:**  $\text{sum}(3) = 6$ ; each frame holds \$ra and n; peak 4 frames (32 bytes).

**Galti mat karna:** (1) Non-leaf mein \$ra save na karna → galat jagah return ya infinite loop. (2) Unbalanced stack: -8 push kiya, +4 pop kiya. (3) Call ke baad n ko \$a0 se padhna (woh badal chuka); stack se lw karo. (4) Push ke liye \$sp badhana; stack neeche badhta hai, push = **minus**. (5)  $\text{fact}(n)$  mein frames = n ( $\text{fact}(1)$  bhi frame banata hai):  $\text{fact}(4) = 4$  frames,  $\text{fact}(5) = 40$  bytes.

**Trick:** "Neeche jao to rakho (push = -), upar aao to uthao (pop = +)". Non-leaf ka niyam: "jal karega? to \$ra bachayega".

**Ek line mein:** Stack neeche badhta hai (\$sp - to push); leaf ko \$ra save nahi karna, non-leaf ko \$ra (aur call ke baad chahiye values) save karne hi padte hain;  $\text{fact}(3) = 3 \text{ frames} \times 8 = 24 \text{ bytes}$ , unwind 1 → 2 → 6.

**Exam mein is unit se kya aata hai → kaise solve karein**

| Question type                                        | Method (steps)                                                                                                                                                                                                                              |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Simple statement / expression to MIPS                | 1. Variables ko diye gaye registers. 2. Ek operation per instruction, beech ke results \$t mein. 3. Constant ke liye addi.                                                                                                                  |
| Array access $A[k]$ / $A[i]$                         | 1. Constant k: offset = $4k \rightarrow \text{lw/sw } 4k(\text{base})$ . 2. Variable i: $\text{sll } \$t, i, 2 \rightarrow \text{add base} \rightarrow \text{lw } 0(\$t)$ . 3. Address = base + $4i$ ( $A[5] \rightarrow +20$ ).            |
| if ( $a == b$ ) / first instruction MCQ              | Ulti condition: $\text{bne } a, b, \text{Skip}$ . ( $a != b \rightarrow \text{beq}$ .)                                                                                                                                                      |
| if / else translation (5 marks)                      | 1. Ulti condition pe branch to Else. 2. if-body. 3. j Done. 4. Else: else-body. 5. Done:. 6. Flowchart (D12.2a) + "j Done kyun" explain.                                                                                                    |
| $a < b, a > b, a \geq b, a \leq b$                   | $\text{slt}$ se flag. $a > b$ aur $a \leq b$ ke liye operands ulte ( $\text{slt } t, b, a$ ). Phir flag ko \$zero se $\text{beq/bne}$ (table 12.2).                                                                                         |
| Value of $\text{slt}$ / pseudo expansion             | $\text{slt } d, a, b: a < b \rightarrow 1 \text{ else } 0$ ( $\text{slt } \$t3, \$t1, \$t0$ with $\$t0 = 5, \$t1 = 9 \rightarrow 9 < 5? \rightarrow 0$ ). $\text{bge} = \text{slt} + \text{beq}$ ; $\text{blt} = \text{slt} + \text{bne}$ . |
| while / for loop translation                         | Init (label se pehle) → Loop: test ulte condition pe End → body → increment → j Loop → End:. while ( $i < n$ ) → $\text{bge } \$t0, \$t1, \text{End}$ .                                                                                     |
| Loop output / iteration count                        | Trace table banao: i, condition, variable. for ( $i = 0; i < 10; i++$ ) = 10 iterations, sum 45. Slide ka blt version ek extra iteration.                                                                                                   |
| Array loop (sum / max)                               | Loop skeleton + andar $\text{sll}, \text{add}, \text{lw}$ . Max ke liye andar $\text{slt max}, A[i] + \text{beq Skip}$ . Pointer method: +4 per element.                                                                                    |
| jal / jr / \$ra numericals                           | jal: $\$ra = \text{PC} + 4$ ( $0x00400020 \rightarrow 0x00400024 = 4194340$ ), phir jump. jr \$ra = wapas. Return value \$v0.                                                                                                               |
| Function with calling convention ( $\text{sum}(n)$ ) | main: \$a0 set, jal, move from \$v0. Callee: kaam, result \$v0, jr \$ra. Leaf + sirf \$t → stack nahi.                                                                                                                                      |

|                                             |                                                                                                                                                                                                                    |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Recursive function + stack frames (6 marks) | 1. Prologue: addi \$sp, -8; sw \$ra, 4; sw \$a0, 0. 2. Base case → \$v0. 3. \$a0 badlo, jal. 4. lw \$a0, 0(\$sp), combine. 5. lw \$ra; addi \$sp, +8; jr \$ra. 6. Frames draw karo + "\$ra aur n kyun save" likho. |
| Frames / bytes at peak                      | fact(n): n frames × 8 bytes (fact(3) = 24, fact(5) = 40). sum(n) with base n == 0: n + 1 frames.                                                                                                                   |
| Push / pop instruction MCQ                  | Push k words: addi \$sp, \$sp, -4k. Pop: addi \$sp, \$sp, +4k. Stack neeche badhta hai.                                                                                                                            |
| Leaf vs non-leaf                            | Leaf: koi call nahi → \$ra save nahi; \$s use kare to \$s save. Non-leaf: \$ra save zaroori + call ke baad chahiye values.                                                                                         |

## Unit 13 · The Single-Cycle Datapath

**Is unit ki kahani:** Units 1–12 mein humne parts banaye (MUX, adder, ALU, register file, memory) aur MIPS ki "language" seekhi (R / I / J formats, lw, sw, beq). Ab in parts ko wires se jod ke ek **chalta hua processor** banayenge jo har instruction ko **ek hi clock cycle** mein poora kar de. Exam ka sabse pakka sawaal yahin se aata hai: "Trace <instruction> and fill the control signals". Unit ke end mein dekhenge ki ye design slow kyun hai, aur wahi problem Unit 14 (pipelining) solve karta hai.

### 13.1 Recap: ISA → Hardware

**Pehle socho:** Restaurant ka **menu** batata hai kya order kar sakte ho (paneer tikka, dal, roti). **Kitchen** batata hai woh dish banti kaise hai (kaunsa chef, kaunsa tawa, kis order mein). Same menu ke liye alag-alag kitchen ho sakte hain: ek chhota dhaba, ek bada hotel. Computer mein **ISA = menu** (kaunse instructions hain: add, lw, sw, beq) aur **microarchitecture = kitchen** (hardware jo unhe chalata hai). Is unit mein hum kitchen banayenge.

**Definition:** The **ISA** (instruction set architecture) is the programmer-visible contract: the instructions, registers and formats. The **microarchitecture** is the hardware implementation of the ISA and consists of the **datapath** (the units that store, move and transform data) and the **control unit** (which decodes the opcode and tells the datapath what to do). In a **single-cycle** implementation every instruction completes in exactly one clock cycle ( $CPI = 1$ ).

**Mushkil shabd, aasaan matlab:**

#### **datapath**

Woh saare hardware blocks jinse data guzarta hai: PC, memories, register file, ALU, MUX, adders. Kitchen ke tawe, chulhe, bartan.

#### **control unit**

Chef ka "head". Instruction ka opcode padh ke batata hai: is baar kaunsa MUX kaunsa input le, register mein likhna hai ya nahi, memory padhni hai ya nahi.

#### **clock cycle / clock period ( $T_c$ )**

Clock ki ek "tick" se agli tick tak ka time. Jaise 800 ps (picosecond,  $1 \text{ ps} = 10^{-12} \text{ s}$ ).

#### **CPI**

Cycles Per Instruction: ek instruction ko kitne clock cycles lagte hain. Single-cycle mein har instruction = 1 cycle, to  $CPI = 1$ .

#### **subset**

Hum poora MIPS nahi banate, sirf ye instructions: **add, sub, and, or, slt** (R-type), **lw, sw** (memory), **beq** (branch), aur extension mein **addi, j**.

### Kaise kaam karta hai (ISA se hardware tak ka plan):

1. ISA ki list dekho: kaunse instructions chalane hain (upar wala subset).
2. Har instruction ko kya chahiye, likho. Sabko instruction memory se fetch chahiye. Zyada-tar ko registers padhne hain. Sabko ALU chahiye (add, address, compare). Sirf lw / sw ko data memory chahiye. R-type, lw, addi ko register mein likhna hai.
3. Har zaroorat ke liye ek hardware block rakho (13.3a mein list).
4. Jahan ek input pe do alag source aa sakte hain, wahan ek MUX lagao (13.3b).
5. Control unit banao jo opcode dekh ke saare MUX aur enable signals set kare (13.3c).
6. Single-cycle mein ye sab ek hi lambe clock cycle mein ho jaata hai. Cycle ke end wale clock edge pe PC, register file aur data memory update hote hain.

Teen design possible hain same ISA ke liye: **single-cycle** (ek lamba cycle per instruction,  $CPI = 1$ , slow clock), **multicycle** (har instruction kai chhote cycles), aur **pipelined** (kai instructions ek saath alag stages mein, Units 14-15).

**Formula (performance):** Execution time = Instruction count  $\times$  CPI  $\times T_c$

Single-cycle:  $CPI = 1$ , lekin  $T_c$  = sabse slow instruction (lw) ka time. Example:  $10^6$  instructions  $\times 1 \times 800$  ps =  $800 \times 10^6$  ps = 0.8 ms.

### Solved example (write the answer like this):

*Differentiate ISA and microarchitecture. What are the characteristics of a single-cycle implementation? (3 marks)*

1. **ISA** is the contract between software and hardware: instruction formats, registers, opcodes (the "what"). **Microarchitecture** is how a particular processor implements it: datapath + control (the "how"). One ISA (MIPS) can have many microarchitectures.
2. **Single-cycle characteristics:** (a) each instruction finishes in one clock cycle, so  $CPI = 1$ ; (b) the clock period must fit the slowest instruction (lw), so the clock is slow; (c) each hardware unit can be used only once per cycle, so we need separate instruction and data memories and separate adders (PC + 4, branch target) besides the ALU.

**Answer:** ISA = what (contract); microarchitecture = how (datapath + control). Single-cycle:  $CPI = 1$ , period = slowest instruction, units duplicated.

**Galti mat karna:** "Single-cycle = ek cycle = fast" sochna galat hai.  $CPI = 1$  hai, lekin woh ek cycle **bahut lamba** hai (lw jitna). Speed =  $CPI \times T_c$  dono se decide hoti hai.

**Ek line mein:** ISA = menu, microarchitecture = kitchen (datapath + control); single-cycle mein har instruction ek lambe cycle mein, CPI = 1.

## 13.2 Fetch-Decode-Execute Cycle

**Pehle socho:** Tum recipe book se khana bana rahe ho. Har step ke liye: (1) bookmark wale page se agla step **padho**, bookmark aage khiskao; (2) samjho kya karna hai, kaunsa masala chahiye; (3) **karo**; (4) zaroorat ho to fridge (storage) se cheez lo ya rakho; (5) result plate mein **rakh do**. Phir agla step. CPU bhi har instruction ke saath yahi 5 kaam karta hai, forever. Bookmark ka naam hai **PC**.

**Definition:** The **instruction cycle** is the sequence a CPU repeats for every instruction: **Fetch** the instruction from memory at address PC and set  $PC \leftarrow PC + 4$ ; **Decode** it and read the source registers; **Execute** the operation in the ALU; access data **Memory** if the instruction is a load or store; and **Write back** the result to a register.

**Mushkil shabd, aasaan matlab:**

### PC (Program Counter)

32-bit register jisme **agle instruction ka address** hota hai. Bookmark. Jaise  $PC = 0x00400000$  matlab agla instruction memory address  $0x00400000$  pe pada hai.

### IR (Instruction Register)

Fetch kiya hua 32-bit instruction yahan rakha jaata hai (single-cycle mein ye seedha instruction memory ka output hai).

### PC + 4

Har MIPS instruction 32 bit = **4 bytes** ka hai, aur memory ka har address ek byte ko point karta hai. Isliye agla instruction 4 address aage hai.  $PC = 0x0040000C \rightarrow PC + 4 = 0x00400010$  (hex:  $C + 4 = 12 + 4 = 16 = 0x10$ , to  $0C \rightarrow 10$ ).

### decode

Instruction ke bits ko todna: opcode [31-26] dekh ke pata chalta hai kaunsa instruction hai, rs / rt fields se registers padhe jaate hain.

### write-back

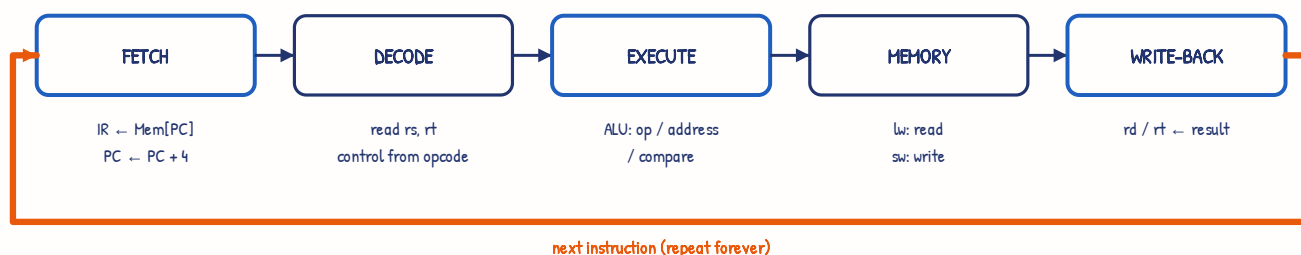
Result ko destination register mein likhna.

**Kaise kaam karta hai (ek instruction ke 5 steps):**

1. **F (Fetch):** PC jo address bata raha hai, instruction memory se us address ka 32-bit word padho. Saath hi  $PC \leftarrow PC + 4$  (agle instruction pe bookmark).
2. **D (Decode):** Opcode control unit ko jaata hai. rs aur rt registers register file se padhe jaate hain.

3. **E (Execute)**: ALU kaam karta hai: R-type ke liye operation (add, sub), lw / sw ke liye address = base + offset, beq ke liye compare (subtract karke dekho zero aaya ya nahi).
4. **M (Memory)**: Sirf lw (memory se padho) aur sw (memory mein likho). Baaki instructions ye step skip karte hain.
5. **WB (Write-back)**: R-type, lw, addi apna result register mein likhte hain. sw aur beq kuch nahi likhte.
6. Phir PC naya instruction point kar raha hai → wapas step 1. Single-cycle mein ye paanchon steps **ek hi clock cycle ke andar** ho jaate hain.

| Step      | add \$t2, \$t0, \$t1                                 | lw \$t0, 4(\$t1)              | sw \$t0, 4(\$t1)               | beq \$t0, \$t1, L                                                        |
|-----------|------------------------------------------------------|-------------------------------|--------------------------------|--------------------------------------------------------------------------|
| <b>F</b>  | $IR \leftarrow \text{Mem}[PC], PC \leftarrow PC + 4$ | same                          | same                           | same                                                                     |
| <b>D</b>  | read \$t0, \$t1                                      | read \$t1                     | read \$t1, \$t0                | read \$t0, \$t1                                                          |
| <b>E</b>  | ALU: $\$t0 + \$t1$                                   | ALU: $\$t1 + 4$ (address)     | ALU: $\$t1 + 4$ (address)      | ALU: $\$t0 - \$t1$ (compare); target = $PC + 4 + \text{offset} \times 4$ |
| <b>M</b>  | skip                                                 | read Mem[address]             | Mem[address] $\leftarrow$ \$t0 | skip                                                                     |
| <b>WB</b> | $\$t2 \leftarrow \text{sum}$                         | $\$t0 \leftarrow \text{data}$ | skip                           | skip (but may replace PC)                                                |



Single-cycle: all five steps happen within ONE long clock cycle. Pipelined (Unit 14): each step is its own stage.

#### Fetch-decode-execute-memory-write-back loop

How to draw: Five boxes in a row with arrows, and one arrow looping from write-back back to fetch; write what each step does under it.

**Diagram kaise padhein:** Left se right paanch boxes: FETCH → DECODE → EXECUTE → MEMORY → WRITE-BACK. Har box ke neeche likha hai us step mein kya hota hai. Neeche wala lamba arrow WRITE-BACK se wapas FETCH pe jaata hai: matlab ek instruction khatam, agla shuru. Trace lw ka: FETCH (instruction lao, PC + 4) → DECODE (\$t1 padho) → EXECUTE (\$t1 + 4) → MEMORY (us address se data) → WRITE-BACK (\$t0 mein data).

#### Solved example 1 (Lab question type):

*Which steps does add \$t2, \$t0, \$t1 use? Which instruction skips the write-back step?*

1. add ko instruction chahiye → **Fetch** haan.
2. \$t0, \$t1 padhne hain → **Decode** haan.
3. ALU mein jodna hai → **Execute** haan.
4. add memory ko chhoota hi nahi (MIPS mein sirf lw / sw memory use karte hain) → **Memory** nahi.

5. Result \$t2 mein jaata hai → **Write-back** haan.

6. sw memory mein likhta hai, kisi register mein nahi → sw write-back skip karta hai.

**Answer:** add uses F, D, E, WB (skips M). sw skips WB.

### Solved example 2:

An instruction is fetched from address 0x0040001C. What is the PC after the fetch step?

1. PC + 4 karna hai. Last hex digit: C = 12.  $12 + 4 = 16$ .

2. 16 hex mein = 0x10: last digit 0, aur 1 ka carry agle digit mein jaata hai.

3. Agla digit 1 tha →  $1 + 1 = 2$ .

4. Baaki digits same: 0x00400020.

**Answer:** PC = 0x00400020 (PC is incremented by 4 bytes).

**Galti mat karna:** (1) PC + 1 mat likho, MIPS mein PC + 4 hai (byte address, 4-byte instruction). (2) "add memory step use karta hai" galat hai. (3) "sw register mein likhta hai" galat hai: sw ka \$t0 sirf *padha* jaata hai.

**Trick:** add M skip, sw WB skip, beq dono skip. Sirf lw paanchon steps use karta hai, isliye wahi sabse lamba hai (13.5).

**Ek line mein:** Har instruction: Fetch ( $IR \leftarrow \text{Mem}[PC]$ ,  $PC \leftarrow PC + 4$ ) → Decode (registers padho) → Execute (ALU) → Memory (sirf lw / sw) → Write-back (R-type, lw, addi).

## 13.3a The Datapath Components (what each part does and why it is there)

**Pehle socho:** Kitchen setup karne se pehle list banate ho: fridge (ingredients), recipe stand, chopping board, gas, plates. Har cheez kisi zaroorat ki wajah se hai. Datapath ka bhi aisa hi ek "shopping list" hai. Har block ko "ye kya karta hai" aur "iske bina kaunsa instruction nahi chalega" ke saath yaad karo, to diagram khud ban jaayega.

**Definition:** The single-cycle datapath consists of **state elements** (PC, register file, instruction memory, data memory), which hold values, and **combinational elements** (ALU, adders, sign-extend unit, shift-left-2, multiplexers), which compute outputs from inputs. Each element is used at most once per clock cycle.

**Mushkil shabd, aasaan matlab:**

**state element**

Jo value **yaad rakhta** hai (clock edge pe likhta hai). PC, register file, data memory.

### combinational element

Jiska output sirf abhi ke inputs pe depend karta hai, kuch yaad nahi rakhta. ALU, adder, MUX.

### sign-extend

16-bit number ko 32-bit banana, **sign bit (bit 15) ko upar copy karke**.  $0x0004 (+4) \rightarrow 0x00000004$ .

$0xFFFFC (-4) \rightarrow 0xFFFFFFF C$  (abhi bhi  $-4$ ). Upar zero bhar dete to  $-4$  galti se  $+65532$  ban jaata.

### shift-left-2

Bits ko 2 jagah left khiskao, right mein do 0 daalo =  $\times 4$ .  $3 = 0011 \rightarrow 1100 = 12$ . Branch offset "kitne instructions" mein hota hai, address "bytes" mein chahiye, isliye  $\times 4$ .

### MUX (multiplexer)

Railway points: do (ya zyada) tracks mein se ek ko aage jaane deta hai. Select = 0  $\rightarrow$  input 0 pass, select = 1  $\rightarrow$  input 1 pass.

### Zero flag

ALU ka 1-bit output: result 0 hai to Zero = 1. beq ise use karta hai:  $\$t1 - \$t2 = 0$  matlab dono barabar.

### read port / write port

Register file ek saath 2 registers padh sakta hai (Read data 1, Read data 2) aur 1 register likh sakta hai (Write register + Write data, tabhi jab RegWrite = 1).

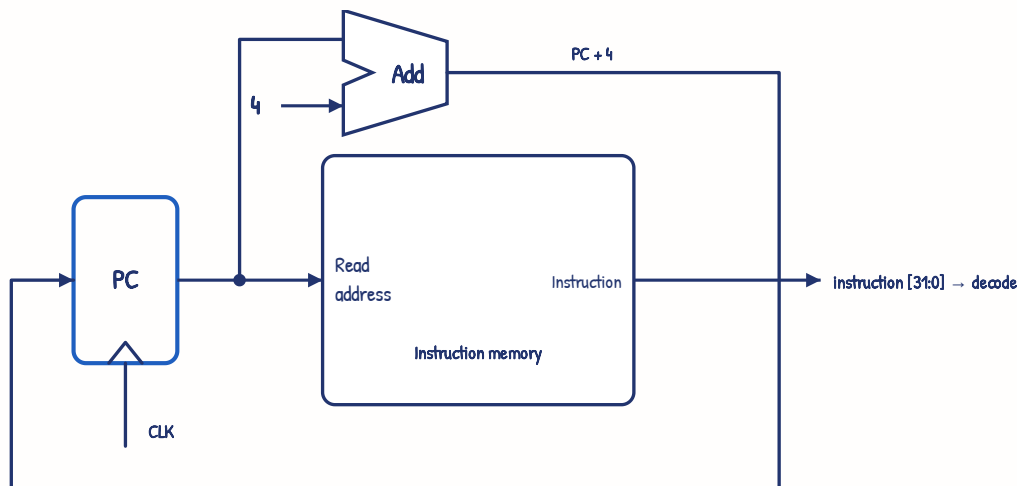
### Kaise kaam karta hai (har component, ek ek karke):

1. **PC:** 32-bit register, agle instruction ka address. Har clock edge pe naya value leta hai ( $PC + 4$ , ya branch / jump target). *Kyun:* bina PC ke pata hi nahi kahan se instruction lena hai.
2. **Instruction memory:** Input = address (PC), output = 32-bit instruction. Sirf padhi jaati hai. *Kyun:* program yahin rakha hai.
3. **PC + 4 adder:** Ek alag adder jiska ek input PC aur doosra constant 4. *Kyun alag:* ALU us cycle mein instruction ka kaam kar raha hai (single-cycle mein ek unit ek hi baar use ho sakti hai).
4. **Register file:** 32 registers. Read register 1  $\leftarrow$  bits [25-21] (rs), Read register 2  $\leftarrow$  bits [20-16] (rt), Write register  $\leftarrow$  rt ya rd (RegDst MUX se). Outputs Read data 1, Read data 2. *Kyun:* MIPS ke saare operands registers mein hote hain.
5. **Sign-extend:** bits [15-0] (immediate / offset) ko 32 bit banata hai. *Kyun:* ALU 32-bit hai; lw / sw ka offset, addi ka constant, beq ka offset 16-bit hote hain.
6. **ALU:** do 32-bit inputs, ek 4-bit "ALU control" input (kaunsa operation), outputs = result + Zero. *Kyun:* add / sub / and / or / slt, address calculation, beq ka compare, sab yahin.
7. **Data memory:** Address (ALU result se), Write data (Read data 2 se), Read data output; MemRead aur MemWrite signals. *Kyun:* lw / sw ke liye.
8. **Shift-left-2 + branch adder:** sign-extended offset  $\times 4$ , phir  $PC + 4$  mein jodo  $\rightarrow$  branch target. *Kyun alag adder:* ALU us waqt compare kar raha hai.

9. **MUXes (4 basic + 1 for jump):** RegDst, ALUSrc, MemtoReg, PCSrc (+ Jump). Har MUX wahan hai jahan ek input ke do possible source hain (13.3b mein detail).

10. **Control unit + ALU control:** opcode [31-26] → main control (saare signals); ALUOp + funct [5-0] → ALU control (4 bits).

| Bits    | Field              | Kahan jaata hai                               |
|---------|--------------------|-----------------------------------------------|
| [31-26] | opcode             | main control unit                             |
| [25-21] | rs                 | Read register 1                               |
| [20-16] | rt                 | Read register 2 <b>and</b> RegDst MUX input 0 |
| [15-11] | rd                 | RegDst MUX input 1                            |
| [15-0]  | immediate / offset | sign-extend                                   |
| [5-0]   | funct              | ALU control                                   |
| [25-0]  | jump address       | shift-left-2 → jump target                    |



Every instruction starts here: PC addresses the instruction memory; an adder computes PC + 4, written into the PC at the next edge.

#### Instruction fetch unit

How to draw: PC box feeding the instruction memory address; the same PC into an adder with constant 4; adder output looping back to the PC input.

**Diagram kaise padhein:** Left mein PC box. Uska output do jagah jaata hai: (1) seedha instruction memory ke "Read address" pe, (2) upar Add box mein jahan doosra input "4" hai. Add ka output (PC + 4) lamba wire banke wapas PC ke input pe aata hai: agle clock edge pe PC = PC + 4. Neeche CLK ka triangle batata hai ki PC sirf clock edge pe badalta hai. Trace: PC = 0x00400000 → instruction memory 0x00400000 wala word deta hai → adder 0x00400004 banata hai → edge pe PC = 0x00400004.

#### Solved example 1 (MCQ type):

*Which instruction bits drive Read register 1, and which drive Read register 2?*

1. R-format: op [31-26] | rs [25-21] | rt [20-16] | rd [15-11] | shamt [10-6] | funct [5-0].
2. rs hamesha padha jaata hai → Read register 1 ← [25-21].

3. *rt* bhi padha jaata hai (R-type ka doosra operand, *sw* ka data, *beq* ka doosra register) → Read register 2 ← [20-16].

4. *rd* sirf *likhne* ke liye hai, isliye woh RegDst MUX mein jaata hai, read port pe nahi.

**Answer:** Read register 1 ← [25-21] (*rs*); Read register 2 ← [20-16] (*rt*).

### Solved example 2 (write the answer like this):

*Why does the single-cycle datapath need separate instruction and data memories and extra adders?*

1. In a single-cycle design, every instruction finishes in one clock cycle, so each hardware unit can be used **only once** per cycle.
2. *lw* and *sw* need to fetch the instruction **and** access data in the same cycle, so one memory is not enough → separate instruction memory and data memory.
3. In the same cycle the ALU is busy with the instruction's own operation, so  $PC + 4$  and the branch target ( $PC + 4 + \text{offset} \times 4$ ) need their own adders.

**Answer:** one use per unit per cycle → two memories +  $PC + 4$  adder + branch adder.

**Galti mat karna:** (1) *rd* ko Read register 2 pe mat jodo, woh *rt* [20-16] hai. (2) Branch offset pe shift-left-2 bhoolna: offset instructions mein hai,  $\times 4$  karke bytes banao. (3) Sign-extend ko zero-extend mat samjho: upar sign bit copy hota hai.

**Ek line mein:**  $PC \rightarrow \text{instruction memory} \rightarrow (rs, rt) \text{ register file} + \text{sign-extend} \rightarrow \text{ALU} \rightarrow \text{data memory} \rightarrow \text{wapas register file}$ ; saath mein  $PC + 4$  adder aur branch adder (shift-left-2), aur jahan do source milte hain wahan MUX.

## 13.3b Building the Datapath Step by Step (why each MUX is added)

**Pehle socho:** Ek gaon mein pehle ek hi sadak hai. Jab doosre gaon se bhi sadak aa ke **usi pul** pe milni hai, to pul pe ek signal / gate lagana padta hai jo batata hai abhi kis taraf ki gaadi jaayegi. Datapath mein bhi: pehle sirf R-type ke liye wires banao. Jab *lw* aata hai aur kisi input pe **doosra source** chahiye hota hai, wahan ek MUX lagao. Har MUX ek "do raaste, ek pul" ki kahani hai.

**Definition:** The datapath is built incrementally: start with the fetch unit, add the parts needed for R-type, then extend it for *lw*, *sw*, *beq* and *j*. Whenever two different sources must drive the same input, a **multiplexer** is inserted and its select line becomes a control signal.

**Mushkil shabd, aasaan matlab:**

**select line**

MUX ka control input. 2:1 MUX mein 1 bit: 0 → upar wala (input 0), 1 → neeche wala (input 1).

### branch target

Jahan beq jump karega:  $PC + 4 + (\text{offset} \times 4)$ . Offset 16-bit signed hai (−32768 se +32767 instructions), isliye peeche (loop) aur aage dono ja sakte ho.

### PCSrc

PC ke aage wale MUX ka select. PCSrc = Branch AND Zero.

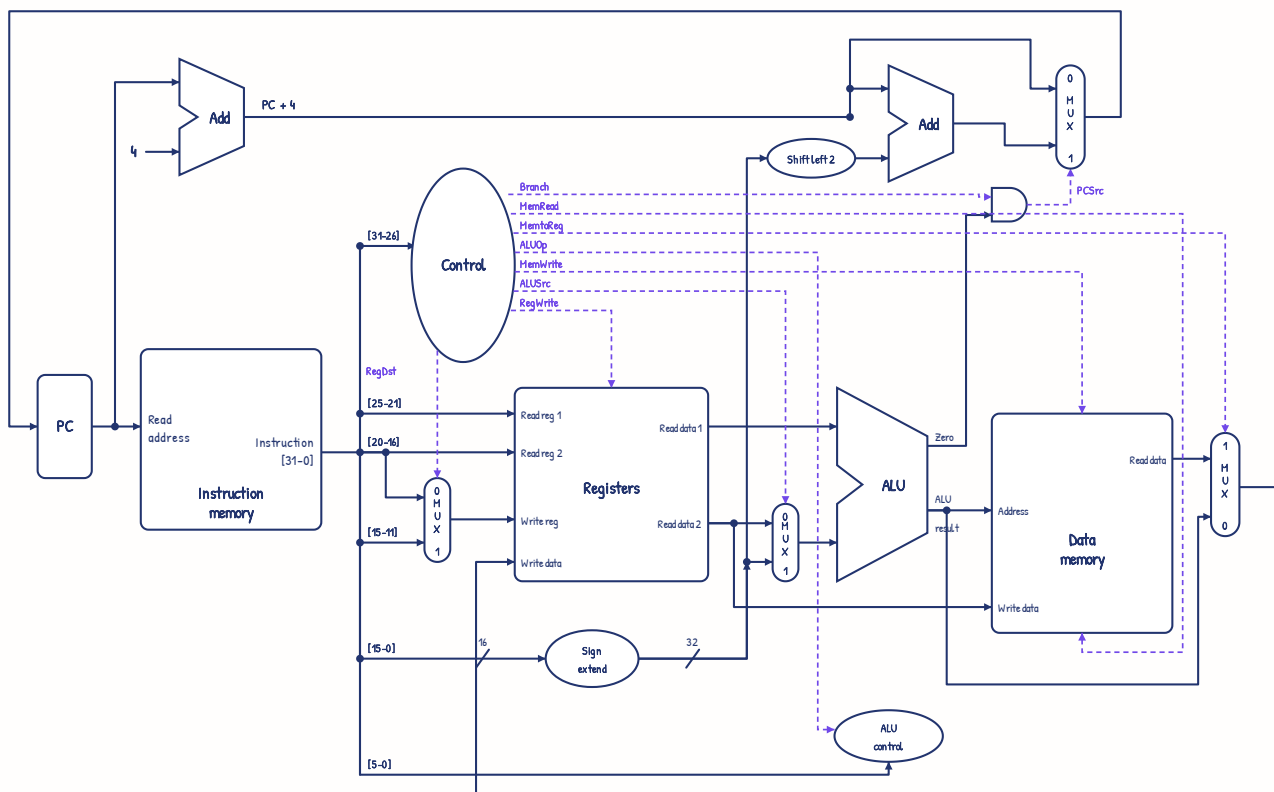
### jump target

j ke liye: (PC + 4) ke upar ke 4 bits [31:28] + 26-bit address × 4. Total 4 + 26 + 2 = 32 bits.

### Kaise kaam karta hai (6 building steps):

1. **Fetch:** PC → instruction memory; PC → adder (+4) → wapas PC. (Diagram D13.3b.)
2. **R-type (add \$t1, \$t2, \$t3):** rs, rt register file mein → Read data 1 aur Read data 2 → ALU ke dono inputs → ALU result → register file ka Write data; Write register = rd [15-11]. Abhi koi MUX nahi chahiye.
3. **lw (\$t2 ← Mem[\$t1 + 4]):** teen nayi problems aati hain.
  - (a) ALU ka doosra input ab register nahi, **offset** hai → sign-extend jodo, aur ALU ke doosre input pe ab do source (Read data 2 ya immediate) → **ALUSrc MUX**.
  - (b) Data memory jodo (address = ALU result). Register mein ab likhna hai memory ka data, R-type mein ALU result → Write data pe do source → **MemtoReg MUX**.
  - (c) lw ka destination **rt** [20-16] hai (l-format mein rd field hai hi nahi), R-type ka rd [15-11] → Write register pe do source → **RegDst MUX**.
4. **sw (Mem[\$t1 + 4] ← \$t2):** address lw jaisa hi. Store karne wala data \$t2 = rt, jo already Read data 2 pe aa raha hai → bas ek wire Read data 2 → data memory "Write data". **Koi naya MUX nahi.**
5. **beq (\$t1 == \$t2 ? jump):** ALU subtract karta hai, Zero flag nikalta hai. Target banane ke liye sign-extend → shift-left-2 → **branch adder** ( $PC + 4 + \text{offset} \times 4$ ). Ab agla PC do jagah se aa sakta hai (PC + 4 ya target) → **PCSrc MUX**, jiska select = Branch AND Zero (ek AND gate).
6. **j (extension):** target =  $PC + 4[31:28]$  joined with  $(\text{instr}[25-0] \times 4)$ . Agla PC ka teesra source → **Jump MUX** (PCSrc MUX ke baad), select = Jump.

| MUX      | Input 0                | Input 1                 | Kis instruction ki wajah se aaya   |
|----------|------------------------|-------------------------|------------------------------------|
| RegDst   | rt [20-16]             | rd [15-11]              | lw (destination rt) vs R-type (rd) |
| ALUSrc   | Read data 2 (register) | sign-extended immediate | lw / sw / addi need the offset     |
| MemtoReg | ALU result             | data memory Read data   | lw writes memory data              |
| PCSrc    | PC + 4                 | branch target           | beq (select = Branch · Zero)       |
| Jump     | output of PCSrc MUX    | jump target             | j                                  |



Single-cycle datapath with control (Patterson & Hennessy style) - dashed = control signals

### Single-cycle MIPS datapath with control (as printed on the class worksheets)

How to draw: Five big blocks left to right (PC, instruction memory, registers, ALU, data memory); add the PC + 4 adder and loop; insert RegDst, ALUSrc, MemtoReg and PCSrc MUXes; control ellipse above the register file with dashed lines to each MUX, memory and RegWrite; finally sign-extend, shift-left-2, branch adder and AND gate.

**Diagram kaise padhein:** Left se right paanch bade blocks: PC, Instruction memory, Registers, ALU, Data memory. Upar wala adder PC + 4 banata hai; uske right mein doosra adder (branch adder) jisme shift-left-2 ka output aata hai. Patle lambe boxes jin pe "MUX" likha hai, woh MUX hain (har input ke paas 0 / 1 likha hai): register file ke Write register ke aage RegDst, ALU ke neeche wale input pe ALUSrc, data memory ke right mein MemtoReg, top-right mein PCSrc. **Dashed lines** control signals hain jo beech ke "Control" oval se nikalte hain (opcode [31-26] usme jaata hai). ALU ke neeche "ALU control" box hai jisme funct [5-0] aur ALUOp jaate hain. Trace R-type: rs, rt → Read data 1, 2 → ALUSrc 0 → ALU → MemtoReg 0 → Write data; rd → RegDst 1 → Write register.

### Solved example 1 (exam favourite):

State the function of each multiplexer in the single-cycle datapath.

1. **RegDst:** chooses the destination register number: rt [20-16] (0) for lw / addi, rd [15-11] (1) for R-type.
2. **ALUSrc:** chooses the ALU's second operand: Read data 2 (0) for R-type / beq, sign-extended immediate (1) for lw / sw / addi.
3. **MemtoReg:** chooses the value written to the register: ALU result (0) or data read from memory (1, lw).
4. **PCSrc:** chooses the next PC: PC + 4 (0) or branch target (1); select = Branch AND Zero.

Answer: RegDst = which register, ALUSrc = which ALU operand, MemtoReg = which data to write back, PCSrc = which next PC. (Basic datapath has 4 MUXes; with jump, 5.)

### Solved example 2:

A j instruction is at address 0x00400018 and its 26-bit address field is 0x0100008. Find the jump target.

1.  $PC + 4 = 0x0040001C$ . Iske upar ke 4 bits [31:28] = hex ka pehla digit = 0  $\rightarrow$  0000.
2. Address field  $\times 4$  (shift-left-2):  $0x0100008 \times 4 = 0x0400020$  ( $8 \times 4 = 32 = 0x20$ ,  $1 \times 4 = 4$  at the  $0x100000$  place).
3. Jodo: 0000 (4 bits) +  $0x0400020$  ka 28-bit roop =  $0x00400020$ .

Answer: jump target =  $0x00400020$ .

**Galti mat karna:** PCSrc ko control unit ka seedha output mat bolo. Control sirf **Branch** deta hai; PCSrc = Branch AND Zero (ALU se). Branch = 1 lekin registers unequal  $\rightarrow$  Zero = 0  $\rightarrow$  PCSrc = 0  $\rightarrow$  PC + 4.

**Trick:** Har MUX ka naam batata hai woh kya chunta hai: **Reg-Dst** = register destination, **ALU-Src** = ALU source, **Mem-to-Reg** = memory to register (1 matlab haan, memory ka data). **Input 1** hamesha "naya / special" wala hai (rd, immediate, memory data, branch target).

**Ek line mein:** R-type se shuru, lw ne 3 MUX joda (ALUSrc, MemtoReg, RegDst), sw ne sirf ek wire, beq ne branch adder + PCSrc MUX, j ne Jump MUX.

## 13.3c Control Signals and the Main Control Table

**Pehle socho:** Railway station pe tracks (datapath) hamesha wahi rehte hain. **Station master** (control unit) har train (instruction) ke liye decide karta hai: kaunse points kis taraf karne hain (MUX selects) aur kaunse signal green karne hain (write enables). Train ka number (opcode) dekh ke woh ek fixed chart se settings uthata hai. Woh chart hi **control table** hai, aur exam mein yahi bharna hota hai.

**Definition:** The **main control unit** is a combinational circuit that takes the 6-bit opcode and produces the control signals RegDst, ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp (2 bits) and Jump. The **ALU control** unit uses ALUOp and the funct field to produce the 4-bit ALU operation (two-level decoding). A value **X (don't care)** means the signal may be 0 or 1 because its effect is not used by that instruction.

**Mushkil shabd, aasaan matlab:**

**control signal**

Ek wire jiska 0 / 1 datapath ko ek decision batata hai. Jaise MemWrite = 1 → "is cycle memory mein likho".

### asserted / deasserted

Asserted = 1 (ON), deasserted = 0 (OFF).

### write enable

RegWrite, MemWrite. Ye 1 ho tabhi clock edge pe likhai hoti hai. 0 → kuch nahi badlega, chahe input pe kuch bhi ho.

### X (don't care)

"Farak nahi padta". Jaise RegWrite = 0 hai to Write register kaunsa chuna (RegDst) koi farak nahi, kyunki likhna hi nahi hai. Designer X ko 0 ya 1 jo sasta ho rakh sakta hai.

### ALUOp

2-bit signal main control se: 00 = "add karo", 01 = "subtract karo", 10 = "funct field dekho".

### funct

R-type ke bits [5-0]: add 100000, sub 100010, and 100100, or 100101, slt 101010. Saare R-type ka opcode 000000 hai, funct batata hai asli operation.

### two-level decoding

Pehle main control opcode se ALUOp banata hai, phir ALU control (ALUOp + funct) se 4-bit operation. Main control chhota rehta hai.

### Har signal, simple words mein:

| Signal         | 0 matlab                                                                  | 1 matlab                                           |
|----------------|---------------------------------------------------------------------------|----------------------------------------------------|
| RegDst         | Write register = rt [20-16]                                               | Write register = rd [15-11]                        |
| ALUSrc         | ALU ka 2nd input = Read data 2 (register)                                 | ALU ka 2nd input = sign-extended immediate         |
| MemtoReg       | register mein ALU result likho                                            | register mein memory se aaya data likho            |
| RegWrite       | register file mein kuch mat likho                                         | clock edge pe Write register mein Write data likho |
| MemRead        | data memory mat padho                                                     | Address wale location ka data Read data pe do      |
| MemWrite       | memory mat badlo                                                          | Address pe Write data likho                        |
| Branch         | ye branch nahi hai                                                        | ye beq hai; Zero = 1 ho to PC = target             |
| ALUOp (2 bits) | 00 = add (lw / sw / addi), 01 = subtract (beq), 10 = funct dekho (R-type) |                                                    |
| Jump           | normal next PC                                                            | PC = jump target                                   |

### Full main control table (exam mein yahi bharna hai):

| Instr  | RegDst | ALUSrc | MemtoReg | RegWrite | MemRead | MemWrite | Branch | ALUOp | Jump |
|--------|--------|--------|----------|----------|---------|----------|--------|-------|------|
| R-type | 1      | 0      | 0        | 1        | 0       | 0        | 0      | 10    | 0    |
| lw     | 0      | 1      | 1        | 1        | 1       | 0        | 0      | 00    | 0    |
| sw     | X      | 1      | X        | 0        | 0       | 1        | 0      | 00    | 0    |
| beq    | X      | 0      | X        | 0        | 0       | 0        | 1      | 01    | 0    |
| addi   | 0      | 1      | 0        | 1        | 0       | 0        | 0      | 00    | 0    |
| j      | X      | X      | X        | 0        | 0       | 0        | X      | XX    | 1    |

### Har value ka WHY (row by row):

- R-type:** destination rd → RegDst 1. Dono operands registers → ALUSrc 0. Result ALU se → MemtoReg 0. Register likhna hai → RegWrite 1. Memory nahi → MemRead 0, MemWrite 0. Branch nahi → 0. Operation funct batayega → ALUOp 10.
- lw:** destination rt → RegDst 0. Address = rs + offset → ALUSrc 1. Register mein memory data → MemtoReg 1. RegWrite 1. Memory padhni hai → MemRead 1, MemWrite 0. Branch 0. Address ke liye add → ALUOp 00.
- sw:** koi register nahi likhna → RegWrite 0, isliye RegDst aur MemtoReg ka chunav bekaar → X. Address = rs + offset → ALUSrc 1. Memory mein likhna → MemWrite 1, MemRead 0. Branch 0. ALUOp 00 (add).
- beq:** register nahi likhna → RegWrite 0 → RegDst, MemtoReg = X. Compare do registers ka → ALUSrc 0. Memory nahi → 0, 0. Branch 1. Compare = subtract → ALUOp 01.
- addi:** destination rt → RegDst 0. Constant → ALUSrc 1. Result ALU se → MemtoReg 0. RegWrite 1. Memory 0, 0. Branch 0. Add → ALUOp 00.
- j:** sirf PC badalna hai. Jump MUX PCSrc wale raaste ko ignore karta hai, isliye Branch bhi X; ALU ka output kahin use nahi → ALUSrc, ALUOp X. Lekin RegWrite, MemRead, MemWrite = 0 pakka.

**X ka rule:** X sirf un signals pe jinka asar is instruction mein kahin nahi pahunchta. **Write enables** (RegWrite, MemWrite) kabhi X nahi: agar galti se 1 ho gaya to koi register ya memory word kharab ho jaayega. MemRead bhi 0 rakhte hain (bekaar read na ho). Branch sirf j mein X.

### ALU control (second level):

| ALUOp | funct  | Operation          | ALU control (4 bits) |
|-------|--------|--------------------|----------------------|
| 00    | XXXXXX | add (lw, sw, addi) | 0010                 |
| 01    | XXXXXX | subtract (beq)     | 0110                 |
| 10    | 100000 | add                | 0010                 |
| 10    | 100010 | sub                | 0110                 |
| 10    | 100100 | AND                | 0000                 |
| 10    | 100101 | OR                 | 0001                 |
| 10    | 101010 | slt                | 0111                 |

### Solved example 1 (mock question):

Fill the control signals (Branch, MemtoReg, MemRead, MemWrite, ALUOp, ALUSrc, RegWrite, RegDst) for `addi $t0, $t1, -4`.

1. `addi` I-format hai: `rs = $t1` padho, constant `-4` jodo, result `rt = $t0` mein.
2. Branch? Nahi → **Branch = 0**.
3. Register mein ALU ka result jaata hai, memory ka nahi → **MemtoReg = 0**.
4. Memory chhooni nahi → **MemRead = 0, MemWrite = 0**.
5. ALU ko add karna hai → **ALUOp = 00**.
6. Doosra operand constant (`0xFFFC` sign-extend → `0xFFFFFFC = -4`) → **ALUSrc = 1**.
7. `$t0` likhna hai → **RegWrite = 1**; `$t0` `rt` field mein hai → **RegDst = 0**.

**Answer: Branch 0, MemtoReg 0, MemRead 0, MemWrite 0, ALUOp 00, ALUSrc 1, RegWrite 1, RegDst 0.**

### Solved example 2:

`sub $s0, $s1, $s2`: give the ALUOp and the 4-bit ALU control input. Why does the main control not output the ALU operation directly?

1. `sub` R-type hai → main control **ALUOp = 10** ("funct dekho").
2. funct of `sub` = `100010` → table se ALU control = `0110` (subtract).
3. Saare R-type ka opcode same (`000000`) hai, isliye opcode se operation pata hi nahi chal sakta; funct chahiye. Two-level decoding se main control sirf opcode dekhta hai aur chhota rehta hai.

**Answer: ALUOp = 10, ALU control = 0110; R-type operation is in funct, not opcode.**

**Galti mat karna:** (1) ALUOp khaali chhodna (class ki har sample sheet mein chhoda gaya tha; free marks gaye). (2) `lw` ke liye **RegDst = 1** likhna: `lw` ka destination `rt` hai → 0. (3) R-type mein **MemRead = X** likhna: 0 likho. (4) **RegWrite** ya **MemWrite** pe X likhna: kabhi nahi.

**Trick:** Columns **RegDst ALUSrc MemtoReg RegWrite MemRead MemWrite Branch ALUOp** ke order mein: **R = 1001 000 10**, **lw = 0111 100 00**, **sw = X1X0 010 00**, **beq = X0X0 001 01**. ALUOp: "**lw/sw add, beq sub, R funct**" = 00, 01, 10.

**Ek line mein:** Opcode → control signals; MUX selects batate hain kaunsa raasta, **RegWrite / MemWrite** batate hain likhna hai ya nahi; X sirf **RegDst / MemtoReg** pe jab **RegWrite = 0** (aur j mein unused signals pe).

## 13.4 Tracing Different Instruction Types (add, lw, sw, beq)

**Pehle socho:** Courier ka parcel track karte ho: "Delhi hub → Jaipur hub → local office → ghar". Trace karna matlab yahi: instruction ka data **kis block se kis block tak jaata hai**, aur raaste mein har MUX pe kaunsa gate khula tha. Class ki chaaron worksheets yahi poochti hain: "Trace <instruction> and Fill the Control Signals".

**Definition:** Tracing an instruction on the datapath means highlighting the path its data takes from the PC to its destination (register, memory or PC), stating which MUX input is selected at each multiplexer, and writing the value of every control signal for that instruction.

**Mushkil shabd, aasaan matlab:**

### active path

Woh wires jinpe is instruction ka kaam ka data chal raha hai (diagram mein orange). Baaki wires pe bhi kuch value hoti hai, par use nahi hoti.

### Zero flag (reminder)

ALU result 0 → Zero = 1. beq mein  $\$t1 - \$t2 = 0$  → barabar.

### offset (beq)

"PC + 4 se kitne instructions aage".  $3 \rightarrow 3 \times 4 = 12$  bytes aage.

### base + offset

lw / sw ka address: register (base) + constant (offset).  $4(\$t1) = \$t1 + 4$ .

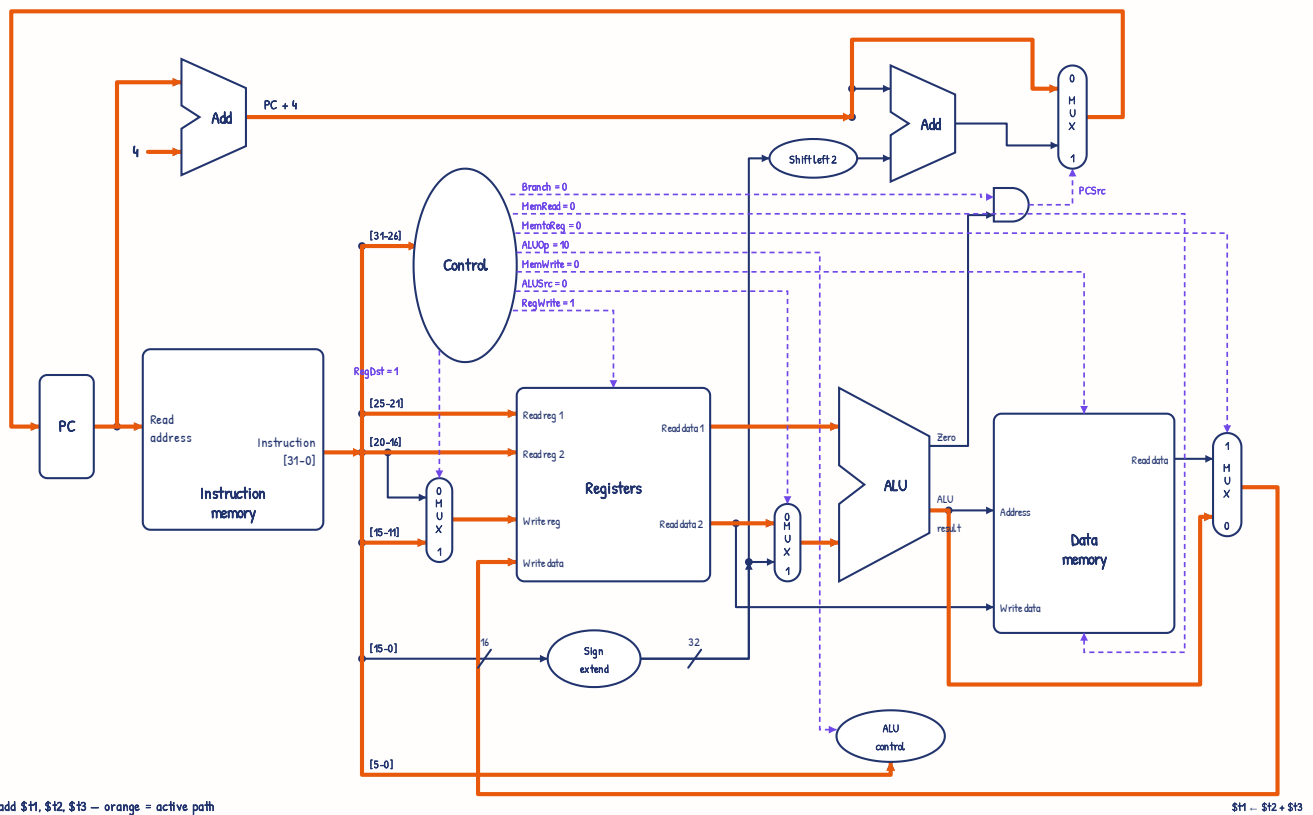
**Kaise trace karein (4 steps, har baar):**

1. **Fetch path** darken karo: PC → instruction memory; PC → +4 adder.
2. **Fields follow karo:** [25-21] → Read reg 1, [20-16] → Read reg 2 (aur RegDst input 0), [15-11] → RegDst input 1, [15-0] → sign-extend, [5-0] → ALU control.
3. **Har MUX pe** jo input use hua usse darken karo aur uska 0 / 1 likho.
4. **Destination pe khatam karo** (register, memory ya PC) aur control row bharo; jis MUX ka output use nahi hua wahan X.

**Trace 1 (actual numbers):** add  $\$t1, \$t2, \$t3$  at PC = 0x00400000, with  $\$t2 = 7$ ,  $\$t3 = 5$ .

1. Fetch: instruction memory[0x00400000] = 0x014B4820 = 000000 01010 01011 01001 00000 100000.  
Adder: PC + 4 = 0x00400004.
2. Fields: op = 000000 (R-type), rs = 01010 = 10 =  $\$t2$ , rt = 01011 = 11 =  $\$t3$ , rd = 01001 = 9 =  $\$t1$ , funct = 100000 (add).
3. Register file: Read data 1 =  $\$t2 = 7$ , Read data 2 =  $\$t3 = 5$ .
4. ALUSrc = 0 → ALU ko 5 milta hai (immediate nahi). ALUOp = 10 + funct 100000 → ALU control 0010 (add). ALU result =  $7 + 5 = 12$ , Zero = 0.

5. Data memory use nahi (MemRead 0, MemWrite 0). MemtoReg = 0 → Write data = 12.
6. RegDst = 1 → Write register = rd = 9 (\$t1). RegWrite = 1.
7. Branch = 0 → PCSrc = 0 → next PC = 0x00400004. Clock edge pe: \$t1 = 12, PC = 0x00400004.



Trace: R-type add \$t1, \$t2, \$t3

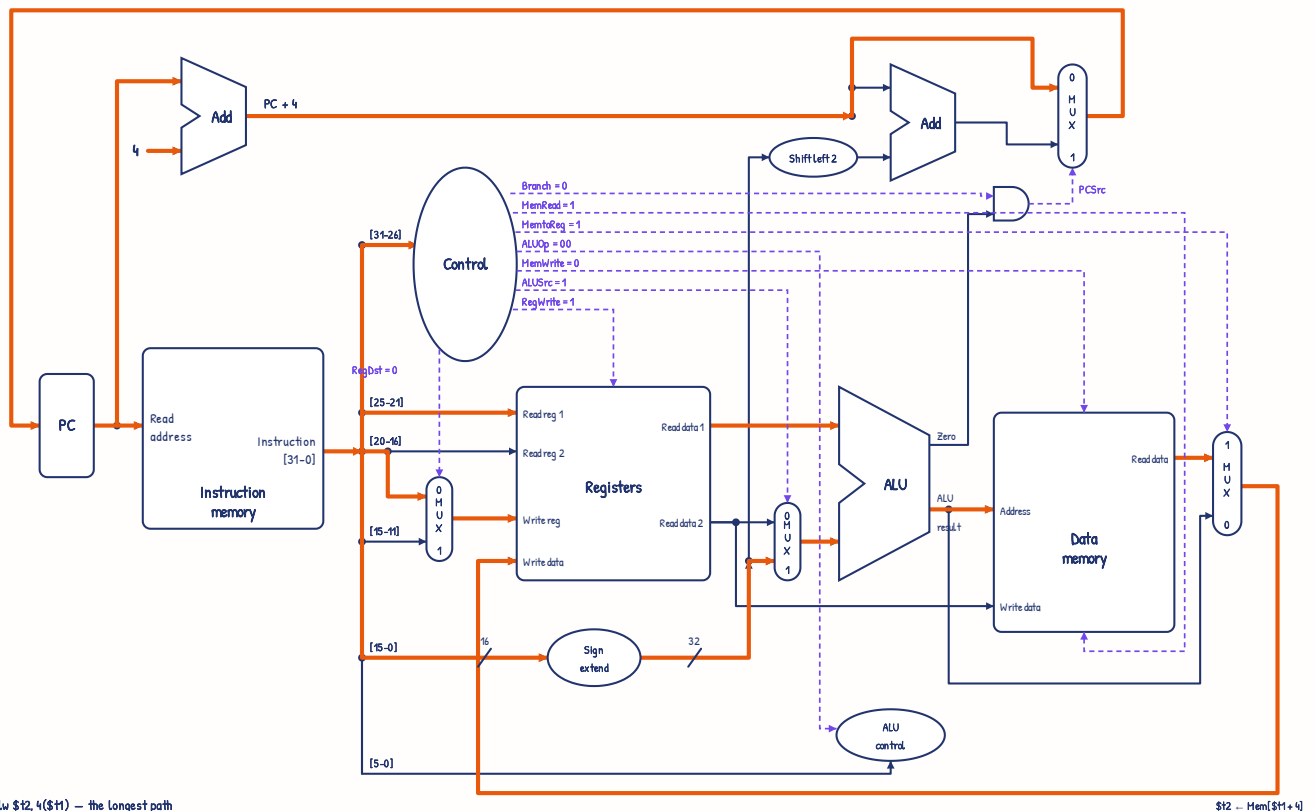
How to draw: Fetch path; rs and rt into the register file; Read data 1 and 2 into the ALU (ALUSrc 0); ALU result through MemtoReg 0 back to Write data; rd through RegDst 1 to Write reg.

**Diagram kaise padhein:** Orange = active path. Upar se: PC → instruction memory, PC → +4 → PCSrc MUX ka input 0 → wapas PC. Instruction se rs, rt register file mein; rd neeche se RegDst MUX ke input 1 mein. Read data 1 seedha ALU, Read data 2 ALUSrc MUX ke input 0 se ALU. ALU result data memory ko bypass karke MemtoReg MUX ke input 0 se neeche wale lambe wire se wapas Write data pe. Funct [5-0] wire neeche ALU control tak jaata hai. Control oval se nikalti har dashed line pe uski value likhi hai (Branch = 0, MemRead = 0, MemtoReg = 0, ALUOp = 10, MemWrite = 0, ALUSrc = 0, RegWrite = 1, RegDst = 1). Dhyan do: MemtoReg MUX mein **1 upar** (memory data) aur **0 neeche** (ALU result) hai.

**Trace 2: lw \$t2, 4(\$t1)** (0x8D2A0004) with \$t1 = 0x10010000 and Mem[0x10010004] = 99.

1. Fetch + PC + 4 (same as before).
2. Fields: op 100011 (lw), rs = 01001 = \$t1, rt = 01010 = \$t2, imm = 0x0004.
3. Read data 1 = \$t1 = 0x10010000. (Read data 2 = \$t2 ka purana value bhi niklega, par use nahi hoga.)
4. Sign-extend: 0x0004 → 0x00000004. ALUSrc = 1 → ALU ka 2nd input = 4.
5. ALUOp = 00 → add. ALU result = 0x10010000 + 4 = 0x10010004 → data memory Address.
6. MemRead = 1 → Read data = 99.

7. MemtoReg = 1 → Write data = 99. RegDst = 0 → Write register = rt = \$t2. RegWrite = 1. Edge pe \$t2 = 99.



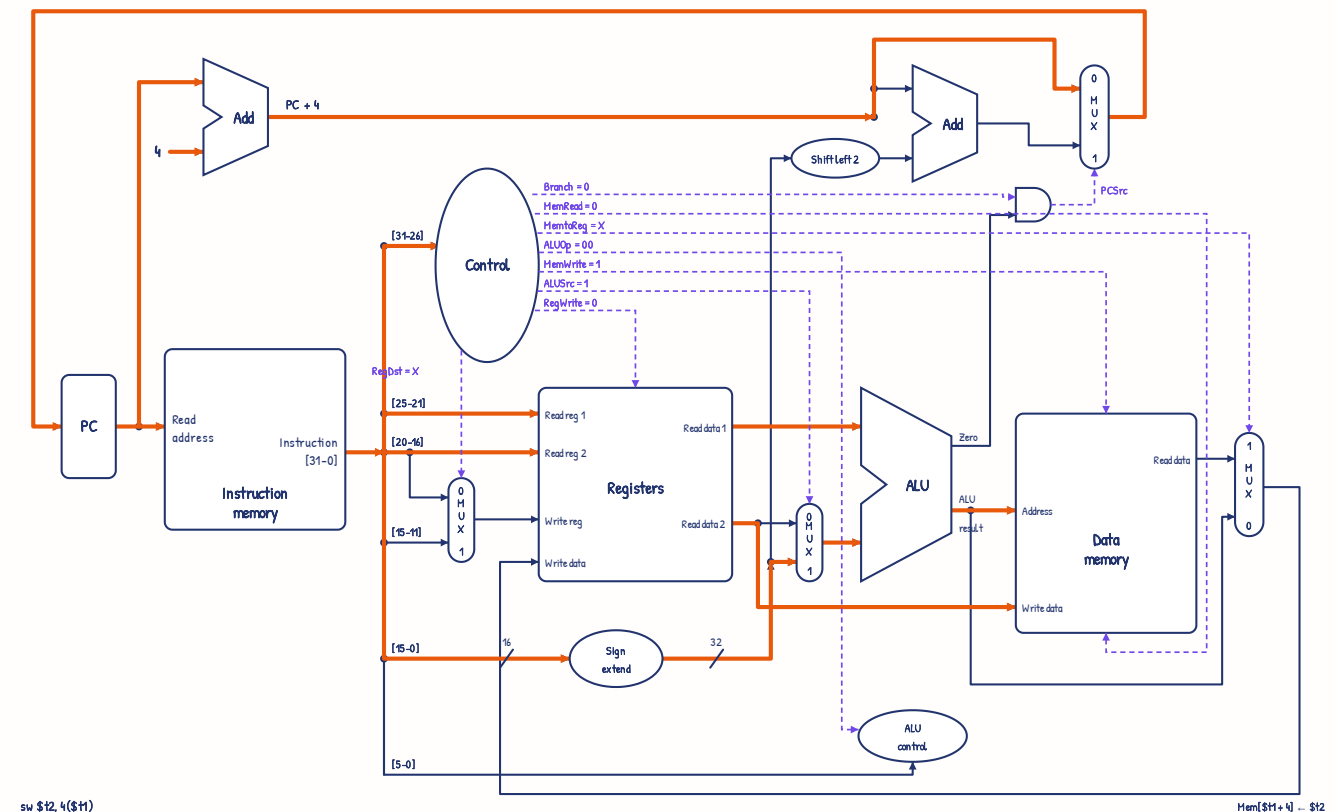
Trace: lw \$t2, 4(\$t1)

How to draw: Fetch; rs → Read data 1 → ALU; imm → sign-extend → ALUSrc 1 → ALU; ALU result → data memory address; Read data → MemtoReg 1 → Write data; rt → RegDst 0.

**Diagram kaise padhein:** Ye **sabse lamba raasta** hai: instruction memory → register file → (immediate sign-extend → ALUSrc input 1) → ALU → data memory → MemtoReg input 1 → register file. rt wala wire RegDst MUX ke input 0 pe orange hai. Isi lambai ki wajah se lw clock period decide karta hai (13.5).

**Trace 3: sw \$t2, 4(\$t1)** (0xAD2A0004) with \$t1 = 0x10010000 and \$t2 = 42.

1. Fetch + PC + 4.
2. rs = \$t1 → Read data 1 = 0x10010000; rt = \$t2 → Read data 2 = 42.
3. Sign-extend 4, ALUSrc = 1, ALUOp = 00 → Address = 0x10010004.
4. Read data 2 (42) seedha data memory ke "Write data" pe. MemWrite = 1 → edge pe Mem[0x10010004] = 42.
5. RegWrite = 0: koi register nahi badla, isliye RegDst = X, MemtoReg = X.



sw \$t2, 4(\$t1)

Mem[\$t1+4] ← \$t2

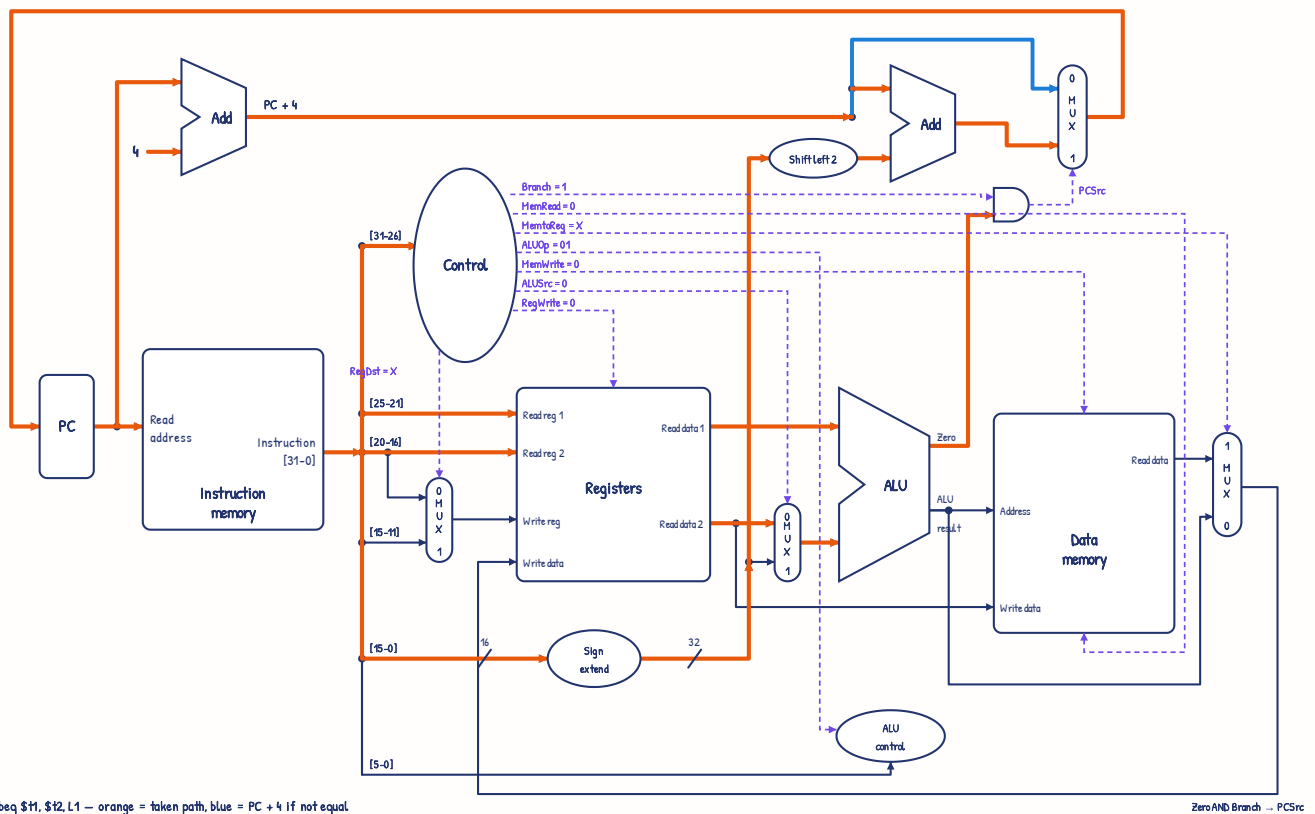
Trace: sw \$t2, 4(\$t1)

How to draw: Fetch; rs → ALU with sign-extended 4 (ALUSrc 1) → address; Read data 2 → data memory Write data; nothing returns to the register file.

**Diagram kaise padhein:** lw jaisa address path, lekin data memory pe raasta khatam: neeche se Read data 2 ka wire memory ke Write data mein jaata hai. Register file ki taraf wapas koi orange wire nahi.

**Trace 4: beq \$t1, \$t2, L1** at PC = 0x00400010, offset = 3 (0x112A0003).

1. Fetch; PC + 4 = 0x00400014.
2. Read data 1 = \$t1, Read data 2 = \$t2. ALUSrc = 0 → ALU ko dono registers. ALUOp = 01 → subtract.
3. Saath mein: offset 3 → sign-extend → shift-left-2 = 12 = 0xC → branch adder: 0x00400014 + 0xC = 0x00400020.
4. **Case A:** \$t1 = 5, \$t2 = 5 → 5 - 5 = 0 → Zero = 1. Branch = 1 → PCSrc = 1 AND 1 = 1 → PC = 0x00400020 (branch taken).
5. **Case B:** \$t1 = 5, \$t2 = 7 → 5 - 7 = -2 ≠ 0 → Zero = 0 → PCSrc = 0 → PC = 0x00400014 (not taken).
6. RegWrite = 0, MemRead = 0, MemWrite = 0; RegDst = X, MemtoReg = X.



Trace: beq \$t1, \$t2, L1

How to draw: Fetch; rs and rt into the ALU (ALUSrc 0) to subtract; offset → sign-extend → shift-left-2 → branch adder with PC + 4; Zero AND Branch selects the PCSrc MUX input.

**Diagram kaise padhein:** Orange = taken path: offset → sign-extend → shift-left-2 → branch adder → PCSrc input 1 → PC. Blue = not-taken path (PC + 4 → PCSrc input 0). ALU se Zero wire upar AND gate tak jaata hai, AND ka doosra input Branch (dashed, control se).

**Kaunsa instruction kaunse units use karta hai:**

| Instr  | Instr mem | Reg read   | ALU        | Data mem | Reg write | Branch adder used |
|--------|-----------|------------|------------|----------|-----------|-------------------|
| R-type | ✓         | ✓ (rs, rt) | ✓ op       | no       | ✓ rd      | no                |
| lw     | ✓         | ✓ (rs)     | ✓ address  | ✓ read   | ✓ rt      | no                |
| sw     | ✓         | ✓ (rs, rt) | ✓ address  | ✓ write  | no        | no                |
| beq    | ✓         | ✓ (rs, rt) | ✓ subtract | no       | no        | ✓                 |

### Solved example 1 (mock passage, exam-level):

lw \$s0, 8(\$sp) runs on the single-cycle datapath. \$sp = 0x7FFFFFF8 and the word at 0x7FFFF000 is 25.

(a) What address does the ALU compute? (b) What is written into \$s0 and with which MemtoReg? (c) Machine code? (d) Control row.

- (a) Address = \$sp + 8. Last digits: 0xF8 + 0x08 = 248 + 8 = 256 = 0x100 → last two digits 00, carry 1. Carry chain: 0xEF + 1 = 0xF0. So 0x7FFFFFF8 + 8 = **0x7FFFF000**.
- (b) MemRead = 1 → memory gives 25. MemtoReg = 1 → **25** goes to \$s0 (not the address).

3. (c)  $op = 100011$ ,  $rs = \$sp = 29 = 11101$ ,  $rt = \$s0 = 16 = 10000$ ,  $imm = 0000\ 0000\ 0000\ 1000$ . Group in 4s:  $1000\ 1111\ 1011\ 0000\ 0000\ 0000\ 0000\ 1000 = \mathbf{0x8FB00008}$ .
4. (d) Branch 0, MemtoReg 1, MemRead 1, MemWrite 0, ALUOp 00, ALUSrc 1, RegWrite 1, RegDst 0.

**Answer:** address  $0x7FFFF000$ ;  $\$s0 = 25$  via MemtoReg = 1; code  $0x8FB00008$ ; row 0 1 1 0 00 1 1 0.

### Solved example 2 (write the answer like this, 5 marks):

Trace  $sw\ \$t2, 4(\$t1)$  and fill the control signals.

1. **Fetch:** PC  $\rightarrow$  instruction memory gives the instruction; PC + 4 computed by the adder.
2. **Register read:**  $rs = \$t1$  (Read data 1, base address) and  $rt = \$t2$  (Read data 2, data to store).
3. **Address:** offset 4 is sign-extended, ALUSrc = 1 selects it, ALUOp = 00  $\rightarrow$  ALU adds: address =  $\$t1 + 4$ .
4. **Memory:** Read data 2 goes to Write data; MemWrite = 1  $\rightarrow$   $Mem[\$t1 + 4] \leftarrow \$t2$ .
5. **No write-back:** RegWrite = 0, so RegDst and MemtoReg are don't care. PCSrc = 0  $\rightarrow$  PC = PC + 4.

**Answer:** Branch 0, MemtoReg X, MemRead 0, MemWrite 1, ALUOp 00, ALUSrc 1, RegWrite 0, RegDst X.

**Galti mat karna:** (1) lw mein MemtoReg = 0 likhna  $\rightarrow$  register mein address chala jaayega, data nahi. (2) beq mein ALUSrc = 1 likhna: beq do *registers* compare karta hai, offset ALU mein nahi jaata (woh branch adder mein jaata hai). (3) beq target = PC + offset likhna: sahi =  $PC + 4 + offset \times 4$ .

**Ek line mein:** Trace = fetch path + fields + har MUX ka chuna hua input + destination; R-type rd mein ALU result, lw rt mein memory data, sw memory mein Read data 2, beq PC mein target (sirf Zero = 1 pe).

## 13.5 Critical Path and Clock Period

**Pehle socho:** School trip ki bus har stop pe utna rukti hai jitna **sabse slow** bachche ko time lagta hai, kyunki sab ek hi bus mein hain. Single-cycle CPU mein har instruction ko **ek hi size ka** clock cycle milta hai. To cycle itna lamba hona chahiye ki **sabse slow instruction (lw)** bhi khatam ho jaaye. Baaki instructions (add, beq) jaldi khatam ho ke bas **khaali baithe intezaar karte hain**. Yahi waste pipelining ki wajah hai.

**Definition:** The **critical path** is the longest combinational delay path between two state elements; it sets the minimum clock period. In the single-cycle MIPS datapath the critical path is that of **lw**: instruction memory  $\rightarrow$  register read  $\rightarrow$  ALU  $\rightarrow$  data memory  $\rightarrow$  register write (setup). Clock period  $T_c$  = delay of the slowest instruction, and clock frequency  $f = 1 / T_c$ .

### Mushkil shabd, aasaan matlab:

#### propagation delay

Input badalne ke baad output sahi hone mein lagne wala time. Jaise ALU = 200 ps.

#### critical path

Sabse lamba raasta (delay ka total sabse zyada). Chain ki sabse kamzor kadi jaisa: poora system usi ki speed se chalega.

#### ps, ns, GHz

$1 \text{ ps} = 10^{-12} \text{ s}$ ,  $1 \text{ ns} = 1000 \text{ ps}$ .  $f = 1 / T$ .  $1 / 1 \text{ ns} = 1 \text{ GHz}$ .  $1 / 800 \text{ ps} = 1 / 0.8 \text{ ns} = 1.25 \text{ GHz}$ .

#### instruction class

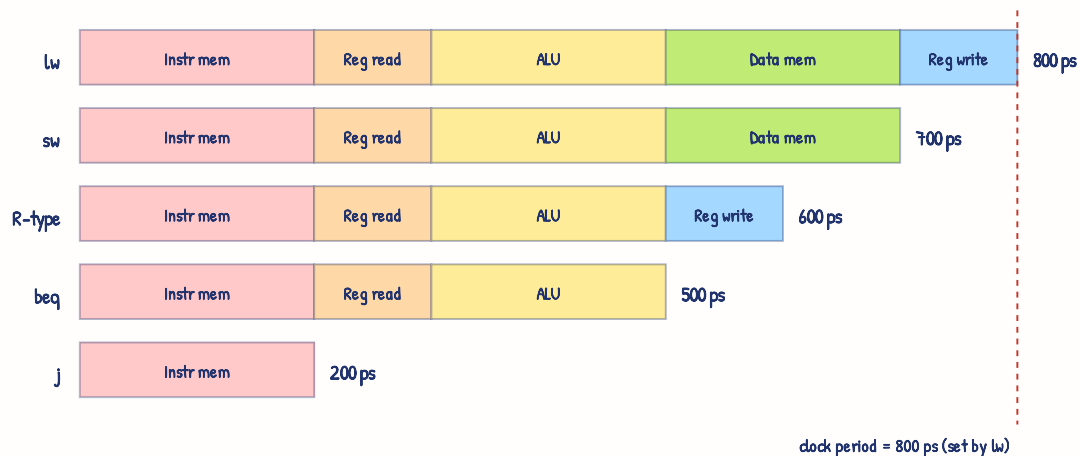
Same raasta lene wale instructions ka group: R-type, lw, sw, beq, j.

### Kaise calculate karen:

1. Har instruction class ke liye likho kaunse units use hote hain (13.4 wali table): R-type = IMem + Reg read + ALU + Reg write; lw = IMem + Reg read + ALU + DMem + Reg write; sw = IMem + Reg read + ALU + DMem; beq = IMem + Reg read + ALU; j = IMem.
2. Diye hue delays jodo, har class ka total.
3. Sabse bada total = clock period (ye hamesha lw aayega).
4. Frequency =  $1 / \text{period}$ .
5. Waste = period - us instruction ka apna time.

| Class  | IMem | Reg read | ALU | DMem | Reg write | Total         |
|--------|------|----------|-----|------|-----------|---------------|
| R-type | 200  | 100      | 200 | no   | 100       | 600 ps        |
| lw     | 200  | 100      | 200 | 200  | 100       | <b>800 ps</b> |
| sw     | 200  | 100      | 200 | 200  | no        | 700 ps        |
| beq    | 200  | 100      | 200 | no   | no        | 500 ps        |
| j      | 200  | no       | no  | no   | no        | 200 ps        |

(Patterson & Hennessy delays; MUX, sign-extend aur PC setup ko 0 maana hai.)



Single-cycle: every instruction gets the full 800 ps, so beq wastes 300 ps and j wastes 600 ps (P&H delays; MUX, sign-extend, PC setup ignored).

### Critical path: delay per instruction class (lw is longest)

How to draw: Horizontal stacked bars, one per instruction class, segments for instruction memory, register read, ALU, data memory, register write; a vertical line at the lw total marks the clock period.

**Diagram kaise padhein:** Har row ek instruction class hai, rangeen tukde uske units (Instr mem, Reg read, ALU, Data mem, Reg write) ki lambai = delay. Right mein total. Vertical line 800 ps pe = clock period, lw ki lambai ne set ki. beq ki bar 500 pe khatam, line tak ka khaali hissa = 300 ps waste. j ka waste 600 ps.

**Formula:**  $T_c = \max(\text{total delay of each instruction class}) = t_{\text{IMem}} + t_{\text{RegRead}} + t_{\text{ALU}} + t_{\text{DMem}} + t_{\text{RegWrite}}$  (lw)

$$f = 1 / T_c \cdot \text{Program time} = N \times 1 \times T_c$$

Harris & Harris version (more detailed):  $T_c = t_{\text{pcq,PC}} + t_{\text{mem}} + t_{\text{RFread}} + t_{\text{ALU}} + t_{\text{mem}} + t_{\text{mux}} + t_{\text{RFsetup}}$ ;  $t_{\text{pcq}} = \text{PC ka clock-to-Q delay}$ ,  $t_{\text{RFsetup}} = \text{register file setup time}$ .

### Solved example 1 (easy):

*Delays: instruction memory 200, register read 100, ALU 200, data memory 200, register write 100 ps. Find the single-cycle clock period, frequency, and the time wasted by beq in each cycle.*

1.  $lw = 200 + 100 + 200 + 200 + 100 = 800 \text{ ps}$  (sabse lamba).
2.  $T_c = 800 \text{ ps}$ .
3.  $f = 1 / 800 \text{ ps} = 1 / (0.8 \times 10^{-9} \text{ s}) = 1.25 \times 10^9 \text{ Hz} = 1.25 \text{ GHz}$ .
4.  $beq = 200 + 100 + 200 = 500 \text{ ps}$ . Waste =  $800 - 500 = 300 \text{ ps}$ .

**Answer:**  $T_c = 800 \text{ ps}$ ,  $f = 1.25 \text{ GHz}$ , beq wastes 300 ps per cycle.

### Solved example 2 (mock exam-level):

*Single-cycle delays: instruction memory 250, register read 150, ALU 200, data memory 250, register write 150 ps. Find the delay of each instruction class, the clock period, the clock frequency, and the time to run  $10^6$  instructions.*

1. R-type =  $250 + 150 + 200 + 150 = 750$  ps.
2. lw =  $250 + 150 + 200 + 250 + 150 = 1000$  ps.
3. sw =  $250 + 150 + 200 + 250 = 850$  ps.
4. beq =  $250 + 150 + 200 = 600$  ps. j = 250 ps.
5. Sabse bada = lw = 1000 ps  $\rightarrow T_c = 1000$  ps = 1 ns.
6.  $f = 1 / 1 \text{ ns} = 1 \text{ GHz}$ .
7. Time =  $10^6 \times 1 (\text{CPI}) \times 1 \text{ ns} = 10^6 \text{ ns} = 1 \text{ ms}$ .

**Answer:** R 750, lw 1000, sw 850, beq 600, j 250 ps;  $T_c = 1000$  ps;  $f = 1 \text{ GHz}$ ;  $10^6$  instructions take 1 ms.

### Solved example 3 (Harris & Harris formula):

$t_{pcq} = 30$ ,  $t_{mem} = 250$ ,  $t_{RFread} = 150$ ,  $t_{ALU} = 200$ ,  $t_{mux} = 25$ ,  $t_{RFsetup} = 20$  ps. Find  $T_c$ .

1. Memory do baar aata hai (instruction memory aur data memory).
2.  $T_c = 30 + 250 + 150 + 200 + 250 + 25 + 20$ .
3.  $30 + 250 = 280$ ;  $+ 150 = 430$ ;  $+ 200 = 630$ ;  $+ 250 = 880$ ;  $+ 25 = 905$ ;  $+ 20 = 925$ .

**Answer:**  $T_c = 925$  ps ( $f = 1 / 925 \text{ ps} \approx 1.08 \text{ GHz}$ ).

### Solved example 4 (why single-cycle is inefficient, write like this):

Why is the single-cycle design inefficient? How much faster would an ideal variable-length clock be for a mix of 25 % lw, 10 % sw, 45 % R-type, 15 % beq, 5 % j (P&H delays)?

1. Every instruction gets the full 800 ps, although R-type needs only 600, beq 500, j 200. Also each unit is busy only for a part of the cycle (when the ALU works, the instruction memory sits idle).
2. Average needed time =  $0.25 \times 800 + 0.10 \times 700 + 0.45 \times 600 + 0.15 \times 500 + 0.05 \times 200$ .
3. =  $200 + 70 + 270 + 75 + 10 = 625$  ps.
4. Speedup =  $800 / 625 = 1.28$ .
5. Better fix: cut the datapath into 5 stages of about 200 ps and overlap instructions (pipelining, Unit 14): one instruction can finish every 200 ps  $\rightarrow$  up to  $800 / 200 = 4$  times faster.

**Answer:** time wasted on short instructions and idle units; variable clock would be 1.28x faster; pipelining fixes it better.

**Galti mat karna:** (1) R-type ka delay clock period le lena: period = max = lw. (2) beq ya R-type mein data memory ka delay jod dena. (3) Average delay ko clock period bolna: single-cycle mein har cycle same length ka hota hai, average sirf "ideal variable clock" ke comparison ke liye hai.

**Trick:** P&H numbers yaad: **8-7-6-5-2** ( $\times 100$  ps) = lw, sw, R, beq, j.

**Ek line mein:** Single-cycle clock = lw ka poora raasta (IMem + Reg + ALU + DMem + Reg); CPI 1 hai par clock slow, chhote instructions time waste karte hain, isliye pipelining.

### Exam mein is unit se kya aata hai → kaise solve karein

| Question type                                                        | Method (steps)                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trace <instruction> and fill the control signals (5 marks, diagram)  | 1. Datapath draw karo (5 blocks + adders + 4 MUX + control). 2. Fetch path darken. 3. Fields follow: rs, rt, rd / imm. 4. Har MUX pe chuna input mark (0 / 1). 5. Control row: Branch, MemtoReg, MemRead, MemWrite, <b>ALUOp</b> , ALUSrc, RegWrite, RegDst; X sirf RegDst / MemtoReg jab RegWrite = 0. |
| Fill one control signal (MCQ: RegDst for lw, ALUOp for beq)          | Instruction ka kaam socho: destination rt ya rd? Doosra operand register ya constant? Memory padhni / likhni? ALU add / sub / funct? Table se match karo.                                                                                                                                               |
| Function of a MUX / match MUX to what it selects                     | RegDst = rt / rd, ALUSrc = Read data 2 / immediate, MemtoReg = ALU / memory, PCSrc = PC + 4 / branch target (Branch AND Zero).                                                                                                                                                                          |
| Which bits go where                                                  | [31-26] control, [25-21] Read reg 1, [20-16] Read reg 2 + RegDst 0, [15-11] RegDst 1, [15-0] sign-extend, [5-0] ALU control.                                                                                                                                                                            |
| beq: next PC / branch target                                         | 1. PC + 4. 2. offset × 4. 3. Jodo = target. 4. Registers barabar → Zero = 1 → PC = target, warna PC + 4.                                                                                                                                                                                                |
| lw with values (address, value written)                              | Address = base register + sign-extended offset (hex addition carefully). Written value = memory data (MemtoReg = 1), address nahi.                                                                                                                                                                      |
| ALU control input                                                    | ALUOp 00 → 0010, 01 → 0110, 10 → funct: add 0010, sub 0110, and 0000, or 0001, slt 0111.                                                                                                                                                                                                                |
| Critical path / clock period numerical                               | 1. Har class ke units ka delay jodo. 2. Max = $T_c$ (lw). 3. $f = 1 / T_c$ . 4. Program time = $N \times T_c$ . 5. Waste = $T_c$ - instruction ka time.                                                                                                                                                 |
| Fetch-decode-execute steps / which step an instruction skips         | F D E M WB likho; add M skip, sw WB skip, beq dono skip; $PC \leftarrow PC + 4$ in fetch.                                                                                                                                                                                                               |
| ISA vs microarchitecture; single-cycle traits; why separate memories | Menu vs kitchen; CPI = 1, period = slowest (lw); har unit ek baar per cycle → 2 memories + 2 extra adders.                                                                                                                                                                                              |

## Unit 14 · Pipelining

**Is unit ki kahani:** Unit 13 ka single-cycle processor ek instruction poora khatam karke hi agla shuru karta hai, aur har cycle lw jitna lamba (800 ps) hai. Is unit mein idea hai: **kaam ko 5 stages mein kaato aur alag-alag instructions ko alag stages mein ek saath chalao**, jaise laundry ya factory assembly line. Clock chhota ho jaata hai aur har cycle ek instruction khatam hota hai. Kuch formulas (cycles, speedup) aur pipeline diagram yahan se aate hain. Jab instructions ek doosre pe depend karte hain to problem aati hai: woh hazards hain (Unit 15). (Is unit ki class slides nahi thi; content Patterson & Hennessy se hai.)

### 14.1–14.2 The Problem with Single-Cycle, and the Laundry Analogy

**Pehle socho:** Hostel mein 4 dosto ke kapde (loads A, B, C, D) dhone hain. Har load: **wash** 30 min, **dry** 30 min, **fold** 30 min, **stash** (almirah mein rakhna) 30 min. Bewakoof tareeka: A ka poora kaam (2 ghante) khatam, phir B shuru. Smart tareeka: A washer se nikal ke dryer mein gaya, **usi waqt B washer mein daal do**. Machine khaali kyun baithe? Ek load ko utna hi time lagta hai, lekin saare loads bahut jaldi khatam ho jaate hain. CPU mein: single-cycle mein jab ALU kaam kar raha hai, instruction memory khaali baithi hai. Pipelining us khaali machine ko agle instruction ke liye use karta hai.

**Definition: Pipelining** is an implementation technique in which the execution of an instruction is divided into stages and multiple instructions are overlapped, each in a different stage, so that (once the pipeline is full) one instruction completes every clock cycle. It improves **throughput** (instructions completed per unit time), not the **latency** of a single instruction.

**Mushkil shabd, aasaan matlab:**

#### stage

Kaam ka ek tukda jo ek hardware hissa karta hai. Laundry mein wash / dry / fold / stash; CPU mein IF / ID / EX / MEM / WB.

#### latency

Ek item ko shuru se khatam hone mein kitna time. Ek load ki latency = 2 ghante, pipelining ke baad bhi 2 ghante.

#### throughput

Har unit time mein kitne items khatam hote hain. Pipelined laundry mein har 30 min ek load ready.

#### fill time

Shuru mein pipeline bharne ka time: pehla load akela 4 stages se guzarta hai, tab tak kuch khatam nahi hota.

#### idle

Khaali baitha hua. Single-cycle mein ALU kaam kare to instruction memory idle.

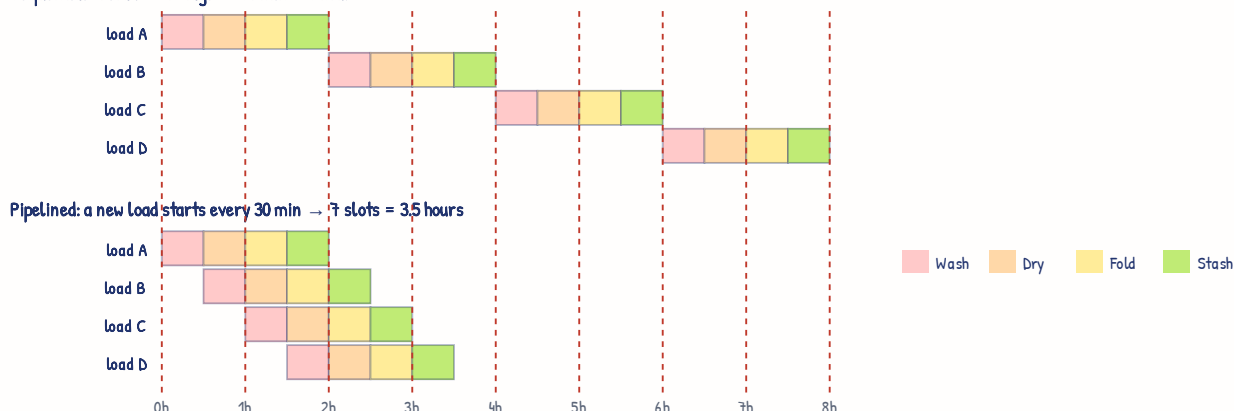
### Kaise kaam karta hai (laundry, 4 loads, 4 stages × 30 min):

1. **Sequential:** har load  $4 \times 30 = 120 \text{ min} = 2 \text{ h}$ . 4 loads  $\times 2 \text{ h} = 8 \text{ h}$ .
2. **Pipelined:** 30-min ke "slots" mein socho. Slot 1: A wash. Slot 2: A dry, B wash. Slot 3: A fold, B dry, C wash. Slot 4: A stash, B fold, C dry, D wash (sab machines busy). Slot 5: B stash, C fold, D dry. Slot 6: C stash, D fold. Slot 7: D stash.
3. Total slots = 7 = (4 stages + 4 loads - 1). Time =  $7 \times 30 \text{ min} = 210 \text{ min} = 3.5 \text{ h}$ .
4. Har load ki latency abhi bhi 2 h (A: slot 1 se 4). Lekin pipeline full hone ke baad **har 30 min ek load** khatam hota hai.
5. Speedup =  $8 / 3.5 \approx 2.29$ . Bahut saare loads ho to speedup  $\rightarrow 4$  (= number of stages).
6. **CPU pe lagao:** single-cycle mein cycle = 800 ps (lw). Datapath ko 5 stages (har ek  $\approx 200 \text{ ps}$ ) mein kaato, beech mein registers lagao. Clock = 200 ps, aur full pipeline mein har 200 ps ek instruction khatam.

| Load | Slot 1 | Slot 2 | Slot 3 | Slot 4 | Slot 5 | Slot 6 | Slot 7 |
|------|--------|--------|--------|--------|--------|--------|--------|
| A    | Wash   | Dry    | Fold   | Stash  |        |        |        |
| B    |        | Wash   | Dry    | Fold   | Stash  |        |        |
| C    |        |        | Wash   | Dry    | Fold   | Stash  |        |
| D    |        |        |        | Wash   | Dry    | Fold   | Stash  |

**Laundry ke rules jo CPU pe bhi lagte hain:** (1) sab stages same time lein to best; **sabse slow stage speed decide karta hai** (dryer 60 min ho to har 60 min ek load). (2) Stages ke beech kaam rakhne ki jagah chahiye (CPU mein pipeline registers). (3) Problem tab aati hai jab ek load ko kuch chahiye jo pichhla load abhi khatam nahi kiya (CPU mein hazards).

Sequential: 4 loads  $\times$  4 stages  $\times$  30 min = 8 hours



Each load still takes 2 h (latency unchanged); throughput rises from 1 load per 2 h to 1 load per 30 min. Speedup  $8 / 3.5 \approx 2.3 \rightarrow 4$  for many loads.

### Laundry: sequential vs pipelined

How to draw: Two Gantt charts on the same time axis: top one staircase of 16 boxes end-to-end; bottom one each load shifted by one slot, overlapping.

**Diagram kaise padhein:** Upar wala chart sequential hai: 16 rangeen boxes ek ke baad ek, seedhi seedhi (staircase), 8 h tak. Neeche wala pipelined hai: har load ki row ek slot right khiski hui, boxes overlap karte hain, 3.5 h pe khatam. Har rang ek stage (legend: Wash, Dry, Fold, Stash). Kisi bhi ek vertical line (time) pe neeche wale chart mein dekho: zyada se zyada 4 alag rang = 4 machines ek saath busy.

### Solved example 1 (easy):

*Laundry with 4 stages of 30 min each. How long do 6 loads take (a) sequentially, (b) pipelined? (c) Speedup?*

1. (a) Ek load =  $4 \times 30 = 120 \text{ min} = 2 \text{ h}$ . 6 loads =  $6 \times 2 = 12 \text{ h}$ .
2. (b) Slots = stages + loads - 1 =  $4 + 6 - 1 = 9$ . Time =  $9 \times 30 \text{ min} = 270 \text{ min} = 4.5 \text{ h}$ .
3. (c) Speedup =  $12 / 4.5 = 2.67$ .

**Answer: 12 h sequential, 4.5 h pipelined, speedup  $\approx 2.67$ .**

### Solved example 2:

*The dryer takes 60 min, the other three stages 30 min. Once the pipeline is full, how often does a load finish?*

1. Washer har 30 min ek load de sakta hai, lekin dryer har load pe 60 min leta hai.
2. Loads dryer ke aage line mein ruk jaayenge. Pipeline ki speed = sabse slow stage.

**Answer: one load every 60 min (slowest stage sets the pace).**

### Solved example 3 (write the answer like this):

*Why is a single-cycle processor inefficient, and how does pipelining help? (3 marks)*

1. In a single-cycle processor the clock period is fixed by the slowest instruction (lw, 800 ps); shorter instructions (R-type 600, beq 500 ps) waste the rest of the cycle.
2. Each unit is busy only for part of the cycle; e.g. the instruction memory is idle while the ALU works.
3. Pipelining divides the datapath into stages separated by pipeline registers and overlaps instructions, so all units work at once. The clock period becomes one stage (about 200 ps) and, once full, one instruction completes per cycle.

**Answer: wasted time + idle units  $\rightarrow$  split into stages and overlap  $\rightarrow$  shorter clock, CPI  $\approx 1$ , higher throughput.**

**Galti mat karna:** (1) "Pipelining se ek instruction jaldi khatam hota hai" galat hai: latency same rehti hai (ya thodi badhti hai). Throughput badhta hai. (2) Pipelined time =  $n \times \text{stage time}$  mat likho; shuru ka fill time  $(k - 1)$  bhoolna common galti hai.

**Trick: "Machine kabhi khaali na baithe."** Pipelining = har machine ko har slot mein koi na koi kaam.

**Ek line mein:** Single-cycle mein units idle aur chhote instructions time waste karte hain; pipelining kaam ko stages mein kaat ke instructions overlap karta hai: latency same, throughput zyada.

### 14.3 The 5-Stage MIPS Pipeline and Pipeline Registers

**Pehle socho:** Factory assembly line mein har station ke beech ek **tray** hota hai. Station 1 apna kaam tray mein rakhta hai; bell bajti hai (clock); tray agle station ko milta hai aur station 1 naya kaam shuru karta hai. Bina tray ke, station 1 ka naya kaam purane ko overwrite kar dega. CPU mein ye trays hi **pipeline registers** hain.

**Definition:** The classic MIPS pipeline has five stages: **IF** (instruction fetch), **ID** (instruction decode and register read), **EX** (execute or address calculation), **MEM** (data memory access) and **WB** (write back). Between consecutive stages are **pipeline registers**, named **IF/ID**, **ID/EX**, **EX/MEM**, **MEM/WB**, which hold everything a stage produced (data, register numbers and control signals) for the next stage.

**Mushkil shabd, aasaan matlab:**

**pipeline register**

Do stages ke beech ka register (tray). Naam dono stages ka: IF/ID = IF aur ID ke beech. Total 4.

**control travels with the instruction**

Control signals ID mein bante hain, phir instruction ke saath tray se tray aage jaate hain. EX wale (RegDst, ALUOp, ALUSrc) EX mein use, MEM wale (Branch, MemRead, MemWrite) MEM mein, WB wale (MemtoReg, RegWrite) WB mein.

**write first half, read second half**

Register file cycle ke pehle aadhe mein likhta hai, doosre aadhe mein padhta hai. Isliye WB mein likha value usi cycle ID wala instruction padh leta hai.

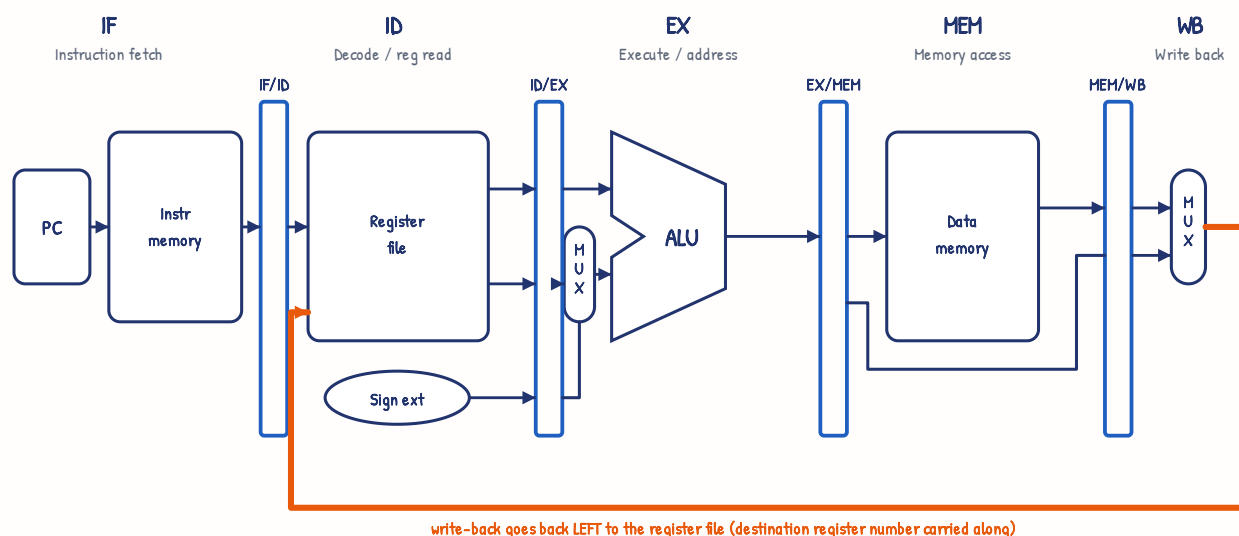
**Kaise kaam karta hai (har stage ka kaam):**

| Stage                            | Kya hota hai                                               | Hardware                                   |
|----------------------------------|------------------------------------------------------------|--------------------------------------------|
| <b>IF</b> instruction fetch      | $IR \leftarrow IMem[PC], PC \leftarrow PC + 4$             | PC, instruction memory, +4 adder           |
| <b>ID</b> decode / register read | rs, rt padho; immediate sign-extend; control signals banao | register file (read), sign-extend, control |
| <b>EX</b> execute                | ALU operation, lw / sw address, beq compare + target       | ALU, ALUSrc MUX, branch adder              |
| <b>MEM</b> memory access         | lw padhta, sw likhta; baaki kuch nahi                      | data memory                                |
| <b>WB</b> write back             | result rd / rt mein likho                                  | MemtoReg MUX, register file (write)        |

1. Har clock edge pe har stage apna result aage wale pipeline register mein daalta hai.

2. Usi edge pe har stage peechhe wale register se naya instruction le leta hai.
3. Isliye kisi bhi cycle mein 5 alag instructions 5 alag stages mein hote hain.
4. Destination register number (rd / rt) bhi ID/EX → EX/MEM → MEM/WB mein saath chalta hai, kyunki WB tak IF/ID mein koi aur instruction aa chuka hota hai.
5. Clock period = sabse slow stage (+ pipeline register ka thoda overhead).

**MIPS pipeline ke liye aasaan kyun hai (Lecture 10):** (1) saare instructions 32-bit fixed length: IF bina decode kiye agla fetch kar sakta hai; (2) kam formats, rs / rt hamesha same bits mein: decode ke saath hi registers padh lo; (3) memory sirf lw / sw: address EX mein, access MEM mein; (4) memory access aligned.



Pipeline registers (blue bars) hold everything a stage produces — data, register numbers and control — for the next stage.

#### Pipelined datapath with IF/ID, ID/EX, EX/MEM, MEM/WB registers

How to draw: Draw the single-cycle blocks in a row (PC + IMem, register file, ALU, data memory, write-back MUX) and insert a tall thin register between each pair; label them IF/ID, ID/EX, EX/MEM, MEM/WB; show the write-back wire returning to the register file.

**Diagram kaise padhein:** Upar IF, ID, EX, MEM, WB labels batate hain kaunsa hissa kaunsa stage. Neele lambe patle bars pipeline registers hain (IF/ID, ID/EX, EX/MEM, MEM/WB). Left se right: PC + instruction memory (IF) → register file + sign-extend (ID) → ALU (EX) → data memory (MEM) → MUX (WB). Orange wire WB se **wapas left** register file tak jaata hai: result likhne ke liye (ye ek hi wire right-to-left chalti hai). Trace lw: CC1 IF, CC2 ID (base register padha), CC3 EX (address), CC4 MEM (data), CC5 WB (register mein).

#### Solved example 1 (MCQ type):

(a) In which stage does lw compute its memory address? (b) How many pipeline registers are there? (c) Why must the destination register number travel to WB?

1. (a) Address = base + offset ek addition hai → ALU → **EX**. Memory padhna MEM mein.
2. (b) Stages ke beech ki jagah: 5 stages → 4 gaps → IF/ID, ID/EX, EX/MEM, MEM/WB = **4**.
3. (c) Jab lw WB mein pahunchta hai, IF/ID mein 3 instructions baad wala instruction hai. Uske rd / rt bits galat honge. Isliye lw ka apna destination number tray-dar-tray saath chalna chahiye.

**Answer: (a) EX, (b) 4, (c) because each pipeline register holds a different instruction by then.**

### Solved example 2 (write the answer like this, 5 marks):

Draw the 5-stage MIPS pipeline with its pipeline registers and explain each stage.

1. Draw D14.3a: PC/IMem | IF/ID | Reg file + sign-extend | ID/EX | ALU | EX/MEM | Data memory | MEM/WB | write-back MUX, with the write-back wire returning to the register file.
2. IF: fetch instruction at PC,  $PC \leftarrow PC + 4$ . ID: decode, read rs and rt, sign-extend, generate control. EX: ALU operation or address calculation, branch compare. MEM: lw reads / sw writes data memory. WB: write the result to rd or rt.
3. Pipeline registers hold data, register numbers and control for the next stage; control is generated in ID and carried along.
4. Register file is written in the first half and read in the second half of a cycle.

**Answer:** 5 stages IF ID EX MEM WB + 4 pipeline registers IF/ID, ID/EX, EX/MEM, MEM/WB.

**Galti mat karna:** Pipeline register ka naam ek stage pe mat rakho ("ID register" galat), naam hai IF/ID (do stages ke beech). Aur WB ka wire right se left jaata hai, ise diagram mein bhoolna mat.

**Trick:** "I Do EXercise, MEMorize, Write Back" = IF, ID, EX, MEM, WB.

**Ek line mein:** IF → ID → EX → MEM → WB, beech mein 4 trays (IF/ID, ID/EX, EX/MEM, MEM/WB) jo data, register number aur control aage le jaate hain.

## 14.4 Pipeline Diagrams (stages × cycles)

**Pehle socho:** Railway timetable ek table hai: rows mein trains, columns mein time, aur cell mein station ka naam. Pipeline diagram bilkul yahi hai: rows = instructions, columns = clock cycles, cell = us cycle mein instruction kis stage mein hai. Isse turant dikhta hai kitne cycles lage aur kaun kab kahan tha.

**Definition:** A multi-cycle pipeline diagram shows instructions as rows and clock cycles (CC1, CC2, CC3) as columns; each cell names the stage the instruction occupies in that cycle. In an ideal pipeline instruction  $i$  enters IF in cycle  $i$ , and  $n$  instructions in a  $k$ -stage pipeline take  $k + (n - 1)$  cycles.

**Mushkil shabd, aasaan matlab:**

**CC (clock cycle)**

CC1 = pehla cycle, CC5 = paanchwa.

**staircase / diagonal**

Har agli row ek column right khiski: kyunki har cycle ek naya instruction aata hai.

**in flight**

Abhi pipeline ke andar (shuru ho gaya, khatam nahi hua).

## Independent instructions

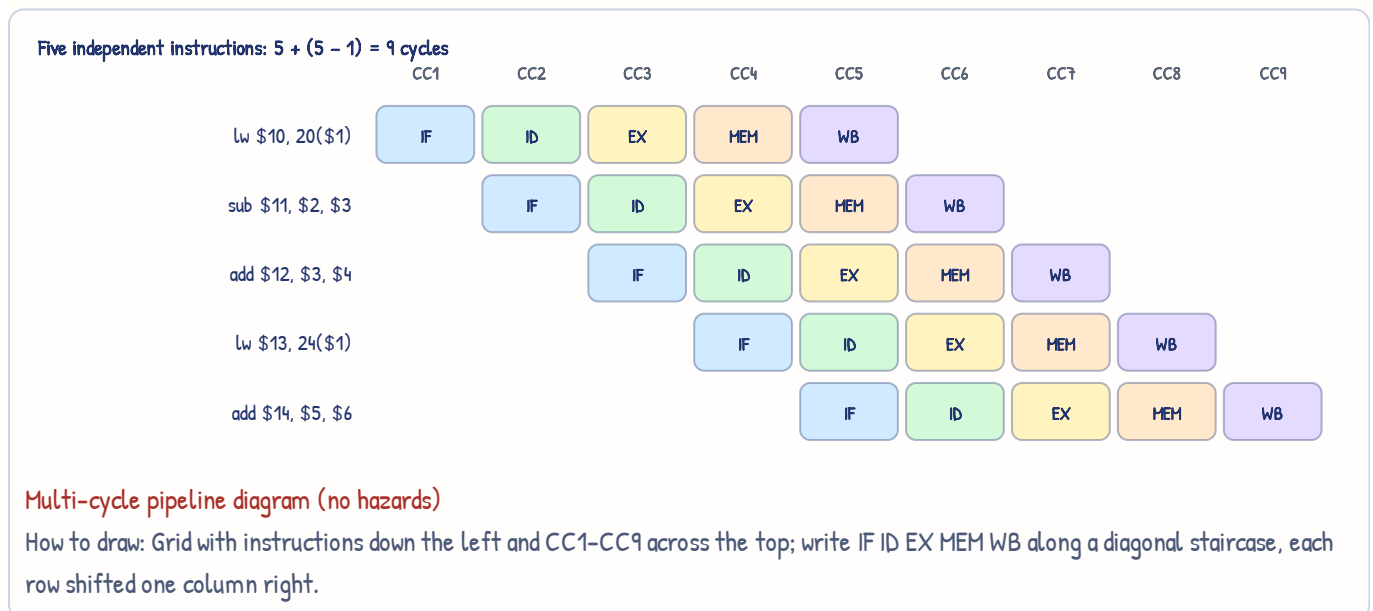
Jo ek doosre ka result use nahi karte, isliye koi rukawat (stall) nahi.

### Kaise banayein:

1. Left mein instructions likho, upar CC1, CC2 likho.
2. Instruction 1: CC1 IF, CC2 ID, CC3 EX, CC4 MEM, CC5 WB.
3. Instruction 2: ek column right: CC2 se shuru.
4. Instruction i: CC i mein IF. Stage s (IF = 1, ID = 2, EX = 3, MEM = 4, WB = 5) cycle  $i + s - 1$  mein.
5. Last instruction (n) ka WB = cycle  $n + 5 - 1 = n + 4$ . Yahi total cycles.
6. Kisi column ko upar se neeche padho: us cycle mein kaun kis stage mein.

| Instruction        | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 | CC8 | CC9 |
|--------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| lw \$10, 20(\$1)   | IF  | ID  | EX  | MEM | WB  |     |     |     |     |
| sub \$11, \$2, \$3 |     | IF  | ID  | EX  | MEM | WB  |     |     |     |
| add \$12, \$3, \$4 |     |     | IF  | ID  | EX  | MEM | WB  |     |     |
| lw \$13, 24(\$1)   |     |     |     | IF  | ID  | EX  | MEM | WB  |     |
| add \$14, \$5, \$6 |     |     |     |     | IF  | ID  | EX  | MEM | WB  |

CC5 column dekho: WB, MEM, EX, ID, IF. Paanchon stages busy, paanch instructions in flight. Ye "pipeline full" ka matlab hai.



**Diagram kaise padhein:** Upar CC1 to CC9. Har row ek instruction, rangeen boxes us cycle ki stage. Har rang ek stage hai (IF ek rang, ID doosra). Rows ek ek column right khiski hain (staircase). Last box CC9 mein (add \$14 ka WB) = total 9 cycles.

**Formula:** Total cycles (no stalls) =  $k + (n - 1) \cdot$  instruction i, stage s  $\rightarrow$  cycle  $i + s - 1$

k = number of stages (5), n = number of instructions. Pehle instruction ko k cycles, phir har naye instruction ke liye +1.

### Solved example 1 (easy):

How many cycles do 5 independent instructions take in a 5-stage pipeline? And 100 instructions?

1.  $k = 5, n = 5: 5 + (5 - 1) = 5 + 4 = 9$  cycles.
2.  $k = 5, n = 100: 5 + (100 - 1) = 5 + 99 = 104$  cycles.
3. Check: non-pipelined 5-stage machine pe  $100 \times 5 = 500$  cycles lagte.

**Answer: 9 cycles and 104 cycles.**

### Solved example 2:

In the diagram above, which stage is the third instruction (add \$12) in during CC5? Which instruction is in MEM during CC6?

1. add \$12 = instruction 3  $\rightarrow$  IF in CC3.  $CC5 - CC3 = 2$  stages aage: IF (3)  $\rightarrow$  ID (4)  $\rightarrow$  EX (5).  
Formula:  $s = 5 - 3 + 1 = 3 = EX$ .
2. CC6 mein MEM ( $s = 4$ ):  $i = 6 - 4 + 1 = 3 \rightarrow$  instruction 3 = add \$12.

**Answer: add \$12 is in EX during CC5; add \$12 is in MEM during CC6.**

**Galti mat karna:** (1) Doosre instruction ko bhi CC1 se shuru karna: har instruction ek cycle baad aata hai. (2) Total =  $5 \times n$  likhna: woh non-pipelined hai. Pipelined =  $5 + n - 1$ .

**Ek line mein:** Rows = instructions, columns = cycles, har row ek column right;  $n$  instructions,  $k$  stages  $\rightarrow k + n - 1$  cycles.

## 14.5 Throughput, Latency and Speedup (numericals)

**Pehle socho:** Pipelining kitna fayda deta hai? Laundry mein 4 stages the to max 4 guna. Lekin agar ek machine (dryer) slow ho, ya har tray ko shift karne mein thoda time lage, to fayda kam. Exam mein yahi numericals aate hain: non-pipelined time, pipelined time, speedup, aur "speedup exactly k kyun nahi".

**Definition:** **Latency** is the time one instruction takes from IF to WB. **Throughput** is the number of instructions completed per unit time. **Speedup** = (execution time without pipelining) / (execution time with pipelining). With  $k$  perfectly balanced stages, no register overhead and no hazards, speedup approaches  $k$  for a large number of instructions (ideal speedup).

**Mushkil shabd, aasaan matlab:**

**balanced stages**

Saare stages ka delay barabar (jaise sab 200 ps). Unbalanced = alag alag (250, 350, 150).

### register overhead

Har pipeline register ka extra time (setup + clock-to-Q). Jaise 20 ps. Har cycle mein jud jaata hai.

### cycle time ( $t_{cycle}$ )

Pipelined clock period = sabse slow stage + overhead. Clock sabke liye ek hai, to slowest stage ke hisaab se.

### T (non-pipelined time per instruction)

Saare stages ka total delay (bina registers ke). Ek instruction ek baar mein poora.

### Kaise solve karein (har numerical mein yahi 6 steps):

1.  $T$  = saare stage delays ka sum (non-pipelined, ek instruction).
2. Non-pipelined total =  $n \times T$ .
3.  $t_{cycle} = \max(\text{stage delays}) + \text{register overhead}$ .
4. Pipelined total =  $(k + n - 1) \times t_{cycle}$ .
5. Speedup = non-pipelined / pipelined. Bahut bade  $n$  ke liye speedup  $\approx T / t_{cycle}$ .
6. Latency (pipelined) =  $k \times t_{cycle}$ ; throughput  $\approx 1$  instruction per  $t_{cycle}$ .

### Formula:

Non-pipelined time =  $n \times T$ , where  $T$  = sum of stage delays

Pipelined time =  $(k + n - 1) \times t_{cycle}$ , where  $t_{cycle}$  = slowest stage + register overhead

Speedup =  $n \times T / [(k + n - 1) \times t_{cycle}]$  · balanced, no overhead: Speedup =  $n \cdot k / (k + n - 1) \rightarrow k$

Latency =  $k \times t_{cycle}$  · Throughput =  $1 / t_{cycle}$  (pipeline full)

$n$  = instructions,  $k$  = stages.

### Solved example 1 (equal stages, easy):

*A 5-stage pipeline has each stage 2 ns (ignore register overhead). For 100 instructions find the non-pipelined time, pipelined time, speedup, latency and throughput.*

1.  $T = 5 \times 2 = 10$  ns.
2. Non-pipelined =  $n \times T = 100 \times 10 = 1000$  ns.
3.  $t_{cycle} = \max = 2$  ns (sab barabar).
4. Pipelined =  $(5 + 100 - 1) \times 2 = 104 \times 2 = 208$  ns.
5. Speedup =  $1000 / 208 = 4.81$  (ideal 5;  $n$  bada ho to aur 5 ke paas).
6. Latency =  $5 \times 2 = 10$  ns (same as non-pipelined). Throughput =  $1 \text{ instruction} / 2 \text{ ns} = 0.5 \times 10^9$  instructions/s = 500 million per second.

**Answer: 1000 ns vs 208 ns, speedup  $\approx 4.81$ , latency 10 ns, throughput 1 per 2 ns.**

### Solved example 2 (unequal stages + overhead, exam-level):

Stage delays are 250, 350, 150, 300, 200 ps and each pipeline register adds 20 ps. For 1000 instructions find the non-pipelined time, pipelined time, speedup, the limit of speedup for very many instructions, and the latency of one instruction.

1.  $T = 250 + 350 + 150 + 300 + 200 = 1250$  ps.
2. Non-pipelined =  $1000 \times 1250 = 1,250,000$  ps =  $1.25$   $\mu$ s.
3. Slowest stage = 350 ps.  $t_{\text{cycle}} = 350 + 20 = 370$  ps.
4. Pipelined =  $(5 + 1000 - 1) \times 370 = 1004 \times 370 = 371,480$  ps  $\approx 0.371$   $\mu$ s.
5. Speedup =  $1,250,000 / 371,480 = 3.36$ .
6. Very many instructions: speedup  $\rightarrow T / t_{\text{cycle}} = 1250 / 370 = 3.38$  (5 nahi, kyunki stages unbalanced + overhead).
7. Latency =  $5 \times 370 = 1850$  ps: non-pipelined ke 1250 ps se **zyada**! Har instruction ko har stage mein 370 ps milta hai chahe use 150 chahiye.

**Answer: 1.25  $\mu$ s vs 0.371  $\mu$ s; speedup  $\approx 3.36$  (limit 3.38); latency 1850 ps.**

### Solved example 3 (P&H textbook, single-cycle vs pipelined):

Stage delays 200 (IF), 100 (ID), 200 (EX), 200 (MEM), 100 (WB) ps. Compare 3 lw instructions on the single-cycle (800 ps) and pipelined machine. What is the speedup for a very long program? Why not 5?

1. Single-cycle clock =  $200 + 100 + 200 + 200 + 100 = 800$  ps. 3 lw =  $3 \times 800 = 2400$  ps.
2. Pipelined clock = slowest stage = 200 ps. 3 lw =  $(5 + 3 - 1) \times 200 = 7 \times 200 = 1400$  ps.
3. Long program: speedup  $\rightarrow 800 / 200 = 4$ .
4. Not 5 because the stages are unbalanced (ID and WB need only 100 ps but get 200 ps). Register overhead and hazards reduce it further.
5. One lw latency =  $5 \times 200 = 1000$  ps (more than 800!), but one lw finishes every 200 ps.

**Answer: 2400 ps vs 1400 ps; long-run speedup 4, not 5, because of unbalanced stages.**

**Galti mat karna:** (1) Clock = average stage delay lena: clock = **maximum** (+ overhead). (2) Overhead sirf ek baar jodna: har cycle mein judta hai. (3) "Latency kam hoti hai" likhna: latency same ya zyada; throughput badhta hai.

**Trick: "Slowest + overhead = cycle; k + n - 1 cycles; ideal speedup = k."**

**Ek line mein:** Pipelined time =  $(k + n - 1) \times (\text{slowest stage} + \text{overhead})$ ; speedup  $\rightarrow k$  sirf balanced, bina overhead, bina hazards; latency nahi ghatti, throughput badhta hai.

## 14.6 Hazards Preview

**Pehle socho:** Laundry mein teen tarah ki problem ho sakti hai: (1) washer-dryer combo ek hi machine hai, do loads ek saath nahi (resource ki kami); (2) load B ka "fold" tabhi ho sakta hai jab A ka dupatta bhi aa jaaye jo abhi dryer mein hai (data ka intezaar); (3) pata nahi agla load kaunsa daalna hai jab tak koi decide na kare (decision ka intezaar). CPU mein yahi teen **hazards** hain.

**Definition:** A **hazard** is a situation in which the next instruction cannot execute in its planned clock cycle. **Structural hazard:** two instructions need the same hardware resource in the same cycle. **Data hazard:** an instruction needs a result that an earlier instruction has not yet produced. **Control hazard:** the next instruction to fetch depends on a branch whose outcome is not yet known.

**Mushkil shabd, aasaan matlab:**

### stall / bubble

Pipeline mein ek cycle ka "khaali" slot, instruction rok diya gaya. Detail Unit 15.

### forwarding

Result ko register file mein likhe jaane se pehle hi seedha agle instruction ke ALU tak bhej dena. Detail Unit 15.

### split I / D memory

Instruction aur data ke liye alag memories (alag L1 caches), isliye IF aur MEM ek saath chal sakte hain.

**Teen hazards, ek ek example ke saath:**

1. **Structural:** agar ek hi memory ho to CC4 mein lw ka MEM aur 4th instruction ka IF dono memory maangenge. MIPS fix: alag instruction aur data memory; register file pehle aadhe mein likhe, doosre mein padhe.
2. **Data:** add \$s0, \$t0, \$t1 phir sub \$t2, \$s0, \$t3. sub ko \$s0 CC3 mein (ID) chahiye, lekin add use CC5 (WB) mein likhega. Fix: forwarding; load ke baad 1 stall.
3. **Control:** beq ke baad kaunsa instruction fetch karein? Branch ka faisla baad mein hota hai, IF tab tak ruk nahi sakta. Fix: stall, predict (not-taken), delayed branch.
4. RISC ka load/store rule hazards ko pakadna aasaan banata hai: sirf lw / sw memory chhoote hain, aur sab registers ID mein padhte hain, WB mein likhte hain.

### Solved example (write the answer like this, 3 marks):

*Name and explain the three types of pipeline hazards with one example each.*

1. **Structural:** two instructions need the same resource in the same cycle, e.g. a single memory needed by lw (MEM) and a later instruction (IF) in CC4. Solved by separate instruction and data memories.

2. **Data:** an operand is not ready, e.g. add \$s0, \$t0, \$t1 followed by sub \$t2, \$s0, \$t3. Solved by forwarding (and stalls).
3. **Control:** the next PC depends on an unresolved branch, e.g. beq. Solved by stalling, branch prediction or delayed branch.

**Answer:** structural (resource), data (operand not ready), control (branch decides next PC).

**Galti mat karna:** Branch ki problem ko data hazard mat bolo, woh **control** hazard hai. Aur MIPS mein single-memory structural hazard nahi hota, kyunki memories alag hain.

**Trick: S-D-C = Seat, Data, Choice:** seat (hardware) nahi, data nahi aaya, choice (branch) nahi pata.

**Ek line mein:** Hazard = agla instruction apne time pe nahi chal sakta; structural (hardware), data (value ready nahi), control (branch).

**Exam mein is unit se kya aata hai → kaise solve karein**

| Question type                                     | Method (steps)                                                                                                                                 |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Laundry analogy, sequential vs pipelined time     | Sequential = loads × stages × stage time. Pipelined = (stages + loads – 1) × stage time. Latency same, throughput badhta.                      |
| Cycles for n instructions in a k-stage pipeline   | $k + (n - 1)$ . Example: 5 stages, 100 instructions → 104.                                                                                     |
| Draw a pipeline diagram / which stage in CC x     | Rows = instructions, columns = CC; row i starts IF in CC i; stage s in cycle $i + s - 1$ .                                                     |
| Non-pipelined vs pipelined time, speedup          | 1. $T$ = sum of stages. 2. $n \times T$ . 3. $t_{\text{cycle}}$ = max stage + overhead. 4. $(k + n - 1) \times t_{\text{cycle}}$ . 5. Divide.  |
| Ideal speedup / why not k                         | Ideal = k (balanced, no overhead, no hazards, large n). Real less: unbalanced stages, register overhead, hazards, fill time.                   |
| Latency vs throughput (define + compute)          | Latency = $k \times t_{\text{cycle}}$ (can exceed single-cycle). Throughput = 1 instruction per $t_{\text{cycle}}$ .                           |
| Five stages + pipeline registers (draw / explain) | IF ID EX MEM WB table (work + hardware); registers IF/ID, ID/EX, EX/MEM, MEM/WB carry data, register numbers, control; WB wire goes back left. |
| Name the three hazards with examples              | Structural (one memory, CC4), data (add \$s0 → sub uses \$s0), control (beq).                                                                  |

## Unit 15 · Pipeline Hazards

**Is unit ki kahani:** Unit 14 mein pipeline ideal tha: har cycle ek naya instruction, koi rukawat nahi. Asli programs mein instructions ek doosre pe depend karte hain (add ka result sub use karta hai), hardware share hota hai, aur branch ke baad pata nahi agla instruction kaunsa. In situations ko **hazards** kehte hain. Is unit mein: hazards ke types, **stall** (rukna), **forwarding** (result jaldi pahunchana), **load-use** ka 1 stall, **compiler scheduling** (order badalna), branch handling, aur  $CPI = 1 + \text{stalls}$  wala formula. Saari cycle tables site ke pipeline tool (PIPE.schedule) ke convention se bani hain: register file pehle aadhe cycle mein likhta hai, doosre aadhe mein padhta hai; instruction aur data memory alag. (Class slides nahi thi; content Patterson & Hennessy se.)

### 15.1a Hazard Taxonomy and the Structural Hazard

**Pehle socho:** Flat mein 4 log, **ek hi bathroom**. Subah 8 baje do log ek saath ready: ek ko rukna padega. Ye hardware (resource) ki kami hai = **structural hazard**. Agar tumhe dost ki notes chahiye jo usne abhi likhi hi nahi = **data hazard**. Agar pata hi nahi aaj college jaana hai ya nahi jab tak notice na aaye = **control hazard**. Teenon mein result ek hi: **kaam apne time pe shuru nahi ho sakta**.

**Definition:** A **hazard** is a situation in which the next instruction cannot execute in its planned clock cycle. A **structural hazard** occurs when two instructions need the same hardware resource in the same cycle; a **data hazard** occurs when an instruction needs an operand that an earlier instruction has not yet produced; a **control hazard** occurs when the address of the next instruction depends on a branch that has not yet been resolved.

**Mushkil shabd, aasaan matlab:**

**resource**

Koi hardware hissa: memory, ALU, register file port.

**stall (bubble)**

Instruction ko ek cycle rok dena; us cycle pipeline ka woh stage khaali (nop) chalta hai. Table mein "stall" likhte hain.

**unified vs split memory**

Unified = instructions aur data ek hi memory mein (von Neumann). Split = alag instruction memory / cache aur data memory / cache (Harvard style, Unit 9). MIPS pipeline split use karta hai.

**Kaise kaam karta hai:**

| Hazard | Wajah | Example | Fix |
|--------|-------|---------|-----|
|--------|-------|---------|-----|

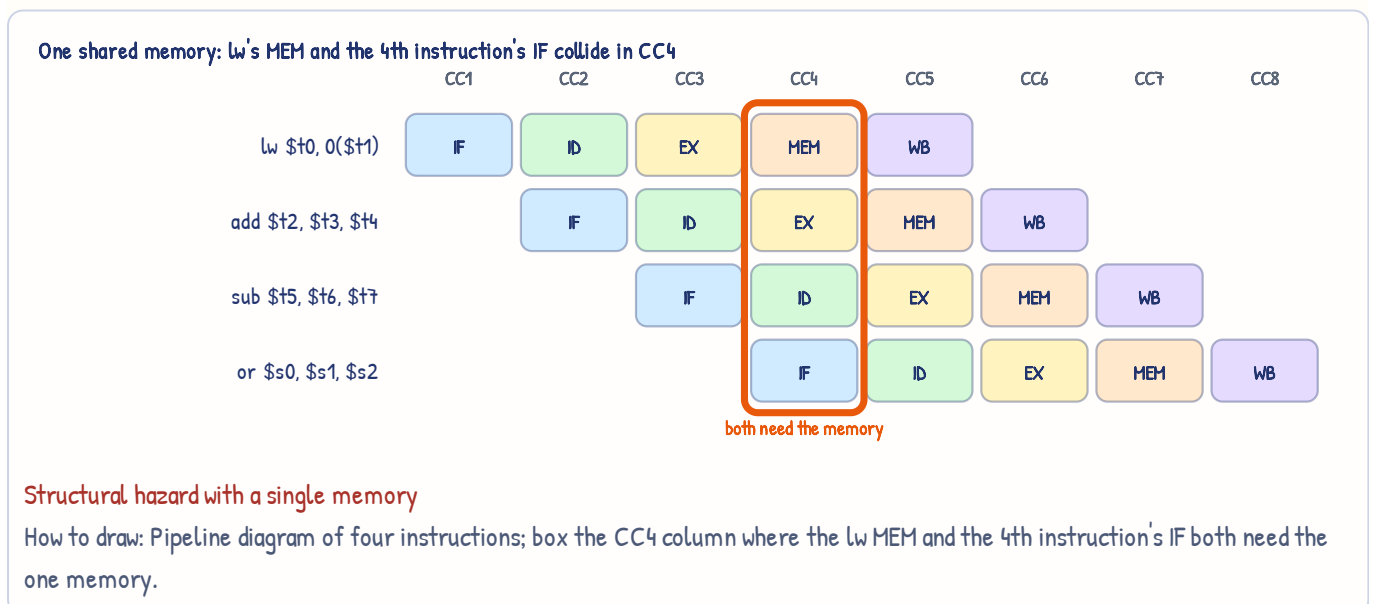
|                   |                                        |                              |                                                                                           |
|-------------------|----------------------------------------|------------------------------|-------------------------------------------------------------------------------------------|
| <b>Structural</b> | do instructions, ek resource, ek cycle | single memory for IF and MEM | resource duplicate karo (split I / D memory), register file write / read halves, ya stall |
| <b>Data</b>       | operand abhi bana nahi                 | add \$s0 → sub uses \$s0     | forwarding, stall (load-use), compiler scheduling                                         |
| <b>Control</b>    | agla PC branch pe depend               | beq                          | stall, branch in ID, predict not-taken / dynamic prediction, delayed branch               |

### Structural hazard ka example (agar memory ek hi ho):

1. lw CC1 mein IF, CC4 mein MEM (data memory padhni hai).
2. Chautha instruction (or) CC4 mein IF karna chahta hai (instruction memory padhni hai).
3. Ek hi memory hai → CC4 mein dono ko chahiye → **conflict** (yellow cells).
4. Bina fix ke: or ka IF ek cycle rukega (stall), sab kuch ek cycle late.
5. MIPS ka fix: instruction aur data ke liye alag memory (split L1-I / L1-D cache). Phir CC4 mein lw data memory use karta hai, or instruction memory: koi jhagda nahi.

| Instruction          | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 | CC8 |
|----------------------|-----|-----|-----|-----|-----|-----|-----|-----|
| lw \$t0, 0(\$t1)     | IF  | ID  | EX  | MEM | WB  |     |     |     |
| add \$t2, \$t3, \$t4 |     | IF  | ID  | EX  | MEM | WB  |     |     |
| sub \$t5, \$t6, \$t7 |     |     | IF  | ID  | EX  | MEM | WB  |     |
| or \$s0, \$s1, \$s2  |     |     |     | IF  | ID  | EX  | MEM | WB  |

Register file pe bhi WB (likhna) aur ID (padhna) ek hi cycle mein ho sakte hain; isliye register file pehle aadhe cycle mein likhta hai aur doosre aadhe mein padhta hai, aur usme 2 read ports + 1 write port hain.



**Diagram kaise padhein:** 4 instructions ki pipeline diagram, CC1 to CC8. CC4 wala column box kiya hua hai: usme lw ka MEM aur or ka IF dono hain, dono ko memory chahiye. Neeche label "both need the memory". Baaki columns mein koi do instructions ek hi resource nahi maang rahe.

### Solved example (write the answer like this, 5 marks):

Classify pipeline hazards with one example and one solution each. Show a structural hazard on a pipeline diagram.

1. **Structural:** two instructions need the same resource in one cycle. Example: with one memory, lw's MEM and the 4th instruction's IF both occur in CC4. Solution: separate instruction and data memories.
2. **Data:** an instruction needs a result not yet written. Example: add \$s0, \$t0, \$t1 ; sub \$t2, \$s0, \$t3. Solution: forwarding (stall for load-use).
3. **Control:** next PC unknown until a branch resolves. Example: beq. Solution: predict not-taken / delayed branch / stall.
4. Draw the 4-instruction table above and box CC4.

Answer: structural = resource, data = operand, control = branch; diagram with the CC4 conflict.

**Galti mat karna:** Resource conflict ko data hazard mat bolo. Aur ye mat likho ki MIPS 5-stage pipeline mein IF-MEM structural hazard hota hai: split memories ki wajah se nahi hota (sirf "agar ek memory ho to" wala example hai).

**Ek line mein:** Hazard = instruction apne cycle mein nahi chal sakta; structural (resource), data (operand), control (branch); single memory ka IF-MEM conflict split I / D memory se hat jaata hai.

## 15.1b Control Hazards: Branch Handling and CPI with Stalls

**Pehle socho:** Highway pe aage do raaste (fork) hain, aur board tab dikhega jab tum fork ke paas pahunchoge. Teen options: (1) **ruk jao** jab tak board na dikhe (stall); (2) **andaza lagao** ki seedha jaana hai aur chalte raho; galat nikla to wapas aao (predict not-taken + flush); (3) fork se pehle ek aisa kaam kar lo jo dono raaston mein karna hi hai, jaise petrol bharwana (delayed branch). CPU ko beq ke baad agla instruction **turant fetch karna hai**, lekin beq ka faisla baad ke stage mein hota hai.

**Definition:** A **control hazard** (branch hazard) arises because the pipeline must fetch the next instruction before the branch outcome is known. The **branch penalty** is the number of cycles lost. It is handled by **stalling** until the branch resolves, **predicting not-taken** (fetch sequentially and flush the wrong instructions if the branch is taken), **dynamic prediction**, or the **delayed branch** (the instruction after the branch, in the delay slot, is always executed).

**Mushkil shabd, aasaan matlab:**

**branch resolved**

Pata chal gaya ki branch lega ya nahi (taken / not taken) aur target kya hai.

**taken / not taken**

Taken = registers barabar, PC = target. Not taken = PC + 4 pe hi chalo.

### flush

Galti se fetch kiye instruction ko pipeline se nikaal dena (usko nop bana dena), taaki woh kuch likhe nahi.

### branch penalty

Branch ki wajah se kitne cycles barbaad hue. Branch ID mein decide ho to 1; MEM mein decide ho to 3.

### delay slot

Branch ke turant baad wali jagah. Delayed branch mein ye instruction hamesha chalta hai, branch le ya na le.

### dynamic prediction

Hardware pichhli baar ka result yaad rakhta hai (1-bit ya 2-bit counter, branch target buffer) aur usi hisaab se guess karta hai. Loops mein bahut sahi guess karta hai.

### Kaise kaam karta hai:

1. **Branch kab decide hota hai:** basic P&H pipeline mein compare EX mein aur PC update MEM mein → beq ke peeche 3 instructions galat raaste ke fetch ho chuke hote hain (penalty 3). Compare aur target adder ko ID mein le aao → penalty sirf 1 cycle.
2. **Option 1, stall:** beq ke baad ek cycle kuch fetch mat karo; ID mein faisla hote hi sahi instruction fetch karo. Har branch pe 1 cycle jaata hai.
3. **Option 2, predict not-taken:** maan lo branch nahi lega, PC + 4 wala fetch karte raho. Not taken → 0 loss. Taken → galat instruction flush, target fetch → 1 cycle loss (branch in ID).
4. **Option 3, delayed branch (MIPS):** branch ke baad wala instruction hamesha chalta hai. Compiler waha koi aisa instruction rakhta hai jo dono case mein chahiye (branch se pehle wala independent instruction). Kuch na mile to nop.
5. **Option 4, dynamic prediction:** history ke basis pe taken / not-taken guess, galat guess pe flush.

**Stall (branch decided in ID):** agla instruction CC2 ki jagah CC3 mein fetch hota hai.

| Instruction        | CC1 | CC2   | CC3 | CC4 | CC5 | CC6 | CC7 |
|--------------------|-----|-------|-----|-----|-----|-----|-----|
| beq \$t1, \$t2, L  | IF  | ID    | EX  | MEM | WB  |     |     |
| next (PC + 4 or L) |     | stall | IF  | ID  | EX  | MEM | WB  |

**Predict not-taken, branch NOT taken:** koi loss nahi.

| Instruction                   | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 |
|-------------------------------|-----|-----|-----|-----|-----|-----|
| beq \$t1, \$t2, L (not taken) | IF  | ID  | EX  | MEM | WB  |     |
| add (PC + 4, correct)         |     | IF  | ID  | EX  | MEM | WB  |

**Predict not-taken, branch TAKEN:** add (galat) flush, target CC3 mein fetch: 1 cycle loss.

| Instruction               | CC1 | CC2 | CC3   | CC4 | CC5 | CC6 | CC7 |
|---------------------------|-----|-----|-------|-----|-----|-----|-----|
| beq \$t1, \$t2, L (taken) | IF  | ID  | EX    | MEM | WB  |     |     |
| add (PC + 4, wrong)       |     | IF  | flush |     |     |     |     |
| L: target                 |     |     | IF    | ID  | EX  | MEM | WB  |

**Basic pipeline (branch decided in MEM), taken:** 3 galat instructions flush, penalty 3.

| Instruction               | CC1 | CC2 | CC3 | CC4 | CC5   | CC6 | CC7 | CC8 | CC9 |
|---------------------------|-----|-----|-----|-----|-------|-----|-----|-----|-----|
| beq \$t1, \$t2, L (taken) | IF  | ID  | EX  | MEM | WB    |     |     |     |     |
| wrong 1                   |     | IF  | ID  | EX  | flush |     |     |     |     |
| wrong 2                   |     |     | IF  | ID  | flush |     |     |     |     |
| wrong 3                   |     |     |     | IF  | flush |     |     |     |     |
| L: target                 |     |     |     |     | IF    | ID  | EX  | MEM | WB  |

**Delayed branch ka example:**

|                      |                                                |
|----------------------|------------------------------------------------|
| Before:              | After (delay slot filled):                     |
| add \$s1, \$s2, \$s3 | beq \$t1, \$t2, L                              |
| beq \$t1, \$t2, L    | add \$s1, \$s2, \$s3 # delay slot, always runs |

add ko branch ke baad le jaana safe hai kyunki add \$t1 / \$t2 ko nahi badalta, aur add dono raaston mein waise bhi chalna tha. Ab us slot mein waste nahi.

**Formula (CPI with stalls):**  $CPI = 1 + (\text{stall cycles per instruction})$

Branch stalls per instruction = (fraction of branches)  $\times$  (fraction that pays the penalty)  $\times$  (penalty cycles)

Time =  $N \times CPI \times t_{\text{cycle}}$  · speedup over non-pipelined (balanced k stages) =  $k / CPI$

Always stall: har branch penalty deta hai. Predict not-taken: sirf taken branches penalty dete hain.

**Solved example 1 (easy):**

20 % of instructions are branches. The pipeline always stalls 1 cycle on a branch. Find the CPI.

1. Stalls per instruction =  $0.20 \times 1 = 0.20$ .
2.  $CPI = 1 + 0.20 = 1.20$ .

**Answer: CPI = 1.20.**

**Solved example 2 (exam-level):**

20 % of instructions are branches and 60 % of branches are taken. The clock is 200 ps. Compare the CPI and the time for  $10^6$  instructions for: (a) predict not-taken with branch in ID (penalty 1), (b) predict not-taken with branch in MEM (penalty 3).

1. Taken branches per instruction =  $0.20 \times 0.60 = 0.12$ .
2. (a) Stalls =  $0.12 \times 1 = 0.12 \rightarrow CPI = 1.12$ .

3. (a) Time =  $10^6 \times 1.12 \times 200 \text{ ps} = 224 \times 10^6 \text{ ps} = 224 \mu\text{s}$ .
4. (b) Stalls =  $0.12 \times 3 = 0.36 \rightarrow \text{CPI} = 1.36$ .
5. (b) Time =  $10^6 \times 1.36 \times 200 \text{ ps} = 272 \times 10^6 \text{ ps} = 272 \mu\text{s}$ .
6. Comparison: branch ko ID mein decide karne se  $272 \rightarrow 224 \mu\text{s}$ ;  $272 / 224 = 1.21$  guna tez.

**Answer: (a) CPI 1.12, 224  $\mu\text{s}$ ; (b) CPI 1.36, 272  $\mu\text{s}$ .**

**Galti mat karna:** (1) Predict not-taken mein not-taken branches pe bhi penalty jod dena: unka loss 0 hai. (2) Branch hazard ko data hazard bolna. (3) Delayed branch mein slot ka instruction "sirf taken pe" chalta hai sochna: woh hamesha chalta hai.

**Trick: "Ruko, Andaza, Pehle-kaam"** = stall, predict not-taken, delayed branch. Penalty: ID mein faisla = 1, MEM mein = 3.

**Ek line mein:** Branch ka faisla der se hota hai; stall (har branch 1 cycle), predict not-taken (sirf taken pe 1), delayed branch (slot hamesha chalta);  $\text{CPI} = 1 + \text{branch\%} \times \text{penalty wala\%} \times \text{penalty}$ .

## 15.2 RAW, WAR, WAW: Why Only RAW Matters In-Order

**Pehle socho:** Whiteboard pe ek number likha hai. (1) Tumhe woh number **padhna** hai jo dost abhi **Likhne** wala hai: tumhe intezaar karna hoga (RAW). (2) Tum pehle padhoge, phir dost mitaake naya likhega: agar dost jaldi kar de to tum galat padhoge (WAR). (3) Dono likhenge: aakhir mein jo baad wala hai uska number rehna chahiye (WAW). Hamare 5-stage pipeline mein sab **ek line mein, ek hi order mein** chalte hain, isliye sirf pehla wala (RAW) problem karta hai.

**Definition:** **RAW (read after write, true dependence):** a later instruction reads a register that an earlier one writes. **WAR (write after read, anti-dependence):** a later instruction writes a register that an earlier one reads. **WAW (write after write, output dependence):** two instructions write the same register. In the in-order 5-stage MIPS pipeline only RAW causes hazards.

**Mushkil shabd, aasaan matlab:**

**dependence**

Do instructions ka ek hi register se rishta, jisse unka order badalna risky hai.

**in-order**

Instructions program ke order mein hi stages se guzarte hain; koi aage nahi nikalta.

**register renaming**

Out-of-order CPUs mein WAR / WAW hataane ki technique: alag physical register de do, naam ka takraav khatam.

### Kaise pehchaanein (I1 pehle, I2 baad mein):

1. **RAW:** I1: add \$s0, \$t0, \$t1; I2: sub \$t2, \$s0, \$t3. I2 padhta hai \$s0, jo I1 likhta hai.
2. **WAR:** I1: sub \$t2, \$s0, \$t3; I2: add \$s0, \$t4, \$t5. I2 likhta hai \$s0, jo I1 padhta hai.
3. **WAW:** I1: add \$s0, \$t0, \$t1; I2: sub \$s0, \$t2, \$t3. Dono \$s0 likhte hain.
4. **Sirf RAW kyun:** har instruction registers **ID (stage 2)** mein padhta hai aur **WB (stage 5)** mein likhta hai, aur order same rehta hai. Baad wala instruction kabhi pehle wale se pehle WB tak nahi pahunch sakta → WAW nahi. Baad wale ka WB (stage 5) pehle wale ke ID (stage 2) ke baad hi aata hai → WAR nahi.
5. WAR / WAW sirf out-of-order ya alag-alag length ke pipelines mein problem hain; wahan register renaming fix hai.

### Solved example (MCQ type):

Classify: (a) add \$s0, \$t0, \$t1; sub \$t2, \$s0, \$t3. (b) lw \$t0, 0(\$s1); addi \$s1, \$s1, 4 (dependence on \$s1).

1. (a) I1 likhta \$s0, I2 padhta \$s0 → read after write → **RAW**.
2. (b) I1 (lw) \$s1 padhta hai (base address), I2 (addi) \$s1 likhta hai → write after read → **WAR**. In-order pipeline mein ye koi hazard nahi.

**Answer: (a) RAW, (b) WAR (harmless in the 5-stage in-order pipeline).**

**Galti mat karna:** Naam "kya pehle hua" ke order mein hai: RAW = pehle write (I1), phir read (I2). add \$s0 → sub uses \$s0 ko WAR mat bolo.

**Ek line mein:** RAW = true dependence, sirf yahi 5-stage in-order MIPS mein hazard; WAR / WAW nahi hote kyunki sab ID mein padhte, WB mein likhte, same order mein.

## 15.3 The Naive Fix: Stalls (Bubbles)

**Pehle socho:** Tiffin dabba line mein ek dabbawala ko doosre ka label chahiye jo abhi chhapa nahi. Seedha tareeka: **woh ruk jaaye**, aur uske peeche wale bhi (line hai, koi overtake nahi karta). Line mein jo khaali jagah banti hai, use **bubble** kehte hain. Kaam sahi hota hai, par time barbaad hota hai.

**Definition:** A **stall** (pipeline bubble) is a cycle in which an instruction is held in its stage, together with all instructions behind it, until its operand is available; the empty slot that travels down the pipeline behaves like a **nop**. Without forwarding, a dependent instruction must wait until the producer has written the register file in WB.

**Mushkil shabd, aasaan matlab:**

**nop**

"No operation": aisa instruction jo kuch nahi karta. Bubble = hardware ka banaya nop.

### distance

Producer aur consumer ke beech kitne instructions ka gap. Turant agla = distance 1.

### hazard detection unit

ID stage mein ek circuit jo dependence pakad ke stall karta hai.

### interlock

Hardware khud rokna (stall). Early MIPS ("Microprocessor without Interlocked Pipeline Stages") ye kaam compiler pe chhodta tha: compiler nop daalta tha.

### Kaise kaam karta hai (no forwarding):

1. add \$s0 apna result CC5 (WB) mein register file mein likhta hai, pehle aadhe cycle mein.
2. sub ko \$s0 ID mein padhna hai. Normally sub ka ID CC3 mein hota. Wahan \$s0 purana hai → galat.
3. sub ko tab tak ID mein rok do jab tak padhai CC5 ke doosre aadhe mein na ho (wahan nayi value mil jaayegi).
4. CC3 → CC5 = 2 cycles extra → **2 stalls**. sub ka EX CC4 ki jagah CC6 mein.
5. Peeche wala and bhi ruka (IF mein), sab 2 cycle late. Total 7 ki jagah 9 cycles.
6. Rule: distance 1 → 2 stalls; distance 2 → 1 stall; distance 3 ya zyada → 0 stalls.
7. **Hardware stall kaise karta hai:** (a) PC aur IF/ID register ko update nahi hone deta (wahi instructions dobara fetch / decode); (b) ID/EX mein jaane wale saare control signals 0 kar deta hai → EX, MEM, WB mein kuch nahi likha jaata = bubble.

### add → sub (distance 1), no forwarding: 9 cycles, 2 stalls

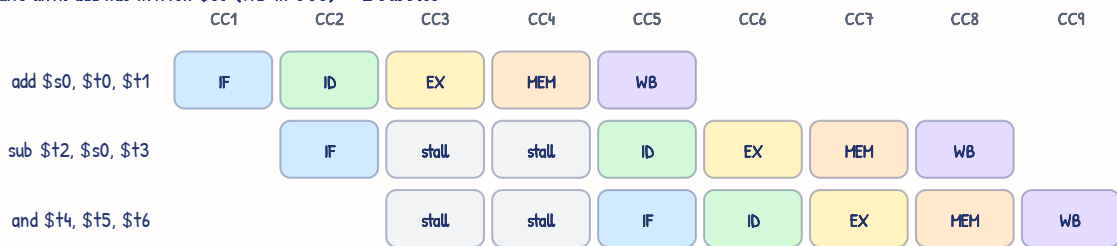
| Instruction          | CC1 | CC2 | CC3 | CC4   | CC5   | CC6 | CC7 | CC8 | CC9 |
|----------------------|-----|-----|-----|-------|-------|-----|-----|-----|-----|
| add \$s0, \$t0, \$t1 | IF  | ID  | EX  | MEM   | WB    |     |     |     |     |
| sub \$t2, \$s0, \$t3 |     | IF  | ID  | stall | stall | EX  | MEM | WB  |     |
| and \$t4, \$t5, \$t6 |     |     | IF  | stall | stall | ID  | EX  | MEM | WB  |

Table padhna: sub ki row mein "ID stall stall" = sub teen cycles ID mein hi ruka hai, aur aakhri (CC5) mein sahi \$s0 padhta hai. and IF mein ruka hai. (D15.3a figure stalls ko ID se pehle dikhata hai; count same hai, 2.)

### add → (independent or) → sub (distance 2), no forwarding: 8 cycles, 1 stall

| Instruction          | CC1 | CC2 | CC3 | CC4 | CC5   | CC6 | CC7 | CC8 |
|----------------------|-----|-----|-----|-----|-------|-----|-----|-----|
| add \$s0, \$t0, \$t1 | IF  | ID  | EX  | MEM | WB    |     |     |     |
| or \$t7, \$t5, \$t6  |     | IF  | ID  | EX  | MEM   | WB  |     |     |
| sub \$t2, \$s0, \$t3 |     |     | IF  | ID  | stall | EX  | MEM | WB  |

No forwarding: sub waits until add has written \$s0 (WB in CC5) — 2 bubbles



### Stalls (bubbles) without forwarding

How to draw: Pipeline diagram: add normal; sub fetched in CC2 but held, with two stall boxes, so its ID lines up with add's WB in CC5; every later instruction shifts right too.

**Diagram kaise padhein:** add normal staircase mein (CC1 to CC5). sub CC2 mein fetch hota hai, phir do "stall" boxes (CC3, CC4), aur ID CC5 mein: bilkul add ke WB wale column mein (same cycle likho aur padho). and bhi do cycle right khisak gaya. Last WB CC9 mein = 9 cycles.

**Formula:**  $CPI = 1 + (\text{stall cycles per instruction}) \cdot \text{Total cycles} = (k + n - 1) + \text{total stalls}$

### Solved example 1 (easy):

No forwarding. add \$s0, \$t0, \$t1; sub \$t2, \$s0, \$t3; and \$t4, \$t5, \$t6. How many stalls and total cycles?

1. sub ko \$s0 chahiye, distance 1 → 2 stalls.
2. and independent hai, lekin sub ke peeche hai, to woh bhi 2 late (koi extra stall nahi).
3. Ideal =  $5 + (3 - 1) = 7$ . Total =  $7 + 2 = 9$ .

**Answer: 2 stalls, 9 cycles.**

### Solved example 2 (CPI, exam-level):

No forwarding. In a program 30 % of instructions use the result of the immediately previous instruction and another 10 % use the result of the instruction two before. Find the CPI and the execution time of  $10^6$  instructions at 200 ps.

1. Distance 1 → 2 stalls:  $0.30 \times 2 = 0.60$  stall cycles per instruction.
2. Distance 2 → 1 stall:  $0.10 \times 1 = 0.10$ .
3.  $CPI = 1 + 0.60 + 0.10 = 1.70$ .
4. Time =  $10^6 \times 1.70 \times 200 \text{ ps} = 340 \times 10^6 \text{ ps} = 340 \mu\text{s}$  (ideal CPI 1 hota to 200  $\mu\text{s}$ ).

**Answer: CPI = 1.70, time = 340  $\mu\text{s}$ . Isliye forwarding chahiye (15.4).**

**Galti mat karna:** (1) 3 stalls ginna: register file ka "pehle likho, phir padho" bhoool gaye. (2) Sirf dependent instruction ko rokna aur peeche walon ko aage nikal jaane dena: in-order pipeline mein sab ke sab rukte hain.

**Trick:** Bina forwarding: **distance 1 → 2, distance 2 → 1, distance 3 → 0** (stalls + distance = 3).

**Ek line mein:** Stall = dependent instruction (aur peeche wale) rok do jab tak value register file mein na aaye; no forwarding mein back-to-back dependence = 2 bubbles; CPI = 1 + stalls per instruction.

## 15.4 Forwarding (Bypassing)

**Pehle socho:** Dost ne exam ka answer nikaal liya hai, lekin abhi register (notebook) mein likh raha hai. Tumhe notebook ka intezaar kyun karna? Woh **seedha bol de**, kaam ho gaya. CPU mein add ka result EX stage ke end mein ban jaata hai (CC3), WB (CC5) tak rukne ki zaroorat nahi. Pipeline register (EX/MEM ya MEM/WB) se **seedha ALU ke input pe** bhej do. Isse kehte hain forwarding.

**Definition: Forwarding** (bypassing) is a technique in which a result is sent directly from a pipeline register (EX/MEM or MEM/WB) to the ALU input of a later instruction, instead of waiting for it to be written into the register file. A **forwarding unit** compares register numbers and drives multiplexers (ForwardA, ForwardB) in front of the ALU.

**Mushkil shabd, aasaan matlab:**

### EX/MEM → EX

Producer abhi MEM stage mein hai (ek instruction aage). Uska ALU result EX/MEM register mein pada hai → consumer ke EX mein bhejo.

### MEM/WB → EX

Producer WB stage mein hai (do instructions aage). Result MEM/WB register mein → consumer ke EX mein bhejo.

### ForwardA / ForwardB

ALU ke do inputs (A = rs wala, B = rt wala) ke aage 3-input MUX ka select. 00 = register file, 10 = EX/MEM, 01 = MEM/WB.

### ID/EX.Rs

"ID/EX register mein jo instruction hai uska rs number." Dot ke pehle register ka naam, baad mein field.

**Kaise kaam karta hai:**

1. Har cycle forwarding unit EX wale instruction ke source register numbers (ID/EX.Rs, ID/EX.Rt) ko aage wale instructions ke destination (EX/MEM.Rd, MEM/WB.Rd) se compare karta hai.
2. **Match EX/MEM ke saath** (aur woh instruction sach mein register likhta ho, aur  $Rd \neq \$zero$ ): ForwardA = 10 → ALU ko EX/MEM wala result.
3. **Match MEM/WB ke saath** (aur EX/MEM ke saath match nahi): ForwardA = 01 → MEM/WB wala result.
4. Koi match nahi: ForwardA = 00 → register file wali value.

5. Rt ke liye bilkul same rules ForwardB pe.

6. Distance 3 ya zyada: register file khud handle karta hai (pehle aadhe mein likha, doosre mein padha).

```
EX hazard:  if (EX/MEM.RegWrite and EX/MEM.Rd != 0
               and EX/MEM.Rd == ID/EX.Rs)           ForwardA = 10
MEM hazard: if (MEM/WB.RegWrite and MEM/WB.Rd != 0
               and not (EX hazard for Rs)
               and MEM/WB.Rd == ID/EX.Rs)           ForwardA = 01
(same two conditions with Rt give ForwardB)
```

**Har condition ka WHY: RegWrite** check: sw / beq register nahi likhte, unka "Rd" bekaar hai. **Rd ≠ 0**: \$zero hamesha 0 rehna chahiye, usko likhne ki koshish forward nahi karni. **"not EX hazard"**: agar dono mein match ho to **sabse naya** (EX/MEM) result jeetna chahiye.

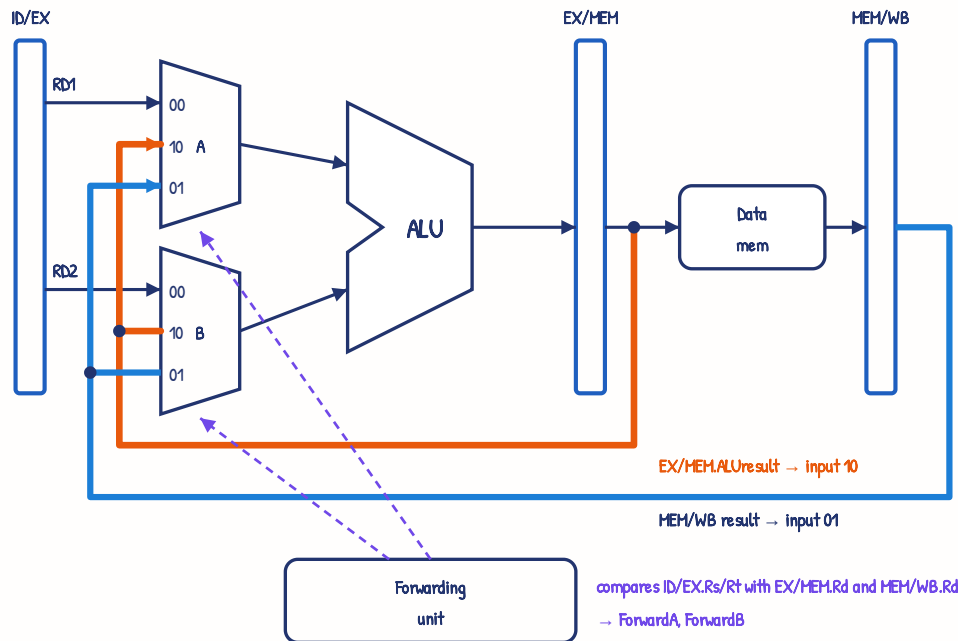
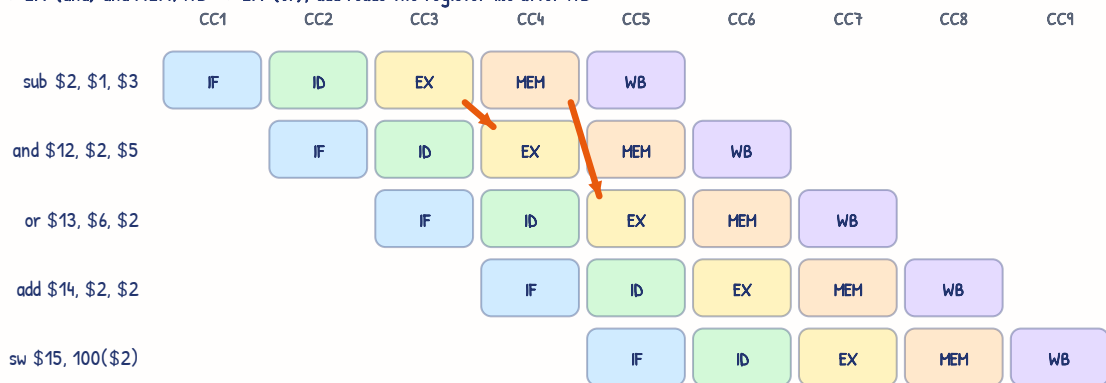
**P&H chain with forwarding: 9 cycles, 0 stalls** (\$1 = 10, \$3 = 4 maan lo, to sub ka result \$2 = 6)

| Instruction        | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 | CC8 | CC9 |
|--------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| sub \$2, \$1, \$3  | IF  | ID  | EX  | MEM | WB  |     |     |     |     |
| and \$12, \$2, \$5 |     | IF  | ID  | EX  | MEM | WB  |     |     |     |
| or \$13, \$6, \$2  |     |     | IF  | ID  | EX  | MEM | WB  |     |     |
| add \$14, \$2, \$2 |     |     |     | IF  | ID  | EX  | MEM | WB  |     |
| sw \$15, 100(\$2)  |     |     |     |     | IF  | ID  | EX  | MEM | WB  |

- CC3: sub EX mein  $10 - 4 = 6$  banata hai. Edge pe  $6 \rightarrow$  EX/MEM.
- CC4: and EX mein.  $EX/MEM.Rd = \$2 = ID/EX.Rs (\$2) \rightarrow$  **ForwardA = 10**, and ko 6 milta hai.
- CC5: or EX mein. or \$13, \$6, \$2 mein  $\$2 = rt$ . Ab sub MEM/WB mein  $\rightarrow$  **ForwardB = 01**. (EX/MEM mein ab and hai, uska Rd = \$12, match nahi.)
- CC5: add \$14, \$2, \$2 ID mein register file padhta hai. Usi cycle sub WB pehle aadhe mein  $\$2 = 6$  likh deta hai  $\rightarrow$  sahi value, forwarding nahi chahiye.
- CC6: sw \$15, 100(\$2) ID mein: \$2 already register file mein.
- Bina forwarding yahi chain 11 cycles leti (and aur or dono ruke): 2 cycles bache.

**Priority example:** add \$1, \$1, \$2; add \$1, \$1, \$3; add \$1, \$1, \$4. Teesre add ke EX (CC5) mein \$1 do jagah hai: MEM/WB mein (pehle add ka, purana) aur EX/MEM mein (doosre add ka, naya). Rule "not EX hazard" ki wajah se **EX/MEM** (ForwardA = 10) chuna jaata hai: sahi, kyunki program order mein sabse naya value wahi hai.

Forwarding: EX/MEM → EX (and) and MEM/WB → EX (or); add reads the register file after WB



### Forwarding paths EX/MEM → EX and MEM/WB → EX

How to draw: Pipeline diagram of the sub-and-or-add-sw chain with arrows from the END of sub's EX to the START of and's EX, and from the end of sub's MEM to or's EX; below, the ALU with two 3-input MUXes fed by the register file, EX/MEM and MEM/WB, controlled by the forwarding unit.

**Diagram kaise padhein:** Upar wali pipeline diagram mein orange arrows forwarding hain: sub ke EX ke end se and ke EX ke start tak (EX/MEM → EX), aur sub ke MEM ke end se or ke EX tak (MEM/WB → EX). Arrows hamesha aage (right / neeche) jaate hain, peeche time mein kabhi nahi. Neeche wala hardware: ID/EX se RD1, RD2 do 3-input MUX (A, B) mein, unke inputs 00 (register file), 10 (EX/MEM se orange wire), 01 (MEM/WB se wire). Neeche "Forwarding unit" box ID/EX.Rs / Rt ko EX/MEM.Rd aur MEM/WB.Rd se compare karke MUX selects (dashed) deta hai.

### Solved example 1 (MCQ type):

and \$12, \$2, \$5 directly follows sub \$2, \$1, \$3. Where does and's first ALU operand come from, and what is ForwardA? How many stalls with forwarding?

1. Distance 1: jab and EX mein hai, sub MEM mein hai, uska result EX/MEM register mein.
2. \$2 and ka rs hai → ALU input A → ForwardA = 10.
3. Result time pe mil gaya → 0 stalls.

**Answer:** from the EX/MEM pipeline register, ForwardA = 10, 0 stalls.

**Solved example 2 (write the answer like this, 5 marks):**

*Explain forwarding with the forwarding hardware and write the EX-hazard and MEM-hazard conditions.*

1. An ALU result is ready at the end of EX, two cycles before WB. Forwarding sends it from the EX/MEM or MEM/WB pipeline register directly to the ALU inputs of later instructions.
2. Hardware: two 3-input MUXes in front of the ALU (ForwardA, ForwardB; 00 register file, 10 EX/MEM, 01 MEM/WB) and a forwarding unit that compares ID/EX.Rs and ID/EX.Rt with EX/MEM.Rd and MEM/WB.Rd.
3. EX hazard: EX/MEM.RegWrite and EX/MEM.Rd  $\neq$  0 and EX/MEM.Rd = ID/EX.Rs  $\rightarrow$  ForwardA = 10.
4. MEM hazard: MEM/WB.RegWrite and MEM/WB.Rd  $\neq$  0 and not EX hazard and MEM/WB.Rd = ID/EX.Rs  $\rightarrow$  ForwardA = 01. (Same for Rt / ForwardB.)
5. Draw the sub-and-or-add-sw diagram with the two arrows.

**Answer:** as above; ALU-to-ALU dependences need 0 stalls with forwarding.

**Galti mat karna:** (1) Forwarding arrow ko peeche ki taraf (pichhle cycle mein) draw karna: data future se past mein nahi ja sakta. (2) Rd  $\neq$  0 check bhoolna. (3) Dono match hon to MEM/WB se forward karna: naya EX/MEM jeetta hai.

**Trick:** "1 aage  $\rightarrow$  EX/MEM (10), 2 aage  $\rightarrow$  MEM/WB (01), 3 aage  $\rightarrow$  register file (00)."

**Ek line mein:** Result ko register file ka intezaar kiye bina EX/MEM ya MEM/WB se seedha ALU tak bhejo; ALU  $\rightarrow$  ALU dependence mein 0 stall.

## 15.5 The Load-Use Hazard: Forwarding + One Stall

**Pehle socho:** add ka result EX ke end mein ban jaata hai, isliye agla instruction usse turant le sakta hai. Lekin lw ka data memory se aata hai, yaani MEM stage ke end mein, ek cycle aur baad. Agla instruction use apne EX ki shuruaat mein chahta hai, jo bilkul usi cycle mein hai jisme lw abhi memory padh raha hai. Data abhi bana hi nahi; forwarding time mein peeche nahi bhej sakta. Isliye ek cycle rukna hi padega.

**Definition:** A **load-use hazard** occurs when an instruction uses the register loaded by the immediately preceding lw. Because the loaded data is available only at the end of the MEM stage, one stall cycle is required even with forwarding; after the stall the value is forwarded from MEM/WB to EX.

**Mushkil shabd, aasaan matlab:**

### load-use

lw ke turant baad wala instruction usi register ko use kare. lw \$2, 20(\$1); and \$4, \$2, \$5.

### MemRead (ID/EX.MemRead)

ID/EX mein baitha instruction load hai, iski pehchaan.

### IF/ID.Rs, IF/ID.Rt

ID stage wale (agle) instruction ke source registers.

### Kaise kaam karta hai:

1. lw \$2, 20(\$1): CC3 EX mein address =  $\$1 + 20$  (jaise  $\$1 = 100 \rightarrow 120$ ). CC4 MEM mein Mem[120] padha, data CC4 ke end mein ready.
2. and \$4, \$2, \$5 normally CC4 mein EX karta: use CC4 ki **shuruaat** mein \$2 chahiye. Data abhi nahi  $\rightarrow$  impossible.
3. Hazard detection unit (ID mein, CC3): ID/EX.MemRead = 1 aur ID/EX.Rt (\$2) = IF/ID.Rs (\$2)  $\rightarrow$  stall.
4. Stall: PC aur IF/ID hold, ID/EX mein bubble (control 0). and ka EX CC5 mein chala jaata hai.
5. CC5 mein lw MEM/WB mein hai  $\rightarrow$  ForwardA = 01  $\rightarrow$  and ko \$2 mil jaata hai.
6. Bina forwarding yahi 2 stalls hote (WB tak wait). lw aur use ke beech ek independent instruction ho to 0 stall.

```
if (ID/EX.MemRead and
    (ID/EX.Rt == IF/ID.Rs or ID/EX.Rt == IF/ID.Rt))
    stall the pipeline (hold PC and IF/ID, insert a bubble in ID/EX)
```

### lw $\rightarrow$ add (with forwarding): 7 cycles, 1 stall

| Instruction          | CC1 | CC2 | CC3 | CC4   | CC5 | CC6 | CC7 |
|----------------------|-----|-----|-----|-------|-----|-----|-----|
| lw \$t0, 0(\$t1)     | IF  | ID  | EX  | MEM   | WB  |     |     |
| add \$t2, \$t0, \$t3 |     | IF  | ID  | stall | EX  | MEM | WB  |

### P&H example (with forwarding): 9 cycles, 1 stall

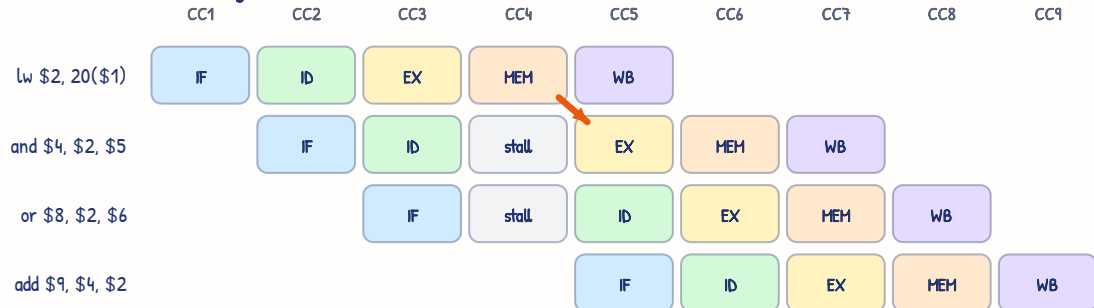
| Instruction       | CC1 | CC2 | CC3 | CC4   | CC5 | CC6 | CC7 | CC8 | CC9 |
|-------------------|-----|-----|-----|-------|-----|-----|-----|-----|-----|
| lw \$2, 20(\$1)   | IF  | ID  | EX  | MEM   | WB  |     |     |     |     |
| and \$4, \$2, \$5 |     | IF  | ID  | stall | EX  | MEM | WB  |     |     |
| or \$8, \$2, \$6  |     |     | IF  | stall | ID  | EX  | MEM | WB  |     |
| add \$9, \$4, \$2 |     |     |     |       | IF  | ID  | EX  | MEM | WB  |

and ID mein ek extra cycle ruka (stall), or IF mein. CC5: and ko \$2 MEM/WB se; or CC5 mein ID karta hai aur register file se \$2 padh leta hai (lw ne usi cycle likha); add \$9 ko \$4 MEM/WB se (and se), \$2 register file se.

### Ek independent instruction beech mein: 7 cycles, 0 stalls (MEM/WB $\rightarrow$ EX forwarding)

| Instruction          | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 |
|----------------------|-----|-----|-----|-----|-----|-----|-----|
| lw \$t0, 0(\$t1)     | IF  | ID  | EX  | MEM | WB  |     |     |
| or \$t7, \$t5, \$t6  |     | IF  | ID  | EX  | MEM | WB  |     |
| add \$t2, \$t0, \$t3 |     |     | IF  | ID  | EX  | MEM | WB  |

Load-use: one bubble, then MEM/WB → EX forwarding (cannot forward back in time)



**Load-use: 1 stall + forwarding**

How to draw: lw normal; and fetched next with one stall box before its EX; arrow from the end of lw's MEM to the start of and's EX; later instructions shift right by one.

**Diagram kaise padhein:** lw normal (CC1 to CC5). and ki row mein ID ke baad ek stall box (CC4), phir EX CC5 mein. Orange arrow lw ke MEM ke end (CC4) se and ke EX ki shuruat (CC5) tak: MEM/WB → EX forwarding. Dhyan do arrow time mein aage jaata hai; stall na hota to arrow seedha upar-neeche (same cycle) hota, jo impossible hai.

**Formula:** Load-use stalls per instruction = (fraction of loads) × (fraction of loads followed immediately by a use) × 1

CPI = 1 + load-use stalls + branch stalls

### Solved example 1 (easy):

With forwarding: lw \$t0, 0(\$t1); add \$t2, \$t0, \$t3. Stalls and total cycles? Without forwarding?

1. Load ke turant baad use → 1 stall (forwarding ke saath).
2. Ideal = 5 + (2 - 1) = 6. Total = 6 + 1 = 7.
3. Without forwarding: add ko WB tak rukna → 2 stalls → 6 + 2 = 8.

**Answer:** with forwarding 1 stall, 7 cycles; without 2 stalls, 8 cycles.

### Solved example 2 (CPI, exam-level):

A pipeline has full forwarding. 25 % of instructions are loads and 40 % of loads are immediately followed by an instruction that uses the loaded value. 15 % of instructions are branches, each costing 1 stall cycle. Find the CPI and the speedup over an unpipelined machine (balanced 5 stages).

1. Load-use stalls =  $0.25 \times 0.40 \times 1 = 0.10$ .
2. Branch stalls =  $0.15 \times 1 = 0.15$ .
3. ALU → ALU dependences: forwarding se 0.

4.  $CPI = 1 + 0.10 + 0.15 = 1.25$ .

5. Unpipelined: har instruction 5 stage-times. Pipelined: 1.25 stage-times per instruction. Speedup =  $5 / 1.25 = 4$ .

**Answer:  $CPI = 1.25$ , speedup = 4.**

### Solved example 3 (write the answer like this):

*Why can't forwarding alone resolve a load-use hazard?*

1. The loaded value is produced only at the end of the MEM stage.
2. The next instruction needs it at the beginning of its EX stage, which is the same cycle as the load's MEM.
3. Forwarding can only send a value forward in time, not backward, so the dependent instruction must be stalled one cycle; then the value is forwarded MEM/WB → EX.
4. Detection:  $ID/EX.MemRead$  and  $(ID/EX.Rt = IF/ID.Rs \text{ or } ID/EX.Rt = IF/ID.Rt)$ .

**Answer: data ready after MEM, needed at EX of the next instruction in the same cycle → 1 stall + MEM/WB forwarding.**

**Galti mat karna:** (1) "Forwarding se load-use stall bhi khatam" likhna: 1 stall bachta hai. (2) Forwarding arrow lw ke EX se draw karna: EX mein sirf address bana hai, data MEM ke end mein. (3) lw ke baad independent instruction ho to bhi stall lagana: tab 0 stall.

**Ek line mein:** Load ka data MEM ke end mein aata hai, agle instruction ko EX ki shuruat mein chahiye, isliye forwarding ke saath bhi 1 stall, phir MEM/WB → EX.

## 15.6 Compiler Scheduling (reordering to remove load-use stalls)

**Pehle socho:** Maggi banate waqt paani ubalne ka intezaar karte ho (stall). Smart tareeka: **usi waqt pyaaz kaat lo**, jo baad mein chahiye hi tha. Kaam ka order badla, result wahi, time bacha. Compiler bhi yahi karta hai: lw ke baad ka khaali slot kisi aise instruction se bhar deta hai jo **us load pe depend nahi karta**.

**Definition: Compiler (instruction) scheduling** is the reordering of independent instructions by the compiler so that a load is not immediately followed by an instruction that uses its result, removing load-use stalls without changing the program's result. All RAW, WAR and WAW dependences (through registers and memory) must be preserved.

**Mushkil shabd, aasaan matlab:**

### Independent instruction

Jo na load ka register padhta hai, na koi aisa register likhta hai jo beech ke instructions use karte hain.

### legal reordering

Order badalne ke baad bhi har register aur memory location mein wahi final value aaye.

### load delay slot

lw ke turant baad ki jagah; yahan independent instruction rakho.

### Kaise karein (steps):

1. Har lw dhundo jiske turant baad wala instruction uska register use karta hai (load-use pair).
2. Aage ya peeche se ek **independent** instruction dhundo (aksar doosra lw).
3. Check karo: use upar khiskaane se koi RAW / WAR / WAW nahi tootna chahiye (jo register woh likhta hai, beech mein koi padh / likh nahi raha; jo woh padhta hai, beech mein koi likh nahi raha; store ke upar load tabhi jab address alag ho).
4. Use load-use pair ke beech rakh do.
5. Cycles dobara gino:  $5 + (n - 1) + \text{bachae hue stalls}$ .

**Easy example (with forwarding).** Original: lw \$t0, 0(\$s1); add \$t1, \$t0, \$s2; sw \$t1, 4(\$s1); lw \$t3, 8(\$s1). lw → add load-use = 1 stall →  $5 + 3 + 1 = 9$  cycles. lw \$t3 kisi pe depend nahi karta aur koi usse pehle \$t3 use nahi karta → upar khiskao:

| Instruction          | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 | CC8 |
|----------------------|-----|-----|-----|-----|-----|-----|-----|-----|
| lw \$t0, 0(\$s1)     | IF  | ID  | EX  | MEM | WB  |     |     |     |
| lw \$t3, 8(\$s1)     |     | IF  | ID  | EX  | MEM | WB  |     |     |
| add \$t1, \$t0, \$s2 |     |     | IF  | ID  | EX  | MEM | WB  |     |
| sw \$t1, 4(\$s1)     |     |     |     | IF  | ID  | EX  | MEM | WB  |

Ab add aur lw \$t0 ke beech lw \$t3 hai: add ko \$t0 MEM/WB se, sw ko \$t1 EX/MEM se. 0 stalls, 8 cycles (9 se 8).

**P&H example:** C code  $a = b + e$ ;  $c = b + f$ ; with b, e, f, a, c at offsets 0, 4, 8, 12, 16 from \$t0.

|                              |                             |
|------------------------------|-----------------------------|
| Original (2 load-use stalls) | Scheduled (0 stalls)        |
| lw \$t1, 0(\$t0)             | lw \$t1, 0(\$t0)            |
| lw \$t2, 4(\$t0)             | lw \$t2, 4(\$t0)            |
| add \$t3, \$t1, \$t2 # stall | lw \$t4, 8(\$t0) # moved up |
| sw \$t3, 12(\$t0)            | add \$t3, \$t1, \$t2        |
| lw \$t4, 8(\$t0)             | sw \$t3, 12(\$t0)           |
| add \$t5, \$t1, \$t4 # stall | add \$t5, \$t1, \$t4        |
| sw \$t5, 16(\$t0)            | sw \$t5, 16(\$t0)           |

**Original: 13 cycles** ( $5 + 6 = 11 \text{ ideal} + 2 \text{ stalls}$ )

| Instruction          | CC1 | CC2 | CC3 | CC4 | CC5   | CC6 | CC7 | CC8 | CC9   | CC10 | CC11 | CC12 | CC13 |
|----------------------|-----|-----|-----|-----|-------|-----|-----|-----|-------|------|------|------|------|
| lw \$t1, 0(\$t0)     | IF  | ID  | EX  | MEM | WB    |     |     |     |       |      |      |      |      |
| lw \$t2, 4(\$t0)     |     | IF  | ID  | EX  | MEM   | WB  |     |     |       |      |      |      |      |
| add \$t3, \$t1, \$t2 |     |     | IF  | ID  | stall | EX  | MEM | WB  |       |      |      |      |      |
| sw \$t3, 12(\$t0)    |     |     |     | IF  | stall | ID  | EX  | MEM | WB    |      |      |      |      |
| lw \$t4, 8(\$t0)     |     |     |     |     |       | IF  | ID  | EX  | MEM   | WB   |      |      |      |
| add \$t5, \$t1, \$t4 |     |     |     |     |       |     | IF  | ID  | stall | EX   | MEM  | WB   |      |
| sw \$t5, 16(\$t0)    |     |     |     |     |       |     |     | IF  | stall | ID   | EX   | MEM  | WB   |

**Scheduled: 11 cycles** (0 stalls)

| Instruction          | CC1 | CC2 | CC3 | CC4 | CC5 | CC6 | CC7 | CC8 | CC9 | CC10 | CC11 |
|----------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|
| lw \$t1, 0(\$t0)     | IF  | ID  | EX  | MEM | WB  |     |     |     |     |      |      |
| lw \$t2, 4(\$t0)     |     | IF  | ID  | EX  | MEM | WB  |     |     |     |      |      |
| lw \$t4, 8(\$t0)     |     |     | IF  | ID  | EX  | MEM | WB  |     |     |      |      |
| add \$t3, \$t1, \$t2 |     |     |     | IF  | ID  | EX  | MEM | WB  |     |      |      |
| sw \$t3, 12(\$t0)    |     |     |     |     | IF  | ID  | EX  | MEM | WB  |      |      |
| add \$t5, \$t1, \$t4 |     |     |     |     |     | IF  | ID  | EX  | MEM | WB   |      |
| sw \$t5, 16(\$t0)    |     |     |     |     |     |     | IF  | ID  | EX  | MEM  | WB   |

Kyun legal: lw \$t4 sirf \$t0 padhta hai (koi nahi badalta) aur \$t4 likhta hai (beech mein koi \$t4 use nahi karta); woh Mem[\$t0 + 8] padhta hai, aur jis sw (\$t3 → 12(\$t0)) ke upar gaya woh alag address likhta hai. Isme ek extra register (\$t4 alag rakhna) laga: RISC mein 32 registers hone ka ek fayda.

### Solved example (MCQ type):

lw \$t1, 0(\$t0); add \$t2, \$t1, \$t3; sub \$t4, \$t5, \$t6. Which reordering is ILLEGAL: (a) move sub between lw and add, (b) move sub before lw, (c) move add before lw?

- (a) sub \$t1 / \$t2 se koi lena-dena nahi → legal, aur load-use stall bhi hat jaata hai (best choice).
- (b) sub kisi pe depend nahi → legal (lekin stall nahi hatega).
- (c) add ko \$t1 chahiye jo lw laata hai (RAW). add pehle chala to purana \$t1 use karega → **illegal**.

**Answer: (c) is illegal; (a) is the useful legal move (removes the stall).**

**Galti mat karna:** (1) Load ko us store ke upar le jaana jo *usi* address pe likhta hai (memory RAW tootega). (2) Do instructions jo same register likhte hain unka order badalna (WAW). (3) Scheduled version mein bhi stall ginna jab beech mein ek independent instruction aa gaya ho.

**Trick: "lw ke baad gap bharo, dependence mat todo."** Sabse aasaan filler: agla lw upar khiskao.

**Ek line mein:** Compiler independent instructions (aksar loads) ko upar khiska ke load-use slot bharta hai; P&H example 13 → 11 cycles; har dependence bachani zaroori.

## Exam mein is unit se kya aata hai → kaise solve karein

| Question type                                              | Method (steps)                                                                                                                                                                                                                                           |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Classify a situation as structural / data / control        | Resource ka jhagda → structural. Value ready nahi → data. Next PC branch pe depend → control.                                                                                                                                                            |
| RAW / WAR / WAW identify                                   | I1 likhe, I2 padhe → RAW. I1 padhe, I2 likhe → WAR. Dono likhein → WAW. 5-stage in-order mein sirf RAW hazard.                                                                                                                                           |
| Stalls without forwarding                                  | Distance d (1 = turant agla): stalls = 3 - d (minimum 0). Total = 5 + (n - 1) + stalls. Table: dependent instruction ID mein ruka, peeche wala IF mein.                                                                                                  |
| Stalls with forwarding                                     | ALU → ALU: 0. lw → turant use: 1. lw, ek instruction, use: 0.                                                                                                                                                                                            |
| Draw cycle table with stalls / forwarding                  | 1. Staircase banao. 2. Har dependence pe dekho value kab ready (ALU: EX end, lw: MEM end) aur kab chahiye (EX start). 3. Gap ho to stall cells daalo, peeche wale sab shift. 4. Forwarding arrow ready-cycle ke end se use-cycle ki start tak (aage hi). |
| Where is the value forwarded from / ForwardA value         | Producer 1 aage → EX/MEM (10); 2 aage → MEM/WB (01); 3 aage → register file (00). Rs → ForwardA, Rt → ForwardB.                                                                                                                                          |
| Write forwarding conditions / load-use detection condition | EX: EX/MEM.RegWrite, Rd ≠ 0, EX/MEM.Rd = ID/EX.Rs → 10. MEM: same with MEM/WB and "not EX hazard" → 01. Load-use: ID/EX.MemRead and ID/EX.Rt = IF/ID.Rs or IF/ID.Rt → stall.                                                                             |
| Why load-use needs a stall                                 | Data MEM ke end mein, chahiye agle ke EX start pe (same cycle); time mein peeche nahi bhej sakte → 1 stall + MEM/WB → EX.                                                                                                                                |
| Reorder code to remove stalls; cycles before / after       | Load-use pair dhundo → independent instruction (doosra lw) beech mein rakho → dependences check → cycles = 5 + n - 1 + stalls (P&H: 13 → 11).                                                                                                            |
| Branch handling (stall, predict not-taken, delayed branch) | Stall: har branch penalty. Predict not-taken: sirf taken pe flush (ID mein faisla → 1, MEM → 3). Delayed: slot ka instruction hamesha chalta, compiler useful instruction rakhta.                                                                        |
| CPI with stalls numerical                                  | $CPI = 1 + \sum (\text{fraction} \times \text{stall cycles})$ . Branch: branch% × taken% × penalty. Load-use: load% × use-next% × 1. Time = $N \times CPI \times t_{\text{cycle}}$ ; speedup = $k / CPI$ .                                               |

# Last-hour cheat sheet

## Gates

XOR =  $A'B + AB'$  (differ) · XNOR =  $A'B' + AB$  (equal) ·  
NAND/NOR universal

## Laws

$A + AB = A \cdot A(A + B) = A \cdot A + A'B = A + B \cdot AB + AB' = A$   
De Morgan:  $(AB)' = A' + B'$ ,  $(A + B)' = A'B'$

## Canonical / K-map

SOP:  $1 \rightarrow X$ ,  $0 \rightarrow X'$  · POS:  $0 \rightarrow X$ ,  $1 \rightarrow X'$   
Gray 00 01 11 10 · group  $2^k$  removes  $k$  variables · corners wrap

## Universal gate counts

NAND: NOT 1, AND 2, OR 3, XOR 4 · NOR: NOT 1, OR 2, AND 3  
SOP  $\rightarrow$  NAND-NAND · POS  $\rightarrow$  NOR-NOR

## MUX / decoder

$2:1Y = S'0 + S1$  ·  $n$  selects  $\rightarrow 2^n$  inputs · decoder  $D_k = m_k$

## Adders

HA:  $S = A \oplus B$ ,  $C = AB$  · FA:  $S = A \oplus B \oplus C_{in}$ ,  $C_{out} = AB + C_{in}(A \oplus B)$   
CLA:  $G = AB$ ,  $P = A \oplus B$ ,  $C(i+1) = G_i + P_i C_i$  · ALU: add 0010, sub 0110

## Numbers

2's range  $-2^{n-1} \dots 2^{n-1} - 1$  ·  $-x = \text{invert} + 1$  · MSB weight  $-2^{n-1}$   
 $A - B = A + B' + 1$  ·  $V = C_3 \oplus C_4$  ·  $CF = C_4$  · sign-extend = copy MSB

## Latches / FFs

NOR SR: 11 forbidden · NAND SR: 00 forbidden ·  $Q^+ = S + R'Q$   
D latch: transparent when  $EN = 1$  · D FF:  $Q^+ = D$  at the edge  
JK:  $Q^+ = JQ' + K'Q$  · T:  $Q^+ = T \oplus Q$  ·  $T = Q \oplus Q^+$

## Timing

$T_c \geq t_{pcq} + t_{pd} + t_{setup}$  ·  $f = 1/T_c$  · hold:  $t_{ccq} + t_{cd} \geq t_{hold}$   
ring oscillator  $T = 2 \cdot N \cdot t_{pd}$  ·  $1/ns = 1\text{ GHz}$

## Counters / registers

FFs =  $\lceil \log_2 N \rceil$  · ripple delay  $n \cdot t_{pd}$  · bit  $k = f/2^{(k+1)}$   
 $T0 = 1$ ,  $T1 = Q0$ ,  $T2 = Q0Q1$  · load enable  $Q^+ = LD + L'Q$   
ring  $n$ , Johnson  $2n$ , LFSR  $2^n - 1$  · SIPO = UART RX, PISO = UART TX

## Register file / FSM

$32 \times 32$ , 2 read + 1 write · address bits =  $\log_2(\text{regs})$   
read = MUX (combinational), write = decoder + WE (clocked)  
Moore: output in circle; Mealy: on arrow · 1011: Mealy 4 states, Moore 5

## Memory

SRAM 6T, no refresh, cache · DRAM 1T1C, refresh 64 ms, destructive read  
refresh interval =  $64\text{ ms} \div \text{rows}$  ( $8192 \rightarrow 7.8\text{ }\mu\text{s}$ ) · AMAT = hit + miss rate  $\times$  penalty

## Architecture

VN: one bus (bottleneck, Backus 1977) · Harvard: two buses · modern: split L1  
bandwidth =  $\text{bits/cycle} \times f \div 8$  · RISC = fixed length + load/store · CPU time =  $IC \times CPI \times T$

## MIPS

R 6-5-5-5-5-6 · I 6-5-5-16 · J 6-26 · \$zero 0, \$v 2-3, \$a 4-7, \$t 8-15, \$s 16-23, \$sp 29, \$ra 31  
 $EA = rs + \text{signext}(\text{imm})$  · branch =  $PC + 4 + \text{imm} \times 4$  · jump =  $PC + 4[31:28] \parallel \text{addr} \parallel 00$   
add 0x20, sub 0x22, slt 0x2A, lw 0x23, sw 0x2B, beq 4, addi 8, j 2  
div: LO = quotient, HI = remainder ·  $A[i] = \text{base} + 4i$  · jal: \$ra =  $PC + 4$

## Datapath

R: RegDst 1, ALUSrc 0, MemtoReg 0, RegWrite 1, ALUOp 10  
lw: 0, 1, 1, 1, MemRead 1, ALUOp 00 · sw: X, 1, X, 0, MemWrite 1, 00  
beq: X, 0, X, 0, Branch 1, ALUOp 01 · PCSrc = Branch · Zero · lw = critical path

## Pipeline

IF ID EX MEM WB · cycles =  $k + n - 1$  · speedup  $\rightarrow k \cdot \text{clock} = \text{slowest stage}$   
stalls: 2 (no forwarding) / 0 (forwarding) / 1 (load-use)